

z/OS
3.1

*ISPF Dialog Tag Language Guide and
Reference*



Note

Before using this information and the product it supports, read the information in [“Notices” on page 495.](#)

This edition applies to IBM® z/OS® 3.1 (5655-ZOS) and to all subsequent releases and modifications until otherwise indicated in new editions.

Last updated: 2024-01-20

© **Copyright International Business Machines Corporation 1989, 2024.**

US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

Figures.....	ix
Tables.....	xvii
Preface.....	xxi
About this document.....	xxi
Who should use this document.....	xxi
How to read the syntax diagrams.....	xxi
z/OS information.....	xxv
How to provide feedback to IBM.....	xxvii
Summary of changes.....	xxix
Summary of changes for z/OS 3.1.....	xxix
What's in the library?.....	xxxii
Part 1. Guide to the Dialog Tag Language (DTL).....	1
Chapter 1. Introduction to the Dialog Tag Language (DTL).....	3
Why the Dialog Tag Language (DTL)?.....	3
What is the Dialog Tag Language?.....	4
Dialog elements.....	5
Application panels.....	5
Help panels.....	6
Messages.....	7
Application command table.....	8
Key mapping lists.....	8
Variables and variable classes.....	8
What is the ISPF conversion utility?.....	9
Chapter 2. How to use the Dialog Tag Language (DTL).....	11
Syntax conventions.....	11
Attributes and values.....	11
Tag text.....	12
Text formatting.....	13
Nesting tags.....	14
Markup declarations.....	15
Declaring the document type.....	15
Including comments in your markup.....	16
Defining entities and parameter entities.....	17
Embedding source files.....	22
Runtime substitution variables.....	23
Predefined entities.....	23
Chapter 3. Getting started: designing application panels.....	27
Defining application panels: the PANEL tag.....	27
The panel title.....	29

Panel size (width and depth).....	29
Key mapping lists.....	29
Associated help panels.....	29
Panel defaults.....	30
Cursor placement.....	30
Other panel attributes.....	31
Defining action bars and pull-downs.....	33
Coding an action bar definition.....	34
Pull-down choice actions.....	35
Action bar help.....	35
Preselected pull-down choices.....	36
Mnemonic choice selection.....	36
Defining the panel body.....	37
Panel instructions.....	37
The AREA tag.....	38
Scrollable areas.....	39
Multiple AREA tags.....	42
The DYNAMIC AREA tag.....	43
The GRAPHIC AREA tag.....	43
The DIVIDER tag.....	44
The REGION tag.....	45
Defining a command area.....	48
Defining panel defaults.....	50
Chapter 4. Variables and variable classes.....	53
Declaring variables.....	53
Defining variable classes.....	54
Variable class types.....	54
Character variables.....	54
Numeric variables.....	56
Variable validation.....	56
Translate lists.....	56
Validity checks.....	60
Overriding variable classes.....	70
Chapter 5. Application panel fields.....	71
Field prompts.....	71
Defining data fields.....	74
Data field width.....	75
Data field descriptions.....	75
Data field help.....	76
Other data field attributes.....	77
Defining selection fields.....	79
Single-choice fields.....	80
Multiple-choice fields.....	81
Menu-choice fields.....	83
Model-choice fields.....	84
Tutor-choice fields.....	84
Selection field help.....	85
Selection width.....	85
Other selection field attributes.....	86
Data columns.....	89
Defining list fields.....	91
List group headings.....	92
List column width.....	94
Other list column attributes.....	94
Defining group headings.....	96
Defining point-and-shoot fields.....	96

Defining scrollable fields.....	97
Chapter 6. Information regions and help panels.....	101
Defining an information region.....	101
Defining basic text.....	102
Paragraphs.....	102
Headings.....	104
Lines.....	104
Examples.....	105
Figures.....	106
Defining lists.....	108
Note lists.....	108
Simple lists.....	109
Unordered lists.....	110
Ordered lists.....	112
Definition lists.....	114
Parameter lists.....	118
Nesting tags within lists.....	119
Nesting lists within lists.....	120
Alerting users: notes, warnings, cautions, and attention.....	121
Notes (NOTE, NT and NOTEL tags).....	121
Attention and warning (ATTENTION and WARNING tags).....	124
Caution (CAUTION tag).....	125
Emphasizing panel text.....	126
Highlighted phrases.....	126
Reference phrases.....	127
Using information regions with other panel elements.....	129
Help panels.....	130
Defining help panels.....	130
Defining help panel text.....	131
Chapter 7. Messages.....	137
Defining messages.....	137
Specifying message severity.....	138
Short messages.....	139
Assigning messages.....	139
Displaying messages.....	140
Variable substitution.....	141
Chapter 8. The application command table.....	143
Defining the application command table.....	143
Specifying command actions.....	144
Truncating commands.....	145
Chapter 9. Defining key mapping lists.....	147
Assigning keys and actions.....	147
ISPF default key list.....	147
Displaying keys.....	148
Defining help for key list.....	148
Chapter 10. Using the conversion utility.....	151
Using the ISPF-supplied invocation panels.....	151
Invocation panel.....	151
Panel input fields.....	152
Panel options.....	153
Converting multiple DTL source files.....	156
Calling help.....	156
Using CUA panel attributes.....	156

Conversion utility syntax.....	156
Conversion utility general information.....	162
Converting multiple panels.....	165
ISPF conversion utility messages.....	165
Return codes.....	166
Conversion results.....	166
Conversion utility file names.....	166
Default data set names.....	167
Part 2. Dialog Tag Language (DTL) reference.....	169
Chapter 11. Markup declarations and DTL macro reference.....	171
Document-type declaration.....	171
Description.....	171
Example.....	171
Entity declarations.....	172
Description.....	173
Conditions.....	173
Example.....	174
Sample entity definitions.....	175
DTL macros.....	177
Chapter 12. Tag reference.....	179
Rules for variable names.....	179
Rules for “%variable” names.....	179
AB (Action Bar).....	179
ABC (Action Bar Choice).....	182
ACTION (Action).....	184
AREA (Area).....	189
ASSIGNI (Assignment List Item).....	195
ASSIGNL (Assignment List).....	196
ATTENTION (Attention).....	198
ATTR (Attribute).....	200
BOTINST (Bottom Instruction).....	204
CAUTION (Caution).....	205
CHDIV (Choice Divider).....	207
CHECKI (Validity Check Item).....	210
CHECKL (Validity Check List).....	218
CHOFLD (Choice Data Field).....	219
CHOICE (Selection Choice).....	226
CMD (Command Definition).....	233
CMDACT (Command Action).....	234
CMDAREA (Command Area).....	237
CMDTBL (Command Table).....	243
COMMENT (Comment).....	245
COMPOPT (Compiler Options).....	247
COPYR (Copyright).....	249
DA (Dynamic Area).....	250
DD (Definition Description).....	255
DDHD (Definition Description Header).....	257
DIVIDER (Area Divider).....	258
DL (Definition List).....	261
DLDIV (Definition List Divider).....	265
DT (Definition Term).....	267
DTACOL (Data Column).....	269
DTAFLD (Data Field).....	275
DTAFLDD (Data Field Description).....	284

DTDIV (Definition Term Divider).....	286
DTHD (Definition Term Header).....	287
DTHDIV (Definition Term Header Divider).....	288
DTSEG (Definition Term Segment).....	290
FIG (Figure).....	291
FIGCAP (Figure Caption).....	294
GA (Graphic Area).....	295
GENERATE (Generate).....	298
GRPHDR (Group Header).....	300
HELP (Help Panel).....	303
HELPDEF (Help default).....	311
Hn (Heading).....	313
HP (Highlighted Phrase).....	315
INFO (Information Region).....	317
KEYI (Key Item).....	319
KEYL (Key List).....	322
LI (List Item).....	325
LINES (Lines).....	327
LIT (Literal).....	329
LP (List Part).....	330
LSTCOL (List Column).....	332
LSTFLD (List Field).....	341
LSTGRP (List Group).....	345
LSTVAR (List Variable).....	348
M (Mnemonic).....	351
MSG (Message).....	352
MSGMBR (Message Member).....	356
NOTE (Note).....	358
NOTEL (Note List).....	361
NT (Note).....	364
OL (Ordered List).....	367
P (Paragraph).....	370
PANDEF (Panel Default).....	372
PANEL (Panel).....	376
PARML (Parameter List).....	386
PD (Parameter Description).....	388
PDC (Pull-Down Choice).....	390
PDSEP (Pull-Down Separator).....	394
PLDIV (Parameter List Divider).....	394
PNLINST (Panel Instruction).....	396
PS (Point-and-Shoot).....	398
PT (Parameter Term).....	400
PTDIV (Parameter Term Divider).....	402
PTSEG (Parameter Term Segment).....	404
REGION (Region).....	405
RP (Reference Phrase).....	411
SCRFLD (Scrollable Field).....	413
SELFLD (Selection Field).....	421
SL (Simple List).....	433
SOURCE (Source).....	435
T (Truncation).....	437
TEXTLINE (Text Line).....	439
TEXTSEG (Text Segment).....	439
TOPINST (Top Instruction).....	441
UL (Unordered List).....	443
VARCLASS (Variable Class).....	445
VARDCL (Variable Declaration).....	449
VARLIST (Variable List).....	450

VARSUB (Variable Substitution).....	452
WARNING (Warning).....	454
XLATI (Translate Item).....	456
XLATL (Translate List).....	458
XMP (Example).....	460
Appendix A. Dialog Tag Language (DTL) tags.....	463
Appendix B. Accessibility.....	493
Notices.....	495
Terms and conditions for product documentation.....	496
IBM Online Privacy Statement.....	497
Policy for unsupported hardware.....	497
Minimum supported hardware.....	497
Programming Interface Information.....	498
Trademarks.....	498
Index.....	499

Figures

1. Sample syntax diagram.....	xxii
2. Application panel.....	5
3. Help panel.....	7
4. Message displayed in message area.....	8
5. Text formatting rules.....	13
6. Entity reference for text substitution.....	18
7. Entity reference for text substitution.....	19
8. Entity reference for text substitution and file embedding.....	20
9. Parameter entities.....	21
10. Cursor placement.....	31
11. Action bar and pull-down.....	34
12. Top and bottom instructions.....	38
13. AREA MARGINW=10.....	39
14. Scrollable panel area.....	41
15. Application panel area.....	41
16. Scrollable panel area.....	42
17. Multiple horizontal areas.....	43
18. Area dividers.....	45
19. Vertical region.....	46
20. Horizontal region.....	47
21. Horizontal region.....	48
22. Command area.....	49
23. Command area.....	50

24. Variable translation results.....	57
25. Variable translation.....	58
26. Variable translation.....	59
27. Variable translation.....	59
28. Variable translation.....	60
29. Prompt locations.....	72
30. Prompt widths.....	73
31. Data fields.....	74
32. Data field description.....	76
33. Single-choice selection field.....	81
34. Multiple-choice selection field.....	83
35. Sample option menu.....	84
36. selection width, definingSelection field SELWIDTH attribute.....	86
37. Data column.....	90
38. List field.....	92
39. List group.....	93
40. Scrollable field.....	98
41. Scrollable field within a list column.....	99
42. Paragraph.....	103
43. Multiple paragraphs.....	103
44. Headings (H1-H2).....	104
45. LINES.....	105
46. XMP.....	106
47. Figure with rules.....	107
48. Figure caption.....	108

49. Simple list.....	109
50. Compact simple list.....	110
51. Unordered list.....	110
52. Nested unordered lists.....	112
53. Ordered list.....	112
54. Nested ordered lists.....	114
55. Definition list.....	115
56. Definition list (BREAK=ALL).....	117
57. Definition list (BREAK=FIT).....	118
58. Parameter list.....	119
59. Nested paragraph within a list.....	119
60. List part.....	120
61. Nested unordered list in a definition list.....	121
62. Note (NOTE tag).....	122
63. Note (NOTEL tag).....	123
64. Note (NT tag).....	124
65. Warning.....	125
66. Caution.....	125
67. Highlighted phrase example.....	127
68. Reference phrase example.....	128
69. Reference phrase example of help attribute.....	129
70. Information region.....	130
71. Help panel.....	132
72. Help panel (example 1 of 4).....	133
73. Help panel (example 2 of 4).....	133

74. Help panel (example 3 of 4).....	133
75. Help panel (example 4 of 4).....	134
76. Help panel (example 1 of 3).....	135
77. Help panel (example 2 of 3).....	135
78. Help panel (example 3 of 3).....	135
79. Data field and message.....	140
80. Displayed function key area.....	149
81. Conversion utility invocation panel (ISPCP01).....	151
82. Panel ISPCP04.....	156
83. ISPF Dialog Tag Language conversion utility - confirm cancel.....	164
84. ISPF Dialog Tag Language conversion utility - recursive invoke.....	164
85. Entities and parameter entities.....	175
86. Action bar.....	182
87. Action bar choices.....	184
88. Application panel area.....	194
89. Application panel area.....	194
90. Application panel area.....	195
91. Attention statement.....	200
92. Bottom instructions.....	205
93. Caution statement.....	207
94. Choice divider.....	209
95. Choice data fields.....	226
96. Selection field choices.....	232
97. Command area.....	243
98. Comment text added to a panel.....	247

99. Copyright statement added to a panel.....	250
100. Definition descriptions.....	257
101. Definition description header.....	258
102. Area dividers.....	261
103. Definition List (BREAK=NONE).....	264
104. Definition List (BREAK=FIT).....	264
105. Definition List (BREAK=ALL).....	265
106. Definition list dividers.....	267
107. Definition Terms.....	269
108. PMTWIDTH, ENTWIDTH, FLDSPACE, and DESWIDTH attributes.....	273
109. Data column.....	275
110. Data fields.....	284
111. Data field descriptions.....	285
112. Definition term divider.....	287
113. Definition term header.....	288
114. Definition term header divider.....	290
115. Definition term segment.....	291
116. Figure.....	294
117. Figure caption.....	295
118. Generated panel.....	300
119. Group heading.....	303
120. Help panel (example 1 of 3).....	310
121. Help panel (example 2 of 3).....	310
122. Help panel (example 3 of 3).....	310
123. Headings.....	315

124. HP (Highlighted Phrase).....	317
125. Information region.....	319
126. Key Items.....	322
127. Function keys.....	324
128. List items.....	327
129. LINES.....	329
130. List part.....	332
131. List columns.....	339
132. List columns.....	340
133. List field.....	345
134. List group.....	348
135. List variable.....	350
136. Messages.....	356
137. Message member.....	358
138. Note.....	361
139. NOTEL.....	364
140. NT.....	367
141. Ordered lists.....	370
142. Paragraphs.....	372
143. Application panel.....	386
144. Parameter list.....	388
145. Parameter descriptions.....	390
146. Pull-down choices.....	393
147. Parameter list divider.....	396
148. Panel instructions.....	398

149. Point-and-shoot fields.....	400
150. Parameter terms.....	402
151. Parameter term divider.....	403
152. Parameter term segment.....	405
153. Regions.....	409
154. Using WIDTH and DEPTH attributes.....	411
155. Reference phrase example.....	413
156. Reference phrase example.....	413
157. List field.....	420
158. List variable.....	421
159. Selection fields.....	432
160. Selection menu.....	433
161. Simple list.....	435
162. Top instructions.....	443
163. Unordered list.....	445
164. Variable substitution.....	454
165. Warning statement.....	456
166. Example.....	462

Tables

1. The equivalence of an option and valid value to conversion utility syntax.....	153
2. Default data set names used for conversion utility.....	167
3. The tag you can code within an AB definition.....	181
4. Tags you can code within an ABC definition.....	183
5. Tags you can code within an AREA definition on an application panel.....	192
6. Tags you can code within an AREA definition on a help panel.....	193
7. Tags you can code within an ASSIGNL definition.....	197
8. Tags you can code within an ATTENTION definition.....	199
9. Tags you can code within a BOTINST definition.....	205
10. Tags you can code within a CAUTION definition.....	206
11. Tags you can code within a CHDIV definition.....	208
12. Tags you can code within a CHECKL definition.....	218
13. Tags you can code within a CHOFLD definition.....	224
14. Host Display indicators for particular TYPEs.....	228
15. The tags you can code within a CHOICE definition.....	231
16. The tags you can code within a CMD definition.....	234
17. ISPF application command table.....	234
18. ISPF application command table.....	237
19. The tags you can code within a CMDAREA definition.....	242
20. The tags you can code within a CMDTBL definition.....	244
21. ISPF Application Command Table.....	245
22. The tags you can code within a DA definition.....	255
23. The tags you can code within a DD definition.....	256

24. The tags you can code within a DDHD definition.....	258
25. The tags you can code within a DIVIDER definition.....	260
26. The tags you can code within a DL definition.....	263
27. The tags you can code within a DLDIV definition.....	266
28. The tags you can code within a DT definition.....	268
29. The tags you can code within a DTACOL definition.....	274
30. The tags you can code within a DTAFLD definition.....	282
31. The tags you can code within a DTAFLDD definition.....	285
32. The tags you can code within a DTHD definition.....	288
33. The tag you can code within a DTSEG definition.....	290
34. The tags you can code within a FIG definition.....	292
35. The tags you can code within a FIGCAP definition.....	294
36. The tags you can code within a GENERATE definition.....	299
37. The tags you can code within a GRPHDR definition.....	302
38. ISPHelp keylist and assignments.....	308
39. ISPHLP2 keylist and assignments.....	308
40. The tags you can code within a HELP definition.....	309
41. The tags you can code only within an H2, H3, or H4 definition.....	314
42. The tags you can code within an INFO definition.....	318
43. The tags you can code within a KEYL definition.....	324
44. The tags you can code within an LI definition.....	326
45. The tags you can code within a LINES definition.....	328
46. The tags you can code within an LP definition.....	331
47. The tags you can code within a LSTCOL definition.....	338
48. The tags you can code within a LSTFLD definition.....	343

49. The tags you can code within a LSTGRP definition.....	346
50. Tags you can code within an LSTVAR definition.....	349
51. Tags you can code within a MSG definition.....	355
52. Tags you can code within an MSGMBR definition.....	357
53. Tags you can code within a NOTE definition.....	360
54. Tags you can code within a NOTEL definition.....	363
55. Tags you can code within an NT definition.....	366
56. Tags you can code within an OL definition.....	369
57. Tags you can code within a P definition.....	371
58. Tags you can code within a PANEL definition.....	384
59. Tags you can code within a PARML definition.....	387
60. Tags you can code within a PD definition.....	389
61. Tags you can code within a PDC definition.....	392
62. Tags you can code within a PLDIV definition.....	395
63. Tags you can code within a PNLINST definition.....	397
64. Tags you can code within a PT definition.....	401
65. Tags you can code within a PTSEG definition.....	404
66. Tags you can code within a REGION definition.....	407
67. Tags you can code within a REGION tag on a help panel.....	408
68. Order in which scroll indicator fields are created when FLDSPPOS=BELO is specified.....	417
69. Order in which scroll indicator fields are created.....	418
70. Tags you can code within a SCRFLD definition.....	418
71. Tags you can code within a SELFLD definition.....	430
72. Tags you can code within an SL definition.....	435
73. ISPF application command table.....	438

74. Tags you can code within a TEXTLINE definition.....	439
75. Tags you can code within a TEXTSEG definition.....	440
76. Tags you can code within a TOPINST definition.....	442
77. Tags you can code within a UL definition.....	445
78. Tags you can code within a VARCLASS definition.....	448
79. Tags you can code within a VARLIST definition.....	451
80. Tags you can code within a WARNING definition.....	455
81. Tags you can code within an XLATI definition.....	457
82. Tags you can code within an XLATL definition.....	459
83. Tags you can code within an XMP definition.....	461
84. Tag summary.....	463

Preface

This document describes how to use the Dialog Tag Language (DTL), the tag-based markup language you use to create these ISPF dialog elements:

- Application panels
- Help panels
- Application command tables
- Messages
- Key lists

It also explains how to use the ISPF conversion utility to convert the source files that contain DTL markup into ISPF panel language source or execution time format.

About this document

This document is organized into two parts:

- Part 1 guides you through the steps involved in using the Dialog Tag Language for designing, defining, and converting dialog elements for ISPF applications.
- Part 2 provides an alphabetical reference of the Dialog Tag Language tags and markup declarations.

Who should use this document

This document is for ISPF application developers who want to use Dialog Tag Language (DTL) to create dialog elements for ISPF applications.

How to read the syntax diagrams

The syntactical structure of commands described in this document is shown by means of syntax diagrams.

Figure 1 on page xxii shows a sample syntax diagram that includes the various notations used to indicate such things as whether:

- An item is a keyword or a variable.
- An item is required or optional.
- A choice is available.
- A default applies if you do not specify a value.
- You can repeat an item.

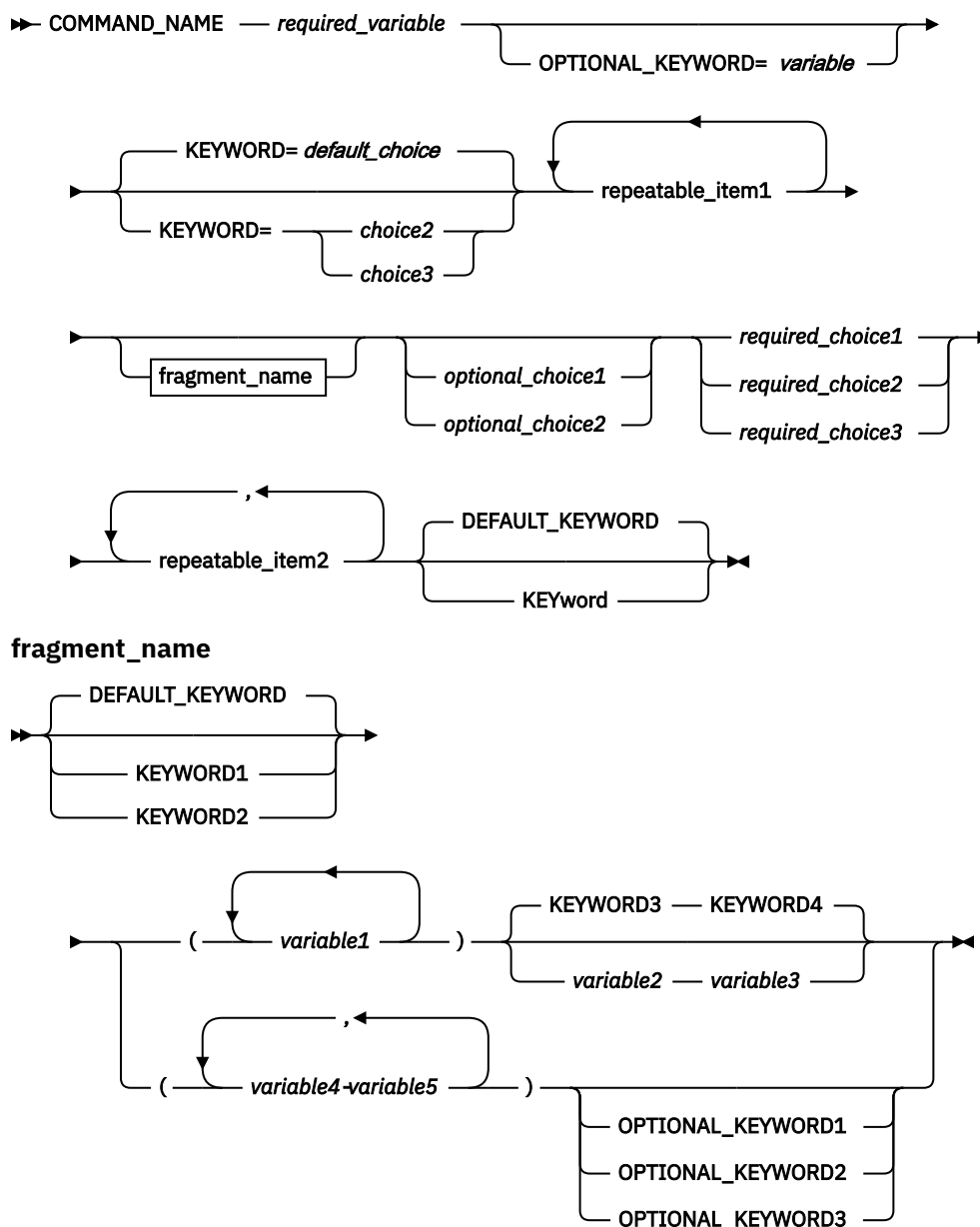


Figure 1. Sample syntax diagram

Here are some tips for reading and understanding syntax diagrams:

Order of reading

Read the syntax diagrams from left to right, from top to bottom, following the path of the line.

The ➡ symbol indicates the beginning of a statement.

The → symbol indicates that a statement is continued on the next line.

The ▶ symbol indicates that a statement is continued from the previous line.

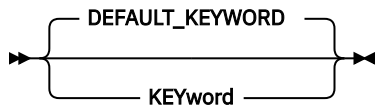
The ⇨ symbol indicates the end of a statement.

Keywords

Keywords appear in uppercase letters.

➡ COMMAND_NAME ⇨

Sometimes you only need to type the first few letters of a keyword, The required part of the keyword appears in uppercase letters.



In this example, you could type "KEY", "KEYW", "KEYWO", "KEYWOR" or "KEYWORD".

The abbreviated or whole keyword you enter must be spelled exactly as shown.

Variables

Variables appear in lowercase letters. They represent user-supplied names or values.

➡ *required_variable* ➡

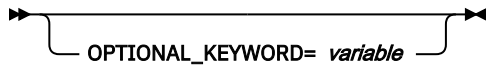
Required items

Required items appear on the horizontal line (the main path).

➡ **COMMAND_NAME** — *required_variable* ➡

Optional items

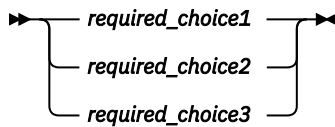
Optional items appear below the main path.



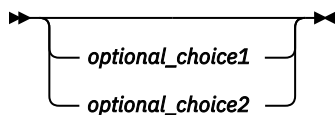
Choice of items

If you can choose from two or more items, they appear vertically, in a stack.

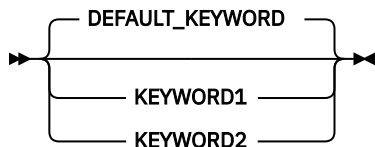
If you *must* choose one of the items, one item of the stack appears on the main path.



If choosing one of the items is optional, the entire stack appears below the main path.



If a default value applies when you do not choose any of the items, the default value appears above the main path.



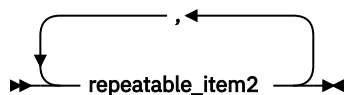
Repeatable items

An arrow returning to the left above the main line indicates an item that can be repeated.



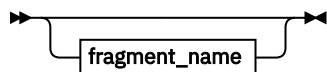
Who should use this document

If you need to specify a separator character (such as a comma) between repeatable items, the line with the arrow returning to the left shows the separator character you must specify.



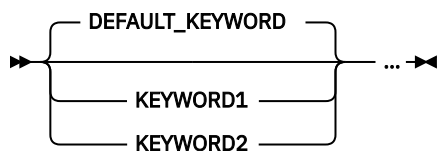
Fragments

Where it makes the syntax diagram easier to read, a section or *fragment* of the syntax is sometimes shown separately.



⋮

fragment_name



z/OS information

This information explains how z/OS references information in other documents and on the web.

When possible, this information uses cross-document links that go directly to the topic in reference using shortened versions of the document title. For complete titles and order numbers of the documents for all products that are part of z/OS, see *z/OS Information Roadmap*.

To find the complete z/OS library, go to [IBM Documentation \(www.ibm.com/docs/en/zos\)](http://www.ibm.com/docs/en/zos).

How to provide feedback to IBM

We welcome any feedback that you have, including comments on the clarity, accuracy, or completeness of the information. For more information, see [How to send feedback to IBM](#).

Summary of changes

This information includes terminology, maintenance, and editorial changes. Technical changes or additions to the text and illustrations for the current edition are indicated by a vertical line to the left of the change.

Note: IBM z/OS policy for the integration of service information into the z/OS product documentation library is documented on the z/OS Internet Library under [IBM z/OS Product Documentation Update Policy](http://www.ibm.com/docs/en/zos/latest?topic=zos-product-documentation-update-policy) (www.ibm.com/docs/en/zos/latest?topic=zos-product-documentation-update-policy).

Summary of changes for z/OS 3.1

The following changes are made for z/OS 3.1.

There are maintenance changes to this document for z/OS 3.1 ISPF.

What's in the library?

You can order the ISPF books using the numbers provided below.

Title

Order Number

z/OS ISPF Dialog Developer's Guide and Reference

SC19-3619-40

z/OS ISPF Dialog Tag Language Guide and Reference

SC19-3620-40

z/OS ISPF Edit and Edit Macros

SC19-3621-40

z/OS ISPF Messages and Codes

SC19-3622-40

z/OS ISPF Planning and Customizing

GC19-3623-40

z/OS ISPF Reference Summary

SC19-3624-40

z/OS ISPF Software Configuration and Library Manager Guide and Reference

SC19-3625-40

z/OS ISPF Services Guide

SC19-3626-40

z/OS ISPF User's Guide Vol I

SC19-3627-40

z/OS ISPF User's Guide Vol II

SC19-3628-40

Part 1. Guide to the Dialog Tag Language (DTL)

This part contains these chapters:

- [Chapter 1, “Introduction to the Dialog Tag Language \(DTL\),” on page 3](#)
An introduction to the Dialog Tag Language (DTL) and descriptions of the dialog elements you define with the Dialog Tag Language. A brief description of the ISPF Conversion Utility is also included.
- [Chapter 2, “How to use the Dialog Tag Language \(DTL\),” on page 11](#)
An explanation of the syntax conventions of the Dialog Tag Language.
- [Chapter 3, “Getting started: designing application panels,” on page 27](#)
How to design application panels.
- [Chapter 4, “Variables and variable classes,” on page 53](#)
How to declare variables, define variable classes, and validate variables.
- [Chapter 5, “Application panel fields,” on page 71](#)
How to define interactive fields for application panels.
- [Chapter 6, “Information regions and help panels,” on page 101](#)
How to define information regions and help panels.
- [Chapter 7, “Messages,” on page 137](#)
How to define messages.
- [Chapter 8, “The application command table,” on page 143](#)
How to define application commands and the application command table.
- [Chapter 9, “Defining key mapping lists,” on page 147](#)
How to define key mapping lists.
- [Chapter 10, “Using the conversion utility,” on page 151](#)
How to convert your DTL source files into ISPF panel language source format or executable preprocessed ISPF format.

Chapter 1. Introduction to the Dialog Tag Language (DTL)

The Dialog Tag Language (DTL) is a tag-based language used to define many of the elements that make up the type of application known as a *dialog*, the communication between a person and a computer. You can define these elements using DTL, and use them in your ISPF applications.

The elements you produce with DTL are used by ISPF as the user interface for your ISPF applications. The programs you write using ISPF services and a programming language use the dialog elements you create for an application.

The overview of DTL, and the dialog elements you create with DTL, are provided in these topics:

- **Why the Dialog Tag Language?**

This topic explains why you would want to use DTL to create elements for ISPF applications.

- **What is the Dialog Tag Language?**

This topic explains what the Dialog Tag Language is and how it works.

- **Dialog elements**

This topic explains and illustrates the dialog elements. These elements are:

- Application panels
- Help panels
- Messages
- An application command table
- Key mapping lists.

- **Variables and variable classes**

This topic discusses the definition of variables you include in dialog element definitions.

- **What is the ISPF Conversion Utility?**

This topic describes the conversion utility, the compiler you use to convert your DTL source files for use by ISPF.

Why the Dialog Tag Language (DTL)?

If you are already familiar with a tag-based markup language, such as HTML (Hypertext Markup Language), you will find that DTL is very similar. IBM created DTL for many of the same reasons that HTML was created:

- Markup tags are easy to use. Because tag names are short and relate directly to the structure of the dialog elements, they are also easy to remember.
- DTL lends flexibility to application development. Panels can be quickly changed without your having to tediously line up text and fields. This gives you greater control over application development and updates.
- DTL provides consistency when many programmers are working on the same application, or when programmers who are new to your company must update existing applications. Since each programmer is using the same tags, only minor adjustments may be needed to achieve complete uniformity.
- DTL techniques improve the way in which interactive programs, like ISPF applications, are developed. The language concentrates on the role of the various elements and their interrelationships, and ISPF takes care of their form and appearance at run time.

What is the Dialog Tag Language?

- DTL also enforces some formatting rules defined by the Systems Application Architecture® Common User Access (CUA), so you do not have to be familiar with all of the CUA formatting rules. Therefore, the CUA skills required by programmers who are developing CUA-conforming applications are significantly reduced.
- DTL enables multicultural support and the conversion utility provides NLS translations for certain key words.

In other words, DTL is an application development and maintenance system that is sophisticated, flexible, and easy to use.

Examples of DTL usage by ISPF are provided in data set ISP.SISPGxxx, where xxx is a standard ISPF language identifier. Consult your ISPF system administrator for the actual location of these examples.

What is the Dialog Tag Language?

In “Why the Dialog Tag Language (DTL)?” on page 3 we referred to DTL as a tag-based markup language that is similar to HTML. The two have much in common. For example, *markup* is a term that is usually associated with documentation. It is an old typesetting term that formerly meant the instructions with which a document was “marked up” to show how the document should be set in type.

Today, this definition has been expanded to include information that is added to a document to enable a person or system to process it. Just as markup information can describe a document's characteristics or the processing to be performed, it can also describe the characteristics or processing related to dialog elements. This is where the tags come in.

We call DTL a *tag-based* markup language simply because the markup consists of tags that determine not only what each element is, but also how it is processed. To convert the dialog elements into a format that is usable by ISPF, you must convert them to ISPF elements with ISPD TLC, the ISPF conversion utility. (See “What is the ISPF conversion utility?” on page 9 for more information.)

Another thing that DTL and HTML have in common, of course, is the tags themselves, which have these similarities:

- They are very short and easy to remember.
- They are often accompanied by text.
- Many DTL tags are almost identical to corresponding HTML tags.

These are all reasons that familiarity with HTML makes it easy to learn DTL. The preceding bulleted, or *unordered*, list can be created in both HTML and DTL using this tagging:

```
<ul>
  <li>They are very short and easy to remember.</li>
  <li>They are often accompanied by text.</li>
  <li>Many DTL tags are almost identical to corresponding HTML tags.</li>
</ul>
```

Here, the and tags, respectively, begin and end the unordered list. This type of list is called an unordered list because the list items are not numbered. The individual list items are defined by the tags and consist of the accompanying text.

As you can see from the preceding example, DTL tags act as control words that specify how the text of source files is interpreted by the conversion utility. This concept is based on the Standard Generalized Markup Language (SGML), which is a standard of the International Standards Organization (ISO). The conventions of the Dialog Tag Language are based on the SGML standard.

After you are finished marking up a source file, use the conversion utility to convert the file into a format usable by your ISPF application. In addition to processing the file, the conversion utility also checks and verifies the syntax of your markup, and notifies you of any errors. After conversion, the elements you defined in your source file are stored within ISPF libraries.

You can use ISPF dialog test facilities to display application panels and messages after they have been converted. Displaying your panels is a good idea to make sure they format properly.

You should now have a basic understanding of DTL and how it works. The next section builds on this understanding by describing the types of elements that you can define with DTL.

Dialog elements

This topic provides a descriptive overview of the dialog elements you can create for an ISPF application.

These elements include:

- [“Application panels” on page 5](#)
- [“Help panels” on page 6](#)
- [“Messages” on page 7](#)
- [“Application command table” on page 8](#)
- [“Key mapping lists” on page 8](#)

Application panels

Application panels are the primary element of the user interface for an application. They allow users to interact with your application through the use of data fields, selection fields, and other interactive fields. Application panels appear in primary and pop-up windows.

[Figure 2 on page 5](#) shows a full-screen application panel. Following that is a list of the elements that make up an application panel.

```

File  Search  Help
-----
                        Library Card Registration

Type in patron's name and card number if applicable.

Then, select an action bar choice.

Date . . . : 12/29/90
Card No. . . _____ (A 7-digit number)
Name . . . . _____ (Last, First, M.I.)
Address . . _____

Choose one of the following          Check valid branches
--  1. New                          - North Branch
    2. Renewal                      - South Branch
    3. Replacement                  - East Branch
                                   - West Branch

Enter a command ==> _____
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel

```

Figure 2. Application panel

Application panel elements

Action bar

The action bar appears in the top portion of the panel. It contains keyword choices that provide users access to available actions for the current panel. When the user selects an action bar choice, a pull-down containing choices appears directly below the action bar choice.

Panel title

The panel title appears below the action bar.

Panel body

The panel body serves as the main work area of the panel. The panel body contains the input and output fields, selection fields, and other text.

Additionally, the panel body can contain optional top and bottom instructions, which provide instructional text to the user. Top instructions appear below the panel title and above the interactive fields on the panel. Top instructions tell the user how to interact with the panel. Bottom instructions appear below the interactive fields on the panel. Bottom instructions tell the user how to interact with the panel, or how to continue with the application.

Message area

ISPF uses the message area (or message pop-ups) to display messages to users while they are working in the panel.

Command area

The optional command area (or command line) consists of two components: the command field prompt and the command entry field. Application users can use the command entry field to enter commands or requests to the ISPF application.

Function key area

The optional function key area, which appears at the bottom of the panel immediately below the command area (if one is defined), contains the key assignments for dialog actions valid for the application panel. The user can request that function keys not be displayed.

Note: The message area and the command area for panels defined with DTL appear at the bottom of the panel if the user has selected the "Command line at bottom" option on the ISPF Settings panel, or the application has set ZPLACE to BOTTOM. For more information on placement options, refer to the discussion of the ISPF Settings panel in the [z/OS ISPF Dialog Developer's Guide and Reference](#).

Chapter 3, "Getting started: designing application panels," on page 27 tells you how to define application panels and panel elements.

Help panels

Help panels appear in pop-up windows in response to user requests for assistance during ISPF application sessions. ISPF processes these help requests and displays the help panels.

Using DTL, you can create help panels that provide help for:

- An entire application panel (extended help or panel help)
- A specific field on an application panel (contextual help or field help)
- Messages (message help)
- The function key area (keys help).

Figure 3 on page 7 illustrates a help panel. Following that is a list that defines each of the elements that make up a help panel.

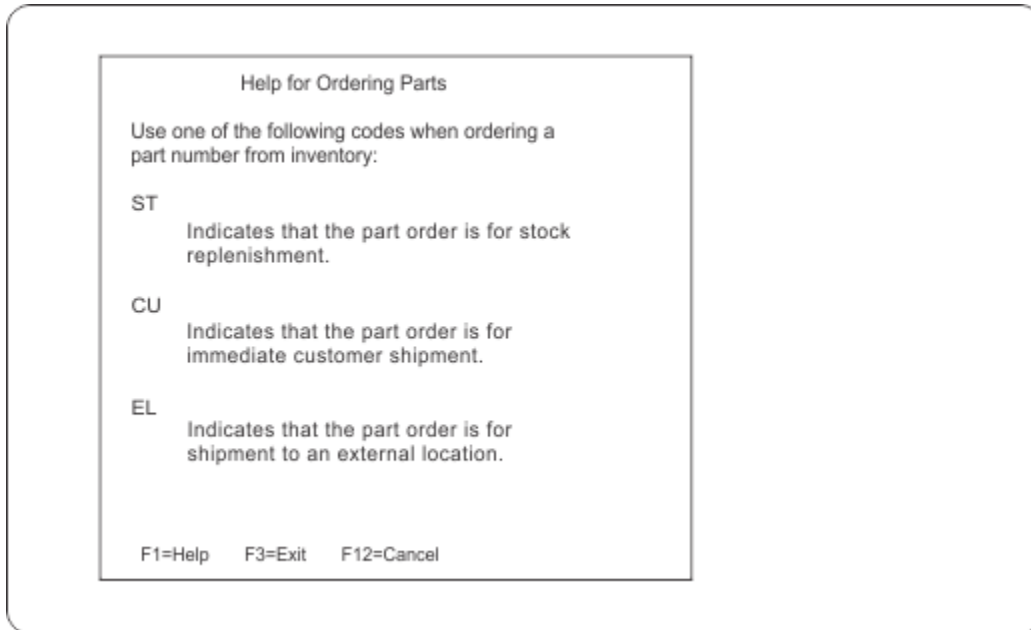


Figure 3. Help panel

Help panel elements

Help panel title

The help panel title appears at the topmost portion of the panel, followed by a blank line separating the panel title from the panel body. If the help panel text exceeds the defined depth of the help panel, a scrolling indicator appears in the right margin of the blank line following the panel title.

Help panel body

The help panel body contains the text of the help panel.

Text within the help panel is protected, which means that the user cannot interact with the text. You define this static text within an *information region* in the panel definition.

Function key area

If you are creating a help panel that does not end with a scrollable area, note that ISPF reserves 4 lines at the bottom of the panel for function keys. The display of keys in the function key area is controlled by the user through the ISPF FKA command.

Chapter 6, “Information regions and help panels,” on page 101 tells you how to define help panels and information regions.

Messages

You can use DTL to define messages that display in response to a user request or action, or that provide additional information. Messages can confirm a user-requested action, report an error in user input, or notify the user of an error or exception condition. [Figure 4 on page 8](#) illustrates a message displayed in the message area of an application panel (highlighting added).

```

File  Search  Help
-----
                        Library Card Registration

Type in patron's name and card number if applicable.

Then, select an action bar choice.

Date . . . : 12/29/90
Card No. . . : _____ (A 7-digit number)
Name . . . : _____ (Last, First, M.I.)
Address . . : _____

Choose one of the following          Check valid branches
--  1. New                          - North Branch
    2. Renewal                      - South Branch
    3. Replacement                  - East Branch
                                   - West Branch

You must type your name in the Name field.
Enter a command ==>
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel

```

Figure 4. Message displayed in message area

The messages you define for an application are stored within *message members*. You use DTL to define the messages and message members.

[Chapter 7, “Messages,” on page 137](#) provides a complete description of defining messages and message members.

Application command table

You can use DTL to define commands that perform actions requested by the user. The valid commands for an application are defined and stored within an internal application command table. You can define only one command table for an application.

Valid commands include those assigned to pull-down choices, function keys, and commands entered in command entry fields.

[Chapter 8, “The application command table,” on page 143](#) tells you how to define commands and application command tables.

Key mapping lists

The key assignments that are active for an application are defined and stored within key mapping lists. These key assignments allow the user to request commands and other actions through the use of function keys. Key assignments for your application are displayed in the function key area of application panels.

[Chapter 9, “Defining key mapping lists,” on page 147](#) tells you how to define key assignments and key mapping lists.

Variables and variable classes

Variables are used to communicate information between an application and the user. Each variable you define for a DTL-defined dialog element can be *declared*, or identified, within a variable list. In addition, each variable can be associated with a variable class that defines its type and length characteristics. The variable class can also be used to define translations and validity checks that are used when a value is displayed on a panel or entered by a user.

[Chapter 4, “Variables and variable classes,” on page 53](#) tells you how to declare variables and define variable classes.

What is the ISPF conversion utility?

ISPD TLC is the ISPF conversion tool that converts Dialog Tag Language (DTL) source files to ISPF panel language source format or executable preprocessed ISPF format. ISPF provides you with an invocation panel that allows you to specify a number of options for the conversion, or you can use conversion utility command syntax from the command line of your terminal. [Chapter 10, “Using the conversion utility,” on page 151](#) provides a complete description of both methods.

What is the ISPF conversion utility?

Chapter 2. How to use the Dialog Tag Language (DTL)

This chapter describes the tag syntax conventions and mark-up declarations of the Dialog Tag Language (DTL). It also explains how to use the DTL to create dialog element source files for your ISPF applications.

The markup style of DTL is based on the International Standards Organization (ISO) Standard Generalized Markup Language (SGML). Markup languages allow you to specify, through the use of tags, how the text of a file is to be formatted for use by an application. Because DTL is a markup language, you must follow special rules and conventions when using it to define dialog elements.

Syntax conventions

The DTL tags act as control words that determine how the text in the source files is used. Each tag is enclosed within a set of delimiter symbols that distinguish the tag as a control word (as opposed to general text). *Start* tags, which initiate text interpretation, are preceded by the start tag open delimiter (<) and followed by the close delimiter (>). *End* tags, which explicitly end text interpretation, are preceded by the end tag open delimiter (</) and followed by the close delimiter (>).

For example, the DTL tags used to define the beginning and end of an application panel are the PANEL tag and its matching end tag, which look like this:

```
<panel>  
</panel>
```

DTL tags are free-form. Indentation of nested tags can be helpful for DTL source file readability.

All of the text that you define between a start and end tag is the tag *definition*. The DTL tag data extends to the right boundary of the source file. Therefore, DTL source files cannot contain line sequence numbers. The characteristics of the tag determine how the text or other tags coded within the tag definition is formatted.

End tags are not required for all DTL tags. In many cases, the tag is implicitly ended by other start tags. For this reason, optional end tags are not used in the markup examples in this information. [Chapter 12, "Tag reference," on page 179](#) contains a detailed description of each DTL tag, and indicates when a tag needs a corresponding end tag.

Attributes and values

Many DTL start tags contain *attributes* and *values* that define various physical and operating characteristics of the dialog elements. While most attributes and values are optional, or contain default settings, some are required.

For example, the PANEL tag has a required NAME attribute that must be specified to identify the panel. The value you assign to the NAME attribute must be unique for each panel in a source file. This PANEL tag has the NAME value "panel1":

```
<panel name=panel1>  
</panel>
```

The PANEL tag also has two optional attributes, DEPTH and WIDTH, whose values specify the dimensions of the panel. For these types of attributes, you specify a numeric value.

```
<panel name=panel1 depth=20 width=40>  
</panel>
```

Values for some of the tag attributes are predetermined; that is, you can choose from one of a number of keyword values for the tag. For example, the FIG (figure) tag has a FRAME attribute that specifies the top and bottom borders of the figure. The value you assign to the FRAME attribute can be either RULE, which

produces a visible border above and below the figure, or NONE, which results in a figure without a border. No other value is acceptable for the FRAME attribute.

RULE is the default value, which means that the figure formats with visible borders if you do not specify the FRAME attribute.

The markup for a figure without ruled borders looks like this:

```
<fig frame=none>  
</fig>
```

When coding attribute values you must use single or double quotes to enclose values that contain characters other than A-Z, a-z, 0-9, a hyphen (-), or a period (.).

For example, the value assigned to the TYPE attribute of this VARCLASS tag contains a blank, so the value must be enclosed in quotes:

```
<varclass name=boolean type='char 1'>
```

Some attributes can be assigned either a specific value, such as a number or a character string, or a variable name. To distinguish a variable name from a specific value, precede the variable name with a percent (%) sign. This convention is called *% notation*. The percent sign distinguishes the variable name from a specific value. To specify a string that begins with a %, you must code an additional % before the string to distinguish it from a variable name. (For example, to specify the string "%abc", code "%%abc").

Here is an example where the ACTION attribute uses % notation to specify a variable named "varname":

```
<cmdact action='%varname'>
```

The length of any attribute value is limited to 253 characters, unless stated otherwise. This includes the lengths of any entity references that are a part of the value.

Generally, you can code tags, attributes, and values in uppercase, lowercase, or mixed case; the results are always the same regardless of case. The conventions you must follow for case-sensitive processing for each tag are described in [Chapter 12, "Tag reference," on page 179](#).

Tag text

The content or text of a tag is coded immediately following the start tag. This is the actual text that is subject to formatting and translation. The text is processed according to the type of tag it follows.

For example, the text following this P (paragraph) tag is the actual text that appears in the panel after formatting:

```
<p>The copy command allows you to copy single or multiple forms.
```

Because the tag text is processed according to the tag characteristics, not the way it is written in the source file, the paragraph could also be marked up using more than one line, like this:

```
<p>  
The copy command allows you to  
copy single or multiple forms.
```

The formatted result is the same in either case.

In most cases, there is no limit to the amount of text you can code. However, keep in mind that the text of some tags, such as the title of a PANEL tag, should be limited because of size constraints of the panel they are coded within. [Chapter 12, "Tag reference," on page 179](#) describes text length restrictions (if they exist) for each of the tags.

In most cases, multiple lines of text are concatenated. Concatenation, leading blanks, and trailing blanks are processed in this way:

- Leading and trailing blanks between lines of text are not preserved. Instead, they are compressed to a single blank when the lines are concatenated.
- The first line of tag text may start on the same line as the start tag, or on the next line. The formatted result is the same.

The text of some tags, such as the FIG, LINES, and XMP tag, allow you to control where lines break. That is, within the range of the tag, each output line is ended at the same point that you ended the input line. With these tags, multiple lines are not concatenated, and all blanks are preserved.

Text formatting

ISPF determines if the text is to be formatted according to English rules or Asian rules, based on the language specified on the conversion utility invocation. If the language is JAPANESE, CHINESE, CHINESE, or KOREAN, ISPF uses the Japanese, Traditional Chinese, Simplified Chinese, or Korean text formatting rules, respectively. If JAPANESE language is specified and the KANA option is also specified, ISPF uses the Japanese Katakana formatting rules. Otherwise, the English formatting rules are used.

English rules for text formatting

Text exceeding the width of the available panel space is wrapped to the next line. The text is split at blanks. However, if any word exceeds the panel space, then the word splits and continues on the next line.

Asian rules for text formatting

Some characters should not be placed at the beginning of a line, and some should not be placed at the end of a line. These beginning-inhibited and ending-inhibited characters are different among the languages but the required process is the same. Thus, ISPF uses the same text formatting process for these Asian languages, but uses a different beginning-and-ending-inhibitor character table for each of the languages.

The text is first split into *words*. An SBCS *word* is delimited by blanks, or SO/SI characters. Then any beginning inhibitors are stripped from the beginning of the word and treated as separate words, and any ending inhibitors are stripped from the end of the word and treated as separate words.

Adjoining DBCS alphanumeric characters (that is, Ward 42 characters) are treated as one DBCS *word*. Then any beginning inhibitors are stripped from the beginning of the word and treated as separate words, and any ending inhibitors are stripped from the end of the word and treated as separate words. All other non-Ward 42 double-byte characters are treated as separate DBCS *words*.

If a word exceeds the available panel space, then the word splits and continues on the next line. If the text consists of mixed data and does not fit in one line within the specified width, the first position is always reserved for a SO character (if first word is double-byte) or for a blank (if the first word is single-byte). This allows the text to be aligned properly.

Words that exceed the width of the available panel space are wrapped to the next line according to following rules:

... CE-1 CE CB CB+1 ...				
CE-1	CE	CB	CB+1	Process
any	B,X	B	X,E	Backward
E	E	X,B	X,E	Backward
X,B	E	any	any	Forward
X,B	X	B	B	Forward
— any other —				No process

Figure 5. Text formatting rules

Where:

CE-1 and CE

Last two words that fit on line

CB and CB+1

First two words on next line

E

Ending inhibitor

B

Beginning inhibitor

X

Neither

Forward

Move CE to next line

Backward

Move CB to previous line

No process

Split as is

Note: If words CE or CB are single-byte words and are more than 1 character, or if CE or CB are double-byte words and are more than 1 double-byte character, then no special processing is used; the line is split as is.

When your panel contains several successive lines of mixed data from different tags, the alignment of a short text string can appear to be shifted 1 byte further left than the surrounding text. This occurs because a text string that fits on one line does not have the leading position reserved for the SO character to use as many positions on the screen as possible.

You can control the alignment of successive lines of mixed data by adding a string of DBCS blanks to the end of a short text string. This forces the SO character position to be reserved during formatting.

SBCS and DBCS blanks that end or begin a line are deleted.

Nesting tags

It is often necessary to code certain tags (and their text) within the definition of other tags (between the start and end tags). This is called *nesting*.

A good example of nesting is the relationship between the DL (definition list) tag, the DT (definition term) tag, and the DD (definition description) tag. The DL tag specifies a definition list and the DT and DD tags specify the terms and descriptions of the items within the definition list. Consequently, the DT and DD tags must be nested within a DL tag and its matching end tag if the list is to format properly.

Here is an example:

```
<dl>
  <dt>This is a definition term.
  <dd>This is a definition description.
  <dt>Another term.
  <dd>Another description.
</dl>
```

Note: Although it isn't required, we indented the nested tags in this example to illustrate nesting levels. You can also do this in your own source files.

There are several tags that must be nested within the actual text of another start tag. These tags serve to identify a condition for the text. In this example, the nested CMD tag follows the CMDTBL start tag and precedes the CMDTBL end tag. The T (truncation) tag nested within the text of the CMD tag provides truncation of the command text.

```
<CMDTBL APPLID=conv>
  <CMD NAME=delete>Del<T>ete
```

```
<CMDACT ACTION=setverb>
</CMDTBL>
```

Nesting tags can take on many different forms and can be complex. For example, some tags allow multiple tags or multiple occurrences of the same tag to be nested, while other tags do not allow nesting of any tags. You can also nest levels of certain tags, that is, nested tags within other nested tags. Additionally, in many instances, you must nest certain tags within other tags. The tag descriptions in [Chapter 12, “Tag reference,” on page 179](#) describe the allowed and required conditions for nesting each of the DTL tags.

Including comments in the generated panel or message member

You can use the COMMENT tag to add comments to the generated panel or message member file. The TYPE attribute specifies the panel section for the comment. TYPE = END is automatically used for message member processing. You provide the comment text in a manner similar to the paragraph tag. ISPD TLC flows the text to a width of 66 bytes and adds "/" before and "/" after each resulting comment line.

Including Copyright Statements in the Generated Panel or Message Member

You can use the COPYR tag to add a copyright statement to the generated panel or message member. The copyright statement is placed in the panel immediately following the)END panel section line, or immediately following the last message in the message member. The text of the COPYR tag is limited to 66 bytes. ISPD TLC adds "/" before and "/" after the copyright text. Each COPYR tag adds one line to the generated panel.

Markup declarations

In addition to tag markup, you can also include *markup declarations* in your source files to define other, related information. Markup declarations are control statements that specify how other markup (such as tags) within a source file is to be interpreted.

For example, in order for the compiler to recognize your source files as being intended for DTL conversion to ISPF elements, you must include a *document type* declaration at the beginning of each source file.

Like tags, markup declarations must be enclosed within a set of delimiter symbols so the compiler can distinguish the declaration as a control statement. All markup declarations are preceded by the <! symbol and followed by the > symbol.

Note: For multicultural support users of DTL the <! symbol can be replaced with the <: symbol.

DTL supports three types of markup declarations:

- Document type declarations
- Comments
- Entity declarations.

Declaring the document type

You must declare the *document type* before you can convert a source file that contains the tag markup for dialog elements. Do this by coding the DOCTYPE declaration at the beginning of the source file. The DOCTYPE declaration looks like this:

```
<!doctype dm system>
```

Where:

<!

Begins the markup declaration

DOCTYPE

Identifies the declaration as a document type declaration

DM

Specifies that the source file contains tags used to define dialog elements for a Dialog Manager application

SYSTEM

Indicates that the syntax rules for defining elements are contained in an external file

>

Closes the markup declaration.

External files that are embedded (through the use of entity declarations) within the source file intended for conversion cannot contain a DOCTYPE declaration. They are converted using the DOCTYPE declaration of the source file they are embedded within. For more information about entity declarations and embedding external files within source files, see [“Defining entities and parameter entities” on page 17](#).

Including comments in your markup

If you want to include notes, reminders, or other text that you don't want processed in your source files, you can insert them as comments, and the conversion utility ignores them.

Note: You cannot place comments within any of the DTL tags. A comment placed within a start or end tag causes the tag to end, and the text following the comment is treated as part of the tag content.

Like document type declarations, comments must be enclosed within markup declaration delimiters (<!-->). However, you must also delimit comments within markup declarations by preceding and following a comment with two dashes (--), like this:

```
<!-- This is the text of the comment -->
```

Because the dashes act as comment delimiters, you can use them in any markup declaration. For example, you can include a comment within a DOCTYPE declaration:

```
<!doctype dm system -- DECLARE DOCUMENT TYPE -->
```

Here is a comment that generates a warning message because the second set of dashes is interpreted as the end of a comment and the text "Provides help for ordering" is treated as an additional markup declaration:

```
<!-- Panel DMH022 -- Provides help for ordering -->
```

If you delete one of the dashes in the second set of dashes, or use another symbol, no error occurs.

```
<!-- Panel DMH022 - Provides help for ordering -->
```

This block comment produces a warning message because of the odd number of dashes in the first and last lines of the block:

```
<!------->
<!--This source file contains all of the -->
<!-- help panels for the application -->
<!------->
```

We could avoid this problem by using a different symbol between the comment dashes, like this:

```
<!--*****-->
<!--This source file contains all of the -->
<!-- help panels for the application -->
<!--*****-->
```

ISPD TLC accepts comments which start with the 4 characters "<!--" and end with the 3 characters "-->". The minimum valid comment is 7 characters ("<!-->").

You cannot nest comments within other comments. You can, however, code multiple comments within a markup declaration, like this:


```
<!-- Here a comment --
-- THERE A COMMENT --
-- Everywhere a comment, COMMENT-->
```

As you can see, each of the comments begin and end correctly with the comment delimiters.

You can use comment delimiters to temporarily ignore multiple lines (or a block) of DTL source text. The block of text might include one or more DTL tags. To *comment out* a block of text, place an "open comment" delimiter before the first line of the text, and a "close comment" delimiter after the last line of text. For example:

```
<!--
<p> This is a multiple line of text block
<p> It is commented out for compile purposes
-->
```

When commenting out multiple lines of DTL source, use the MCOMMENT compiler option when coding the ISPDTLC invocation syntax, or select the **Process multiple line comment blocks** option on the ISPDTLC invocation panel.

Defining entities and parameter entities

You can define, or *declare* frequently used words, phrases, and longer character strings in your source file as *entities* or *parameter entities* that represent text in the source file. You declare them within the DOCTYPE statement of your source file. After you have declared them, you refer to the names of the entities in place of the word or phrase in the text. This saves you time when marking up your text, and allows you to globally change the defined words or phrases in one place in the source file.

You can use entities and parameter entities for these purposes:

- To replace single characters in text that are considered special characters. This can include characters not available on a particular keyboard, or characters that have special meaning to the compiler, such as the tag start delimiter (<), that you want to treat as normal text.

DTL provides you with a set of predefined single-character entities. See [“Predefined entities” on page 23](#) for a list of these entities.

- To replace strings of text, such as words, phrases, and longer text strings used frequently in the source file text.
- To embed entire files in a source file. This is useful for breaking up a source file into smaller, more manageable files, and for declaring entities that are shared by different source files.

When you refer to an entity in the text of a source file, you must precede the entity reference with an ampersand (&) and follow it with a semicolon (;) or a blank space. The text defined by the entity replaces the entity reference in the formatted text.

Entities

Entities are symbolic statements that represent text strings in a source file. Like other markup declarations, entity declarations must be enclosed within markup declaration delimiters (<! >). In addition, you must place entity declarations within the *declaration subset* of the DOCTYPE statement.

The declaration subset is delimited by left and right brackets ([]) or parentheses () and is coded within the DOCTYPE statement. If left and right brackets are coded, they must have the hex values of ‘AD’ and ‘BD’ respectively.

Within the markup declaration delimiters, you declare the entity with the term "entity", the name you are assigning to the entity, and the text string the name represents. The text string of the entity must be enclosed in single or double quotes.

```
<!doctype dm system (
<!entity name "text string">
)>
```

Entity names must have these characteristics:

- 1-17 characters
- The first character must be alphabetic (A-Z, a-z, @, #, or \$)
- Remaining characters, if any, can be A-Z, a-z, @, #, \$, 0-9, or _
- Entity names are case-sensitive.
- Entity names of more than 8 bytes must contain at least 1 underscore character.

This example declares an entity named "guar" for the phrase "full, unconditional, money-back guarantee":

```
<!doctype dm system [  
  <!entity guar "full, unconditional, money-back guarantee">  
>]
```

Now that we've declared the entity, we can use the entity name in our source file text instead of the entire text string. To specify an entity name in text, you must precede the name with an ampersand (&) and follow it with a semicolon (;) or a blank, as we did in this panel text:

```
<!doctype dm system [  
  <!entity guar "full, unconditional, money-back guarantee">  
>]  
<panel name=widget21 width=40>Widgets  
  <area>  
    <info width=38>  
      <p>You'll love the wide selection of merchandise  
        in our Widgets department.  
      <p>And, like all of our merchandise, Widgets come  
        with our &guar;.  
    </info>  
  </area>  
</panel>
```

As long as we declared the entity properly, the compiler recognizes the entity reference in the source file and replaces it with the text of the entity declaration. [Figure 6 on page 18](#) shows the result.

```
Widgets  
  
You'll love the wide selection of  
merchandise in our Widgets department.  
  
And, like all of our merchandise,  
Widgets come with our full,  
unconditional, money-back guarantee.
```

Figure 6. Entity reference for text substitution

We can refer to the same entity in the text of the source file as many times as we like. If we should ever want to change the text of the entity, we only have to do it in one place: the declaration subset.

A change to the previous example will show you what we mean.

```
<!doctype dm system [  
  <!entity guar "partial, conditional, non-refundable guarantee">  
>]  
<panel name=widget22 width=40>Widgets
```

```

<area>
  <info width=38>
    <p>You'll love the wide selection of merchandise
      in our Widgets department.
    <p>And, like all of our merchandise, Widgets come
      with our &guar;.
  </info>
</area>
</panel>

```

The only change we made was to the text of the entity declaration, not the entity name. Following reformatting, the text of the entity reference now looks like this:

```

      Widgets

You'll love the wide selection of
merchandise in our Widgets department.

And, like all of our merchandise,
Widgets come with our partial,
conditional, non-refundable guarantee.

```

Figure 7. Entity reference for text substitution

If, for any reason you need to change the name of an entity, be sure to update all of the references to the entity name in your text.

You can also define the text of an entity in an external file and refer to that file in an entity declaration. If you do this, you must include the SYSTEM parameter in the entity declaration, to indicate to the conversion utility that the file is external.

Note: You must include the external file in the concatenation of DTL source files defined to the conversion utility.

For example, we'll define a text string we want to use as an entity in our source file in a file called WIDGETS. Here are the contents of the WIDGETS file:

```
doohickeys, whatnots, and gizmos
```

To declare this file in the entity declaration in our source file, we code it like this, with the SYSTEM parameter:

```

<!doctype dm system [
<!entity guar "full, unconditional, money-back guarantee">
<!entity widgets system>
]>

```

If we want to use the text string in our source file, we refer to the entity "widgets" (in this case, the file name also serves as the entity name).

```

<!doctype dm system [
<!entity guar "full, unconditional, money-back guarantee">
<!entity widgets system>
]>
<panel name=widget23 width=42>More Widgets
  <area>
    <info width=40>
      <p>The fine selection of items in our Widgets department

```

```
includes &widgets;.
<p>And, like all of our merchandise, Widgets come with
our &guar;.
</info>
</area>
</panel>
```

Figure 8 on page 20 shows the formatted result.

```
More Widgets

The fine selection of items in our
Widgets department includes doohickeys,
whatnots, and gizmos.

And, like all of our merchandise,
Widgets come with our full,
unconditional, money-back guarantee.
```

Figure 8. Entity reference for text substitution and file embedding

Anytime we want to update or change the text of the entity, we only need to change the text in the WIDGETS file.

In the previous example, the name "widgets" serves as the external file name and as the entity name.

The SYSTEM parameter may optionally be followed by the file name for the included file. When the SYSTEM parameter is used but no file name is provided, the entity name is used as the file name.

For instance, if you want to declare a different entity name for the WIDGETS file, "things" for example, code it like this in the entity declaration:

```
<!doctype dm system [
<!entity guar "full, unconditional, money-back guarantee"
><!entity things system "widgets">
]>
```

Refer to the entity name, *things*, like this:

```
<!doctype dm system [
<!entity guar "&full, unconditional, money-back guarantee">
<!entity things system "widgets">
]>
<panel name=widget24 width=42>More Widgets
  <area>
    <info width=40>
      <p>The fine selection of items in our Widgets department
includes &things;.
      <p>And, like all of our merchandise, Widgets come with
our &guar;.
    </info>
  </area>
</panel>
```

The formatted result of this markup is the same as that shown in [Figure 8 on page 20](#), assuming no changes were made to the text of the WIDGETS file.

Parameter entities

Parameter entities allow you to place multiple entity declarations within an external file and refer to them within a source file. To embed the entities into the source file, you must declare the external file as a parameter entity. A parameter entity is identified by a percent symbol (%) following the term "entity" and followed by a space and the entity name. See [“Entity declarations” on page 172](#) for the syntax description. You refer to a parameter entity within the DOCTYPE statement by preceding the entity name with a percent symbol (%) and following it with a semicolon (;). This embeds the parameter entity file and allows its entities to be referred to in the source file.

For example, we've declared all of our entities within an external file called SYMBOLS. Here are the contents of the SYMBOLS file:

```
<!ENTITY sb "ShelfBrowse">
<!ENTITY cotime "ten days">
<!ENTITY xcotime "five days">
<!ENTITY nttime "three days">
<!ENTITY nitem "red checkout card">
<!ENTITY lfine "ten cents">
<!ENTITY cophone "555-1234">
```

The conversion utility locates the parameter entity using these rules for entity external files.

We can embed the SYMBOLS file into the declaration subset of the source file with a parameter entity declaration within the DOCTYPE statement. As long as we declare the parameter entity and refer to it properly, we can use any of the declared entities in the external file in the text of the source file.

```
<!doctype dm system
[<!entity % SYMBOLS system> %SYMBOLS;]>
<panel name=chkout width=40 depth=22>Library Checkout Periods
  <area>
    <info width=38>
      <p>&sb; allows you to check out an inventory
        item for a maximum of &cotime;.
        However, you can renew the item for an additional
        &xcotime; by calling in your card number to our
        checkout phone line (&cophone;) any time of day.
      <p>If an inventory item is a new shelf item
        (indicated by the &nitem;), you may only reserve it for
        a maximum of &nttime;.
        You may not renew a new shelf item.
      <p>There is a fine of &lfine; per day for all
        items returned late.
    </info>
  </area>
</panel>
```

[Figure 9 on page 21](#) shows the formatted result.

Library Checkout Periods

ShelfBrowse allows you to check out an inventory item for a maximum of ten days. However, you can renew the item for an additional five days by calling in your card number to our checkout phone line (555-1234) any time of day.

If an inventory item is a new shelf item (indicated by the red checkout card), you may only reserve it for a maximum of three days. You may not renew a new shelf item.

There is a fine of ten cents per day for all items returned late.

Figure 9. Parameter entities

Markup declarations

Parameter entity names must have these characteristics:

- 1-8 characters
- The first character must be alphabetic (A-Z, a-z, @, #, or \$)
- Remaining characters, if any, must be A-Z, a-z, @, #, \$, or 0-9
- Parameter entity names are case-sensitive.

Embedding source files

You can also use entities to embed entire files within your source file. For example, you could define common variables for several panels in your source file in a separate file. These separate files are stored as members of any input library specified to ISPD TLC. Here is markup that shows the contents of a file called VARDEFS.

```
<varclass name=titlcls type='char 50'>
<varclass name=bookcls type='char 20'>
<varclass name=pagecls type='char 5'>
<varclass name=datecls type='char 8'>

<varlist>
  <vardcl name=title      varclass=titlcls>
  <vardcl name=author     varclass=bookcls>
  <vardcl name=publish    varclass=bookcls>
  <vardcl name=pages      varclass=pagecls>
  <vardcl name=curdate    varclass=datecls>
</varlist>
```

Another common markup file could be defined for an action bar. Here is markup that shows a portion of the contents of a file called ACTNBAR.

```
<ab>
  <abc>File
    <pdcl>Add Entry
      <action run=add>
    <pdcl>Delete Entry
      <action run=delete>
    <pdcl>Update Entry
      <action run=update>
    <pdcl>Exit
      <action run=exit>
  <abc>View
  :
  <abc>Options
  :
  <abc>Help
  :
</ab>
```

We can embed these files in a source file by coding entity references to the files in the source file DOCTYPE statement.

```
<!doctype dm system [
<!entity actnbar system>
<!entity vardefs system]>>

&vardefs;

<panel name=dfdxmp21>Library Inventory

&actnbar;

  <topinst>To add a book to the inventory, complete the fields below,
    and then press Enter.
  <area>
    <dtafld datavar=title usage=in pmtwidth=14>Title
    <dtafld datavar=author usage=in pmtwidth=14>Author
    <dtafld datavar=publish pmtwidth=14>Publisher
    <dtafld datavar=pages usage=in pmtwidth=14>Number of pages
    <divider type=solid gutter=3>
    <dtafld datavar=curdate usage=out pmtwidth=20>Today's date is
  </area>
</panel>
```

The variable definitions in VARDEFS are referred to by the data fields in the panel because the file was embedded into the source file through the entity declaration. In the previous example, the entry width information for each field is obtained from the variable definitions.

File embed entity names must have these characteristics:

- 1-8 characters
- The first character must be alphabetic (A-Z, a-z, @, #, or \$)
- Remaining characters, if any, must be A-Z, a-z, @, #, \$, or 0-9
- Entity names are case-sensitive.

Runtime substitution variables

If you need to include a dialog variable within your panel source that will be substituted at run time, the output panel must be created to contain an "&variable" string. An example would be a reference to an ISPF variable such as &ZDATE.

The conversion utility always tries to substitute each "&variable" found at conversion time with the available entity definitions. If the conversion utility can't find an entity definition, it issues a warning message, and then passes the original "&variable" into the output panel.

To avoid the warning message, you can use the predefined entity "&". You can code the variable in the tag source as "&&variable" to make "&variable" appear in the panel.

You should use caution when designing panels that contain runtime substitution variables. The regular panel formatting process might not leave sufficient space in the panel text line for the variable value to be inserted. For example, a variable name of "&date" that requires 10 positions (YYYY/MM/DD) should be coded as "&date(10);".

Predefined entities

The Dialog Tag Language provides you with a set of predefined entities that you can use in your source files. You can use them when the symbol you want is not present on your keyboard, or conflicts with a conversion utility delimiter symbol.

You do not need to declare a predefined entity to use it. If you use the entity in your source file as you would an entity that you declare within your document subset, the conversion utility performs the substitution for you. You should always use the pre-defined entities for all symbols that are used as part of the tag language syntax.

The Dialog Tag Language predefined entities include:

- >sym;**
greater than (>)
- <sym;**
less than (<)
- :**
colon (:)
- &**
ampersand (&)
- ;**
semicolon (;)
- .**
period (.)
- "e;**
single quote (')
- &dquote;**
double quote (")

Markup declarations

–

short dash (–)

¬

not symbol (¬)

&us;

underscore (_)

∨

logical or (||)

&sl;

back slash (\)

&lbrk;

left bracket ([)

&rbrk;

right bracket (])

&lbrc;

left brace ({)

&rbrc;

right brace (})

−

minus sign (-)

+

plus sign (+)

&rbl;

required blank ()

&tpl;

text placeholder ()

&eqsym;

equal sign (=)

&rdb;

required SBCS blank in DBCS mode ()

&percent;

percent sign (%)

˙

dot (.)

&cmdpmt

command prompt (= = >)

&rprr

right pointer (-->)

Any of these predefined entities can be coded with a replication factor. For example, >sym(5); creates the string '>>>>>' in the substituted text.

Multicultural support text strings are also accessible as entities:

&more

More

&caution

CAUTION

¬e

Note

&warning

Warning

&command

Command

&alpha

abcdefghijklmnopqrstuvwxyz

&scroll

Scroll

&option

Option

&horizdiv

|

&multihst

Enter "/" to select option

&multigui

Check box to select option

&release

Release

&maintlvl

Level:

&created

Created -

&datetext

Date:

&timetext

Time:

¬es

Notes

&attentn

Attention

&tutorial

Tutorial

Points to remember:

1. Some of the symbols defined in the preceding list do not display on some non-programmable terminals.
2. The &rbld; predefined entity creates one blank in the resulting panel text. To place three required blanks in a text string, for example, you should code &rbld;&rbld;&rbld; in your tag source file.
3. The &tpl; predefined entity uses a hex FF code to reserve a space in DTL formatted text. After formatting is completed, the hex FF character is replaced by a blank. As with any predefined entity, you can change this default to another value. The current value of &tpl; is used for post-formatting text replacement. Thus, if you prefer to use an @ as the reserved space character, define the entity in this way:

```
<! ENTITY TPL '@'>
```

If multiple reserved spaces are required, you could use these entity definitions to reserve 10 characters:

```
<! ENTITY TPL '@'>
<! ENTITY MYTPL '@@@@@@@@@@'>
```

To use your own entity name, first define TPL to override the system default character for text replacement. Second, add your entity definition, using the specified override character. When the &tpl; is changed, be careful to select a character that is not otherwise used in your panel.

Markup declarations

4. The `&rdb;` predefined entity generates an SBCS blank when ISPD TLC is processing in DBCS mode, or a null character when processing in SBCS mode.
5. The `˙` predefined entity generates a dot (or period) character in the text. The number of spaces following the `˙` in the DTL source is maintained in the formatted panel.

Chapter 3. Getting started: designing application panels

Each application panel you create serves a specific purpose, with unique fields, messages, and help information defined for each one. This chapter explains how to define elements that are common among application panels. This includes defining the application panels, and the interactive elements of panels, including action bars, instruction text, and command areas. We also tell you how to arrange the contents of your application panels using panel regions and dividers.

The PANDEF tag allows you to define in one place common attributes and values for the panels in your application—see [“Defining panel defaults” on page 50](#).

Defining application panels: the PANEL tag

You use the panel tag, its associated attributes, and the required panel end tag to define an application panel and the specific characteristics of the panel.

The PANEL start and end tags define the beginning and ending of an application panel. The PANEL start tag defines:

- Panel name
- Name of the help panel for the application panel
- Name of the panel default
- Dimensions of the panel
- Associated key mapping list
- KEYLTYPE value
- APPLID value
- Cursor placement
- CCSID number
- MENU keyword
- PRIME keyword
- TUTOR keyword
- WINDOW value
- WINTITLE value
- APPTITLE value
- PAD value
- PADC value
- OUTLINE value
- EXPAND value.
- MSGLINE value
- TITLINE value
- CMDLINE value
- ATTRUSE value
- ENDATTR value
- TYPE value
- SMSG value

- LMSG value
- ASIS keyword
- ACTBAR keyword
- MERGESAREA value
- PANELSTMT value
- ENTKEYTEXT value
- IMAPNAME value
- IMAPROW value
- IMAPCOL value
- TMARGIN value
- BMARGIN value
- ERRORCHECK value
- ZUP value
- ZCONT value
- AUTONRET value
- AUTOTCMD value
- Panel title text

With the exception of the required NAME attribute used to identify the name of the application panel, all of the attributes for the PANEL tag are optional. Many attributes have default values that the conversion utility assumes if you do not specify the attribute. This topic describes these attributes, and how to use them.

The PANEL start and end tags look like this, respectively:

```
<panel name=mainpan>
.
.</panel>
```

In the preceding example, we included the required NAME attribute and its value *mainpan* on the PANEL start tag. ISPF requires that each panel definition contain this attribute and an associated value to identify the panel. The panel name is also used as the panel ID when the panel ID is displayed. The “NAME=*” notation sets the panel name to be the same as the member name of the input DTL source file. If multiple panel definitions have been combined within a single source file, then this notation should be used for only one panel definition within the file.

The *panel name* must follow the naming convention described in [“Rules for variable names”](#) on page 179.

Note: During conversion when the PREP option is active, the conversion utility uses a temporary PDS to store ISPF source format panels.

The file name for interactive use is: *tsoprefix*.TEMPDTLW.DTLPAN*nn*, or, if the TSO NOPREFIX profile option is in effect, the file name is: *tsouserid*.TEMPDTLW.DTLPAN*nn*. *nn* is the screen number.

For batch, the file name is: *tsoprefix*.TEMPDTLW.DTLBATCH.Ttttttt.Rnnnnn, or, if the TSO NOPREFIX profile option is in effect, the file name is: *tsouserid*.TEMPDTLW.DTLBATCH.Ttttttt.Rnnnnn.

The batch file name is uniquely created for each ISPD TLC invocation by including the system time and a random number as the last two qualifiers of the name.

The ISPPREP utility is called to convert all of the generated panels from ISPF source format to preprocessed format at one time to improve performance.

The panel title

The text that appears as the title of the panel is called the title text. You define the title text by coding it as tag text for the PANEL start tag.

This example uses the text "Catalog Ordering System" as title text:

```
<panel name=mainpan>Catalog Ordering System
:
</panel>
```

Panel size (width and depth)

Use the DEPTH and WIDTH attributes of the PANEL tag to define the size of an application panel. The PANEL tag has a default WIDTH value of 76 characters and a default DEPTH value of 22 lines. If you specify WINDOW=NO, the default WIDTH is 80 and the default DEPTH is 24. These are the values the conversion utility assumes if you do not specify dimensions for WIDTH and DEPTH.

Here is an example that defines the panel size as 60 characters wide and 15 lines deep:

```
<panel name=mainpan width=60 depth=15>Catalog Ordering System
:
</panel>
```

To make the width of the panel 76 characters (the default width), we only need to specify a value for DEPTH, as in this markup:

```
<panel name=mainpan depth=15>Catalog Ordering System
:
</panel>
```

This results in a panel with a default width of 76 characters and a specified depth of 15 lines.

Because you can display application panels in pop-ups, you should allow for pop-up borders (added by ISPF at run time) when you define the WIDTH and DEPTH values for application panels. When the panel is displayed in a pop-up, ISPF adds two lines to the depth specified and 4 characters to the width specified for pop-up borders. Remember that ISPF cannot display a panel whose size exceeds the device size and issues an error message at run time in this situation.

Key mapping lists

To specify the function keys that are active for an application panel, use the KEYLIST attribute of the PANEL tag. This attribute specifies the name of the key mapping list you define for use with the panel. A key mapping list contains the keys that are active while the panel is displayed. The key mapping list also specifies what command is run when each key is pressed.

This PANEL definition refers to a key mapping list named *key01*:

```
<panel name=mainpan keylist=key01>Catalog Ordering System
:
</panel>
```

For more information about defining key mapping lists, see [Chapter 9, "Defining key mapping lists,"](#) on [page 147](#).

Associated help panels

To provide help for an application panel (also called *extended help*), specify the name of the associated help panel with the HELP attribute of the PANEL tag. The help panel you specify appears when the user requests extended help while in the application panel or when contextual help is requested for an item on the panel, but no contextual help is available for the item. The help panel you specify is also displayed when the user requests extended help while in a contextual help panel associated with an item on the panel.

This panel definition refers to a help panel named *ordhelp*:

```
<panel name=mainpan help=ordhelp>Catalog Ordering System
:
</panel>
```

[“Help panels” on page 130](#) tells you how to create help panels for your application.

Panel defaults

The PANEL tag attribute PANDEF provides the name of a panel default definition. Attribute values defined on the named PANDEF tag are used for the current panel unless the attribute has also been specified on the PANEL tag.

Cursor placement

The PANEL tag attributes, CURSOR, CSRINDEX, and CSRPOS, allow you to specify where the cursor is placed when the panel is initially displayed. If you do not specify a specific cursor position, ISPF places the cursor in the first field in the PANEL definition that can contain the cursor.

Use the CURSOR attribute to specify the field that is to contain the cursor. Use the CSRINDEX and CSRPOS attributes to identify positions within the field you specify with the CURSOR attribute. CSRINDEX and CSRPOS can only be used when the CURSOR attribute is used.

The CURSOR attribute

Use the CURSOR attribute to specify the value of the NAME attribute of a CHOICE or SELFLD tag, or the value of the DATAVAR attribute of a CHOFLD, DTAFLD or LSTCOL tag. Here are the characteristics of cursor placement:

CHOFLD

The cursor appears in the first character position of the choice field. Cursor positioning is valid only when the USAGE attribute of the CHOFLD tag specifies INPUT or BOTH.

CHOICE

The cursor appears in the entry field of the specified choice in a multiple-choice selection field.

DTAFLD

The cursor appears in the first character position of the data field. Cursor positioning is valid only when the USAGE attribute of the DTAFLD tag specifies INPUT or BOTH.

LSTCOL

The cursor appears in the first row in the list column. Cursor positioning is valid only when the USAGE attribute of the LSTCOL tag specifies INPUT or BOTH.

SELFLD

The cursor appears in the entry field of the specified single-choice selection field.

Chapter 5, “Application panel fields,” on page 71 provides a complete description of the types of interactive fields you can define for your application panels.

You can also place the cursor in the command area of the panel by specifying *cmdarea* as the CURSOR value.

[“Defining a command area” on page 48](#) provides a complete description of the CMDAREA tag.

Here is an example where the CURSOR attribute specifies the data field DATAVAR value *place*. When the panel is initially displayed, the cursor appears in the first character position of that field. [Figure 10 on page 31](#) shows the formatted result.

```
<!doctype dm system>
<panel name=mainpan1 cursor=place>Travel Agency
  <selfld name=dest selwidth=50 pmtwidth=15>Destinations:
    <choice>London
    <choice>Madrid
    <choice>Paris
    <choice>Zurich
```

```

</selfld>
<divider>
<dtafld datavar=place entwidth=9 pmtwidth=5>Other
<cmdarea>
</panel>

```

This example, and other examples in this chapter, include tag markup for elements such as fields and variables that have not yet been discussed, to illustrate the formatting characteristics of some tags. The syntax of these elements are not important for the purposes of these examples. Syntax conventions of these elements in discussed in later chapters of this document.

Travel Agency

Destinations:
 -- 1. London
 2. Madrid
 3. Paris
 4. Zurich

Other -----

Command ==> -----

Figure 10. Cursor placement

If no cursor placement was specified in the PANEL tag for the preceding example, the cursor would appear in the entry field of the **Destinations** single-choice selection field when the panel is initially displayed.

The CSRINDEX attribute

To place the cursor in a table row within a list field, use the CURSOR attribute to specify the data variable name for a list column within the list field, and the CSRINDEX attribute to specify the table row number where the cursor should be placed. The value you assign to CSRINDEX must be numeric.

The CSRPOS attribute

If you use the CURSOR attribute to place the cursor within an input-only, or input/output data field or list column, or the command area, you can also define a specific character position for the cursor using the CSRPOS attribute.

The value you assign to the CSRPOS attribute must be numeric. This numeric value indicates the number of character positions from the left margin of the field where the cursor is placed, where a 1 specifies that the cursor should be in the first character position.

Other panel attributes

See “[PANEL \(Panel\)](#)” on page 376 for more information.

KEYLTYPE

This attribute is used to add the SHARED keyword to the KEYLIST parameter of the)PANEL statement.

APPLID

This attribute is used to add the application ID to the KEYLIST parameter of the)PANEL statement.

CCSID

This attribute specifies the coded-character-set identifier as defined by the Character Data Representation Architecture.

MENU

This attribute specifies that the panel is an ISPF menu selection panel.

PRIME

This attribute is used with the MENU attribute to specify an ISPF primary option menu.

TUTOR

This attribute specifies that the panel is to be an ISPF tutorial panel.

WINDOW

This attribute is used to control the generation of the WINDOW keyword on the panel)BODY statement.

WINTITLE

This attribute is used to add a title on a pop-up window border.

APPTITLE

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PAD

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

PADC

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

OUTLINE

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

EXPAND

This attribute causes the ISPF EXPAND keyword to be added to the panel)BODY statement.

MSGLINE

This attribute controls the provision for a long message line in the generated panel. When MSGLINE=NO, the blank line for the long message is not added to the panel)BODY section.

TITLINE

This attribute controls the provision for a panel title line in the generated panel. When TITLINE=NO, the title line is not added to the panel)BODY section. This attribute allows a panel formatted as a dynamic area to provide the panel title as part of the dynamic area data.

CMDLINE

This attribute controls the automatic addition of the command area to a menu selection or table display panel. When CMDLINE=NO, the command area is not automatically generated when the CMDAREA tag is not present in the DTL source file.

ATTRUSE

This attribute controls the use of panel attribute characters in the range of x'01' through x'2F'. When ATTRUSE=YES, dynamic area attributes (specified with the ATTR tag) can be assigned low-order hex values normally reserved for use by the conversion utility.

ENDATTR

This attribute specifies that when the last attribute on any panel body line is 'normal text' (CUA), it is replaced by the default 'text' (ISPF) attribute.

TYPE

This attribute specifies that the panel is used for host display, for display by a client that is using the JSON API, or both.

SMSG

This attribute provides the name of the field where the short message is to be placed.

LMSG

This attribute provides the name of the field where the long message is to be placed.

ASIS

This attribute specifies that the command and long message fields are to appear on the display as specified in the generated panel definition. When ASIS is specified, any user request specified on the Settings panel, or by setting the system variable ZPLACE is ignored.

ACTBAR

This attribute causes the action bar information for the panel to be generated, overriding the NOACTBAR invocation option.

MERGESAREA

This attribute specifies that a panel with a single scrollable area be reformatted to combine the scrollable area into the panel body.

PANELSTMT

This attribute controls the creation of the)PANEL statement.

ENTKEYTEXT

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAME

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPROW

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPCOL

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

TMARGIN

This attribute provides the number of blank lines to format at the top of the panel as a top margin.

BMARGIN

This attribute provides the number of blank lines to format at the bottom of the panel as a bottom margin.

ERRORCHECK

This attribute specifies that error checking code is added to the)PROC panel section.

ZUP

This attribute provides the name of the tutorial panel to be assigned to the ZUP variable.

ZCONT

This attribute provides the name of the tutorial panel to be assigned to the ZCONT variable.

Defining action bars and pull-downs

To create a consistent user interface, you should design your applications according to the object-action process sequence defined by the SAA Common User Access. The action bar is a major user interface component that helps you achieve consistency in your applications.

The action bar is the panel element located at the top of an application panel that contains action bar choices for the panel. Each action bar choice represents a group of related choices that appear in the pull-down associated with the action bar choice. When the user selects an action bar choice, the associated pull-down appears directly below the action bar choice. Pull-downs contain choices that, when selected by the user, perform actions that apply to the contents of the panel.

Panel design note:

ISPF and DTL provide the tools to help you create the object-action process sequence in your application, but it is your responsibility as an application designer to ensure that the contents of your action bar are actions that can be applied to the objects contained within your panel.

Typically, application panels intended for display within primary windows contain action bars that present the user with all of the available actions that apply to that panel. Application panels that are displayed as pop-ups should not include the action bar. Instead, actions for a pop-up panel are presented in the function key area.

Figure 11 on page 34 shows an action bar with the File pull-down menu displayed.

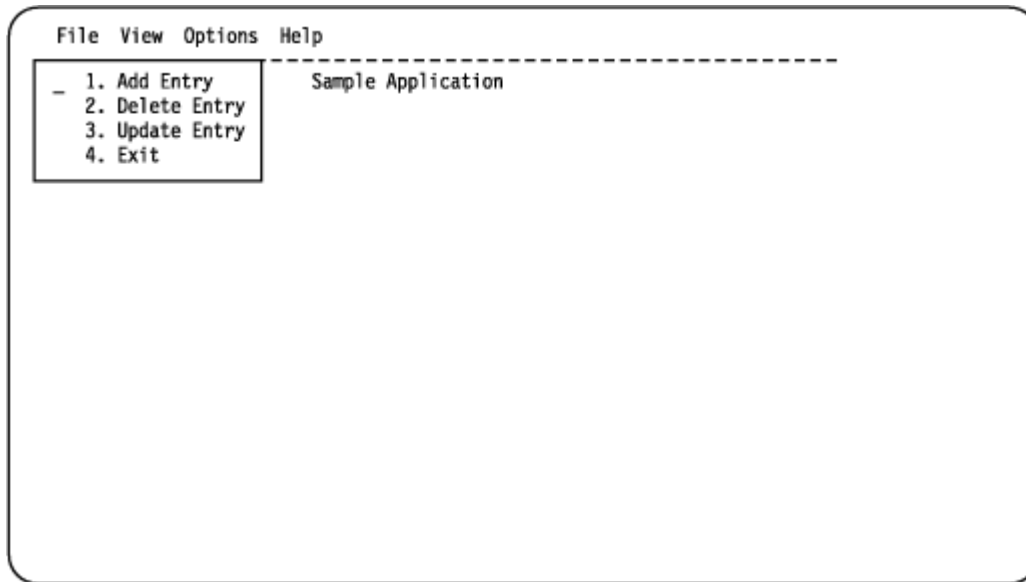


Figure 11. Action bar and pull-down

The tags you use to create the action bar and its pull-down menus are:

AB

To start an action bar definition. The required AB end tag ends an action bar definition.

ABC

To define each of the action bar choices.

PDC

To define the choices on the pull-down associated with an action bar choice.

ACTION

To specify an action to be taken when the pull-down choice is selected. The ACTION tag is coded within the PDC tag.

M

To specify a mnemonic character for action bar choice or pull-down choice selection.

Coding an action bar definition

This list describes how to code an action bar definition:

- Code the AB start tag immediately after the PANEL start tag and before any other tags in the panel.
- Following the AB start tag, code an ABC tag for each action bar choice in the action bar. The text you specify on the ABC tag is the text that appears in the action bar as the action bar choice.
- Code the associated PDC tags within the ABC tags. The text you specify on the PDC tag is the text that appears as the pull-down choice.

- Following each PDC tag, code one or more ACTION tags to specify what type of action occurs when that pull-down choice is selected by the user.

The ACTION tag RUN attribute (and its *internal-command-name*) define a command action for the pull-down choice. If you define multiple ACTION tags for a pull-down choice, one of which contains a RUN value, code the RUN action last, because any actions specified after a RUN action are ignored.

- End the action bar definition with the required AB end tag.

Here is an example that shows the markup for the action bar shown in [Figure 11 on page 34](#). The detailed markup for the **File** pull-down is included.

```
<panel name=mainpan2 depth=15>Sample Application
  <ab>
    <abc>File
      <pdc>Add Entry
        <action run=add>
      <pdc>Delete Entry
        <action run=delete>
      <pdc>Update Entry
        <action run=update>
      <pdc>Exit
        <action run=Exit>
    <abc>View
  :
  <abc>Options
  :
  <abc>Help
  :
  </ab>
  :
</panel>
```

Pull-down choice actions

A pull-down choice provides an immediate action to the user. To ensure that a pull-down choice performs an immediate action, you should code an ACTION tag that specifies the RUN attribute for each pull-down choice. The value you assign to RUN tells ISPF which command to run when the user selects the choice.

In the preceding markup example, each ACTION definition uses the RUN attribute to specify a command. Each of these commands must be defined within the command table for the application. [Chapter 8, “The application command table,” on page 143](#) tells you how to define commands for an application.

In addition to the RUN action, you can specify other types of actions to occur when a pull-down choice is selected. The SETVAR and TOGVAR attributes on the ACTION tag can be used to set and toggle variables which the application can use to determine the processing to perform.

Remember, any SETVAR or TOGVAR actions for a pull-down choice must be coded before any ACTION definition specifying the RUN action, because actions coded after RUN are ignored.

A pull-down choice may be marked as unavailable. The UNAVAIL attribute is used to provide a variable name that is used by ISPF to determine the availability of the pull-down choice. When the variable value is 1, the pull-down choice is unavailable.

Action bar help

You can provide help for each action bar choice and pull-down choice with the HELP attribute on the ABC and PDC tags, respectively. By specifying the name of a help panel or message for the action bar choice or pull-down choice, ISPF knows which help information to display when the user requests help on that choice. If you do not specify help for a pull-down choice, the help for the action bar choice is displayed, when the user requests help. If there is no help defined for the action bar choice, the extended help panel is displayed.

Defining action bars and pull-downs

This example adds the HELP attribute to each of the action bar choices and pull-down choices in the action bar defined in [“Coding an action bar definition” on page 34](#). The values specified with each HELP attribute are the NAME values of defined help panels.

```
<!doctype dm system>
<panel name=mainpan3 width=50 depth=15>Sample Application
  <ab>
    <abc help=hfile>File
      <pd help=hnew>Add Entry
        <action run=add>
      <pd help=hopen>Delete Entry
        <action run=delete>
      <pd help=hsave>Update Entry
        <action run=update>
      <pd help=hexit>Exit
        <action run=Exit>
    <abc help=hview>View
  :
  <abc help=hoption>Options
  :
  <abc help=hhhelp>Help
  :
</ab>
:
</panel>
```

In the preceding example, we defined a help panel named *hhhelp* for the Help action bar choice.

Common User Access requires that you put the Help action bar choice as the last action bar choice in an action bar definition. You should code the Help action bar pull-down in this way:

```
<abc help=hhhelp>Help
  <pd>Extended help
    <action run=exhelp>
  <pd>Keys help
    <action run=keyshelp>
</abc>
```

[“Help panels” on page 130](#) tells you how to define help panels.

Preselected pull-down choices

You can define a pull-down choice as being *preselected* with the CHECKVAR and MATCH attributes of the PDC tag. The CHECKVAR attribute specifies the name of a variable that you set at run time to indicate if the pull-down choice should be preselected. The MATCH attribute defines a value that causes the choice to be preselected. ISPF compares the value of the variable named for the CHECKVAR attribute to the MATCH value, and if they are equal, the choice appears preselected when the pull-down is displayed.

Continuing with the library application, assume that the user can view the files in the library sorted by name, owner, date, or size. Preselecting a pull-down choice provides a visual cue to the user of the current sort order.

To preselect any of the pull-down choices, the same CHECKVAR value is specified for each choice, and a unique MATCH value is specified for each choice. The application variable specified with CHECKVAR is set to the MATCH value to indicate the sorting option being used. The variable specified with CHECKVAR is changed each time the sorting option is changed. This provides a visual reminder to the user of how the files are sorted.

Mnemonic choice selection

ISPF supports mnemonic selection of action bar choices.

Mnemonic selection of action bar choices is automatically determined by ISPD TLC when a non-DBCS conversion is in process. When DBCS is specified, mnemonics are not automatically generated. The default mnemonic character generation can be overridden by adding the MNEMGEN=NO attribute to the AB tag for non-DBCS conversions. The mnemonic character that is selected is the first alphabetic

or numeric character from the current action bar choice description text that is not already used as a mnemonic character within the action bar. If a unique mnemonic character cannot be selected, the conversion utility issues a message. DBCS characters cannot be specified as mnemonics. See [“M \(Mnemonic\)” on page 351](#) for a description of how to provide a mnemonic character that is not part of the normal choice description.

Mnemonic selection of action bar choices may be specified by placing the M tag immediately in front of the character to be used as a mnemonic within the ABC text.

The automatic mnemonic generation does not replace any valid mnemonic specified by the M tag. (If the mnemonic character specified by the M tag is a duplicate of a mnemonic character previously selected by the generation process, a message is issued and ISPD TLC attempts to replace the duplicate value that was specified.) This processing allows the combination of specific character selection with the automatic generation feature, as long as the characters automatically generated and the characters specified (by the M tag) are unique.

```
<!doctype dm system (
  <!ENTITY actnfile system>
  <!ENTITY actnoptn system>
  <!ENTITY actnhelp system>
)>

<panel name=pdcxmp1>Sample Application
  <ab>
    &actnfile;

    <abc>View
      <pdccheckvar=sorttype match=N>Name
        <action run=name>
      <pdccheckvar=sorttype match=O>Owner
        <action run=owner>
      <pdccheckvar=sorttype match=D>Date
        <action run=date>
      <pdccheckvar=sorttype match=S>Size
        <action run=size>

    &actnoptn;
    &actnhelp;
  </ab>

  <topinst>
  <area>
  </area>
  <cmdarea>
</panel>
```

If the application sets the variable *sorttype* to "D" before the panel is displayed, then the **Date** choice is preselected.

Defining the panel body

In this topic we tell you how to use DTL to define elements of the panel body such as instruction text, areas, regions, and dividers.

Panel instructions

DTL provides you with the TOPINST, PNLINST, and BOTINST tags to define instructions for your application panels. None of the tags have required end tags associated with them.

Use the instruction tags to provide text that tells the user how to interact with the panel or how to continue with an application.

If the COMPACT attribute is not specified, a blank line is added to the panel after each TOPINST tag and before each PNLINST or BOTINST tag.

You must code the top and bottom instruction tags outside of the portion of the panel defined with the AREA tag and its matching end tag. ([“The AREA tag” on page 38](#) explains how to use the AREA tag). Code the TOPINST tag immediately after the action bar definition (or the PANEL start tag if the panel does

not contain an action bar). Code the BOTINST following the main body of the panel, before the PANEL end tag. You may code PNLINST tags within the AREA tag.

This application panel markup contains both types of instructions. [Figure 12 on page 38](#) shows the results.

```
<!doctype dm system>
<panel name=mainpan5>Item Selection
  <topinst>Select one of the following items and press Enter.
    <selfld name=itemtyp selwidth=76>
      <choice>Automotive
      <choice>Hardware
      <choice>Health and beauty
      <choice>Lawn and garden
      <choice>Sporting goods
    </selfld>
  <botinst>To exit the application, press F3.
</panel>
```

Item Selection

Select one of the following items and press Enter.

-
1. Automotive
 2. Hardware
 3. Health and beauty
 4. Lawn and garden
 5. Sporting goods

To exit the application, press F3.

Figure 12. Top and bottom instructions

The AREA tag

The AREA tag (and its matching end tag) defines the main portions of the panel body. You code all of the interactive fields for the panel within AREA definitions.

Add an AREA definition to the previous application panel markup.

```
<!doctype dm system>
<panel name=mainpan6 depth=18>Item Selection
  <topinst>Select one of the following items and press Enter.
    <area>
      <selfld name=itemtyp selwidth=76>
        <choice>Automotive
        <choice>Hardware
        <choice>Health and beauty
        <choice>Lawn and garden
        <choice>Sporting goods
      </selfld>
    </area>
  <botinst>To exit the application, press F3.
</panel>
```

As stated in [“Panel instructions” on page 37](#), you must code the top and bottom instruction tags outside of the AREA definition. In this example, we coded only a selection field within the AREA definition.

The AREA tag has an optional MARGINW attribute that you can use to specify the width of the panel body margins. This is useful for arranging fields on a panel.

The MARGINW attribute has a default value of 1. You can specify a different value to increase the size of the panel body margins. For example, we could specify a margin width for the AREA in the preceding markup example.

```
<!doctype dm system>
<panel name=mainpan7>Item Selection
  <topinst>Select one of the following items and press Enter.
    <area marginw=10>
      <selfld name=itemtyp selwidth=58>
        <choice>Automotive
        <choice>Hardware
        <choice>Health and beauty
        <choice>Lawn and garden
        <choice>Sporting goods
      </selfld>
    </area>
  <botinst>To exit the application, press F3.
</panel>
```

We specified a margin width of 10. Here is how the panel looks now:

```

                                Item Selection

Select one of the following items and press Enter.

--  1.  Automotive
    2.  Hardware
    3.  Health and beauty
    4.  Lawn and garden
    5.  Sporting goods

To exit the application, press F3.
```

Figure 13. AREA MARGINW=10

Scrollable areas

You specify a scrollable area with the Dialog Tag Language by coding the AREA tag and specifying the DEPTH, EXTEND, and DIV attributes for the area. When the DEPTH attribute is present, the conversion utility generates the)AREA section in the panel definition, along with the corresponding)ATTR and)BODY entries for the scrollable area.

Help panels generated by the Conversion Utility that contain all of the help panel text within an AREA tag (with DEPTH specified) are not split into separate panels. The conversion utility places the text in an)AREA section, which allows you to define panels up to the display size limit of ISPF.

If you specified DEPTH to signal the creation of a panel with a scrollable area, you can also specify the EXTEND and DIV attributes.

You can specify EXTEND=ON to allow the panel to expand to the logical window size. If you intend to have the panel in a pop-up window, you should not code the EXTEND attribute. Panels that specify EXTEND=ON cannot be preprocessed.

You use the DIV attribute to control the creation of a divider line before and after the scrollable area.

If you specify DIV=BLANK, a blank divider line is added before and after the area. If you specify DIV=SOLID, a visible divider is created. The visible divider formats with an attribute byte on each end of the line of dashes, which causes the line to appear with a 1-character "space" on both ends. Omitting the DIV attribute or specifying DIV=NONE causes the area to be created without divider lines.

The conversion utility uses the DEPTH attribute value to reserve a fixed amount of space in the panel body. This space, together with the divider lines, if specified, is considered as part of the body within the depth limit specified (or defaulted) on the PANEL tag. When EXTEND=OFF, the minimum depth for a scrollable area is two lines, one for the scrolling indicator line and at least one line of displayable text.

Here is markup that shows how to code a scrollable panel. [Figure 14 on page 41](#), [Figure 15 on page 41](#), and [Figure 16 on page 42](#) show the formatted result.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar2 system>
  <!entity sampabc system>)>
&sampvar2;

<PANEL NAME=scrarea3 KEYLIST=keylxml>File-A-Case
<AB>
&sampabc;
</AB>
<TOPINST COMPACT>
  Type in client's name and case number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
<DTAFLD DATAVAR=caseno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=25>Case No
  <DTAFLDD>(A 7-digit number)
<DTAFLD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Name
  <DTAFLDD>(Last, First, M.I.)
<DTAFLD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
<DIVIDER>
<SELFLD NAME=casesel PMTWIDTH=30 PMTLOC=before SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=case MATCH=civ>Civil
  <CHOICE CHECKVAR=case MATCH=estate>Real Estate
  <CHOICE CHECKVAR=case MATCH=environ>Environmental
</SELFLD>
</AREA>
<AREA DEPTH=6>
<SELFLD TYPE=multi PMTWIDTH=35 SELWIDTH=50>Check type of offense committed
  <CHOICE NAME=patin HELP=patin CHECKVAR=val>Patent Infringement
  <CHOICE NAME=defa HELP=defame CHECKVAR=def>Defamation
  <CHOICE NAME=cont HELP=cont CHECKVAR=con>Breach of Valid Contract
  <CHOICE NAME=priv HELP=priv CHECKVAR=pri>Invasion of Privacy
  <CHOICE NAME=incr HELP=incr CHECKVAR=icr>Interference with Contractual
  Relations
  <CHOICE NAME=disp HELP=disp CHECKVAR=dis>Improper Disposal of Medical
  By-Products
  <CHOICE NAME=fraud HELP=fraud CHECKVAR=fra>Fraud
</SELFLD>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```



```

File  Search  Help
-----
                        File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case No  . . . _____ (A 7-digit number)
Name . . . . _____ (Last, First, M.I.)
Address . . _____

Choose one of the following      --  1.  Civil
                                      2.  Real Estate
                                      3.  Environmental
                                      More:      +

Check type of offense committed
- Patent Infringement
- Defamation
- Breach of Valid Contract
Enter a command ==>
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel

```

Figure 14. Scrollable panel area

After scrolling, the panel looks like this:

```

File  Search  Help
-----
                        File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case No  . . . _____ (A 7-digit number)
Name . . . . _____ (Last, First, M.I.)
Address . . _____

Choose one of the following      --  1.  Civil
                                      2.  Real Estate
                                      3.  Environmental
                                      More:      - +

- Breach of Valid Contract
- Invasion of Privacy
- Interference with Contractual Relations
- Improper Disposal of Medical By-products
Enter a command ==>
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel

```

Figure 15. Application panel area

After scrolling, the last choice in the list is visible.

```

File  Search  Help
-----
                                File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case No   . . . _____ (A 7-digit number)
Name      . . . _____ (Last, First, M.I.)
Address   . . . _____

Choose one of the following      --  1.  Civil
                                      2.  Real Estate
                                      3.  Environmental
                                      More:  -

-  Invasion of Privacy
-  Interference with Contractual Relations
-  Improper Disposal of Medical By-products
-  Fraud
Enter a command ==>
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel

```

Figure 16. Scrollable panel area

Multiple AREA tags

The default AREA tag formatting arranges areas vertically within the panel body.

The WIDTH and DIR attributes of the AREA tag allow areas to be formatted horizontally.

Here is markup that shows horizontal areas. [Figure 17 on page 43](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM(
  <!entity sampvar2 system>
  <!entity sampabc system>)>
&sampvar2;

<PANEL NAME=scrarea4 KEYLIST=keylxmlp>File-A-Case
<AB>
&sampabc;
</AB>
<CMDAREA>Enter a command
<TOPINST COMPACT>
  Type in client's name and case number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA width=50 dir=horiz>
<DTAFLD DATAVAR=caseno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=21>Case No
  <DTAFLLDD>(A 7-digit number)
<DTAFLD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=8>Name
  <DTAFLLDD>(Last, First, M.I.)
<DTAFLD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
<DIVIDER>
<SELFLD NAME=casesel PMTWIDTH=11 PMTLOC=before SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=case MATCH=civ>Civil
  <CHOICE CHECKVAR=case MATCH=estate>Real Estate
  <CHOICE CHECKVAR=case MATCH=environ>Environmental
</SELFLD>
</AREA>
<AREA width=26 dir=horiz>
<SELFLD TYPE=multi PMTWIDTH=24 SELWIDTH=26 depth=10>
  Check type of offense
  <CHOICE NAME=patin HELP=patin CHECKVAR=val>Patent Infringement
  <CHOICE NAME=defa HELP=defame CHECKVAR=def>Defamation
  <CHOICE NAME=cont HELP=cont CHECKVAR=con>Breach of Valid Contract
  <CHOICE NAME=priv HELP=priv CHECKVAR=pri>Invasion of Privacy
  <CHOICE NAME=incr HELP=incr CHECKVAR=icr>Interference with Contractual
  Relations
  <CHOICE NAME=disp HELP=disp CHECKVAR=dis>Improper Disposal of Medical
  By-Products
  <CHOICE NAME=fraud HELP=fraud CHECKVAR=fra>Fraud

```

Here are the contents of the scrollable area:

- Patent Infringement
- Defamation
- Breach of Valid Contract
- Invasion of Privacy
- Interference with Contractual Relations
- Improper Disposal of Medical By-Products
- Fraud

Figure 17. Multiple horizontal areas

You specify a dynamic area in the)BODY section by coding the DA and ATTR tags. The DA tag is used to define the dynamic area in the panel)BODY section. The ATTR tag is used to specify the)ATTR section entries for DATAIN, DATAOUT, and CHAR attribute types used within the dynamic area. A dynamic area allows you to specify an area of the panel to format with your application. See the [*z/OS ISPF Dialog Developer's Guide and Reference*](#) for more information.

You specify a graphic area in the panel)BODY section by coding the GA tag. A Graphic area allows you to define a specific portion of the screen for a GDDM display. See the [*z/OS ISPF Dialog Developer's Guide and Reference*](#) for more information.

The DIVIDER tag

You can separate the elements on a panel or the regions you define for a panel with the DIVIDER tag. A DIVIDER definition produces either a blank or visible divider line, depending on the value you assign to the TYPE attribute of the DIVIDER tag. The visible divider line can be a dashed line or a solid line, or it can contain text.

The default value, NONE, produces a blank divider line. The values DASH, SOLID, and TEXT produce a visible divider line.

For horizontally formatted dividers,

- When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line.
- When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

Both SOLID and DASH use the "|" character (obtained from the ISPF literals table) for vertically formatted dividers. The value TEXT (used in combination with the FORMAT attribute) is valid only for dividers within vertical regions and specifies that blank padding is used for the supplied text.

The GAP attribute specifies whether the divider line completely crosses the panel area or region that contains the divider, or if a 1-character gap is to remain at either end of a horizontally formatted divider. The valid values for the GAP attribute are YES (the default), and NO.

The value you assign to GUTTER specifies the size (in characters) of the total width of the divider. For vertical formatting the default is 1, because ISPF allots 1 line of screen space for the divider. For horizontal formatting the default GUTTER size is 3, because an attribute byte is placed both before and after the divider character. Any value more than the default is split to either side of the divider. If the GUTTER value is an even number, the conversion utility increases the number by 1 so that the divider is centered within the defined width. The GUTTER attribute is useful for creating blank space on a panel.

The NOENDATTR attribute is valid only when formatting dividers within horizontal regions. When NOENDATTR is specified, the ending attribute is not added to the divider. With NOENDATTR and a GUTTER size of 1, a divider of one blank character can be created. With a GUTTER size of 2, TYPE=SOLID can be used to produce a visible divider.

The FORMAT attribute is valid only when formatting dividers within vertical regions. The FORMAT attribute must be specified to have ISPD TLC process the text provided with the DIVIDER tag. FORMAT specifies the text placement within the divider line as START, CENTER, or END.

Here is an example where there are two DIVIDER tags defined. The first DIVIDER does not specify a TYPE attribute, and produces a blank horizontal line. The second DIVIDER specifies TYPE=SOLID, and produces a visible divider.

```
<!doctype dm system>
<panel name=fields1>Selections
  <area>
    <dtacol selwidth=24 pmtwidth=15>
      <selfld name=item>Pick an item:
        <choice>Widget
        <choice>Doohickey
        <choice>Gizmo
      </selfld>
      <divider>
      <selfld name=color>Pick a color:
        <choice>Red
        <choice>Green
      </selfld>
      <divider type=solid gap=no>
      <selfld name=size>Pick a size:
        <choice>Minuscule
        <choice>Behemoth
      </selfld>
    </dtacol>
  </area>
  <botinst>To exit the application, press F3.
</panel>
```

Figure 18 on page 45 shows the result:

```

                                Selections

Pick an item:
--  1.  Widget
   2.  Doohickey
   3.  Gizmo

Pick a color:
--  1.  Red
   2.  Green
-----
Pick a size:
--  1.  Minuscule
   2.  Behemoth

To exit the application, press F3.

```

Figure 18. Area dividers

The dashed line in the second divider in the preceding example extends across the entire AREA definition to both margins because we specified GAP=NO in the DIVIDER definition.

The REGION tag

You can further define the areas of your panel, and how you want the information in the areas arranged, with the REGION tag. Using one or more regions within a PANEL definition provides an easy way of arranging the elements on a panel. Like the PANEL and AREA tags, the REGION end tag is required.

The DIR (direction) attribute of the REGION tag specifies how the elements within a region are arranged, either horizontally or vertically. The default value is VERT, which arranges the elements within the region vertically. This means that if you do not specify a horizontal region (DIR=HORIZ), or if you do not define a region at all, the panel elements are arranged vertically by default.

In this example, the selection fields are arranged vertically, because no DIR value is defined for the REGION tag.

```

<!doctype dm system>
<panel name=fields2>Selections
  <area>
    <region>
      <dtacol selwidth=24 pmtwidth=15>
        <selfld name=item>Pick an item:
          <choice>Widget
          <choice>Doohickey
          <choice>Gizmo
        </selfld>
        <divider>
        <selfld name=color>Pick a color:
          <choice>Red
          <choice>Green
        </selfld>
        <divider type=solid gap=no>
        <selfld name=size>Pick a size:
          <choice>Minuscule
          <choice>Behemoth
        </selfld>
      </dtacol>
    </region>
  </area>
  <botinst>To exit the application, press F3.
</panel>

```

```

                                Selections

Pick an item:
--  1.  Widget
   2.  Doohickey
   3.  Gizmo

Pick a color:
--  1.  Red
   2.  Green
-----

Pick a size:
--  1.  Minuscule
   2.  Behemoth

To exit the application, press F3.
```

Figure 19. Vertical region

We'll specify the `HORIZ` value for the region to change the layout of the selection fields to horizontal. Figure 20 on page 47 shows the result.

```

<!doctype dm system>
<panel name=fields3>Selections
  <area>
    <region dir=horiz>
      <dtacol selwidth=20 pmtwidth=15>
        <selfld name=item>Pick an item:
          <choice>Widget
          <choice>Doohickey
          <choice>Gizmo
        </selfld>
        <divider type=solid gutter=5>
        <selfld name=color>Pick a color:
          <choice>Red
          <choice>Green
        </selfld>
        <divider type=solid gutter=5>
        <selfld name=size>Pick a size:
          <choice>Minuscule
          <choice>Behemoth
        </selfld>
      </dtacol>
    </region>
  </area>
  <botinst>To exit the application, press F3.
</panel>
```

```

                                Selections

Pick an item:      |      Pick a color:      |      Pick a size:
--  1.  Widget      |      --  1.  Red      |      --  1.  Minuscule
    2.  Doohickey    |      2.  Green      |      2.  Behemoth
    3.  Gizmo        |      |

```

To exit the application, press F3.

Figure 20. Horizontal region

In the markup for this example, we also changed the format of the DIVIDER tags to provide additional space and a visible line between the selection fields. We did this by specifying TYPE=SOLID and GUTTER=5 on each of the DIVIDER tags. Also the divider lines are now vertical. That's because of the way DTL handles dividers within regions. DTL adheres to these formatting rules for DIVIDER tags within regions:

- A DIVIDER tag coded within a vertical region formats horizontally.
- A DIVIDER tag coded within a horizontal region formats vertically.

Here is markup to show how REGION and DIVIDER tags format under different circumstances. This example shows both horizontal and vertical regions, as well as solid and blank dividers.

```

<!doctype dm system>
<panel name=mainpan8>Application
  <topinst>Complete the information below and press Enter.
  <area>
    <dtafld datavar=name entwidth=25 pmtwidth=9>Name
    <dtafld datavar=addr entwidth=25 pmtwidth=9>Address
    <region dir=horiz>
      <dtafld datavar=city pmtwidth=9 entwidth=25>City
      <dtafld datavar=stat pmtwidth=5 entwidth=2>State
      <dtafld datavar=zip pmtwidth=8 entwidth=5>Zip code
    </region>
    <divider type=solid gutter=3>
    <region dir=horiz>
      <selfld name=grade pmtwidth=32 selwidth=33>Highest education level
      <choice>Some high school
      <choice>High school graduate
      <choice>Some college
      <choice>College graduate
      <choice>Some post-graduate work
      <choice>Post graduate degree
    </selfld>
    <divider gutter=5>
    <region>
      <info width=30>
        <p compact>Complete if applicable:
      </info>
      <dtafld datavar=grad pmtwidth=10 entwidth=11>Date of graduation
      <dtafld datavar=field pmtwidth=10 entwidth=11>Field of study
    </region>
  </region>
</area>
</panel>

```

Figure 21 on page 48 shows how the preceding markup formats.

Application

Complete the information below and press Enter.

Name . . _____
Address _____
City . . _____ State __ Zip code _____

Highest education level	Complete if applicable:
-- 1. Some high school	Date of graduation _____
2. High school graduate	Field of study . . _____
3. Some college	
4. College graduate	
5. Some post-graduate work	
6. Post graduate degree	

Figure 21. Horizontal region

This is an example of nesting regions. The data fields for entering the graduation date and field of study are arranged in a vertical region that is nested within a horizontal region.

The ALIGN, DEPTH, EXTEND, INDENT, LOCATION, WIDTH and GRPBOX attributes allow additional formatting control. The DEPTH and EXTEND attributes are used with scrollable regions. ALIGN, INDENT, LOCATION, and WIDTH affect the placement of fields within the region and the placement of the region within the panel. GRPBOX allows for the display of a title for the region. The title is specified following the REGION tag ending delimiter.

Defining a command area

Many applications are dependent on a command area in their panels. You define a command area and specify the prompt text of the command area with the CMDAREA tag. The conversion utility supplies the prompt symbol (===>) and provides the entry field in the command area for user input.

The conversion utility always formats the command area at the top of the panel. An ISPF runtime option determines the actual display location of the command line.

The command area contains an entry field and command prompt text, and is normally displayed at the bottom of an application panel. Users can enter commands in the command entry field. All commands entered into the command entry field are validated against the commands you define within the application command table and the ISPF-provided commands. For more information about defining the application command table, see [Chapter 8, “The application command table,” on page 143](#).

```
<!doctype dm system (<!entity actnbar system>)>
<panel name=cmdxmp1>Application Name
  &actnbar;
  <topinst>Sample command area panel
    <area>
    </area>
    <CMDAREA>
  </panel>
```

Here is how the command area displays on the panel:



Figure 22. Command area

In [Figure 22 on page 49](#) we did not specify the text of the command prompt, so the conversion utility automatically added the text "Command" (or its translated equivalent), which is the default text. If we wanted to specify something other than this text, we could have coded it as tag text, as in this example:

```
<!doctype dm system (<!entity actnbar system>)>
<panel name=cmdxmp2>Application Name
  &actnbar;
  <topinst>Sample command area panel
  <area>
  </area>
  <cmdarea>Enter a command
</panel>
```

You can code up to 59 bytes of prompt text on a standard 76-byte width panel when overriding the default text. Here is how the command prompt looks now:



Figure 23. Command area

Data entered on the command line can be forced to uppercase either by specifying CAPS = ON or by including a VARCLASS tag to define the command area and an XLATL tag to specify translation to uppercase. (The type attribute defines the space available on a standard 76 character width panel using the default command prompt.)

```
<varclass name=vccmd type='char 59'>
  <xlatl format=upper>
  </xlatl>
</varclass>

<varlist>
  <varclass name=zcmd varclass=vccmd>
</varclass>
```

The AUTOTAB, CAPS, CMDLEN, CMDLOC, ENTWIDTH, NAME, NOINIT, NOJUMP, OUTLINE, PAD, PADC, PLACE, PMTTEXT, PSBUTTON, PSVAL, PSVAR, SRCAPS, SCROLLTAB, SCROLLVAR, and SCRHELP are attributes that control formatting, initialization, and presentation of the command area.

Defining panel defaults

DTL provides a tag that makes it easier to define attributes and values that are common for multiple application panels: the PANDEF (panel default) tag. This tag must be coded in the source file before any panels it is providing defaults for.

The default PANEL values you can define with the PANDEF tag are:

- The panel dimensions (DEPTH and WIDTH)
- The help panel
- The key mapping list
- The KEYLTYPE value
- The CCSID number
- The WINDOW value
- The WINTITLE value
- The APPTITLE value
- The PAD value

- The PADC value
- The OUTLINE value
- The EXPAND value.
- The MERGESAREA value
- APPLID value
- ENTKEYTEXT value
- IMAPNAME value
- IMAPROW value
- IMAPCOL value
- TMARGIN value
- BMARGIN value

You can use a PANDEF tag to define all of these values, or some of them. You can also override a specific panel default value for a referencing panel by specifying the attribute on the PANEL tag.

For instance, if you create a series of panels that all have the same dimensions and that all refer to the same help panel and key mapping list, you can define these values in a PANDEF definition, and refer to that definition in each of the application panels that use those values. The DTL compiler does the rest of the work for you, as long as the default definition is available as part of the same source file as the panels that refer to it.

For example, if you are creating a series of panels that all share the same values, you could create a PANDEF definition like this:

```
<!doctype dm system>

<pandef id=printdef help=prnthlp depth=20 width=70 keylist=printkey>
```

And refer to the panel default like this on all of the panels in that series:

```
<panel name=panel01 pandef=printdef>A Panel
:
</panel>

<panel name=panel02 pandef=printdef>Another Panel
:
</panel>
```

When you compile this source file, the PANDEF definition provides those values for the panels that refer to the panel default.

You can also use the PANDEF tag to define common values for individual PANEL attributes. For instance, if the only commonality between application panels is the dimensions, you can use a panel default to define the dimensions and refer only to those values in the application panel definitions:

```
<!doctype dm system>

<pandef id=size depth=20 width=70>

<panel name=panel01 help=help01
keylist=keylsta pandef=size>A Panel
:
</panel>

<panel name=panel02 help=help02
keylist=keylstb pandef=size>Another Panel
:
</panel>
```

To change the dimensions of the application panels that refer to a panel default, you only have to make the change in one place: the PANDEF definition.

To override a PANDEF value, you must specify that value in the PANEL definition. Here is an example of a panel default that defines both dimensions and a help panel. While all three PANEL definitions refer to the

Defining panel defaults

panel default, the panel with the NAME value *panel03* specifies a different help panel, and thus overrides the PANDEF HELP value.

```
<!doctype dm system>
<pandef id=pandef01 depth=20 width=70 help=help01>
<panel name=panel01 pandef=pandef01>
:
</panel>
<panel name=panel02 pandef=pandef01>
:
</panel>
<panel name=panel03 pandef=pandef01 help=help02>
:
</panel>
```

You can also define multiple panel defaults within a single source file, like this:

```
<!doctype dm system>
<pandef id=pandef01 depth=20 width=70 help=help01>
<pandef id=pandef02 depth=10 width=50 help=help02 keylist=klist01>
<panel name=panel01 pandef=pandef01>
:
</panel>
<panel name=panel02 pandef=pandef02>
:
</panel>
<panel name=panel03 pandef=pandef01 help=help02>
:
</panel>
```

Chapter 4. Variables and variable classes

Much of the information displayed within dialog elements is derived directly from the tags used to define it. Other information is obtained dynamically when the application is running, such as:

- Data that the user supplies
- Data that the application supplies
- Data that ISPF supplies.

In all of these cases, the data is derived from values specified in variables.

DTL provides you with tags to *declare* variables and to define the characteristics of these variables using variable classes. Variables and variable classes are considered global because they can be referred to by more than one element within the same source file. All variables referred to by dialog elements should be declared. Variable names and variable classes should be used consistently throughout dialog elements that are used in the same application.

Variables declared using DTL are accessible to your application through the dialog variable pools and variable services provided by ISPF. Within ISPF display processing, all variable values are in character format. ISPF transforms display variables between their dialog program format and internal display processing character format when retrieving and storing variable values.

Note: Although the conversion utility processes all of the variable information provided in your DTL source file and issues suppressible warning messages for missing VARDCL tags during the processing of several other tags, such as DTAFLD and LSTCOL, ISPF does not require any of the tags described in the chapter to generate a valid ISPF panel.

The conversion utility supports the SOURCE tag as an alternative means of placing variable processing and validation statements directly into the ISPF panel.

Declaring variables

You declare variables for dialog elements by coding variable declarations in a variable list and specifying the variable class associated with each declared variable.

The variable list (VARLIST) tag and its required end tag define the variable list. You code the variable list after any variable classes and before any other tags.

To declare variables, use VARDCL (variable declaration) tags within the VARLIST definition. The VARDCL tag has two required attributes, NAME and VARCLASS.

NAME

NAME specifies the variable used within the DTL source file.

For example, a data field definition includes a variable name in the DATAVAR attribute to specify the variable that receives data when the user enters data in the field.

VARCLASS

VARCLASS specifies the variable class associated with the variable declaration. Variable classes define the format and length of variable data plus translations and checks to perform on the data.

Here is an example where the variable list contains two variable declarations, referred to by the data fields in the application panel:

```
<!doctype dm system>

<varclass name=authorc type='char 40'>
<varclass name=catnumc type='char 10'>

<varlist>
  <vardcl name=author varclass=authorc>
  <vardcl name=catnum varclass=catnumc>
</varlist>
```

```
<panel name=books1>Book Title Search
  <area>
    <dtacol pmtwidth=20>
      <dtafld entwidth=40 datavar=author>Author
      <dtafld entwidth=10 datavar=catnum>Catalog number
    </dtacol>
  </area>
</panel>
```

Note: The ISPF Dialog Tag Language conversion utility does not require that you code the VARCLASS, VARDCL, or VARLIST tags for a successful generation of a panel, command table, or message member that includes variables. If the conversion utility finds a variable that does not have an associated VARDCL definition, it issues a suppressible warning message.

The use of the VARCLASS, VARDCL, and VARLIST tags is required if you want to use the facilities provided by the CHECKL and XLATL tags.

Defining variable classes

To complete the preceding example, we must code the variable classes that are referred to with the VARDCL VARCLASS attributes. The variable class information must be defined if the conversion utility is to generate)INIT and)PROC section statements for variable translations and validations. ([“Variable validation” on page 56](#) tells you how to define translations and validity checks.)

The VARCLASS (variable class) tag defines a variable class. You include variable classes in the same source file as the dialog elements and variable list that refer to them. Additionally, you must code variable classes in the source file before the variable list and dialog element definitions. You do this by coding variable classes following the DOCTYPE statement or by coding this information in an external file and embedding the file following the DOCTYPE statement.

There are two required attributes associated with the VARCLASS tag: NAME and TYPE.

NAME

NAME is used to identify and refer to the variable class.

TYPE

TYPE defines the format and entry-field width for variable data.

In addition to these required attributes, the VARCLASS tag has an optional MSG attribute. This attribute specifies the message to be displayed if the variable fails any defined validity checks and no message is defined for the XLATL or CHECKL tags. [Chapter 7, “Messages,” on page 137](#) tells you how to define messages.

Variable class types

DTL supports character variables and numeric variables. In addition, the conversion utility uses the length specified in the TYPE attribute value of the VARCLASS tag to determine the entry width of fields associated with the VARCLASS tag if this width is not defined with the tag used to create the field. For more information about defining field entry widths, see [Chapter 5, “Application panel fields,” on page 71](#).

Character variables

You can specify whether single-byte characters, double-byte characters, or mixed double-byte and single-byte characters are permitted, as well as the maximum number of bytes the variable can accept. Here is a description of each type:

Type

Description

'CHAR maximum-length'

Specifies a single-byte character string.

'DBCS maximum-length'

Specifies a double-byte character string. The maximum length must be an even number.

'MIXED maximum-length'

Specifies a character string containing single-byte characters, double-byte characters, or both.

Note: This type is treated as CHAR if the system does not support double-byte characters.

'ANY maximum-length'

Specifies a character string containing single-byte characters, double-byte characters, or both. It is processed by the conversion utility as MIXED.

'EBCDIC maximum-length'

Specifies a single-byte character string.

'%varname maximum-length'

Specifies that a variable name is used for TYPE in the Panel definition. The application must ensure a valid type is set before the panel is displayed.

'VMASK maximum-length'

A VEDIT statement is added to the generated panel. The 'maximum-length' is the default length for associated variables. The application must use the VMASK service with a user-specified mask value.

'ITIME'

A VEDIT statement is added to the generated panel for associated variables. The default length for the variables is 5. The application must use the VMASK service with a mask of ITIME.

'STDTIME'

A VEDIT statement is added to the generated panel for associated variables. The default length for the variables is 8. The application must use the VMASK service with a mask of STDTIME.

'IDATE'

A VEDIT statement is added to the generated panel for associated variables. The default length for the variables is 8. The application must use the VMASK service with a mask of IDATE.

'STDDATE'

A VEDIT statement is added to the generated panel for associated variables. The default length for the variables is 10. The application must use the VMASK service with a mask of STDDATE.

'JDATE'

A VEDIT statement is added to the generated panel for associated variables. The default length for the variables is 6. The application must use the VMASK service with a mask of JDATE.

'JSTD'

A VEDIT statement is added to the generated panel for associated variables. The default length for the variables is 8. The application must use the VMASK service with a mask of JSTD.

We add the variable classes for authorc and catnumc to the markup example found in [“Declaring variables”](#) on page 53. We assume an author's last name has a maximum of 40 characters, and a catalog number is 10 characters.

```
<!doctype dm system>

<varclass name=authorc type='char 40'>
<varclass name=catnumc type='char 10'>

<varlist>
  <vardcl name=author varclass=authorc>
  <vardcl name=catnum varclass=catnumc>
</varlist>

<panel name=books2>Book Title Search
  <area>
    <dtacol pmtwidth=20>
      <dtafld entwidth=40 datavar=author>Author
      <dtafld entwidth=10 datavar=catnum>Catalog number
    </dtacol>
  </area>
</panel>
```

Numeric variables

You can use the NUMERIC type to ensure that a valid number is entered in the associated field. You can specify the total number of digits (up to 16) allowed in the number and the number of fractional digits allowed. The conversion utility generates a VER(variable ENUM) statement for input validation.

Here is an example of a variable class (*pricevar*) where the data entered in the associated field has a maximum number of five digits, two of which are fractional:

```
<varclass name=pricevar
  type='numeric 5 2'>
```

If you do not specify an entry width with the tag that defines the associated field, the conversion utility calculates an entry width for the field based on the NUMERIC value and allow for a sign, thousands separators, and a decimal point.

Variable validation

DTL allows you to define translate lists and validity checks as part of the variable class definition by using tags nested within the VARCLASS tag. These built-in translations and checks are especially useful because ISPF automatically performs them on variable values, so the dialog application does not need to.

Note: Translations and checks are performed only on variable values that are intended for display. For instance, before displaying the data from a variable specified on the DATAVAR attribute of a DTAFLD tag, ISPF performs any specified translations on the variable retrieved from the application to construct the correct display value. However, ISPF does not perform translations on a variable specified as the CHECKVAR attribute of a CHOICE tag.

Translate lists

Translate lists provide a means of translating a displayed variable value into a different dialog variable pool value, and vice versa. Translation can occur on input (when the user enters a value), on output (the value stored in the variable pool is translated before the user sees it), or both. This is based on the USAGE value of the tag that refers to a variable using a variable class with translate lists.

To associate a translate list with a variable class, code the XLATL (translate list) tag and its required end tag within the VARCLASS definition.

The type of translation is determined by the value assigned to the FORMAT attribute of the XLATL tag. The two types of translations supported are:

- Uppercase translation
- Item translation

There is an optional MSG attribute on the XLATL tag that allows you to specify your own message to display when input translation specified by the XLATL does not result in a match. For information about defining your own messages, see [Chapter 7, “Messages,” on page 137](#).

Upper

Allows you to translate a value to uppercase. To specify this translation, code FORMAT=UPPER on the XLATL tag. This translation is always successful.

We'll add a translate list to the authorc variable class in the example under [“Numeric variables” on page 56](#). The translate list converts the author's name to uppercase.

```
<varclass name=authorc type='CHAR 40' msg=liba001>
  <xlatl format=upper>
</xlatl>
```

This figure shows the results on input and output translations for the previous example:

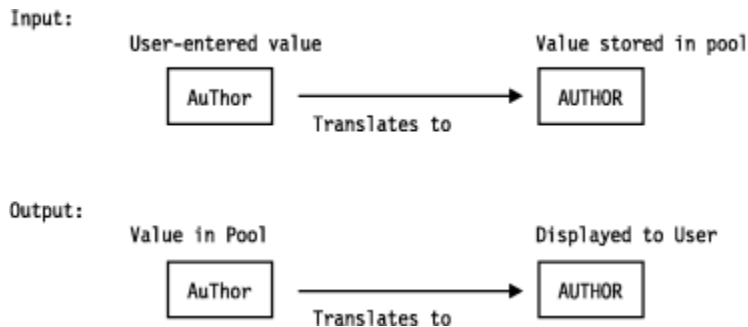


Figure 24. Variable translation results

Item translation

Allows you to translate an internal variable value to a displayed value (or vice-versa) on an item-for-item basis. To specify this translation, either code `FORMAT=NONE` on the `XLATL` tag or omit the `FORMAT` attribute because this is the default. You define the list of possible internal values and the corresponding display values they should be translated to, or from, using the `XLATI` (translate item) tags nested within the `XLATL` tag.

To specify an internal value (the value in the variable pool) for a translate item, use the `VALUE` attribute on the `XLATI` tag. The `XLATI` tag text specifies what the user sees (for output) and enters (for input).

The display value is the `XLATI` tag text. If a display value of all blanks or a display value in which leading, trailing, or embedded blanks are preserved is desired, use the `LIT` tag and its required end tag to indicate that blanks are significant.

An explicit match is achieved during translation processing as follows:

- On input, an explicit match occurs when the value the user enters matches one of the specified display values in the translate list. An explicit match also occurs when a display value is omitted (indicating any value is acceptable) and the corresponding internal value is specified.
- On output, an explicit match occurs when the value from the variable pool matches one of the specified internal values in the translate list. An explicit match also occurs when an internal value is omitted (indicating any value is acceptable) and the corresponding display value is specified.

Omitting both the internal value and the display value does not produce an explicit match. This case is discussed further on in this topic.

Translate list processing is case-sensitive. To ensure that a match results when the user enters the correct display value but in a different or mixed case, code an uppercase conversion translate list before the value translate list.

Here is an example where the variable class `dayc` uses an internal value for days of the week that is different from the display value. The comparisons are on uppercase values, because `FORMAT=UPPER` is provided before the item translation list.

```

<!doctype dm system>

<varclass name=dayc type='CHAR 9'>
  <xlatl format=upper>
  </xlatl>
  <xlatl msg=liba004>
    <xlati value=1>SUNDAY
    <xlati value=2>MONDAY
    <xlati value=3>TUESDAY
    <xlati value=4>WEDNESDAY
    <xlati value=5>THURSDAY
    <xlati value=6>FRIDAY
    <xlati value=7>SATURDAY
  </xlatl>

```

This figure shows how variable values of variable class `dayc` are translated on input and output.

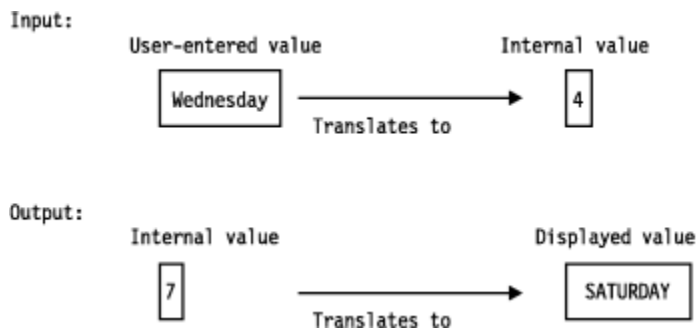


Figure 25. Variable translation

The previous example shows one translate list with a finite number of translation items. This example assumes that the only possible internal values are 1-7 and the only possible display values are the days of the week. For input fields, a match must be found in this list, or the translation fails and message liba004 is displayed to the user.

Here is an example which allows allow a nonmatching value to be passed on for further processing (either to another translate list or to the validity checks that follow) by coding an XLATI tag without an internal value or a display value, to indicate that any value is acceptable:

```

<!doctype dm system>

<varclass name=dayc type='CHAR 9'>
  <xlatl format=upper>
  </xlatl>
  <xlatl>
    <xlati value=1>SUNDAY
    <xlati value=2>MONDAY
    <xlati value=3>TUESDAY
    <xlati value=4>WEDNESDAY
    <xlati value=5>THURSDAY
    <xlati value=6>FRIDAY
    <xlati value=7>SATURDAY
    <xlati>
  </xlatl>
  
```

Because multiple translate lists are permitted, we can expand this example to accept either the days of the week spelled out or their accepted abbreviations. Because the last XLATI tag in the first translate list has no internal or displayed value, the input value are passed on for further translate list or validity checking.

```

<!doctype dm system>

<varclass name=dayc type='CHAR 9'>
  <xlatl format=upper>
  </xlatl>
  <xlatl>
    <xlati value=1>SUNDAY
    <xlati value=2>MONDAY
    <xlati value=3>TUESDAY
    <xlati value=4>WEDNESDAY
    <xlati value=5>THURSDAY
    <xlati value=6>FRIDAY
    <xlati value=7>SATURDAY
    <xlati>
  </xlatl>
  <xlatl>
    <xlati value=1>SUN
    <xlati value=2>MON
    <xlati value=3>TUES
    <xlati value=4>WED
    <xlati value=5>THUR
    <xlati value=6>FRI
    <xlati value=7>SAT
  </xlatl>
  
```

It is possible to omit only the internal value to indicate that any internal value is acceptable. This affects input and output translate processing differently. When translating on input, the value is not translated

before being stored in the variable pool. When translating on output, any value not already matched is translated to the displayed value.

In the following example, the *branchc* variable class illustrates translate processing when only the internal value is omitted.

```
<!doctype dm system>

<varclass name=branchc type='CHAR 3'>
  <xlat1 format=upper>
  </xlat1>
  <xlat1>
    <xlati value=1>RAL
    <xlati>CRY
  </xlat1>
```

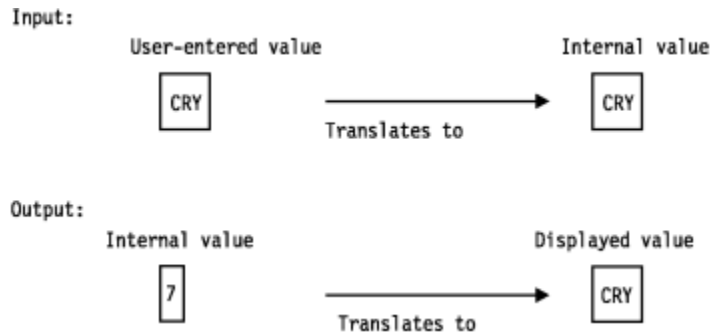


Figure 26. Variable translation

It is also possible to omit only the display value to indicate that any display value is acceptable. This affects input and output translate processing differently. When translating on input, any value not already matched is translated to the internal value. When translating on output, the internal value is not translated before it is displayed.

Here is a similar example, but with the *branchc* variable class changed, to show translate processing when only the display value is omitted:

```
<!doctype dm system>

<varclass name=branchc type='CHAR 3'>
  <xlat1 format=upper>
  </xlat1>
  <xlat1>
    <xlati value=1>RAL
    <xlati value=2>
  </xlat1>
```

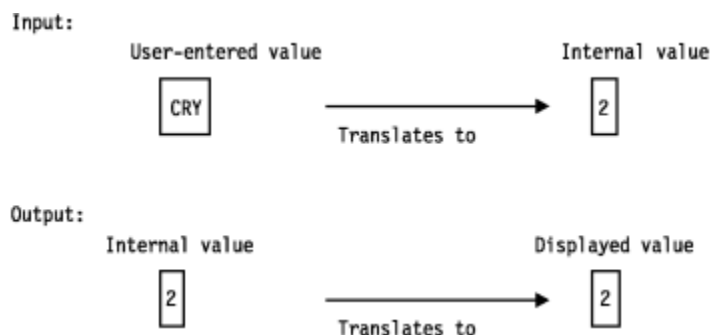


Figure 27. Variable translation

It is possible to specify that less than the full input value be entered by the use of the TRUNC attribute. Output translation is not affected.

We'll change the *branchc* variable class again to illustrate:

```
<!doctype dm system>

<varclass name=branchc type='CHAR 3'>
  <xlat1 format=upper>
  </xlat1>
  <xlat1 format=none trunc=1>
    <xlati value=1>RAL
    <XLATI VALUE=2>
  </xlat1>
```

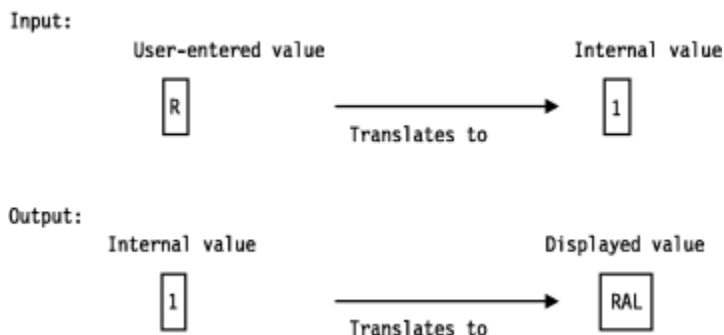


Figure 28. Variable translation

Validity checks

You use validity checks to automatically verify data input by the user. Code validity checks after any translate lists.

To associate a validity check with a variable class, code the CHECKL (check list) tag and its required end tag either following the last translate list, or if no translate list exists, following the VARCLASS start tag. The individual check item that defines the test to perform is coded using the CHECKI (check item) tag nested within the check list. You can code one CHECKI tag in a CHECKL definition. However, you can code multiple CHECKL tags within a variable class definition.

There is an optional MSG attribute on the CHECKL tag that allows you to specify your own message to display when the entered value fails the test. If you do not specify a message, ISPF Dialog Manager supplies a default message for you. For more information about defining your own messages, see [Chapter 7, “Messages,” on page 137](#).

A value entered by the user must pass the check item test for the check list to be considered successful. Furthermore, because there can be multiple check lists defined, all check lists must be successful for the validation to be successful.

The TYPE attribute of the CHECKI tag allows you to specify the various validity tests of the input. You can define these types of validity check:

- [“RANGE” on page 61](#)
- [“ALPHA” on page 61](#)
- [“VALUES” on page 62](#)
- [“VALUESX” on page 62](#)
- [“CHARS” on page 62](#) (limited to characters for HEX, BIT, and NUM tests)
- [“FILEID” on page 63](#)
- [“DSNAME” on page 63](#)
- [“DSNAMEF” on page 63](#)
- [“DSNAMEFM” on page 63](#)
- [“DSNAMEPQ” on page 63](#)
- [“DSNAMEQ” on page 64](#)
- [“NAME” on page 64](#)

- [“NAMEF” on page 64](#)
- [“DBCS” on page 64](#)
- [“EBCDIC” on page 65](#)
- [“MIX” on page 65](#)
- [“ALPHAB” on page 65](#)
- [“PICT” on page 65](#)
- [“PICTCN” on page 66](#)
- [“LISTV” on page 66](#)
- [“LISTVX” on page 66](#)
- [“LEN” on page 67](#)
- [“ENUM” on page 67](#)
- [“BIT” on page 67](#)
- [“NUM” on page 67](#)
- [“HEX” on page 68](#)
- [“INCLUDE” on page 68](#)
- [“IDATE” on page 68](#)
- [“STDDATE” on page 68](#)
- [“JDATE” on page 69](#)
- [“JSTD” on page 69](#)
- [“ITIME” on page 69](#)
- [“STDTIME” on page 69](#)
- [“IPADDR4” on page 69](#)

RANGE

To perform a range test, specify the check item TYPE attribute as RANGE. A range check allows you to check a value within a numeric range including the end points. The PARM1 attribute specifies the lower bound; PARM2 specifies the upper bound. The range delimiters can include 16 digits, and may contain a preceding sign (- or +).

Here is an example where a range check for a NUMERIC variable class ensures that catalog numbers are in the range 50 to 90000000:

```
<!doctype dm system>

<varclass name=catnumc type='NUMERIC 8'>
  <check1 msg=liba005>
    <checki type=range parm1=50 parm2=90000000>
  </check1>
```

The conversion utility generates an ISPF range verification statement in the)PROC section.

ALPHA

To perform an alphabetic test, specify the check item TYPE attribute as ALPHA. An alpha check limits the characters allowed to A-Z, a-z, #, \$, and @.

Here is an example where an alpha check in the *authorc* variable class ensures that authors' names are alphabetic:

```
<!doctype dm system>

<varclass name=authorc type='CHAR 40'>
  <check1 msg=liba006>
```

```
<checki type=alpha>
</check1>
```

The conversion utility generates an ISPF alpha verification statement in the)PROC section.

VALUES

To perform a values test, specify the check item TYPE attribute as VALUES. A values test allows you to specify a list of values. The value the user enters must match one of the values in the list. The PARM1 attribute must have the value EQ. The PARM2 attribute specifies the list of values. Because case is respected in a VALUES check, if you want case to be ignored, you must code an UPPER translation and code the values all in uppercase.

Here is an example where a check in a variable class named *subject* ensures that the value entered is MATH, SCIENCE, ENGLISH, or HISTORY:

```
<!doctype dm system>

<varclass name=subject type='char 10'>
  <xlat1 format=upper>
</xlat1>
  <check1 msg=liba008>
    <checki type=values parm1=eq
      parm2='MATH SCIENCE ENGLISH HISTORY'>
  </check1>
```

The conversion utility generates a LIST verification statement in the)PROC section.

VALUESX

The check item type VALUESX is the opposite of VALUES. This test allows you to specify a list of values that are not valid. The PARM1 attribute must have the value NE. The PARM2 attribute specifies the list of values that are not valid. The value the user enters cannot match any of the values specified in the list. Because case is respected in a VALUESX check, if you want case to be ignored, you must code an UPPER translation and code the values all in uppercase.

Here is an example where a check in a variable class named *subject* ensures that the value entered is not MATH, SCIENCE, ENGLISH, or HISTORY:

```
<!doctype dm system>

<varclass name=subject type='char 10'>
  <xlat1 format=upper>
</xlat1>
  <check1 msg=liba008>
    <checki type=valuesx parm1=ne
      parm2='MATH SCIENCE ENGLISH HISTORY'>
  </check1>
```

The conversion utility generates a LISTX verification statement in the)PROC section.

CHARS

The conversion utility supports BIT, HEX and NUM validation with TYPE=CHARS. The PARM1 attribute must have the value EQ. The PARM2 attribute value can be either "01" for BIT validation, "0123456789ABCDEFabcdef" for HEX validation, or "0123456789" for NUM validation.

Here is an example where a check list in the *hexc* variable class validates hexadecimal values:

```
<!doctype dm system>

<varclass name=hexc type='char 2'>
  <check1 msg=liba008>
    <checki type=chars parm1=eq parm2='0123456789ABCDEFabcdef'>
  </check1>
```

The conversion utility generates an ISPF hex verification statement in the)PROC section.

FILEID

To perform a FILEID test, specify the check item TYPE attribute as FILEID.

Here is an example where a FILEID check in the *infile* variable class ensures that specified variables contain a valid file ID in CMS syntax:

```
<!doctype dm system>

<varclass name=infile type='CHAR 20'>
  <check1 msg=liba010>
    <checki type=fileid>
  </check1>
```

The conversion utility generates an ISPF FILEID verification statement in the)PROC section.

DSNAME

To perform a DSNAME test, specify the check item TYPE attribute as DSNAME.

Here is an example where a DSNAME check in the *namefile* variable class ensures that the specified variables contain a valid TSO file name:

```
<!doctype dm system>

<varclass name=namefile type='CHAR 44'>
  <check1 msg=liba011>
    <checki type=dsname>
  </check1>
```

The conversion utility generates a DSNAME verification statement in the)PROC section.

DSNAMEF

To perform a DSNAMEF test, specify the check item TYPE attribute as DSNAMEF.

Here is an example where a DSNAMEF check in the *namefile* variable class ensures that the specified variables contain a valid TSO file name:

```
<!doctype dm system>

<varclass name=namefile type='CHAR 44'>
  <check1 msg=liba011>
    <checki type=dsnamef>
  </check1>
```

The conversion utility generates a DSNAMEF verification statement in the)PROC section.

DSNAMEFM

To perform a DSNAMEFM test, specify the check item TYPE attribute as DSNAMEFM.

Here is an example where a DSNAMEFM check in the *namefile* variable class ensures that the specified variables contain a valid TSO file name:

```
<!doctype dm system>

<varclass name=namefile type='CHAR 44'>
  <check1 msg=liba011>
    <checki type=dsnamefm>
  </check1>
```

The conversion utility generates a DSNAMEFM verification statement in the)PROC section.

DSNAMEPQ

To perform a DSNAMEPQ test, specify the check item TYPE attribute as DSNAMEPQ.

Here is an example where a DSNAMEPQ check in the *namefile* variable class ensures that the specified variables contain a valid TSO file name:

```
<!doctype dm system>

<varclass name=namefile type='CHAR 44'>
  <check1 msg=liba011>
    <checki type=dsnamepq>
  </check1>
```

The conversion utility generates a DSNAMEPQ verification statement in the)PROC section.

DSNAMEQ

To perform a DSNAMEQ test, specify the check item TYPE attribute as DSNAMEQ.

Here is an example where a DSNAMEQ check in the *namefile* variable class ensures that the specified variables contain a valid TSO file name:

```
<!doctype dm system>

<varclass name=namefile type='CHAR 44'>
  <check1 msg=liba011>
    <checki type=dsnameq>
  </check1>
```

The conversion utility generates a DSNAMEQ verification statement in the)PROC section.

NAME

To perform a NAME test, specify the check item TYPE attribute as NAME.

Here is an example where a NAME check in the *chapters* variable class ensures that the variable contains a valid name, obeying the rules of member names:

```
<!doctype dm system>

<varclass name=chapters type='CHAR 8'>
  <check1 msg=liba012>
    <checki type=name>
  </check1>
```

The conversion utility generates a NAME verification statement in the)PROC section.

NAMEF

To perform a NAMEF test, specify the check item TYPE attribute as NAMEF.

Here is an example where a NAMEF check in the *chapters* variable class ensures that the variable contains a valid name, obeying the rules of member names:

```
<!doctype dm system>

<varclass name=chapters type='CHAR 8'>
  <check1 msg=liba012>
    <checki type=namef>
  </check1>
```

The conversion utility generates a NAMEF verification statement in the)PROC section.

DBCS

To perform a DBCS test, specify the check item TYPE attribute as DBCS.

Here is an example of a DBCS check in the *dbdesc* variable class. This ensures that specified variables contain valid DBCS characters.


```
<!doctype dm system>

<varclass name=dbdesc type='DBCS 12'>
  <check1 msg=liba013>
    <checki type=dbcs>
  </check1>
```

The conversion utility generates a DBCS verification statement in the)PROC section.

EBCDIC

To perform an EBCDIC test, specify the check item TYPE attribute as EBCDIC.

Here is an example where an EBCDIC check in the *title* variable class ensures that specified variables contain valid EBCDIC characters:

```
<!doctype dm system>

<varclass name=title1 type='CHAR 40'>
  <check1 msg=liba014>
    <checki type=ebcdic>
  </check1>
```

The conversion utility generates an EBCDIC verification statement in the)PROC section.

MIX

To perform a MIX test, specify the check item TYPE attribute as MIX.

Here is an example where a MIX check in the title2 variable class ensures that specified variables contain valid DBCS and EBCDIC characters:

```
<!doctype dm system>

<varclass name=title2 type='CHAR 40'>
  <check1 msg=liba015>
    <checki type=mix>
  </check1>
```

The conversion utility generates a MIX verification statement in the)PROC section.

ALPHAB

To perform an ALPHAB test, specify the check item TYPE attribute as ALPHAB. An ALPHAB check limits the characters allowed to A-Z or a-z. Blanks are not allowed.

Here is an example where an ALPHAB check in the *chapters* variable class ensures that chapter names are alphabetic:

```
<!doctype dm system>

<varclass name=chapters type='CHAR 8'>
  <check1 msg=liba016>
    <checki type=alphab>
  </check1>
```

The conversion utility generates an ALPHAB verification statement in the)PROC section.

PICT

To perform a PICT check, specify the check item TYPE attribute as PICT. A PICT check allows you to specify a pattern used to validate the variable. The PARM1 attribute must have the value EQ. The PARM2 attribute contains the validation character string.

Here is an example where a PICT check in the socsec variable class validates the format of a social security number:

```
<!doctype dm system>

<varclass name=socsec type='CHAR 11'>
  <check1 msg=liba017>
    <checki type=pict parm1=eq parm2='nnn-nn-nnnn'>
  </check1>
```

The conversion utility generates a PICT verification statement in the)PROC section.

PICTCN

To perform a PICTCN check, specify the check item TYPE attribute as PICTCN. A PICTCN check allows you to specify a pattern containing required characters to validate the variable. The PARM1 attribute contains a mask character. The PARM2 attribute contains the field-mask. The PARM3 attribute contains the validation string.

Here is an example where a PICTCN check in the socsec variable class validates the format of a social security number, including the hyphen (-) character in positions 4 and 7:

```
<!doctype dm system>

<varclass name=socsec type='CHAR 11'>
  <check1 msg=liba017>
    <checki type=pictcn parm1='*' parm2='***-**-****'
      parm3='nnn-nn-nnnn'>
  </check1>
```

The conversion utility generates a PICTCN verification statement in the)PROC section.

LISTV

To perform a LISTV check, specify the check item TYPE attribute as LISTV. A LISTV test allows you to provide a variable name that has been defined by your application to contain a list of valid variable values. The PARM1 attribute must have the value EQ. The PARM2 attribute must be a variable name entered with "%" as the first character.

Here is an example where a LISTV check in the *majors* variable class validates major subjects, providing the application has previously defined the listitem variable to contain the value "MATH SCIENCE ENGLISH HISTORY":

```
<!doctype dm system>

<varclass name=majors type='CHAR 8'>
  <check1 msg=liba018>
    <checki type=listv parm1=eq parm2=%listitem>
  </check1>
```

The conversion utility generates a LISTV verification statement in the)PROC section.

LISTVX

The check item type LISTVX is the opposite of LISTV. A LISTVX test allows you to provide a variable name that has been defined by your application to contain a list of variable values that are not valid. PARM1 attribute must have the value NE. The PARM2 attribute must be a variable name entered with "%" as the first character.

Here is an example where a LISTVX check in the *majors* variable class validates major subjects, providing the application has previously defined the listitem variable to contain the value "MATH SCIENCE ENGLISH HISTORY":

```
<!doctype dm system>

<varclass name=majors type='CHAR 8'>
  <check1 msg=liba018>
```

```
<checki type=listvx parm1=ne parm2=%listitem>
</check1>
```

The conversion utility generates a LISTVX verification statement in the)PROC section.

LEN

To perform a LEN check, specify the check item TYPE attribute as LEN. A LEN test allows you to validate the length of the variable. The PARM1 attribute can be a relational operator or a variable name that contains a relational operator. Valid relational operators are EQ, LT, GT, LE, GE, NE, NG, or NL. If a variable name is used, it must be preceded by a “%”. The PARM2 value can be either a number or a variable name that contains the number. If you enter a number, it must be in the range of 1-99999. If you use a variable name, it must be preceded by a “%”.

Here is an example where a LEN check in the *chapters* variable class validates the length of chapter names:

```
<!doctype dm system>

<varclass name=chapters type='CHAR 8'>
  <check1 msg=liba019>
    <checki type=len parm1=le parm2=8>
  </check1>
```

The conversion utility generates a LEN verification statement in the)PROC section.

ENUM

To perform an ENUM check, specify the check item TYPE attribute as ENUM. An ENUM check allows you to verify a variable as extended numeric. ISPF verifies variable values for correct decimal and comma notation plus correct sign placement.

Here is an example where an ENUM check in the *quantity* variable class ensures that specified variables are in correct extended numeric format:

```
<!doctype dm system>

<varclass name=quantity type='CHAR 10'>
  <check1 msg=liba020>
    <checki type=enum>
  </check1>
```

The conversion utility generates an ENUM verification statement in the)PROC section.

BIT

To perform a BIT check, specify the check item TYPE as BIT. A BIT check allows you to verify that a variable contains only 0's and 1's.

Here is an example where a BIT check in the *choices* variable class ensures that specified variables are in BIT format:

```
<!doctype dm system>

<varclass name=choices type='CHAR 1'>
  <check1 msg=liba021>
    <checki type=bit>
  </check1>
```

NUM

To perform a NUM check, specify the check item TYPE attribute as NUM. A NUM check allows you to verify a variable as a numeric character 0-9.

Here is an example where a NUM check in the *numbers* variable class ensures that specified variables are numeric:

```
<!doctype dm system>

<varclass name=numbers type='CHAR 5'>
  <check1 msg=liba022>
    <checki type=num>
  </check1>
```

HEX

To perform a HEX check, specify the check item TYPE attribute as HEX. A HEX check allows you to specify a variable that contains only hexadecimal characters (0-9, A-F).

Here is an example where a HEX check in the hexc variable class validates hexadecimal values:

```
<!doctype dm system>

<varclass name=hexc type='CHAR 2'>
  <check1 msg=liba008>
    <checki type=hex>
  </check1>
```

INCLUDE

To perform an INCLUDE check, specify the TYPE attribute as INCLUDE, and, at a minimum, the PARM2 attribute as ALPHA, ALPHAB, or NUM. The PARM1 and PARM3 attributes are optional.

Here is an example where an INCLUDE check in the incl variable class allows an embedded blank and validates the values for both the ALPHA and NUM characters:

```
<!doctype dm system>

<varclass name=incl type='CHAR 10'>
  <check1 msg=liba023>
    <checki type=include parm1=IMBLK parm2=ALPHA parm3=NUM>
  </check1>
```

Note: See the *z/OS ISPF Dialog Developer's Guide and Reference* for more information about panel variable verification.

IDATE

To perform an IDATE check, specify the TYPE attribute as IDATE. An IDATE check allows you to validate an 8 character international date, including the national language date delimiter. The format for the United States is YY/MM/DD.

This example validates an IDATE:

```
<!doctype dm system>

<varclass name=idate type='CHAR 8'>
  <check1 msg=liba024>
    <checki type=idate>
  </check1>
```

STDDATE

To perform an STDDATE check, specify the TYPE attribute as STDDATE. An STDDATE check allows you to validate a 10 character standard date, including the national language date delimiter. The format for the United States is YYYY/MM/DD.

This example validates an STDDATE:

```
<!doctype dm system>
```

```
<varclass name=stddate type='CHAR 10'>
  <check1 msg=liba025>
    <checki type=stddate>
  </check1>
```

JDATE

To perform a JDATE check, specify the TYPE attribute as JDATE. A JDATE check allows you to validate a 6 character Julian date. The format is YY.DDD.

This example validates a JDATE:

```
<!doctype dm system>

<varclass name=jdate type='CHAR 6'>
  <check1 msg=liba026>
    <checki type=jdate>
  </check1>
```

JSTD

To perform a JSTD check, specify the TYPE attribute as JSTD. A JSTD check allows you to validate an 8 character Julian date. The format is YYYY.DDD.

This example validates a JSTD:

```
<!doctype dm system>

<varclass name=jstd type='CHAR 8'>
  <check1 msg=liba026>
    <checki type=jstd>
  </check1>
```

ITIME

To perform an ITIME check, specify the TYPE attribute as ITIME. An ITIME check allows you to validate a 5 character international time, including the national language time delimiter. The format for the United States is HH:MM.

This example validates an ITIME:

```
<!doctype dm system>

<varclass name=itime type='CHAR 5'>
  <check1 msg=liba027>
    <checki type=itime>
  </check1>
```

STDTIME

To perform a STDTIME check, specify the TYPE attribute as STDTIME. A STDTIME check allows you to validate an 8 character standard time, including the national language time delimiter. The format for the United States is HH:MM:SS.

This example validates a STDTIME:

```
<!doctype dm system>

<varclass name=stdtime type='CHAR 8'>
  <check1 msg=liba028>
    <checki type=stdtime>
  </check1>
```

IPADDR4

To perform a IPADDR4 check, specify the TYPE attribute as IPADDR4. A IPADDR4 check allows you to verify a 15 character IP address of the format xxx.xxx.xxx.xxx.

Variable validation

This example validates an IPADDR4:

```
<!doctype dm system>  
  
  <varclass name=ipaddr4 type='CHAR 15'>  
    <check1 msg=liba029>  
      <checki type=ipaddr4>  
    </check1>
```

Overriding variable classes

Some tags, such as DTAFLD, allow you to specify a different variable class for a variable other than the default one that was specified when the variable was declared using the VARDCL tag. This is called an overriding variable class and is used to perform different translates and validity checks from those provided by the default variable class.

Chapter 5. Application panel fields

Most of the direct interaction that takes place between the user and the application is through the use of interactive fields. They provide a means for the user to communicate data to the application, as well as receive data from the application.

The type of interaction the user has with the application depends on the task. The task, in turn, determines the fields' characteristics. The appearance of the fields, the application's response to user input, and assistance such as messages and help information must all be considered when defining an interactive field.

This topic explains how to use the Dialog Tag Language to define these types of fields and their operating characteristics:

- Data fields
- Selection fields
- List fields.

This topic begins with a description of field prompts for data fields and selection fields.

Field prompts

A field prompt is static, descriptive text that explains the field it is associated with. Data fields and selection fields support the use of field prompts. To define a field prompt for a data field or selection field, specify the prompt text as the tag text on the DTAFLD and SELFLD tags.

The PMTLOC attribute defines the location of the prompt using one of these values:

PMTLOC = ABOVE

The prompt appears above and left-aligned with the field. This is the default for selection fields.

PMTLOC = BEFORE

The prompt appears directly in front of and on the same line as the field. This is the default for data fields.

You should define the amount of space the prompt uses by specifying the PMTWIDTH attribute on the DTAFLD and SELFLD tags. If the prompt text is longer than the width you specify on PMTWIDTH, the prompt is word-wrapped on multiple lines. Using the PMTWIDTH attribute can ensure that multiple fields with prompts are aligned evenly. If you do not specify PMTWIDTH, the field prompt length is determined by the length of the prompt text.

When PMTLOC=BEFORE, the conversion utility inserts leader dots at the end of the prompt text to fill the specified prompt width. For output-only data fields, a colon is used in place of the last leader dot. For fields with this prompt location, it is a good idea to specify a PMTWIDTH with a value that allows for leader dots after the prompt text. This lends consistency to the panel appearance. The conversion utility issues a warning message when there is insufficient space for leader dots.

[Figure 29 on page 72](#) shows how prompts appear.

Application Name

Name -----
Address . . -----
City -----
State --

Age --

1. 0 - 12
2. 13 - 19
3. 20 - 29
4. 30 - 49
5. 50 - 64
6. over 65

Method of payment

-- 1. Cash
2. Check
3. Credit card

Payment

Figure 29. Prompt locations

The **Name**, **Address**, **City**, and **State** data fields show the prompts in front of the fields (PMTLOC=BEFORE), as does the **Age** field, which shows a prompt for a selection field. The same prompt width is used on the first five fields so that they align evenly. The **Method of payment** and **Payment** fields demonstrate having the prompt above the field (PMTLOC=ABOVE).

Here is the markup used to demonstrate the field prompts in [Figure 29 on page 72](#):

```
<!doctype dm system>

<varclass name=sampcls type='char 20'>
<varclass name=statcls type='char 2'>
<varclass name=numcls type='numeric 8 2'>
<varclass name=char1 type='char 1'>

<varlist>
  <vardcl name=name varclass=sampcls>
  <vardcl name=addr varclass=sampcls>
  <vardcl name=city varclass=sampcls>
  <vardcl name=stat varclass=statcls>
  <vardcl name=pay varclass=numcls>
  <vardcl name=age varclass=char1>
  <vardcl name=paymeth varclass=char1>
</varlist>

<panel name=pmt01>Application Name
  <area>
    <dtafld datavar=name entwidth=20 pmtwidth=12>Name
    <dtafld datavar=addr entwidth=20 pmtwidth=12>Address
    <dtafld datavar=city entwidth=20 pmtwidth=12>City
    <dtafld datavar=stat entwidth=2 pmtwidth=12>State
    <divider>
      <region dir=horiz>
        <selfld name=age selwidth=20 pmtloc=before pmtwidth=12>Age
          <choice>0 - 12
          <choice>13 - 19
          <choice>20 - 29
          <choice>30 - 49
          <choice>50 - 64
          <choice>over 65
        </selfld>
        <divider gutter=5>
        <selfld name=payment selwidth=24 pmtwidth=20>Method of payment
          <choice>Cash
          <choice>Check
          <choice>Credit card
        </selfld>
      </region>
    
```



```

<divider>
<dtafld datavar=pay entwidth=11 pmtloc=above pmtwidth=7>Payment
</area>
</panel>

```

Figure 30 on page 73 shows how the prompt width can affect the appearance of the prompt text.

```

Application Name

This is a very, very long prompt -----

This is a
very, very
long prompt -----

Here is another long prompt used to show word-wrapping of prompts
-- 1. Choice 1
   2. Choice 2

Here is
another long
prompt used to
show
word-wrapping
of prompts
-- 1. Choice 1
   2. Choice 2

```

Figure 30. Prompt widths

The prompts in the two data fields are formatted differently. The prompt text of the first data field is not wrapped. It formats on one line, using as much space as necessary (up to the maximum available formatting width). The second data field has the same prompt text, with a prompt width that is less than the amount of space needed, so the prompt text is wrapped to as many lines as are needed. Similarly, the two selection fields also demonstrate how the prompt text appears based on the prompt width. The prompt text of data fields and selection fields can be displayed differently by omitting or specifying different values for the PMTWIDTH attribute.

Here is the markup that demonstrates the field prompts in [Figure 30 on page 73](#):

```

<!doctype dm system>

<varclass name=sampcls type='char 10'>
<varclass name=char1 type='char 1'>

<varlist>
  <vardcl name=samplea varclass=sampcls>
  <vardcl name=sampleb varclass=sampcls>
  <vardcl name=samplec varclass=char1>
  <vardcl name=sampled varclass=char1>
</varlist>

<panel name=pmt02>Application Name
  <area>
    <dtacol entwidth=10 selwidth=76>
    <dtafld datavar=samplea>This is a very, very long prompt
    <divider>
    <dtafld datavar=sampleb pmtwidth=12>This is a very, very long prompt
    <divider>
    <selfld name=samplec>Here is another long prompt used to show
      word-wrapping of prompts
      <choice>Choice 1
      <choice>Choice 2
    </selfld>
    <divider>
    <selfld name=sampled pmtwidth=14>Here is another long prompt used to show
      word-wrapping of prompts
      <choice>Choice 1

```

```
<choice>Choice 2
</selfld>
</dtacol>
</area>
</panel>
```

Defining data fields

Data fields are used to display variable data and to allow the user to enter data. To define a data field, use the DTAFD tag. Every data field must have an associated variable, which is specified on the required DATAVAR attribute. Like all variables used on the panel, the variable named on the DATAVAR attribute can be declared using the VARDCL tag.

The purpose of the data field is defined using one of these values on the USAGE attribute of the DTAFD tag:

IN

Defines an entry (input-only) data field. An entry data field allows the user to enter data. When an entry field is initially displayed, it is padded with underscore characters, unless the data is right-justified.

OUT

Defines an output-only data field. An output-only data field is used to display the current value of the variable associated with the data field. The user cannot tab to or interact with an output-only field.

BOTH

Defines an input/output data field. When an input/output field is initially displayed, the current value of the associated variable is displayed, and the user can enter data into the field as well. If you do not specify the USAGE attribute, BOTH is the default.

Data fields support field prompts, which can be placed in front of or above the data field.

This panel contains examples of all three types of data fields:

```

Library Inventory

To add a book to the inventory, complete the fields below, and then press
Enter.

Title . . . . _____
Author . . . . _____
Publisher . . SPOTH AND CRICK
Number of
pages . . . . _____

-----

Today's date is . : 08-10-89
```

Figure 31. Data fields

Here is the markup for [Figure 31 on page 74](#):

```
<!doctype dm system>

<varclass name=titlcls type='char 50'>
<varclass name=bookcls type='char 20'>
```

```

<varclass name=pagecls type='numeric 5'>
<varclass name=datecls type='char 8'>

<varlist>
  <vardcl name=title varclass=titlcls>
  <vardcl name=author varclass=bookcls>
  <vardcl name=publish varclass=bookcls>
  <vardcl name=pages varclass=pagecls>
  <vardcl name=curdate varclass=datecls>
</varlist>

<panel name=dfdxmlp1a>Library Inventory
  <topinst>To add a book to the inventory, complete the fields below,
    and then press Enter.
  <area>
    <dtafld datavar=title usage=in pmtwidth=14>Title
    <dtafld datavar=author usage=in entwidth=20 pmtwidth=14>Author
    <dtafld datavar=publish entwidth=20 pmtwidth=14>Publisher
    <dtafld datavar=pages usage=in entwidth=5 pmtwidth=14>Number of pages
    <divider type=solid gutter=3 gap=no>
    <dtafld datavar=curdate usage=out entwidth=8 pmtwidth=20>Today's date is
  </area>
</panel>

```

In the previous example, there are three input-only data fields, an input/output data field, and an output-only data field. The value of the associated variable is not displayed in an input-only data field, so when the panel is initially displayed, the **Title**, **Author**, and **Number of pages** fields are blank. The **Publisher** data field assumes the default, BOTH, so the current value of the associated variable, *publish*, is displayed in the data field when the panel is initially displayed. The output-only data field is used to display the current date. The user cannot interact with this data field, since it is used only to display variable data. The user can enter data into any of the data fields except the output-only field.

Data field width

The width of a data field is determined by the value you specify for the ENTWIDTH attribute of the DTAFLD tag. You should specify ENTWIDTH for all data fields. In the previous example, ENTWIDTH is specified for each DTAFLD tag except for the **Title** field, whose length is determined as discussed next.

If you do not specify a value for ENTWIDTH, the width of the data field is determined by the value specified for the TYPE attribute of the VARCLASS tag associated with the variable named in the DTAFLD DATAVAR attribute. For example, the **Title** field in Figure 31 on page 74 has an entry width of 50 as determined by the variable class *titlcls*, which has the TYPE value “char 50”. This variable class is associated with the data field through the variable declaration *title*, which is specified as the data field's DATAVAR attribute value. For more information about variables and variable classes, see [Chapter 4](#), “Variables and variable classes,” on page 53.

The formatted width of the field is 2 positions more than the ENTWIDTH value to provide for an attribute byte both before and after the field. The maximum width for an entry field is the remaining available formatting width in the panel.

Note: The conversion utility tracks the remaining width available for use. For data fields, the width of the entry field has first priority, followed by the prompt width, and then the description width.

Data field descriptions

In addition to a field prompt, you can provide additional descriptive text for a data field using the DTAFLDD (data field description) tag. You code the DTAFLDD tag following the definition of the data field being described. The DTAFLDD tag has no attributes or required end tag. Multiple data field descriptions can be coded if necessary, and each description begins a new line.

The data field description appears to the right of the entry field, taking up as much room as is available, unless you have used the DESWIDTH attribute of the DTAFLD tag to specify a width for the description. If the DESWIDTH attribute is defined, the data field description is displayed within the description width specified (or defaulted), and word-wrapped on multiple lines, if necessary.

This panel contains data field descriptions.

```
Library Inventory

To add a book to the inventory, complete the fields below, then press
Enter.

Title . . . . . -----
Author . . . . . ----- Last name, First name, M.I.
Publisher . . . SPOTH AND CRICK
Total number of
pages . . . . . ----- (1 - 99999)
```

Figure 32. Data field description

Here is the markup used to generate the panel in [Figure 32 on page 76](#):

```
<!doctype dm system>

<varclass name=titlcls type='char 50'>
<varclass name=bookcls type='char 20'>
<varclass name=pagecls type='numeric 5'>

<varlist>
  <vardcl name=title varclass=titlcls>
  <vardcl name=author varclass=bookcls>
  <vardcl name=publish varclass=bookcls>
  <vardcl name=pages varclass=pagecls>
</varlist>

<panel name=dfdcmp4>Library Inventory
  <topinst>To add a book to the inventory, complete the fields below,
  then press Enter.
  <area>
    <dtacol pmtwidth=15>
      <dtafld datavar=title usage=in entwidth=50>Title
      <dtafld datavar=author usage=in entwidth=20 deswidth=30>Author
      <dtafld datavar=publish usage=in entwidth=20>Publisher
      <dtafld datavar=pages usage=in entwidth=5 deswidth=15>
        Total number of pages
      <dtafld>(1 - 99999)
    </dtacol>
  </area>
</panel>
```

Data field help

ISPF allows you to provide help on a data field using the HELP attribute on the DTAFLD tag. If you specify the name of a help panel or message for a data field, ISPF knows which help information to display when the user selects help on the data field. If you do not specify help for a data field, the extended help panel (specified with the HELP attribute of the enclosing PANEL tag) is displayed.

Here is an example that shows how to provide help for data fields:

```
<!doctype dm system>

<varclass name=titlcls type='char 50'>
```

```

<varclass name=bookcls type='char 20'>
<varclass name=pagecls type='numeric 5'>

<varlist>
  <vardcl name=title    varclass=titlcls>
  <vardcl name=author   varclass=bookcls>
  <vardcl name=publish  varclass=bookcls>
  <vardcl name=pages    varclass=pagecls>
</varlist>

<panel name=dfdxmp5>Library Inventory
  <topinst>To add a book to the inventory, complete the fields below,
  then press Enter.
  <area>
    <dtacol pmtwidth=15>
      <dtafld datavar=title    help=hlptitl entwidth=50>Title
      <dtafld datavar=author   help=hlpauth entwidth=20 deswidth=30>Author
        <dtafldd>Last name, First name, M.I.
      <dtafld datavar=publish  help=hlppubl entwidth=20>Publisher
      <dtafld datavar=pages    help=hlppage entwidth=5  deswidth=15>
        Total number of pages
      <dtafldd>(1 - 99999)
    </dtacol>
  </area>
</panel>

```

Other data field attributes

There are several other attributes you can specify to tailor a data field to meet the requirements of your application. See [“DTAFLD \(Data Field\)” on page 275](#) for more information. Here is a list that describes each of the remaining DTAFLD attributes and what you can do with them:

REQUIRED

This attribute allows you to indicate if the data field requires input. When you assign a value of YES to this attribute, the user must enter data into the field before ISPF accepts the panel as valid. The default REQUIRED value is NO. This attribute is only valid for data fields defined as input-only or as input/output.

MSG

This attribute identifies the message that should be displayed when the user does not enter any data into an input-required data field. If you do not specify this attribute, ISPF displays a default message. This attribute is valid only if REQUIRED=YES.

[Chapter 7, “Messages,” on page 137](#) tells you how to define application messages.

ALIGN

This attribute allows you to align the variable data within the data field. The default value for ALIGN is *start*, which aligns the data from the left side of the data field. You can also center the data within the field with the *center* value, or justify the data from the right side of the field with the *end* value.

AUTOTAB

This attribute provides automatic cursor movement between data fields. If you specify AUTOTAB=YES for a data field, the cursor automatically moves to the next field that is capable of input. If no other field capable of input exists on the panel, the cursor returns to the beginning of the data field.

DISPLAY

The value you assign to this attribute, either *yes* (the default) or *no*, determines if the data appears on the screen when the user enters it. One way to use DISPLAY=NO is for defining a password.

VARCLASS

This attribute allows you to override the variable class that is specified on the variable declaration (VARDCL) for the data field's data variable (DATAVAR). See [Chapter 4, “Variables and variable classes,” on page 53](#) for a description of variables and variable classes.

FLDSpace

This attribute specifies the space reserved for the data-entry field. When the FLDSpace value is larger than the entry width plus any attributes, blanks are added following the data-entry field. This provides spacing before DTAFLDD tag descriptions.

NOENDATTR

This attribute specifies that no ending attribute character is placed after the data field. NOENDATTR is valid only when WINDOW=NO is specified or when data fields are being formatted within a horizontal region.

PAD

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a "%".

PADC

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a "%".

OUTLINE

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a "%".

PMTFMT

This attribute controls the generation of prompt leader characters. The default is to create CUA leader dots.

PSVAR

This attribute provides the name of a variable that is to be set when a DTAFLD is clicked on for point-and-shoot selection.

PSVAL

This attribute provides the value to be placed in the field specified by the PSVAR attribute.

PAS

This attribute provides a variable name that contains the value ON to enable point-and-shoot for this data field, or OFF to disable point-and-shoot. When PSVAR and PSVAL have been specified without the PAS attribute, the point-and-shoot field is automatically enabled.

CSRGRP

The CSRGRP attribute, in combination with the PAS attribute, enables the use of a cursor group by a client that is using the JSON API.

EXPAND

The EXPAND attribute, used in combination with EXPAND=xy on the PANEL definition, causes the expand characters to be added to the DTAFLD definition, effectively allowing ENTWIDTH to expand.

FLDWIDTH

The FLDWIDTH attribute, used in combination with WINDOW=NO on the PANEL definition, provides the width of a data field that spans multiple lines.

ATTRCHANGE

The ATTRCHANGE attribute specifies that, if required, an additional)ATTR section entry (which can apply to multiple fields) be created instead of a unique ".ATTR" override entry for the current field.

INIT

The INIT attribute provides an initial value for the data field.

DBALIGN

The DBALIGN attribute is used only for DBCS language conversion when PMTLOC=ABOVE to align the prompt text with the data field.

DEPTH

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAME

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAMEP

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PLACE

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PMTSKIP

This attribute, used during horizontal field formatting of input fields, specifies that the cursor should move past the prompt text to the input field.

DESSKIP

This attribute, used during horizontal field formatting of input fields, specifies that the cursor should move past the description text to the next input field.

FLDTYPE

This attribute specifies whether CUA or traditional ISPF attribute definitions are used.

COLOR

When FLDTYPE=ISPF, this attribute specifies the color of the field.

INTENS

When FLDTYPE=ISPF, this attribute specifies the intensity of the field.

HILITE

When FLDTYPE=ISPF, this attribute specifies the highlighting for the field.

ATTRCHAR

This attribute provides a user selected panel attribute for the data field.

CAPS

This attribute specifies whether the field is displayed in uppercase characters.

NOJUMP

This attribute specifies that the JUMP function is disabled for the data field.

AUTOTYPE

This attribute specifies whether ISPF panel logic is added to support the AUTOTYPE function.

AUTOVOL

This attribute specifies an associated volume name when AUTOTYPE = DSN.

AUTODMEM

This attribute specifies whether a member name is part of the data set name when AUTOTYPE = DSN.

VARDCL

This attribute specifies whether the field name is validated to the panel variables specified with the VARDCL tag.

Defining selection fields

Selection fields allow the user to select from a group of choices on an application panel. You can specify if only one choice can be selected from a selection field, or if multiple choices are allowed.

In either case, you use the same DTL tags to define a selection field. The SELFLD (selection field) tag and its required end tag define a selection field. The CHOICE (selection choice) tag defines a choice within a selection field. You code the CHOICE tags between the SELFLD start and end tags, like this:

```
<selfld>
  <choice>
  <choice>
  <choice>
</selfld>
```

Each CHOICE tag defines a choice within the selection field.

Like data fields, selection fields support field prompts, which can be placed in front of or above the selection field. Field prompts are described in [“Field prompts” on page 71](#).

To define the selection field type use the TYPE attribute of the SELFLD tag. The values you can assign to TYPE are:

SINGLE

Specifies the selection field as being a single-choice field. Choices in a single-choice selection field appear in a list with an entry field preceding the first choice in the list. The conversion utility prefixes the text of each choice with a number, so the selection field choices are numbered sequentially. Users indicate choice selection by typing the number of the choice they want in the entry field.

MULTI

Specifies the selection field as being a multiple-choice field. Choices in a multiple-choice selection field appear in a list with a single-character entry field preceding each choice. Users indicate choice selection by typing any nonblank character in the entry fields.

MENU

Specifies the selection field as being a menu-choice field. Choices in a menu-choice selection field are similar to those in a single-choice selection field. TYPE=MENU is valid only when the MENU keyword has been specified on the PANEL tag.

MODEL

Specifies the selection field as being a model-choice field. Choices in a model-choice selection field are similar to those in a menu-choice selection field. TYPE=MODEL is valid only when the MENU keyword has been specified on the PANEL tag.

TUTOR

Specifies the selection field as being a tutor-choice field. Choices in a tutor-choice selection field are similar to those in a menu-choice selection field. TYPE=TUTOR is valid only when the MENU keyword has been specified on the PANEL tag.

The CHOICE tag has two attributes associated with it that are important when defining a selection field: CHECKVAR and MATCH. The CHECKVAR and MATCH attributes are used to preselect choices in the selection field. The CHECKVAR attribute can also communicate to the application which selections were made by the user.

The value specified on the CHECKVAR attribute is the name of a dialog variable that is defined by the application. Both the application and ISPF can set the check variable. Here are topics that describe how the CHECKVAR and MATCH attributes are used for each type of selection field.

Single-choice fields

Use a single-choice selection field when you have a fixed set of choices that are mutually exclusive. That is, the user can select only one of the choices by typing the choice number in the entry field. You can specify the preselected choice in a single-choice selection field so that one item is already selected when the panel is displayed. The user can either leave the preselected choice or enter a different choice number.

To preselect choices in a single-choice selection field, and to find out which choice was selected by the user, you should specify the CHECKVAR and MATCH attributes for each CHOICE tag. For a single-choice field, all of the enclosed choices should refer to the same check variable, but they should have unique MATCH values. The example markup shows how this is coded:

```
<!doctype dm system>

<varclass name=daycls type ='char 1'>

<varlist>
  <vardcl name=day varclass=daycls>
  <vardcl name=choice varclass=daycls>
</varlist>

<panel name=singsel>Schedule Appointments
  <topinst>Choose the most convenient day for your appointment,
    then press Enter.
  <area>
    <selfld name=choice selwidth=30 pmtwidth=9>Weekdays:
      <choice checkvar=day match=M>Monday
      <choice checkvar=day match=T>Tuesday
      <choice checkvar=day match=W>Wednesday
      <choice checkvar=day match=H>Thursday
```



```

    <choice checkvar=day match=F>Friday
  </selfd>
</area>
</panel>

```

To preselect a certain choice, set the check variable, *day*, to the match value for that choice. Assume that the check variable, *day*, is set to M before the panel is displayed. When the panel is displayed, the choice, **Monday**, is selected as shown in [Figure 33 on page 81](#).

Schedule Appointments

Choose the most convenient day for your appointment, then press Enter.

Weekdays:

- 1 1. Monday
2. Tuesday
3. Wednesday
4. Thursday
5. Friday

Figure 33. Single-choice selection field

If the user decides that another day is more convenient, another choice might be selected. This causes the check variable to be updated with the match value of the newly selected choice. For example, if the user selects **Friday** (by typing "5" in the entry field), the check variable, *day*, contains "F" when control is returned to the application.

Note: The TYPE attribute does not have to be specified on a single-choice selection field because TYPE=SINGLE is the default. However, you must specify the NAME attribute for single-choice selection fields.

Multiple-choice fields

Use a multiple-choice selection field when you have several choices for the user, but they are not mutually exclusive. Each choice acts independently as a toggle, and selecting one of the choices does not affect any of the other choices in the selection field.

To preselect choices in a multiple-choice selection field, and to find out which choices were selected by the user, specify the CHECKVAR, MATCH, and NOMATCH attributes for each CHOICE tag.

On a multiple-choice selection field, define a unique check variable for each enclosed CHOICE. You can let the MATCH value default to 1, or specify the MATCH attribute with a value of your choice. Also, you can let the NOMATCH value default to 0, or specify the NOMATCH attribute with a value of your choice. Here is how a multiple-choice selection field is coded:

```

<!doctype dm system>

<varclass name=sampcls type='char 1'>

<varlist>
  <vardcl name=dry varclass=sampcls>
  <vardcl name=cut varclass=sampcls>
  <vardcl name=per varclass=sampcls>

```

```
<vardcl name=fac varclass=sampcls>
<vardcl name=man varclass=sampcls>
<vardcl name=ped varclass=sampcls>
<vardcl name=ch1 varclass=sampcls>
<vardcl name=ch2 varclass=sampcls>
<vardcl name=ch3 varclass=sampcls>
<vardcl name=ch4 varclass=sampcls>
<vardcl name=ch5 varclass=sampcls>
<vardcl name=ch6 varclass=sampcls>
</varlist>

<panel name=multsel>Schedule Appointments
  <area>
    <dtacol pmtwidth=45 selwidth=76>
      <selfld type=multi>Choose the services needed, then press Enter.
        <choice name=ch1 checkvar=dry>Dry haircut
        <choice name=ch2 checkvar=cut>Shampoo, haircut, and style
        <choice name=ch3 checkvar=per>Permanent or body wave
        <choice name=ch4 checkvar=fac>Facial
        <choice name=ch5 checkvar=man>Manicure
        <choice name=ch6 checkvar=ped>Pedicure
      </selfld>
    </dtacol>
  </area>
</panel>
```

You specify preselected choices for a multiple-choice selection field just as you would for a single-choice selection field. Set the check variable for the preselected choices to the match values (or the default value of 1) for those choices. When a choice is preselected, a slash (/) is displayed in the entry field preceding the choice.

When the user types a value in an entry field in a multiple-choice selection field, ISPF toggles the choice in this way:

- If the choice is already selected and the user enters a blank in the entry field, ISPF deselects the choice and sets the check variable to the NOMATCH value for the choice, or to 0 if the NOMATCH attribute is not specified.
- If the choice is not selected and the user types a nonblank character in the entry field, ISPF selects the choice and sets the check variable to the MATCH value for the choice, or to 1 if the MATCH attribute is not specified. If the choice is not selected, ISPF sets the check variable to the NOMATCH value for the choice, or to 0 if the NOMATCH attribute is not specified.

In the preceding markup, the MATCH attribute was not specified, so the check variables toggle between 0 and 1 (the default MATCH and NOMATCH values) as the user selects and deselects items.

Because ISPF is setting the check variable, you should not use the SETVAR or the TOGVAR attributes of the ACTION tag to refer to the check variable.

Figure 34 on page 83 shows how the multiple-choice selection field in the preceding markup appears with the choices **Facial** and **Pedicure** preselected.

Schedule Appointments

Choose the services needed, then press Enter.

- Dry haircut
- Shampoo, haircut, and style
- Permanent or body wave
- / Facial
- / Manicure
- / Pedicure

Figure 34. Multiple-choice selection field

Menu-choice fields

Use a menu-choice selection field to create an ISPF option menu. Menu-choice fields are similar to single-choice fields. That is, the user can select only one of the choices presented. The entry field for this type of selection field is the command line, which is formatted with the word *Option* instead of *Command*. As with single-choice selections, you can specify a preselected choice so that one item is already selected when the panel is displayed.

The CHOICE tag is followed by an ACTION tag which specifies the type of selection (PANEL, PGM, CMD, or EXIT), and other attributes required by the ISPF SELECT service.

When creating an option menu, the MENU keyword is required on the PANEL tag. The optional PRIME keyword causes the creation of a primary option menu. The SELFLD tag must specify TYPE=MENU. Depending on the panel being created, the SELFLD tag attributes ENTWIDTH, FCHOICE, and TRAIL, and the CHOICE tag attribute SELCHAR might be required. See [Chapter 12, “Tag reference,” on page 179](#) for more information on the PANEL, SELFLD, CHOICE, and ACTION tags.

The example markup creates a sample option menu:

```
<!doctype dm system (>
<!-- MENU selection panel example -->
<panel name=menusel1 menu>Sample Option Menu
  <topinst>Enter a selection choice

  <region indent=4>
    <selfld type=menu entwidth=1 selwidth=40>
      <choice checkvar=xtest1 match=a>Select Command
        <action run=tstch1 type=cmd parm='1234'
          newappl=aaaa passlib newpool suspend
          lang=crcx nocheck mode=fscr>
      <choice checkvar=xtest1 match=b>Select Panel
        <action run=tstch2 type=panel
          addpop newappl=aaaa passlib newpool suspend>
      <choice checkvar=xtest1 match=c>Select Program
        <action run=tstch3 type=pgm parm=abcd
          newappl=aaaa passlib newpool suspend
          nocheck mode=fscr>
      <choice checkvar=xtest1 match=x>Exit
        <action run=exit type=exit>
    </selfld>
  </region>
```

```
<cmdarea>  
</panel>
```

The resulting panel is:

Sample Option Menu

Enter a selection choice

- 1 Select Command
- 2 Select Panel
- 3 Select Program
- 4 Exit

Option ==> _____

Figure 35. Sample option menu

Model-choice fields

Use a model-choice selection field to create an ISPF edit model selection menu. Model-choice fields are similar to single-choice or menu-choice fields. That is, the user can select only one of the choices presented. The entry field for this type of selection field is the command line, which is formatted with the word *Option* instead of *Command*. As with single-choice or menu-choice selections, you can specify a preselected choice so that one item is already selected when the panel is displayed.

The CHOICE tag is followed by an ACTION tag which specifies the type of selection (PANEL, PGM, CMD, or EXIT), and other attributes required by the ISPF SELECT service.

When creating an edit model menu, the MENU keyword is required on the PANEL tag. The SELFLD tag must specify TYPE=MODEL. Depending on the panel being created, the SELFLD tag attributes ENTWIDTH, FCHOICE, and TRAIL, and the CHOICE tag attributes SELCHAR, HIDE, and TRUNC might be required. See [Chapter 12, “Tag reference,” on page 179](#) for more information about the PANEL, SELFLD, CHOICE, and ACTION tags.

Tutor-choice fields

Use a tutor-choice selection field to create an ISPF tutorial selection menu. Tutor-choice fields are similar to menu-choice fields. That is, the user can select only one of the choices presented. The entry field for this type of selection field is the command line, which is formatted with the word *Option* instead of *Command*. As with menu-choice selections, you can specify a preselected choice so that one item is already selected when the panel is displayed.

The CHOICE tag is followed by an ACTION tag that must specify the type of selection as PANEL, and other attributes required by the ISPF SELECT service.

When creating a tutorial menu, the MENU keyword is required on the PANEL tag. The SELFLD tag must specify TYPE=TUTOR. Depending on the panel being created, the SELFLD tag attributes ENTWIDTH and FCHOICE, and the CHOICE tag attribute SELCHAR might be required. See [Chapter 12, “Tag reference,” on page 179](#) for more information on the PANEL, SELFLD, CHOICE, and ACTION tags.

Selection field help

ISPF enables you to provide help on selection fields. For single-choice selection fields, you specify the name of a help panel or message for the selection field with the `HELP` attribute of the `SELFLD` tag. For multiple-choice selection fields, you specify the name of a help panel or message for each of the choices in the selection field with the `HELP` attribute of the `CHOICE` tags. For menu-choice, model-choice, or tutor-choice fields, the selection field is the command line. The name of the help panel or message must be provided on the `CMDAREA` tag. If you specify help for a single-choice selection field, a menu-choice selection field, or for choices in a multiple-choice selection field, ISPF displays that help information when the user requests help and the cursor is on that panel element. If there is no help defined, the extended help panel is displayed.

Here is an example that shows how to code a help panel for a single-choice selection field:

```
<selfld name=choice
  help=dayhelp>Weekdays:
  <choice checkvar=day match=M>Monday
  <choice checkvar=day match=T>Tuesday
  <choice checkvar=day match=W>Wednesday
  <choice checkvar=day match=H>Thursday
  <choice checkvar=day match=F>Friday
</selfld>
```

This example shows how to code help panels for choices in a multiple-choice selection field.

```
<selfld type=multi>Choose the services needed:
  <choice name=ch1 help=dryhlp>Dry haircut
  <choice name=ch2 help=cuthlp>Shampoo, haircut, and style
  <choice name=ch3 help=permhlp>Permanent or body wave
  <choice name=ch4 help=facehlp>Facial
  <choice name=ch5 help=manihlp>Manicure
  <choice name=ch6 help=pedihlp>Pedicure
</selfld>
```

Selection width

The `SELWIDTH` attribute of the `SELFLD` tag should be used to define the amount of space taken up by the *choice-description-text* of each `CHOICE` tag. This attribute is used to control the formatting of panels defined with horizontal regions. If you do not specify a `SELWIDTH` value, the conversion utility reserves the remaining available formatting width for the text.

When specifying an explicit `SELWIDTH` value, you must take into consideration the components of the selection field, as well as the *choice-description-text*. The conversion utility reserves a number of positions on the lines that selection field choices appear on for the entry fields, 3270 attributes, and, in the case of single-choice, menu-choice, model-choice, and tutor-choice selection fields, the choice prefixes. See the `SELWIDTH` attribute in [“SELFLD \(Selection Field\)”](#) on page 421 for a discussion of the amount of space reserved for each choice type.

These reserved positions must be added to the length of the *choice-description-text* in the `SELWIDTH` value you specify. Here is an example of markup that contains two selection fields, one single-choice and one multiple-choice, within a horizontal region. To format the selection fields properly, ensure that the `SELWIDTH` values you specify are adequate for the reserved positions and the *choice-description-text*. The largest *choice-description-text* in the first selection field is 9 characters, which, when combined with the 10 reserved positions in the field, means you must specify a `SELWIDTH` value of at least 19. The largest *choice-description-text* in the second selection field is 27 characters, which, when combined with the 5 reserved positions in the field, means you must specify a `SELWIDTH` value of at least 32.

```
<!doctype dm system>
<varclass name=char1 type='char 1'>
<varclass name=char2 type='char 2'>

<varlist>
  <vardcl name=person varclass=char2>
  <vardcl name=ch1 varclass=char1>
  <vardcl name=ch2 varclass=char1>
  <vardcl name=ch3 varclass=char1>
  <vardcl name=ch4 varclass=char1>
```

```
<vardcl name=ch5 varclass=char1>
<vardcl name=ch6 varclass=char1>
</varlist>

<panel name=servsel>Service Selections
  <topinst>Select the stylist and services you want, then press Enter.
  <area>
    <region dir=horiz>
      <selfld name=person selwidth=19 pmtwidth=15>Stylist
        <choice checkvar=stylst match=1>Cecilia
        <choice checkvar=stylst match=2>Dana
        <choice checkvar=stylst match=3>Laurel
        <choice checkvar=stylst match=4>Pierce
        <choice checkvar=stylst match=5>Stephenie
      </selfld>
      <divider>
      <selfld type=multi selwidth=32 pmtwidth=15>Services
        <choice name=ch1 checkvar=dry>Dry haircut
        <choice name=ch2 checkvar=cut>Shampoo, haircut, and style
        <choice name=ch3 checkvar=per>Permanent or body wave
        <choice name=ch4 checkvar=fac>Facial
        <choice name=ch5 checkvar=man>Manicure
        <choice name=ch6 checkvar=ped>Pedicure
      </selfld>
    </region>
  </area>
</panel>
```

Here is the formatted result:

```

Service Selections

Select the stylist and services you want, then press Enter.

Stylist
--  1. Cecilia
   2. Dana
   3. Laurel
   4. Pierce
   5. Stephenie

Services
--  Dry haircut
--  Shampoo, haircut, and style
--  Permanent or body wave
--  Facial
--  Manicure
--  Pedicure
```

Figure 36. Selection field SELWIDTH attribute

Other selection field attributes

There are several other attributes you can specify to tailor a selection field to meet the requirements of your application. See “SELFLD (Selection Field)” on page 421 for more information. Here is a list that describes each of the remaining SELFLD attributes and what you can do with them:

ENTWIDTH

This attribute controls the entry width for single-choice, menu-choice, model-choice, and tutor-choice selections.

REQUIRED

This attribute allows you to indicate if the single-choice selection field requires input. When you assign a value of YES to this attribute, the user must enter data into the field before ISPF accepts the panel as valid. The default REQUIRED value is NO.

MSG

This attribute identifies the message that should be displayed when the user does not enter any data into the selection field. If you do not specify this attribute, ISPF displays a default message. This attribute is valid only if REQUIRED=YES.

Chapter 7, “Messages,” on page 137 tells you how to define application messages.

FCHOICE

This attribute controls the first choice number for single-choice, menu-choice, model-choice, and tutor-choice selections. The value can be either 0 or 1.

AUTOTAB

This attribute provides automatic cursor movement between fields. If you specify AUTOTAB=YES for a selection field, the cursor automatically moves to the next field that is capable of input. If no other field capable of input exists on the panel, the cursor returns to the selection field.

DEPTH

This attribute specifies that the selection list is to be formatted as a scrollable area. A list formatted into multiple columns (using CHOICECOLS) is formatted as multiple scrollable areas.

EXTEND

This attribute is valid only when DEPTH has been specified and specifies that the scrollable area is to be expanded at run time to the size of the logical screen.

TRAIL

This attribute is used with menu-choice selections to specify the name of one or more variables that applications use to obtain TRAIL information created by option menu selection processing.

CHOICECOLS

This attribute is used to specify the number of columns to create for the selection list. When multiple columns are requested, the number of choices placed in each column is obtained from the CHOICEDEPTH attribute.

CHOICEDEPTH

This attribute specifies the number of choices to be formatted into each column of choices. If more choice entries are specified than can be formatted in the available number of columns specified by the CHOICECOLS attribute, the remaining choice entries are placed in the rightmost (or only) available column for the current SELFLD tag.

CWIDTHS

This attribute specifies the number of bytes to be allocated for each column of CHOICE entries. The ‘w1 w2...wn’ notation provides the number of bytes for each column. You may use an asterisk or a number combined with an asterisk to specify a proportional allocation of column space.

PAD

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

PADC

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

OUTLINE

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

SELMSG

This attribute specifies the message that is displayed when an invalid single-choice entry is selected.

SELMSGU

This attribute specifies the message that is displayed when an unavailable single-choice entry is selected.

INIT

This attribute controls the single-choice and multiple-choice selection field variables initialization in the panel)INIT section.

VERIFY

This attribute controls the single-choice verification and menu-choice, model-choice, or tutor-choice selection logic generation in the panel)PROC section.

REFRESH

This attribute controls the creation of the REFRESH statement in the)REINIT section for multi-choice selection variables.

SELFMT

This attribute controls the placement of the choice selection character(s) within the width specified by ENTWIDTH.

CHKBOX

This attribute enables the display of multiple-choice fields as check boxes by a client that is using the JSON API.

ZGUI

This attribute controls the creation of the VGET (ZGUI) statement created as part of the)INIT section for multiple-choice selection definitions using the "&multipmt" built-in ENTITY.

CSRGRP

This attribute, in combination with CHKBOX=YES, provides a cursor group identification for multi-choice selections.

TSIZE

This attribute provides the number of bytes to indent multiple lines of CHOICE text.

LISTTYPE

This attribute controls the display of single-choice selection lists.

LISTREF

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

LISTDEPTH

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

DBALIGN

This attribute, used for DBCS fields when PMTLOC=ABOVE, specifies alignment of the prompt text with the selection input field.

NOSEL

This attribute provides a value to be placed in the CHECKVAR variable (specified by the CHOICE tag), when no selection is made from a single-choice selection list.

SELDEFAULT

This attribute specifies a default choice selection for a single-choice selection list.

PMTSKIP

This attribute, used during horizontal field formatting, specifies that the cursor should move past the prompt text to the input field.

FLDTYPE

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

COLOR

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

INTENS

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

HILITE

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

SELCHECK

This attribute is used with menu-choice selection to specify that panel logic be included in selection processing to check for selection choices that are not valid.

Data columns

The DTACOL (data column) tag can be used to define values for data fields and selection fields that are coded within the data column. If you have a group of data fields and selection fields on the same application panel, the DTACOL tag is a convenient short-cut for ensuring alignment of the fields.

The DTACOL tag has these attributes:

PMTWIDTH

Applies to data fields and selection fields

ENTWIDTH

Applies to data fields only

DESWIDTH

Applies to data fields only

SELWIDTH

Applies to selection fields only

FLDSpace

Applies to data fields only

PAD

Applies to data fields only

PADC

Applies to data fields only

OUTLINE

Applies to data fields only

PMTFMT

Applies to data fields only

AUTOTAB

Applies to data fields only

ATTRCHANGE

Applies to data fields only

PMTLOC

Applies to data fields only

DBALIGN

Applies to data fields only

VARCLASS

Applies to data fields only

REQUIRED

Applies to data fields only

CAPS

Applies to data fields only

These attributes serve the same purposes in DTACOL definitions as they do in CHOFLD, DTAFLD, and SELFLD definitions. The only difference is that when you use them with a DTACOL tag, they define those values for all of the data fields and selection fields coded between the DTACOL start and end tags.

Here is an example of markup that uses a data column to define a prompt width, entry width, and description width for the data fields and the selection field coded within the data column. Because we

want to limit the entry width of the **State** and **Zip code** fields, we defined ENTWIDTH values in the DTAFDL definitions for these fields that override the DTACOL ENTWIDTH value.

```
<!doctype dm system>

<varclass name=sampcls type='char 30'>
<varclass name=statcls type='char 2'>
<varclass name=zipcls type='char 5'>
<varclass name=char1cls type='char 1'>

<varlist>
  <vardcl name=name varclass=sampcls>
  <vardcl name=addr varclass=sampcls>
  <vardcl name=city varclass=sampcls>
  <vardcl name=stat varclass=statcls>
  <vardcl name=day varclass=char1cls>
  <vardcl name=zipc varclass=zipcls>
</varlist>

<panel name=dcolxmp>Schedule Appointments
  <topinst>Enter your name and address and
  choose the most convenient day for your appointment.
  <area>
    <dtacol pmtwidth=12 entwidth=30 deswidth=29 selwidth=30>
      <dtafld datavar=name>Name
        <dtafldd>Last, First, M.I.
      <dtafld datavar=addr>Address
        <dtafldd>If it applies, include apartment number
      <dtafld datavar=city>City
      <dtafld datavar=stat entwidth=2>State
        <dtafldd>Use 2-character abbreviation
      <dtafld datavar=zipc entwidth=5>Zip code
      <divider type=solid gutter=3>
      <selfld name=day pmtloc=before>Weekdays
        <choice>Monday
        <choice>Tuesday
        <choice>Wednesday
        <choice>Thursday
        <choice>Friday
      </selfld>
    </dtacol>
  </area>
</panel>
```

Here is how the panel formats:

```

                                Schedule Appointments

Enter your name and address and choose the most convenient day for your
appointment.

Name . . . . _____ Last, First, M.I.
Address . . _____ If it applies, include
                                apartment number

City . . . . _____
State . . . . __ Use 2-character abbreviation
Zip code . . _____

-----

Weekdays . . __ 1. Monday
                  2. Tuesday
                  3. Wednesday
                  4. Thursday
                  5. Friday
```

Figure 37. Data column

Defining list fields

A list field is used to display ISPF table data in column format, and to allow the user to enter data in the column rows. The list field supports vertical scrolling if all of the data in the list field is not visible.

If you define a list field in a panel, the ISPF application program must use the TBDISPL service to display the panel.

The tags you use to define a list field are:

LSTFLD

To define the list field. A matching end tag is required.

LSTGRP

To define column group headings. A matching end tag is required.

LSTCOL

To define a column within a list field. You code a LSTCOL tag for each column of data in the list field.

LSTVAR

To define a variable model line.

A list field can contain one or more columns of data, where each column can be input-only, output-only, or input/output, as defined by the USAGE attribute on the LSTCOL tag. These are the values you can specify on the USAGE attribute:

IN

Defines an input-only list column. An input-only column is underscore-filled when it is initially displayed, unless the data is right-justified, and the user can enter data into any of the rows in the input column.

OUT

Defines an output-only list column. When the panel is initially displayed, output-only columns display the value of the ISPF table variable associated with the list column. The user cannot interact with an output-only list column.

BOTH

Defines an input/output list column. Input/output list columns display the value of the ISPF table variable associated with the list column when the panel is initially displayed, as well as allowing the user to enter data into any of the rows in the column. BOTH is the default value for the USAGE attribute.

The data that is associated with each list column is specified on the DATAVAR attribute of the LSTCOL tag. Like all variables used on the panel, the data variable should be declared using the VARDCL tag.

The conversion utility builds a model section into the converted application panel. The model section begins with a)MODEL header statement, which includes the variables named by the DATAVAR attributes of each of the LSTCOL tags defined within the LSTFLD.

Application panels defined using the LSTFLD tag must be displayed using the ISPF TBDISPL service. You can specify the optional ROWS=SCAN attribute on the LSTFLD tag to indicate that only those rows meeting the criteria established by a previous TBSARG service are to be displayed.

You can define a column heading for any of the list columns in the list field by specifying the column heading text as the tag text on the LSTCOL tag. You can specify the optional DIV attribute on the LSTFLD tag to create a divider line between the display of table rows. The column headings do not scroll when the list field is scrolled.

A scroll amount field can be placed at the right end of the command line by specifying the SCROLLVAR attribute on the LSTFLD tag. Field level help for the SCROLLVAR field is specified using the SCRHELP attribute. The scroll amount field is displayed in uppercase characters when the SCRCAPS=ON attribute is specified.

This panel shows a list field with six columns. The first column is output-only, and the remaining columns are input/output.

ROW 1 to 9 of 9

Scheduling Account Visits

Enter the account name in the appropriate time slot.

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 08:59	-----	-----	-----	-----	-----
09:00 - 09:59	-----	-----	-----	-----	-----
10:00 - 10:59	-----	Simmons	-----	-----	-----
11:00 - 11:59	-----	-----	-----	-----	-----
12:00 - 12:59	-----	-----	Douglass	Campbell	-----
01:00 - 01:59	-----	-----	-----	-----	-----
02:00 - 02:59	-----	-----	-----	-----	-----
03:00 - 03:59	-----	-----	-----	-----	-----
04:00 - 04:59	-----	-----	-----	-----	-----

***** Bottom of data *****

Command ==> _____ Scroll ==> CSR
 F1=Help F2=Split F3=Exit F9=Swap F12=Cancel

Figure 38. List field

Here is the markup we used to create the panel:

```
<!doctype dm system>

<varclass name=timecls type='char 13'>
<varclass name=vc1 type='char 9'>

<varlist>
  <vardcl name=timecol varclass=timecls>
  <vardcl name=moncol varclass=vc1>
  <vardcl name=tuecol varclass=vc1>
  <vardcl name=wedcol varclass=vc1>
  <vardcl name=thrcol varclass=vc1>
  <vardcl name=fricol varclass=vc1>
</varlist>

<panel name=lstfld2>Scheduling Account Visits
<topinst>Enter the account name in the appropriate time slot.
<area>
  <lstfld scrollvar=scrhmt scrvhlp=scrhlp>
    <lstcol datavar=timecol usage=out colwidth=13>
    <lstcol datavar=moncol colwidth=9>Monday
    <lstcol datavar=tuecol colwidth=9>Tuesday
    <lstcol datavar=wedcol colwidth=9>Wednesday
    <lstcol datavar=thrcol colwidth=9>Thursday
    <lstcol datavar=fricol colwidth=9>Friday
  </LSTFLD>
</area>
<cmdarea>
</panel>
```

List group headings

You can define additional headings for the columns in a list field using the LSTGRP (list group) tag and its matching end tag. You can define a list group for a single list column or for multiple list columns. You nest the list columns you want to provide additional heading text for within the LSTGRP definition.

At least one field from the first line of the model set must be included within a LSTGRP definition.

The HEADLINE attribute of the LSTGRP tag allows you to place dashes in the list group heading. This is handy for list groups that span across several list columns. Specify HEADLINE=YES to produce a dashed list group heading.

The ALIGN attribute of the LSTGRP tag allows you to control the format position of the list group heading. The default value is CENTER. The heading can be left- or right-justified by specifying the values START or END, respectively.

Here is an example where a LSTGRP definition is added to the list field shown in [Figure 38 on page 92](#).

ROW 1 to 9 of 9

Scheduling Account Visits

Enter the account name in the appropriate time slot.

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 08:59					
09:00 - 09:59					
10:00 - 10:59		Simmons			
11:00 - 11:59					
12:00 - 12:59			Douglass	Campbell	
01:00 - 01:59					
02:00 - 02:59					
03:00 - 03:59					
04:00 - 04:59					

***** Bottom of data *****

Command ==> _____ Scroll ==> CSR
 F1=Help F2=Split F3=Exit F9=Swap F12=Cancel

Figure 39. List group

The text of the list group, **Appointments** is centered within the dashes. Here is how we coded the list group:

```
<!doctype dm system>

<varclass name=timecls type='char 13'>
<varclass name=vc1 type='char 9'>

<varlist>
  <vardcl name=timecol varclass=timecls>
  <vardcl name=moncol varclass=vc1>
  <vardcl name=tuecol varclass=vc1>
  <vardcl name=wedcol varclass=vc1>
  <vardcl name=thrcol varclass=vc1>
  <vardcl name=fricol varclass=vc1>
</varlist>

<panel name=lstgrp2>Scheduling Account Visits
  <topinst>Enter the account name in the appropriate time slot.
  <area>
    <lstfld scrollvar=scr1amt scrvhelpt=scrhelp>
      <lstcol datavar=timecol usage=out colwidth=13>
        <lstgrp headline=yes>Appointments
          <lstcol datavar=moncol colwidth=9>Monday
          <lstcol datavar=tuecol colwidth=9>Tuesday
          <lstcol datavar=wedcol colwidth=9>Wednesday
          <lstcol datavar=thrcol colwidth=9>Thursday
          <lstcol datavar=fricol colwidth=9>Friday
        </lstgrp>
      </lstfld>
    </area>
  </cmdarea>
</panel>
```

List column width

You can use the COLWIDTH attribute of the LSTCOL tag to determine the data width to be used by the column. If you do not specify this attribute, the data width and column formatting width are determined by the actual length of the *column-heading*. If the width of the *column-heading* text is greater than the COLWIDTH, it is used as the column formatting width.

The minimum width value is 1 and the maximum is the remaining available panel (or region) width. If the *column-heading* and the COLWIDTH attribute are omitted, the data width and column formatting width are determined by the TYPE value of the associated VARCLASS. If a VARCLASS TYPE value is not available, the size of the column variable name (specified by the DATAVAR attribute) determines the width.

You should code the COLWIDTH attribute with a value equal to the length of the table data variable.

Other list column attributes

There are several other attributes that can be used in the LSTCOL tag. Many of these attributes are the same as attributes on the DTAFLD tag. This list describes these LSTCOL attributes and how they are used:

ALIGN

This attribute aligns the variable data within the list column. The default value for ALIGN is start, which aligns the data from the left side of the column. You can also center the data within the column with the center value, or align the data to the right side of the column with the end value. The attribute value end is useful for right-aligning numbers within an output-only column, because numbers are typically right-aligned.

ATTRCHANGE

This attribute specifies that, if required, an additional)ATTR section entry (which can apply to multiple fields) be created instead of a unique ".ATTR" override entry for the current field.

AUTOTAB

This attribute specifies automatic tabbing. If you assign a value of YES to this attribute, the cursor automatically moves to the next field that is capable of user input when the user enters the last character in the current list column. The default value for AUTOTAB is NO. This attribute is only valid for list columns defined as input-only or as input/output.

CAPS

This attribute specifies whether the data column is displayed in uppercase characters.

CLEAR

This attribute specifies that the column is a table extension variable, which should be cleared before the row is displayed. Column names with the CLEAR attribute are identified by the CLEAR keyword on the)MODEL statement.

COLOR

When COLTYPE=ISPF, this attribute specifies the color for the column.

COLSPACE

The COLSPACE attribute specifies the total number of bytes for the column width, including the leading and trailing attributes, and the trailing blank for input fields. The use of the COLSPACE attribute causes column heading text longer than the COLSPACE value to be flowed into multiple lines.

COLTYPE

The COLTYPE attribute specifies the attribute type to be used for the column.

CSRGRP

This attribute, in combination with the PAS attribute, enables the use of a cursor group by a client that is using the JSON API.

DISPLAY

This attribute specifies whether the data column is visible when the panel is displayed.

FORMAT

This attribute specifies how the data column and its column heading are formatted. If you do not specify this attribute, or if you specify the attribute value START, then the column formats as in ISPF Version 3.1 and ISPF Version 3.2.

HELP

This attribute specifies the help panel name to display when the user requests help on the list column.

HILITE

When COLTYPE=ISPF, this attribute specifies the highlighting for the column.

INTENS

When COLTYPE=ISPF, this attribute specifies the intensity for the column.

LINE

This attribute specifies the model line that contains the variable. You can specify lines 1-8.

MSG

This attribute identifies the message that should be displayed when the user does not enter any data into an input-required list column. If you do not specify this attribute, ISPF displays a default message. This attribute is valid only if REQUIRED=YES. [Chapter 7, “Messages,” on page 137](#) tells you how to define application messages.

NOENDATTR

This attribute specifies that no ending attribute character is placed after the data column. NOENDATTR is ignored for the last data column on each model line. See [“LSTCOL \(List Column\)” on page 332](#) for more information about the NOENDATTR attribute.

OUTLINE

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”. See [“LSTCOL \(List Column\)” on page 332](#) for more information about the OUTLINE attribute.

PAD

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”. See [“LSTCOL \(List Column\)” on page 332](#) for more information about the PAD attribute.

PADC

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”. See [“LSTCOL \(List Column\)” on page 332](#) for more information about the PADC attribute.

PAS

This attribute is used to control the generation of the point-and-shoot indicator for table display panels. You can define this attribute as a variable name preceded by a “%”.

POSITION

This attribute allows you to specify the starting position of the data column. The POSITION value must be greater than the end of the last formatted data column for that model line and less than the right panel margin. Column formatting for adding the data column and text takes place after the starting position has been established. See [“LSTCOL \(List Column\)” on page 332](#) for more information.

REQUIRED

This attribute indicates if this column is required to have input for any modified row. For input-required columns (REQUIRED=YES), ISPF does not validate the panel unless the user has entered data into that column. If you do not specify this attribute, input is not required on the list column. This attribute is only valid for list columns defined as input-only or as input/output.

TEXT

This attribute specifies a short description of the data column. Text can be placed before or after the data column. See [“LSTCOL \(List Column\)” on page 332](#) for more information.

TEXTLOC

This attribute specifies the location of the TEXT relative to the data column. Text can be placed on either side of the data column. See [“LSTCOL \(List Column\)” on page 332](#) for more information.

TEXTFMT

This attribute specifies the format of the text within the length of the text area. The text can be left-justified, centered, or right-justified. See [“LSTCOL \(List Column\)” on page 332](#) for more information.

TEXTLEN

This attribute specifies the amount of space to reserve for formatting the descriptive text. This helps you line up text on different model lines, and if the space reserved is longer than the descriptive text, TEXTLEN permits formatting within the reserved space with the TEXTFMT attribute. See [“LSTCOL \(List Column\)” on page 332](#) for more information.

TEXTSKIP

This attribute specifies the cursor should move past the text to the next input field.

VARCLASS

This attribute allows you to override the variable class that is specified on the variable declaration (VARDCL) for the list column's data variable (DATAVAR). See [Chapter 4, “Variables and variable classes,” on page 53](#) for a description of variables and variable classes.

Defining group headings

The Group Header (GRPHDR) tag defines a group heading in the panel)BODY section.

The FORMAT attribute is used to control the type of text formatting. You can choose formatting similar to the LINES tag or the P tag. For example, if FORMAT=NONE, the text formats as if you used a LINES tag. However, if FORMAT=START, CENTER, or END, the text flows to multiple lines and is formatted at the right, center or left part of the space reserved for the group heading.

Here is a short description of the other available attributes:

WIDTH

This attribute specifies the number of columns reserved for the group heading. The default value is the remaining panel width.

FMTWIDTH

This attribute specifies the number of columns (of the WIDTH value) to use for formatting the group heading. The default is the WIDTH value. By specifying a FMTWIDTH that is less than the WIDTH value, the group heading text can be formatted on multiple lines.

INDENT

This attribute specifies the number of bytes that the group heading is to be indented.

HEADLINE

This attribute specifies whether dashes are added to span the width of the group heading not occupied by text.

DIV

This attribute specifies the type of divider line to be placed before and after the group heading text.

DIVLOC

This attribute specifies whether the divider is to be added before the group heading, after the group heading, or both before and after the group heading.

COMPACT

This attribute causes the group heading to format without a blank line before the group heading.

STRIP

This attribute causes leading and trailing blanks to be removed from the group heading text.

Defining point-and-shoot fields

The Point-and-Shoot (PS) tag is used to identify a portion of panel)BODY section text to be used for point-and-shoot selection. When a point-and-shoot selection is made, a variable is set to a specified value before normal)PROC section processing. The PS tag attributes identify the variable name and the value associated with each point-and-shoot selection.

The PS tag requires a matching end tag to indicate the end of the point-and-shoot text.

See the *z/OS ISPF Dialog Developer's Guide and Reference* for more information about point-and-shoot selection.

Defining scrollable fields

A scrollable field can be used when the size of the field defined on the panel is smaller than the amount of data to be displayed. With the cursor placed in the field, the LEFT and RIGHT commands can be used to scroll the data displayed. In addition, the EXPAND command can be used to display the data in a popup window.

With DTL, fields that can be made scrollable are defined using the DTAFLD or LSTCOL tags. A field is made scrollable by nesting a SCRFLD tag in the DTAFLD or LSTCOL tag. Here are the attributes of the SCRFLD tag that allow you to specify dialog variables to contain scroll indicators. The conversion utility generates output fields on the panel to allow the scroll indicators to be displayed along with the scrollable field:

INDVAR

A 2-byte left and right scroll indicator that shows whether left and right scrolling can be performed.

LINDVAR

A 1-byte left scroll indicator that shows whether left scrolling can be performed.

RINDVAR

A 1-byte right scroll indicator that shows whether right scrolling can be performed.

SINDVAR

A separator scroll indicator that shows the length of the scrollable field and whether left and right scrolling can be performed.

LCOLIND

A left column position indicator that shows the position of the character currently displayed in the leftmost byte of the scrollable field.

RCOLIND

A right column position indicator that shows the position of the character currently displayed in the rightmost byte of the scrollable field.

SCALE

A scale indicator showing the positions of the columns currently displayed in the scrollable field.

Here is the markup used for the Data Columns example (see Figure 37 on page 90), modified to display the Name and Address fields as scrollable fields. The Name field is displayed with a separator scroll indicator and the Address field is displayed with a scale indicator. The conversion utility automatically generates the separator scroll indicator below the Name field and the scale indicator below the Address field.

```
<!doctype dm system>
<varclass name=sampcls type='char 30'>
<varclass name=statcls type='char 2'>
<varclass name=zipcls type='char 5'>
<varclass name=char1cls type='char 1'>

<varlist>
  <vardcl name=name varclass=sampcls>
  <vardcl name=addr varclass=sampcls>
  <vardcl name=city varclass=sampcls>
  <vardcl name=stat varclass=statcls>
  <vardcl name=day varclass=char1cls>
  <vardcl name=zipc varclass=zipcls>
</varlist>

<panel name=scrixmp depth=24>Schedule Appointments
  <topinst>Enter your name and address and
  choose the most convenient day for your appointment.
  <area>
    <dtacol pmtwidth=12 entwidth=30 deswidth=29 selwidth=30>
      <dtafld datavar=name>Name
        <dtafldd>Last, First, M.I.
        <scrfld dispen=50 sindvar=namesi>
      <dtafld datavar=addr>Address
        <scrfld dispen=80 scale=addrsi>
      <dtafld datavar=city>City
```

```
<dtafld datavar=stat entwidth=2>State
  <dtafldd>Use 2-character abbreviation
<dtafld datavar=zipc entwidth=5>Zip code
<divider type=solid gutter=3>
<selfld name=day pmtloc=before>Weekdays
  <choice>Monday
  <choice>Tuesday
  <choice>Wednesday
  <choice>Thursday
  <choice>Friday
</selfld>
</dtacol>
</area>
</panel>
```

This is how the panel displays:

Schedule Appointments

Enter your name and address and choose the most convenient day for your appointment.

Name Veryveryverylongsurname, Alexa Last, First, M.I.
 ----->

Address . . Apartment 52b, 446 Verylongstr
 -----1-----2-----3

City _____

State . . . Use 2-character abbreviation

Zip code . . _____

Weekdays . . 1. Monday
 2. Tuesday
 3. Wednesday
 4. Thursday
 5. Friday

Command ===> _____ Scroll ===> CSR
 F1=Help F2=Split F3=Exit F9=Swap F12=Cancel

Figure 40. Scrollable field

When the scrollable field is defined using the LSTCOL tag the conversion utility automatically generates, along with the column heading, output fields for any scroll indicators you specify. Here is the markup used for the List Group Headings example (see [Figure 39 on page 93](#)), modified to display the Appointment data in scrollable fields. This would allow more information than just the account name to be stored and displayed in the Appointment data. A scale indicator is displayed with the heading for each day's column.

```
<!doctype dm system>

<varclass name=timecls type='char 13'>
<varclass name=vc1 type='char 9'>

<varlist>
  <vardcl name=timecol varclass=timecls>
  <vardcl name=moncol varclass=vc1>
  <vardcl name=tuecol varclass=vc1>
  <vardcl name=wedcol varclass=vc1>
  <vardcl name=thrcol varclass=vc1>
  <vardcl name=fricol varclass=vc1>
</varlist>

<panel name=scrxmp2>Scheduling Account Visits
  <topinst>Enter the appointment details in the appropriate time slot.
  <area>
    <lstfld scrollvar=scr1amt scrvhelp=scrhelp>
    <lstcol datavar=timecol usage=out colwidth=13>
```

```
<lstgrp headline=yes>Appointments
  <lstcol datavar=moncol colwidth=9>Monday
    <scrfld displen=30 scale=monscl>
  <lstcol datavar=tuecol colwidth=9>Tuesday
    <scrfld displen=30 scale=tuescl>
  <lstcol datavar=wedcol colwidth=9>Wednesday
    <scrfld displen=30 scale=wedscl>
  <lstcol datavar=thrcol colwidth=9>Thursday
    <scrfld displen=30 scale=thrscl>
  <lstcol datavar=fricol colwidth=9>Friday
    <scrfld displen=30 scale=friscl>
</lstgrp>
</lstfld>
</area>
<cmdarea>
</panel>
```

This is how the panel displays:

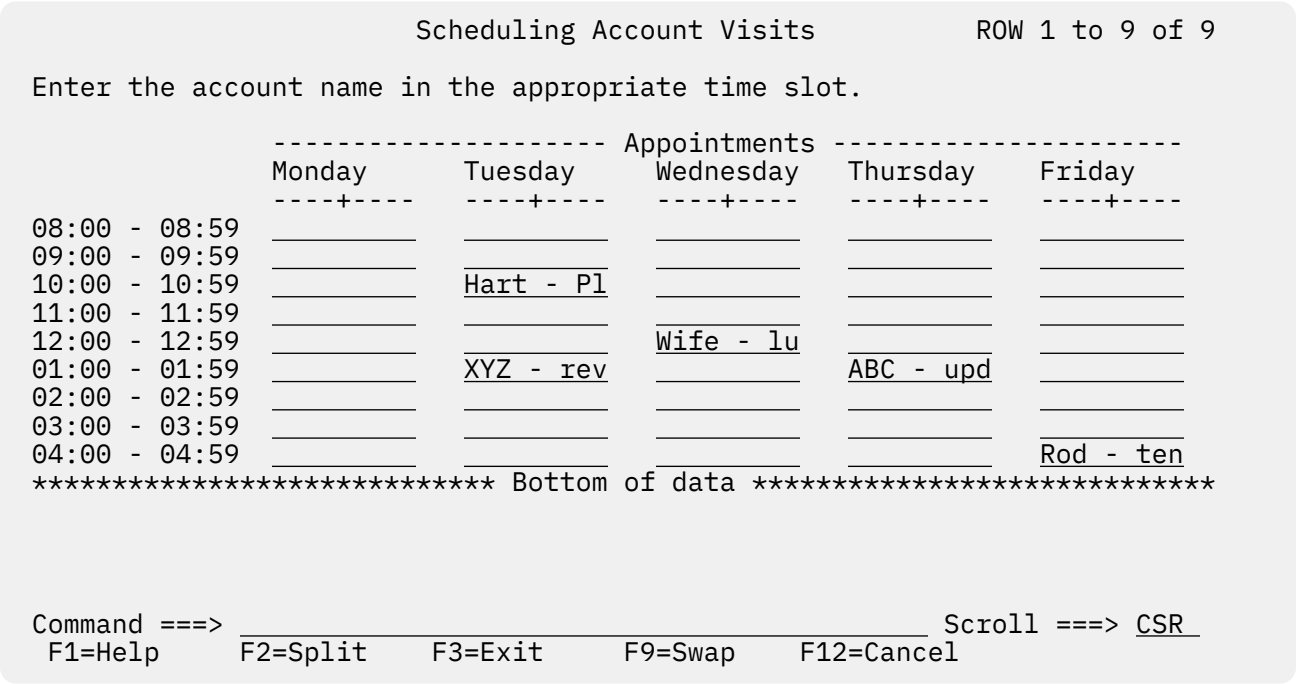


Figure 41. Scrollable field within a list column

Chapter 6. Information regions and help panels

Some of the information displayed on panels is *static*, or fixed text that the user does not interact with directly. This includes text such as top instructions and bottom instructions, prompt text, and data-field description text. DTL provides you with another method of defining static text for application panels using *information regions*.

Defining an information region on a panel allows you more flexibility for defining static text on a panel. The tags you use to define the text of information regions are much more versatile than the tags you use to define other types of static text, which means you can be more creative in the text you define.

In addition to using information regions on application panels, you must use them to define the text on help panels you define for your application. This chapter explains how to define information regions on application panels, and how to define help panels for your applications.

Defining an information region

Use the INFO tag and its required end tag to define an information region on a panel. You can code an information region within an AREA, HELP, PANEL, or REGION definition.

Here is an example of an INFO definition:

```
<panel name=infopan width=42 depth=16>Information
  <area>
    <info>
    </info>
  </area>
</panel>
```

The INFO tag has an optional WIDTH attribute that defines the width of the information region. If the value you assign the INFO WIDTH attribute is greater than the WIDTH available in the panel, the conversion utility resets the value to the available width.

Note: You should code the WIDTH attribute if the information region is part of an application panel definition that uses horizontal region capability.

The INFO tag only defines an information region. It does not define the text of the information region. DTL provides you with a set of tags that define the text in information regions. These tags are:

- ATTENTION
- CAUTION
- DL (definition list)
- FIG (figure)
- Hn (heading)
- HP (highlighted phrase)
- LINES
- NOTE
- NOTEL (note list)
- NT (note)
- OL (ordered list)
- P (paragraph)
- PARML (parameter list)
- RP (reference phrase)
- PS (point-and-shoot)

Defining basic text

- SL (simple list)
- UL (unordered list)
- WARNING
- XMP (example).

With the exception of HP, PS, and RP, these tags can be coded *only* within an INFO definition. The next section explains how to use each of these tags and some other tags that complement these tags within information regions.

Defining basic text

You can define a lot of text using four basic units:

- Paragraphs
- Headings
- Lines
- Examples

Paragraphs

The tag you use most often in information regions is the P (paragraph) tag. Use the P tag to arrange text as you would arrange a paragraph in your usual writing (to join one or more sentences related by their subject matter into a single block of text).

When the paragraph text formats for display, the text starts at the current margin and the words automatically wrap to fit within the margin. In addition, the conversion utility normally inserts a blank line before each paragraph.

The P tag has an optional attribute, COMPACT, which causes the blank line before the paragraph to be omitted. The P tag does not require a matching end tag.

We'll illustrate the use of the P tag with this example:

```
<!doctype dm system>
<panel name=infopan1 width=42>Information
  <area>
    <info width=40>
      <p>This is a paragraph.
        This sentence is also part of the paragraph.
    </info>
  </area>
</panel>
```

Notice that we coded the second sentence of the paragraph on a different line. It doesn't matter, because the conversion utility treats it as part of the same paragraph and formats it accordingly. That is, two blanks are automatically inserted between the sentences. Here is how the paragraph looks:

Information

This is a paragraph. This sentence is also part of the paragraph.

Figure 42. Paragraph

As you can see, the text of the paragraph is left-justified on the panel and the words automatically wrap to fit within the defined dimensions of the information region.

We'll add another paragraph to the panel to illustrate how two paragraphs format:

```
<!doctype dm system>
<panel name=infopan2 width=42>Information
  <area>
    <info width=40>
      <p>This is a paragraph.
        This sentence is also part of the paragraph.
      <p>Here is another paragraph.
        Paragraphs are useful for providing
        information on panels.
    </info>
  </area>
</panel>
```

[Figure 43 on page 103](#) shows the result:

Information

This is a paragraph. This sentence is also part of the paragraph.

Here is another paragraph. Paragraphs are useful for providing information on panels.

Figure 43. Multiple paragraphs

In addition to the placement and wrapping of the text, the compiler separated the paragraphs with a blank line.

Headings

The Hn (heading) tag allows you to place headings in an information region. You use these headings to define topics and subtopics of information. You can define four levels of headings:

H1

Centers text in the information region. Use this heading level to identify a main topic of information.

H2, H3, H4

Formats text against the left margin of the information region. Use one of these heading levels to identify subtopics of information.

You must code headings sequentially. The conversion utility adds a blank line to the information region before and after the formatted heading text. The heading tags have no attributes associated with them, and they don't require an end tag.

Here is markup that contains an information region using two heading levels and paragraphs following each one.

```
<!doctype dm system>
<panel name=infopan3 width=42>Information
  <area>
    <info width=40>
      <h1>A Main Topic
      <p>Notice how the heading is in the
        center of the information region?
      <h2>A Subtopic
      <p>This heading is left-justified.
      <h2>Another Subtopic
      <p>Here's another level-two heading.
    </info>
  </area>
</panel>
```

Here is the formatted result:

```

      Information
      A Main Topic

Notice how the heading is in the center
of the information region?

A Subtopic

This heading is left-justified.

Another Subtopic

Here's another level-two heading.
```

Figure 44. Headings (H1-H2)

Lines

Occasionally, you'll want to present text that you don't want formatted by the compiler, or that you want to show "as is". You can use the LINES (lines) tag and its required end tag to do this. All text coded within a LINES definition is treated as unformatted text, and you can position the text however you like on each line. If the text line is too long to fit in the available width, the conversion utility truncates the text and issues a warning message.

The LINES tag requires an end tag.

There are many ways to use a LINES definition. Here we use it for a quotation:

```
<!doctype dm system>
<panel name=specact width=48>Special Activities
  <area>
    <info width=46>
      <lines>
        Between the dark and daylight,
        When the night is beginning to lower,
        Comes a pause in the days' occupations,
        That's known as the children's hour.
          -Longfellow
      </lines>
      <p>Every Tuesday evening at seven
        o'clock, we present the Children's Hour,
        a one-hour recital of selected children's
        stories in our children's section.
      </p>
    </info>
  </area>
</panel>
```

Our quotation appears just the way we marked it up:

```
Special Activities

Between the dark and daylight,
When the night is beginning to lower,
Comes a pause in the days' occupations,
That's known as the children's hour.
    -Longfellow

Every Tuesday evening at seven o'clock, we
present the Children's Hour, a one-hour
recital of selected children's stories in our
children's section.
```

Figure 45. LINES

Examples

The XMP (example) tag is similar to the LINES tag, in that it allows you to code unformatted text. However, the text of an XMP definition is indented two spaces from the current margin, as opposed to the text of a LINES definition, which is not indented from the current margin.

Like a LINES definition, you should avoid coding lines of text in an XMP definition that exceed the available formatting width of the information region. If the text exceeds the defined width, it is truncated.

The XMP tag requires a matching end tag.

Here's the formatted result of an example using the XMP tag:

```
Order a Toy

Type the catalog number of the toy you want to order
and press Enter. The number must be a 6-digit number.

For example:

Catalog Number. . . 581678
```

Figure 46. XMP

The markup for the previous panel looks like this:

```
<!doctype dm system>
<panel name=toy1 width=57>Order a Toy
<area>
<info width=55>
<p>Type the catalog number of the toy you want to order
and press Enter.
The number must be a 6-digit number.
<p>For example:
<xmp>
Catalog Number. . . 581678
</xmp>
</info>
</area>
</panel>
```

Figures

The FIG (figure) tag is yet another way you can code text that isn't formatted. It works just like the LINES tag, except you can add a ruled border above and below the figure to separate it from the rest of the panel. You can also provide a caption for the figure using the FIGCAP tag.

Like the LINES and XMP tags, the FIG tag requires an end tag.

To define the ruled borders for the figure, use the FRAME attribute of the FIG start tag. The FRAME attribute has two values, RULE, which is the default, and NONE. Because RULE is the default value, you don't need to specify this attribute if you want ruled lines above and below the figure. To create a figure without rules, specify NONE as the FRAME value.

The figure in this panel formats with a ruled border:

```
<!doctype dm system>
<panel name=toy2 width=57>Order a Toy
<area>
<info width=55>
<p>Type the catalog number of
the toy you want to order and
press Enter.
The number must be a 6-digit number.
<p>For example:
<xmp>
Catalog Number. . . 581678
</xmp>
<p>A description of the toy will appear.
<fig>
    ZOOM-A-GO DAREDEVIL SET

    Your kids will have hours of excitement
```

```

    playing with this full set of action toys.
    Requires 80 "AA" batteries.  Not included.
</fig>
</info>
</area>
</panel>

```

Here is the formatted panel:

```

                                Order a Toy

Type the catalog number of the toy you want to order
and press Enter. The number must be a 6-digit number.

For example:

    Catalog Number. . . 581678

A description of the toy will appear.

-----

                                ZOOM-A-GO DAREDEVIL SET

Your kids will have hours of excitement
playing with this full set of action toys.
Requires 80 "AA" batteries.  Not included.

-----

```

Figure 47. Figure with rules

If we wanted the figure to appear without a ruled border, we would have specified `FRAME=NONE` for the `FIG` tag.

The `FIG` tag also has an optional `WIDTH` attribute that allows you to specify how the figure is aligned in the information region. The valid values for `WIDTH` are `PAGE` and `COL`. `PAGE`, which is the default value, aligns the figure along the left margin of the information region. `COL` indicates that the figure is aligned along the current left margin; that is, the current margin defined by the tag the figure is nested in. This is useful, for example, for aligning figures within list items.

Figure captions (`FIGCAP`) tag

To add a caption to the figure in [Figure 47 on page 107](#), use a `FIGCAP` tag and caption text within the figure definition, like this:

```

<!doctype dm system>
<panel name=toy3 width=57>Order a Toy
<area>
<info width=55>
<p>Type the catalog number of
the toy you want to order and
press Enter.
The number must be a 6-digit number.
<p>For example:
<xmp>
Catalog Number. . . 581678
</xmp>
<p>A description of the toy will appear.
<fig>
    ZOOM-A-GO DAREDEVIL SET

    Your kids will have hours of excitement
    playing with this full set of action toys.
    Requires 80 "AA" batteries.  Not included.
<figcap>Zoom-A-Go Daredevil Set
</fig>
</info>
</area>
</panel>

```

The figure caption appears just below the bottom figure rule:

```
Order a Toy

Type the catalog number of the toy you want to order
and press Enter. The number must be a 6-digit number.

For example:

    Catalog Number. . . 581678

A description of the toy will appear.

-----

        ZOOM-A-GO DAREDEVIL SET

Your kids will have hours of excitement
playing with this full set of action toys.
Requires 80 "AA" batteries. Not included.

-----

Zoom-A-Go Daredevil Set
```

Figure 48. Figure caption

Defining lists

Sometimes you want to present information to the user that is not appropriate in paragraph form, such as list items, a sequence of items or actions, or definitions. For these situations (and many others), you can use the DTL list tags to format your text appropriately.

You can create these types of lists:

Note lists

Format as numbered lists of notes under a header called **Notes**.

Simple lists

Format as indented lists of items without any preceding identifiers.

Unordered lists

Format as indented lists of items with each item preceded by a bullet (o), a hyphen (-), or dashes (--), depending on the level of nesting.

Ordered lists

Format as indented lists of items with each item preceded by a number or letter indicating its sequence in the list.

Definition lists

Format in two columns, with terms in one column and their matching descriptions in the other. You can also specify headings for each column in the list. (This list is a definition list.)

Parameter lists

Format in two columns. This list is specifically designed to identify and define parameter terms.

The list items in note lists, simple lists, unordered lists, and ordered lists are created with the list item (LI) tag. The LI tag does not require an end tag. It is implicitly ended by another LI tag, an LP tag, or the end tag of the list it is coded within.

Note lists

See [“Alerting users: notes, warnings, cautions, and attention” on page 121](#) for an example showing the use of note lists.

Simple lists

A simple list is the least complex type of list. Use a simple list when the information you are presenting does not follow a sequential pattern or when bullets are not required to discriminate one list item from another.

Figure 49 on page 109 illustrates a simple list.

```

Virtues

Around the child bend all the three sweet
graces:

    Faith

    Hope

    Charity

```

Figure 49. Simple list

This is the markup for the panel:

```

<!doctype dm system>
<panel name=slistx1 width=44>Virtues
  <area>
    <info width=42>
      <p>Around the child bend all the
three sweet graces:
      <sl>
        <li>Faith
        <li>Hope
        <li>Charity
      </sl>
    </info>
  </area>
</panel>

```

We used the SL tag and its matching end tag to define the simple list. We defined each of the list items by nesting the LI tags definition. within the simple list

As you can see, our simple list formatted with a blank line between each of the list items. For cases where you need to conserve space, you can use the COMPACT attribute to format the list without blank lines between the list items.

Code the COMPACT attribute within the SL start tag (before the tag close delimiter), like this:

```

<!doctype dm system>
<panel name=slistx2 width=44>Virtues
  <area>
    <info width=42>
      <p>Around the child bend all the
three sweet graces:
      <sl compact>
        <li>Faith
        <li>Hope
        <li>Charity
      </sl>
    </info>
  </area>
</panel>

```

Defining lists

Now the simple list is compacted:

```
Virtues
Around the child bend all the three sweet
graces:
    Faith
    Hope
    Charity
```

Figure 50. Compact simple list

You can also nest simple lists within other lists. The list items format at different indentation levels, based on the level of nesting.

The indentation for the list item is based on the SPACE attribute of the LI tag and the enclosing list tag. When SPACE=NO (or the SPACE attribute is not present) the list item indentation is 4 spaces. When SPACE=YES, the indentation is 3 spaces. See [Chapter 12, “Tag reference,” on page 179](#) for additional information about the LI, SL, OL, and UL tags.

Unordered lists

Unordered lists are similar to simple lists, except each list item is preceded by symbol that is dependent on the nesting level of the list. You don't have to supply the symbols—the conversion utility does that for you.

Use an unordered list if the list items are long and you don't want to imply any particular sequence in the list.

Here is an unordered list:

```
Window Shopper
With Window Shopper, you can order many
wonderful things, such as:
o  Raindrops on roses
o  Whiskers on kittens
o  Bright copper kettles
o  Warm woolen mittens
o  Brown paper packages tied up with string
And many more of your favorite things!
```

Figure 51. Unordered list

Here is the markup for this unordered list:

```

<!doctype dm system>
<panel name=winshop width=48>Window Shopper
  <area>
    <info width=46>
      <p>With Window Shopper, you can order many wonderful things,
        such as:
        <ul>
          <li>Raindrops on roses
          <li>Whiskers on kittens
          <li>Bright copper kettles
          <li>Warm woolen mittens
          <li>Brown paper packages tied up with string
        </ul>
      <p>And many more of your favorite things!
    </info>
  </area>
</panel>

```

For our unordered list, we used the UL tag and its matching end tag. As you can see, even though we didn't code the bullet symbols (o) in the markup, they appear in front of each of the list items in the unordered list.

We could make this list compact like our simple list example because the COMPACT attribute is also valid for the UL tag. Likewise, we could use the SPACE attribute to control indentation of the list items for the UL tag.

You can also define levels of unordered lists; that is, you can nest unordered lists within other unordered lists. When you do this, the symbols preceding the list items in each level of the list vary, depending on the level of nesting. Specifically, the list items in the first (or only) level of unordered list are preceded by bullets (o), as shown in [Figure 51 on page 110](#). If you nest another unordered list within an unordered list, the list items in that list are preceded by hyphen symbols (-). A third-level unordered list has dashes (--) preceding the list items. The nested tag text is aligned according to the level of nesting.

To show how this works, we'll create a panel with three levels of unordered lists.

```

<!doctype dm system>
<panel name=ulists width=42>Nested Unordered Lists
  <area>
    <info width=40>
      <ul>
        <li>First level, first item
        <li>First level, second item
          <ul>
            <li>Second level, first item
            <li>Second level, second item
              <ul>
                <li>Third level, only item
              </ul>
            </ul>
          </ul>
        <li>Back to the first level
      </ul>
    </info>
  </area>
</panel>

```

Here is how this panel looks:

```
Nested Unordered Lists
o  First level, first item
o  First level, second item
    - Second level, first item
    - Second level, second item
        -- Third level, only item
o  Back to the first level
```

Figure 52. Nested unordered lists

If you nest more than three levels of unordered lists, the sequence of bullets, hyphens, and dashes repeats. For example, a fourth level would be preceded by bullets, a fifth level by hyphens, and so on.

Remember, all lists must be explicitly ended with the appropriate list end tag.

Ordered lists

Ordered lists imply an outline sequence to the list items by preceding each of the list items with a number or character depending on the level of nesting.

Here is an ordered list:

```
Window Shopper

After you have placed your order with Window
Shopper, you should...

1.  Press the Enter key to leave the Order Panel.
2.  Go to the receiving desk located at the front
    of the store.
3.  Give the cashier the pink copy of your
    receipt.
4.  Take your purchases home, and enjoy!
```

Figure 53. Ordered list

You don't supply the numbers for the list items in your markup; they are generated automatically. This saves you time when you revise ordered lists, because you can insert, delete, or rearrange list items without renumbering them yourself.

Here is the markup we used for this list:

```
<!doctype dm system>
<panel name=winshop2 width=52>Window Shopper
  <area>
    <info width=50>
      <p>After you have placed your order with Window Shopper, you should...
```



```

    <ol>
      <li>Press the Enter key to leave the Order Panel.
      <li>Go to the receiving desk located at the front of the store.
      <li>Give the cashier the pink copy of your receipt.
      <li>Take your purchases home, and enjoy!
    </ol>
  </info>
</area>
</panel>

```

Like other types of lists, you can nest ordered lists within other lists. And, like unordered lists, the levels of the lists you nest determine the characters that precede the list items.

Specifically, the conversion utility uses this sequence when processing list items in nested ordered lists:

- First-level list items are preceded by sequential numbers followed by a period and 2 spaces ¹.
- Second-level list items are preceded by sequential lowercase alphabetic characters followed by a period and 2 spaces ¹.
- Third-level list items are preceded by sequential numbers followed by a close parentheses symbol and 2 spaces ¹.
- Fourth-level list items are preceded by sequential lowercase alphabetic characters followed by a close parentheses symbol and 2 spaces ¹.

Note: Each level beyond the first level indents 4¹ spaces.

The sequence of nesting is repeated for levels of nesting beyond the fourth level. For example, the list items in a fifth level of nesting are preceded by sequential numbers followed by a period.

To show you what this looks like, we'll nest three levels of ordered lists in this markup. We'll use the COMPACT attribute in the third level to conserve space.

```

<!doctype dm system>
<panel name=olists width=42>Nested Ordered Lists
  <area>
    <info width=40>
      <ol>
        <li>Step one (first level)
        <li>Step two (first level)
          <ol>
            <li>Step one (second level)
            <li>Step two (second level)
              <ol compact>
                <li>Step one (third level)
                <li>Step two (third level)
              </ol>
            <li>Step three (second level)
          </ol>
        <li>Step three (first level)
      </ol>
    </info>
  </area>
</panel>

```

Here is how the DTL compiler formats this panel:

¹ The default indentation for a list item is 4 spaces. When the SPACE=YES attribute is coded, the indentation is 3 spaces. See the LI and OL tag descriptions in [Chapter 12, “Tag reference,” on page 179](#) for more information.

```
Nested Ordered Lists

1. Step one (first level)
2. Step two (first level)
   a. Step one (second level)
   b. Step two (second level)
      1) Step one (third level)
      2) Step two (third level)
   c. Step three (second level)
3. Step three (first level)
```

Figure 54. Nested ordered lists

Definition lists

Definition lists allow you to identify a list of words or phrases and their corresponding definitions. A simple definition list formats as a two-column list: the terms you define appear in the left column, and the definitions for the terms appear in the right column. Definition lists are slightly more complex than the previous lists we've discussed, because of the additional tags required to construct them.

The tags used to create a definition list are:

DL

Begins a definition list. The required end tag ends the list.

DT

Identifies the term being defined. The definition term is formatted in the left column of the list. It does not require an end tag.

DD

Identifies the term description. Each definition description is formatted in the right column of the list, immediately opposite or below its associated term. It does not require an end tag.

You can also create headings for definition list columns. There are two additional tags that you can use to do this. They are:

DTHD

Defines a header for the definition term column.

DDHD

Defines a header for the definition description column.

Both of these tags are optional for creating definition lists. We'll show you how you can use them to enhance definition lists later on in this topic.

Here is an example of a definition list:

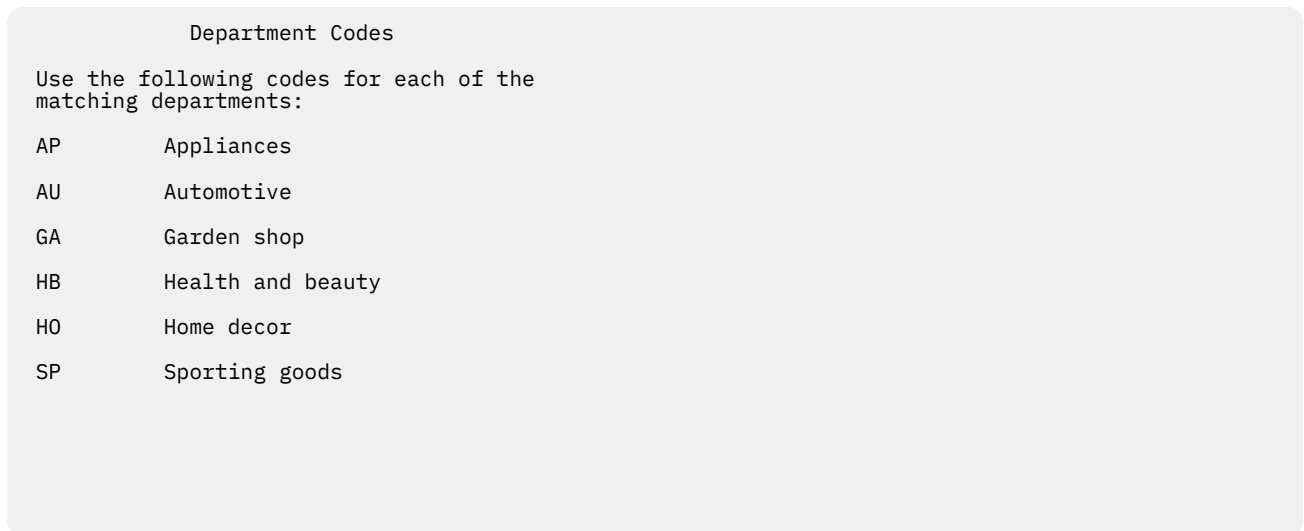


Figure 55. Definition list

Here is the markup:

```
<!doctype dm system>
<panel name=deptcode width=42>Department Codes
  <area>
    <info width=40>
      <p>Use the following codes for each of the
        matching departments:
      <dl>
        <dt>AP
        <dd>Appliances
        <dt>AU
        <dd>Automotive
        <dt>GA
        <dd>Garden shop
        <dt>HB
        <dd>Health and beauty
        <dt>HO
        <dd>Home decor
        <dt>SP
        <dd>Sporting goods
      </dl>
    </info>
  </area>
</panel>
```

A definition list can contain multiple definition terms. The TSIZE attribute of the enclosing DL tag specifies the number of DT tags in a group and their respective widths. For example, TSIZE='10 5' specifies 2 definition term columns with sizes of 10 and 5 characters, respectively.

The DL tag has optional attributes:

TSIZE

specifies the space allocated for the term column or columns

BREAK

indicates if the definition formats on the same line as its associated term

COMPACT

determines if there is a space between each set of terms and descriptions

NOSKIP

suppresses the blank line normally placed before the list

INDENT

controls the indentation from the current left margin

FORMAT

controls the location of the definition term within the TSIZE space

DIVEND

determines whether a divider character is inserted following the DDHD and DD tag text

SPLIT

controls the format of the last DT tag in a multiple DT tag group

Use the TSIZE attribute to specify how much space you want for the definition term column (or columns). The default value is 10 bytes, which also sets the default number of DT tags to **one**. If you want to specify more (or less) space than the default, or multiple DT tags, use the TSIZE attribute to assign the value you want.

Use the BREAK attribute to specify where the definition descriptions are supposed to start (on the same line as the definition terms or on the next line). The BREAK attribute can be specified as NONE, ALL, or FIT.

NONE

The definition descriptions start on the same lines as the definition terms.

ALL

All of the definition descriptions start on the line after the definition terms.

FIT

The definition descriptions are to start on the next line only when the definition term does not fit in the allocated space and spills over into the description area.

The definition list in [Figure 55 on page 115](#) used the default BREAK=NONE. We'll define another list that uses BREAK=ALL.

```
<!doctype dm system>
<panel name=reverb1 width=52>Reverberations
  <area>
    <info width=50>
      <p>Reverberations is one of the most popular brands of electronic
        components available today.
        We stock the following Reverberations components:
        <dl break=all>
          <dt>CD Player Unit
          <dd>With auto-search, auto-off, power door, and
            a two-year warranty.
          <dt>Receiver
          <dd>Digital, 6-speaker hookup, and built-in
equalizer.
          <dt>Tape deck
          <dd>Supports metal and chrome cassettes, and comes with
            a two-year warranty.
        </dl>
      </info>
    </area>
  </panel>
```

[Figure 56 on page 117](#) shows how this definition list formats.

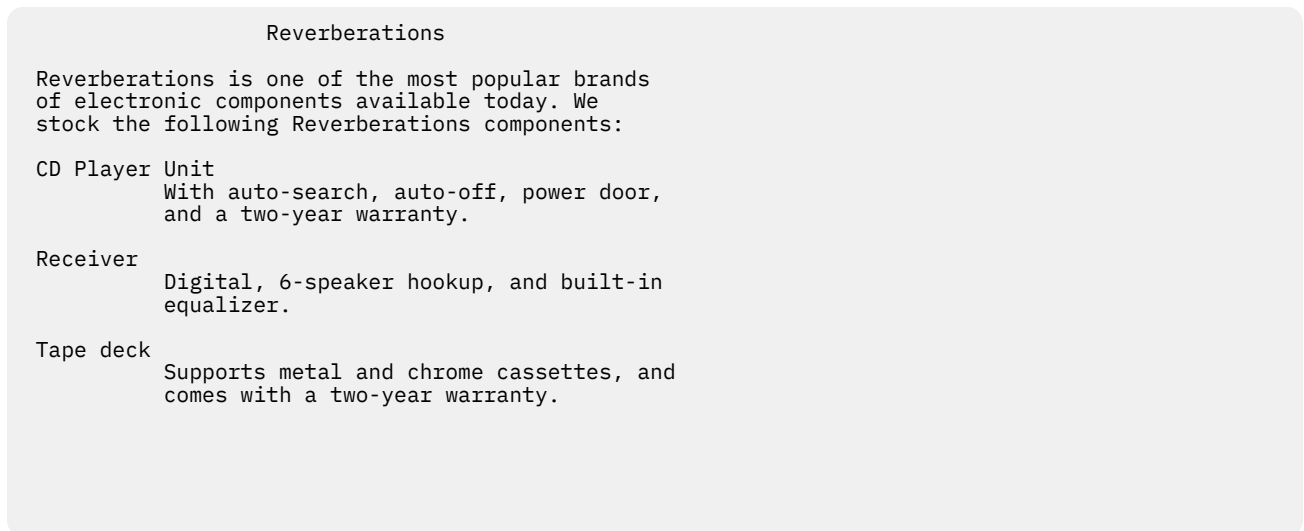


Figure 56. Definition list (BREAK=ALL)

Because the TSIZE and BREAK attributes lend versatility to definition lists, we can rearrange this list practically any way we want. We'll change the BREAK value to FIT, and increase the TSIZE to 13 to show you what we mean. We'll also add headings to the list to show you how they format.

```
<!doctype dm system>
<panel name=reverb2 width=52>Reverberations
  <area>
    <info width=50>
      <p>Reverberations is one of the most popular brands of electronic
        components available today.
        We stock the following Reverberations components:
      <dl tsize=13 break=fit>
        <dt>Component
        <dd>Features
        <dt>CD Player Unit
        <dd>With auto-search, auto-off, power door, and
          a two-year warranty.
        <dt>Receiver
        <dd>Digital, 6-speaker hookup, and built-in equalizer.
        <dt>Tape deck
        <dd>Supports metal and chrome cassettes, and comes with
          a two-year warranty.
      </dl>
    </info>
  </area>
</panel>
```

Here is how the panel looks now:

Reverberations	
Reverberations is one of the most popular brands of electronic components available today. We stock the following Reverberations components:	
Component	Feature
CD Player Unit	With auto-search, auto-off, power door, and a two-year warranty.
Receiver	Digital, 6-speaker hookup, and built-in equalizer.
Tape deck	Supports metal and chrome cassettes, and comes with a two-year warranty.

Figure 57. Definition list (BREAK=FIT)

Parameter lists

Parameter lists are another way of defining terms in a list form. You use a parameter list when the terms you are defining are related to the application in some way (for example, showing codes or parameters).

The tags you use to create parameter lists are the PARML tag and its required end tag, the PT (parameter term) tag, and the PD (parameter description) tag. The parameter list tags work a lot like the definition list tags in defining terms and descriptions, except there are no tags for defining list headings.

The PARML tag also contains the TSIZE, BREAK, COMPACT, INDENT, and SKIP attributes. The TSIZE default value is 10 bytes, as it is for definition lists. However, the BREAK default value for parameter lists is ALL, instead of NONE, as in definition lists. Thus, the parameter descriptions format on the lines following the parameter terms unless you specify otherwise.

A parameter list can contain multiple parameter terms. The TSIZE attribute of the enclosing PARML tag specifies the number of PT tags in a group and their respective widths. For example, TSIZE='10 5' specifies 2 parameter term columns with sizes of 10 and 5 characters, respectively.

Here is the markup for a typical parameter list:

```
<!doctype dm system>
<panel name=ordnum width=52>Order Numbers
  <area>
    <info width=50>
      <p>The order number assigned to each inventory item
        represents specific information about the item.
      <p>Specifically,
      <parml>
        <pt>123
        <pd>The first 3 digits represent the
          department the item is stocked in.
        <pt>456
        <pd>The fourth, fifth, and sixth digits
          represent the item.
        <pt>78
        <pd>The seventh and eighth digits represent
          the lot number of the item.
      </parml>
    </info>
  </area>
</panel>
```

Here is the formatted parameter list:

Order Numbers

The order number assigned to each inventory item represents specific information about the item.

Specifically,

- | | |
|-----|---|
| 123 | The first 3 digits represent the department the item is stocked in. |
| 456 | The fourth, fifth, and sixth digits represent the item. |
| 78 | The seventh and eighth digits represent the lot number of the item. |

Figure 58. Parameter list

Nesting tags within lists

The format of your lists isn't confined to only list items. You can also nest other tags within the list items. For example, if a list item requires an additional paragraph, you can nest a P tag following the list item.

This markup contains an ordered list with a paragraph nested within the second list item.

```
<!doctype dm system>
<panel name=winshop3 width=52>Window Shopper
  <area>
    <info width=50>
      <p>After you have placed your order with Window Shopper, you should...
      <ol>
        <li>Press the Enter key to leave the Order Panel.
        <li>Go to the receiving desk located at the front of the store.
          <p>Don't forget to bring your receipt!
        <li>Give the cashier the pink copy of your receipt.
        <li>Take your purchases home, and enjoy!
      </ol>
    </info>
  </area>
</panel>
```

The paragraph text follows the indentation of the preceding list item, like this:

Window Shopper

After you have placed your order with Window Shopper, you should...

1. Press the Enter key to leave the Order Panel.
2. Go to the receiving desk located at the front of the store.

Don't forget to bring your receipt!
3. Give the cashier the pink copy of your receipt.
4. Take your purchases home, and enjoy!

Figure 59. Nested paragraph within a list

The List Part (LP) tag

If you want to insert unindented text in a list, use the list part (LP) tag. The LP tag is useful for providing information about the list items that follow it.

We added a list part to the panel shown in [Figure 59 on page 119](#):

```
<!doctype dm system>

<panel name=winshop4 width=52>Window Shopper
  <area>
    <info width=50>
      <p>After you have placed your order with Window Shopper, you should...
      <ol>
        <li>Press the Enter key to leave the Order Panel.
        <li>Go to the receiving desk located at the front of the store.
          <p>Don't forget to bring your receipt!
        <li>Give the cashier the pink copy of your receipt.
          <lp>Occasionally, the item you ordered won't be in stock.
            If this occurs, the cashier will be happy to delete
              the item from your order.
        <li>Take your purchases home, and enjoy!
      </ol>
    </info>
  </area>
</panel>
```

Here is the formatted result:

```
Window Shopper

After you have placed your order with Window
Shopper, you should...

1. Press the Enter key to leave the Order Panel.

2. Go to the receiving desk located at the front
   of the store.

   Don't forget to bring your receipt!

3. Give the cashier the pink copy of your
   receipt.

Occasionally, the item you ordered won't be in
stock. If this occurs, the cashier will be happy
to delete the item from your order.

4. Take your purchases home, and enjoy!
```

Figure 60. List part

Nesting lists within lists

In [“Unordered lists” on page 110](#) and [“Ordered lists” on page 112](#) we showed you how to define levels of nested unordered and ordered lists. You can also nest different types of lists within other lists.

Here is an example of an unordered list nested within a definition list:


```

Payment Procedures
  Methods of Payment
    Cash
      Of course, we always accept cash!
    Charge
      Your charge card is welcome here! We
      accept the following charge cards:
        o BigCharge
        o MoneyCard
        o Plastic Express
    Personal Check
      We gladly welcome your personal check,
      with the proper identification.

```

Figure 61. Nested unordered list in a definition list

Here is the markup we used to create the nested lists in [Figure 61 on page 121](#):

```

<!doctype dm system>

<panel name=payment width=52>Payment Procedures
  <area>
    <info width=50>
      <h1>Methods of Payment
      <dl tsize=9 break=all>
        <dt>Cash
          <dd>Of course, we always accept cash!
        <dt>Charge
          <dd>Your charge card is welcome here!
          We accept the following charge cards:
            <ul compact>
              <li>BigCharge
              <li>MoneyCard
              <li>Plastic Express
            </ul>
          <dt>Personal check
            <dd>We gladly welcome your personal check,
            with the proper identification.
        </dl>
      </info>
    </area>
  </panel>

```

You can nest any type of list within another list. Remember, whenever you nest lists, be sure that you end each level with its proper end tag.

Alerting users: notes, warnings, cautions, and attention

DTL provides you with tags that you can use to alert the user to certain text that warrants special attention. Whether you are noting a minor aspect of the application or alerting the user to the risk of possible damage to programs or data, you can alert the user appropriately.

This topic discusses these tags:

- “Notes (NOTE, NT and NETEL tags)” on [page 121](#)
- “Attention and warning (ATTENTION and WARNING tags)” on [page 124](#)
- “Caution (CAUTION tag)” on [page 125](#)

Notes (NOTE, NT and NETEL tags)

The NOTE, NETEL, and NT tags format as noted text. Use notes to emphasize minor points.

When you use either the NOTE or NT tag, you get the text "Note:" followed by a space before the text you specify. However, the text is formatted differently depending on which tag you use.

The NOTEL tag is formatted with the first line containing the text "Notes:" followed by a numbered list of note information provided by the tag.

The NOTE tag

If the text is a single paragraph, you use the NOTE tag. The text is formatted as an unindented block, like a paragraph. The NOTE tag does not require a matching end tag.

You use the NOTE tag like this:

```
<!doctype dm system>
<panel name=widget61 width=50>Widgets
  <area>
    <info width=48>
      <p>Choose the type of Widget you want to order by placing
        the cursor on the field and pressing Enter.
      <note>If the Widget you wish to order is not in stock, please
        refer to the "Back Order" panel to place an order.
    </info>
  </area>
</panel>
```

Figure 62 on page 122 shows how it formats.

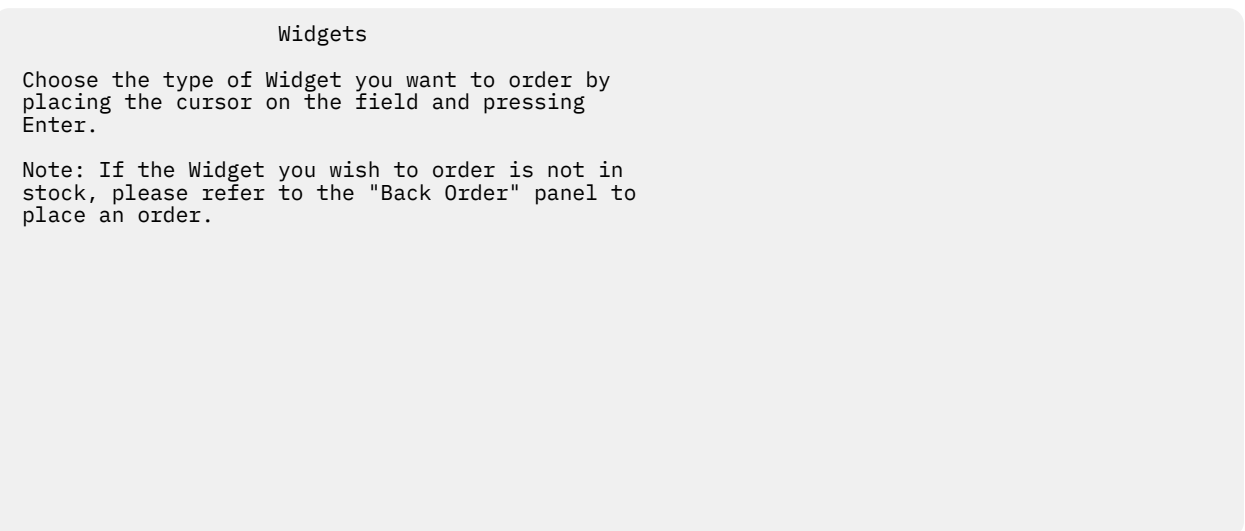


Figure 62. Note (NOTE tag)

The NOTEL tag

If more than one note is used for special attention information, you use the NOTEL tag. Each note is provided by a separate LI tag. The notes are numbered similar to the format described in [“Ordered lists” on page 112](#). You use either the P or LP tag to add any additional paragraphs in the NOTEL definition. Use the required end tag to end the NOTEL definition.

In this example, 2 notes are used, 1 with more than one paragraph. We use the NOTEL tag and its required end tag along with LI tags to define the notes, and a P tag for the additional paragraph.

```
<!doctype dm system>
<panel name=widget63 width=50>Widgets
  <area>
    <info width=48>
      <p>Choose the type of Widget you want to order by placing
        the cursor on the field and pressing Enter.
      <notel>
        <li>If the Widget you wish to order is not in stock, please
          refer to the "Back Order" panel to place an order.
        <li>Back-ordered Widgets usually arrive within three days.
          <p>Please check again in three days.
        </notel>
      <p>If you want to order more than one Widget, specify the quantity
```

```

        and press Enter.
    </info>
</area>
</panel>

```

Notice that the P tag in the note is coded *before* the NOTEL end tag, indicating that the second paragraph belongs in the note.

This is how the panel looks now:

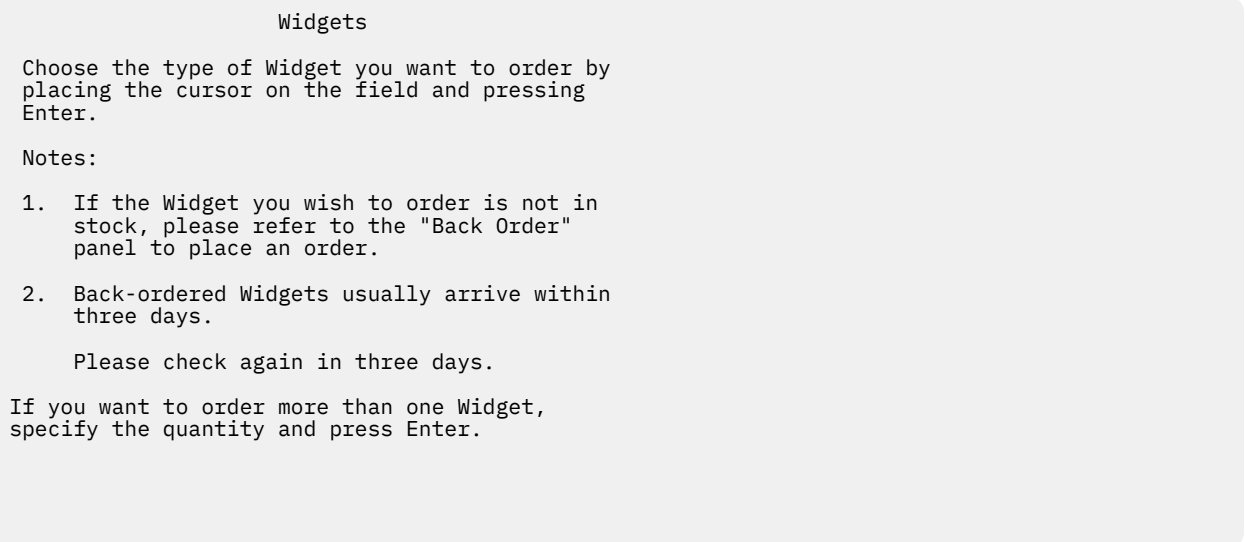


Figure 63. Notel (NOTEL tag)

As you can see, the text of the NOTEL tag is formatted as a list under the "Notes:" heading. The text of the P tag is indented to match the list items.

The NT tag

If the note requires more than one paragraph, you use the NT tag. You use the P tag to add any additional paragraphs in the NT definition. Use the required end tag to end the NT definition.

Another difference between the NOTE and NT tag is that the NT tag indents the note text from the left panel margin.

In this example, the note is longer than one paragraph. We use the NT tag and its required end tag to define the note, and a P tag for each additional paragraph.

```

<!doctype dm system>
<panel name=widget62 width=50>Widgets
  <area>
    <info width=48>
      <p>Choose the type of Widget you want to order by placing
        the cursor on the field and pressing Enter.
      <nt>If the Widget you wish to order is not in stock, please
        refer to the "Back Order" panel to place an order.
      <p>Back-ordered Widgets usually arrive within three days.
      </nt>
      <p>If you want to order more than one Widget, specify the quantity
        and press Enter.
    </info>
  </area>
</panel>

```

Notice that the P tag in the note is coded *before* the NT end tag, indicating that the second paragraph belongs in the note.

This is how the panel looks now:

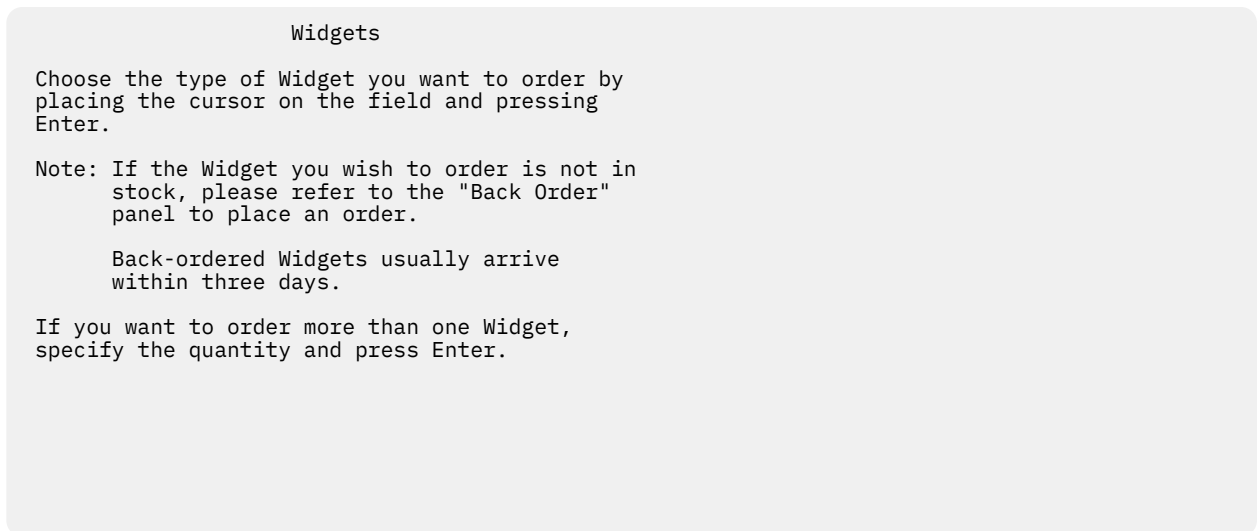


Figure 64. Note (NT tag)

As you can see, the text of the NT tag is indented, as is the text of the P tag coded within the NT tag.

Attention and warning (ATTENTION and WARNING tags)

Attention statements and warning statements alert the user of a possible risk involved with a user action, or of existing error conditions.

You must immediately precede the ATTENTION or WARNING tag with a P (paragraph) tag, LI (list item) tag, or LP (list part) tag. The warning statement formats with the term "Warning:" before the text. The attention statement formats with the term "Attention:" before the text.

The ATTENTION and WARNING tags have no associated attributes and require a matching end tag.

Here is the markup for a warning statement that formats as a paragraph.

```
<!doctype dm system>
<panel name=addfile width=50>Changing a File
  <area>
    <info width=48>
      <p>After you have made the desired changes
        to the file, press Enter to save the changes.
      <p><warning>Pressing Enter saves
        ALL changes made to the file.
        You can cancel this operation by pressing
        the F12=Cancel key.
      </warning>
    </info>
  </area>
</panel>
```

Here is the formatted result:

Changing a File

After you have made the desired changes to the file, press Enter to save the changes.

Warning: Pressing Enter saves ALL changes made to the file. You can cancel this operation by pressing the F12=Cancel key.

Figure 65. Warning

Caution (CAUTION tag)

Caution statements indicate the greatest degree of severity. Like the WARNING tag, the CAUTION tag has a required end tag, and must be preceded by a P (paragraph) tag, LI (list item) tag, or LP (list part) tag. The caution statement formats with the term "CAUTION:" followed by the caution text on the next line.

```
<!doctype dm system>
<panel name=delfile width=50>Deleting a File
  <area>
    <info width=48>
      <p>To delete a file, type the file name in the
        "Delete this file" field and press Enter.
      <p>A message appears asking for verification.
        To continue the delete operation, press Enter.
      <p><b><caution>Verifying the delete operation
        permanently deletes the file from your records.
        There is no chance of recovery.
      </caution>
    </info>
  </area>
</panel>
```

Here is the formatted result:

Deleting a File

To delete a file, type the file name in the "Delete this file" field and press Enter.

A message appears asking for verification. To continue the delete operation, press Enter.

CAUTION:
Verifying the delete operation permanently deletes the file from your records. There is no chance of recovery.

Figure 66. Caution

Emphasizing panel text

You can emphasize text on application panels or on help panels with highlighting by using the HP (highlighted phrase) tag. You can also highlight words or phrases to indicate that additional information is available by using the RP (reference phrase) tags. On a color terminal, the emphasized text displays in a CUA defined color, or whatever color you set with the Color Change Utility.

Highlighting requires the use of 3270 attribute bytes to control the display of highlighted text. The attribute positions before and after the highlighted text display as blank spaces. These attributes might limit the formatting of your highlighted phrase or reference phrase.

Here is an example of highlighting:

```
<HP>To cancel this option</HP>, press the F12 key.
```

Here is the result:

```
To cancel this option, press the F12 key.
```

You can prevent this situation by writing statements that do not require punctuation following an HP or an RP end tag.

Highlighted phrases

The HP (highlighted phrase) tag provides emphasis through highlighting. You can focus the user's attention to particular sections of the panel text by highlighting words, phrases, or entire paragraphs.

The HP tag requires a matching end tag to indicate the end of a highlighted phrase.

This markup example shows the use of the HP tag:

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=date TYPE='char 8'>
<VARCLASS NAME=numcls TYPE='numeric 7'>
<VARCLASS NAME=namecls TYPE='char 25'>
<VARCLASS NAME=char1cls TYPE='char 1'>
<VARCLASS NAME=char7cls TYPE='char 7'>

<VARLIST>
  <VARDCL NAME=whchsrch VARCLASS=char1cls>
  <VARDCL NAME=curdate VARCLASS=date>
  <VARDCL NAME=cardno VARCLASS=numcls>
  <VARDCL NAME=name VARCLASS=namecls>
  <VARDCL NAME=address VARCLASS=namecls>
  <VARDCL NAME=cardsel VARCLASS=char1cls>
  <VARDCL NAME=card VARCLASS=char7cls>
  <VARDCL NAME=north VARCLASS=char1cls>
  <VARDCL NAME=south VARCLASS=char1cls>
  <VARDCL NAME=east VARCLASS=char1cls>
  <VARDCL NAME=west VARCLASS=char1cls>
</VARLIST>

<PANEL NAME=hlitep>Library Card Registration
<AB>
  <ABC>File
    <PDC>Add Entry
      <ACTION RUN=add>
    <PDC>Delete Entry
      <ACTION RUN=delete>
    <PDC>Update Entry
      <ACTION RUN=update>
    <PDC>Exit
      <ACTION RUN=exit>
  <ABC>Search
    <PDC CHECKVAR=whchsrch MATCH=1>Search on name
      <ACTION SETVAR=whchsrch VALUE=1>
      <ACTION RUN=search>
    <PDC CHECKVAR=whchsrch MATCH=2>Search on card number
      <ACTION SETVAR=whchsrch VALUE=2>
      <ACTION RUN=search>
  <ABC>Help
```

```

<PDC>Extended Help...
  <ACTION RUN=exhelp>
<PDC>Keys Help...
  <ACTION RUN=keyshelp>
</AB>

<TOPINST>Type in <HP>patron's name</HP> and <HP>card number</HP>
  (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
  <DTAFLD DATAVAR=cuidate USAGE=out ENTWIDTH=8>Date
  <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
    <DTAFLDD>(A 7-digit number)
  <DTAFLD DATAVAR=name>Name
    <DTAFLDD>(Last, First, M.I.)
  <DTAFLD DATAVAR=address>Address
  </DTACOL>
  <DIVIDER>
  <REGION DIR=horiz>
  <SEFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
  one of the following
    <CHOICE CHECKVAR=card MATCH=new>New
    <CHOICE CHECKVAR=card MATCH=irenew>Renewal
    <CHOICE CHECKVAR=card MATCH=replace>Replacement
  </SEFLD>
  <SEFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
    <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
    <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
    <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
    <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
  </SEFLD>
  </REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

Here is the formatted result:

```

File  Search  Help
-----
                        Library Card Registration

Type in patron's name and card number (if applicable).

Then select an action bar choice.

Date . . . : 08/29/90
Card No. . . : _____ (A 7-digit number)
Name . . . : _____ (Last, First, M.I.)
Address . . : _____

Choose one of the following          Check valid branches
--  1.  New                          _ North Branch
    2.  Renewal                      _ South Branch
    3.  Replacement                  _ East Branch
                                      _ West Branch

Enter a command ===> _____
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel

```

Figure 67. Highlighted phrase example

Reference phrases

The RP (reference phrase) tag allows you to highlight words or phrases on panels to indicate that additional help information is available. When a help panel with reference phrases is displayed, the cursor is positioned in the first reference phrase. When an application panel containing reference phrases is

displayed, the cursor is positioned to the first reference phrase or panel input field, unless the cursor setting has been specified by the application. The reference phrase is an input-capable field so that the user can tab to successive reference phrases on the panel. The reference phrase text is refreshed whenever the panel is displayed.

When the user places the cursor on a reference phrase and requests help, the reference phrase panel or message is displayed. Reference phrase help panels themselves can also contain reference phrases. When a user cancels a reference phrase help, the panel from which the user requested the reference phrase help is displayed again. All other help facilities, such as KEYSHELP and EXHELP, are available from a reference phrase help panel.

The RP tag requires a matching end tag to indicate the end of the reference phrase text.

This markup example shows the use of the RP tag.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=french1 depth=12>Help for Masters Degree in French Literature
<area>
<info>
<p>
The Masters in French Literature (MFL) Program is also available
to students interested in
<rp help=liteve>evening studies.</rp>
<p>
Please consult your program advisers for details before registering for
a class.
</info>
</area>
</help>

<help name=liteve depth=13>Help for Evening Studies
<area>
<info>
<p>
Evening Studies offered by the French Literature
graduate program are available to students
interested in part-time and full-time studies.
All core courses and many electives are offered
in the evening on a rotating basis. Please
consult your program advisers for details before
registering for a class.
</info>
</area>
</help>
```

Here is the formatted result:

```
Help for Masters Degree in French Literature

The Masters in French Literature (MFL) Program
is also available to students interested in
evening studies.

Please consult your program advisers for details
before registering for a class.

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

Figure 68. Reference phrase example

Accordingly, when the user selects the reference phrase **evening studies**, the help panel specified by the HELP attribute (help=liteve) is displayed.

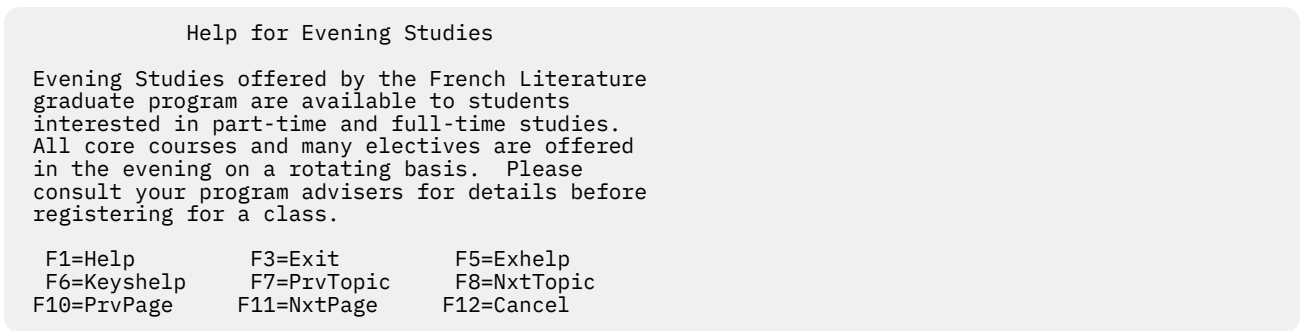


Figure 69. Reference phrase example of help attribute

The *help-panel-name* attribute specifies the name of the help panel to be displayed if the reference phrase is selected.

Using information regions with other panel elements

You can use information regions to complement the other elements of an application panel in many different ways. For example, you can use an information region to provide additional information for fields on an application panel.

Here is a markup example where the information region uses a paragraph and a compact ordered list to tell the user how to interact with the panel fields. [Figure 70 on page 130](#) shows the formatted result.

```
<!doctype dm system>

<VARCLASS NAME=selcls TYPE='char 1'>

<VARLIST>
  <VARDCL NAME=day VARCLASS=selcls>
  <VARDCL NAME=time VARCLASS=selcls>
</VARLIST>

<panel name=appmnt>Make an Appointment
  <area>
    <info width=74>
      <p>To schedule an appointment, you must choose one
        selection from each field.
      <ol compact>
        <li>Choose a day from the first field.
        <li>Choose a time slot from the second field.
        <li>After you have completed both fields, press
          Enter to log your appointment and leave the panel.
        </ol>
      </info>
      <divider type=solid gutter=3>
      <region dir=horiz>
        <region>
          <selfld name=day selwidth=20 pmtwidth=9>Weekdays:
            <choice>Monday
            <choice>Tuesday
            <choice>Wednesday
            <choice>Thursday
            <choice>Friday
          </selfld>
        </region>
        <divider gutter=8>
        <region>
          <selfld name=time selwidth=20 pmtwidth=5>Time:
            <choice>9:00
            <choice>10:00
            <choice>11:00
            <choice>12:00
            <choice>1:00
            <choice>2:00
            <choice>3:00
            <choice>4:00
          </selfld>
        </region>
      </region>
    </area>
```

```
</area>
</panel>
```

Make an Appointment

To schedule an appointment, you must choose one selection from each field.

1. Choose a day from the first field.
2. Choose a time slot from the second field.
3. After you have completed both fields, press Enter to log your appointment and leave the panel.

```
-----
Weekdays:                                Time:
--  1.  Monday                            --  1.  9:00
    2.  Tuesday                           2.  10:00
    3.  Wednesday                         3.  11:00
    4.  Thursday                          4.  12:00
    5.  Friday                            5.  1:00
                                           6.  2:00
                                           7.  3:00
                                           8.  4:00
```

Figure 70. Information region

Help panels

This topic shows you how to use the DTL to define help panels that provide help to users while they are using an ISPF application. We also show you how to link help panels with application panels.

Defining help panels

The HELP tag and its required end tag define a help panel. The HELP start tag indicates the beginning of a help panel definition, and the HELP end tag closes the definition. All of the other tags that compose a help panel are coded between these two tags. You also use the HELP tag to define the help panel title in the same way you code panel title text with the PANEL tag, as tag content.

Here is an example of the HELP tag and its matching end tag:

```
<help name=help01>Help Panel Title
</help>
```

In this example we added the required NAME attribute and value to the HELP start tag. The NAME value you assign must follow the standard naming convention described in [“Rules for variable names” on page 179](#).

The value you assign to NAME is the value that elements such as application panels, fields, and messages use to provide help to the user.

For example, if we define the help we want to provide for an application panel in a help panel with the NAME value *help01*, we would specify that help panel like this in the PANEL definition:

```
<panel name=panel01 width=60 depth=18 help=help01>Application
```

The help panel *help01* would appear when the user requests help for that application panel.

Like the PANEL tag, the HELP tag has WIDTH and DEPTH attributes that define the dimensions of the panel. However, help panels differ from application panels. If the DEPTH attribute is specified on the AREA tag, a single panel is created with a scrollable section to allow the display of longer sections of help

text. Otherwise, the conversion utility generates as many help panels as needed (up to 37) for the help text content you define. This means that you can define text for a help panel that exceeds the defined depth, and, even though the text may not appear in the initial display of the panel, the user can view the text through page scrolling. Examples of both types of help panel scrolling are shown in [“A scrollable panel”](#) on page 132 and [“Multiple panels in sequence”](#) on page 134.

Because ISPF displays all DTL-defined help panels in pop-ups, the WIDTH and DEPTH values you specify must allow for the addition of two lines (depth) and 4 characters (width) for pop-up borders. Therefore, WIDTH=76 and DEPTH=22 are the maximum values that can be used with 80-by-24 display devices. The HELP panel default values are WIDTH=50 and DEPTH=10.

Typically, you would define help panel values of WIDTH=60 and DEPTH=22 or less. The specified depth must include allowance for the panel title line and its separator. A help panel that does not end with a scrollable area also reserves four lines for the function key area.

Defining help panel text

The text you define for help panels cannot be modified by the user; it is for information purposes only. To define this text, use an information region and the tags associated with information regions. The INFO tag and its matching end tag are required in help panel definitions.

You can also use AREA definitions within help panels. Remember to code the entire INFO definition (start and end tag) within the AREA definition, just as you would on an application panel. Here is an example:

```
<help name=help01>Help Panel Title
  <area>
    <info>
  :
    </info>
  </area>
</help>
```

You can use any of the information region tags discussed in this chapter in a help panel. For example, you use the P (paragraph) tag to define a paragraph of text on a help panel the same way you use it to define a paragraph on an application panel.

Here is a help panel markup that has two paragraphs, an unordered list, a figure and figure caption to define the help text. The specification of DEPTH=28 is valid only if the display terminal has 30 or more display lines. [Figure 71 on page 132](#) shows the formatted result.

```
<!doctype dm system>

<help name=olcthlp depth=28 width=50>Help for Online Catalog
<area>
<info>
<p>The Online Catalog provides
you with:
<ul compact>
<li>The book title
<li>Catalog number
<li>Page count
<li>The author
<li>A brief description
</ul>
<p>Here is an example:
<fig>
  The Yellow Subroutine
  365 Pages          1234.56
  John-Paul Georgeringo

  A young band of British programmers embarks on
  a voyage across a perilous "sea" language in
  search of FORTRAN and fame.
<figcaption>Online Catalog Example
</fig>
</info>
</area>
</help>
```

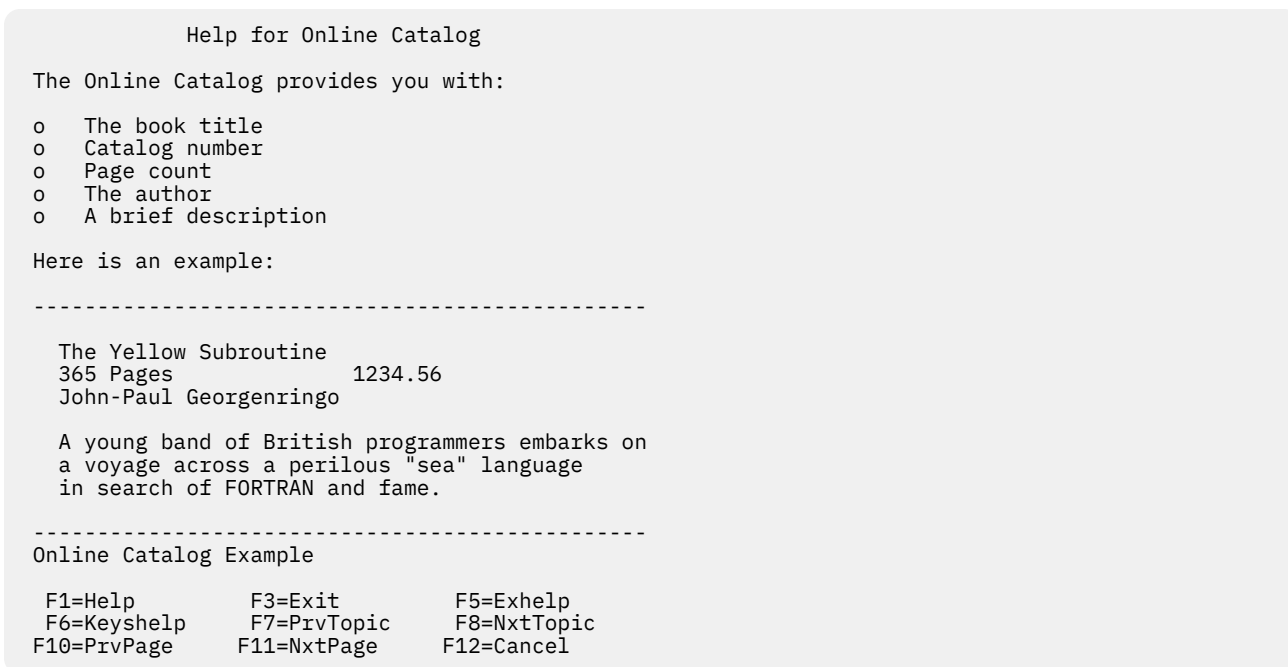


Figure 71. Help panel

In Figure 71 on page 132, all of the text was displayed because the depth we defined for the help panel was large enough to accommodate the text. However, the amount of help you want to provide for your users can vary, and it's not always possible to display all of the help text you define in the initial panel display, especially when you don't, or can't, specify a large DEPTH value for the help panel.

Depending on the use of the AREA tag, the conversion utility generates multiple panels or a single scrollable help panel.

This help panel markup includes an information region that contains a paragraph, a definition list, and two unordered lists nested within the definition list.

A scrollable panel

The addition of the DEPTH attribute on the AREA tag illustrates a scrollable panel.

```
<!DOCTYPE DM SYSTEM>

<help name=helpscr width=46 depth=16>ShelfBrowse for Kids
<area depth=10>
  <info>
    <p>ShelfBrowse can help you
    find any kind of book you are looking for.
    The two main categories for books are:
    <dl tsize=12>
      <dthd>Book
      <ddhd>Description
      <dt>Fiction
      <dd>Fiction books are stories
      that never really happened.
      The writer made them up.
      For example:
      <ul>
        <li>Fairy Tales
        <li>Mysteries
        <li>Science fiction stories
      </ul>
      <dt>Nonfiction
      <dd>Nonfiction books are about
      things that really exist.
      For example:
      <ul>
        <li>History books
        <li>Reference books
        <li>How-to books
```

```

    </ul>
  </dl>
</info>
</area>
</help>

```

When initially displayed, the first part of the scrollable text is visible. For this example, to scroll down, place the cursor on the first or last displayed line of text, and press Enter or the RIGHT (F11) key. Use the LEFT (F10) key to scroll up.

```

HELPSCR      ShelfBrowse for Kids

                                     More:      +

ShelfBrowse can help you find any kind of
book you are looking for. The two main
categories for books are:

Book          Description
Fiction       Fiction books are stories that
               never really happened. The

F1=Help       F3=Exit       F5=Exhelp
F6=Keyshelp   F7=PrvTopic   F8=NxtTopic
F10=PrvPage   F11=NxtPage  F12=Cancel

```

Figure 72. Help panel (example 1 of 4)

After you scroll down, this panel appears:

```

HELPSCR      ShelfBrowse for Kids

                                     More:      - +
               never really happened. The
               writer made them up. For
               example:

               o   Fairy Tales

               o   Mysteries

               o   Science fiction stories

F1=Help       F3=Exit       F5=Exhelp
F6=Keyshelp   F7=PrvTopic   F8=NxtTopic
F10=PrvPage   F11=NxtPage  F12=Cancel

```

Figure 73. Help panel (example 2 of 4)

After you scroll down, this panel appears:

```

HELPSCR      ShelfBrowse for Kids

                                     More:      - +
               o   Science Fiction stories

Nonfiction    Nonfiction books are about
               things that really exist. For
               example:

               o   History books

               o   Reference books

F1=Help       F3=Exit       F5=Exhelp
F6=Keyshelp   F7=PrvTopic   F8=NxtTopic
F10=PrvPage   F11=NxtPage  F12=Cancel

```

Figure 74. Help panel (example 3 of 4)

After you scroll down, this panel appears:

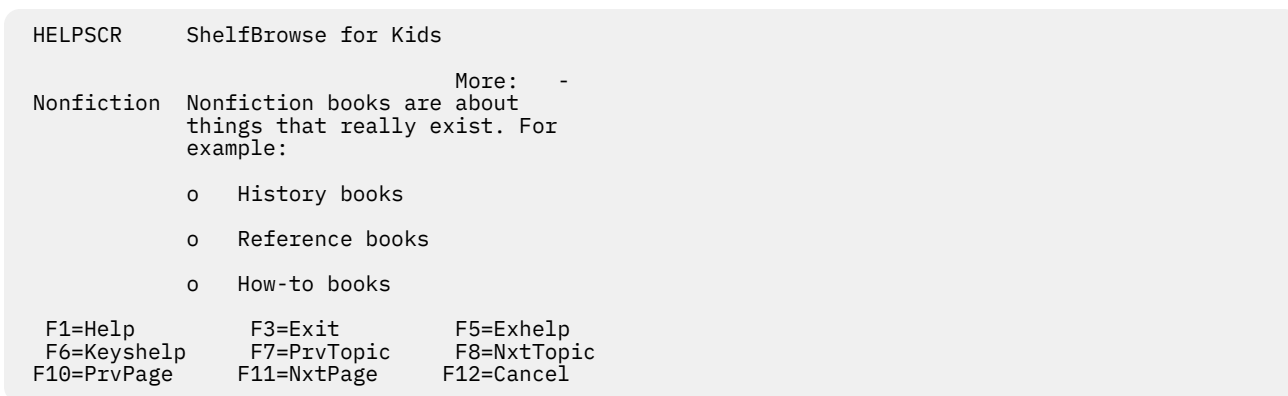


Figure 75. Help panel (example 4 of 4)

There is only one additional line to display, so the scroll has moved the scrollable text up only one line.

Multiple panels in sequence

If no AREA tag is present or the AREA tag does not contain the DEPTH attribute, multiple help panels are generated. ISPF simulates scrolling by displaying the set of multiple help panels in sequence.

If the help panel contains additional text, the conversion utility provides an indicator at the top of the panel to notify the user. If additional text exists, the text **More:** is displayed followed by a + sign. Following scrolling, if additional text stills exists, the indicator displays as “**More: - +**”, indicating scrolling is possible in either direction. If, following scrolling, no more text is available through scrolling forward, but text is available by scrolling backward, the indicator displays as “**More: -**”. Scrolling function keys are defined by tutorial processing.

Here is markup that uses the previous example without a DEPTH attribute on the AREA tag to generate multiple help panels. Because all of the data does not fit in one help panel, the conversion utility created three panels: HELPSB, HELPSBX0, and HELPSBX1. The panels are displayed individually by tutorial processing. Figures [Figure 76 on page 135](#), [Figure 77 on page 135](#), and [Figure 78 on page 135](#) show the formatted results with the function key area displayed in its short form.

```

<!DOCTYPE DM SYSTEM>

<help name=helpsb width=46 depth=16>ShelfBrowse for Kids
<area>
  <info>
    <p>ShelfBrowse can help you
    find any kind of book you are looking for.
    The two main categories for books are:
    <dl tsize=12>
      <dthd>Book
      <ddhd>Description
      <dt>Fiction
      <dd>Fiction books are stories
      that never really happened.
      The writer made them up.
      For example:
      <ul>
        <li>Fairy Tales
        <li>Mysteries
        <li>Science fiction stories
      </ul>
      <dt>Nonfiction
      <dd>Nonfiction books are about
      things that really exist.
      For example:
      <ul>
        <li>History books
        <li>Reference books
        <li>How-to books
      </ul>
    </dl>
  </info>
</area>
</help>
  
```

```
HELPSB      ShelfBrowse for Kids      More:      +
ShelfBrowse can help you find any kind of
book you are looking for. The two main
categories for books are:

Book        Description
Fiction     Fiction books are stories that
            never really happened. The
            writer made them up. For
            example:

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

Figure 76. Help panel (example 1 of 3)

```
HELPSBX0    ShelfBrowse for Kids      More:      - +
o Fairy Tales
o Mysteries
o Science fiction stories

Nonfiction  Nonfiction books are about
            things that really exist. For
            example:

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

Figure 77. Help panel (example 2 of 3)

```
HELPSBX1    ShelfBrowse for Kids      More:      -
o History books
o Reference books
o How-to books

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

Figure 78. Help panel (example 3 of 3)

You can use any of the tags provided for information regions to define the text of the information regions in your help panels.

Chapter 7. Messages

You use messages to communicate information to users; that is, information that you, the application developer, believe they need to know. Typically, this would be information regarding user actions, status, or problems that need correction. Additionally, ISPF issues messages when needed to inform users of situations that ISPF handles.

Dialog Tag Language provides you with a way to define application-provided messages. You use ISPF services to handle the display of messages. When the application calls for a message to be displayed, ISPF places it either in the message area of the application panel or within a pop-up window, known as a *message pop-up*.

Messages are defined according to their purpose and severity. The four types of messages you can define are:

Information

To provide information about a user-requested action.

Warning

To provide information about conditions the user may need to be aware of.

Action

To alert the user to an exception condition that requires a response from the user to correct the situation.

Critical

To alert the user to an exception condition that requires a response from the user to correct the situation. Critical messages are similar to action messages.

This chapter tells you how to define messages for your applications. For a complete description of ISPF message processing, refer to [z/OS ISPF Dialog Developer's Guide and Reference](#).

Defining messages

The messages you define using DTL are stored in *message members*. Each regular message member can contain up to 10 messages. The conversion utility stores the message members in an ISPF message file for the application. The DTL tags you use to define messages and message members are:

MSGMBR

Defines a message member. The MSGMBR tag requires an end tag.

MSG

Defines a message within a message member. The text of the MSG tag is the text that appears as the message. Each message can be up to 512 bytes in length after variable substitution. [“Variable substitution” on page 141](#) describes variable substitution in messages.

You assign an identifier to each message within a message member. The message identifier is composed of two required attribute values: the NAME attribute value of the MSGMBR tag and the SUFFIX attribute value for the MSG tag.

The NAME attribute you specify for the MSGMBR tag can consist of 1-5 uppercase or lowercase alphabetic characters and 2 numeric characters.

The SUFFIX attribute values for each of the MSG tags you code within a MSGMBR definition must consist of either 1 numeric character (0-9) or a numeric character (0-9) and an optional suffix character as defined for ISPF messages. Each of the values must be unique (a message suffix cannot be defined twice in a message member).

```
<!doctype DM system>

<msgmb r name=maia00>
  <msg suffix=0>You cannot type a number in the Name field.
  <msg suffix=1>Please include your first name in the Name field.
```

Defining messages

```
<msg suffix=2>Unrecognized character in Name field. Please correct.  
<msg suffix=3>Unrecognized character in Address field. Please correct.  
<msg suffix=4>You cannot type a number in the City field.  
<msg suffix=5>Unrecognized character in City field. Please correct.  
<msg suffix=6>You cannot type a number in the State field.  
<msg suffix=7>You must type two letters in the State field.  
<msg suffix=8>The Zip code exceeds the maximum length.  
<msg suffix=9>You cannot type an alphabetic character in the Zip field.  
</msgmbr>
```

The value of the MSG SUFFIX attribute, when added to the MSGMBR NAME value, forms the message identifier for that message. For example, the message identifier for this message: "You must type two letters in the State field". is *maia007*. If you specify *maia007* as the MSG value on a CHECKL tag, this message is displayed when ISPF detects the error as a result of input validation.

In addition to SUFFIX, the MSG tag has an optional HELP attribute that allows you to identify a help panel for the message. for information about defining help panels, see [Chapter 6, "Information regions and help panels,"](#) on page 101.

Specifying message severity

The severity you assign a message determines if the alarm is sounded when the message is displayed. You can specify the severity of a message with the MSGTYPE attribute of the MSG tag. ISPF accepts one of four values for the MSGTYPE attribute: INFO (the default value), WARNING, ACTION, or CRITICAL. The value can be supplied as a variable name.

Information Messages

Use the default value INFO when you want to provide the user with feedback about the state of the application.

```
<msgmbr name=orda00>  
  <msg suffix=0>Your order is being processed. Please wait...  
</msgmbr>
```

Warning Messages

Warning messages tell users that a potentially undesirable situation could occur. Users only need to respond to the message to continue, although corrective action may be required later. ISPF displays warning messages with an alarm.

```
<msgmbr name=orda00>  
  <msg suffix=0>Your order is being processed. Please wait...  
  <msg suffix=1 msgtype=warning>Your request for the engraving  
  option is not valid.  
  Please check your request, and correct it if necessary.  
</msgmbr>
```

Action and Critical Messages

Action and critical messages both represent the highest degree of severity. They tell users about exception conditions that require a response. The user must respond with a specific action to continue with the application. ISPF displays these messages with an alarm.

Action messages may appear in a pop-up or in the panel message area. Critical messages always appear in a pop-up.

```
<!doctype dm system>  
  
<msgmbr name=orda00>  
  <msg suffix=0>Your order is being received. Please wait...  
  <msg suffix=1 msgtype=warning>Your request for  
  the engraving option is not valid.
```

```
Please check your request, and correct it if necessary.
<msg suffix=2 msgtype=action>The data you have
entered is incorrect.
Please reenter the data.
</msgmbtr>
```

Short messages

The SMSG attribute enables you to specify a short message. The short message does not conform to CUA architecture, but it is supported for ISPF compatibility.

Assigning messages

Some of the DTL tags have an optional MSG attribute that you use to specify a message-identifier. The message text associated with the message-identifier specified is displayed when conditions you define for the tag are not met by the user.

This list contains the DTL tags that have MSG attributes associated with them, and describes the conditions for each.

CHECKL

Use the MSG attribute of the CHECKL tag to specify a message ISPF displays when the user's input fails the validation check defined for the check list.

CHOFD

Use the MSG attribute of the CHOFD tag to specify a message ISPF displays when the user does not provide input for a required field. You can only assign a message to a data field when the REQUIRED attribute has a value of YES.

DTAFD

Use the MSG attribute of the DTAFD tag to specify a message ISPF displays when the user does not provide input for a required field. You can only assign a message to a data field when the REQUIRED attribute has a value of YES.

LSTCOL

Use the MSG attribute of the LSTCOL tag to specify a message ISPF displays when the user does not provide input for a required entry. You can only assign a message to a list column when the REQUIRED attribute has a value of YES.

SELFLD

Use the MSG attribute of the SELFLD tag to specify a message ISPF displays when the user does not provide input for a required single-choice selection field. You can only assign a message to a selection field when the REQUIRED attribute has a value of YES.

Use the SELMSG attribute of the SELFLD tag to specify a message ISPF displays when the user selects an invalid choice.

Use the SELMSGU attribute of the SELFLD tag to specify a message ISPF displays when the user selects an unavailable choice.

VARCLASS

Use the MSG attribute of the VARCLASS tag to specify a message ISPF displays when the user's input fails the validity check defined by the VARCLASS TYPE attribute.

Note: The message specified by the MSG attribute of a VARCLASS tag is also used if enclosed checks (CHECKL tag) or translations (XLATL tag) do not include the MSG attribute.

XLATL

Use the MSG attribute of the XLATL tag to specify a message that ISPF displays when the user's input fails a specified translation.

ISPF displays a default message for most of the situations listed above if you do not specify the MSG attribute.

To show you how messages are associated with DTL tags, here is an example that defines a data field that requires input from the user. It also defines a message member that contains the warning message ISPF

displays if the user does not provide input for the field. [Figure 79 on page 140](#) shows the displayed panel and message.

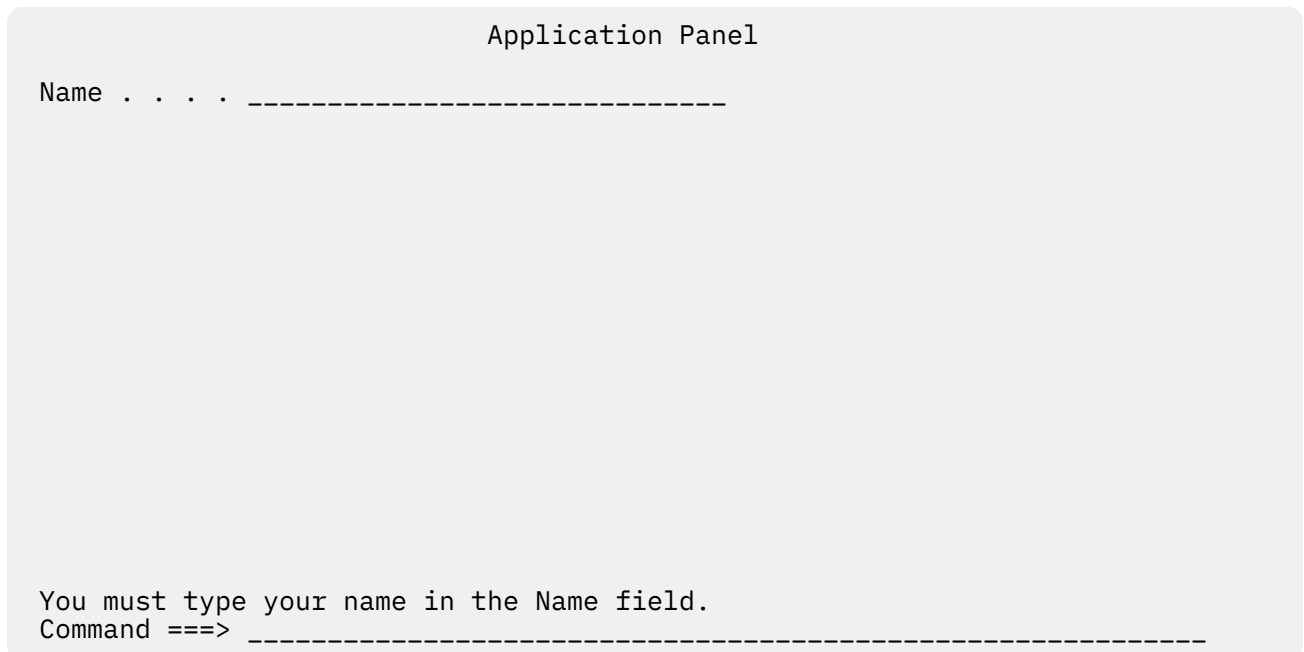
```
<!doctype dm system>

<varclass name=namecls type='char 30'>

<varlist>
  <vardcl name=name varclass=namecls>
</varlist>

<msgmbr name=ordb00>
  <msg suffix=0 msgtype=warning>You must type your name in the Name field.
</msgmbr>

<panel name=msgxmp1>Application Panel
  <dtafld datavar=name pmtwidth=12 required=yes msg=ordb000>Name
<cmdarea>
</panel>
```



Application Panel

Name -----

You must type your name in the Name field.
Command ==> -----

Figure 79. Data field and message

Displaying messages

You can specify how a message is to be displayed, either in the panel message area or a pop-up, using the LOCATION attribute of the MSG tag.

There are five valid values you can assign to LOCATION: AREA (the default), MODAL, MODAL(L), MODELESS, and MODELESS(L). AREA specifies that the message is to appear in the panel message area, unless the text of the message exceeds the length of the message area. If the text of the message exceeds the message area length, ISPF displays the message in a pop-up.

If you want a message that requires a response from the user to appear in a pop-up, specify the MODAL or MODAL(L) value for the LOCATION attribute. This is useful for presenting warning and action messages that have a good deal of text.

If you want a message that does not require a response from the user to appear in a pop-up, specify the MODELESS or MODELESS(L) value for the LOCATION attribute.

For further discussion of these LOCATION values, see [“MSG \(Message\)” on page 352](#).

Here is an example message member markup that contains three messages, each of them with a different LOCATION value. The second and third messages display in pop-up windows.

```
<!doctype dm system>
<msgmbr name=orda01>
  <msg suffix=0>Your order is being received. Please wait...
  <msg suffix=1 msgtype=warning location=modeless>Your request for
the engraving option is not valid.
Please check your request, and correct it if necessary.
  <msg suffix=2 msgtype=action location=modal>The data you have
entered is incorrect.
Please reenter the data.
</msgmbr>
```

Variable substitution

You can specify a variable in the text of a message by using the VARSUB (variable substitution) tag. When the message is displayed, ISPF inserts the current value of the variable into the text of the displayed message.

You code the VARSUB tag within the text of the message where you want the substitution to be made. You use the required VAR attribute of the VARSUB tag to specify the name of a declared variable whose value is substituted in the message text.

Here is an example that uses two variable substitutions in the text of the message "msga001". The first VARSUB specifies the variable *invvar*, which provides an invoice number in the message. The second VARSUB specifies the variable *datevar*, which provides a date in the message.

```
<!doctype dm system>

<varclass name=invoices type='char 10'>
<varclass name=updates type='char 8'>

<varlist>
  <vardcl name=invvar varclass=invoices>
  <vardcl name=datevar varclass=updates>
</varlist>

<msgmbr name=msga00>
  <msg suffix=0>Your request is being processed.
  <msg suffix=1>The invoice number you have requested,
    <varsub var=invvar>, was last updated on
    <varsub var=datevar>.
</msgmbr>
```


Chapter 8. The application command table

In addition to the commands in the ISPF system command table, DTL provides a way to define and store commands that are specific to your application. You can also define commands that override the ISPF system commands. You define and store these commands within a *command table* for your application. These application-specific commands define the responses to commands entered by the user in the command entry field and commands linked to pull-down choices and key mapping lists.

You can define only one command table for an application. ISPF locates the command table using the defined *application-identifier* for the command table.

For a complete description of ISPF command processing and a list of the ISPF system commands, refer to the [z/OS ISPF User's Guide Vol I](#).

Note: You can use the TSO ISPCMDTB command to convert existing command tables to DTL. To use ISPCMDTB, ensure that the command table is in your table concatenation (ISPCMDTB), type TSO ISPCMDTB *applid* (where *applid* is the application id of the command table). This places you in an edit session containing the DTL version of the command table. Use the editor CREATE or REPLACE commands to save the table to your DTL source data set.

Defining the application command table

The tags you use to define an application command table are:

CMDTBL

Begins the definition of an application command table. The required end tag ends the definition.

CMD

Defines a command within an application command table. You code the CMD tags within a CMDTBL definition (between the start and end tags).

CMDACT

Defines the action taken by ISPF when a user enters a command. You code the CMDACT tag following the command (CMD) with which it is associated.

The CMDTBL tag has a required APPLID attribute that you use to define the *application identifier* for the command table. ISPF uses the value you assign with the APPLID attribute to identify the command table. The value you assign to APPLID must be the same as the runtime application identifier specified when the application starts.

The value you assign as an application identifier can have a maximum of 4 characters, and the first character must be A-Z, a-z, @, #, or \$.

Any remaining characters can be either A-Z, a-z, @, #, \$ or 0-9. Lowercase characters are translated to their uppercase equivalents. Additionally, ISPF reserves the application identifier ISPx, where x is any character including the space character. Do not use any of these for an APPLID value.

The conversion utility uses the application identifier as a prefix to the string CMDS to form the name of the command table library. For example, the APPLID value, *demo*, results in the application command table name DEMOCMDS.

Command tables are updated using ISPF table services. Input is obtained from the ISPTLIB DDname allocation and output is written to the ISPTABL DDname allocation. For the description of how to allocate libraries before you start ISPF, and for more information about the use of ISPTLIB and ISPTABL, see the [z/OS ISPF User's Guide Vol I](#).

When a user enters a command in a command-entry field or through a pull-down choice or function key, ISPF searches the command tables defined for the user. The tables are searched in this order (provided that a table is present and defined):

1. Application command table

2. User command tables
3. Site command tables
4. System command table

Note: Up to three user and site command tables can be defined in the ISPF Configuration table. The search order of the site and system command table can be reversed if specified as such in the ISPF Configuration table.

If the command is found in a command table, ISPF performs the action defined in that command table for that command. If the command is not found in any of the command tables, ISPF passes the command to the application program for processing. If any of the command tables are not present, ISPF skips to the next command table in the hierarchy.

Use the CMD tag to define each of the commands within the application command table. The CMD tag has a required NAME attribute that you use to identify the *internal-command-name* for the command. The value you assign as an *internal-command-name* must not exceed 8 characters, and the first character must be alphabetic. Any remaining characters can be either alphabetic or numeric.

Here is a markup example that shows a source file that contains an application command table, a key mapping list, and a panel with an action bar. The command table contains commands that are mapped to the RUN attributes of the ACTION tags associated with the pull-down choices and to the CMD attributes of the KEYI tags.

```
<!doctype dm system>

<cmdtbl applid=brws>
  <cmd name=quit>quit
  <cmdact action=...>
  <cmd name=send>send
  <cmdact action=...>
</cmdtbl>

<keyl name=panlkeys>
  <keyi key=f4 cmd=quit>
  <keyi key=f6 cmd=send>
</keyl>

<panel...>
  <ab>
    <abc>Actions
    <pd>Quit
    <action run=quit>
    <pd>Send
    <action run=send>
    <pd>Exit
    <action run=exit>
  </ab>
  :
</panel>
```

Because ISPF provides the EXIT command, it is not defined within the application command table. When the EXIT command is entered, ISPF finds it in the system command table.

Specifying command actions

You must specify a CMDACT tag for each of the CMD tags you define within an application command table so that ISPF can process these commands. You use the CMDACT tag to define the action taken for the command. Code the CMDACT tag immediately after the CMD tag it is associated with.

The ACTION attribute

The CMDACT tag has a required attribute, ACTION, which you use to specify the ISPF command action. Here is a list of ISPF command actions you can assign. You can also assign some of the ISPF-provided system commands listed in “CMDACT (Command Action)” on page 234, and you can specify command actions dynamically at run time as discussed in [“Specifying command actions dynamically” on page 145](#).

ALIAS

To allow a command to have an alternate name, such as using QUIT as an alias for EXIT.

PASSTHRU

To pass the command to the application. The *internal-command-name* and any command parameters are passed to the dialog in the ISPF ZCMD system variable.

SETVERB

To pass the command to the application. The *internal-command-name* is passed to the dialog in the ZVERB system variable, and the parameters (if any) are passed to the dialog in the ZCMD system variable.

The ALIAS command action provides you with a way to define synonyms for commands. The *internal-command-name* you define for the ALIAS attribute value defines the command to be processed. You must enclose the keyword ALIAS, the *internal-command-name*, and any optional parameters within quotes.

When you define an ALIAS command action, you must code that command's CMD and CMDACT tags before the command the ALIAS represents. ISPF first searches the application-defined commands, and then the ISPF system commands. It must locate the ALIAS definition before the aliased command.

Here is an example where we've added the commands PREV and NEXT to the application command table. We want "PREV" and "NEXT" to be aliases for the ISPF system commands BACKWARD and FORWARD. Because the BACKWARD and FORWARD commands are provided by ISPF, we do not need to define them in the application command table. ISPF locates the aliases before the ISPF system commands they refer to.

Additionally, this example shows the CMDACT for the SEND command set to PASSTHRU, because we want the application program to process the SEND command.

```
<cmdtbl applid=brws>
  <cmd name=quit>quit
    <cmdact action='alias exit'>
  <cmd name=send>send
    <cmdact action=passthru>
  <cmd name=prev>
    <cmdact action='alias backward'>
  <cmd name=next>
    <cmdact action='alias forward'>
</cmdtbl>
```

Specifying command actions dynamically

You can also specify a variable as the value for the ACTION attribute of the CMDACT tag. ISPF substitutes the value of the variable at run time when the command is processed. The runtime value of the variable must be one of the ISPF-supported command actions. You specify the variable using the % notation in the ACTION value.

Here is an example where we specified the variable *scroll* as a command action for the SCROLL command. When the user issues the SCROLL command, ISPF obtains the value of the variable *scroll* from the variable pool to determine the action to be taken. The application can then control the direction of scrolling by setting the variable *scroll* to FORWARD or BACKWARD, or to NOP if no scrolling is possible.

```
<!doctype dm system>

<cmdtbl applid=abcd>
  <cmd name=scroll>scroll
    <cmdact action='%scroll'>
  :
</cmdtbl>
```

Truncating commands

Instead of forcing the user to enter the full command name when typing a command in the command area, you can a shortcut for the user by defining *command truncations* for commands. The user can issue a truncated command in the command area by entering the minimum number of characters you specify for the command.

Defining the application command table

To specify truncation for a command, you code the T (truncation) tag within the *external-command-name* of the command.

For example, to specify "qu" as the minimum command for the QUIT command, you add the T tag to the *external-command-name*, like this:

```
<cmdtbl applid=brws>
  <cmd name=quit>qu<t>it
    <cmdact action='alias exit'>
  :
</cmdtbl>
```

The T tag follows the characters you specify as the minimum command.

With this truncation, the user can issue the QUIT command by typing the command in one of these ways:

```
qu
qui
quit
```

However, you should be careful to avoid adding truncations that duplicate other truncations in the command table. For example, these two truncations define minimum commands ("co") that are identical:

```
<cmdtbl applid=brws>
  <cmd name=comp>co<t>mpare
    <cmdact action=passthru>
  <cmd name=copy>co<t>py
    <cmdact action=passthru>
</cmdtbl>
```

The preceding definition would cause the conversion utility to issue a warning message.

To avoid this type of duplication, place the T tag appropriately in the CMD tag content. The duplication shown in the example can be avoided by coding the truncations in this way:

```
<cmdtbl applid=brws>
  <cmd name=comp><com<t>pare
    <cmdact action=passthru>
  <cmd name=copy><cop<t>y
    <cmdact action=passthru>
</cmdtbl>
```

Chapter 9. Defining key mapping lists

Every application panel has keys that map to valid actions for the panel. You define these key assignments within key mapping lists. The key assignments map a key to a command defined within the application command table or to an ISPF-provided command. You use the KEYLIST attribute of the HELP, HELPDEF, PANEL or PANDEF tags to name the key mapping list to use for a panel. If a keylist is not specified, ISPF provides the default key mapping list used for help panels. ISPF also provides a default key mapping list used when application panels do not refer to an application-defined KEYLIST.

The tags you use to define key mapping lists are:

KEYL

To define a key mapping list. The required end tag ends the key mapping list definition.

KEYI

To define a key assignment and specify the command ISPF processes when the user presses the key, and specify the label for the key if it is displayed in the function key area.

You can code multiple KEYI (key item) tags within a KEYL (key list) definition. You code a KEYI tag for each key that is defined for the key mapping list.

Keylists are updated using ISPF table services. Input is obtained from the ISPTLIB DDname allocation and output is written to the ISPTABL DDname allocation. See the description of how to allocate libraries before starting ISPF in the [z/OS ISPF User's Guide Vol I](#) for more information about the use of ISPTLIB and ISPTABL.

Assigning keys and actions

The KEYL tag starts a key mapping list definition and provides the name of the key mapping list. You specify the key mapping list to be used with the KEYLIST attribute of the HELP, HELPDEF, PANEL, or PANDEF tag.

Each KEYI definition within a key mapping list maps a key assignment with a command. The command can be defined in the application command table, one of the user command tables or site command tables, the system command table, or it can be one of the ISPF-provided commands. The required KEY and CMD attributes of the KEYI tag match the key with the command.

The KEYI definition in this example maps the F2 key on the user's keyboard with the SEARCH command in the application command table.

```
<!doctype dm system>

<cmdtbl applid=abcd>
  <cmd name=search>Search
  <cmdact action=pass thru>
</cmdtbl>

<keyl name=panlkeys>
  <keyi key=f2 cmd=search>Search
</keyl>

<panel name=panl01 keylist=panlkeys>
  :
</panel>
```

When the user presses the F2 key during the display of an application panel that refers to this key mapping list, ISPF processes the SEARCH command.

ISPF default key list

ISPF provides a default key mapping list named ISPKYLST for application panels. If you do not specify a key mapping list to be associated with a panel (using the KEYLIST attribute of the PANEL or PANDEF

tag), ISPF uses the keys defined for ISPKYLST to display in the function key area of the panel when it is displayed. See [“PANEL \(Panel\)” on page 376](#) for information about coding the PANEL tag.

The key mappings for ISPKYLST are:

Key	Command
-----	---------

F1	HELP
-----------	------

F2	SPLIT
-----------	-------

F3	EXIT
-----------	------

F9	SWAP
-----------	------

F12	CANCEL
------------	--------

F13	HELP
------------	------

F14	SPLIT
------------	-------

F15	EXIT
------------	------

F21	SWAP
------------	------

F24	CANCEL
------------	--------

ISPF provides a default key mapping list named ISPHELP for help panels. If you do not specify a key mapping list to be associated with a panel (using the KEYLIST attribute of the HELP or HELPDEF tag), ISPF uses the keys defined for ISPHELP to display in the function key area of the panel when it is displayed. See [“HELP \(Help Panel\)” on page 303](#) for information about coding the HELP tag and [Table 38 on page 308](#) for key mappings of the ISPHELP keylist.

You can override the ISPF default key mapping list by specifying a KEYLIST attribute in the panel definition. All keys that you want to be active, including those for ISPF-provided commands, must be specified in the key mapping list referred to by the KEYLIST attribute.

Displaying keys

While all of the key assignments you define in a key mapping list are valid for the application panels that refer to the list, they only appear in the function key area (FKA) of the panel under these conditions:

- You specify that the key is to be displayed by including FKA=YES in the KEYI tag, and
- The user has not turned off display of the function key area.

You use the FKA attribute of the KEYI tag to specify whether the key is to appear in the panel's function key area. The default FKA value, NO, means that the key does not appear. You must specify FKA=YES for the key to be displayed in the function key area.

When function keys are displayed in the function key area, the key you assign is displayed followed by an equal sign and the FKA text defined for the KEYI tag.

Defining help for key list

The conversion utility supports a keys help panel name on the KEYL tag. This allows a keys help panel to be associated with the key list. You can use the KEYLIST utility to add, change, or delete a keylist help panel name.

Alternatively, the application can provide the help panel name in the ZKEYHELP variable. However, the panel name specified as the keylist help panel either on the KEYL tag or by the KEYLIST utility overrides the panel name supplied by the ZKEYHELP variable.

Here is an example where we want only the F2, F3, and F6 keys to appear in the panel function key area, with F2 mapped to the SEARCH command defined in the application command table, F3 mapped to the EXIT command, and F6 mapped to the KEYSHELP command. We also want F1 to be active to support the ISPF HELP command. No other function keys are to be active for this key mapping list. To obtain this result, we define the function key mapping list like this:

```
<!doctype dm system>

<cmdtbl applid=abcd>
  <cmd name=search>Search
  <cmdact action=passthru>
</cmdtbl>
<keyl name=panlkeys help=panlkeyh>
  <keyi key=f1 cmd=help>
  <keyi key=f2 cmd=search fka=yes>Search
  <keyi key=f3 cmd=exit fka=yes>Exit
  <keyi key=f6 cmd=keyshelp fka=yes>Keyshelp
</keyl>

<panel name=panl01 keylist=panlkeys>
:
</panel>
```

This is how the function key area appears when panel "panl01" is displayed:

```
F2=Search F3=Exit F6=Keyshelp
```

Figure 80. Displayed function key area

Chapter 10. Using the conversion utility

The ISPF conversion utility is a tool that converts Dialog Tag Language (DTL) source files into ISPF panel language source format or executable preprocessed ISPF format. There are two methods of invoking the conversion utility: using the ISPF-supplied invocation panels, or using the conversion utility syntax. In either case, the conversion utility must be run under ISPF control. In this chapter we explain both methods of calling the conversion utility.

Using the ISPF-supplied invocation panels

Type this command on the command line to invoke the conversion utility and display the ISPF invocation panel:

```
ISPD TLC
```

Invocation panel

This panel appears:

Figure 81. Conversion utility invocation panel (ISPCP01)

There are many options on this panel, so you need to scroll forward several times to view them all.

You must specify:

- At least one file containing DTL source
- The panel output file
- The message output file

The language selection defaults to the current ISPF session language. The current selected language is displayed as an information field on the panel.

Select the national language you want by using the Language action bar pull-down to enter a number corresponding to the supported ISPF language. The language is used to provide formatting rules for tag text. See [“Text formatting” on page 13](#) for more information.

Panel input fields

Additional information about the panel input fields follows:

Member Name

If the member name is left blank or entered as a member pattern, a member list is displayed. You can select one or more members to be converted from the member list.

DTL Source data set - n

You can specify up to three additional DTL source libraries on the invocation panel. See [“Additional DTL source files” on page 155](#) for more information.

Panel data set

If no panel output is required, you can specify NULLFILE or DUMMY in place of the panel output file name.

Message data set

if no message output is required, you can specify NULLFILE or DUMMY in place of the message output file name.

Log data set

The log file name is optional. If it is not specified and the messages are to be written to disk, log output is written to the ISPF log file. If the log file is a PDS, a member name must be provided. You may specify an asterisk to tell the conversion utility to use the input GML source file member name as the output log file member name. However, if the input GML member is in the special DTLLST file list format (discussed in [“Conversion utility general information” on page 162](#)) then a separate log file member is created for each source member converted.

List data set

The list file name is optional. If it is not specified, list output is written to the ISPF list file. If the list file is a PDS, a member name must be provided. You may specify an asterisk to tell the conversion utility to use the input DTL source file member name as the output list file member name. However, if the input GML member is in the special DTLLST file list format (discussed in [“Conversion utility general information” on page 162](#)) then a separate list file member is created for each source member converted.

SCRIPT data set

The SCRIPT file name is optional. If a SCRIPT output file is requested, it must be a PDS file. Member names for the SCRIPT file are the same as the panel file.

Tables data set

The Tables file name is optional. If a tables file name is provided, it must be an 80-byte fixed-length PDS file. When a tables file is provided, keylist and command table output is placed in this file.

Keylist Application ID

The optional Keylist Application ID is used when the APPLID attribute is omitted on the HELP, PANEL, KEYL, or CMDTBL tags. This is the equivalent of the ID provided by the KEYAPPL option described in [“Conversion utility syntax” on page 156](#).

Conversion status message interval

When the conversion utility is running in interactive mode and the "Place ISPD TLC Messages in log file" option is selected and the "List Source Convert Messages" option is deselected, a status message containing the name of the current DTL source file member being converted is displayed in the long message area. This message provides a conversion status when you are converting multiple members using the DTLLST format member option. See [“Converting multiple panels” on page 165](#) for more information about the DTLLST syntax. The default message interval value is 1 which displays the message for each member processed. This value can be set to 0 to suppress the message or to a value that refreshes the message after a specified number of members have been converted.

DISPLAY(W) option check interval

When the conversion utility is running in test mode and either the DISPLAY or DISPLAYW option is selected, the converted panel is displayed for visual verification. A panel is displayed periodically after the converted panel has been displayed to enable the user to control the DISPLAY or DISPLAYW function.

The "DISPLAY(W) option check interval" option on the invocation panel controls the frequency of the DISPLAY or DISPLAYW control function panel appearance. The default value is 1, so that the control function panel is displayed after each converted panel display. The control panel enables you to continue using the same display interval, cancel the DISPLAY or DISPLAYW option, or change the control panel display interval.

All files specified must be preallocated.

When the log or list file is a PDS file and the member name is not an asterisk, all of the conversion results are placed in the specified member. If the file name or member name is changed, the pending log or list information is written to the previously specified member and a new log or list is generated beginning with the next conversion. When the conversion utility ends, pending log and list files are written.

The log and list files can be either fixed length or variable length, with or without printer control. When the file is allocated with print control specified, the conversion utility output begins in column 2; column 1 is blank. When print control is not specified, the conversion utility output begins in column 1.

Panel options

The conversion utility options are displayed either as a multi-choice selection list by scrolling the invocation panel, or in a series of multi-choice selection list panels with related options, through the Options pull-down on the action bar.

You select the options by entering a "/" in front of the option description. If you want to deselect an option, you must leave the selection choice field blank. These options are initially set to the default values described in ["Conversion utility syntax" on page 156](#). This table shows how the options and their valid values are equivalent to conversion utility syntax. Note that *b* represents a blank.

<i>Table 1. The equivalence of an option and valid value to conversion utility syntax</i>	
Options	Valid values
Replace Panel/Message/SCRIPT/Keylist/ Command Members	/ is equivalent to REPLACE, the default. <i>b</i> is equivalent to NOREPLACE.
Preprocess Panel Output	/ is equivalent to PREP, the default. <i>b</i> is equivalent to NOPREP.
Place ISPD TLC Messages in log file	<i>b</i> is equivalent to SCREEN, the default. / is equivalent to DISK.
Suppress Messages (ISPF extensions)	<i>b</i> is equivalent to NOMSGSUPP, the default. / is equivalent to MSGSUPP.
Suppress Messages (CUA exceptions)	<i>b</i> is equivalent to NOCUASUPP, the default. / is equivalent to CUASUPP.
Use CUA Panel Attributes	/ is equivalent to CUAATTR, the default. <i>b</i> is equivalent to NOCUAATTR.

<i>Table 1. The equivalence of an option and valid value to conversion utility syntax (continued)</i>	
Options	Valid values
Generate Statistics on Panel/Message/Script Members	<i>/</i> is equivalent to STATS, the default. <i>b</i> is equivalent to NOSTATS.
Generate List file	<i>b</i> is equivalent to NOLISTING, the default. <i>/</i> is equivalent to LISTING.
Generate List file with substitution	<i>b</i> is equivalent to NOFORMAT, the default. <i>/</i> is equivalent to FORMAT.
Generate SCRIPT file	<i>b</i> is equivalent to NOSCRIPT, the default. <i>/</i> is equivalent to SCRIPT.
Replace Log File Members	<i>/</i> is equivalent to LOGREPL, the default. <i>b</i> is equivalent to NOLOGREPL.
Replace List File Members	<i>/</i> is equivalent to LISTREPL, the default. <i>b</i> is equivalent to NOLISTREPL.
List Source Convert Msgs	<i>b</i> is equivalent to NOLSTVIEW, the default. <i>/</i> is equivalent to LSTVIEW.
Use Expanded Message Format	<i>b</i> is equivalent to NOMSGEXPAND, the default. <i>/</i> is equivalent to MSGEXPAND.
Allow DBCS	<i>b</i> is equivalent to NODBCS, the default. <i>/</i> is equivalent to DBCS.
Specify KANA	<i>/</i> is equivalent to KANA.
Specify NOKANA	<i>/</i> is equivalent to NOKANA.
Create panels with Action bars	<i>/</i> is equivalent to ACTBAR, the default. <i>b</i> is equivalent to NOACTBAR.
Create panels with JSON API client display controls	<i>/</i> is equivalent to GUI, the default. <i>b</i> is equivalent to NOGUI.
Add ISPD TLC version/timestamp to panel	<i>/</i> is equivalent to VERSION, the default. <i>b</i> is equivalent to NOVERSION.
Combine scrollable areas into panel body	<i>b</i> is equivalent to NOMERGESAREA, the default. <i>/</i> is equivalent to MERGESAREA.
Display converted panels (*)	<i>b</i> is equivalent to NODISPLAY, the default. <i>/</i> is equivalent to DISPLAY.

Table 1. The equivalence of an option and valid value to conversion utility syntax (continued)	
Options	Valid values
Display converted panels in a window (*)	<i>b</i> is equivalent to NODISPLAYW, the default. / is equivalent to DISPLAYW.
Bypass data set name validation (after first cycle).	<i>b</i> is equivalent to DSNCHK, the default. / is equivalent to NODSNCHK.
Enable graphic character display	/ is equivalent to GRAPHIC, the default. <i>b</i> is equivalent to NOGRAPHIC.
Use full names in place of Z variables	<i>b</i> is equivalent to ZVARS, the default. / is equivalent to NOZVARS.
Align DBCS prompt text with entry field	<i>b</i> is equivalent to NODBALIGN, the default. / is equivalent to DBALIGN.
Preserve leading blanks when <i>space</i> is not specified	<i>b</i> is equivalent to NOPLEB, the default. / is equivalent to PLEB.
Process multiple line comment blocks	<i>b</i> is equivalent to NOMCOMMENT, the default. / is equivalent to MCOMMENT.
Display additional DTL source data set list	<i>b</i> —second input panel is not displayed. / —second input panel is displayed.
(*) If you specify DISPLAY or DISPLAYW, ISPDTLC must be run in test mode (Option 7) to force display processing to use the current generated panel. An error message is issued if ISPDTLC is not being run in test mode and either option is specified.	

All of the entries from the panel (or panels) are saved in the user's profile.

Additional DTL source files

A second input panel is displayed for entry of up to twelve additional DTL source file data set names when you perform one or more of these actions:

- You place the cursor on the point-and-shoot panel phrase "DTL input files 5-16" and press Enter.
- You select the "Display additional DTL source data set list" option from either the scrollable area of the main panel or the Miscellaneous section of the Options action bar pull-down.
- You enter XDTL on the command line.
- You select Option 7 from the Commands action bar pull-down

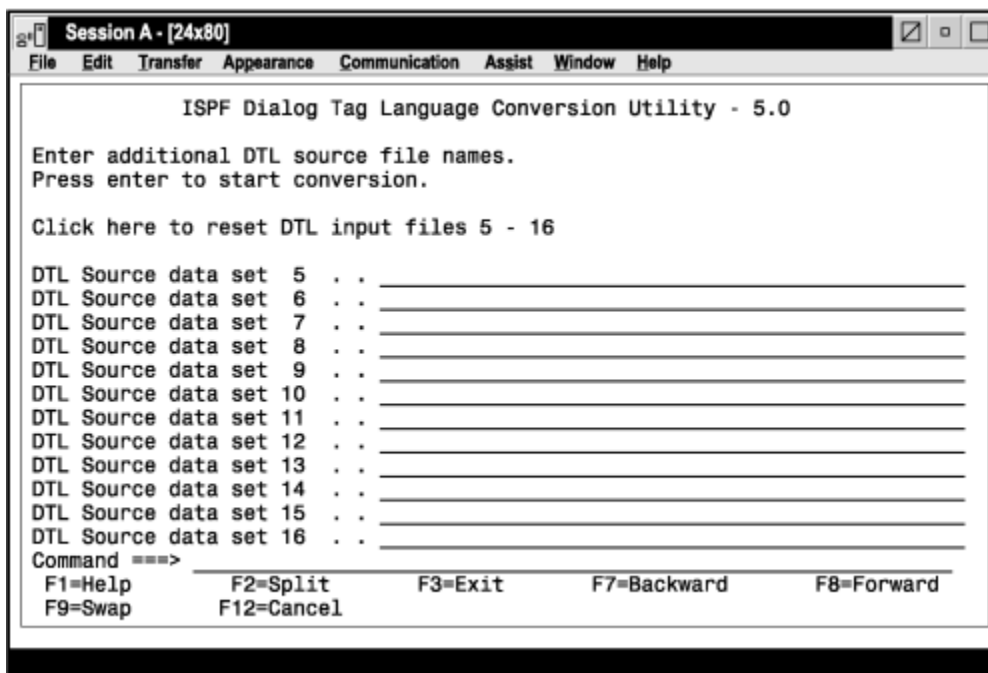


Figure 82. Panel ISPCP04

The entries for DTL source data sets 2-16 can be reset from the first invocation panel by placing the cursor on the point-and-shoot field "Click here to reset DTL input files 2-16" and pressing Enter. The panel redisplay with all entries except "DTL Source data set-1" reset to blanks.

Similarly, DTL source data sets 5-16 can be reset from the additional source files panel by placing the cursor on the point-and-shoot field "Click here to reset DTL input files 5-16" and pressing Enter. The invocation panel redisplay with all entries on the additional source files panel reset to blanks.

Converting multiple DTL source files

When ISPF finishes processing the DTL source you specify, the conversion utility displays the invocation panel again to convert another DTL source file. This cycle continues until you exit or cancel the conversion utility.

Calling help

You can get help for any field on the conversion utility invocation panel by moving the cursor to the field and pressing the F1 key.

Using CUA panel attributes

CUA defines the default colors and emphasis techniques for individual panel elements. The conversion utility generates panel element attributes that ISPF defines. The NOCUAATTR option can be used to create panels with attributes that are not based on CUA defined elements.

See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for more information about panel attributes.

Conversion utility syntax

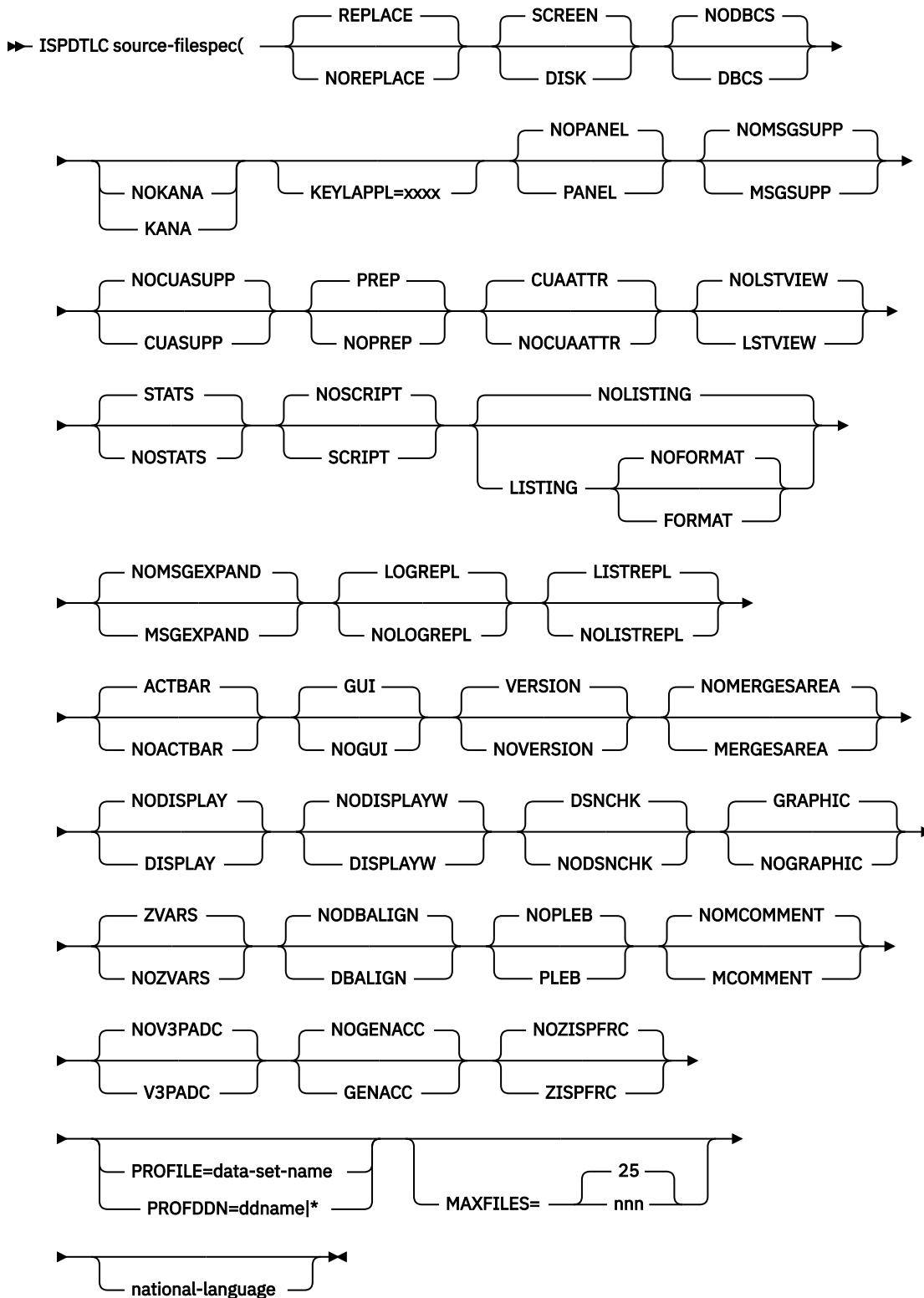
This topic provides an alternative way to invoke the conversion utility. This alternative way provides compatibility with previous ISPD TLC releases, and allows you to issue multiple calls from a user-specified EXEC file. To read the conversion utility syntax, see ["How to read the syntax diagrams"](#) on page xxi for more information.

You can view the allowable syntax and a description of the options by entering this command on the ISPF command line:

```
ISPD TLC ?
```

This command causes the general help panel to be displayed. The first line of information contains the ISPD TLC version, APAR, and PTF numbers.

This diagram shows the conversion utility syntax:



As shown in this diagram, when you specify options, a left parenthesis "(" is required before the first option. If you specify mutually exclusive options such as SCREEN and DISK, the conversion utility issues an error message and stops processing.

The syntax description follows:

source-filespec

Specify the *source-filespec* as a member of a partitioned data set (PDS) that contains the DTL source to be converted to ISPF dialog elements. The first-level qualifier is the "user ID" and the second-level qualifier is "GML" for the input data set name unless the PROFILE option is specified to override the default.

Note: The conversion utility output is stored as commands, keylists, messages and panels. A single source file might result in any or all of these objects. The source file might contain multiple command tables, keylists, message members or panels. The names for the output objects are provided by the CMDTBL, KEYL, MSGMBR, PANEL, and HELP tags. See the descriptions of these tags for additional information.

REPLACE | NOREPLACE

Indicates whether members generated by the conversion utility replace existing members of the same name. If you specify NOREPLACE, the conversion utility issues a warning message for each existing member with the same name, but does not overwrite the existing member. If you specify REPLACE, the conversion utility overwrites any existing member with the same name. REPLACE and NOREPLACE affect keylists, commands, messages, panels, and SCRIPT files.

SCREEN | DISK

Indicates where to send information, warning, and error messages that occur while running the conversion utility. If you specify SCREEN (the default), conversion messages are sent to the display screen. If you specify DISK, conversion messages are sent to the designated log file.

Note: If your messages are not being written to the ISPF log, the specified conversion utility log file must be preallocated. If your messages are being written to the ISPF log, the ISPF Settings option must specify that an ISPF log is to be created.

Running the conversion utility with the DISK option causes additional messages to be appended to the existing sequential ISPD TLC log file or the ISPF log. When using the conversion utility log file, a separator record indicating the date and time of the execution is written to the log file before any messages.

Messages are written to the screen automatically when:

- The conversion utility detects errors during initialization.
- System I/O errors occur.

DBCS | NODBCS

Indicates whether DBCS validation is performed on tag text following the tag suffix ">". Errors found during DBCS validation cause the conversion utility to issue error or warning messages. DBCS shift-out and shift-in characters are considered part of the text, thereby contributing to the length of the text.



Attention: DBCS strings cannot span records. That is, DBCS shift-out and shift-in characters (shift-in characters end the DBCS string) must be on the same record. The conversion utility ends with a severe error for incorrectly formed DBCS strings. If DBCS is specified and no language is specified, the default language is Japanese.

KANA | NOKANA

Indicates whether the KANA keyword is added to the)BODY statement on panels and the message ID line of messages. There is no default. If KANA is specified and no language is specified, the default language is Japanese. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for more information.

KEYLAPPL=xxxx

The KEYLAPPL=xxxx option, where "xxxx" is equal to the 1–4 character application ID, must be specified when the user includes a key list or lists in the DTL source and the APPLID attribute is omitted on the KEYL tag. The application ID is used by the conversion utility to write to the correct key list file.

Note: You cannot use "ISP" as an application ID, because the conversion utility is running as an ISP application.

See *z/OS ISPF Dialog Developer's Guide and Reference* for restrictions on updating key lists.

PANEL | NOPANEL

The PANEL keyword forces the conversion utility to display the invocation panel even if a *source-filespec* has been entered. The PANEL keyword is disregarded when the conversion utility is running in a batch job.

MSGSUPP | NOMSGSUPP

The MSGSUPP keyword causes the conversion utility to suppress warning messages concerning panel formatting.

CUASUPP | NOCUASUPP

The CUASUPP keyword causes the conversion utility to suppress warning messages concerning CUA Architecture non-compliance.

PREP | NOPREP

The NOPREP keyword causes the preprocessing of the output panel to be bypassed. Panel output is stored in ISPF panel format.

CUAATTR | NOCUAATTR

The NOCUAATTR option can be used to create panels with attributes that are not based on CUA defined elements. CUAATTR causes panels to be created using CUA attribute types as defined in the *z/OS ISPF Dialog Developer's Guide and Reference*.

Note: If you specify NOCUAATTR, the conversion utility issues a message and changes the default GRAPHIC option to NOGRAPHIC because GRAPHIC support is implemented only for CUA attributes.

LSTVIEW | NOLSTVIEW

The LSTVIEW keyword causes the conversion utility to display the "converting source file" message in line mode when the user has routed the log file messages to DISK. NOLSTVIEW causes the "converting source file" message to be displayed as a long message in full-screen mode. The NOLSTVIEW keyword is disregarded when the conversion utility is running in a batch job; the "converting source file" message is written to file SYSTSPRT.

STATS | NOSTATS

The NOSTATS keyword causes the conversion utility to bypass the creation of member statistics on created panels and messages. STATS and NOSTATS affect messages, panels, and SCRIPT files.

SCRIPT | NOSCRIPT

The SCRIPT keyword causes the conversion utility to create a panel image template as a member of a file allocated to DTLSCR. The panel image template has BookMaster® tags included so that it may be incorporated into documentation files. Input and output fields in the panel image are shown as underscores. Runtime substitution variables are shown as "&varname". Editing is required to supply appropriate information for input and output fields and "&varname" values.

Note: The specified conversion utility SCRIPT output file must be preallocated.

LISTING | NOLISTING

The LISTING keyword causes the conversion utility to create a list file of the processed source GML records. This file is allocated to DTLLIST or if no file name is provided to the conversion utility, the list is added to the standard ISPF list data set. The file you provide can be in either sequential or partitioned format.

Note: If your messages are not being written to the ISPF list file, the specified conversion utility list file must be preallocated.

Indentation of nested tags (to a limit of 30 columns) is provided for readability. The listing is limited to an 80-column format. Tag contents that would extend beyond the right column are flowed to multiple lines.

The formatted listing is unchanged from the original DTL source file except for indentation processing.

FORMAT | NOFORMAT

The FORMAT keyword causes the conversion utility to create a list file of the source GML records after entity substitution is performed. (The FORMAT keyword implies the LISTING keyword.) The number at the left side of the list indicates the file nest level. If the LISTING keyword is specified in combination with the NOFORMAT keyword, all substitution is bypassed and the listing can be used as a formatted input GML file.

MSGEXPAND | NOMSGEXPAND

The MSGEXPAND keyword causes the conversion utility to expand the warning and error messages to include an indicator of the major type of tag in process (PANEL, HELP, KEYL, MSGMBR, CMDTBL) along with the object name.

LOGREPL | NOLOGREPL

Indicates whether members generated by the conversion utility replace existing log file PDS members of the same name. If you specify NOLOGREPL, the conversion utility issues a warning message for each existing member with the same name but does not overwrite the existing member.

LISTREPL | NOLISTREPL

Indicates whether members generated by the conversion utility replace existing list file PDS members of the same name. If you specify NOLISTREPL, the conversion utility issues a warning message for each existing member with the same name but does not overwrite the existing member.

ACTBAR | NOACTBAR

Indicates whether the ISPF panel statements for action bars are added to the generated panel. If you specify NOACTBAR, the panel sections for)ABC,)ABCINIT, and)ABCPROC and the action bar lines from the panel body are not added to the output panel. (The DTL source for action bar creation is syntax-checked in all cases.)

If a PANEL tag includes the keyword ACTBAR, this option is ignored for that panel.

GUI | NOGUI

The NOGUI keyword causes the panel keywords for mnemonics and check boxes to be removed from the generated panel.

If you specify MNEMGEN=YES on the AB tag or CHKBX=YES on the SELFLD tag, this option is ignored for the specified tag. This option can be overridden by specifying the TYPE attribute on the PANEL tag.

VERSION | NOVERSION

Indicates whether the ISPD TLC version number, maintenance level, and member creation date and time are added as comments following the)END panel statement and the last message of a message member. In addition, VERSION causes the conversion language (ENGLISH, GERMAN, JAPANESE, and so on), Panel ID, and ISPF version to be added to the)ATTR panel statement line as a comment. If you specify NOVERSION, the comments are not added to the generated panel or message.

Note: If the PREP conversion option has been specified, the comments are not part of the final panel because they are not processed by the ISPPREP utility.

NOMERGESAREA | MERGESAREA

Indicates whether scrollable areas are merged into panel body sections. Merging occurs only when the entire scrollable area can be contained within the panel body, allowing for the function key area.

This option can be overridden by specifying the MERGESAREA attribute on the HELP or PANEL tag.

NODISPLAY | DISPLAY

Indicates whether the converted panel is displayed by the conversion utility immediately after the panel is created. The display is in full-screen format. The DISPLAY keyword is disregarded when the conversion utility is running in a batch job.

Note: If you specify DISPLAY, ISPD TLC must be run in test mode (Option 7) to force display processing to use the current generated panel. An error message is issued if ISPD TLC is not being run in test mode and this option is specified.

DISPLAY causes each converted panel to be displayed until the user enters DISPLAY OFF on the command line of a displayed panel or selects option 2 from the display control panel. The control panel is displayed periodically, according to the interval specified in the "DISPLAY(W) option check interval" field on the invocation panel, or from the Miscellaneous choice on the Options action bar.

NODISPLAYW | DISPLAYW

Indicates whether the converted panel is displayed by the conversion utility immediately after the panel is created. The display is within a window. The DISPLAYW keyword is disregarded when the conversion utility is running in a batch job.

Note: If you specify DISPLAYW, ISPD TLC must be run in test mode (Option 7) to force display processing to use the current generated panel. An error message is issued if ISPD TLC is not being run in test mode and this option is specified.

DISPLAYW causes each converted panel to be displayed until the user enters DISPLAY OFF on the command line of a displayed panel or selects option 2 from the display control panel. The control panel is displayed periodically, according to the interval specified in the "DISPLAY(W) option check interval" field on the invocation panel, or from the Miscellaneous choice on the Options action bar.

DSNCHK | NODSNCHK

Indicates whether file validation is performed on the files specified on the interactive panel after the first conversion cycle has been completed. If you specify NODSNCHK and any specified file is unavailable, the conversion fails when the conversion utility attempts to use the file. The NODSNCHK keyword is disregarded when the conversion utility is running in a batch job.

GRAPHIC | NOGRAPHIC

Indicates, for host display only, whether the action bar separator line and visible horizontal divider lines display as dashed lines or as solid lines. The GRAPHIC option can be overridden by the tag definition that generates the line. See tag attribute descriptions for

- [“AB \(Action Bar\)” on page 179,](#)
- [“AREA \(Area\)” on page 189,](#)
- [“CHDIV \(Choice Divider\)” on page 207,](#)
- [“DA \(Dynamic Area\)” on page 250,](#)
- [“DIVIDER \(Area Divider\)” on page 258,](#)
- [“GA \(Graphic Area\)” on page 295,](#)
- [“GRPHDR \(Group Header\)” on page 300,](#)
- [“LSTFLD \(List Field\)” on page 341, and](#)
- [“LSTGRP \(List Group\)” on page 345](#)

for information about the creation of action bar separator and various types of visible divider lines. If you specify NOGRAPHIC, the action bar separator line and visible divider lines are created as dashed lines.

Note: If you specify NOCUAATTR, the conversion utility issues a message and change the default GRAPHIC option to NOGRAPHIC because GRAPHIC support is implemented only for CUA attributes.

ZVARS | NOZVARS

Indicates whether variable names are formatted as Z variables. If you specify NOZVARS, the variable name is used in panel)BODY or)AREA formatting unless the variable name is longer than the defined field width.

DBALIGN | NODBALIGN

For DBCS language conversions only. Indicates whether fields with PMTLOC=ABOVE are aligned so that the first position of the prompt text is formatted above the first position of the field.

PLEB | NOPLEB

Indicates whether leading blanks in ENTITY text strings are processed. This option is effective only for ENTITY definitions that do not specify the "space" keyword.

MCOMMENT | NOMCOMMENT

Indicates whether multiple line comment blocks, starting with <! -- or <: -- and ending with the first --> found are valid. Comment blocks can include DTL tags.

NOV3PADC | V3PADC

Indicates whether ISPD TLC Version 3 padding is added to global definitions for input fields in the)ATTR panel section. When ISPD TLC is invoked with the V3PADC option, the ISPF keyword PADC('_') is added to input attribute definitions if there is no PAD or PADC attribute specified on the PANEL tag.

GENACC | NOGENACC

Indicates whether DTL formatting of leader dots will be contiguous dots (.....), and whether default formatting of multiple column, single choice selection lists will be in left-to-right, top-to-bottom order.

NOZISPFRC | ZISPFRC

Indicates whether the compiler places the final return code into the ISPF shared pool variable ZISPFRC. This option is not supported by the panel interface.

PROFILE=*data-set-name* | **PROFDDN=***ddname* | **PROFDDN=***

The PROFILE or PROFDDN option provides access to the data set name that contains the conversion utility defined ddnames and associated PDS or sequential file names to be used by the conversion utility during I/O. A sample profile member ISPD TLP is included in the ISPSLIB skeleton library.

The *data-set-name* value must be a fully qualified data set name that specifies either a sequential or a partitioned data set. If the profile entry is part of a partitioned data set, then the member name must be included in the data-set-name specification.

The *ddname* value specifies a ddname allocated to a profile data set.

The "*" value specifies that the ddnames used in the conversion are found as preallocated files. See ["Default data set names"](#) on page 167 for the ddnames used by ISPD TLC.

The profile data set and all data sets defined within the profile must be preallocated.

MAXFILES=25|nnn

The maximum number of nested embed files that the compiler can process. If the message ISPC810E is issued this number can be increased to as high as 999. Doing so does not guarantee a successful compilation and the DTL may still have to be restructured.

national-language

Specifies the language rules to be used for formatting the tag text. Supported language keywords are:

CHINESES	ENGLISH	ITALIAN	PORTUGUE	UPPERENG
CHINESET	FRENCH	JAPANESE	SGERMAN	
DANISH	GERMAN	KOREAN	SPANISH	

Note: ISPF must have been installed to support the language requested by the conversion utility.

Conversion utility general information

The output panel library can be defined as either fixed length or variable length. A fixed-length record library must have a record length of 80, 132, or 160 bytes. Record lengths for variable-length record libraries must be increased by 4. Variable-length libraries defined with a record length other than 84, 136, or 164 are treated as the next smaller standard size. Thus, a variable-length file of 255 bytes is treated as 164, and a variable-length file of 100 bytes is treated as 84.

The NOPREP option directs the conversion utility to write the panels being processed directly to the specified panel output file in the ISPF source format. The overall width for the created panels is limited by the record length of the designated file. Thus, if you have specified a panel library with a fixed length of 80 bytes (or a variable length of 84 bytes), the maximum panel width allowed on the PANEL tag is 80.

The PREP (default) option causes the creation of a temporary panel library to receive the ISPF source panel format file. The temporary library is created with a record length of 160 bytes. Multiple panels created in PREP mode are stored in the temporary library and converted through one call to ISPPREP. When all of the panels are converted, the temporary library is deleted. ISPPREP is called by the conversion utility when you do this:

- Change the name of the output panel library on the ISPD TLC invocation panel and then convert another panel.
- Deselect the Preprocess Panel Output option on the ISPD TLC invocation panel and then convert another panel.
- Change the Generate Statistics on Panel/Message/Script Members option on the ISPD TLC invocation panel and then convert another panel.
- Enter "PREP" on the command line of the ISPD TLC invocation panel or select "PREP" from the Commands action bar pull-down.
- Exit from the conversion utility.

ISPPREP is also called when:

- The number of extents of the temporary library exceeds 5.
- The number of members written to the temporary library exceeds 50.

ISPPREP output for panels longer than 80 bytes can be stored in a panel library with a fixed record length of 80 (or a variable record length of 84). Thus, you can create larger than standard panels in PREP mode while directing the final panel output to a library defined with a standard length. It is the developer's responsibility to ensure that the WIDTH specified on the PANEL tag is appropriate for the device intended to display the panel.

When the log or list files are specified as members of a partitioned data set, and the log or list file member name is specified as an asterisk (*) the member is written before the invocation panel is redisplayed. Otherwise, the log or list file is stored in memory (and added to for additional DTL source conversions) until one of these occurrences:

- The output log or list data set name is changed and another conversion is performed.
- The member name of the log or list file is changed on the invocation panel and another conversion is performed.
- The input DTL source member name is changed when the log or list member name is specified as an asterisk.
- You enter on the command line or select from the Commands action bar pull-down:

SAVELOG

to save the log file

SAVELIST

to save the list file

SAVEALL

to save both log and list files.

- You exit the conversion utility.

When the log file is specified as a partitioned data set, messages issued when the conversion utility ends are directed to the screen.

When the CANCEL command is entered, ISPD TLC displays a cancellation confirmation panel. This panel provides options for disposition of pending log and list file members and for any panels to be processed by ISPPREP. An option is also provided to ignore the CANCEL command and resume ISPD TLC processing.

ISPF Dialog Tag Language Conversion Utility

You have entered the CANCEL command.
Specify termination processing choice.

- _ 1. Do not save log or list files. Do not preprocess pending panels.
 - 2. Save log and list files only.
 - 3. Preprocess pending panels only.
 - 4. Save pending log and list files. Preprocess pending panels.
 - 5. Ignore CANCEL command and resume processing.
- F1=Help F3=End F4=Return F5=Rfind
F6=Rchange F10=Left F11=Right F12=Cretriev

Figure 83. ISPF Dialog Tag Language conversion utility - confirm cancel

The panel appears with option 1 preselected. You may choose another option to save log and list files only, preprocess pending panels only, save log and list files and preprocess pending panels, or resume processing.

When you enter the SUBMIT command, ISPDTLC creates and submits a batch job, using the file names and options specified on the interactive panel. After the job is submitted, the interactive panel is redisplayed. The batch JCL file is built using the ISPF skeleton ISPDTLB.

You can also run ISPDTLC from ISPF options 4 and 5 and from the workplace member list.

Note: From the workplace member list, enter "T" (TSO) in front of the member name to be processed. On the TSO pop-up panel enter

```
"ISPDTLC / (PANEL RETURN"
```

to run a foreground conversion or

```
"ISPDTLC / (PANEL SUBMIT"
```

to submit a batch job.

After you complete the required ISPDTLC invocation panel fields and press Enter, the conversion runs or the job is submitted, and control is returned to the previous option.

Extremely large DTL input source files (source files that contain multiple panel, message, key list, and application command table definitions) might cause memory capacity to be exceeded. Should this occur, split the DTL input source file into multiple files with fewer panels, message members, key lists, or command table definitions or reduce the record length of the input source file.

When ISPDTLC is invoked recursively, that is, more than 1 time from the same ISPF screen, this panel is displayed.

ISPF Dialog Tag Language Conversion Utility

CAUTION:
ISPDTLC has been invoked recursively.
Results are not predictable.

- Enter processing option.
- 1. CANCEL this invocation of ISPDTLC.
 - 2. Proceed with recursive execution.

F1=Help F3=End F4=Return F5=Rfind
F6=Rchange F10=Left F11=Right F12=Cancel

Figure 84. ISPF Dialog Tag Language conversion utility - recursive invoke

The panel appears with option 1 preselected. If you select option 2, the new invocation is processed. Because of possible region size limitations, results are not predictable.

The recursive invocation check is based on the setting of a profile variable that is unique for each active screen. If the recursive check panel appears following an abend, the profile variable was not properly reset when the abend occurred. In this case, select option 2 to allow ISPDTLC to continue.

If the conversion utility is called without a *source-filespec* or if the PANEL option has been specified, the invocation panel is displayed. If other options have been specified, they are merged with the options from the profile before the display. The PROFILE option is disregarded when the invocation panel is displayed.

The *national-language* selection UPPERENG causes the conversion utility to use the uppercase version of the ENGLISH program literals. In addition, the tag text for all tags except <SOURCE> is translated to uppercase during the conversion process.

The *national-language* selection SGERMAN causes the conversion utility to use a special German-to-Swiss German conversion routine to create Swiss German panels from either German or Swiss German DTL source files.

Converting multiple panels

The *source-filespec* can be a special file which is a list of other files to be converted. When you use this option, you can convert multiple panels with a single call to the conversion utility. The format of the file list is:

```
DTLLST source-filespec 1
DTLLST source-filespec 2
:
```

The format of *source-filespec* is the same as any other call to the conversion utility. Duplicate *source-filespec* names within DTLLST are ignored.

ISPF conversion utility messages

During processing, the conversion utility can issue information, warning, and error messages. For unsupported DTL tags and attributes that generate warning messages, the conversion utility either ignores the tag or attribute, or sets attribute values to the conversion utility defaults. If the conversion causes error messages, the conversion utility does not generate the ISPF file (key list, panel, application command table, or message member) that would have been created had the error not occurred.

In the message listing, the line numbers displayed in the messages might not always match the line numbers of the source file that caused the message. This occurs because the conversion utility must sometimes continue to read the source file until it encounters an end tag or a new tag before issuing a message. You should be able to determine which source line created the message by examining the DTL source file.

There are two options required to suppress all noncritical messages.

- The MSGSUPP option is used to suppress messages related to ISPDTLC formatting.
- The CUASUPP option is used to suppress messages related to CUA architecture deviations allowed by ISPDTLC. Examples include nonstandard use of F1/F13, F3/F15, and F12/F24 keylist commands, and the use of the SMSG attribute on the MSG tag to create a short message.

When each DTL source file conversion is completed, the conversion utility issues a message listing the number of warning and error messages generated. If the MSGSUPP or CUASUPP option(s) have been specified, an additional message is issued with the total number of messages suppressed.

When the conversion utility is finished, it issues a message listing the total number of warning and error messages generated. If the MSGSUPP or CUASUPP option(s) have been specified, a message is issued with the total number of messages suppressed. The end of job messages listing the total number of messages are placed in the ISPF log file, if the log file is available; otherwise the overall totals are written to the terminal.

Return codes

Here is a list of return codes that explains the results of the conversion invocation.

- 0**
No warnings, errors, or severe errors
- 1**
All messages were suppressed.
- 4**
CANCEL command ended ISPD TLC
- 8**
Only warnings were found
- 16**
At least one DTL conversion had at least one error
- 20**
At least one DTL conversion ended with a severe error.

For multiple conversions, the highest return code is used.

Conversion results

The results of the conversion are placed in the shared pool.

- The variable ZDTLRC contains the return code.
- The variable ZDTLNWRN contains the number of warning messages.
- The variable ZDTLNERR contains the number of error messages.
- The variable ZDTLNSUP contains the number of suppressed messages.

Conversion utility file names

The conversion utility is provided as a REXX exec on the ISPF product tape.

The ISPD TLC exec can reside in a CLIST data set allocated to SYSPROC or in an EXEC data set allocated to SYSEXEC. For more information about the use of REXX execs on MVS™, refer to the [z/OS TSO/E REXX User's Guide](#).

Additional Requirements:

- All data sets must be allocated before running the conversion utility. In addition, the conversion utility uses ISPF services to produce command table and key list output, which means that a partitioned data set must be allocated to ISPTABL. See the topic on allocating ISPF libraries in the [z/OS ISPF User's Guide Vol I](#) for more information.
- To allow the user to specify the source and destination data sets when using the conversion utility syntax, seven ddnames have been reserved in an allocation profile with associated data set names to be provided by the user.

Note: ISPD TLC profiles from previous releases can be used without change. However, a warning message is issued if the DTLMIN or DTLNLS ddname records are encountered.

- A sample profile member ISPD TLP is included in the ISPSLIB skeleton library. You can modify the data set names for installation or user use. DTL format comments (`<!-- comment text-->` or `<:-- comment text-->`) can be used in the profile data set or member. Do not modify the DDNAMEs in this table (column one). A sample user updated profile member follows:

DDNAME

Data Set

DTLGML

any.GML.input

DTLPAN

your.panel.output

DTLMSG

your.msg.output

DTLLOG (*)

your.log.output

DTLLIST (*)

your.list.output

DTLSCR

your.script.output

DTLTAB

your.table.output

(*) The sequential data set name associated with the DTLLOG and DTLLIST ddnames should have the same characteristics and attributes as the LOG and LIST data sets for ISPF.

DTLGML is the input file to the conversion utility. The last 6 files are for output and are usually the user's own data sets.

- For compatibility with previous ISPD TLC releases, the user can provide the allocation profile name on invocation:

```
ISPD TLC source-filespec (disk PROFILE=User.profile
```

The data set name following the PROFILE keyword must be a fully qualified data set name. When specifying the data set name, do not include quotes.

The profile data-set-name can specify either a sequential or a partitioned data set. If the profile entry is part of a partitioned data set, then the member name must be included in the data-set-name specification. The profile data set and all data sets defined within the profile must be preallocated.

The profile can contain multiple entries for each ddname. For output files, the first valid data set name in the profile is used. For the input GML file, each data set is checked in the order they are found in the profile for the member name specified. The first match by member name is used as the file to be converted.

When the data set associated with either the DTLLOG or DTLLIST ddname in the profile is a PDS, the member name may be a single asterisk. When the asterisk notation is present, the conversion utility uses the same name for the log or list file as the source GML member name.

Default data set names

Here is a table that shows an example of the default data set names used for the conversion utility. USERID is the user's TSO prefix.

<i>Table 2. Default data set names used for conversion utility</i>			
DDNAME	Data Set	Type	Description
DTLGML	userid.GML	PDS	The DTL source PDS where GML members reside.
DTLPAN	userid.PANELS or NULLFILE or DUMMY	PDS	PDS for panel member output. May be specified as NULLFILE or DUMMY for cases where no panel output is required.
DTLMSG	userid.MSGS or NULLFILE or DUMMY	PDS	PDS for message member output. May be specified as NULLFILE or DUMMY for cases where no message output is required.

<i>Table 2. Default data set names used for conversion utility (continued)</i>			
DDNAME	Data Set	Type	Description
DTLLOG	userid.ISPD TLC.LOG userid.LOGLIB(logmem)	SEQ or PDS	Optional. User's log data set for conversion utility messages. If not specified, log messages are written to the standard ISPF log data set. If file is a PDS, member name must be included in the data set name specification.
DTLLIST	userid.ISPD TLC.LIST userid.LISTLIB(listmem)	SEQ or PDS	Optional. User's list data set for conversion utility messages. If not specified, list messages are written to the standard ISPF list data set. If file is a PDS, member name must be included in the data set name specification.
DTLSCR	userid.SCRIPT	PDS	Optional. PDS for panel member documentation output. The DTLSCR data set is required only if the SCRIPT option is specified.
DTLTAB	userid.TABLES	PDS	Optional. PDS for keylist and command table output. If specified, a LIBDEF is performed for ISPTLIB and ISPTABL and the keylist and command table output is written to the data set.

Part 2. Dialog Tag Language (DTL) reference

This part contains these chapters:

- [Chapter 11, “Markup declarations and DTL macro reference,” on page 171](#)

A reference listing for each DTL markup declaration.

- [Chapter 12, “Tag reference,” on page 179](#)

A reference listing for each DTL tag. Each reference listing contains a syntax diagram and attribute definition list, a description, and examples of usage.

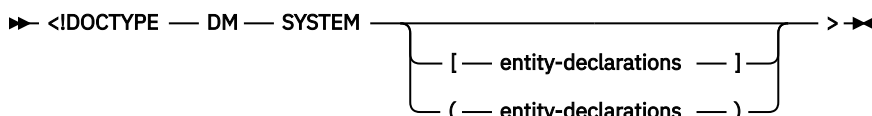
Chapter 11. Markup declarations and DTL macro reference

This chapter provides a detailed look at these items:

- [“Document-type declaration” on page 171](#)
- [“Entity declarations” on page 172](#)
- [“Sample entity definitions” on page 175](#)
- [“DTL macros” on page 177](#)

Document-type declaration

The document-type declaration (DOCTYPE) identifies the source file document type and the rules the source file must follow.



DOCTYPE

Indicates that this is a document-type declaration.

DM

Indicates that this is a DTL source file defining dialog elements.

SYSTEM

Indicates that the rules for the DOCTYPE are contained in an external file.

[(

Indicates the beginning of the declaration subset. Either the left bracket or the open parenthesis can be used to begin the declaration subset. The declaration subset can contain entity declarations and parameter entity references. If the left bracket is coded, it must be the value X'AD'.

entity-declarations

The entity declarations you define for the source file must be coded within the declaration subset. [“Entity declarations” on page 172](#) contains a complete description of entity declarations.

])

Indicates the end of the declaration subset. Either the right bracket or the close parenthesis can be used to end the declaration subset. If the right bracket is coded, it must be the value X'BD'.

Description

A document-type declaration identifies the source file document type and rules the source file must follow.

The DOCTYPE declaration must appear in a DTL source file before any tag markup, although it can be preceded by comments. Files that are embedded in a source file intended for compilation cannot contain a DOCTYPE declaration.

Example

The DOCTYPE statement declares the source file as a DM type file.

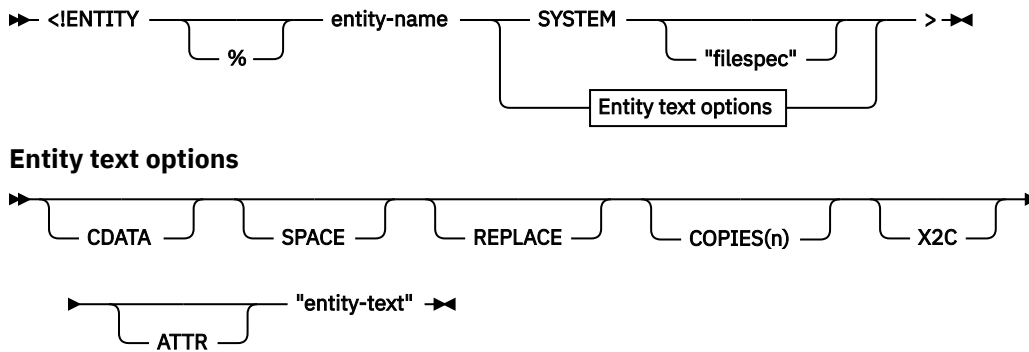
```
<!doctype dm system>
<varclass name=varc type='char 10'>
```

```
<varlist>
  <vardcl name=vard varclass=varc>
</varlist>

<panel name=panel>
:
```

Entity declarations

Entities are symbolic names that are used to insert text into a file.



ENTITY

Indicates this is an entity declaration.

%

Indicates a parameter entity declaration, which must be followed by at least one space.

entity-name

The name of the entity. It must follow these rules:

- Length
 - 1-8 for file embed entity names
 - 1-8 for parameter entity names
 - 1-17 for other entity names
- The first character must be A-Z, a-z, @, #, or \$.
- Remaining characters, if any, can be A-Z, a-z, @, #, \$, or 0-9.

When an entity name is more than 8 bytes in length, one or more of the remaining characters must be an underscore.

- Entity names are case-sensitive.
- The entity name for a parameter entity can be specified as a variable name (that is, **%&varname;**). The resolved name must follow the parameter entity naming rules.

CDATA

Indicates that any delimiter characters in *entity-text* are not interpreted as delimiters. This allows you to define entities with tags in *entity-text* that are not interpreted as tags.

For example the entity-text "`<panel>`" is not interpreted as the PANEL tag if the CDATA keyword is used.

The effect of CDATA is to delay substitution of the variable until all other text manipulation is completed. For example, you should use CDATA to specify an *entity-text* string of blanks as normal text processing removes leading and trailing blanks from text strings.

Note: CDATA cannot be used with parameter entities.

SPACE

Indicates that *entity-text* which spans multiple DTL source file records is formatted like the <p> tag. (Leading and trailing blanks on *entity-text* lines are compressed to a single blank character.) In addition, multiple blanks between words of *entity-text* are compressed to a single blank character.

REPLACE

Indicates that the current *entity-text* is to replace any previous definition of the same entity name.

COPIES(n)

Indicates that the *entity-text* is to be expanded by repeating the provided text *n* times. For example, including COPIES(5) as a keyword with the text specified as "*" causes the entity text to be processed as "*****".

X2C

Indicates that the specified hex format *entity-text* is to be converted to character format. Non-valid hex values are processed as a regular entity character string.

ATTR

Indicates that the specified *entity-text* is a CUA text attribute. Valid values are: CH, CT, DT, ET, FP, NT, PIN, PT, SAC, SI, WASL, and WT. These values are converted to their corresponding attribute byte. Non-valid attribute codes are processed as a regular entity character string.

"entity-text"

The text associated with the entity reference. The text must be enclosed in single or double quotes. The length of the *entity-text* must be less than or equal to 253 bytes.

SYSTEM

Indicates this entity refers to an external file.

"filespec"

The name of the file the entity refers to. The name must be enclosed in single or double quotes. If this is not specified, it defaults to the name of the entity.

The SYSTEM parameter can optionally be followed by the file name for the included file. The file name for MVS is a member name for a file provided on the invocation panel or specified as "DTLGML" entries in the ISPDTLC profile.

Description

Entities are symbolic names that are used to insert text into a file. The text that an entity refers to can be a simple string of characters or it can be the text from an entire file.

An entity reference is used to insert the text associated with the entity. Entities must be declared in the declaration subset of the DOCTYPE declaration before they can be referred to. To refer to the entity in the source file, the entity name is preceded by an ampersand (&) to indicate it is an entity or percent (%) to indicate it is a parameter entity. Both types of entities are ended with a semicolon (;). A blank or the end of the line can be used to end the entity reference instead of the semicolon.

Because entity declarations can only be made within the declaration subset, the parameter entity is the only way to embed a file of entity declarations. Parameter entities are used when an entity reference is needed in the declaration subset. References to parameter entities can only be made in the declaration subset.

References to entities can be made anywhere in the source file *after* the end of the DOCTYPE declaration.

Note: To refer to an entity within a <SOURCE> tag in the source file, the entity name is preceded by a percent (%) instead of an ampersand (&).

Because entity names are case-sensitive, ensure that references to entities are specified correctly.

Conditions

Entities that are declared do not have to be referred to.

Example

This example uses both entities and parameter entities. It embeds the file GLBENT with global entity declarations, and a file with tags and text. It also uses entities and parameter entities that refer to text strings.

The first entity declaration declares the "glbent" parameter entity as an external file.

The file name is defaulted to GLBENT. A parameter entity is used because this file contains entity declarations. Because entity declarations can only be made in the declaration subset, the GLBENT file is embedded with an entity reference within the declaration subset. The entity declarations in GLBENT are for text that is used at the top and bottom of the panel. The "header" entity declaration refers to an external file, and the "footer" is a text string. Both of these entities are referred to in the source file.

The second entity declaration, for "list", is also a parameter entity. This declaration refers to a string, not an external file. The text is the SL tag name, which is referred to in the next two entity declarations. These two declarations, "slist" and "elist", are used as the SL start and end tags. They are defined as entities so the type of list can be changed in one place. To change the list type from a simple list (SL) to an unordered list (UL), change the parameter entity "list" from SL to UL.

This is the content of the source file:

```
<!DOCTYPE DM SYSTEM [
<!ENTITY % glbent SYSTEM -- declaration of global entity file -->
%glbent;<!-- Embeds the global entity file -->
<!ENTITY % list "sl" -- type of list -->
<!ENTITY slist "<%list;>" -- type of list start tag. -- >
<!ENTITY elist "</%list;>" -- type of list end tag. -- >
]>

<panel name=showlist depth=22 width=45>Show Departments
  <area>
    <info width=40>
      &header;

      <p>The floors and departments are shown below:
      &slist;
        <li>First floor
          &slist;
            <li>Toys
            <li>Electronics
          &elist;
        <li>Second floor
          &slist;
            <li>Boys clothes
            <li>Girls clothes
          &elist;
        &elist;
      &footer;
    </info>
  </area>
</panel>
```

This is the content of the embedded file GLBENT:

```
<!ENTITY header SYSTEM "coname">
<!ENTITY footer "<p> We're always glad to help!">
```

This is the content of the embedded file CONAME:

```
<lines>
Frank's Children's World
Barnett, NC
</lines>
```

Figure 85 on page 175 shows the formatted result:

```

Show Departments

Frank's Children's World
Barnett, NC

The floors and departments are shown below:

First floor
    Toys
    Electronics

Second floor
    Boys clothes
    Girls clothes

We're always glad to help!

```

Figure 85. Entities and parameter entities

Sample entity definitions

The tag examples in [Chapter 12, “Tag reference,”](#) on page 179 use entity definitions to create the sample panels. The entities used are called SAMPABC (to define the action bar); SAMPVAR1, SAMPVAR2, and SAMPVAR3 (to provide VARCLASS and VARLIST definitions); and SAMPBODY (to provide a panel body section).

The DTL definitions follow:

SAMPABC:

```

<ABC>File
  <PDC>Add Entry
    <ACTION RUN=add>
  <PDC>Delete Entry
    <ACTION RUN=delete>
  <PDC>Update Entry
    <ACTION RUN=update>
  <PDC>Exit
    <ACTION RUN=exit>
<ABC>Search
  <PDC CHECKVAR=whchsrch MATCH=1>Search on name
    <ACTION SETVAR=whchsrch VALUE=1>
    <ACTION RUN=search>
  <PDC CHECKVAR=whchsrch MATCH=2>Search on card number
    <ACTION SETVAR=whchsrch VALUE=2>
    <ACTION RUN=search>
<ABC>Help
  <PDC>Extended Help...
    <ACTION RUN=exhelp>
  <PDC>Keys Help...
    <ACTION RUN=keyshelp>

```

SAMPVAR1:

```

<VARCLASS NAME=date TYPE='char 8'>
<VARCLASS NAME=numcls TYPE='numeric 7'>
<VARCLASS NAME=namecls TYPE='char 25'>
<VARCLASS NAME=char1cls TYPE='char 1'>
<VARCLASS NAME=char7cls TYPE='char 7'>

<VARLIST>
  <VARDCL NAME=whchsrch VARCLASS=char1cls>
  <VARDCL NAME=curdate VARCLASS=date>
  <VARDCL NAME=cardno VARCLASS=numcls>
  <VARDCL NAME=name VARCLASS=namecls>
  <VARDCL NAME=address VARCLASS=namecls>
  <VARDCL NAME=cardsel VARCLASS=char1cls>
  <VARDCL NAME=card VARCLASS=char7cls>
  <VARDCL NAME=north VARCLASS=char1cls>

```

Sample entity definitions

```
<VARDCL NAME=south VARCLASS=char1cls>
<VARDCL NAME=east VARCLASS=char1cls>
<VARDCL NAME=west VARCLASS=char1cls>
<VARDCL NAME=nth VARCLASS=char1cls>
<VARDCL NAME=sth VARCLASS=char1cls>
<VARDCL NAME=est VARCLASS=char1cls>
<VARDCL NAME=wst VARCLASS=char1cls>
</VARLIST>
```

SAMPVAR2:

```
<VARCLASS NAME=cascls TYPE='char 7'>
<VARCLASS NAME=namecls TYPE='char 25'>
<VARCLASS NAME=addrcls TYPE='char 25'>
<VARCLASS NAME=char1cls TYPE='char 1'>
<VARCLASS NAME=char2cls TYPE='char 2'>

<VARLIST>
  <VARDCL NAME=caseno VARCLASS=cascls>
  <VARDCL NAME=name VARCLASS=namecls>
  <VARDCL NAME=address VARCLASS=addrcls>
  <VARDCL NAME=casesel VARCLASS=char2cls>
  <VARDCL NAME=patin VARCLASS=char1cls>
  <VARDCL NAME=defa VARCLASS=char1cls>
  <VARDCL NAME=cont VARCLASS=char1cls>
  <VARDCL NAME=priv VARCLASS=char1cls>
  <VARDCL NAME=incr VARCLASS=char1cls>
  <VARDCL NAME=disp VARCLASS=char1cls>
  <VARDCL NAME=fraud VARCLASS=char1cls>
</VARLIST>
```

SAMPVAR3:

```
<VARCLASS NAME=namecls TYPE='char 7'>
<VARCLASS NAME=char1cls TYPE='char 1'>
<VARCLASS NAME=char2cls TYPE='char 2'>

<VARLIST>
  <VARDCL NAME=file VARCLASS=namecls>
  <VARDCL NAME=type VARCLASS=char2cls>
  <VARDCL NAME=marg VARCLASS=char2cls>
  <VARDCL NAME=copy VARCLASS=char2cls>
  <VARDCL NAME=duplx VARCLASS=char1cls>
</VARLIST>
```

SAMPBODY:

```
<TOPINST>Type in patron's name and card number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
  <DTAFD DATAVAR=curdate PMTWIDTH=12 ENTWIDTH=8 USAGE=out>Date
  <DTAFD DATAVAR=cardno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=25>Card No
    <DTAFD>>(A 7-digit number)
  <DTAFD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Name
    <DTAFD>>(Last, First, M.I.)
  <DTAFD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
  <DIVIDER>
  <REGION DIR=horiz>
  <SEFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=CARD MATCH=NEW>New
  <CHOICE CHECKVAR=CARD MATCH=RENEW>Renewal
  <CHOICE CHECKVAR=CARD MATCH=REPLACE>Replacement
</SEFLD>
  <SEFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
  <CHOICE NAME=NORTH HELP=NTHHLP CHECKVAR=NTH>North Branch
  <CHOICE NAME=SOUTH HELP=STHHLP CHECKVAR=STH>South Branch
  <CHOICE NAME=EAST HELP=ESTHLP CHECKVAR=EST>East Branch
  <CHOICE NAME=WEST HELP=WSTHLP CHECKVAR=WST>West Branch
</SEFLD>
</REGION>
</AREA>
<CMDAREA>Enter a command
```


DTL macros

A DTL macro is a DTL source member found in the concatenated DTL source libraries allocated as input to ISPD TLC. The macro member can be empty or it can contain any DTL tag coding, including DTL comments.

The macro member is embedded into the current DTL source file when the macro name is encountered during conversion. The file embed process is similar to Entity file embed.

DTL macro tag syntax is similar to regular DTL tag syntax. You invoke a macro by specifying the macro member name using a special DTL tag open delimiter, like this:

```
<?macmemb>
```

The macro member name must conform to the DTL standard member name rules.

When ISPD TLC finds the <? open delimiter, a file embed is performed on the specified member name. The reserved member name **dummy** can be specified to create a *no operation* (NOP) situation during the embed cycle. If the specified member has no records, conversion continues with the next DTL source record.

The content of the macro member can be any valid DTL source input. The member can contain multiple tags and comment records just like any other DTL source file. DTL source file variables, sometimes referred to as entities, are substituted using standard entity processing.

An advantage to using the macro syntax instead of an entity file embed is that you need not code the entity declaration for the file to be embedded. ISPD TLC resolves the required information for you.

Another advantage is that you can specify entity values as part of the macro coding syntax, and bypass the coding of other entity declarations. For example, if the entity variables *subst_var_1*, *subst_var_2*, and *subst_var_3* were coded within the macro using standard DTL syntax (that is, &subst_var_1;, &subst_var_2;, and &subst_var_3;), you could invoke the macro and specify the substitution values like this:

```
<?macmemb subst_var_1=subvalue1 subst_var_2=subvalue2
subst_var_3=subvalue3>
```

ISPD TLC automatically defines the entities with the specified values. The values are stored using entity REPLACE processing, so that if a previous definition exists, it is overwritten. The new definition remains in effect until replaced, and can be referenced by any other part of the DTL source file. Entities defined in this way must not be referenced by the first line of a macro.

Macro tags placed within the document declaration function use the same rules as macro tags found after the document declaration. For example, you can use the macro syntax in place of parameter entities. The parameter entity (really a file of other entity definitions) member **pentmem** can be embedded easily by coding

```
<?pentmem>
```

within the document declaration. This syntax replaces the more complicated parameter entity coding of

```
<:ENTITY % pentmem; system>
%pentmem;
```

In another example, the macro syntax can be used in place of entity tags.

```
<?dummy panel_title='ISPF macro example'
panel_width=60
panel_depth=18>
```

This syntax replaces multiple entity definitions:

```
<:ENTITY panel_title 'ISPF macro example'>
<:ENTITY panel_width '60'>
<:ENTITY panel_depth '18'>
```

In the previous example the macro name **dummy** is used to bypass the file embed and enable the attribute resolution process to establish the entity values.

The macro name **dummy** can also be used within a macro definition to provide default values for macro entity variables. Example:

```
<!-- macro/include ISPZ@EX1 to format a 2 column example -->
<?dummy ?col1_indent=0 ?col1_width=30>

<region dir=horiz>
  region dir=vert width=&col1_width; indent=&col1_indent;>
    <pnlinst>&col1_text;
  </region>

  <region dir=vert>
    <pnlinst>&col2_text;
  </region>
</region>
```

In this example the entity variables *col1_width* and *col1_indent* have default values specified by the **dummy** tag. The special syntax '?variable=value' provides the default values.

The default values for *col1_width* and *col1_indent* are used if you invoke the ISPZ@EX1 macro like this:

```
<?ispz@ex1 col1_text='text left' col2_text='text right'>
```

The default values for *col1_width* and *col1_indent* are overridden by those specified if you invoke ISPZ@EX1 macro like this:

```
<?ispz@ex1 col1_width=40 col1_indent=4
  col1_text='text left' col2_text='text right'
```

Chapter 12. Tag reference

This chapter contains an alphabetical reference of the Dialog Tag Language (DTL) tags.

Each reference listing contains:

- A diagram of the valid syntax for the tag
- A list describing the tag attributes
- A description of the tag
- Conditions of usage
- A table of the tags that can be nested within the tag
- An example of how the tag is used within DTL source markup.

Rules for variable names

Variable names supplied as attribute values on DTL tags must have these characteristics:

- 1-8 characters in length
- The first character must be A-Z, a-z, @, #, or \$.
- Remaining characters, if any, can be A-Z, a-z, @, #, \$, or 0-9.

Lowercase characters are translated to their uppercase equivalents

Names composed of valid characters that are longer than 8 bytes are truncated to 8 bytes. Names that are not valid are set to blank.

Rules for “%variable” names

When a "%varname" notation is found as an attribute value, the "%varname" entry must have these characteristics:

- 2-9 characters in length
- The first character is a “%”.
- The second character must be A-Z, a-z, @, #, or \$.
- Remaining characters, if any, can be A-Z, a-z, @, #, \$, or 0-9.

Lowercase characters are translated to their uppercase equivalents

The first position of a valid name is replaced by an “&”.

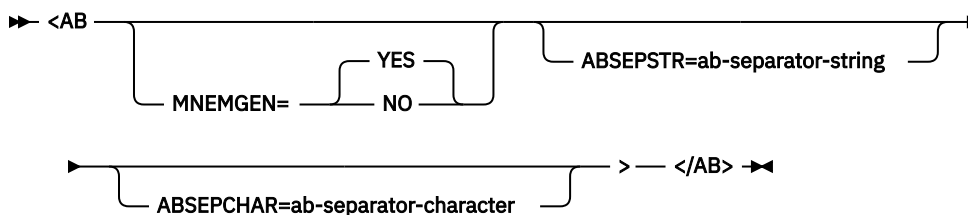
Names composed of valid characters that are longer than 9 bytes are truncated to 9 bytes. Names that are not valid are set to blank.

It is the responsibility of the application to provide a valid value in the variable before the panel is displayed.

AB (Action Bar)

The AB tag defines an action bar on an application panel.

Syntax



Parameters

MNEGEN=YES | NO

Note: When the conversion utility is operating in DBCS mode, the default value for MNEGEN is NO.

This attribute controls the automatic generation of mnemonic characters for the entire action bar. When MNEGEN=NO, mnemonic characters are determined only by the use of the M tag within action bar choice description text. See [“Mnemonic choice selection” on page 36](#) and [“M \(Mnemonic\)” on page 351](#) for additional information.

When MNEGEN=YES, the NOGUI invocation option is ignored and mnemonics are generated automatically.

ABSEPSTR=ab-separator-string

This attribute provides a string of data to be overlaid at the right end of the action bar separator line.

ABSEPCHAR=ab-separator-character

This attribute provides a replacement character for the action bar separator line. When the GRAPHIC invocation option has been specified, the action bar separator defaults to a solid line for host display. You can use the ABSEPCHAR attribute to provide a different character such as a dash.

Comments

The AB tag defines an action bar on an application panel. The action bar appears on the panel above the panel title line. The action bar provides a way for users to view all actions that apply to the panel it is coded within.

The conversion utility inserts a line between the action bar and the panel title line. The GRAPHIC invocation option creates a solid line. NOGRAPHIC creates a dashed line. If required by the length or number of action bar choices, the conversion utility formats multiple lines for the action bar.

ABC tags, which you code within an AB definition, define application panel choices for the action bar. PDC tags, which you code within ABC tag definitions, define the action bar pull-down choices.

To define an action bar and its associated pull-downs, you code the AB tag (and other tags that define the action bar choices and pull-downs) within a PANEL definition.

Restrictions

- The AB tag requires an end tag.
- You must code the AB tag within a PANEL definition. Each application panel can include only one action bar. See [“PANEL \(Panel\)” on page 376](#) for a complete description of this tag.
- You must code at least one ABC tag within an action bar definition.
- To conform to CUA rules, you must include a help action bar choice.

Processing

Table 3. The tag you can code within an AB definition			
Tag	Reference	Usage	Required
ABC	“ABC (Action Bar Choice)” on page 182	Multiple	Yes

Examples

Here is markup that contains the action bar markup for the application panel illustrated in [Figure 86 on page 182](#).

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampbody system>)>
&sampvar1;

<PANEL NAME=ab KEYLIST=keylxml>Library Card Registration
<AB>
<ABC>File
  <PDC>Add Entry
    <ACTION RUN=add>
  <PDC>Delete Entry
    <ACTION RUN=delete>
  <PDC>Update Entry
    <ACTION RUN=update>
  <PDC>Exit
    <ACTION RUN=exit>
<ABC>Search
  <PDC CHECKVAR=whchsrich MATCH=1>Search on name
    <ACTION SETVAR=whchsrich VALUE=1>
    <ACTION RUN=search>
  <PDC CHECKVAR=whchsrich MATCH=2>Search on card number
    <ACTION SETVAR=whchsrich VALUE=2>
    <ACTION RUN=search>
<ABC>Help
  <PDC>Extended Help...
    <ACTION RUN=exhelp>
  <PDC>Keys Help...
    <ACTION RUN=keyshelp>
</AB>
&sampbody;
</PANEL>
```

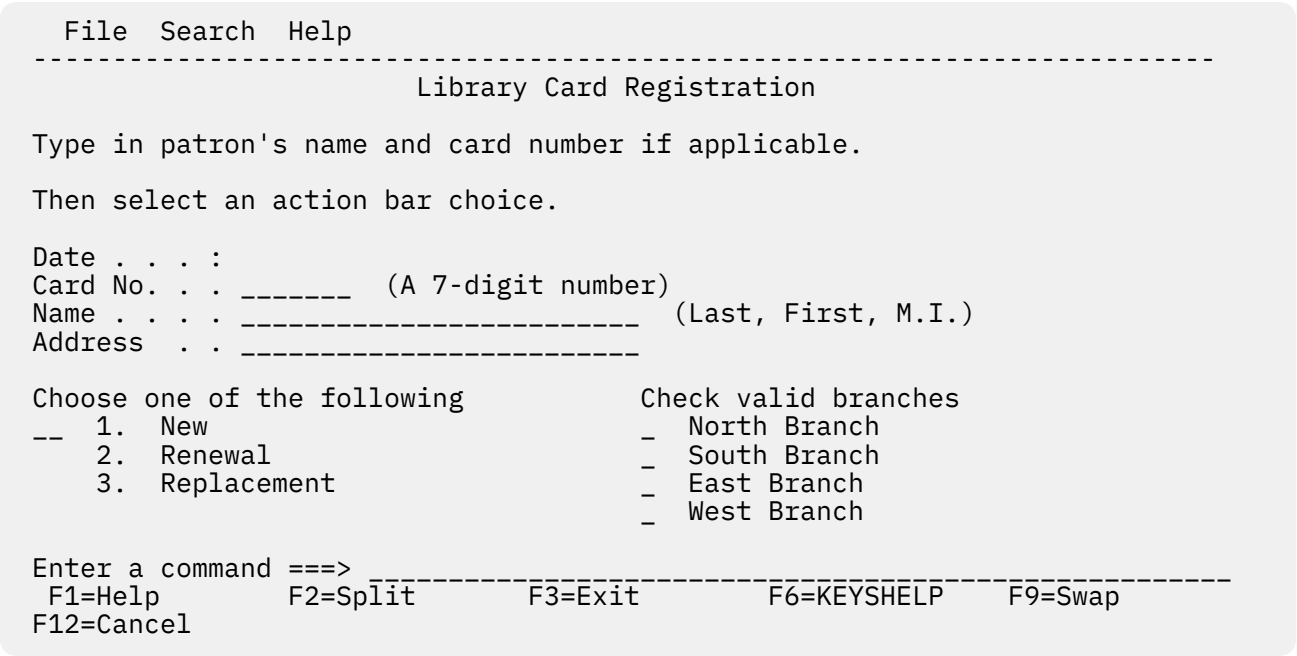
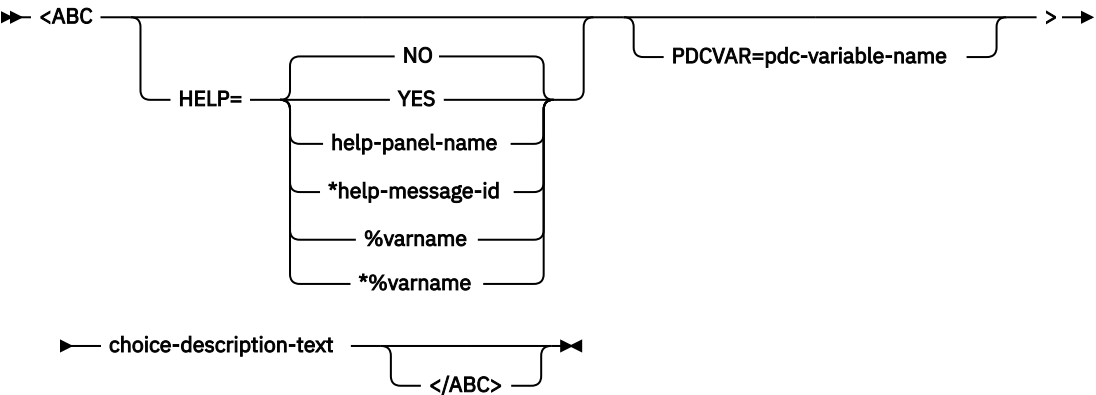


Figure 86. Action bar

ABC (Action Bar Choice)

The ABC tag defines a choice in an action bar and serves as a base for associated pull-down choice tags.

Syntax



Parameters

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%%*/varname

This attribute specifies the help action taken when the user requests help on the action bar choice.

When `HELP=YES`, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When **%%*/varname** is coded, a message variable name is created.

If the user requests help on an action bar choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information on creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

PDCVAR=*pdv-variable-name*

This attribute provides the name of a variable to contain the value of the pull-down choice. When a variable name is provided, it replaces the default ZPDC variable name. The *pdv-variable-name* is not initialized to blank.

The *pdv-variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

choice-description-text

This is the text that appears in the action bar. The text is limited to 64 bytes.

If the *choice-description-text* exceeds the panel width, the conversion utility issues a warning message and truncates the text. If the *choice-description-text* for multiple ABC tags exceeds the panel width, the conversion utility formats a multiple-line action bar.

Comments

The ABC tag defines a choice in an action bar and serves as a base for associated pull-down choice tags. The pull-down choices appear in a pull-down when the action bar choice is selected.

If the text of an action bar choice contains multiple words, multiple blanks between the words are not compressed.

Restrictions

- You must code the ABC tag within an AB definition. See [“AB \(Action Bar\)”](#) on page 179 for a complete description of this tag.
- You must code at least one PDC tag within each ABC definition. See [“PDC \(Pull-Down Choice\)”](#) on page 390 for a complete description of this tag.
- The maximum number of action bar choices that is generated is 40.

Processing

Table 4. Tags you can code within an ABC definition			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
M	“M (Mnemonic)” on page 351	Single	No
PDC	“PDC (Pull-Down Choice)” on page 390	Multiple	Yes
PDSEP	“PDSEP (Pull-Down Separator)” on page 394	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is markup that shows the use of the PDCVAR attribute to specify an application variable for the first action bar choice. It produces the action bar on the application panel shown in [Figure 87](#) on page 184.

ACTION

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampbody system)>
&sampvar1;

<PANEL NAME=abc1 KEYLIST=keylxml>Library Card Registration
<AB>
<ABC PDCVAR=foptns>File
  <PDC>Add Entry
    <ACTION RUN=add>
  <PDC>Delete Entry
    <ACTION RUN=delete>
  <PDC>Update Entry
    <ACTION RUN=update>
  <PDC>Exit
    <ACTION RUN=exit>
<ABC>Search
  <PDC CHECKVAR=whchsrch MATCH=1>Search on name
    <ACTION SETVAR=whchsrch VALUE=1>
    <ACTION RUN=search>
  <PDC CHECKVAR=whchsrch MATCH=2>Search on card number
    <ACTION SETVAR=whchsrch VALUE=2>
    <ACTION RUN=search>
<ABC>Help
  <PDC>Extended Help...
    <ACTION RUN=exhelp>
  <PDC>Keys Help...
    <ACTION RUN=keyshelp>
</AB>
&sampbody;
</PANEL>
```

```
File Search Help
-----
                        Library Card Registration

Type in patron's name and card number if applicable.

Then select an action bar choice.

Date . . . :
Card No. . . _____ (A 7-digit number)
Name . . . . _____ (Last, First, M.I.)
Address . . _____

Choose one of the following          Check valid branches
--  1. New                          _ North Branch
    2. Renewal                      _ South Branch
    3. Replacement                  _ East Branch
                                   _ West Branch

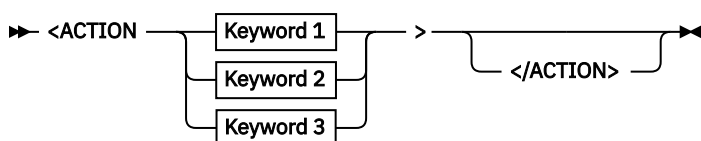
Enter a command ==> _____
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel
```

Figure 87. Action bar choices

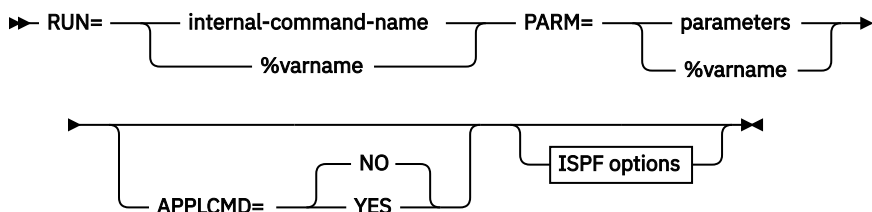
ACTION (Action)

The ACTION tag defines the action that occurs when a pull-down choice or a selection field choice is selected.

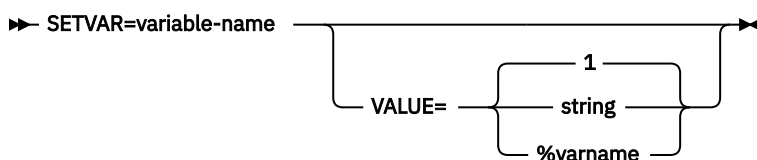
Syntax



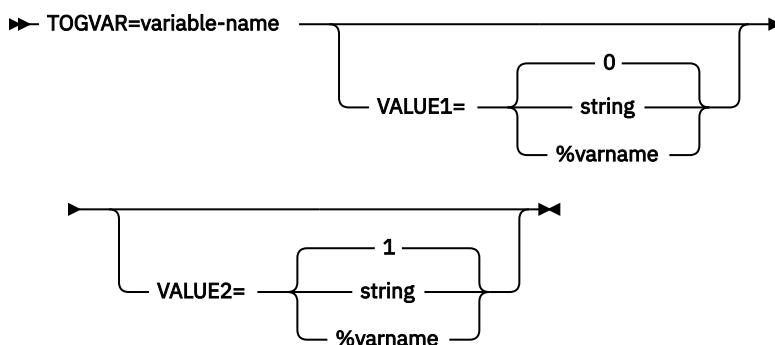
Keyword 1



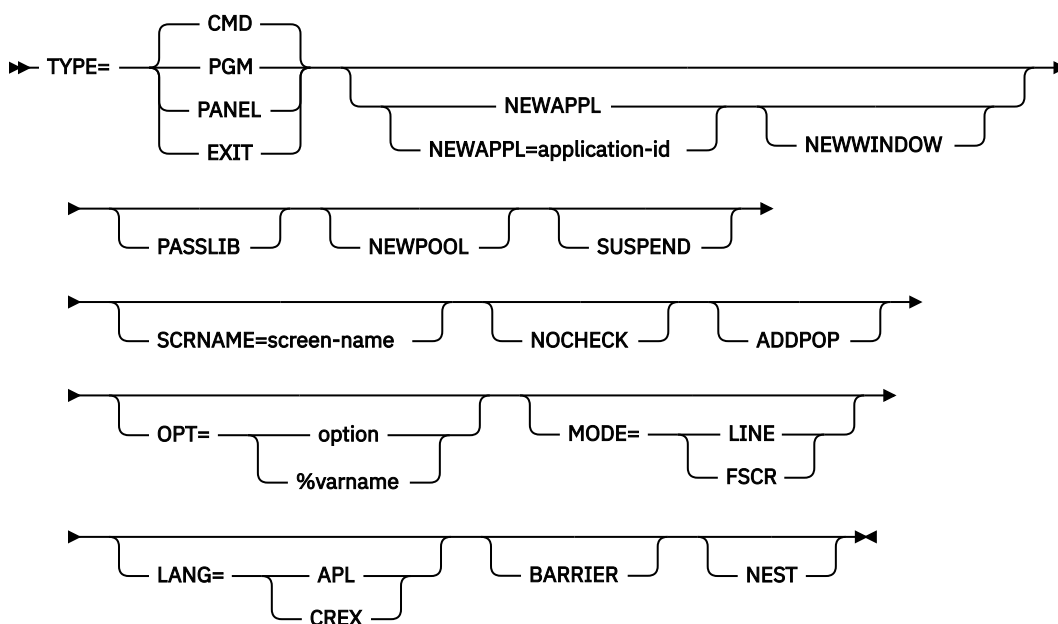
Keyword 2



Keyword 3



ISPF options



Parameters

RUN=internal-command-name | %varname

When the ACTION tag is associated with a PDC tag, this attribute specifies the internal name of a command to be executed. The command is found in the application or system command table unless APPLCMD=YES is specified. The search for the command follows the normal command processing rules. For information on defining commands, see [“CMD \(Command Definition\)” on page 233](#).

The RUN action is an ending action. Thus, if multiple ACTION tags are coded for a given pull-down, those following a RUN action are ignored.

When the ACTION tag is associated with a CHOICE tag (under a SELFLD tag that specifies TYPE=MENU or TYPE=MODEL), the TYPE attribute and related RUN attribute values are:

TYPE

RUN attribute value

CMD

Command name

PGM

Program name

PANEL

Panel name

When the ACTION tag is associated with a CHOICE tag under a SELFLD tag that specifies TYPE=TUTOR, the TYPE attribute is forced to **PANEL**. The RUN attribute must provide a panel name. None of the other ISPF selection menu attributes are valid for tutorial panels.

If TYPE=CMD is specified and the *internal-command-name* should start with a %, you must code an additional % before the *internal-command-name* to distinguish it from a variable name. (For example, to specify the *internal-command-name* “%abc”, code “%%abc”. If TYPE=EXIT is specified, the RUN attribute is required for conversion utility processing, but is not used in the generated panel.

Note: This attribute is not supported if the ACTION tag is associated with a CHOICE tag under a SELFLD tag that specifies TYPE=SINGLE or TYPE=MULTI.

PARM=parameters | %varname

These are the command parameters. These parameters are passed to command processing with the command specified on the RUN attribute. Command processing handles the specified parameters the same way parameters entered in the command area are handled. You can specify the name of a dialog variable (using % notation) whose value at run time is passed as the parameter data. When the ACTION tag is associated with a PDC tag, the conversion utility limits the length of the command parameters to 72 single-byte characters.

When a ACTION tag is used to build a menu selection choice for TYPE=CMD or TYPE=PGM, and the NEWWINDOW attribute has been specified, the conversion utility limits the length of the command parameters to 249 single-byte characters; otherwise, the parameter is added to the selection as coded.

APPLCMD=NO | YES

This attribute specifies whether the command provided by the RUN attribute is to be passed directly to the application, bypassing the command table search. When APPLCMD=YES, the length of the command name is limited to 7 bytes to allow the passthru character ">" to be prefixed to the command name.

This attribute is valid only on an ACTION tag that is associated with a PDC tag.

Here is a list of attributes that are valid only when generating an ISPF selection menu or edit model selection menu. (When the SELFLD tag specifies TYPE=TUTOR, the TYPE attribute is forced to "PANEL" and none of the other ISPF selection menu attributes are valid.)

TYPE=CMD | PGM | PANEL | EXIT

This attribute specifies the type of selection to be generated for the selection menu. The attributes NEWAPPL, NEWWINDOW, PASSLIB, NEWPOOL, SUSPEND, SCRNAME, NOCHECK, ADDPOP, OPT, MODE, LANG, BARRIER, and NEST are not valid when TYPE=EXIT is specified.

NEWAPPL=application-id

The NEWAPPL keyword may be specified with or without an application identifier. This attribute specifies that the NEWAPPL keyword (and the application identifier, if present) are added to the selection menu choice.

NEWWINDOW

This attribute specifies that the selection menu choice is created specifying the ISPSTRT programming interface. The NEWWINDOW attribute is valid only when TYPE=PANEL, TYPE=PGM, or TYPE=CMD.

PASSLIB

This attribute specifies that the PASSLIB keyword is added to the selection menu choice.

NEWPOOL

This attribute specifies that the NEWPOOL keyword is added to the selection menu choice.

SUSPEND

This attribute specifies that the SUSPEND keyword is added to the selection menu choice.

SCRNAME=screen-name

This attribute specifies that the SCRNAME keyword is added to the selection menu choice. ISPF reserved values for *screen-name* are LIST, NEXT, PREV, ON, and OFF.

NOCHECK

This attribute specifies that the NOCHECK keyword is added to the selection menu choice. The NOCHECK attribute is valid only when TYPE=CMD or TYPE=PGM.

ADDPOP

This attribute specifies that the ADDPOP keyword is added to the selection menu choice. The ADDPOP attribute is valid only when TYPE=PANEL.

OPT=option | %varname

This attribute specifies that the OPT keyword is added to the selection menu choice to specify an initial option for the panel. The OPT attribute is valid only when TYPE=PANEL.

MODE=LINE | FSCR

This attribute specifies that the MODE keyword is added to the selection menu choice. The MODE attribute is valid only when TYPE=CMD or TYPE=PGM.

LANG=APL | CREX

This attribute specifies that the LANG keyword is added to the selection menu choice. The LANG attribute is valid only when TYPE=CMD. LANG(CREX) is optional if the compiled REXX has been link-edited to include any of the stubs EAGSTCE, EAGSTCPP, or EAGSTMP.

BARRIER

This attribute specifies that the BARRIER keyword is added to the selection menu choice. The BARRIER attribute is valid only when TYPE=CMD.

NEST

This attribute specifies that the NEST keyword is added to the selection menu choice. The NEST attribute is valid only when TYPE=CMD.

SETVAR=variable-name

This attribute sets a value into a dialog variable. The SETVAR attribute names the variable to set. The *variable-name* must be coded without the leading % sign.

VALUE=1 | string | %varname

This is the value to set into the variable named on the SETVAR attribute. If you code the SETVAR attribute but omit the VALUE attribute, ISPF assigns the variable a value of 1. You can specify the name of a variable (using % notation) whose value at run time sets the value of the variable.

When defining the ACTION tag for selection fields, be aware that the variable name defined in the SELFLD tag for single-choice selection fields or in the CHOICE tag for multiple-choice selection fields

contains the value entered by the user when the selection is made. In addition, if the CHECKVAR attribute is specified in the CHOICE tag, the value of the MATCH attribute associated with the choice is set into the variable named by the CHECKVAR attribute. Therefore, it is not necessary to use the ACTION tag SETVAR attribute for the application to know which selection field choice or choices were made by the user.

TOGVAR=variable-name

This attribute allows you to alternate the value of a single variable between two values. The TOGVAR attribute names the variable to set. The *variable-name* must be coded without the leading % sign.

The function of the TOGVAR action can be depicted as follows:

```
if (TOGVAR-variable-name = VALUE1-string)
  TOGVAR-variable-name = VALUE2-string
else
  TOGVAR-variable-name = VALUE1-string
```

VALUE1=0 | string | %varname

This is the value to set into the variable named on the TOGVAR attribute if it is not currently equal to this value. If you code the TOGVAR attribute, but omit the VALUE1 attribute, the variable is assigned a value of 0. You can specify the name of a variable (using % notation) whose value at run time sets the value of the variable.

VALUE2=1 | string | %varname

This is the value to set into the variable named on the TOGVAR attribute if it is currently equal to the value specified with the VALUE1 attribute. If you code the TOGVAR attribute, but omit the VALUE2 attribute, the variable is assigned a value of 1. You can specify the name of a variable (using % notation) whose value at run time sets the value of the variable.

Comments

The ACTION tag defines the action that occurs when a pull-down choice or a selection field choice is selected. Code the ACTION tag within the PDC or CHOICE definition it is associated with. You can specify multiple ACTION tags for a given choice. The conversion utility builds the logic to carry out the actions in the order in which you code the ACTION tags.

When defining action bar pull-downs, you should code the SETVAR attribute in the ACTION tags associated with each PDC tag if the application needs to know which pull-down choice the user selected. Unlike selection fields, there is no variable name associated with a pull-down definition and the PDC CHECKVAR variable is not set to indicate the user's choice. Therefore, dialogs must refer to the SETVAR *variable-name* to determine the pull-down choice the user has selected.

The TYPE, NEWAPPL, NEWWINDOW, PASSLIB, NEWPOOL, SUSPEND, SCRNAME, NOCHECK, ADDPOP, OPT, MODE, LANG, BARRIER, and NEST attributes are used by the conversion utility to build an ISPF selection menu. They are valid only when they appear on an ACTION tag associated with a CHOICE tag which is nested within a SELFLD tag that specifies TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR (when the SELFLD tag specifies TYPE=TUTOR, the only valid selection menu attribute is TYPE=PANEL). They are not processed in other situations. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for a description of the function of these keywords in ISPF option menus.

Restrictions

- You must code the ACTION tag within the PDC or CHOICE definition it is associated with. See [“PDC \(Pull-Down Choice\)” on page 390](#) and [“CHOICE \(Selection Choice\)” on page 226](#) for descriptions of these tags.
- You must code one (and only one) of these attributes on each ACTION tag: RUN, SETVAR, or TOGVAR.
- You can code the RUN attribute when:
 - The ACTION tag is associated with a PDC tag.

- The ACTION tag is associated with a CHOICE tag under a SELFLD tag that specifies TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names”](#) on page 179.

Processing

None.

Examples

Here is markup where each of the PDC tags have associated ACTION tags that specify the command that is executed when the pull-down choice is selected. Many of the PDC tags have additional ACTION tags associated with them that specify the SETVAR attribute to let the application know which pull-down choice was selected.

The use of ACTION tags associated with CHOICE tags is illustrated in the example for [“PS \(Point-and-Shoot\)”](#) on page 398.

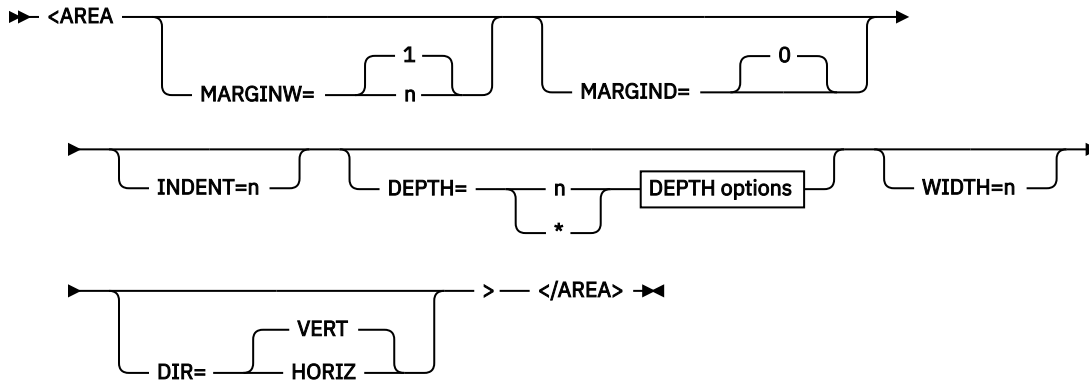
```
<!DOCTYPE DM SYSTEM>

<PANEL NAME=action1>Library Card Listing
<AB>
<ABC>File
  <PDC>Add Entry
    <ACTION SETVAR=fchoice VALUE=add>
    <ACTION RUN=add>
  <PDC>Delete Entry
    <ACTION SETVAR=fchoice VALUE=delete>
    <ACTION RUN=delete>
  <PDC>Update Entry
    <ACTION SETVAR=fchoice VALUE=update>
    <ACTION RUN=update>
  <PDC>Exit
    <ACTION RUN=exit>
<ABC>Sort sequence
  <PDC CHECKVAR=whchsort MATCH=1>Sort on name
    <ACTION SETVAR=whchsort VALUE=1>
    <ACTION RUN=sort>
  <PDC CHECKVAR=whchsort MATCH=2>Sort on card number
    <ACTION SETVAR=whchsort VALUE=2>
    <ACTION RUN=sort>
<ABC>Help
  <PDC>Extended Help...
    <ACTION RUN=exhelp>
  <PDC>Keys Help...
    <ACTION RUN=keyshelp>
</AB>
<TOPINST>Choose the size of the list needed.
<TOPINST>Then select action bar choice "Sort sequence" to
indicate the desired sort sequence.
<AREA>
  <SELFLD NAME=aa PMTWIDTH=30 PMTLOC=before SELWIDTH=38>Choose
  one of the following
    <CHOICE>New this month
    <CHOICE>New this year
    <CHOICE>All (this will take time to process)
  </SELFLD>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

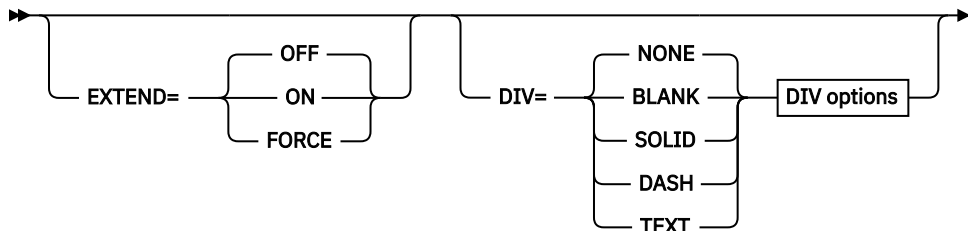
AREA (Area)

The AREA tag defines portions of a panel body, one or more of which can be scrollable.

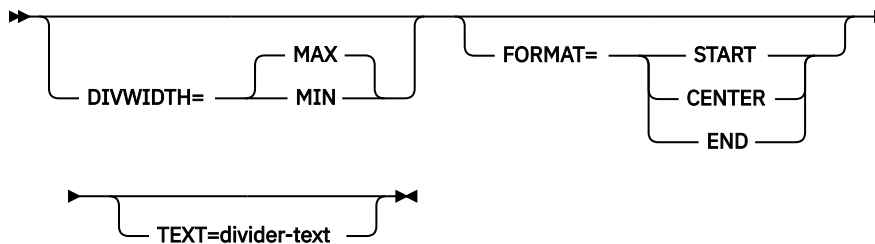
Syntax



DEPTH options



DIV options



Parameters

MARGINW=1 | n

This attribute defines a margin along the left and right sides of the panel area. This attribute allows you to specify the width of the margin in characters. The minimum value you can specify is 1 and the maximum value is 32. If you do not specify a value, the margin is set to 1.

The MARGINW cannot be larger than one half the panel width minus 2. Specification of the MARGINW should always allow enough room in the panel body section of the ISPF panel being generated to contain all non-wrapped data without truncation. Specification of one half the panel width minus 2 results in no panel area in which panel body text can be written.

MARGIND=0

This attribute defines a margin along the top and bottom of the panel area.

The conversion utility only supports a margin depth of zero in an effort to use all of the available space on the panel body. Any definition of margin depth that is not equal to zero is changed to zero.

INDENT=n

This attribute defines the number of columns to indent the current AREA from the current MARGINW value.

DEPTH=n | *

This attribute defines the minimum size of a scrollable panel area. If DEPTH is not specified for HELP panels, the conversion utility generates multiple HELP panels for compatibility with previous releases. When EXTEND=OFF, the minimum DEPTH is 2 lines. When EXTEND=ON, the minimum DEPTH is

1 line. When DEPTH=*, the conversion utility reserves the remaining available panel depth for the scrollable area.

EXTEND=OFF | ON | FORCE

This attribute defines the runtime display size for the scrollable area. If EXTEND=ON is specified, the panel definition is expanded from the minimum DEPTH to the size of the logical screen. Only one EXTEND=ON attribute value is allowed on a panel. The first tag (AREA, DA, GA, REGION, SELFLD) with EXTEND=ON is accepted; the EXTEND attribute on any subsequent AREA tag is ignored.

If you intend to display the panels in a pop-up window, it is recommended that you code EXTEND=OFF.

If the EXTEND attribute is specified without a DEPTH attribute, a warning message is issued and the EXTEND attribute is ignored.

If EXTEND=FORCE is specified within a horizontal area, the EXTEND(ON) keyword is added to the scrollable area attribute statement in the)ATTR panel section. The conversion utility issues a message to advise of a potential error if other panel fields are formatted on or after the last defined line of the scrollable area.

DIV=NONE | BLANK | SOLID | DASH | TEXT

This attribute specifies the type of divider line to be placed before and after the scrollable area. If this attribute is not specified, or has the value NONE, no divider line is generated. The value BLANK produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line. A visible divider formats with a non-displayable attribute byte on each end of the line.

If the DIV attribute is found without the DEPTH attribute, a warning message is issued and the DIV attribute is ignored.

DIVWIDTH=MAX | MIN

This attribute specifies the width of the divider line. If DIVWIDTH=MAX, the divider line extends across the entire width of the panel defined by the AREA tag. If DIVWIDTH=MIN, the divider line is formatted to allow for the MARGINW and INDENT attribute values.

FORMAT=START | CENTER | END

This attribute specifies the position of the *divider-text* within the divider line. You must specify both the FORMAT attribute and the TEXT attribute to create a divider line containing text.

TEXT=divider-text

This attribute specifies the text to be placed on the divider line. You must specify both the FORMAT attribute and the TEXT attribute to create a divider line containing text.

WIDTH=n

This attribute defines the width of a panel area. If WIDTH is not specified the area formats to the remaining available panel width.

DIR=VERT | HORIZ

This attribute allows areas to be placed side by side on the panel. You use the WIDTH attribute in combination with the DIR attribute to tell the conversion utility to position areas horizontally. When the current horizontal AREA right boundary reaches or exceeds the right panel boundary, the next AREA is formatted below the current AREA(s), at the left edge of the panel.

Comments

The AREA tag defines portions of a panel body. The conversion utility uses the DEPTH attribute value to reserve lines in the formatted panel body for a scrollable area. The DEPTH value cannot be more than the number of panel body lines still available for formatting when the AREA tag is processed.

The maximum DEPTH that you can specify is 2 lines less than the DEPTH value specified on the HELP or PANEL tag.

Note:

1. If you specify the CMDAREA tag within your DTL source file, it must appear before the AREA tag when DEPTH=* is specified. The AREA tag DEPTH may have to be adjusted to allow for additional lines which result from tags present within the panel definition following the end AREA tag.
2. For HELP panels, the presence of additional tags following a scrollable area causes the conversion utility to reserve 4 lines at the bottom of the screen to provide for the function key area.

The first line of the scrollable area is always reserved for the scrolling indicator line, which is provided by ISPF at run time. If all of the scrollable portion of the panel is displayed within the available DEPTH, the scroll indicator line is blank; otherwise, the value **"More: +"**, **"More: - +"**, or **"More: -"** appears. On application panels, this portion includes the interactive fields and text of the panel. On help panels, this portion is the area of the panel that contains help text.

The DIR attribute is used to place entire areas side by side on the panel. Formatting within the AREA tag is always in a vertical direction. Panel areas are formatted horizontally when multiple AREA tags (specifying DIR=HORIZ) are placed sequentially in the DTL source file. Any other tag which occurs between an end AREA tag and a start AREA tag causes the overall panel formatting direction to be set to vertical.

Restrictions

- The AREA tag requires an end tag.
- You must code AREA tags within a HELP or PANEL definition. See [“HELP \(Help Panel\)” on page 303](#) and [“PANEL \(Panel\)” on page 376](#) for descriptions of these tags.
- Only one area can be defined with EXTEND=ON. This includes other AREA tags as well as any dynamic area defined by the DA tag, graphic area defined by the GA tag, scrollable section lists defined by the SELFLD tag, or scrollable regions defined by the REGION tag.

Processing

Application panel

<i>Table 5. Tags you can code within an AREA definition on an application panel</i>			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DA	“DA (Dynamic Area)” on page 250	Multiple	No
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
DTACOL	“DTACOL (Data Column)” on page 269	Multiple	No
DTAFLD	“DTAFLD (Data Field)” on page 275	Multiple	No
GA *	“GA (Graphic Area)” on page 295	Single	No
GENERATE	“GENERATE (Generate)” on page 298	Multiple	No
GRPHDR	“GRPHDR (Group Header)” on page 300	Multiple	No
INFO	“INFO (Information Region)” on page 317	Multiple	No
LSTFLD *	“LSTFLD (List Field)” on page 341	Single	No
PNLINST	“PNLINST (Panel Instruction)” on page 396	Multiple	No
REGION	“REGION (Region)” on page 405	Multiple	No
SELFLD	“SELFLD (Selection Field)” on page 421	Multiple	No

Table 5. Tags you can code within an AREA definition on an application panel (continued)

Tag	Reference	Usage	Required
SOURCE	“SOURCE (Source)” on page 435	Multiple	No
Note: Tags marked with * are not valid within an ISPF selection menu panel.			

Help panel

Table 6. Tags you can code within an AREA definition on a help panel

Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
GENERATE	“GENERATE (Generate)” on page 298	Multiple	No
INFO	“INFO (Information Region)” on page 317	Multiple	No
REGION	“REGION (Region)” on page 405	Multiple	No

Examples

Here is an example application panel that contains four data fields and two selection fields coded within the AREA definition. The top instructions and command area are coded outside of the AREA definition. In addition, the panels illustrate a scrollable panel. [Figure 88 on page 194](#), [Figure 89 on page 194](#) and [Figure 90 on page 195](#), show the formatted results.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar2 system>
  <!entity sampabc system>)>
&sampvar2;

<PANEL NAME=area1 KEYLIST=keylxml>File-A-Case
<AB>
&sampabc;
</AB>
<TOPINST COMPACT>
  Type in client's name and case number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
  <DTAFD DATAVAR=caseno PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Case no
  <DTAFDLD>(A 7-digit number)
  <DTAFD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Name
  <DTAFDLD>(Last, First, M.I.)
  <DTAFD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
  <DIVIDER>
  <SEFLD NAME=casesel PMTWIDTH=30 PMTLOC=before SELWIDTH=38>Choose
  one of the following
  <CHOICE CHECKVAR=case MATCH=civ>Civil
  <CHOICE CHECKVAR=case MATCH=estate>Real estate
  <CHOICE CHECKVAR=case MATCH=environ>Environmental
  </SEFLD>
</AREA>
<AREA DEPTH=6>
  <SEFLD TYPE=multi PMTWIDTH=35 SELWIDTH=50>Check type of offense committed
  <CHOICE NAME=patin HELP=patin CHECKVAR=val>Patent infringement
  <CHOICE NAME=defa HELP=defame CHECKVAR=def>Defamation
  <CHOICE NAME=cont HELP=cont CHECKVAR=qua>Breach of valid contract
  <CHOICE NAME=priv HELP=priv CHECKVAR=pri>Invasion of privacy
  <CHOICE NAME=incr HELP=incr CHECKVAR=icr>Interference with
  contractual relations
  <CHOICE NAME=disp HELP=disp CHECKVAR=dis>Improper disposal of
  medical by-products
  <CHOICE NAME=fraud HELP=fraud CHECKVAR=fra>Fraud
  </SEFLD>
</AREA>
```

```
<CMDAREA>Enter a command
</PANEL>
```

```

File  Search  Help
-----
                        File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case no  . .  _____  (A 7-digit number)
Name    . .  .  _____  (Last, First, M.I.)
Address  . .  _____

Choose one of the following  --  1.  Civil
                                   2.  Real estate
                                   3.  Environmental
                                   More:      +

Check type of offense committed
_  Patent infringement
_  Defamation
_  Breach of valid contract
Enter a command ==>
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel
```

Figure 88. Application panel area

After scrolling, the panel looks like this:

```

File  Search  Help
-----
                        File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case no  . .  _____  (A 7-digit number)
Name    . .  .  _____  (Last, First, M.I.)
Address  . .  _____

Choose one of the following  --  1.  Civil
                                   2.  Real estate
                                   3.  Environmental
                                   More:      - +

_  Breach of valid contract
_  Invasion of privacy
_  Interference with contractual relations
_  Improper disposal of medical by-products
Enter a command ==>
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel
```

Figure 89. Application panel area

After scrolling, the last choice in the list is visible.

```

File  Search  Help
-----
                                File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case no   . .  _____  (A 7-digit number)
Name      . .  .  _____  (Last, First, M.I.)
Address   . .  _____

Choose one of the following      --   1.  Civil
                                       2.  Real estate
                                       3.  Environmental

                                           More:   -

-  Invasion of privacy
-  Interference with contractual relations
-  Improper disposal of medical by-products
-  Fraud
-  Enter a command ==> _____
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel

```

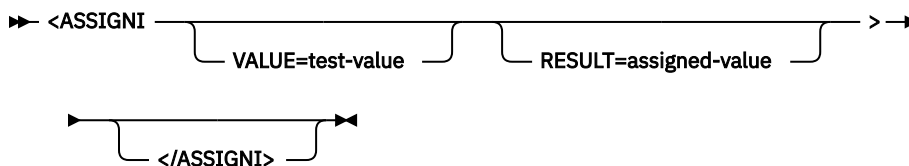
Figure 90. Application panel area

An example of horizontal AREA formatting is shown in “Multiple AREA tags” on page 42.

ASSIGNI (Assignment List Item)

The ASSIGNI tag defines an assignment in an assignment list.

Syntax



Parameters

VALUE=test-value

This attribute specifies the value to be matched when performing the assignment.

The value of the data field variable is compared to the value of each VALUE attribute in succession until a match is found. The *test-value* must be the same case as the value to be matched. You can specify XLATL FORMAT=UPPER within the variable class associated with the data field to convert user input to uppercase before the assignment test is processed.

When ISPF finds a match, it assigns the value in the RESULT attribute to the variable named on the ASSIGNL tag. If ISPF does not find a match, no assignment is made.

If you omit this attribute, any value satisfies the test and ISPF assigns *assigned-value* to the dialog variable.

If a *test-value* appears more than once in the list, the first occurrence is used.

RESULT=assigned-value

This attribute specifies the resulting value of the assignment if a match occurs on the *test-value* specified by VALUE.

Assigned-value specifies the character string value for the conversion utility to assign to the variable named on the ASSIGNL tag. If you omit this attribute, the *test-value* is assigned to the variable.

Comments

The ASSIGNI tag defines an assignment in an assignment list. Each ASSIGNI tag provides information necessary to assign a value (the RESULT attribute) to a variable (specified with the ASSIGNL tag) based on the *test-value* (the VALUE attribute) of the variable named on the DTAFLD tag. As many ASSIGNI tags as are necessary (up to a limit of 126) can be included within the assignment list. The ISPF TRANS() function is used for ASSIGNI processing.

If both the VALUE and RESULT attributes are omitted, the DESTVAR attribute of the ASSIGNL tag is assigned the value of the data field's data variable (DATAVAR).

Restrictions

- You must code an ASSIGNI tag within an ASSIGNL definition. See [“ASSIGNL \(Assignment List\)”](#) on page 196 for a complete description of this tag.

Processing

None.

Examples

Here is source file markup that contains an application panel containing a data field. Within the data field is an assignment list that sets the dialog variable *rmtype* to 1 when "SINGLE" is entered in the data field, or to 2 when "DOUBLE" is entered in the data field. The associated variable declarations and variable classes are also shown.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=roomvar TYPE='char 6'>
  <XLATL FORMAT=upper>
</XLATL>

<VARCLASS NAME=rmtypvar TYPE='char 1'>

<VARLIST>
  <VARDCL NAME=room VARCLASS=roomvar>
  <VARDCL NAME=rmtype VARCLASS=rmtypvar>
</VARLIST>

<PANEL NAME=assigni DEPTH=12 WIDTH=50>Hotel Register
<AREA>
  <DTAFLD DATAVAR=room ENTWIDTH=6 DESWIDTH=20 PMTWIDTH=15>Room type
    <ASSIGNL DESTVAR=rmtype>
      <ASSIGNI VALUE=SINGLE RESULT=1>
      <ASSIGNI VALUE=DOUBLE RESULT=2>
    </ASSIGNL>
    <DTAFLDD>(Single or Double)
  </AREA>
<BOTINST>Press Enter to continue.
</PANEL>
```

ASSIGNL (Assignment List)

The ASSIGNL tag defines an assignment list.

Syntax

➤ <ASSIGNL — DESTVAR=destination-variable-name — > — </ASSIGNL> ➤

Parameters

DESTVAR=destination-variable-name

DESTVAR specifies the variable that receives the assignment value. You can code multiple assignment lists if you need to assign values to additional variables.

Note: If the *destination-variable-name* is a variable name used for another field on the panel, the value of the other field is overlaid by the assignment value. The *destination-variable-name* should only be used for an output field variable.

Comments

The ASSIGNL tag defines an assignment list. ASSIGNI tags, which define the elements of the assignment list, are coded within the ASSIGNL tag.

Assignment lists are optional and provide a means of assigning a value to one variable based on the content of another. ISPF compares the value of the variable specified with the DATAVAR attribute of the DTAFLD tag against the values in the ASSIGNI tags.

Processing of assignment lists occurs at panel end (after translates and checks).

Restrictions

- The ASSIGNL tag requires an end tag.
- You must code an ASSIGNL tag within the DTAFLD definition it is associated with. See [“DTAFLD \(Data Field\)” on page 275](#) for a complete description of this tag.

Processing

Table 7. Tags you can code within an ASSIGNL definition			
Tag	Reference	Usage	Required
ASSIGNI	“ASSIGNI (Assignment List Item)” on page 195	Multiple	No

Examples

In this example markup, the assignment list assigns a value to the variable *decimal* dependent on the value the user enters in the data field variable *hexvar*. The associated variable declarations and variable classes are also shown.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=varcls1 TYPE='char 1'>

<VARCLASS NAME=varcls2 TYPE='char 2'>

<VARLIST>
  <VARDCL NAME=hexvar VARCLASS=varcls1>
  <VARDCL NAME=decimal VARCLASS=varcls2>
</VARLIST>

<PANEL NAME=assign1>Hex to Decimal
<TOPINST>Enter a hexadecimal digit.
<AREA>
<DTAFLD DATAVAR=hexvar PMTWIDTH=23 ENTWIDTH=1>Hexadecimal Value
  <ASSIGNL DESTVAR=decimal>
```

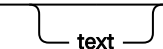
ATTENTION

```
<ASSIGNI VALUE=0>
<ASSIGNI VALUE=1>
<ASSIGNI VALUE=2>
<ASSIGNI VALUE=3>
<ASSIGNI VALUE=4>
<ASSIGNI VALUE=5>
<ASSIGNI VALUE=6>
<ASSIGNI VALUE=7>
<ASSIGNI VALUE=8>
<ASSIGNI VALUE=9>
<ASSIGNI VALUE=a RESULT=10>
<ASSIGNI VALUE=b RESULT=11>
<ASSIGNI VALUE=c RESULT=12>
<ASSIGNI VALUE=d RESULT=13>
<ASSIGNI VALUE=e RESULT=14>
<ASSIGNI VALUE=f RESULT=15>
<ASSIGNI RESULT="??">
</ASSIGNI>
<DTAFD DATAVAR=decimal USAGE=out PMTWIDTH=23 ENTWIDTH=2>Decimal Value
</AREA>
</PANEL>
```

ATTENTION (Attention)

The ATTENTION tag defines text that alerts the user to a risk of possible error conditions in the system.

Syntax

➤ <ATTENTION>  </ATTENTION> ➤

Parameters

text

This is the text of the attention message.

Comments

The ATTENTION tag defines text that alerts the user to a risk of possible error conditions in the system.

The ATTENTION tag is one of the tags that alert the user of a possible risk; CAUTION and WARNING are the others.

Code an attention statement before the text to which it pertains so that the user can read about the possible risks before reading the text.

When an attention statement is displayed, the string "Attention:" (or its translated equivalent) appears on the screen before the text of the statement.

You can code additional paragraphs of text by coding the P (paragraph) tag within an ATTENTION definition. You can also code other tags allowed in an information area within an ATTENTION definition.

Restrictions

- The ATTENTION tag requires an end tag.
- You must code the ATTENTION tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- The ATTENTION tag must be immediately preceded by a P, LI, or LP tag. If the ATTENTION tag is coded on the same line as one of these tags, there can be no intervening blanks. See [“P \(Paragraph\)” on page 370](#), [“LI \(List Item\)” on page 325](#), and [“LP \(List Part\)” on page 330](#) for descriptions of these tags.

- You cannot nest ATTENTION, WARNING or CAUTION tags within each other.

Processing

Table 8. Tags you can code within an ATTENTION definition			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a warning statement. The warning statement starts at the left margin because it is embedded in the LP tag.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=attentn DEPTH=20>Help For Changing a File
<AREA>
<INFO>
  <OL>
    <LI>Type over the existing data
    in the entry fields with the new data.
    <LP><ATTENTION>Performing the next step will save
    all changes and delete the existing data.
    <P>To quit this function without
    deleting the existing data, press F12.
    </ATTENTION>
    <LI>Press Enter to save the
    updated data.
  </OL>
</INFO>
</AREA>
</HELP>
```

Help For Changing a File

1. Type over the existing data in the entry fields with the new data.

Attention: Performing the next step will save all changes and delete the existing data.

To quit this function without deleting the existing data, press F12.

2. Press Enter to save the updated data.

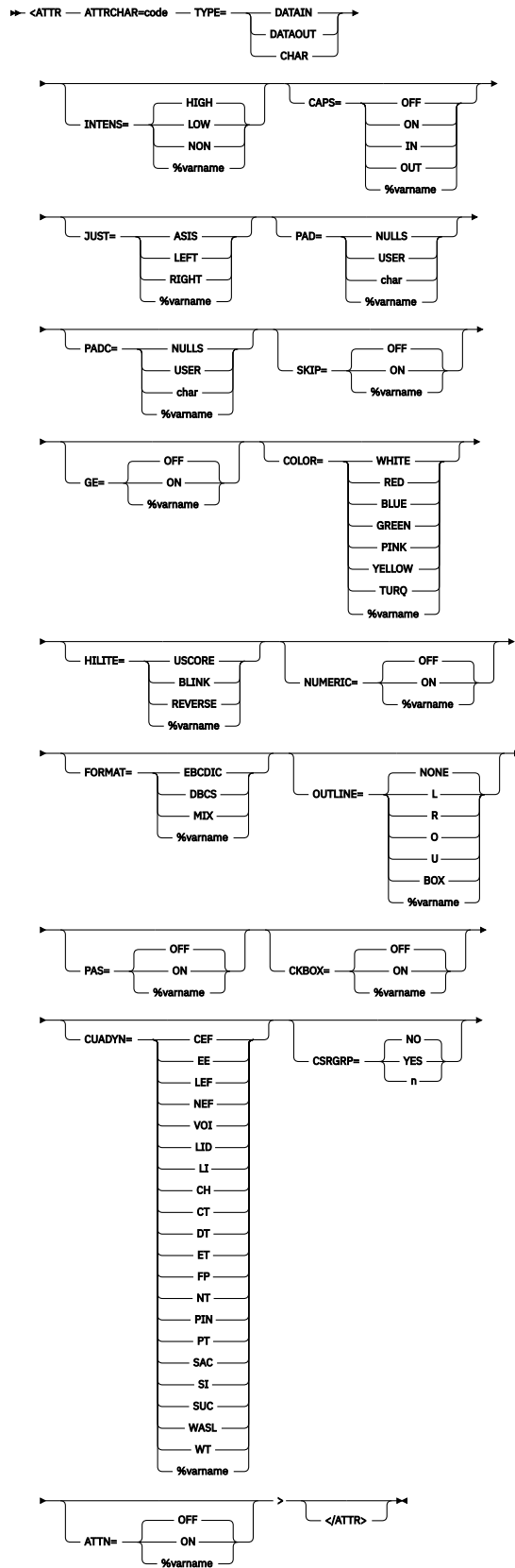
F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

Figure 91. Attention statement

ATTR (Attribute)

The ATTR tag defines a panel attribute used within a dynamic area or a preformatted displayable panel section. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for a complete discussion of panel)ATTR section keywords.

Syntax



Parameters

ATTRCHAR=code

This attribute can be a single character or a two-position entry of valid hex digits. If you enter an incorrect value, a warning message is issued and the value is set to null. Hex entries are converted to character. Hex values '00'-'2F' are reserved for use by the conversion utility.

TYPE=DATAIN | DATAOUT | CHAR

This attribute specifies the characteristic of the field within the dynamic area. Use DATAIN and DATAOUT attribute values for specifying unprotected or protected fields, respectively, within the dynamic area. The CHAR attribute value defines a character attribute that is recognized only when found within a shadow variable.

When the ATTR tag is coded within the GENERATE tag, TYPE can also be specified as any CUA attribute type, or as %varname.

INTENS=HIGH | LOW | NON | %varname

This attribute defines the intensity of a field. You can define this attribute as a variable name preceded by a "%".

CAPS=OFF | ON | IN | OUT | %varname

This attribute specifies the uppercase or lowercase attribute of a field. You can define this attribute as a variable name preceded by a "%".

JUST=ASIS | LEFT | RIGHT | %varname

This attribute specifies how the contents of the field are to be justified when displayed. You can define this attribute as a variable name preceded by a "%".

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a "%".

PADC=NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a "%".

SKIP=OFF | ON | %varname

This attribute specifies the autoskip attribute of the field. You can define this attribute as a variable name preceded by a "%".

GE=OFF | ON | %varname

This attribute specifies whether ISPF places a graphic escape order before the character defined as a character level attribute by TYPE=CHAR. You can define this attribute as a variable name preceded by a "%".

COLOR=WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the field. You can define this attribute as a variable name preceded by a "%".

HILITE=USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute of the field. You can define this attribute as a variable name preceded by a "%".

NUMERIC=OFF | ON | %varname

This attribute specifies whether Numeric Lock is to be activated for data typed in the field. You can define this attribute as a variable name preceded by a "%".

FORMAT=EBCDIC | DBCS | MIX | %varname

This attribute specifies the character format for the field. You can define this attribute as a variable name preceded by a "%".

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a "%".

PAS=OFF | ON | %varname

This attribute controls the availability of the point-and-shoot function for dynamic areas. You can define this attribute as a variable name preceded by a "%".

CKBOX=OFF | ON | %varname

This attribute controls the generation of check boxes for dynamic areas when the panel is displayed by a client that is using the JSON API. You can define this attribute as a variable name preceded by "%".

CUADYN=CEF | EE | LEF | NEF | VOI | LID | LI | CH | CT | DT | ET | FP | NT | PIN | PT | SAC | SI | SUC | WASL | WT | %varname

This attribute specifies a standard CUA attribute for the DATAIN and DATAOUT panel attribute definitions.

Values CEF, EE, LEF, and NEF are valid when TYPE=DATAIN. The remaining values are valid when TYPE=DATAOUT. You can define this attribute as a variable name preceded by a "%".

The conversion utility issues a warning message if the CUADYN attribute is specified and the invocation option is NOCUAATTR.

Note: If you specify other attribute before the CUADYN attribute, the CUADYN attribute can override previously specified attributes. For example:

```
SKIP=ON CUADYN=FP
```

In the above example, CUADYN changes the SKIP attribute to OFF.

CSRGRP=NO | YES | n

The CSRGRP attribute is valid only when TYPE=DATAOUT. When CSRGRP=YES, the conversion utility generates a cursor group number to be used for this DATAOUT attribute. When CSRGRP=n, the number provided is used for this attribute. The PAS attribute must be specified as ON or %varname.

ATTN=NO | YES | %varname

This attribute specifies the attention-select attribute of the field. You can define this attribute as a variable name preceded by a "%".

Comments

The ATTR tag is used to create an entry in the)ATTR panel section for fields to be displayed within a dynamic area, or preformatted panel section.

Restrictions

- You must code an ATTR tag within a Dynamic Area or GENERATE tag definition. See [“DA \(Dynamic Area\)” on page 250](#) and [“GENERATE \(Generate\)” on page 298](#) for a complete description of these tags.
- If ATTRCHAR is not specified, an error is logged and the output panel is not saved.
- If ATTRCHAR is a duplicate of a previously specified attribute, or conflicts with an attribute reserved by the conversion utility, an error is logged and the output panel is not saved.
- If TYPE is not specified, an error is logged and the output panel is not saved. If TYPE is specified, but the value is invalid, the value is set to DATAIN.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

None.

Examples

```

<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

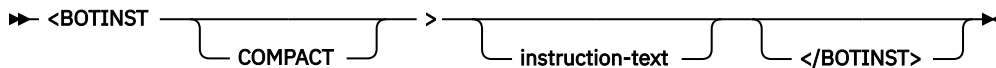
<PANEL NAME=attr KEYLIST=keylxml>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST> Type in patron's name and card number (if applicable)
<AREA>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
    <DTAFLD DATAVAR=curname USAGE=out ENTWIDTH=8>Name
    <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
    <DTAFLDD>(A 7-digit number)
    <DTAFLD DATAVAR=name>Name
    <DTAFLDD>(Last, First, M.I.)
    <DTAFLD DATAVAR=address>Address
  </DTACOL>
  <DIVIDER>
  <DA NAME=darea DIV=solid DEPTH=6 SHADOW=shadwvar>
    <ATTR ATTRCHAR=# TYPE=atain PADC='_' COLOR=BLUE>
    <ATTR ATTRCHAR=| TYPE=ataout COLOR=green>
    <ATTR ATTRCHAR=$ TYPE=char COLOR=red>
  </DA>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

BOTINST (Bottom Instruction)

The BOTINST tag defines bottom instructions for an application panel.

Syntax



Parameters

COMPACT

This attribute causes the bottom instruction to format without a blank line before the text.

instruction-text

This is the text of the bottom instruction. The *instruction-text* must fit in the remaining panel depth.

Comments

The BOTINST tag defines bottom instructions for an application panel. The *instruction-text* formats as a paragraph based on the width of the application panel. You can code multiple paragraphs of instruction text by using a new bottom instruction tag for each new paragraph.

If the COMPACT attribute is not specified, the conversion utility inserts a blank line before the bottom instruction text.

Restrictions

- You must code the BOTINST within a PANEL definition. See [“PANEL \(Panel\)”](#) on page 376 for a complete description of this tag.
- You cannot code a BOTINST tag within an AREA definition. If you define an area for the panel, code the BOTINST tag after the AREA end tag.

Processing

Table 9. Tags you can code within a BOTINST definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

This application panel markup contains two bottom instructions. [Figure 92 on page 205](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=choiccls TYPE='char 2'>
<VARLIST>
  <VARDCL NAME=choices VARCLASS=choiccls>
</VARLIST>

<PANEL NAME=botinst1 WIDTH=35 DEPTH=22>Four Choices
<AREA>
  <SEFLD NAME=choices PMTWIDTH=20 SELWIDTH=33>Choose one:
    <CHOICE>Raindrops on roses
    <CHOICE>Whiskers on kittens
    <CHOICE>Bright copper kettles
    <CHOICE>Warm woolen mittens
  </SEFLD>
</AREA>
<BOTINST>Press Enter to continue.
<BOTINST>Press F12 to cancel.
</PANEL>
```

```
Four Choices

Choose one:
--  1. Raindrops on roses
    2. Whiskers on kittens
    3. Bright copper kettles
    4. Warm woolen mittens

Press Enter to continue.

Press F12 to cancel.
```

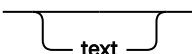
```
F1=Help    F3=Exit    F12=Cancel
```

Figure 92. Bottom instructions

CAUTION (Caution)

The CAUTION tag defines a statement that alerts the user of a possible risk.

Syntax

➤ <CAUTION>  </CAUTION> ➤

Parameters

text

This is the text of the caution statement.

Comments

The CAUTION tag defines a statement that alerts the user of a possible risk. Use the CAUTION tag to alert the user to a condition that might have serious consequences, such as the deletion of a file.

The CAUTION tag is one of the tags that alert the user to a possible risk; the others are ATTENTION and WARNING.

Code a caution statement before the text to which it pertains so that the user can read about the possible risks before reading the text. When a caution statement is displayed, the string "CAUTION:" or its translated equivalent appears on the screen and the caution text displays on the line after.

You can code additional paragraphs of caution text by coding the P (paragraph) tag within a CAUTION definition. You can also code other tags allowed in an information area within a CAUTION definition.

CAUTION text is formatted with an attribute byte that causes it to be emphasized.

Restrictions

- The CAUTION tag requires an end tag.
- A CAUTION tag must be immediately preceded by an LI, LP, or P tag. See [“LI \(List Item\)” on page 325](#), [“LP \(List Part\)” on page 330](#), and [“P \(Paragraph\)” on page 370](#) for descriptions of these tags.
- You must code the CAUTION tag only within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- You cannot nest ATTENTION, CAUTION, or WARNING tags within each other.

Processing

Table 10. Tags you can code within a CAUTION definition			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No

Table 10. Tags you can code within a CAUTION definition (continued)

Tag	Reference	Usage	Required
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a caution statement. [Figure 93 on page 207](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=caution DEPTH=20>Help for DELETE Command
<AREA>
  <INFO>
    <P>The DELETE command erases the specified file from storage.
    <P><CAUTION>
      Issuing the DELETE command permanently
      removes the file from storage.
      There is no possibility of recovery.
    </CAUTION>
    <P>You can exit from the DELETE operation
    by pressing F12.
  </INFO>
</AREA>
</HELP>
```

Help for DELETE Command

The DELETE command erases the specified file from storage.

CAUTION:
Issuing the DELETE command permanently removes the file from storage. There is no possibility of recovery.

You can exit from the DELETE operation by pressing F12.

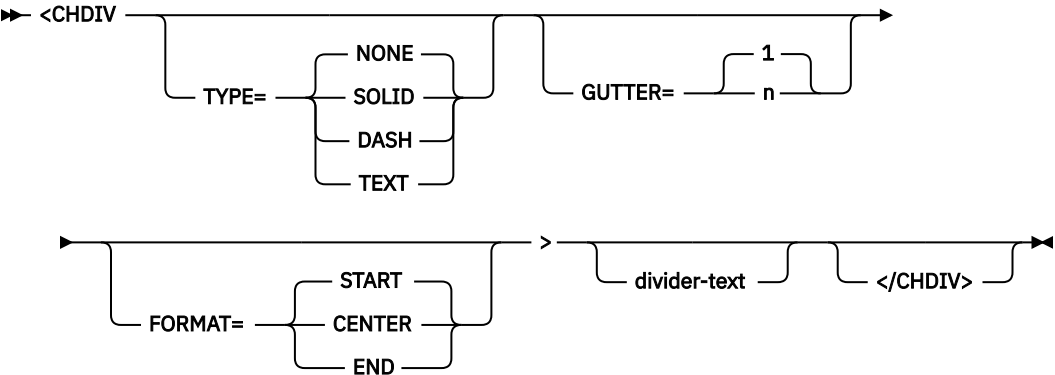
F1=Help F3=Exit F5=Exhelp
F6=Keyshelp F7=PrvTopic F8=NxtTopic
F10=PrvPage F11=NxtPage F12=Cancel

Figure 93. Caution statement

CHDIV (Choice Divider)

The CHDIV tag creates a blank or visible divider between CHOICE tags.

Syntax



Parameters

TYPE=NONE | SOLID | DASH | TEXT

This attribute specifies the type of divider line. The line width is one character.

The default value is NONE, which produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

GUTTER=1 | n

This attribute specifies the total width of the divider. If the GUTTER value is an even number, the conversion utility increases the number by 1 so that the divider is centered within the defined width.

The minimum and default GUTTER value is 1.

FORMAT=START | CENTER | END

This attribute specifies the position of the divider text within the width of the divider line. The divider width is the same as the CHOICE tag text formatting width.

divider-text

This is the text of the choice divider.

Comments

The CHDIV tag creates a blank or solid divider between CHOICE tags.

Restrictions

- You must code the CHDIV tag within an SELFLD definition. See [“SELFLD \(Selection Field\)” on page 421](#) for a description of this tag.

Processing

Table 11. Tags you can code within a CHDIV definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is an example that shows the creation of an ISPF selection menu. The CHDIV tag is included to separate the Exit option from the other selection choices. The FCHOICE attribute specifies that the first selection number is 0. The choice selection for Exit is specified on the CHOICE tag. The ACTION tag for the Exit choice selection specifies both the RUN and TYPE attributes because RUN is required on the ACTION tag and TYPE is necessary to specify the ISPF selection for the generated ZSEL panel statement.

```
<!doctype dm system (>
<!-- Sample selection menu -->
<varclass name=vc1 type='char 80'>
  <xlat1 format=upper>
  </xlat1>

<varlist>
  <vardcl name=zcmd varclass=vc1>
</varlist>

<panel name=chdiv1 menu keylist=keylxmlp>
  Sample Selection Panel with CHDIV tag
  <topinst>This is a selection panel.
  <selfld type=menu pmtloc=before fchoice=0 trail=nextsel
    selwidth=40 pmtwidth=10>Select an option
    <choice checkvar=xtest1 match=a>
      Selection #0 (Command Selch0)
      <action run=Selch0>
    <choice checkvar=xtest1 match=b>
      Selection #1 (Command Selch1)
      <action run=Selch1 parm='1 2 3 4'
        passlib newpool suspend>
    <choice checkvar=xtest1 match=c>
      Selection #2 (Command Selch2)
      <action run=Selch2 parm=1234>
    <choice checkvar=xtest1 match=d>
      Selection #3 (Command Selch3)
      <action run=Selch3 parm=abcd>
    <choice checkvar=xtest1 match=e>
      Selection #4 (Command Selch4)
      <action run=Selch4 parm='a b c d'>
    <chdiv>
    <choice selchar=X>
      Exit
      <action run=exit type=exit>
    </selfld>
  <cmdarea>
</panel>
```

Figure 94 on page 209 shows the formatted result.

```

                                Sample Selection Panel with CHDIV tag
Option ==> -----

This is a selection panel.

Select an
option . . 0  Selection #0 (Command Selch0)
           1  Selection #1 (Command Selch1)
           2  Selection #2 (Command Selch2)
           3  Selection #3 (Command Selch3)
           4  Selection #4 (Command Selch4)

           X  Exit

Option ==> -----
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel
```

Figure 94. Choice divider

CHECKI (Validity Check Item)

The CHECKI tag defines a test of an input value.

Syntax

►► <CHECKI — TYPE= — First set of keywords ►◄

First set of keywords

Parameters

TYPE=

This attribute specifies the type of check to be performed. The valid types are:

RANGE

This allows you to check for an integer value within a range. The specified range includes the end points. The range delimiters can include 16 digits. The range delimiters can also contain a sign (- or +). If no sign is coded, the value is assumed to be positive.

Important: In ISPF, the VER(variable RANGE,lower,upper) statement limits the length of the specified variable to 16 digits. If you specify the CHECKI TYPE=RANGE on a variable that is longer than 16 positions, the variable may not be correctly validated. For example, assume an application developer defines a data field with a length of 20 and defines this validity check for the field:

```
<CHECKI TYPE=RANGE PARM1=1 PARM2=9999999999999999>
```

If the number 12345678901234567890 were entered into the field, only the first 16 digits would be verified and the number would be within the defined range, even though the entire number entered is outside of the defined range.

PARM1=low-bound | %varname

This attribute supplies the low value, if any or the name of a variable that contains the low value. If you do not supply a value, the default is "-" followed by sixteen 9s (that is, -9999...99). Negative values must be coded with the minus sign on the left.

ISPF restrictions on the VER(variable RANGE,lower,upper) apply. The lower value specified in PARM1 can be positive or negative. The length of the lower limit is limited to 16 digits, in addition to the plus or minus sign if used.

PARM2=high-bound | %varname

This attribute supplies the high value, if any or the name of a variable that contains the high value. If you do not supply a value, the default is sixteen 9s (that is, 9999...99). Negative values must be coded with the minus sign on the left.

ISPF restrictions on the VER(variable RANGE,lower,upper) apply. The upper value specified in PARM2 can be positive or negative. The length of the upper limit is limited to 16 digits, in addition to the plus or minus sign if used.

ALPHA

This limits the character set to A-Z, a-z, and #, \$, @. The conversion utility builds the VER(variable ALPHA) statement.

CHARS

Specifies the CHARS check of characters allowed within an input string.

The conversion utility uses the TYPE=CHARS check to support ISPF VER(variable BIT), VER(variable HEX) and VER(variable NUM). BIT, HEX, and NUM are the only types of support provided by ISPF for the TYPE=CHARS check.

PARM1=EQ

This attribute contains EQ to specify that PARM2 contains a value that must be matched. If PARM1 contains any other value, the check is ignored and the conversion utility issues a warning message.

PARM2=character-set

This attribute specifies a set of characters to be matched.

- If TYPE=CHARS, PARM1='EQ', and PARM2='01', the conversion utility generates VER(variable BIT).
- If TYPE=CHARS, PARM1='EQ', and PARM2='0123456789ABCDEFabcdef', the conversion utility generates VER(variable HEX).
- If TYPE=CHARS, PARM1='EQ', and PARM2='0123456789', the conversion utility generates VER(variable NUM).

Note: If one of these options is used, the PARM2 value must be exactly as specified. If PARM2 contains any other value, the check is ignored and the conversion utility issues a warning message.

VALUES

Specifies that the value entered must be the same as one of the values specified in PARM2.

The VER LIST statement built by the conversion utility is case-sensitive to the values entered in PARM2 (no folding of PARM2 to uppercase). This means that ISPF looks for an exact match in the verification process. You can specify XLATL FORMAT=UPPER within the variable class definition before the CHECKL tag to convert user input to uppercase before the VALUES check is processed.

PARM1=EQ

This attribute contains EQ to specify that PARM2 contains values that must be matched. If PARM1 contains any other value, the check is ignored and the conversion utility issues a warning message.

PARM2=value-list

This attribute specifies the list of values. If the list contains more than one value, it must be enclosed in quotes. If a value in the list contains blanks, then it must be enclosed in single quotes and the entire list enclosed in double quotes. Each value in the list must be separated by blanks or enclosed quotes. For example:

```
dog
'dog cat bird lion'
"parsley onion 'black pepper' garlic"
```

The maximum number of values allowed is 100.

Note: You should surround the entire value for PARM2 with double quotes and then surround any value in the list that contains blanks with single quotes. Double quotes surrounding a list are supported by the conversion utility.

The conversion utility generates VER(variable LIST,value-list) from PARM2.

VALUESX

Specifies that the value entered cannot be any of the values specified in PARM2. This is the opposite of VALUES.

The VER LISTX statement built by the conversion utility is case-sensitive to the values entered in PARM2 (no folding of PARM2 to uppercase). This means that ISPF looks for an exact match in the verification process. You can specify XLATL FORMAT=UPPER within the variable class definition before the CHECKL tag to convert user input to uppercase before the VALUES check is processed.

PARM1=NE

this attribute contains ne to specify that parm2 contains values that cannot be entered. If parm1 contains any other value, the check is ignored and the conversion utility issues a warning message.

PARM2=VALUE-LIST

This attribute specifies the list of values. If the list contains more than one value, it must be enclosed in quotes. If a value in the list contains blanks, then it must be enclosed in single quotes and the entire list enclosed in double quotes. Each value in the list must be separated by blanks or enclosed quotes. For example:

```
dog
'dog cat bird lion'
"parsley onion 'black pepper' garlic"
```

The maximum number of values allowed is 100.

Note: You should surround the entire value for PARM2 with double quotes and then surround any value in the list that contains blanks with single quotes. Double quotes surrounding a list are supported by the conversion utility.

The conversion utility generates VER(variable LISTX,value-list) from PARM2.

BIT

Specifies that the variable must be all 0's and 1's. The conversion utility builds the VER(variable BIT) statement.

NAME

Specifies that the variable must contain a valid name, following the rules of member names. The conversion utility builds the VER(variable NAME) statement.

NAMEF

Specifies that the variable must contain a valid name filter. The conversion utility builds the VER(variable NAMEF) statement.

PICT

Specifies that the variable must contain characters that match the corresponding type of character in *pictstring*.

PARM1=EQ

This attribute contains EQ to specify that PARM2 contains values that must be matched. If PARM1 contains any other value, the check is ignored and the conversion utility issues a warning message.

PARM2=pictstring

This 'pictstring' parameter must be the actual value to be used in the generated VER statement. ISPF does not support a variable name for this value.

If PARM2 contains invalid characters as defined by ISPF, the check is ignored and the conversion utility issues a warning message.

The conversion utility builds the VER(variable PICT,pictstring) statement.

PICTCN

Specifies that the variable must contain specific constants along with other standard ISPF picture verification characters.

PARM1=mask-character

The mask-character is any special character that represents itself. It cannot be one of the ISPF picture string characters (C,A,N,X,9,c,a,n,x)

PARM2=field-mask

The field-mask provides the required characters for the field. All other field positions must be represented by the mask-character. For example, if the field is to contain a string VnnRnnMnn (for Version, Release, and Modification) and the mask-character is an asterisk (*), the field mask is V**R**M**.

PARM3=string

The string must be the same length as the field-mask. It contains all of the required characters in the same positions as the field-mask. The positions defined with the mask-character in the field-mask contain one of the standard ISPF picture characters (C,A,N,X,9,c,a,n,x). To complete the example provided for PARM2, the string is VnnRnnMnn. The resulting verification statement is:

```
VER(variable,PICTCN,*,V**R**M**,VnnRnnMnn)
```

The variable is verified for **V** in position 1, **R** in position 4, **M** in position 7, and numeric characters in positions 2,3,5,6,8, and 9.

The conversion utility builds the VER(variable,PICTCN,mask-character,field-mask,string) statement.

NUM

Specifies that the variable must contain all numeric characters (0-9). The conversion utility builds the VER(variable NUM) statement.

DBCS

Specifies that the variable must contain all valid DBCS characters. The conversion utility builds the VER(variable DBCS) statement.

LISTV

Specifies that the variable must be one of the values provided by *varlist*.

PARM1=EQ

This attribute contains EQ to specify that PARM2 contains values that must be matched. If PARM1 contains any other value, the check is ignored and the conversion utility issues a warning message.

PARM2=%varlist

This attribute must be a variable name entered with "%" as the first character. The variable contents are provided by the application and must be a list of valid values.

The conversion utility builds the VER(variable LISTV,&varlist) statement.

LISTVX

Specifies that the variable cannot be any of the values provided by *varlist*. This is the opposite of LISTV.

PARM1=NE

This attribute contains NE to specify that PARM2 contains values that cannot be entered. If PARM1 contains any other value, the check is ignored and the conversion utility issues a warning message.

PARM2=%VARLIST

This attribute must be a variable name entered with "%" as the first character. The variable contents are provided by the application and must be a valid list of excluded values.

The conversion utility builds the VER(variable LISTVX,&varlist) statement.

ALPHAB

Specifies that the variable must be all alphabetic characters (A-Z or a-z). The conversion utility builds the VER(variable ALPHAB) statement.

LEN

Specifies that the variable must satisfy the condition expressed by the relational operator and the expected length.

PARM1=operator | %varname

This attribute can be a relational operator (EQ, LT, GT, LE, GE, NE, NG, or NL) or a variable name that contains a relational operator. If a variable name is entered, it must be preceded by a "%".

PARM2=length | %varname

The parameter must be either a number or a variable name. If a number is entered, it must be in the range of 1-99999. If a variable name is entered, it must be preceded by a "%".

The conversion utility builds the VER(variable operator,length) statement.

EBCDIC

Specifies that the variable must contain all valid EBCDIC characters. The conversion utility builds the VER(variable EBDIC) statement.

ENUM

Specifies that the variable can contain extended numeric notation. The conversion utility builds the VER(variable ENUM) statement.

DSNAME

Specifies that the variable must contain a valid TSO data set name. The conversion utility builds the VER(variable DSNAME) statement.

DSNAMEF

Specifies that the variable must contain a valid TSO data set name filter. The optional member name cannot be specified as a member pattern. A missing close parentheses and ending quotation mark are automatically added by ISPF. The conversion utility builds the VER(variable DSNAMEF) statement.

DSNAMEFM

Specifies that the variable must contain a valid data set name. The optional member name can be specified as a member pattern. A missing close parentheses and ending quotation mark

are automatically added by ISPF. The conversion utility builds the VER(variable DSNAMEFM) statement.

DSNAMEPQ

Specifies that the variable must contain a valid TSO data set name. A missing close parentheses and ending quotation mark are automatically added by ISPF. The conversion utility builds the VER(variable DSNAMEPQ) statement.

DSNAMEQ

Specifies that the variable must contain a valid TSO data set name. A missing ending quotation mark is automatically added by ISPF. The conversion utility builds the VER(variable DSNAMEQ) statement.

MIX

Specifies that the variable must contain all valid DBCS and EBCDIC characters. The conversion utility builds the VER(variable MIX) statement.

HEX

Specifies that the variable must contain all hexadecimal characters (0-9, a-f or A-F). The conversion utility builds the VER(variable HEX) statement.

FILEID

Specifies that the variable must contain a valid file ID in CMS syntax. The conversion utility builds the VER(variable FILEID) statement.

See the [*z/OS ISPF Dialog Developer's Guide and Reference*](#) for additional information concerning panel variable validation.

INCLUDE

Specifies that the variable must contain valid characters from at least one of the ISPF-defined VER groups ALPHA, ALPHAB or NUM.

PARM1=IMBLK

This attribute contains IMBLK to specify that the IMBLK keyword be added to the generated VER statement. If PARM1 contains any other value, it is reset to the value IMBLK.

PARM2=ALPHA | ALPHAB | NUM

This attribute must contain either the value ALPHA, ALPHAB, or NUM. If PARM2 is not specified or contains any other value, the INCLUDE check is ignored and the conversion utility issues a warning message.

PARM3=ALPHA | ALPHAB | NUM

This attribute must contain either the value ALPHA, ALPHAB, or NUM. The value specified for PARM3 should be different than the value specified for PARM2. If the values for PARM2 and PARM3 are the same, the PARM3 value is ignored and the conversion utility issues a warning message.

The conversion utility builds the VER(variable INCLUDE [,IMBLK], parm2 [,parm3]) statement.

IDATE

Specifies that the variable must contain a valid 8 character international date. The conversion utility builds the VER(variable,IDATE) statement.

STDDATE

Specifies that the variable must contain a valid 10 character standard date. The conversion utility builds the VER(variable,STDDATE) statement.

JDATE

Specifies that the variable must contain a valid 6 character Julian date. The conversion utility builds the VER(variable,JDATE) statement.

JSTD

Specifies that the variable must contain a valid 8 character standard Julian date. The conversion utility builds the VER(variable,JSTD) statement.

ITIME

Specifies that the variable must contain a valid 5 character international time. The conversion utility builds the VER(variable,ITIME) statement.

STDTIME

Specifies that the variable must contain a valid 8 character standard time. The conversion utility builds the VER(variable,STDTIME) statement.

IPADDR4

Specifies that the variable must contain a valid 15-position IP address. The conversion utility builds the VER(variable,IPADDR4) statement.

Comments

The CHECKI tag defines a test of an input value. Validity checking occurs only on input.

Restrictions

- You must code the CHECKI tag within a CHECKL definition. The conversion utility supports only one CHECKI within each CHECKL definition. If multiple CHECKI definitions are coded within a single CHECKL definition, the additional CHECKI tags are ignored and are not syntax checked. See [“CHECKL \(Validity Check List\)”](#) on page 218 for a complete description of this tag.
- The conversion utility builds a VER statement in the ISPF)PROC section of the panel definition for a CHECKI tag.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names”](#) on page 179.

Processing

None.

Examples

In this example, variables associated with the variable class *aa* must have a value that contains only alphabetic characters. Values associated with the variable class *bb* can only be SINGLE or DOUBLE.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=aa TYPE='char 18'>
  <CHECKL MSG=msgf881>
    <CHECKI TYPE=ALPHA>
  </CHECKL>
</VARCLASS>

<VARCLASS NAME=bb TYPE='char 6'>
  <XLATL FORMAT=upper>
</XLATL>
  <CHECKL MSG=msgf883>
    <CHECKI TYPE=VALUES PARM1=EQ PARM2="SINGLE DOUBLE">
  </CHECKL>
</VARCLASS>

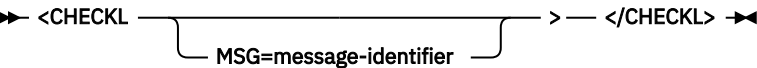
<VARLIST>
  <VARDCL NAME=checka VARCLASS=aa>
  <VARDCL NAME=checkb VARCLASS=bb>
</VARLIST>

<PANEL NAME=checki>CHECKI audits
  <DTAFD DATAVAR=checka ENTWIDTH=18 PMTWIDTH=20>Enter Last Name
  <DTAFD DATAVAR=checkb ENTWIDTH=6 PMTWIDTH=20>Enter Room Type
  <CMDAREA>
</PANEL>
```

CHECKL (Validity Check List)

The CHECKL tag defines a validity check for input variables.

Syntax



Parameters

MSG=message-identifier

This attribute identifies the message to be issued if the value fails the embedded test. The conversion utility adds this *message-identifier* to the VER statement generated by the enclosed CHECKI tag. If you do not specify your own message, ISPF issues a message specified on the enclosing VARCLASS tag or the default message associated with the type of VER statement generated. See “MSG (Message)” on page 352 for information about creating messages.

Comments

The CHECKL tag defines a validity check for input variables. The CHECKI tag coded within the check list performs the validation test.

Field validity checking follows standard ISPF conventions based on the verification statements generated. For details, see “CHECKI (Validity Check Item)” on page 210.

Restrictions

- The CHECKL tag requires an end tag.
- You must code the CHECKL tag within a VARCLASS definition. See “VARCLASS (Variable Class)” on page 445 for a complete description of this tag.
- The CHECKL tag must be coded after all XLATL tags in the same variable class.

Processing

Table 12. Tags you can code within a CHECKL definition			
Tag	Reference	Usage	Required
CHECKI	“CHECKI (Validity Check Item)” on page 210	Single	No

Examples

Here is source file markup that contains two variable classes that each contain a validity check list. The second variable class also contains a translate list that translates variable values to uppercase. Notice that the translate list is coded in the variable class before the validity check list.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=aa TYPE='char 18'>
  <CHECKL MSG=msgf881>
    <CHECKI TYPE=ALPHA>
  </CHECKL>

<VARCLASS NAME=bb TYPE='char 6'>
  <XLATL FORMAT=upper>
</XLATL>
  <CHECKL MSG=msgf883>
    <CHECKI TYPE=VALUES PARM1=EQ PARM2="SINGLE DOUBLE">
  </CHECKL>

<VARLIST>
  <VARDCL NAME=checka VARCLASS=aa>
  <VARDCL NAME=checkb VARCLASS=bb>
</VARLIST>

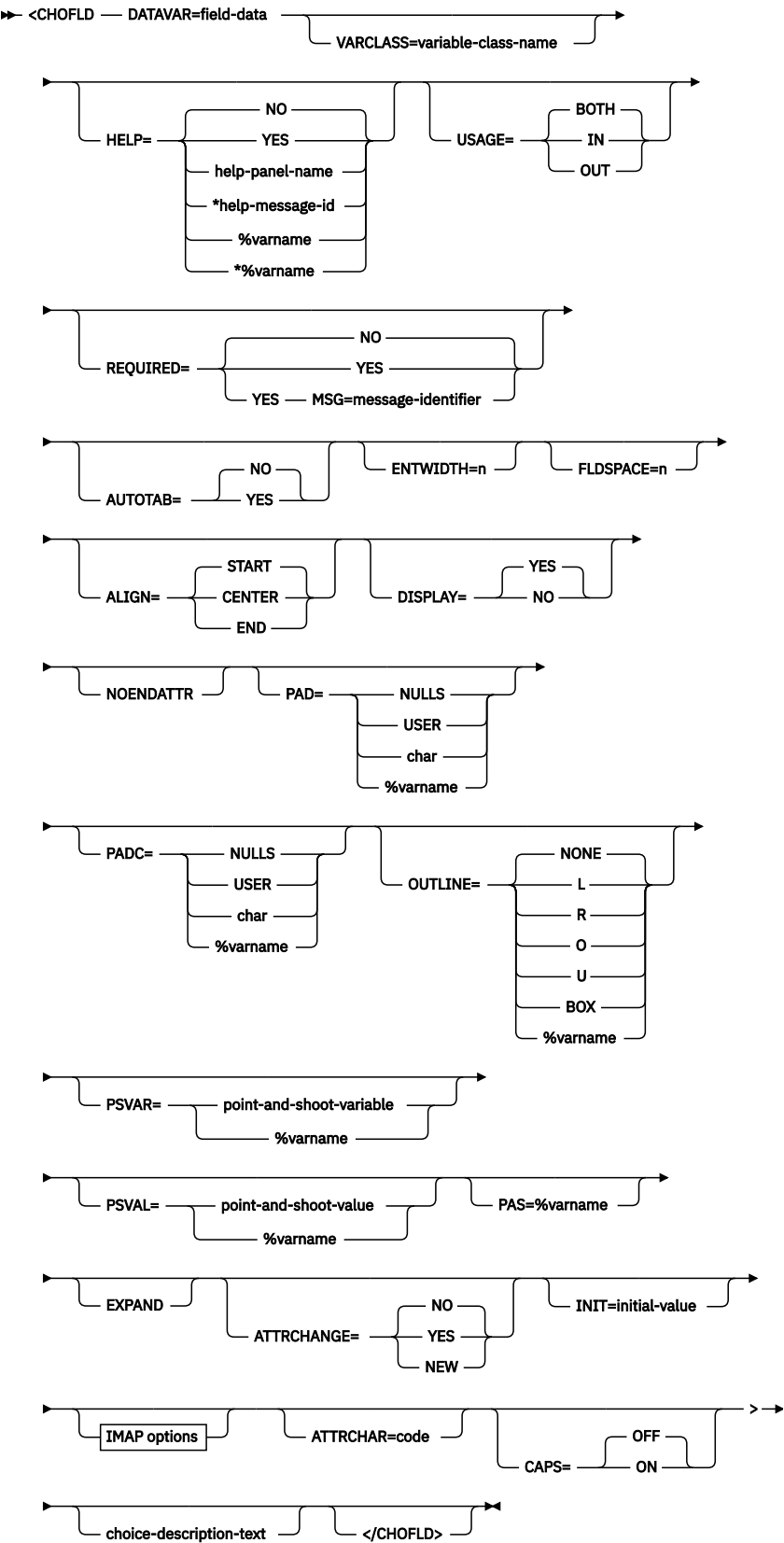
<PANEL NAME=check1>CHECKL audits
  <DTAFLD DATAVAR=checka ENTWIDTH=18 PMTWIDTH=20>Enter Last Name
  <DTAFLD DATAVAR=checkb ENTWIDTH=6 PMTWIDTH=20>Enter Room Type
  <CMDAREA>
</PANEL>

```

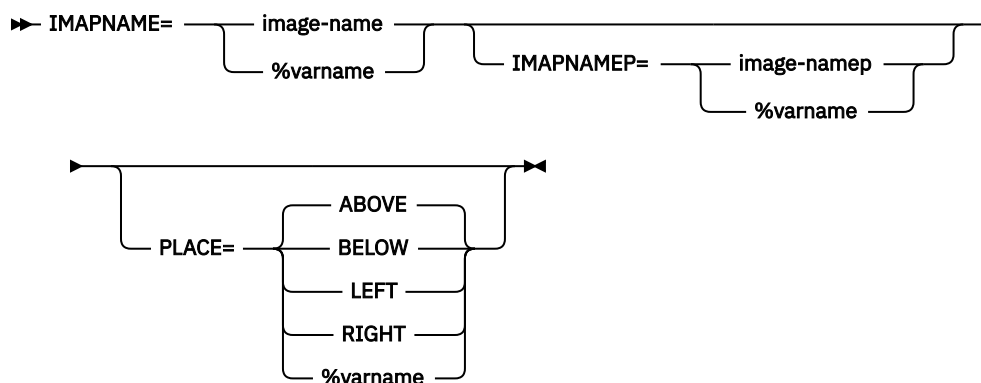
CHOFLD (Choice Data Field)

The CHOFLD tag defines an input field, an output field, or an input/output field within the text of a CHOICE tag.

Syntax



IMAP options



Parameters

DATAVAR=field-data

This attribute specifies the variable name for the data in the field. The value coded must be a variable-name without the leading % notation.

VARCLASS=variable-class-name

This attribute specifies the name of the variable class, defined using a VARCLASS tag, that overrides the default variable class referred to by the VARDCL that declared the data variable for this field.

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for this choice data field. This is field-level help.

When HELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help for the choice data field and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

USAGE=BOTH | IN | OUT

This attribute indicates whether the field is for input only, output only, or both.

REQUIRED=NO | YES

This attribute indicates if the field requires input. This attribute is valid only when USAGE=IN or BOTH.

If REQUIRED=YES is coded, a VER(variable, NONBLANK) statement is built by the conversion utility and placed in the)PROC section of the ISPF panel generated.

MSG=message-identifier

This attribute specifies the message that is displayed when the user does not complete a required entry (defined with the REQUIRED attribute). If you do not specify a *message-identifier*, ISPF displays a default message.

If you specify the MSG attribute and REQUIRED=YES, a VER(variable, NONBLANK, MSG=message-identifier) statement is built by the conversion utility and placed in the)PROC section of the ISPF panel generated. If you specify the MSG attribute and REQUIRED=NO (the default), the conversion utility issues a warning message.

See [“MSG \(Message\)”](#) on page 352 for information about creating messages.

Note: You can specify messages pertaining to other validations using XLATL and CHECKL tags within a VARCLASS definition. See the descriptions of these tags for additional information.

AUTOTAB=NO | YES

When AUTOTAB=YES, the cursor moves to the next field capable of input when the user enters the last character in this field. If no other field capable of user input exists on the panel, the cursor returns to the beginning of this field.

AUTOTAB=YES is valid only when the value for USAGE is either BOTH or IN. If specified, this attribute overrides the AUTOTAB value of the DTACOL tag.

ENTWIDTH=n

This attribute specifies the number of bytes used for the choice data field. The minimum width is 1 and the maximum is the remaining available panel width, less the required amount of space for field attributes. If ENTWIDTH is not provided on either the CHOFLD tag or the enclosing DTACOL tag, the conversion utility uses the width determined by the TYPE value of the associated VARCLASS.

If specified, this attribute overrides the ENTWIDTH value of the DTACOL tag.

FLDSPACE=n

This attribute specifies the number of bytes reserved for the choice data field. The minimum width is 2 and the maximum is the remaining available panel (or region) width. The FLDSPACE value should include the actual entry width plus the number of entry field attributes. If the value specified by ENTWIDTH is less than the specified FLDSPACE value, the entry field is padded with blanks to the FLDSPACE value. This creates blank space between the entry field and following *choice-description-text* and allows you to align description text from successive CHOFLD definitions.

If specified, this attribute overrides the FLDSPACE value of the DTACOL tag.

ALIGN=START | CENTER | END

This attribute specifies the alignment of data within the display field after all translations have been performed. Use this attribute to align the data with the start, the end, or the center of the display field.

This is accomplished in the conversion utility by using an attribute character for the field that specifies JUST(LEFT) if ALIGN=START or JUST(RIGHT) if ALIGN=END. ALIGN=CENTER uses an attribute character for the field that specifies JUST(ASIS).

Alignment occurs if the transformed value of the dialog variable is shorter than the display width of the field. When ALIGN=END, no underscore is padding performed. Instead, blanks are used.

DISPLAY=YES | NO

This attribute specifies whether data displays on the screen as the user types it in. If you specify NO, the data is not displayed. This attribute is useful when creating fields for passwords or other information which you do not want to appear on the screen.

NOENDATTR

This attribute, which is valid only when WINDOW=NO is specified on the PANEL tag or DIR=HORIZ is specified on the REGION tag, specifies that no ending attribute is placed after the choice data field. NOENDATTR is ignored for the last field on each panel line unless WINDOW=NO has been specified on the PANEL tag. NOENDATTR is valid only when the CHOFLD tag is followed by a CHOFLD, CHOICE, or CHDIV tag.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a "%".

If specified, this attribute overrides the PAD value of the DTACOL tag.

PADC=NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a "%".

If specified, this attribute overrides the PADC value of the DTACOL tag.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

If specified, this attribute overrides the OUTLINE value of the DTACOL tag.

PSVAR=point-and-shoot-variable | %varname

This attribute provides the name of a variable that is to be set when a CHOFLD is clicked on for point-and-shoot selection. You can define this attribute as a variable name preceded by a “%”.

The *point-and-shoot-variable* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

PSVAL=point-and-shoot-value | %varname

This attribute provides the value to be placed in the field specified by the PSVAR attribute. You can define this attribute as a variable name preceded by a “%”. To specify a blank value, the " ' " (quotation mark, apostrophe, blank, apostrophe, quotation mark) coding notation should be used.

PAS=%varname

This attribute can be used to provide a variable name to specify ON or OFF for point-and-shoot. When PSVAR and PSVAL have been specified without the PAS attribute, the point-and-shoot field is automatically enabled.

EXPAND

The EXPAND attribute, used in combination with EXPAND=xy on the PANEL definition, causes the expand characters to be added to the CHOFLD definition, effectively allowing ENTWIDTH to expand. The ENTWIDTH value must be specified as 4 or greater for the EXPAND function to operate.

Note: If the PANEL tag attribute EXPAND is defined with no value specified, the CHOFLD tag EXPAND attribute is not used.

ATTRCHANGE=NO | YES | NEW

When ATTRCHANGE=YES or ATTRCHANGE=NEW, the conversion utility formats an additional entry in the panel)ATTR section (that can apply to multiple data fields) instead of creating a unique .ATTR(field-name) entry in the)INIT section for this field. With this option, multiple CHOFLD tags with the same characteristics require fewer panel logic statements. ATTRCHANGE=NEW creates a new entry. ATTRCHANGE=YES uses an existing entry, if possible.

INIT=initial-value

When the INIT attribute is specified, the conversion utility adds a statement to the panel)INIT section to initialize the field to the *initial-value*.

IMAPNAME= image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAMEP=image-namep | %varname

The attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PLACE=ABOVE | BELOW | LEFT | RIGHT

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

ATTRCHAR=code

This attribute can be a single character or a two-position entry of valid hex digits. If you enter an incorrect value, a warning message is issued and the value is set to null. Hex entries are converted to character. Hex values '00'-'2F' are reserved for use by the conversion utility.

CAPS=OFF | ON

When CAPS=ON, the data in the field is displayed in uppercase characters.

choice-description-text

This is the text for the choice data field. The *choice-description-text* appears following the field.

Comments

The CHOFLD tag is similar to the DTAFLD tag. When the enclosing SELFLD tag is defined within a DTACOL tag, the CHOFLD tag uses default values defined by the DTACOL tag in the same way as the DTAFLD tag.

The CHOFLD tag defines an input field, an output field, or an input/output field within CHOICE tag description text on an application panel.

The formatted width of the field is 2 positions more than the ENTWIDTH value to provide for an attribute byte both before and after the field.

When the maximum number of requested attributes for a panel is exceeded, the conversion utility issues error message ISPC804E. The number of requested attributes includes attribute override entries. These are .ATTR entries that are added by the conversion utility for attributes that are specified on CHOFLD, DTACOL, DTAFLD, LSTCOL, and LSTFLD tags. If the same set of attributes is specified on multiple tags, duplicate .ATTR entries are added by default. Adding the parameter ATTRCHANGE=YES to the tags causes the compiler to instead add a single entry in the panel)ATTR section for each unique set of attributes specified. The entry for a set of attributes is then shared by all tags that specify that set of attributes.

Restrictions

- You must code the CHOFLD tag within a CHOICE tag definition. The CHOFLD tag can be placed anywhere within the *choice-description-text*. See [“CHOICE \(Selection Choice\)” on page 226](#) for a description of this tag.
- The variable name specified in the DATAVAR attribute should have an associated VARDCL definition. See [“VARDCL \(Variable Declaration\)” on page 449](#) for a complete description of this tag.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the %varname entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

Table 13. Tags you can code within a CHOFLD definition			
Tag	Reference	Usage	Required
ACTION	“ACTION (Action)” on page 184	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is source file markup that contains an application panel that is similar to the example for the CHOICE tag. In this example, the first selection field is modified to illustrate the CHOFLD tag. The first choice includes a panel input/output field named *cardtype* which must be completed when the *new* choice is selected.

Notice that the reference CHOICE source file has been modified to:

- Add a VARCLASS for the *cardtype* field *before* the include file which has both VARCLASS and VARDCL tags.
- Add a VARDCL for the *cardtype* field *after* the include file which has both VARCLASS and VARDCL tags.
- Add the CHOFLD tag to define the choice data field.
- Add a DTACOL tag definition to allow for a SOURCE tag that provides an audit of *cardtype* only when *new* is selected.

Figure 95 on page 226 shows the formatted result.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>

<VARCLASS NAME=char9cls TYPE='char 9'>
  <XLATL FORMAT=upper>
  </XLATL>

&sampvar1;

<VARLIST>
  <VARDCL NAME=cardtype VARCLASS=char9cls>
</VARLIST>

<PANEL NAME=chofld KEYLIST=keylxmlp>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST>Type in patron's name and card number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
  <DTAFLD DATAVAR=curdate PMTWIDTH=12 ENTWIDTH=8 USAGE=out>Date
  <DTAFLD DATAVAR=cardno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=25>Card No
    <DTAFLDD>(A 7-digit number)
  <DTAFLD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Name
    <DTAFLDD>(Last, First, M.I.)
  <DTAFLD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
  <DIVIDER>
  <REGION DIR=horiz>
  <SELFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
    one of the following
    <CHOICE CHECKVAR=card MATCH=new>
      New      Type:
      <CHOFLD datavar=cardtype autotab=yes entwidth=9>
        (Permanent or Temporary)
    <CHOICE CHECKVAR=card MATCH=renew>Renewal
    <CHOICE CHECKVAR=card MATCH=replace>Replacement
  </SELFLD>
  <DTACOL>
  <SOURCE>
  IF (&CARDBSEL = 1)
    VER(&CARDTYPE,NB,LIST,TEMPORARY,PERMANENT)
  </SOURCE>
  </DTACOL>
  <SELFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
    <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
    <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
    <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
    <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
  </SELFLD>
  </REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

```

File  Search  Help
-----
                        Library Card Registration

Type in patron's name and card number (if applicable).
Then select an action bar choice.

Date . . . : 
Card No. . . : _____ (A 7-digit number)
Name . . . : _____ (Last, First, M.I.)
Address . . : _____

Choose one of the following          Check valid branches
--  1. New      Type: Z              - North Branch
    (Permanent or Temporary)        - South Branch
    2. Renewal                               - East Branch
    3. Replacement                       - West Branch

Enter a command ==>
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel

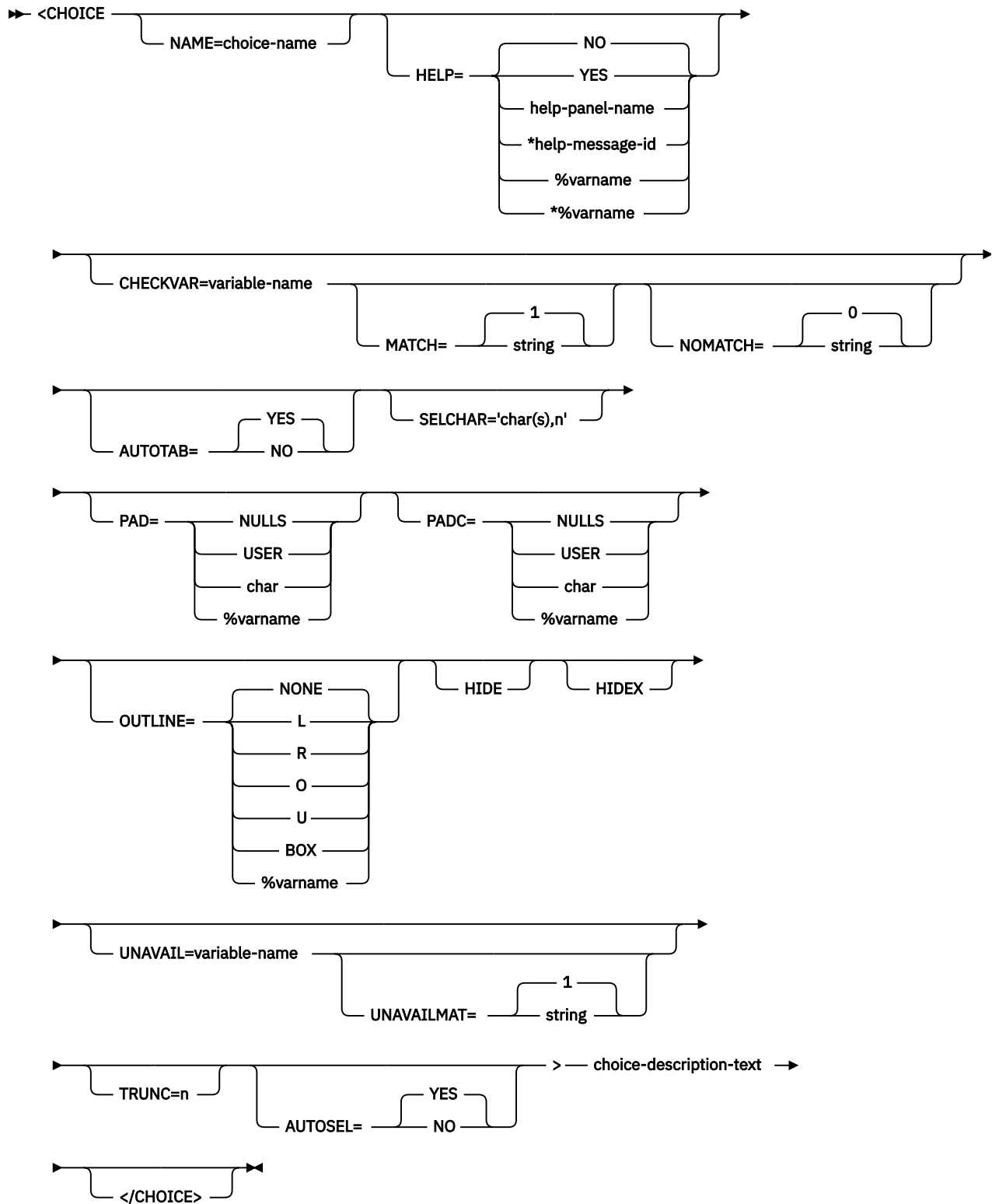
```

Figure 95. Choice data fields

CHOICE (Selection Choice)

The CHOICE tag defines information about a choice in a selection field.

Syntax



Parameters

NAME=choice-name

Specifies the name of the choice. The *choice-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

CHOICE

Note: This attribute is required for choices defined for a multiple-choice selection field because the *choice-name* is used as the input field for multiple choice selections.

For multiple-choice selection fields, the *choice-name* can also be used to position the cursor on the choice or to position a pop-up.

Note: This attribute is not supported by the conversion utility for single-choice selection fields. In this case, the NAME value of the SELFLD tag is used as the field name.

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests for a multiple-choice selection field. This is field-level help.

When HELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MULTI.

CHECKVAR=variable-name

This attribute defines a variable whose value indicates whether the choice is preselected when the selection field is displayed. If the value of the variable is equivalent to the *string* you specify with the MATCH attribute, the item is marked as selected when the panel displays.

The preselection indicator depends on the value of the TYPE attribute from the SELFLD tag.

Table 14. Host Display indicators for particular TYPEs		
TYPE	LISTTYPE	Host Display Indicator
MULTI	n/a	slash
SINGLE	(not used) RADIO LISTBOX DDLIS COMBO	Choice number Choice number Choice number Choice number Choice number
MENU	n/a	Choice number
MODEL	n/a	Choice number
TUTOR	n/a	Choice number

When the SELFLD tag has been specified with TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR, the CHOICE number (or SELCHAR value) is placed in the command line.

The *variable-name* is updated to the value specified by the MATCH attribute when the user selects the choice being defined. For multiple-choice selection fields (SELFLD TYPE=MULTI), if you do not select a choice, or you deselect a choice, the associated *variable-name* is set to the value of the NOMATCH attribute or to 0 if the NOMATCH attribute is not specified.

Use a different variable for *variable-name* than what has been specified for *choice-name*.

Do not use the same variable for the *variable-name* as you use for the *variable-name* specified for the SETVAR or TOGVAR attributes of the ACTION tag.

For single-choice selection fields (SELFLD TYPE=SINGLE), ISPF selection menus (SELFLD TYPE=MENU), edit model selection menus (SELFLD TYPE=MODEL), or tutorial selection menus (SELFLD TYPE=TUTOR), the *variable-name* should be the same for all of the choices. For multiple-choice selection fields (SELFLD TYPE=MULTI), the *variable-name* should be different for each choice.

The CHECKVAR attribute value must be specified without a leading % sign. The *variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

MATCH=1 | string

Defines the value for the check variable that causes the item to be preselected. The *string* can be any character string. MATCH=1 is the default.

NOMATCH=0 | string

Defines the value for setting the check variable when the item is not selected. NOMATCH=0 is the default.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MULTI.

AUTOTAB=YES | NO

When AUTOTAB=YES, the cursor moves to the next field capable of input when the user enters the last character in this field. If no other field capable of user input exists on the panel, the cursor remains on this field.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MULTI.

SELCHAR='char(s),n'

This attribute specifies an alphanumeric character(s) to be used as the selection menu, edit model selection menu, or tutorial selection menu choice in place of the normal numeric value automatically supplied by the conversion utility. The number of characters accepted is controlled by the ENTWIDTH attribute of the SELFLD tag. The *char(s)* value is used as coded, that is, it is not uppercase.

When the HIDE attribute is also specified, the number of characters to be used for the hidden choice selection may be specified as part of the SELCHAR attribute. If specified, the *n* value overrides the number of characters normally obtained from the ENTWIDTH attribute of the SELFLD tag. The *n* value can be a numeric value from 1 to the number of bytes provided as the *char(s)* value, or you can specify an “*” to tell the conversion utility to use all of the *char(s)* provided for the choice selection. The *n* value is ignored when the HIDE attribute is not specified.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MULTI.

PADC=NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MULTI.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MULTI.

HIDE

This attribute causes a choice entry for a single-choice, menu-choice, model-choice, or tutor-choice selection to be removed from the selection list display.

CHOICE

This allows the creation of a numbered selection list when the choice numbers are not continuous by adding a 'dummy' CHOICE tag at the appropriate place in the DTL source. The number assigned to the hidden CHOICE does not appear in the selection list. Normal)INIT and)PROC section entries are not affected.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=SINGLE, TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR.

HIDEX

This attribute causes a choice entry for a model-choice selection to be removed both from the selection list display and from the selection processing.

This attribute is used in combination with the TRUNC attribute and the SELCHAR attribute to supply an alternate CHOICE tag definition with an alternate hidden model selection keyword.

For example, if an edit model panel has a selectable description of "VER", but you also want to allow the full word "VERIFY" to select the same model, two CHOICE tags are required. The first one defines the choice with the text "VER". The alternate CHOICE uses the same SELCHAR information, adds the attribute HIDEX and TRUNC=3, and specifies the tag text as "VERIFY". The conversion utility uses the first definition to build the panel text and the selection processing statement and uses the alternate CHOICE to accept the entry "VERIFY" by truncating it to "VER".

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MODEL.

UNAVAIL=variable-name

This attribute defines a variable whose value indicates whether the choice is available when the selection field is displayed. If the value of the variable is equivalent to the *string* you specify with the UNAVAILMAT attribute (or to the default value "1"), the item is displayed as an unavailable choice.

UNAVAILMAT=1 | string

Defines the value for the UNAVAIL variable that causes the choice to be unavailable. The *string* can be any character string. UNAVAILMAT=1 is the default.

TRUNC=n

This attribute is used for model-choice selection to specify the minimum number of characters required to identify the model choice. If the TRUNC attribute is not specified, the entire model choice name must be used to identify the model selection.

Note: This attribute is valid only when the SELFLD tag has been specified with TYPE=MODEL.

AUTOSEL=YES | NO

This attribute is used for tutor-choice selection to control the automatic selection of this choice by tutorial processing. When AUTOSEL=NO, the choice is not automatically selected.

choice-description-text

The text of the selection choice.

Comments

The CHOICE tag defines a choice within a selection field. The behavior and appearance of the choice depends on whether it is coded within a single-choice, multiple-choice, or menu-choice selection field.

For menu-choice selection fields, the text is preceded by a number (not followed by a period), the input field is the command line, and the choice selection is displayed with the CUA type Normal Text (NT).

For a single-choice selection list:

- When the LISTTYPE attribute of the SELFLD tag is not specified, the text is preceded by a number (followed by a period), the conversion utility provides an input field before the first choice for entry of the number of the selected choice, and the choice selection is displayed with the CUA type Select Available Choices (SAC).
- When LISTTYPE=RADIO is specified on the SELFLD tag, the choice selection can be displayed as a radio button by a client that is using the JSON API.

The field name for single-choice selection fields is the value specified for the NAME attribute of the SELFLD tag. The default field name for an ISPF selection menu choice is the field name used to identify the command line, normally ZCMD.

The text of each choice in a multiple-choice selection field is preceded by an input field. The field name for multiple-choice selection fields is the value specified for the NAME attribute of the CHOICE tag.

You can define an action for each choice using the SETVAR or TOGVAR attribute in an ACTION tag associated with the choice. Typically, an application knows what choice was selected by the application user by the value in the selection field name. The CHOICE field name for a multi-choice selection is set to a "/" when control is returned to the application. The SELFLD field name contains the number of the choice for single choice selection when control is returned to the application. The command line variable name contains the number of a menu selection choice when control is returned to the application. Alternatively, the application can use the value of the check variable or use SETVAR or TOGVAR to set another named variable.

Restrictions

- You must code the CHOICE tag within a SELFLD definition. See [“SELFLD \(Selection Field\)” on page 421](#) for a complete description of this tag.
- If coded within a multiple-choice selection field (SELFLD TYPE=MULTI), the *choice-name* can have an associated VARDCL definition.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).
- If the *choice-description-text* contains HP (Emphasized Text) or RP (Reference Phrase) tags, the UNAVAIL attribute is ignored.

Processing

Table 15. The tags you can code within a CHOICE definition			
Tag	Reference	Usage	Required
ACTION	“ACTION (Action)” on page 184	Multiple	No
CHOFLD	“CHOFLD (Choice Data Field)” on page 219	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is application panel markup that contains two selection fields. The first is a single-choice selection field that can be preselected depending on the value assigned to the variable *card*. When *card* is equal to *new*, *renew*, or *replace*, the selection field's input data field is assigned a value of 1, 2, or 3, respectively; otherwise, it is not preselected and the input data field remains blank.

The second selection field is a multiple-choice selection field. This field can be preselected by assigning values to the variables *nth*, *sth*, *est* and *wst*. If the given variable equals 1, the corresponding selection

field is marked with a /. More than one of the choices may be selected. Any nonblank character in the choice entry-field selects that choice. Preselected choices can be deselected by typing a blank character over the field.

Figure 96 on page 232 shows the formatted result.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=choice1 KEYLIST=keylxml>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST>Type in patron's name and card number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
  <DTAFLD DATAVAR=curdate PMTWIDTH=12 ENTWIDTH=8 USAGE=out>Date
  <DTAFLD DATAVAR=cardno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=25>Card No
    <DTAFLDD>(A 7-digit number)
  <DTAFLD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Name
    <DTAFLDD>(Last, First, M.I.)
  <DTAFLD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
  <DIVIDER>
  <REGION DIR=horiz>
  <SELFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=card MATCH=new>New
  <CHOICE CHECKVAR=card MATCH=renew>Renewal
  <CHOICE CHECKVAR=card MATCH=replace>Replacement
</SELFLD>
  <SELFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
  <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
  <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
  <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
  <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
</SELFLD>
</REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

```
File Search Help
-----
                        Library Card Registration

Type in patron's name and card number (if applicable).

Then select an action bar choice.

Date . . . :
Card No. . . _____ (A 7-digit number)
Name . . . . _____ (Last, First, M.I.)
Address . . _____

Choose one of the following          Check valid branches
--  1.  New                          _ North Branch
    2.  Renewal                      _ South Branch
    3.  Replacement                  _ East Branch
                                    _ West Branch

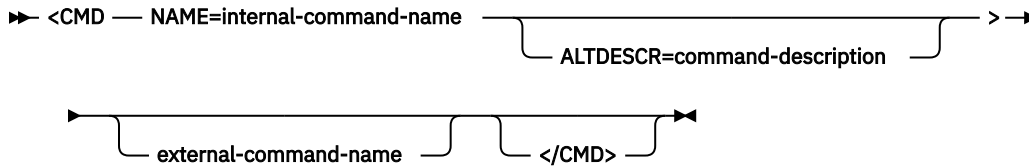
Enter a command ===> _____
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel
```

Figure 96. Selection field choices

CMD (Command Definition)

The CMD tag defines a command within an application command table.

Syntax



Parameters

NAME=internal-command-name

This attribute specifies an internal name for the command. The *internal-command-name* must have these characteristics:

- 2-8 single-byte characters in length
- The first (or only) character must be A-Z, a-z, @, #, or \$.
- Remaining characters, if any, can be A-Z, a-z, @, #, \$, —, or 0-9.

Lowercase characters are translated to their uppercase equivalents.

The *internal-command-name* is used in two ways:

- As the command table search criteria when:
 - A key defined in the current key list is pressed
 - A pull-down choice with an associated RUN action is selected
 - A command is entered in the command area of a panel.
- As the value passed to dialogs when the command action is PASSTHRU or SETVERB. See [“CMDACT \(Command Action\)”](#) on page 234 for more information about the PASSTHRU and SETVERB command actions.

ALTDESCR=command-description

This attribute provides a description of the command. It is placed in the ISPF variable ZCTDESC. The *command-description* text length is limited to 80 bytes.

external-command-name

Specifies the external name for this command.

Note: The *external-command-name* must be equal to the *internal-command-name*. You must use the *external-command-name* to support the ability provided by ISPF for truncated command entry and the T (truncation) tag. For more information, see [“T \(Truncation\)”](#) on page 437.

Comments

The CMD tag defines a command within an application command table. The defined command can be issued by an application user by entering the *internal-command-name* in the panel command area, or pressing a function key, or selecting a pull-down choice that references the command's *internal-command-name*. See [“KEYI \(Key Item\)”](#) on page 319 and [“ACTION \(Action\)”](#) on page 184 for additional information.

The action to be taken when a command is issued is defined with the CMDACT tag. See [“CMDACT \(Command Action\)”](#) on page 234 for information about defining command actions.

Restrictions

- The CMD tag must be coded within a CMDTBL definition. See [“CMDTBL \(Command Table\)”](#) on page 243 for a complete description of this tag.

Processing

Table 16. The tags you can code within a CMD definition			
Tag	Reference	Usage	Required
CMDACT	“CMDACT (Command Action)” on page 234	Single	Yes
T	“T (Truncation)” on page 437	Single	No

Examples

Here is source file markup that contains a command table that defines the commands UPDATE, ADD, DELETE, and SEARCH. The DELETE and UPDATE commands have defined truncations.

```
<!DOCTYPE DM SYSTEM>

<CMDTBL APPLID=conv>
  <CMD NAME=update>Upd<T>ate
    <CMDACT ACTION='alias add'>
  <CMD NAME=add>Add
    <CMDACT ACTION=setverb>
  <CMD NAME=delete>Del<T>ete
    <CMDACT ACTION=passthru>
  <CMD NAME=search>Search
    <CMDACT ACTION=passthru>
</CMDTBL>
```

This table shows the resultant ISPF application command table.

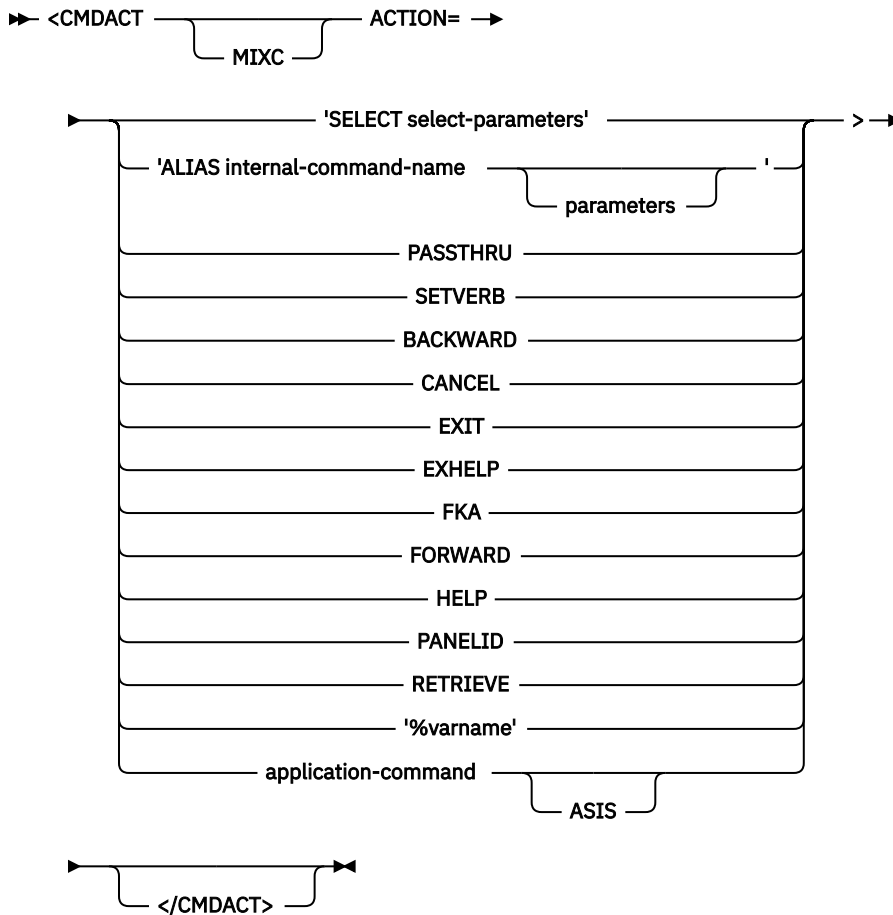
Table 17. ISPF application command table

ZCTVERB	ZCTTRUNC	ZCTACT
UPDATE	3	ALIAS ADD
ADD	0	SETVERB
DELETE	3	PASSTHRU
SEARCH	0	PASSTHRU

CMDACT (Command Action)

The CMDACT tag defines the action that occurs when the associated command is issued.

Syntax



Parameters

MIXC

Specifies that the following ACTION attribute is not to be converted to uppercase.

ACTION=

This attribute indicates the action that should be performed when the associated command is issued. The ACTION attribute value is limited to 240 characters. The value must be one of these:

SELECT select-parameters

Causes the ISPF SELECT service to be issued.

ALIAS internal-command-name

Provides an alternate way to express a command. For example, you can assign QUIT as an alias for the command EXIT.

The ALIAS *internal-command-name* has a maximum length of 8 characters.

In the command table, an alias must precede the command for which it is an alias.

You can create a chain of command aliases in a command table, as long as the result is a valid executable action. The last command and parameter values that ISPF encounters in the alias chain are the ones executed. The command and the parameter values do not necessarily come from the same command definition entry. For example:

Command Name

Command Action

EASYKEY

ALIAS CMD PARM1 PARM2

CMD

ALIAS CMD1 PARM3

CMD1

ALIAS CMD2

In this example, if the EASYKEY command is issued, the command that would ultimately be executed would be CMD2 PARM3.

parameters

If any ALIAS parameters are specified, they take precedence over any parameters included with the command when issued from a command line or the ACTION tag RUN attribute when a pull-down choice is selected.

If the ALIAS *internal-command-name* does not include parameters, ISPF accepts parameters from the command line or ACTION tag.

PASSTHRU

The PASSTHRU action causes the command and any parameters to be passed to the dialog program in the ZCMD dialog variable.

SETVERB

This is an alternate way to pass a command to the dialog. The SETVERB action causes the *internal-command-name* to be passed to the dialog in the ZVERB dialog variable. Any command parameters are passed in the ZCMD dialog variable.

BACKWARD

Specifies the ISPF system command BACKWARD as the command action.

CANCEL

Specifies the ISPF system command CANCEL as the command action.

EXIT

Specifies the ISPF system command EXIT as the command action.

EXHELP

Specifies the ISPF system command EXHELP as the command action.

FKA

Specifies the ISPF system command FKA as the command action.

FORWARD

Specifies the ISPF system command FORWARD as the command action.

HELP

Specifies the ISPF system command HELP as the command action.

PANELID

Specifies the ISPF system command PANELID as the command action.

RETRIEVE

Specifies the ISPF system command RETRIEVE as the command action.

%varname

You can specify a command action dynamically at run time by specifying the name of a variable (using % notation) for the ACTION attribute. If you specify a variable name, ISPF retrieves the action value when the command is issued. The variable value must be one of the actions previously listed.

The "%varname" entry must follow the naming conventions described in [“Rules for “%variable” names” on page 179.](#)

application-command

Specifies an application-unique command as the command action. The command action is created as an ALIAS unless the ASIS keyword is specified.

ASIS

Specifies that the application-unique command is to be created without the ALIAS designation.

Comments

The CMDACT tag defines the action that occurs when the associated command is issued.

Restrictions

- The CMDACT tag must be coded within the CMD definition it is associated with. See [“CMD \(Command Definition\)”](#) on page 233 for a complete description of this tag.
- You must specify the ACTION attribute on the CMDACT tag.

Processing

None.

Examples

Here is source file markup contains a command table that defines the commands UPDATE, ADD, DELETE and SEARCH. The ADD command sets the ZVERB variable equal to *add*. The DELETE command sets the ZCMD variable to *delete*. The UPDATE command is an alias for ADD.

```
<!DOCTYPE DM SYSTEM>

<CMDTBL APPLID=conv>
  <CMD NAME=update>Upd<T>ate
    <CMDACT ACTION='alias add'>
  <CMD NAME=add>Add
    <CMDACT ACTION=setverb>
  <CMD NAME=delete>Del<T>ete
    <CMDACT ACTION=passthru>
  <CMD NAME=search>Search
    <CMDACT ACTION=passthru>
</CMDTBL>
```

This table shows the resultant ISPF application command table.

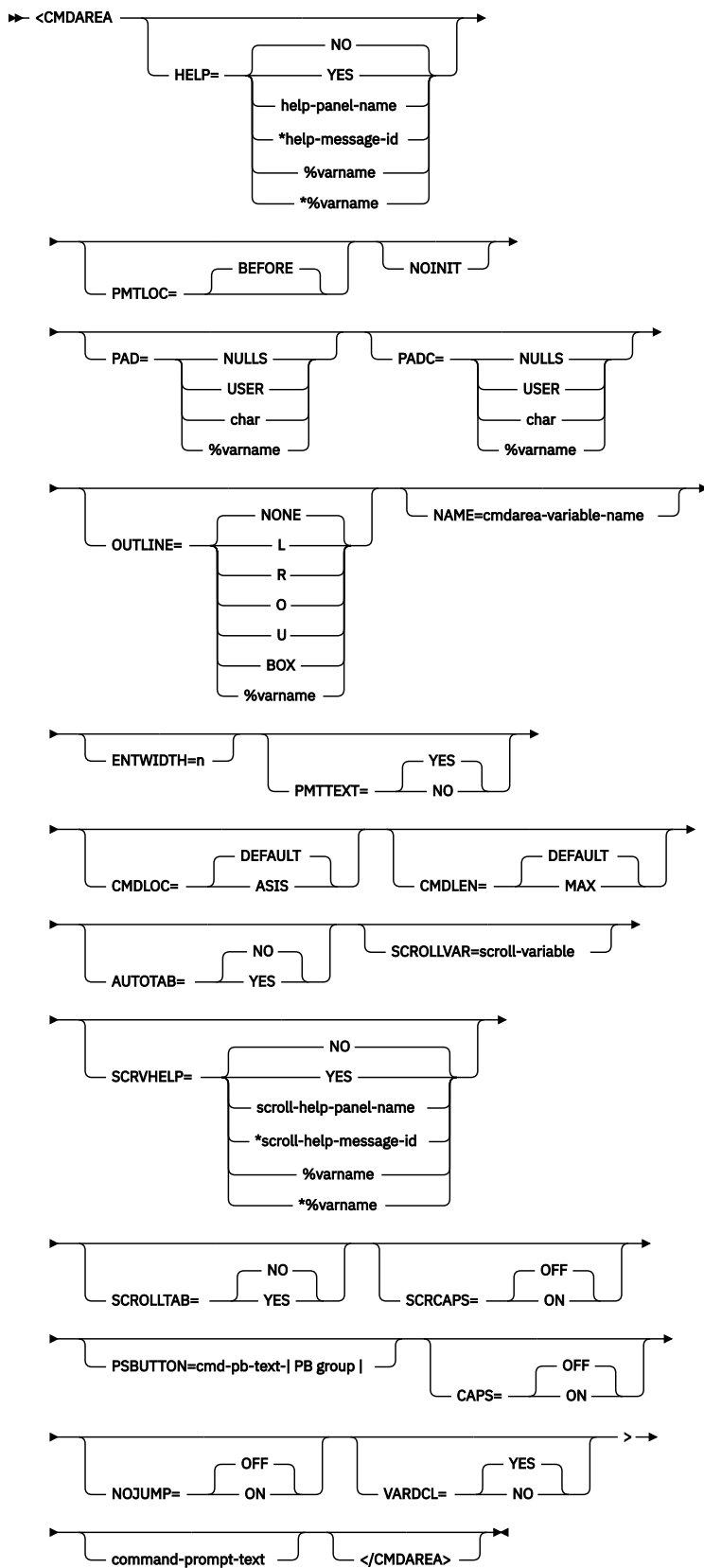
Table 18. ISPF application command table

ZCTVERB	ZCTTRUNC	ZCTACT
UPDATE	3	ALIAS ADD
ADD	0	SETVERB
DELETE	3	PASSTHRU
SEARCH	0	PASSTHRU

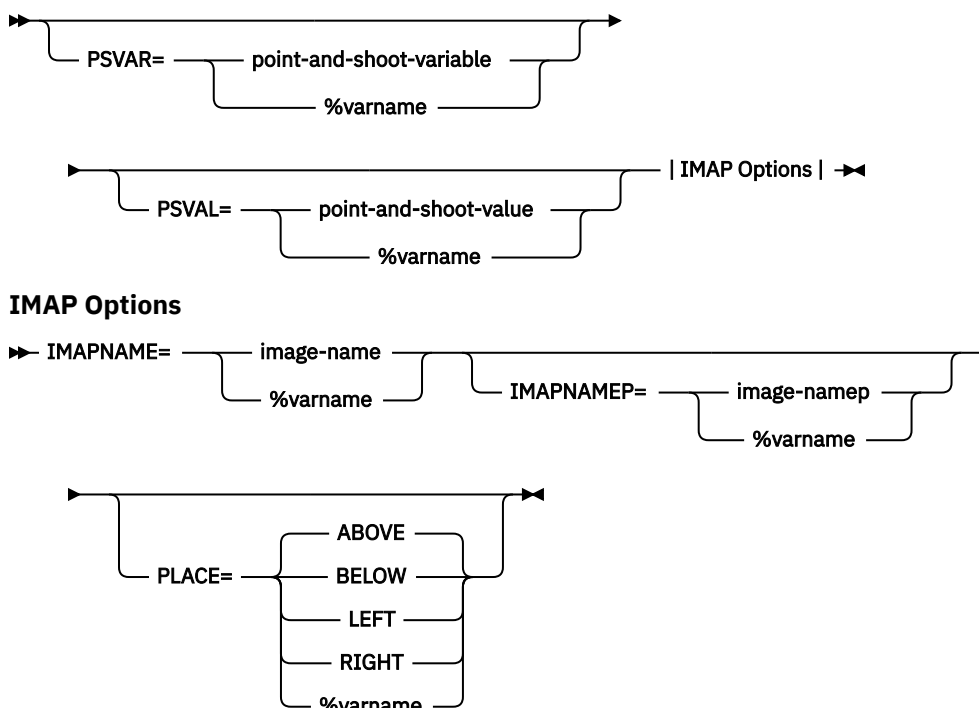
CMDAREA (Command Area)

The CMDAREA tag defines a command entry area on an application panel.

Syntax



PB Group



IMAP Options

Parameters

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for the command area.

When HELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See “HELP (Help Panel)” on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

PMTLOC=BEFORE

This attribute defines the location of the prompt text. The text defined by *command-prompt-text* appears on the same line as the command area entry field.

NOINIT

This attribute controls the initial display of the command line. When this attribute is specified, the ZCMD field is not initialized to blanks before the panel is displayed.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

PADC= NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

NAME=cmdarea-variable-name

This attribute specifies a command area name to replace the default name ZCMD.

The *cmdarea-variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

ENTWIDTH=n

This attribute is used to specify the length of the command field. It is used in combination with WINDOW=NO on the PANEL tag to create a command line which is longer than a single panel line.

PMTTEXT=YES | NO

This attribute is used to control the formatting of the *command-prompt-text*. When PMTTEXT=NO, the *command-prompt-text* is not used, leaving only the “==>” indicator for the command field.

CMDLOC=DEFAULT | ASIS

This attribute is used to control the placement of the command line in the generated panel. When CMDLOC=DEFAULT (or when CMDLOC is not specified) the command area is placed at line 2 in the panel, and the display position is controlled by the option specified on the Settings panel. When CMDLOC=ASIS is specified, the command area is placed in the generated panel in the same relative position as the CMDAREA tag is found in the DTL source, and the Settings option is ignored when the panel is displayed.

CMDLEN=DEFAULT | MAX

This attribute is used to control the length of the command line in the generated panel. When CMDLEN=DEFAULT (or when CMDLEN is not specified) the command line length is taken from the specified (or defaulted) WIDTH attribute of the PANEL tag. When CMDLEN=MAX is specified, the command line length is taken from the record length of the output panel file.

This attribute is valid only when WINDOW=NO is specified on the PANEL tag.

AUTOTAB=NO | YES

When AUTOTAB=YES, the cursor moves to the next input field when you enter the last character in the command field. If there is no other input field on the panel, the cursor returns to the beginning of the command line.

SCROLLVAR=scroll-variable

This attribute specifies the name of a variable that the application uses to obtain scrolling information. The *scroll-variable* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

If the attribute is specified, the conversion utility creates a scroll entry on the command line, providing that the resulting command area allows at least 8 bytes for a command entry.

SCRVHELP=NO | YES | scroll-help-panel-name | *scroll-help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for the field specified with the SCROLLVAR attribute.

When SCRHELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *scroll-help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

SCROLLTAB=NO | YES

When SCROLLTAB=YES, the cursor moves to the next input field when you enter the last character in the scroll amount field. If there is no other input field on the panel, the cursor returns to the beginning of the command line.

SCRCAPS=OFF | ON

When SCRCAPS=ON, the data in the scroll field is displayed in uppercase characters.

PSBUTTON=cmd-pb-text

This attribute requires that the PSVAR and PSVAL attributes also be specified.

This attribute specifies that a command push button is to be placed at the end of the command line, provided that the resulting command area allows at least 8 bytes for a command entry. The push button text area is created as a point-and-shoot field.

PSVAR=point-and-shoot-variable | %varname

This attribute provides the name of a variable that is to be set when the *cmd-pb-text* is clicked on for point-and-shoot selection. You can define this attribute as a variable name preceded by a percent (%) sign.

The *point-and-shoot-variable* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

PSVAL=point-and-shoot-value | %varname

This attribute provides the value to be placed in the field specified by the PSVAR attribute. You can define this attribute as a variable name preceded by a percent (%) sign. To specify a blank value, use the coding notation “' ” (quotation mark, apostrophe, blank space, apostrophe, quotation mark).

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAMEP=image-namep | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PLACE=ABOVE | BELOW | LEFT | RIGHT | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

CAPS=OFF | ON

When CAPS=ON, the data in the field is displayed in uppercase characters.

NOJUMP=OFF | ON

When NOJUMP=ON, the JUMP function is disabled for the field.

VARDCL=YES | NO

When VARDCL=NO the *cmdarea-variable-name* is not checked to the declared variable information provided with the VARCLASS and VARDCL tags.

command-prompt-text

The *command-prompt-text* specifies the prompt text for the command entry area. The maximum prompt text (not including the command area prefix ===>) is 59 bytes for a standard 76 byte-width panel. The conversion utility reserves 8 bytes for a minimum command entry field and 3 additional bytes are required for panel attributes. One blank is placed between the *command-prompt-text* and the command area prefix. One blank is placed between the end of the command line and the right panel boundary (unless the WINDOW=NO attribute has been specified) to prevent the cursor from skipping into the right panel window border. These formatting considerations mean that the maximum length of the *command-prompt-text* for a panel 76 bytes in width is 59. If the length of the *command-prompt-text* exceeds the available space, a message is issued and the *command-prompt-text* is truncated. If your panel requires that the Scroll field be added to the Command line, or the SCROLLVAR attribute is specified in the CMDAREA definition, the *command-prompt-text* must be further reduced to allow for the Scroll field. If your panel specifies the PSBUTTON attribute, the *command-prompt-text* must be further reduced to allow for the Command push button.

If you do not provide *command-prompt-text*, the word "Command" (or its translated equivalent) is the default, unless you are creating an ISPF selection panel, in which case the word "Option" (or its translated equivalent) is the default. The Common User Access command area prefix (===>) is always added automatically in front of the entry field.

Comments

The CMDAREA tag defines a command entry area on an application panel. The command entry area extends to the right side of the panel, unless limited by the ENTWIDTH attribute or the presence of a Scroll field. Application users use the command entry area to enter commands.

Note: If you specify the CMDAREA tag within your DTL source file:

- It must appear before the AREA, DA, GA, REGION, or SELFLD tag when DEPTH=* is specified.
- It must appear before the SELFLD tag when TYPE=MENU and CHECKVAR or UNAVAIL attributes are specified on nested CHOICE tags.

Restrictions

- You must code the CMDAREA tag within a PANEL definition. You can code only one command area definition for each panel. See [“PANEL \(Panel\)” on page 376](#) for a complete description of this tag.
- The data entered on the command line is processed “as is”. To translate the data to uppercase, you must either provide a VARDCL definition for the field ZCMD with a reference to a VARCLASS containing an XLATL tag which specifies FORMAT=UPPER, or specify CAPS=ON.
- You cannot code the CMDAREA tag within an AREA definition. The Command area is generated at the top of the panel source to allow for floating of the command line. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for more information.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

Table 19. The tags you can code within a CMDAREA definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is application panel markup that contains a command area. The *command-prompt-text* "Use this area to enter a command" is specified in the markup to override the default text “Command”. [Figure 97 on page 243](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM>
<VARCLASS NAME=choiccls TYPE='char 2'
<VARCLASS NAME=vccmd TYPE='char 62'>
  <XLATL FORMAT=upper>
  </XLATL>
</VARCLASS>

<VARLIST>
  <VARDCL NAME=sample VARCLASS=choiccls>
  <VARDCL NAME=zcmd VARCLASS=vccmd>
</VARLIST>

<PANEL NAME=cmdarea1>Choose a Virtue
  <TOPINST>Select a choice.
  <AREA>
    <SELFLD NAME=sample PMTWIDTH=10 SELWIDTH=20>Virtues:
      <CHOICE>Faith
      <CHOICE>Hope
      <CHOICE>Charity
    </SELFLD>
  </AREA>
  <BOTINST>Now press Enter.
  <CMDAREA>Use this area to enter a command
</PANEL>

```

Choose a Virtue

Select a choice.

Virtues:

- 1. Faith
- 2. Hope
- 3. Charity

Now press Enter.

Use this area to enter a command ==> _____
 F1=Help F3=Exit F12=Cancel

Figure 97. Command area

CMTDBL (Command Table)

The CMTDBL tag provides support to define the ISPF application command table.

Syntax

```

➤ <CMTDBL — APPLID=application-identifier ————— > — </CMTDBL> ➤
      |
      | NO
      |_____|
      |
      | YES
      |_____|
      |
      SORT=

```

Parameters

APPLID=application-identifier

This attribute specifies an application identifier. This identifier is used as a prefix to the string "CMD5" to form the name of the command table. The *application-identifier* must have these characteristics:

- 1-4 characters in length
- The first (or only) character must be A-Z or a-z @, #, or \$.
- Remaining characters, if any, must be A-Z, a-z, @, #, \$, or 0-9.

Lowercase characters are translated to their uppercase equivalents.

The name of the command table is member name xxxxCMDS, where xxxx represents the *application-identifier*.

Command tables are updated using ISPF table services. Input is obtained from the ISPTLIB DDname allocation and output is written to the ISPTABL DDname allocation. See the description of how to allocate libraries before starting ISPF in the [z/OS ISPF User's Guide Vol I](#) for more information about the use of ISPTLIB and ISPTABL.

SORT=NO | YES

When SORT=YES is specified, the command table is sorted in command-name sequence. Any commands defined as an ALIAS to other commands are placed in the command table first, in command-name sequence. The regular commands follow the ALIAS entries in command-name sequence.

If SORT=NO or the SORT attribute is not specified, commands are placed in the command table in the sequence the CMD tags are encountered in the DTL source file.

Comments

The command table tag provides support to define the ISPF application command table. ACTION tags and definitions of key lists reference the command definitions within an application command table.

Note: To access commands through the use of the key list function keys, specify the KEYLAPPL ID invocation parameter for the conversion utility with the same APPLID value used for the CMDTBL tag.

Note: You can use the TSO ISPCMDTB command to convert existing command tables to DTL. To use ISPCMDTB, ensure that the command table is in your table concatenation (ISPCMDTB), type TSO ISPCMDTB *applid* (where *applid* is the application id of the command table). This places you in an edit session containing the DTL version of the command table. Use the editor CREATE or REPLACE commands to save the table to your DTL source data set.

Restrictions

- The CMDTBL tag requires an end tag.
- You cannot code the CMDTBL tag within any other tag definition.
- You can code only one command table for any application.

Processing

Table 20. The tags you can code within a CMDTBL definition			
Tag	Reference	Usage	Required
CMD	“CMD (Command Definition)” on page 233	Multiple	Yes

Examples

This source file markup contains a command table that defines the commands UPDATE, ADD, DELETE and SEARCH.

```
<!DOCTYPE DM SYSTEM>

<CMDTBL APPLID=conv>
  <CMD NAME=update>Upd<T>ate
  <CMDACT ACTION='alias add'>
  <CMD NAME=add>Add
  <CMDACT ACTION=setverb>
  <CMD NAME=delete>Del<T>ete
  <CMDACT ACTION=passthru>
  <CMD NAME=search>Search
  <CMDACT ACTION=passthru>
</CMDTBL>
```

This table shows the resultant ISPF application command table.

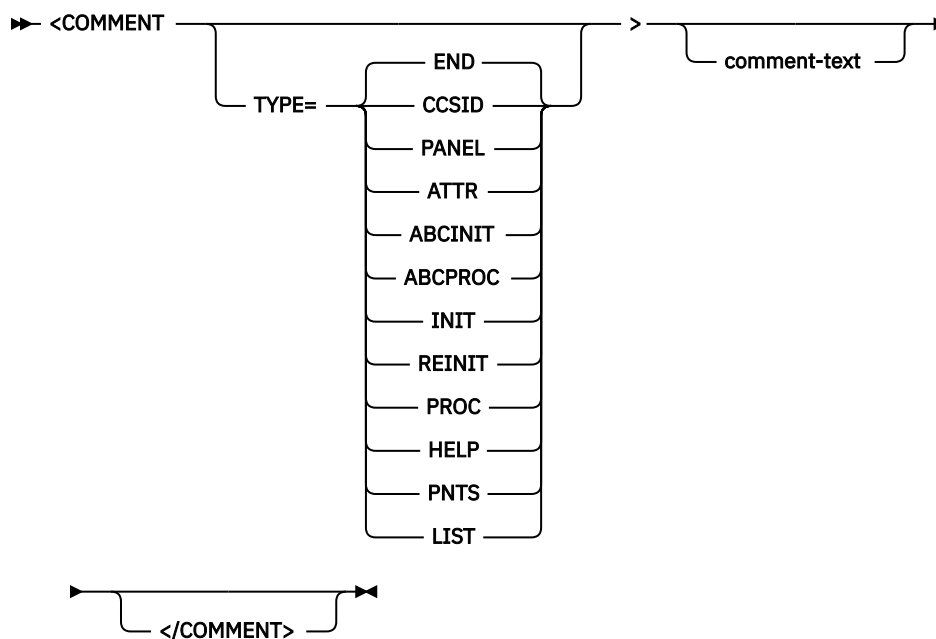
Table 21. ISPF Application Command Table

ZCTVERB	ZCTTRUNC	ZCTACT
UPDATE	3	ALIAS ADD
ADD	0	SETVERB
DELETE	3	PASSTHRU
SEARCH	0	PASSTHRU

COMMENT (Comment)

The COMMENT tag adds comment text to the generated panel or message member.

Syntax



Parameters

TYPE=END | CCSID | PANEL | ATTR | ABCINIT | ABCPROC | INIT | REINIT | PROC | HELP | PNTS | LIST

This attribute specifies the section of the panel that is to contain the comment text. The default is END. TYPE=END is assumed if the COMMENT tag is used within the MSGMBR tag.

COMMENT tags that specify the TYPE as ABCINIT or ABCPROC must follow an ABC or PDC tag.

When a COMMENT tag is coded within a HELP panel, the TYPE value is limited to CCSID, PANEL, ATTR, INIT, PROC, or END.

comment-text

The *comment-text* is flowed to a width of 66 bytes. The conversion utility adds `"/* "` before and `" */"` after the resulting text.

When no *comment-text* is present, a blank comment line is added to the specified (or defaulted) panel section.

Comments

The COMMENT tag adds comments to the generated ISPF format panel. If the PREP conversion option has been specified, the comments are not part of the final panel because they are not processed by the ISPPREP utility.

Lines of text from a COMMENT tag are added to the specified panel section when encountered in the DTL source file.

Note: If the panel section specified is not generated by other conversion processing, comments are formatted in this way:

TYPE

Position of comments

CCSID

Following the)PANEL statement.

LIST

Before the)END statement.

Comments added to the)END panel section are placed following any entries from the COPYR tag and comments containing the ISPD TLC version number and panel creation date. Lines placed in the)END section of a HELP panel are added to each continuation HELP panel.

Restrictions

- You must code the COMMENT tag within an ABC, AREA, CHOICE, DA, DTACOL, DTAFLD, HELP, LSTCOL, LSTFLD, LSTGRP, PANEL, PDC, REGION or SELFLD tag definition.

Processing

None.

Examples

Here is source file markup that contains a comment of several lines that are placed after the)END panel statement. [Figure 98 on page 247](#) shows portion of the ISPF format panel containing the formatted result.

```
<!doctype dm system>
<!-- COMMENT tag example - PANEL tag -->
<!-- )END section - after CMDAREA tag -->

<varclass name=vc1 type='char 10'>
<varclass name=vc2 type='char 6'>
```

```

<varlist>
<vardcl name=lst1 varclass=vc1>
<vardcl name=lst2 varclass=vc2>
</varlist>

<panel name=comment1 depth=19 width=50>
This is panel Comment1

<LSTFLD >
<LSTGRP headline=yes>
<LSTCOL colwidth=10 datavar=lst1 usage=in varclass=vc1 line=1
    required=yes autotab=yes align=end help=h1 msg=abcd101>COL1
<LSTCOL colwidth=6  datavar=lst2 usage=in varclass=vc2 line=2
    required=yes autotab=yes align=end help=h1 msg=abcd101>COL2
</LSTGRP>
</LSTFLD>
<cmdarea>
<comment type=end>
    comment line 1
    comment line 2
    comment line 3
    comment line 4
    comment line 5
    comment line 6
    comment line 7
    comment line 8
    comment line 9
</panel>

:
)END
/* comment line 1 comment line 2 comment line 3 comment line 4      */
/* comment line 5 comment line 6 comment line 7 comment line 8      */
/* comment line 9                                                     */

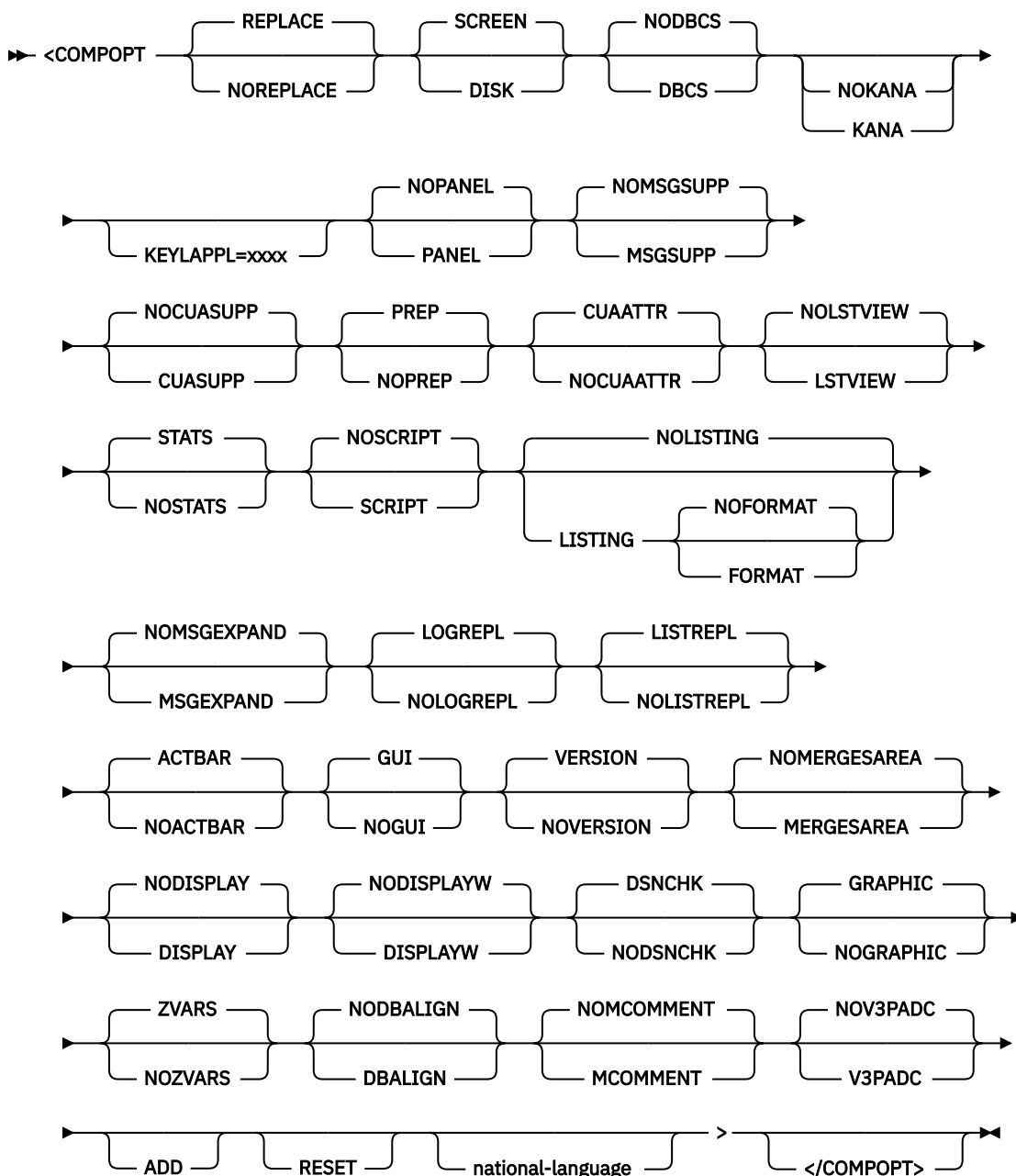
```

Figure 98. Comment text added to a panel

COMPOPT (Compiler Options)

The COMPOPT tag sets compiler options for the current source file.

Syntax



With the exception of ADD and RESET, all of the option keywords used for the COMPOPT tag are the same as those used for [“Conversion utility syntax” on page 156](#). See that chapter for information about these keywords.

The COMPOPT tag keyword RESET restores the conversion utility options to their original invocation settings.

Comments

The COMPOPT tag can be placed within the Doctype definition to encompass the entire DTL source file or it can be placed before the first PANEL, HELP, MSGMBR, KEYL, or CMDTBL tag that requires a compiler option change.

Unless the ADD option is specified when the COMPOPT tag is processed, all conversion utility options except PANEL, DISK, SCREEN, DISPLAY, DISPLAYW, DBCS, and KANA are first reset to the defined default values. The options specified on the COMPOPT tag are then applied.

When the ADD option is included, the original options remain in effect and the options from the COMPOPT tag are added to the current list. ADD overrides any existing option.

The options set by this tag remain in effect for the current source file until another COMPOPT tag is processed. If you are converting a list of members, either from member list selections or from a DTLLST list of members, the conversion utility options are reset to their original invocation settings when the current source file is completed.

The PROFILE and PROFDDN options defined as part of the conversion utility invocation syntax are not supported by the COMPOPT tag.

Restrictions

None.

Processing

None.

Examples

This source file markup contains a compiler options line that specifies the compiler options to be used converting this source file:

```
<!doctype dm system>
<varclass name=vc1 type='char 10'>
<varclass name=vc2 type='char 6'>
<varlist>
<vardcl name=lst1 varclass=vc1>
<vardcl name=lst2 varclass=vc2>
</varlist>

<compopt noprep noreplace>

<panel name=compopt depth=19 width=50>
This is panel Compopt

<LSTFLD >
  <LSTGRP headline=yes>
    <LSTCOL colwidth=10 datavar=lst1 usage=in varclass=vc1 line=1
      required=yes autotab=yes align=end help=h1 msg=abcd101>COL1
    <LSTCOL colwidth=6 datavar=lst2 usage=in varclass=vc2 line=2
      required=yes autotab=yes align=end help=h1 msg=abcd101>COL2
  </LSTGRP>
</LSTFLD>
<cmdarea>
</panel>
```

COPYR (Copyright)

The COPYR tag adds copyright text to the generated panel or message member.

Syntax

```
➡ <COPYR> ┌──────────────────┐ ┌──────────┐ ➡
           │ copyright-text    │ │ </COPYR> │
```

Parameters

copyright-text

The *copyright-text* is limited to 66 bytes. It is automatically formatted as a panel comment with a `"/*` in front and a `*/` following the supplied text.

Comments

The COPYR tag adds copyright information to the panel.

The COPYR tag must be placed before the first PANEL, HELP, or MSGMBR definition within the DTL source file that is to contain the copyright information.

You can use multiple COPYR tags. Each tag creates one comment line, which is placed after the)END panel statement, or the last message in the message member, in the order found in the DTL source.

The *copyright-text* is added to each subsequent panel or message member generated from the same DTL source file member. If the PREP conversion option has been specified, the copyright is not part of the final panel because comments are not processed by the ISPPREP utility.

Restrictions

None.

Processing

None.

Examples

Here is source file markup that contains two copyright lines that are placed after the)END panel statement. [Figure 99 on page 250](#) shows a portion of the ISPF format panel containing the formatted result.

```
<!doctype dm system>
<!-- COPYR tag example - PANEL tag -->

<varclass name=vc1 type='char 10'>
<varclass name=vc2 type='char 6'>
<varlist>
<vardcl name=lst1 varclass=vc1>
<vardcl name=lst2 varclass=vc2>
</varlist>

<copyr>Copyright statement 1
<copyr>Copyright statement 2

<panel name=copyrt1 depth=19 width=50>
This is panel Copyrt1

<LSTFLD >
<LSTGRP headline=yes>
<LSTCOL colwidth=10 datavar=lst1 usage=in varclass=vc1 line=1
  required=yes autotab=yes align=end help=h1 msg=abcd101>COL1
<LSTCOL colwidth=6 datavar=lst2 usage=in varclass=vc2 line=2
  required=yes autotab=yes align=end help=h1 msg=abcd101>COL2
</LSTGRP>
</LSTFLD>
<cmdarea>
</panel>
```

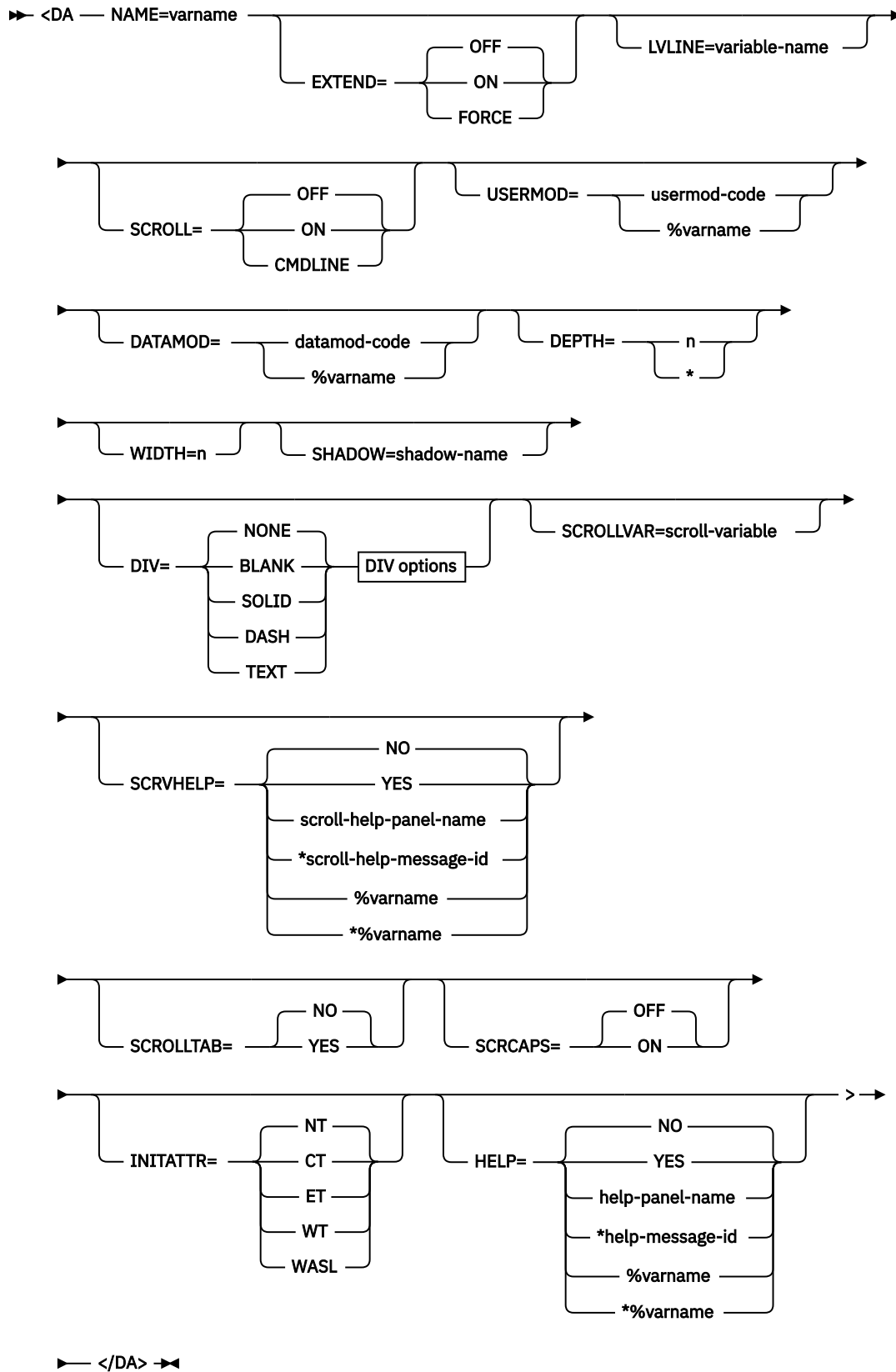
```
:
)END
/* Copyright statement 1 */
/* Copyright statement 2 */
```

Figure 99. Copyright statement added to a panel

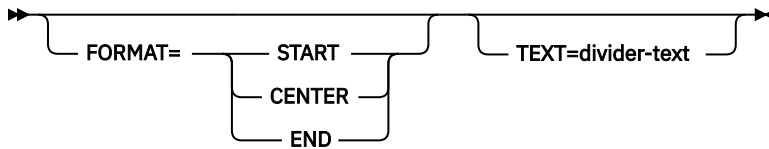
DA (Dynamic Area)

The DA tag defines a dynamic area in the panel)BODY section.

Syntax



DIV options



Parameters

NAME=varname

This attribute defines the name of a dynamic area. This name is the dialog variable specified by the application that contains the data for the dynamic area. The *varname* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

EXTEND=OFF | ON | FORCE

This attribute defines the runtime display size of the dynamic area. If EXTEND=ON is specified, the dynamic area definition is expanded to the size of the logical screen. If you intend to display the panels in a pop-up window, use EXTEND=OFF (which is the default).

If EXTEND=FORCE is specified within a horizontal area or region, the EXTEND(ON) keyword is added to the dynamic area attribute statement in the)ATTR panel section. The conversion utility issues a message to advise of a potential display error if other panel fields are formatted on or after the last defined line of the dynamic area.

LVLIN=variable-name

This attribute allows you to specify the name of a variable that contains the result of the ISPF function LVLIN. The *variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

SCROLL=OFF | ON | CMDLINE

If you specify SCROLL=ON or SCROLL=CMDLINE, ISPD TLC adds the scroll amount field provided by the SCROLLVAR attribute to the command line.

If you specify SCROLL=ON, ISPD TLC also automatically enables scrolling commands by adding SCROLL(ON) to the dynamic area attribute definition.

Note: When SCROLL(ON) is *not* part of the dynamic area attribute definition, data in the scroll amount field is available to the application exactly as entered.

The first dynamic area on a panel that specifies SCROLL=ON or SCROLL=CMDLINE (with a valid SCROLLVAR attribute) controls the creation of the scroll amount field. The specification of the SCROLL attribute on subsequent DA tags is ignored.

USERMOD=usermod-code | %varname

This attribute specifies a single-character or a 2-position hexadecimal value to be substituted for attribute characters in a dynamic area variable following user interaction. You can define this attribute as a variable name preceded by a “%”.

DATAMOD=datamod-code | %varname

This attribute specifies a single-character or a 2-position hexadecimal value to be substituted for attribute characters in a dynamic area following user interaction. You can define this attribute as a variable name preceded by a “%”.

DEPTH=n | *

This attribute specifies the number of lines reserved for the dynamic area definition.

If the DA tag is to be formatted in the panel)BODY section, that is, the tag is not within a scrollable area:

- The maximum DEPTH value is the DEPTH value specified on the PANEL tag, reduced by the number of divider lines (if the DIV attribute is specified) and any other lines previously used by text or interactive fields.
- If the DEPTH value is specified as an asterisk (*), the conversion utility reserves the remaining available panel depth for the dynamic area.

If the DA tag is defined within a scrollable area (see [“AREA \(Area\)”](#) on page 189), * cannot be specified as the depth value. The maximum DEPTH value is limited by the ISPF runtime environment.

WIDTH=n

This attribute specifies the number of columns reserved in the panel)BODY section for the dynamic area definition. If the dynamic area width is less than the PANEL width, the conversion utility adds an attribute byte immediately following the right dynamic area boundary. The minimum width for a dynamic area is the length of *varname* plus two (2) positions. The maximum value is the remaining panel width.

SHADOW=shadow-name

This attribute provides a name for a shadow variable name which is used to define character level attributes within the dynamic area string. The *shadow-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

DIV=NONE | BLANK | SOLID | DASH | TEXT

This attribute specifies the type of divider line to be placed before and after the dynamic area. If this attribute is not specified or has the value NONE, no divider line is generated. The value BLANK produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line. A visible divider line formats with a non-displayable attribute byte on each end of the line.

FORMAT=START | CENTER | END

This attribute specifies the position of the *divider-text* within the divider line. You must specify both the FORMAT attribute and the TEXT attribute to create a divider line containing text.

TEXT=divider-text

This attribute specifies the text to be placed on the divider line. You must specify both the FORMAT attribute and the TEXT attribute to create a divider line containing text.

SCROLLVAR=scroll-variable

This attribute specifies the name of a variable that the application uses to obtain scrolling information. The *scroll-variable* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

SCRVHELP=NO | YES | scroll-help-panel-name | *scroll-help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for the field specified with the SCROLLVAR attribute.

When SCRHELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *scroll-help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

SCROLLTAB=NO | YES

If you specify SCROLLTAB=YES, the cursor moves to the next input field when the user enters the last character in the scroll amount field. If there is no other input field on the panel, the cursor moves to the beginning of the command line.

SCRCAPS=OFF | ON

When SCRCAPS=ON, the data in the scroll field is displayed in uppercase characters.

INITATTR=NT | CT | ET | WT | WASL

This attribute specifies the last attribute found before the start of the dynamic area. This allows the developer control of the initial color for the area. The conversion utility replaces the last attribute found before the dynamic area with the attribute specified.

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies whether help is available for the dynamic area.

When HELP=YES, requesting help when the cursor is within the dynamic area causes control to return to the application. It is the application's responsibility to process the help request. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help in a dynamic area and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

Comments

The DA tag defines a dynamic area in the panel)BODY or)AREA sections.

If you specify the CMDAREA tag within your DTL source file, it must appear before the DA tag when DEPTH=* is specified. The DA tag DEPTH may have to be adjusted to allow for additional lines which result from tags present within the panel definition following the end DA tag.

See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for a discussion of dynamic areas.

Restrictions

- You must code the DA tag within a PANEL, AREA, or REGION tag. If found anywhere else, an error is logged and the output panel is not saved.
- If NAME is not valid or not specified, an error is logged and the output panel is not saved.
- You can use the EXTEND=ON attribute only once within a panel, and EXTEND=ON cannot be specified on a DA tag coded within a scrollable area. If EXTEND is already active, either from a DA tag, or from an AREA, GA, SELFLD or REGION tag, a warning message is logged and the EXTEND attribute is ignored.
- You can use the SCROLLVAR attribute only once within a panel.
- If you specify the SCROLLVAR attribute, you must also specify the attribute SCROLL=ON or SCROLL=CMDLINE.
- The resulting scroll entry on the command line must leave at least eight positions for the command entry field.
- If you specify the SCRHELP attribute, you must also specify the SCROLLVAR attribute.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names”](#) on page 179.

Processing

Table 22. The tags you can code within a DA definition

Tag	Reference	Usage	Required
ATTR	“ATTR (Attribute)” on page 200	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=da KEYLIST=keyl&xp>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST> Type in patron's name and card number (if applicable)
<AREA>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
  <DTAFLD DATAVAR=curdate USAGE=out ENTWIDTH=8>Date
  <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
    <DTAFLDD>(A 7-digit number)
  <DTAFLD DATAVAR=name>Name
    <DTAFLDD>(Last, First, M.I.)
  <DTAFLD DATAVAR=address>Address
</DTACOL>
<DIVIDER>
<DA NAME=darea DIV=solid DEPTH=6 SHADOW=shadwvar>
  <ATTR ATTRCHAR=# TYPE=datain PADC='_' COLOR=BLUE>
  <ATTR ATTRCHAR=| TYPE=dataout COLOR=green>
  <ATTR ATTRCHAR=$ TYPE=char COLOR=red>
</DA>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

DD (Definition Description)

The DD tag defines the description of a term in a definition list.

Syntax



Parameters

definition-description

This is the text for the description of a definition list term.

Comments

The DD tag defines the description of a term in a definition list.

Restrictions

- You must code the DD tag within a DL definition. See [“DL \(Definition List\)” on page 261](#) for a complete description of this tag.
- Each DD tag must follow an associated DT tag within the definition list. You can code only one DD tag for each DT tag.

Processing

Table 23. The tags you can code within a DD definition			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a definition list with three definition descriptions. [Figure 100 on page 257](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=dd DEPTH=22 WIDTH=64>Help for Markup
<AREA>
<INFO>
  <P>Here are some definitions:
  <DL TSIZE=2 BREAK=all>
    <DT>markup
    <DD>Text that is added to document data in order to
    convey information about it.
    There are three types of markup the DTL uses: tags, references,
    and markup declarations.
    <DT>markup declaration
    <DD>Markup that controls how other markup of a document
    is to be interpreted, for example document type and entity declarations.
    <DT>markup language
    <DD>A set of characters, conventions, and rules to control
    the interpretation of document data.
    The Dialog Tag Language is a markup language.
```



```
</DL>
</INFO>
</AREA>
</HELP>
```

Help for Markup

Here are some definitions:

markup

Text that is added to document data in order to convey information about it. There are three types of markup the DTL uses: tags, references, and markup declarations.

markup declaration

Markup that controls how other markup of a document is to be interpreted, for example document type and entity declarations.

markup language

A set of characters, conventions, and rules to control the interpretation of document data. The Dialog Tag Language is a markup language.

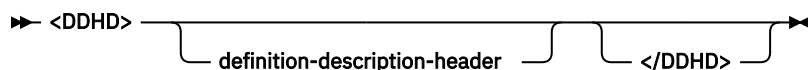
F1=Help F3=Exit F5=Exhelp F6=Keyshelp
F7=PrvTopic F8=NxtTopic F10=PrvPage F11=NxtPage
F12=Cancel

Figure 100. Definition descriptions

DDHD (Definition Description Header)

The DDHD tag defines the heading for the description column of a definition list.

Syntax



Parameters

definition-description-header

This is the text of the definition description header.

Comments

The DDHD tag defines the heading for the description column of a definition list. You can code multiple DDHD tags within a definition list.

The conversion utility inserts a blank line between the header and the list items unless the COMPACT attribute is specified on the DL tag.

Restrictions

- You must code the DDHD tag within a DL definition. See [“DL \(Definition List\)” on page 261](#) for a complete description of this tag.
- Each DDHD tag must be paired with and follow a DTHD tag. See [“DTHD \(Definition Term Header\)” on page 287](#) for a complete description of this tag.

Processing

Table 24. The tags you can code within a DDHD definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains a definition description header with the text “Meaning”. [Figure 101 on page 258](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=ddhd DEPTH=18>Prefix Help
<AREA>
<INFO>
  <P>The following list defines each of the valid prefixes.
  <DL TSIZE=12>
    <DTHD>Prefix
    <DDHD>Meaning
    <DT>AU
    <DD>Automotive
    <DT>HB
    <DD>Health and beauty
    <DT>LG
    <DD>Lawn and garden
    <DT>SG
    <DD>Sporting goods
  </DL>
</INFO>
</AREA>
</HELP>
```

Prefix Help

The following list defines each of the valid prefixes.

Prefix	Meaning
AU	Automotive
HB	Health and beauty
LG	Lawn and garden
SG	Sporting goods

F1=Help

F6=Keyshelp

F10=PrvPage

F3=Exit

F7=PrvTopic

F11=NxtPage

F5=Exhelp

F8=NxtTopic

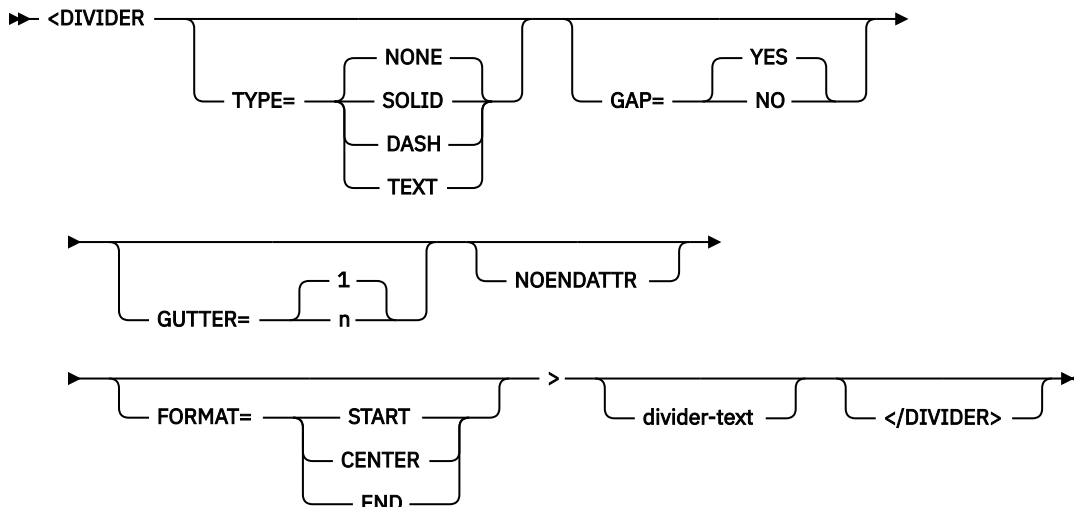
F12=Cancel

Figure 101. Definition description header

DIVIDER (Area Divider)

The DIVIDER tag creates a blank or visible divider within the text portion of an application panel.

Syntax



Parameters

TYPE=NONE | SOLID | DASH | TEXT

This attribute specifies the type of divider line. The line width is one character.

The default value is NONE, which produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

GAP=YES | NO

When GAP=NO, the divider line completely crosses from one side of the text area to the other. When GAP=YES, a 1-character gap remains at each end of the divider line. However, GAP=YES is ignored and set to NO for dividers coded within horizontal regions.

GUTTER=1 | n

This attribute specifies the total width of the divider. If the GUTTER value is an even number, the conversion utility increases the number by 1 so that the divider is centered within the defined width.

The minimum GUTTER value is 1. If GUTTER=1 on a DIVIDER within a horizontal region, then the TYPE value must be NONE.

The default GUTTER value for a DIVIDER within a vertical region is 1. The default GUTTER value for dividers within horizontal regions is 3 to allow for an attribute byte on each side of the divider character.

NOENDATTR

This attribute is valid only when the DIVIDER tag is coded within a horizontal region. It specifies that no ending attribute character is placed after the divider character.

Note: The minimum divider space that can be specified for a horizontal region is 1.

When the GUTTER value is 1, the divider character is set to blank.

When the GUTTER value is 2, a solid divider may be specified. The divider character is placed in the second position of the 2-character GUTTER space.

FORMAT=START | CENTER | END

This attribute specifies the position of the divider text within the width of the divider line.

divider-text

This is the text of the area divider line.

Comments

The DIVIDER tag creates a blank or solid divider within the text portion of an application panel. A horizontally formatted visible divider is created when you specify the TYPE attribute value as SOLID or DASH. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line. A vertically formatted SOLID or DASH divider is the "|" character which is obtained from the ISPF literals table. The direction of the divider is determined by the tag definition it is coded within. Here are the details for formatting for dividers:

- Dividers coded within an AREA, HELP, or PANEL tag definition format horizontally.
- Dividers coded within a vertical region format horizontally.
- Dividers coded within a horizontal region format vertically.

The divider line can be formatted with descriptive text. When this feature is used, the FORMAT attribute must be specified. If FORMAT is not specified, the tag text is ignored. You control the text padding with the TYPE attribute. If TYPE=TEXT, the *divider-text* is padded with blanks. When TYPE=SOLID or TYPE=DASH, the *divider-text* is padded with the specified character.

Restrictions

- You must code the DIVIDER tag within an AREA, DTACOL, HELP, PANEL, or REGION definition. See [“AREA \(Area\)” on page 189](#), [“DTACOL \(Data Column\)” on page 269](#), [“HELP \(Help Panel\)” on page 303](#), [“PANEL \(Panel\)” on page 376](#), and [“REGION \(Region\)” on page 405](#) for descriptions of these tags.

Processing

Table 25. The tags you can code within a DIVIDER definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is application panel markup that contains four DIVIDER definitions. The first divider is blank. The second divider is solid with a gutter size of 2 and a GAP=NO value. The third and fourth dividers are solid. [Figure 102 on page 261](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM(
  <!entity sampvar3 system>)>
&sampvar3;

<PANEL NAME=divider DEPTH=22 WIDTH=70>Print a Document
<AREA>
  <DTACOL PMTWIDTH=20 ENTWIDTH=8 SELWIDTH=40 DESWIDTH=35>
    <DTAFLD DATAVAR=file>File name
    <DTAFLDD>Name of the document to be printed
    <DIVIDER TYPE=none>
    <SELFLD NAME=type PMTLOC=before>Type style for printing
      <CHOICE>Prestige Elite (12 pitch)
      <CHOICE>Courier (10 pitch)
      <CHOICE>Essay Standard (proportional)
      <CHOICE>Essay Bold (proportional)
    </SELFLD>
  </DTACOL>
  <DIVIDER TYPE=solid GUTTER=2 GAP=no>
  <DTACOL PMTWIDTH=20 ENTWIDTH=2 DESWIDTH=35>
    <DTAFLD DATAVAR=marg>Left margin
    <DTAFLDD>Number of spaces in the left margin
    <DIVIDER TYPE=solid>
    <DTAFLD DATAVAR=copy>Copies
    <DTAFLDD>Number of copies
    <DIVIDER TYPE=solid>
    <DTAFLD DATAVAR=duplx ENTWIDTH=1>Duplex
    <DTAFLDD>1 = Yes (Print both sides of paper)
    <DTAFLDD>2 = No (Print one side only)
  </DTACOL>
</AREA>
</PANEL>

```

```

                                Print a Document

File name . . . . . _____ Name of the document to be printed

Type style for
printing . . . . . __  1. Prestige Elite (12 pitch)
                        2. Courier (10 pitch)
                        3. Essay Standard (proportional)
                        4. Essay Bold (proportional)

-----

Left margin . . . . . __  Number of spaces in the left margin
-----
Copies . . . . . __  Number of copies
-----
Duplex . . . . . _  1 = Yes (Print both sides of paper)
                   2 = No (Print one side only)

F1=Help   F3=Exit   F12=Cancel

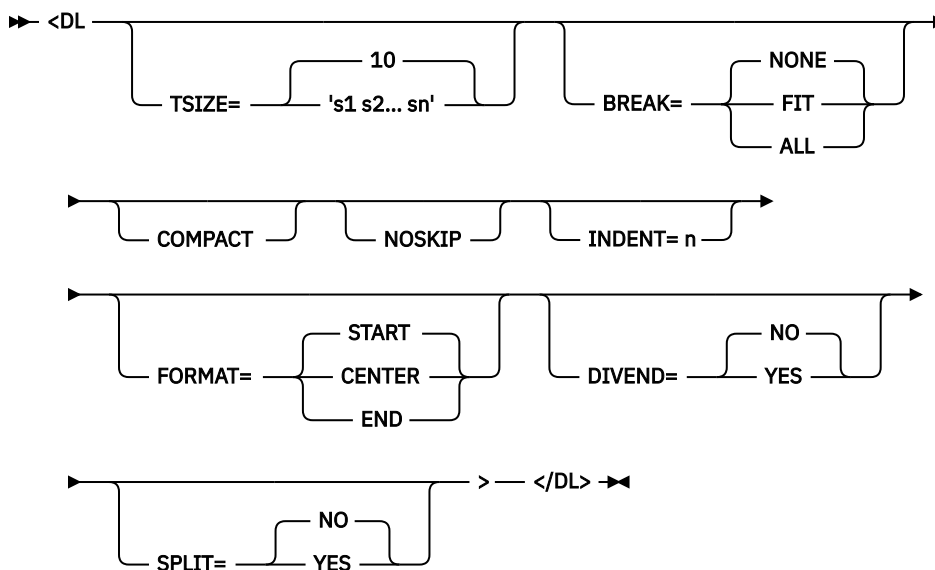
```

Figure 102. Area dividers

DL (Definition List)

The DL tag defines a list of terms and their corresponding definitions within an information region.

Syntax



Parameters

TSIZE=10 | 's1 s2... sn'

This attribute specifies the space to be allocated for the definition term. The default value is 10 characters. The minimum TSIZE value is 0 and the maximum is 40. When multiple TSIZE values are specified, a DT tag must be coded for each value. The sizes are applied to the DT tags in the order the tags are encountered in the DTL source file.

BREAK=NONE | FIT | ALL

This attribute controls the formatting of the definition terms and descriptions. If BREAK=NONE, the term is on the same line as the description, spilling into the description area if the length exceeds TSIZE. If BREAK=FIT, the description is on the line below the term if the term exceeds the TSIZE value. If BREAK=ALL, every definition is on the line below the term.

COMPACT

This attribute causes the list to format without a blank line between the items in the list. If you code DDHD and DTHD tags in a compact definition list, the list formats without a blank line between the headers and list items.

NOSKIP

This attribute causes the list to format without creating a blank line before the first line of the list.

INDENT=n

This attribute specifies that the definition list is to be indented from the current left margin.

FORMAT=START | CENTER | END

This attribute specifies the placement of the DT tag text within the space specified by TSIZE. The DL tag FORMAT setting applies to all of the DT tags within the definition list.

DIVEND=NO | YES

This attribute specifies whether a divider character is formatted following the DDHD and DD tag text. When DIVEND=YES the formatting width of the DDHD and DD text is reduced to allow space for the divider character.

SPLIT=NO | YES

This attribute controls the format of the last DT tag in a multiple DT tag group. It is used only when BREAK=ALL or when BREAK=FIT and the DT tag text length exceeds the TSIZE value. When SPLIT=YES, the text following the last DT tag in the DT group (typically one or two dashes) is placed in front of the first line of the formatted DD tag text. The DL tag SPLIT setting applies to all of the DT tag groups within the definition list.

Comments

The DL tag defines a list of terms and their corresponding definitions within an information region. You use the DT and DD tags to identify the terms that you are defining and their descriptions, respectively. You use the DTHD and the DDHD tags to define headings for the term and description columns in definition lists.

The conversion utility inserts a blank line before the definition list unless NOSKIP is specified.

If you do not specify a TSIZE value, the space allocated for the term size is 10 characters. If any term is longer than 10 characters and BREAK=NONE (the default) is specified, the term extends into the description line. If the term is still too long to fit, it wraps to the next line.

The definition description is an implied paragraph, and can contain any text items. For example, you can insert additional paragraphs in a definition description by using the paragraph (P) tag following the description paragraph. Other tags that you want to nest within the definition list (such as OL, SL, or UL) must follow the DD tag within the list.

Restrictions

- The DL tag requires an end tag.
- You must code the DL tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- If you code DDHD and DTHD tags within the definition list, they must precede the first DT tag.

Processing

<i>Table 26. The tags you can code within a DL definition</i>			
Tag	Reference	Usage	Required
DD	“DD (Definition Description)” on page 255	Multiple	No
DDHD	“DDHD (Definition Description Header)” on page 257	Multiple	No
DLDIV	“DLDIV (Definition List Divider)” on page 265	Multiple	No
DT	“DT (Definition Term)” on page 267	Multiple	No
DTDIV	“DTDIV (Definition Term Divider)” on page 286	Multiple	No
DTHD	“DTHD (Definition Term Header)” on page 287	Multiple	No
DTHDIV	“DTHDIV (Definition Term Header Divider)” on page 288	Multiple	No

Examples

Here is help panel markup that contains a definition list that uses the default BREAK value of NONE, which formats the definition descriptions on the same line as the associated terms. Definition term and description headers are also included. [Figure 103 on page 264](#) shows the formatted result of the markup. [Figure 104 on page 264](#) shows how the same definition list would format with a BREAK value of FIT. [Figure 105 on page 265](#) shows how the same definition list would format with a BREAK value of ALL.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=d1 DEPTH=22 WIDTH=60>Employee Code Help
  <AREA>
    <INFO>
      <P>The following list defines the valid employee codes.
      <DL TSIZE=11>
        <DTHD>Code
        <DDHD>Meaning
        <DT>Full-time
        <DD>Indicates that the employee works a
          regular schedule of 40 hours or more weekly.
        <DT>Part-time
        <DD>Indicates that the employee works a regular
          schedule of 20 to 40 hours weekly.
        <DT>Supplemental
        <DD>Indicates that the employee works less than
          20 hours weekly.
          No regular schedule is in place.
      </DL>
    </INFO>
  </AREA>
</HELP>
```

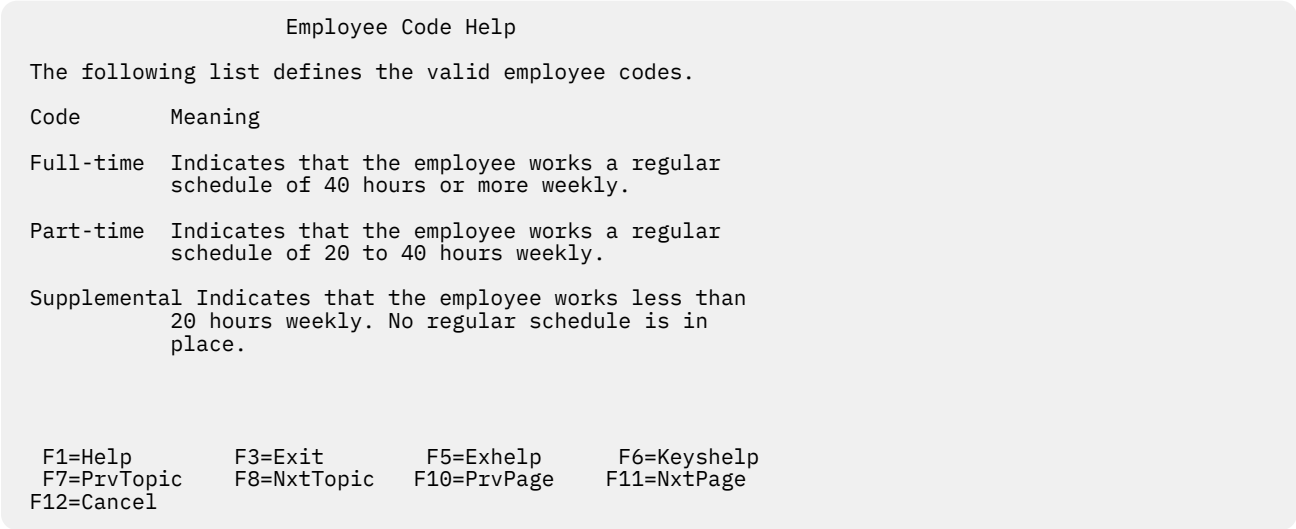


Figure 103. Definition List (BREAK=NONE)

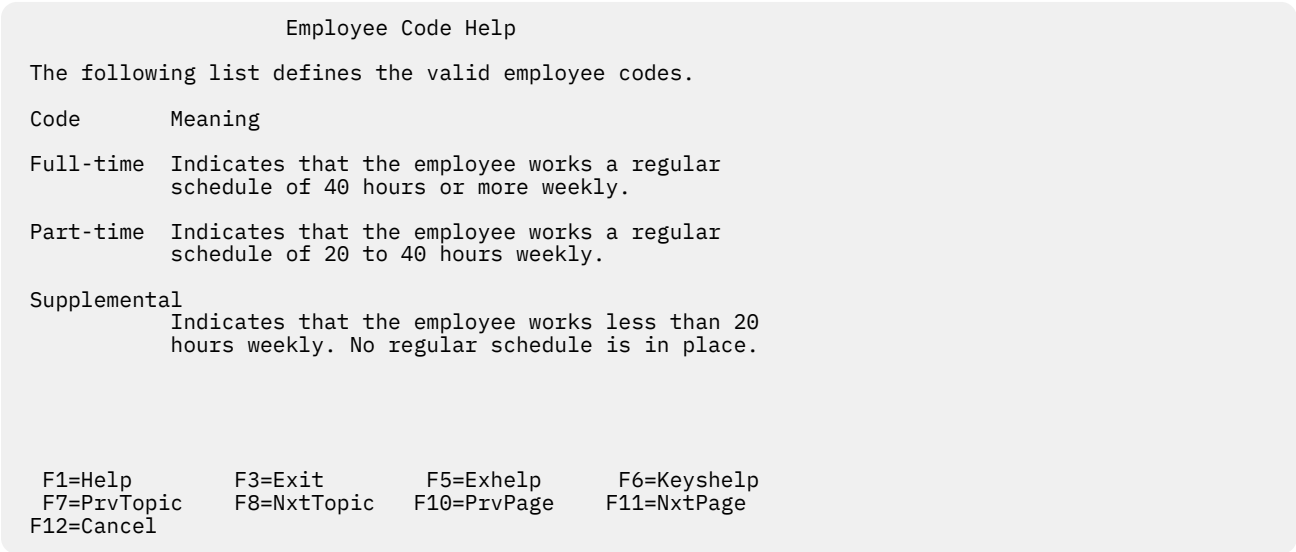


Figure 104. Definition List (BREAK=FIT)

Employee Code Help

The following list defines the valid employee codes.

Code	Meaning
Full-time	Indicates that the employee works a regular schedule of 40 hours or more weekly.
Part-time	Indicates that the employee works a regular schedule of 20 to 40 hours weekly.
Supplemental	Indicates that the employee works less than 20 hours weekly. No regular schedule is in place.

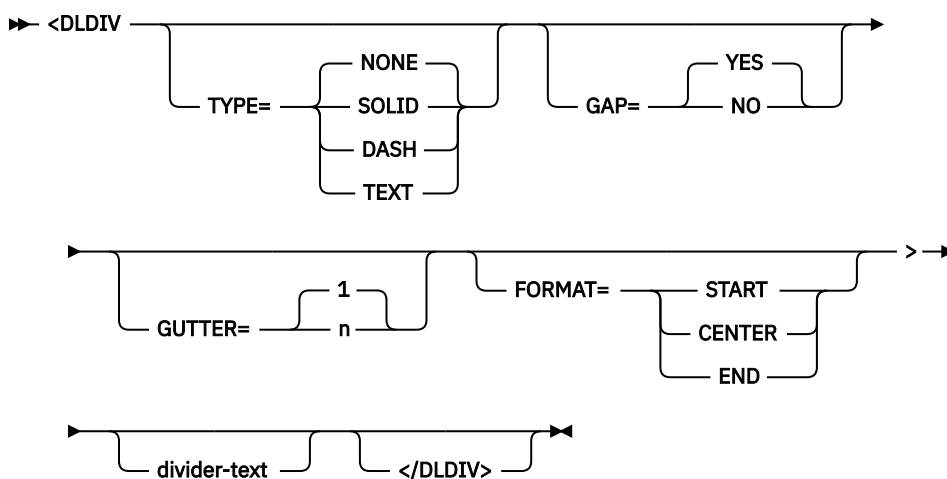
F1=Help	F3=Exit	F5=Exhelp	F6=Keyshelp
F7=PrvTopic	F8=NxtTopic	F10=PrvPage	F11=NxtPage
F12=Cancel			

Figure 105. Definition List (BREAK=ALL)

DLDIV (Definition List Divider)

The DLDIV tag creates a blank or visible divider within the text portion of an application panel.

Syntax



Parameters

TYPE=NONE | SOLID | DASH | TEXT

This attribute specifies the type of divider line. The line width is one character.

The default value is NONE, which produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

GAP=YES | NO

When GAP=NO, the divider line completely crosses from one side of the text area to the other. When GAP=YES, a 1-character gap remains at each end of the divider line.

GUTTER=1 | n

This attribute specifies the total width of the divider. If the GUTTER value is an even number, the conversion utility increases the number by 1 so that the divider is centered within the defined width.

The minimum GUTTER value, and the default, is 1.

FORMAT=START | CENTER | END

This attribute specifies the position of the divider text within the width of the divider line.

divider-text

This is the text of the area divider line.

Comments

The DLDIV tag creates a blank or solid divider within the text portion of an application panel. A horizontally formatted visible divider is created when you specify the TYPE attribute value as SOLID or DASH. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

The divider line can be formatted with descriptive text. When this feature is used, the FORMAT attribute must be specified. If FORMAT is not specified, the tag text is ignored. You control the text padding with the TYPE attribute. If TYPE=TEXT, the *divider-text* is padded with blanks. When TYPE=SOLID or TYPE=DASH, the *divider-text* is padded with the specified character.

Restrictions

- You must code the DLDIV tag within a DL tag.

Processing

Table 27. The tags you can code within a DLDIV definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is an example that shows the use of the DLDIV tag in combination with the multiple DT tag function and the DIVEND attribute of the DL tag. [Figure 106 on page 267](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM>

<HELP NAME=dldiv DEPTH=22 WIDTH=60>Employee Code Help
<AREA>
<INFO>
  <P>The following list defines the valid employee codes.
  <DL TSIZE=14 BREAK=none>
    <DLDIV TYPE=SOLID>
      <DTHD>Code
      <DDHD>Meaning
      <DLDIV TYPE=SOLID>
        <DT NOSKIP>Full-time
        <DD>Indicates that the employee works a
          regular schedule of 40 hours or more weekly.
        <DT>Part-time
        <DD>Indicates that the employee works a regular
          schedule of 20 to 40 hours weekly.
        <DT>Supplemental
        <DD>Indicates that the employee works less than
          20 hours weekly.
          No regular schedule is in place.
      </DL>
    </INFO>
  </AREA>
</HELP>

```

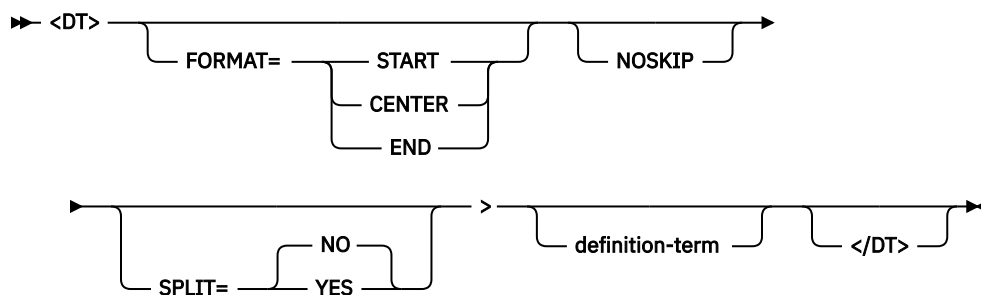
Employee Code Help	
The following list defines the valid employee codes.	
Code	Meaning
Full-time	Indicates that the employee works a regular schedule of 40 hours or more weekly.
Part-time	Indicates that the employee works a regular schedule of 20 to 40 hours weekly.
Supplemental	Indicates that the employee works less than 20 hours weekly. No regular schedule is in place.

Figure 106. Definition list dividers

DT (Definition Term)

The DT tag defines a term in a definition list.

Syntax



Parameters

FORMAT=START | CENTER | END

This attribute specifies the placement of the DT tag text within the space specified by TSIZE. The DT tag FORMAT setting overrides the FORMAT specified in the enclosing DL tag.

NOSKIP

This attribute causes the definition term to be formatted without a blank line before the term. It is used to control the formatting of the definition term when COMPACT has not been specified on the enclosing DL tag. When the DL tag TSIZE attribute specifies that multiple DT tags are to be formatted for each DD tag, NOSKIP should be coded on the first DT tag. It is ignored for the second and subsequent DT tags.

SPLIT

This attribute controls the format of the last DT tag in a multiple DT tag group. It is used only when BREAK=ALL or when BREAK=FIT and the DT tag text length exceeds the TSIZE value. When SPLIT=YES, the text following the last DT tag in the DT group (typically one or two dashes) is placed in front of the first line of the formatted DD tag text. The DT tag SPLIT setting overrides the SPLIT specified in the enclosing DL tag.

definition-term

This is the text of the definition term.

Comments

The DT tag defines a term in a definition list.

Restrictions

- You must code the DT tag within a DL definition. See [“DL \(Definition List\)” on page 261](#) for a complete description of this tag.
- Each DT tag must be paired with and precede a DD tag.

Processing

<i>Table 28. The tags you can code within a DT definition</i>			
Tag	Reference	Usage	Required
DTSEG	“DTSEG (Definition Term Segment)” on page 290	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains a definition list with three definition terms. Each definition term is paired with an associated definition description. [Figure 107 on page 269](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM>

<HELP NAME=dt DEPTH=22 WIDTH=64>Help for Markup
<AREA>
<INFO>
  <P>Here are some definitions:
  <DL TSIZE=2 BREAK=all>
    <DT>markup
    <DD>Text that is added to document data in order to
    convey information about it.
    There are three types of markup the DTL uses: tags, references,
    and markup declarations.
    <DT>markup declaration
    <DD>Markup that controls how other markup of a document
    is to be interpreted, for example document type and entity declarations.
    <DT>markup language
    <DD>A set of characters, conventions, and rules to control
    the interpretation of document data.
    The Dialog Tag Language is a markup language.
  </DL>
</INFO>
</AREA>
</HELP>

```

Help for Markup

Here are some definitions:

markup

Text that is added to document data in order to convey information about it. There are three types of markup the DTL uses: tags, references, and markup declarations.

markup declaration

Markup that controls how other markup of a document is to be interpreted, for example document type and entity declarations.

markup language

A set of characters, conventions, and rules to control the interpretation of document data. The Dialog Tag Language is a markup language.

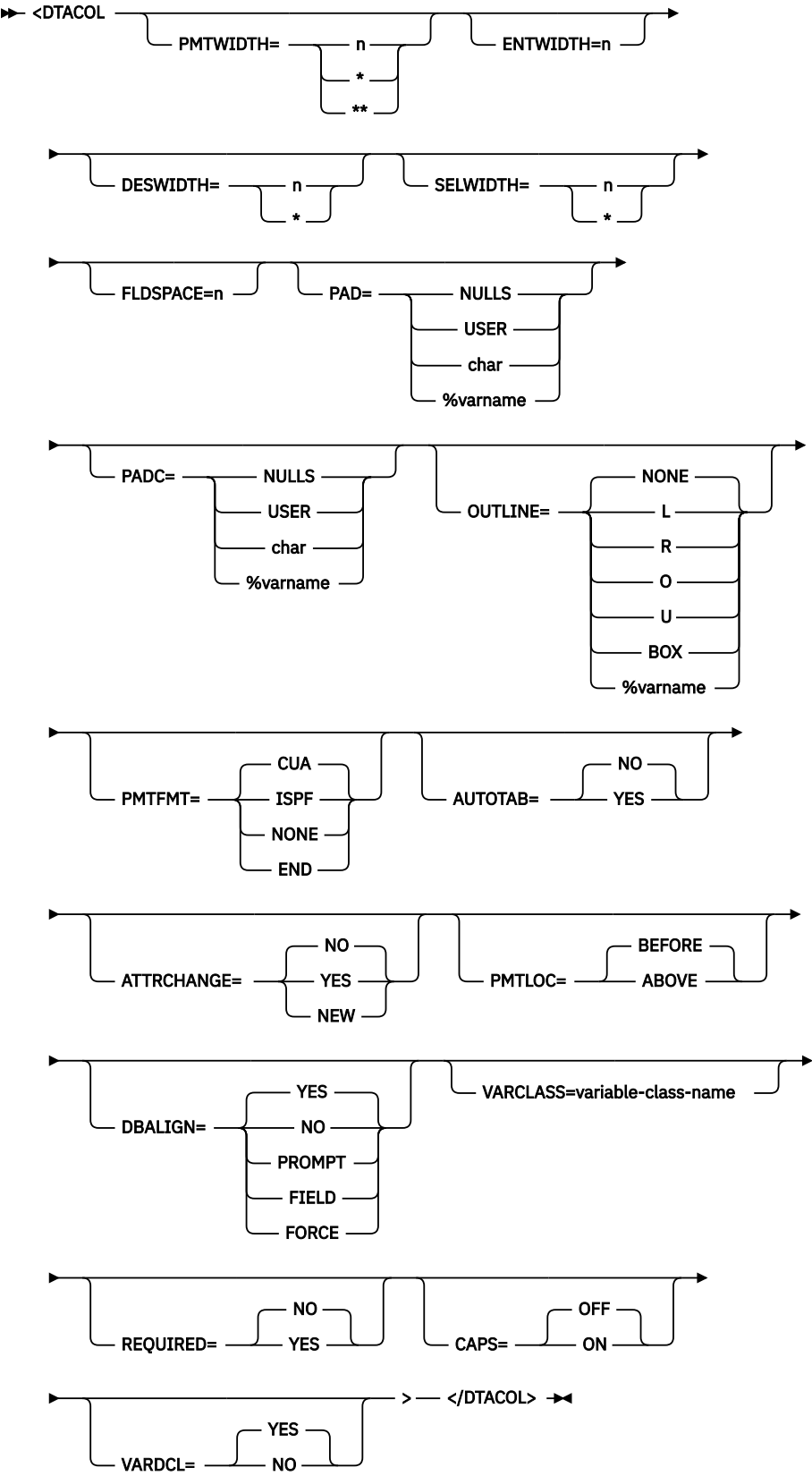
F1=Help	F3=Exit	F5=Exhelp	F6=Keyshelp
F7=PrvTopic	F8=NxtTopic	F10=PrvPage	F11=NxtPage
F12=Cancel			

Figure 107. Definition Terms

DTACOL (Data Column)

The DTACOL tag defines default values for data fields (DTAFLD) and selection fields (SELFLD) that are coded within a DTACOL definition.

Syntax



Parameters

PMTWIDTH=n | * | **

This attribute specifies the number of bytes reserved for prompts for data fields and selection fields coded within the data column. The minimum width is 0 and the maximum is the remaining available panel width. When you specify PMTWIDTH=*, the conversion utility uses the length of the prompt text as the prompt width. When you specify PMTWIDTH=**, the conversion utility uses the maximum available space as the prompt width. If PMTFMT=CUA is specified (or defaulted) and the prompt text has fewer characters than the field allows, leader dots fill the remaining spaces. For output-only data fields, a colon is also added as the last character in the prompt width space. If any prompt contains more characters than the width you specify, the prompt is word-wrapped to fit on multiple lines.

Note: Any field within the data column defining a prompt width overrides the DTACOL PMTWIDTH value.

ENTWIDTH=n

This attribute specifies the number of bytes reserved for data fields coded within the data column. The minimum width is 1 and the maximum is the remaining available panel (or region) width.

Note: Any data field within the data column defining an entry width overrides the DTACOL ENTWIDTH value.

DESWIDTH=n | *

This attribute specifies the number of bytes reserved for the description text of the enclosed DTAFLDD tags. The minimum width is 0. When you specify DESWIDTH=*, the conversion utility uses the length of the description text as the description width. If the text is longer than the width you specify, the text is word-wrapped to fit on multiple lines.

Note: Any data field within the data column defining a description width overrides the DTACOL DESWIDTH value.

SELWIDTH=n | *

This attribute specifies the number of bytes reserved for choices in selection fields coded within the data column. The minimum width value is 1 and the maximum is the remaining available panel width. If the width required by the *choice-text* and its entry field exceeds the specified SELWIDTH value, the text is wrapped to multiple conversion utility will use the remaining available panel (or region) width.

Note: Any selection field within the data column defining a selection width overrides the DTACOL SELWIDTH value.

FLDSPACE=n

This attribute specifies the number of bytes reserved for the data field. The minimum width is 2 and the maximum is the remaining available panel (or region) width. The FLDSPACE value should include the actual entry width plus the number of entry field attributes. If the value specified by ENTWIDTH (plus attributes) is less than the specified FLDSPACE value, the entry field is padded with blanks to the FLDSPACE value. This creates blank space between the entry field and description text provided by the DTAFLDD tag and allows you to align description text from successive DTAFLD definitions.

Note: Any data field within the data column defining field space overrides the DTACOL FLDSPACE value.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a "%".

Note: Any data field within the data column defining PAD overrides the DTACOL PAD value.

PADC=NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a "%".

Note: Any data field within the data column defining PADC overrides the DTACOL PADC value.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

Note: Any data field within the data column defining OUTLINE overrides the DTACOL OUTLINE value.

PMTFMT=CUA | ISPF | NONE | END

This attribute controls the generation of prompt leader characters. The default is to create CUA leader dots. When ISPF is specified, and at least 4 bytes of prompt text space remain following the prompt text, the “===>” character string is placed in the rightmost 4 positions of the prompt text space. When NONE is specified, no leader characters are added to the prompt text. When END is specified, the prompt text is right justified within the prompt text space.

Note: Any data field within the data column defining PMTFMT overrides the DTACOL PMTFMT value.

AUTOTAB=NO | YES

When AUTOTAB=YES, the cursor moves to the next field capable of input when the user enters the last character in this field. If no other field capable of user input exists on the panel, the cursor returns to the beginning of this field.

Note: Any data field within the data column defining AUTOTAB overrides the DTACOL AUTOTAB value.

ATTRCHANGE=NO | YES | NEW

When ATTRCHANGE=YES or ATTRCHANGE=NEW, the conversion utility formats an additional entry in the panel)ATTR section (that can apply to multiple data fields) instead of creating a unique ".ATTR(field-name)" entry in the)INIT section for each field. With this option, multiple DTAFLD tags with the same characteristics require fewer panel logic statements. ATTRCHANGE=NEW creates a new entry. ATTRCHANGE=YES uses an existing attribute, if possible.

Note: Any data field within the data column defining ATTRCHANGE overrides the DTACOL ATTRCHANGE value.

PMTLOC=BEFORE | ABOVE

This attribute defines the prompt location for the enclosed DTAFLD and SELFLD tags.

Note: Any data field or selection field within the data column defining PMTLOC overrides the DTACOL PMTLOC value.

DBALIGN=YES | NO | PROMPT | FIELD | FORCE

This attribute defines the DBALIGN value for the enclosed DTAFLD tags.

Note: Any data field within the data column defining DBALIGN overrides the DTACOL DBALIGN value.

VARCLASS=variable-class-name

This attribute defines the name of the variable class for enclosed CHOFLD and DTAFLD tags.

Note: Any data field within the data column defining VARCLASS overrides the DTACOL VARCLASS value.

REQUIRED=NO | YES

This attribute defines whether the fields for enclosed CHOFLD and DTAFLD tags *require* input.

Note: Any data field within the data column defining REQUIRED overrides the DTACOL REQUIRED value.

CAPS=OFF | ON

This attribute defines whether the fields for enclosed CHOFLD and DTAFLD tags are displayed in uppercase characters.

Note: Any data field within the data column defining CAPS overrides the DTACOL CAPS value.

VARDCL=YES | NO

When VARDCL=NO the data field name is not checked to the declared variable information provided with the VARCLASS and VARDCL tags for enclosed CHOFLD and DTAFLD tags.

Note: Any data field within the data column defining VARDCL overrides the DTACOL VARDCL value.

Comments

The DTACOL tag defines default attribute values for data fields (DTAFLD), choice data fields (CHOFLD), and selection fields (SELFLD) that are coded within a DTACOL definition. This allows you to define common values for fields coded within the data column within a single tag definition.

The xxxWIDTH attributes are convenient for aligning fields on an application panel. Fields are laid out within the data column along boundaries established by the values specified on the DTACOL tag. This example shows those boundaries:

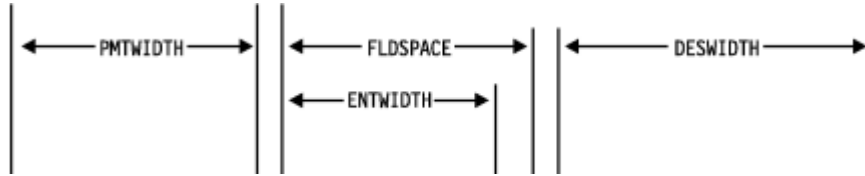


Figure 108. PMTWIDTH, ENTWIDTH, FLDSPACE, and DESWIDTH attributes

The prompt width (PMTWIDTH) is valid for data fields and selection fields coded within the data column description. The entry width (ENTWIDTH), field space (FLDSPACE), and description width (DESWIDTH) are only used by enclosed DTAFLD tags. The selection width (SELWIDTH) is used only by enclosed SELFLD tags. All of the previous cases stated are true only when the enclosed DTAFLD or SELFLD tags do not specify values that override the DTACOL values.

Note: The SELFLD tag does not use the ENTWIDTH, DESWIDTH, FLDSPACE, PAD, PADC, OUTLINE, AUTOTAB, ATTRCHANGE, DBALIGN, VARCLASS, REQUIRED, or CAPS attributes of the DTACOL tag.

If the combined PMTWIDTH, ENTWIDTH, and DESWIDTH values exceed the remaining available panel (or region) width, the conversion utility issues a warning message and attempts to fit the data in the available width by wrapping the text.

For data fields, first priority is given to the entry field. Second and third priorities are given to the prompt and description fields, respectively. These fields use the available width remaining after the width of the entry field is determined.

Note: Word wrapping can result in word truncation if insufficient width is available for the text.

When the maximum number of requested attributes for a panel is exceeded, the conversion utility issues error message ISPC804E. The number of requested attributes includes attribute override entries. These are .ATTR entries that are added by the conversion utility for attributes that are specified on CHOFLD, DTACOL, DTAFLD, LSTCOL, and LSTFLD tags. If the same set of attributes is specified on multiple tags, duplicate .ATTR entries are added by default. Adding the parameter ATTRCHANGE=YES to the tags causes the compiler to instead add a single entry in the panel)ATTR section for each unique set of attributes specified. The entry for a set of attributes is then shared by all tags that specify that set of attributes.

Restrictions

- The DTACOL tag requires an end tag.
- You must code the DTACOL tag within an AREA, PANEL, or REGION definition. You can code a DTACOL definition anywhere within these tags, but the start and end tags must enclose any DTAFLD or SELFLD tags to which it applies. See [“AREA \(Area\)” on page 189](#), [“PANEL \(Panel\)” on page 376](#), and [“REGION \(Region\)” on page 405](#) for descriptions of these tags.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

Table 29. The tags you can code within a DTACOL definition			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
DTAFLD	“DTAFLD (Data Field)” on page 275	Multiple	No
GRPHDR	“GRPHDR (Group Header)” on page 300	Multiple	No
SELFLD	“SELFLD (Selection Field)” on page 421	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is application panel markup that contains a data column that provides default width values for the enclosed data fields and data field descriptions. The ENTWIDTH value specified on the first and second data fields override the ENTWIDTH value specified on the DTACOL tag. [Figure 109 on page 275](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=dtacol2 KEYLIST=keylxml>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST> Type in patron's name and card number (if applicable)
<TOPINST> Then select an action bar choice.
<AREA>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25
    <DTAFLD DATAVAR=curdate USAGE=out ENTWIDTH=8>Date
    <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
      <DTAFLDD>(A 7-digit number)
    <DTAFLD DATAVAR=name>Name
      <DTAFLDD>(Last, First, M.I.)
    <DTAFLD DATAVAR=address>Address
  </DTACOL>
  <DIVIDER>
  <REGION DIR=horiz>
  <SELFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=card MATCH=new>New
  <CHOICE CHECKVAR=card MATCH=renew>Renewal
  <CHOICE CHECKVAR=card MATCH=replace>Replacement
  </SELFLD>
  <SELFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
  <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
  <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
  <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
  <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
  </SELFLD>
  </REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

```

File  Search  Help
-----
                        Library Card Registration

Type in patron's name and card number if applicable.
Then select an action bar choice.

Date . . . : 08/29/90
Card No. . . : _____ (A 7-digit number)
Name . . . : _____ (Last, First, M.I.)
Address . . : _____

Choose one of the following          Check valid branches
--  1.  New                          - North Branch
    2.  Renewal                      - South Branch
    3.  Replacement                  - East Branch
                                      - West Branch

Enter a command ==>
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel

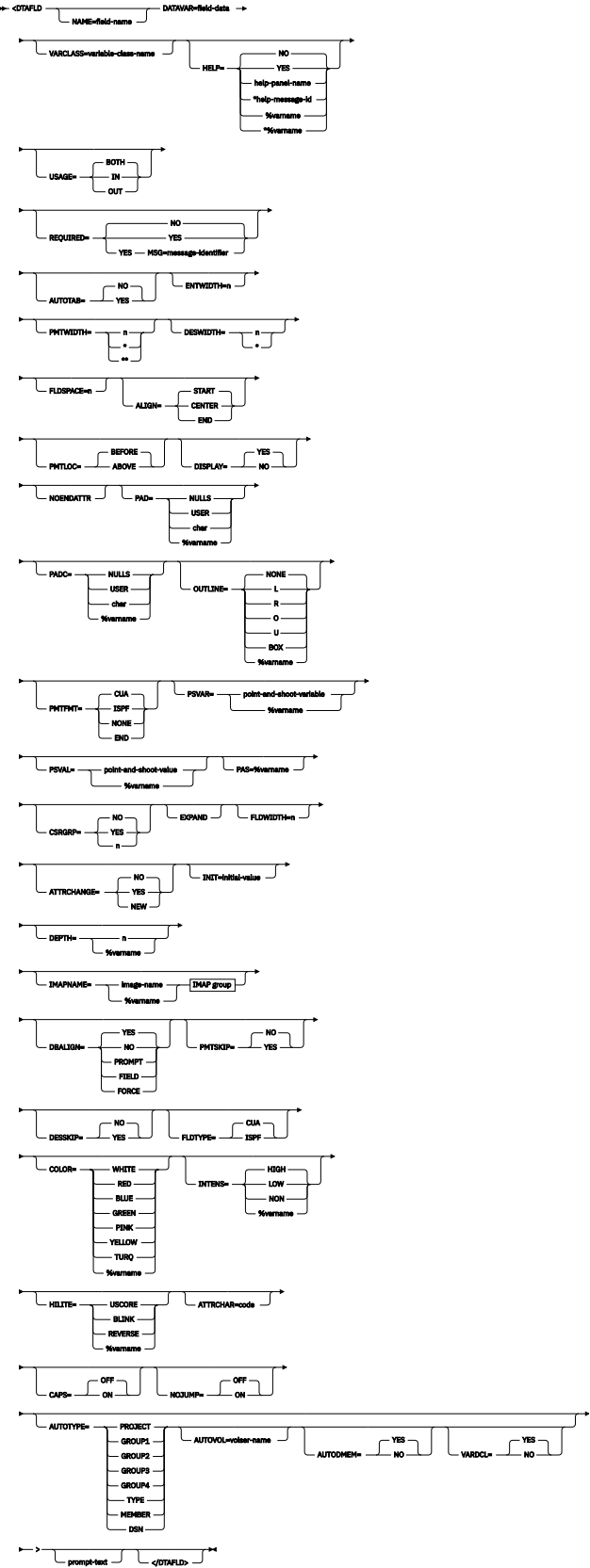
```

Figure 109. Data column

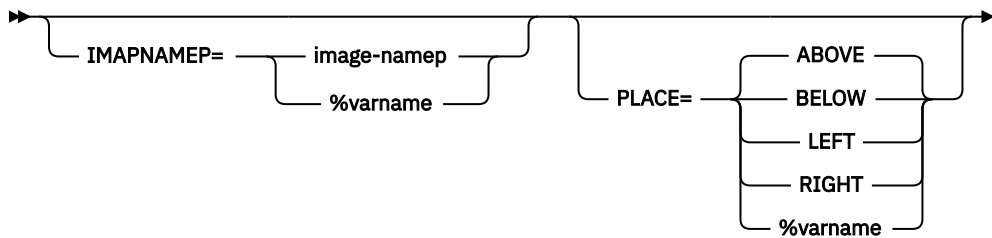
DTAFLD (Data Field)

The DTAFLD tag defines an input field, an output field, or an input/output field on an application panel.

Syntax



IMAP group



Parameters

NAME=field-name

This attribute specifies the name of the field. The *field-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

The *field-name* can be used by:

- The PANEL tag to position the cursor
- The ISPF DISPLAY or TBDISPL services to position the cursor
- The ISPF ADDPOP service to position a pop-up.

DATAVAR=field-data

This attribute specifies the variable name for the data in the field. The value coded must be a variable-name without the leading % notation. The conversion utility considers NAME and DATAVAR to be synonymous. However, the value you assign DATAVAR has precedence. For example, if you specify different values for the DATAVAR and NAME attributes, the conversion utility uses the DATAVAR value as the name of the field on the panel.

Compatibility considerations

DATAVAR is a required attribute for the DTAFLD tag. For compatibility between releases, you can code either the NAME or the DATAVAR attributes, or both.

VARCLASS=variable-class-name

This attribute specifies the name of the variable class, defined using a VARCLASS tag, that overrides the default variable class referred to by the VARDCL that declared the data variable for this field.

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for this data field. This is field-level help.

When HELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help for the data field and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

USAGE=BOTH | IN | OUT

This attribute indicates whether the field is for input only, output only, or both.

For USAGE=OUT, the conversion utility inserts a colon as the last character of the data field prompt to indicate to the user that it is an output-only field.

REQUIRED=NO | YES

This attribute indicates if the field requires input. This attribute is valid only when USAGE=IN or BOTH.

If REQUIRED=YES is coded, a VER(variable, NONBLANK) statement is built by the conversion utility and placed in the)PROC section of the ISPF panel generated.

MSG=message-identifier

This attribute specifies the message that is displayed when the user does not complete a required entry (defined with the REQUIRED attribute). If you do not specify a *message-identifier*, ISPF displays a default message.

If you specify the MSG attribute and REQUIRED=YES, a VER(variable, NONBLANK, MSG=message-identifier) statement is built by the conversion utility and placed in the)PROC section of the ISPF panel generated. If you specify the MSG attribute and REQUIRED=NO (the default), the conversion utility issues a warning message.

See [“MSG \(Message\)” on page 352](#) for information about creating messages.

Note: You can specify messages pertaining to other validations using XLATL and CHECKL tags within a VARCLASS definition. See the descriptions of these tags for additional information.

AUTOTAB=NO | YES

When AUTOTAB=YES, the cursor moves to the next field capable of input when the user enters the last character in this field. If no other field capable of user input exists on the panel, the cursor returns to the beginning of this field.

AUTOTAB=YES is valid only when the value for USAGE is either BOTH or IN. If specified, this attribute overrides the AUTOTAB value of the DTACOL tag.

ENTWIDTH=n

This attribute specifies the number of bytes used for the data field. The minimum width is 1 and the maximum is the remaining available panel width less the required amount of space for field attributes. If ENTWIDTH is not provided on either the DTAFLD tag or the enclosing DTACOL tag, the conversion utility uses the width determined by the TYPE value of the associated VARCLASS.

If specified, this attribute overrides the ENTWIDTH value of the DTACOL tag.

PMTWIDTH=n | * | **

This attribute specifies the number of bytes used for the data field *prompt-text*. The minimum width is 0 and the maximum is the remaining available panel (or region) width less the required amount of space for field attributes. When you specify PMTWIDTH=*, the conversion utility uses the length of the prompt text as the prompt width. When you specify PMTWIDTH=**, the conversion utility uses the maximum available space as the prompt width. If PMTFMT=CUA is specified (or defaulted) and the *prompt-text* has fewer characters than the field allows, leader dots fill the remaining spaces. If any prompt contains more characters than the width you specify, the prompt is word-wrapped to fit on multiple lines. If PMTWIDTH is not specified and *prompt-text* is present, the PMTWIDTH value defaults to the length of the *prompt-text*.

If specified, this attribute overrides the PMTWIDTH value of the DTACOL tag.

DESWIDTH=n | *

This attribute specifies the number of bytes used for the description text of enclosed DTAFLDD tags. The minimum width is 0. When you specify DESWIDTH=*, the conversion utility uses the length of the description text as the description width. If the text is longer than the width you specify, the text is word-wrapped to fit on multiple lines.

If specified, this attribute overrides the DESWIDTH value of the DTACOL tag.

FLDSpace=n

This attribute specifies the number of bytes reserved for the data fields coded within the data column. The minimum width is 2 and the maximum is the remaining available panel (or region) width. The FLDSpace value should include the actual entry width plus the number of entry field attributes. If the value specified by ENTWIDTH is less than the specified FLDSpace value, the entry field is padded with blanks to the FLDSpace value. This creates blank space between the entry field and description

text provided by the DTAFLDD tag and allows you to align description text from successive DTAFLD definitions.

If specified, this attribute overrides the FLDSpace value of the DTACOL tag.

ALIGN=START | CENTER | END

This attribute specifies the alignment of data within the display field after all translations have been performed. Use this attribute to align the data with the start, the end, or the center of the display field.

This is accomplished in the conversion utility by using an attribute character for the field that specifies JUST(LEFT) if ALIGN=START or JUST(RIGHT) if ALIGN=END. ALIGN=Center uses an attribute character for the field that specifies JUST(ASIS).

Alignment occurs if the transformed value of the dialog variable is shorter than the display width of the field. When ALIGN=END, no underscore is padding performed. Instead, blanks are used.

PMTLOC=BEFORE | ABOVE

This attribute specifies whether the *prompt-text* of the data field appears above or in front of the data field.

DISPLAY=YES | NO

This attribute specifies whether data displays on the screen as the user types it in. If you specify NO, the data is not displayed. This attribute is useful when creating fields for passwords or other information which you do not want to appear on the screen.

NOENDATTR

This attribute, which is valid only when WINDOW=NO is specified on the PANEL tag or DIR=HORIZ is specified on the REGION tag, specifies that no ending attribute is placed after the data field. NOENDATTR is ignored for the last field on each panel line unless WINDOW=NO has been specified on the PANEL tag. NOENDATTR is valid only when the DTAFLD tag is followed by a DTAFLD, DTAFLDD, DIVIDER, or SELFLD tag.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

If specified, this attribute overrides the PAD value of the DTACOL tag.

PADC= NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

If specified, this attribute overrides the PADC value of the DTACOL tag.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

If specified, this attribute overrides the OUTLINE value of the DTACOL tag.

PMTFMT=CUA | ISPF | NONE | END

This attribute controls the generation of prompt leader characters. The default is to create CUA leader dots. When ISPF is specified, and at least 4 bytes of prompt text space remain following the prompt text, the “==>” character string is placed in the rightmost 4 positions of the prompt text space. When NONE is specified, no leader characters are added to the prompt text. When END is specified, the prompt text is right justified within the prompt text space.

If specified, this attribute overrides the PMTFMT value of the DTACOL tag.

PSVAR=point-and-shoot-variable | %varname

This attribute provides the name of a variable that is to be set when a DTAFLD is clicked on for point-and-shoot selection. You can define this attribute as a variable name preceded by a “%”.

The *point-and-shoot-variable* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

PSVAL=point-and-shoot-value | %varname

This attribute provides the value to be placed in the field specified by the PSVAR attribute. You can define this attribute as a variable name preceded by a "%". To specify a blank value, the " ' ' " (quotation mark, apostrophe, blank, apostrophe, quotation mark) coding notation should be used.

PAS=%varname

This attribute can be used to provide a variable name to specify ON or OFF for point-and-shoot. When PSVAR and PSVAL have been specified without the PAS attribute, the point-and-shoot field is automatically enabled.

CSRGRP=NO | YES | N

When CSRGRP=YES, the conversion utility generates a cursor group number to be used for this data field. When CSRGRP=n, the number provided is used for this field. The PAS attribute must be specified as %varname.

The CSRGRP attribute is accepted for all data fields. It is used at run time for output fields only.

EXPAND

The EXPAND attribute, used in combination with EXPAND=xy on the PANEL definition, causes the expand characters to be added to the DTAFLD definition, effectively allowing ENTWIDTH to expand. The ENTWIDTH value must be specified as 4 or greater for the EXPAND function to operate.

Note: If the PANEL tag attribute EXPAND is defined with no value specified, the DTAFLD tag EXPAND attribute is not used.

FLDWIDTH=n

The FLDWIDTH attribute, used in combination with WINDOW=NO on the PANEL definition, provides the width of a DTAFLD which spans multiple lines.

FLDWIDTH cannot be used within any horizontal region.

ATTRCHANGE=NO | YES | NEW

When ATTRCHANGE=YES or ATTRCHANGE=NEW, the conversion utility formats an additional entry in the panel)ATTR section (that can apply to multiple data fields) instead of creating a unique ".ATTR(field-name)" entry in the)INIT section for this field. With this option, multiple DTAFLD tags with the same characteristics require fewer panel logic statements. ATTRCHANGE=NEW creates a new entry. ATTRCHANGE=YES uses an existing entry, if possible.

INIT=initial-value

When the INIT attribute is specified, the conversion utility adds a statement to the panel)INIT section to initialize the field to the *initial-value*.

DEPTH=n | %varname

This attribute defines the depth reserved for the field. The minimum value is 1 and the maximum value is the remaining panel depth. This attribute is accepted in order to support existing DTL source files that use it. However, although the space is reserved, point-and-shoot does not function in the additional reserved space..

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAMEP=image-namep | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PLACE=ABOVE | BELOW | LEFT | RIGHT

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

DBALIGN=YES | NO | PROMPT | FIELD | FORCE

This attribute defines the DBALIGN value. DBALIGN is used only for DBCS language conversions when PMTLOC=ABOVE and the DBALIGN invocation option is specified.

When DBALIGN=PROMPT, the start position of the prompt-text is shifted 1 position to the right.

When DBALIGN=FIELD, the start position of the data field is shifted 1 position to the right.

When DBALIGN=YES, and the prompt-text starts with a DBCS character, the data field is shifted. If DBALIGN=YES and the prompt text starts with an SBCS character or the prompt text is not provided, no shifting is done.

When DBALIGN=FORCE, both the prompt-text and the data field are shifted. DBALIGN=YES and DBALIGN=FORCE are useful to align a DTAFLD with another DTAFLD or SELFLD tag.

When DBALIGN=NO, no alignment adjustment is made.

PMTSKIP=NO | YES

This attribute is used for horizontal formatting of input fields. When PMTSKIP=YES, and the previous DTAFLD definition includes the NOENDATTR attribute, the cursor moves past the prompt text to the input field when the user enters the last character in the previous field. If there is no other input field on the panel, the cursor returns to the first input field on the panel.

DESSKIP=NO | YES

This attribute is used for horizontal formatting of input fields. When DESSKIP=YES, and the current DTAFLD definition includes the NOENDATTR attribute, the cursor skips over the description text provided by the DTAFLDD tag to the next input field when the user enters the last character in the current field. If there is no other input field on the panel, the cursor returns to the first input field on the panel.

FLDTYPE=CUA | ISPF

This attribute defines the attribute type to be applied to the field. TYPE=CUA, the default, causes the field to display using the standard CUA attribute. When FLDTYPE=ISPF, a non-CUA attribute entry is generated for the)ATTR section, and you can specify the color, intensity, and highlighting of the attribute. See the COLOR, INTENS, and HILITE attributes that follow for more information. These attributes are not valid when FLDTYPE=CUA.

Note: IF DISPLAY=NO is specified, an .ATTR(...) is created to override this field.

COLOR=WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENS=HIGH | LOW | NON | %varname

This attribute defines the intensity of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

HILITE=USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

ATTRCHAR=code

This attribute can be a single character or a two-position entry of valid hex digits. If you enter an incorrect value, a warning message is issued and the value is set to null. Hex entries are converted to character. Hex values '00'-'2F' are reserved for use by the conversion utility.

CAPS=OFF | ON

When CAPS=ON, the data in the field is displayed in uppercase characters.

NOJUMP=OFF | ON

When NOJUMP=ON, the JUMP function is disabled for the field.

AUTOTYPE=PROJECT | GROUP1 | GROUP2 | GROUP3 | GROUP4 | TYPE | MEMBER | DSN

This attribute specifies that ISPF panel logic be added to support the AUTOTYPE function.

AUTOTYPE=DSN is specified for data set name fields.

The other attribute values are used for ISPF- format project, group, type, and member name fields.

Multiple data fields can be specified with AUTOTYPE=DSN. Only one field can be specified with each of the other listed attribute values.

AUTOVOL = volser name

This attribute specifies an associated panel field for volume name when AUTOTYPE=DSN.

AUTODMEM = YES | NO

This attribute specifies whether a member name is part of the data set name when AUTOTYPE=DSN.

VARDCL = YES | NO

When VARDCL=NO the field name is not checked to the declared variable information provided with the VARCLASS and VARDCL tags.

prompt-text

This is the prompt text for the data field. The *prompt-text* appears in front of or above the field, depending on the setting of the PMTLOC attribute. If you do not specify prompt text, no text appears for the field.

If the *prompt-text* exceeds the width defined for a prompt, it is word-wrapped to multiple lines.

Comments

The DTAFLD tag defines an input field, an output field, or an input/output field on an application panel.

The formatted width of the field is 2 positions more than the ENTWIDTH value to provide for an attribute byte both before and after the field.

If PMTLOC=ABOVE, an attribute is placed both before and after the prompt text reserved space. If PMTLOC=BEFORE (or PMTLOC is not specified), and the DTAFLD is being formatted in a horizontal region, then an additional byte is used for the field prompt attribute when the field prompt is not at the left edge of the panel.

The DTAFLDD tag can be used to provide the description text for the data field.

When the maximum number of requested attributes for a panel is exceeded, the conversion utility issues error message ISPC804E. The number of requested attributes includes attribute override entries. These are .ATTR entries that are added by the conversion utility for attributes that are specified on CHOFLD, DTACOL, DTAFLD, LSTCOL, and LSTFLD tags. If the same set of attributes is specified on multiple tags, duplicate .ATTR entries are added by default. Adding the parameter ATTRCHANGE=YES to the tags causes the compiler to instead add a single entry in the panel)ATTR section for each unique set of attributes specified. The entry for a set of attributes is then shared by all tags that specify that set of attributes.

Restrictions

- You must code the DTAFLD tag within an AREA, DTACOL, PANEL, or REGION definition. See [“AREA \(Area\)” on page 189](#), [“DTACOL \(Data Column\)” on page 269](#), [“PANEL \(Panel\)” on page 376](#), and [“REGION \(Region\)” on page 405](#) for descriptions of these tags.
- The variable name specified in the DATAVAR attribute should have an associated VARDCL definition. See [“VARDCL \(Variable Declaration\)” on page 449](#) for a complete description of this tag.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

Table 30. The tags you can code within a DTAFLD definition			
Tag	Reference	Usage	Required
ASIGNL	“ASSIGNL (Assignment List)” on page 196	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DTAFLDD	“DTAFLDD (Data Field Description)” on page 284	Multiple	No

Table 30. The tags you can code within a DTAFLD definition (continued)

Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SCRFLD	“SCRFLD (Scrollable Field)” on page 413	Single	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is source file markup that contains an application panel with three data fields and the variable declarations and classes associated with the data fields. The **Date** field is an output-only field that displays the current date. The **Name** and **Password** fields are input/output fields. The **Password** field is defined as a required field, and specifies DISPLAY=NO, so the user input for this field is not displayed. A data column specifying a default prompt width for the data fields is also defined. [Figure 110 on page 284](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=date TYPE='char 8'>
<VARCLASS NAME=name TYPE='char 25'>
<VARCLASS NAME=password TYPE='char 8'>

<VARLIST>
  <VARDCL NAME=curname VARCLASS=date>
  <VARDCL NAME=namevar VARCLASS=name>
  <VARDCL NAME=passvar VARCLASS=password>
</VARLIST>

<PANEL NAME=dtafld1 HELP=loghelp>System Logon
<TOPINST>Complete the following fields, then press Enter.
<AREA>
  <DTACOL PMTWIDTH=12>
    <DIVIDER>
      <DTAFLD DATAVAR=curname USAGE=out ENTWIDTH=8 FLDSPACE=27>Date
    <DTAFLDD>(Current Date)
    <DIVIDER>
      <DTAFLD DATAVAR=namevar ENTWIDTH=25 DESWIDTH=25>Name
      <DTAFLDD>(Last, First)
    <DIVIDER>
      <DTAFLD DATAVAR=passvar REQUIRED=yes ENTWIDTH=8 DISPLAY=no>Password
    </DTACOL>
  </AREA>
</PANEL>
```

System Logon

Complete the following fields, then press Enter.

Date . . . : 08/29/90 (Current Date)

Name _____ (Last, First)

Password . .

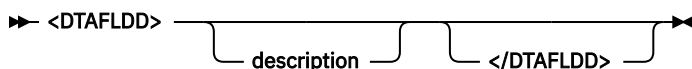
F1=Help F3=Exit F12=Cancel

Figure 110. Data fields

DTAFLDD (Data Field Description)

The DTAFLDD tag defines descriptive text associated with a data field.

Syntax



Parameters

description

This is the descriptive text associated with the data field.

Comments

The DTAFLDD tag defines descriptive text associated with a data field. For example, it could explain what the application user can type into the field.

The text appears in the area defined by the DESWIDTH attribute of the DTAFLD or DTACOL tag.

You can specify more than one DTAFLDD tag for a given field. Each data field description starts a new line.

Restrictions

- You must code the DTAFLDD tag within the DTAFLD definition it is associated with. See [“DTAFLD \(Data Field\)”](#) on page 275 for a complete description of this tag.

Processing

Table 31. The tags you can code within a DTAFLDD definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is application panel markup that contains two data fields that each have associated data field descriptions. [Figure 111 on page 285](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=filecls TYPE='char 8'>
<VARCLASS NAME=copycls TYPE='char 2'>

<VARLIST>
  <VARDCL NAME=file VARCLASS=filecls>
  <VARDCL NAME=copynum VARCLASS=copycls>
</VARLIST>

<PANEL NAME=dtafldd>Print a File
<TOPINST>Type in the name of the file you want to print
and the number of copies, then press Enter.
<AREA>
  <DTAFLD DATAVAR=file PMTWIDTH=12 ENTWIDTH=8 DESWIDTH=30>Filename
    <DTAFLDD>(Maximum of 8 characters)
  <DTAFLD DATAVAR=copynum PMTWIDTH=12 ENTWIDTH=2 DESWIDTH=8>Copies
    <DTAFLDD>(1 - 99)
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

Print a File

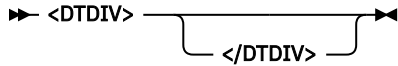
Type in the name of the file you want to print and the number of copies, then press Enter.

```
Filename . . ____ (Maximum of 8 characters)
Copies . . . __ (1 - 99)
```

```
Enter a command ==> _____
F1=Help    F3=Exit    F12=Cancel
```

Figure 111. Data field descriptions

Syntax



Restrictions

Processing

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=dtdiv DEPTH=22 WIDTH=66>Employee Code Help
  <AREA depth=1 extend=on>
    <INFO width=*>
      <P>The following list defines the valid employee codes.
      <DL TSIZE='14 4' BREAK=none COMPACT DIVEND=yes>
        <DLDIV TYPE=solid>
          <DTHDIV>
            <DTHD>Code
            <DTHDIV>
            <DTHD>Flag
            <DTHDIV>
            <DDHD>Meaning
            <DLDIV TYPE=solid>
            <DTDIV>
            <DT NOSKIP>Full-time
            <DTDIV>
            <DT FORMAT=center>F
            <DTDIV>
            <DD>Indicates that the employee works a
              regular schedule of 40 hours or more weekly.
            <DLDIV TYPE=solid>
            <DTDIV>
            <DT>Part-time
            <DTDIV>
            <DT FORMAT=center>P
            <DTDIV>
            <DD>Indicates that the employee works a regular
              schedule of 20 to 40 hours weekly.
            <DLDIV TYPE=solid>
            <DTDIV>
            <DT>Supplemental
            <DTDIV>
            <DT FORMAT=center>S
            <DTDIV>
            <DD>Indicates that the employee works less than
              20 hours weekly.
```

```
No regular schedule is in place.
<DL DIV TYPE=solid>
  </DL>
</INFO>
</AREA>
</HELP>
```

Employee Code Help

The following list defines the valid employee codes.

Code	Flag	Meaning
Full-time	F	Indicates that the employee works a regular schedule of 40 hours or more weekly.
Part-time	P	Indicates that the employee works a regular schedule of 20 to 40 hours weekly.
Supplemental	S	Indicates that the employee works less than 20 hours weekly. No regular schedule is in place.

Figure 112. Definition term divider

DTHD (Definition Term Header)

The DTHD tag defines the heading for the term column of a definition list.

Syntax

➤ <DTHD> — definition-term-header — ➤
 └──────────┘
 </DTHD>

Parameters

definition-term-header

This is the text of the definition term header. The length of the text for the definition term header should be less than the specified TSIZE value in the DL tag. A warning message is issued if the length of the text exceeds the limit.

Comments

The DTHD tag defines the heading for the term column of a definition list. You can code multiple DTHD tags within a definition list.

The conversion utility inserts a blank line between the header and the list items unless the COMPACT attribute is specified on the DL tag.

Restrictions

- You must code the DTHD tag within a DL definition. See [“DL \(Definition List\)”](#) on page 261 for a complete description of this tag.
- Each DTHD tag must be paired with and precede a definition description header (DDHD) tag.

Processing

Table 32. The tags you can code within a DTHD definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains a definition term header with the text “Prefix”. [Figure 113 on page 288](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=dthd DEPTH=18>Prefix Help
<AREA>
<INFO>
  <P>The following list defines each of the valid prefixes.
  <DL TSIZE=12>
    <DTHD>Prefix
    <DDHD>Meaning
    <DT>AU
    <DD>Automotive
    <DT>HB
    <DD>Health and beauty
    <DT>LG
    <DD>Lawn and garden
    <DT>SG
    <DD>Sporting goods
  </DL>
</INFO>
</AREA>
</HELP>
```

Prefix Help

The following list defines each of the valid prefixes.

Prefix	Meaning
AU	Automotive
HB	Health and beauty
LG	Lawn and garden
SG	Sporting goods

F1=Help

F6=Keyshelp

F10=PrvPage

F3=Exit

F7=PrvTopic

F11=NxtPage

F5=Exhelp

F8=NxtTopic

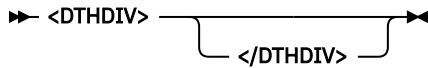
F12=Cancel

Figure 113. Definition term header

DTHDIV (Definition Term Header Divider)

The DTHDIV tag defines a visible vertical divider (|) between multiple DTHD tags.

Syntax



Comments

The DTHDIV tag can be used to create a visual separation between the definition term headings. Each DTHDIV tag adds a vertical bar (plus display control attributes) to the Definition Term Header text.

Restrictions

The DTHDIV tag can be coded before the first DTHD tag, between DTHD tags, or following the last DTHD tag (before the DDHD tag definition).

Processing

None.

Examples

Here is an example that shows the use of the DTHDIV tag in combination with the multiple DT tag function and the DIVEND attribute of the DL tag. Figure 114 on page 290 shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=dthdiv DEPTH=22 WIDTH=66>Employee Code Help
  <AREA depth=1 extend=on>
    <INFO width=*>
      <P>The following list defines the valid employee codes.
      <DL TSIZE='14 4' BREAK=none COMPACT DIVEND=yes>
        <DLDIV TYPE=solid>
          <DTHDIV>
            <DTHD>Code
            <DTHDIV>
            <DTHD>Flag
            <DTHDIV>
            <DDHD>Meaning
            <DLDIV TYPE=solid>
            <DTDIV>
            <DT NOSKIP>Full-time
            <DTDIV>
            <DT FORMAT=center>F
            <DTDIV>
            <DD>Indicates that the employee works a
            regular schedule of 40 hours or more weekly.
            <DLDIV TYPE=solid>
            <DTDIV>
            <DT>Part-time
            <DTDIV>
            <DT FORMAT=center>P
            <DTDIV>
            <DD>Indicates that the employee works a regular
            schedule of 20 to 40 hours weekly.
            <DLDIV TYPE=solid>
            <DTDIV>
            <DT>Supplemental
            <DTDIV>
            <DT FORMAT=center>S
            <DTDIV>
            <DD>Indicates that the employee works less than
            20 hours weekly.
            No regular schedule is in place.
            <DLDIV TYPE=solid>
          </DL>
        </INFO>
      </AREA>
    </HELP>
```

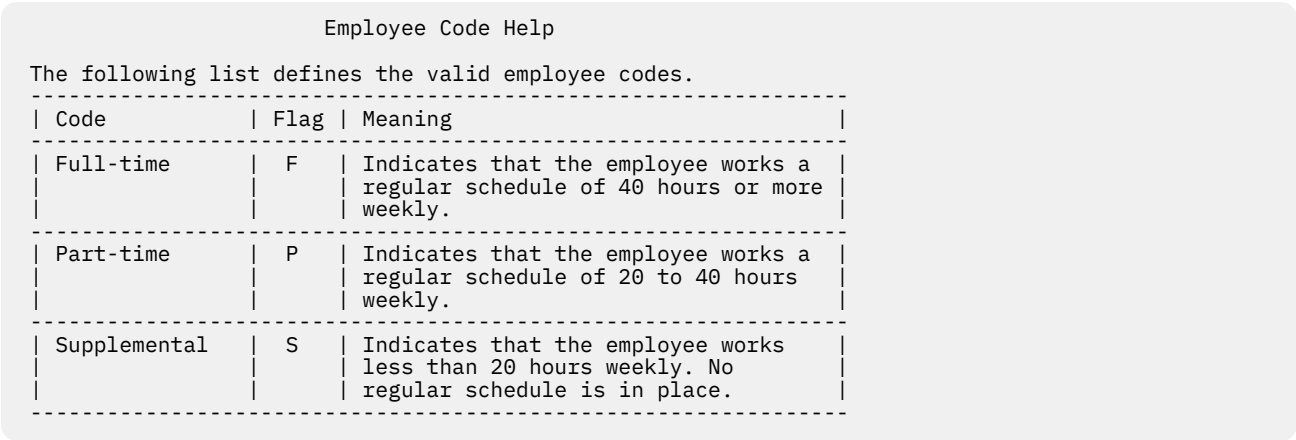
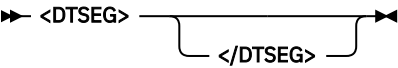


Figure 114. Definition term header divider

DTSEG (Definition Term Segment)

The DTSEG tag defines a segment of the definition term. It is used to provide vertical separation of the DT tag text.

Syntax



Comments

The DTSEG tag is used to create a vertical separation within the definition term. The text following the DTSEG tag is formatted directly under any previous definition term tag text. Multiple DTSEG tags create additional DT text lines.

Use of the DTSEG tag affects the DL tag BREAK attribute. The first (or only) line of DT tag text is processed according to the BREAK attribute of the DL tag. For additional lines, when TSIZE is large enough to accommodate the text segments, the DTSEG text is formatted in front of the associated DD tag text. When TSIZE is not large enough to accommodate the largest segment, all of the DT and DTSEG text is formatted above the associated DD tag text.

Restrictions

- The DTSEG tag can be coded within the text following a DT tag.
- When a DTSEG tag is coded, then all remaining DT tag text for the current DT tag set must follow a DTSEG tag.
- The DT nested tags RP and PS are not supported within DT tag text following any DTSEG tag in a DT/DD tag set.

Processing

Table 33. The tag you can code within a DTSEG definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is an example that shows the use of the DTSEG tag in combination with a multiple DT tag set. The last DT tag includes the SPLIT=yes attribute to format the dash in front of the DD tag text. [Figure 115 on page 291](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM(>

<PANEL NAME=dtseg KEYLIST=ISRHELP APPLID=ISR WINDOW=no PADC=user
      TUTOR ZUP=ISP7R000>Traces - Primary Commands

<CMDAREA CAPS=on>
<AREA DEPTH=1 EXTEND=on>

  <INFO WIDTH=*>
    <P>
      Enter a <hp>Primary Command</hp> in the command input field.
      It is processed after all row modifications and all line commands
      are processed. The following primary commands are valid for the
      Traces options:

    <DL TSIZE="8 1" BREAK=fit INDENT=2>
      <DT>
        LOCATE
        function-name
        (Function Traces) or variable name (Variable Traces)
        <DTSEG>
          LOC or
          <DTSEG>
            L
      <DT SPLIT=yes>-
      <DD>The LOCATE command positions the scrollable display at the
        first (or next) row containing the function name (Function
        Traces option) or the variable name (Variable Traces option).
    </DL>
  </INFO>
</AREA>
</PANEL>
```

```
Tutorial ----- Traces - Primary Commands ----- Tutorial
Command ==> _____
```

Enter a Primary Command in the command input field. It is processed after all row modifications and all line commands are processed. The following primary commands are valid for the Traces options:

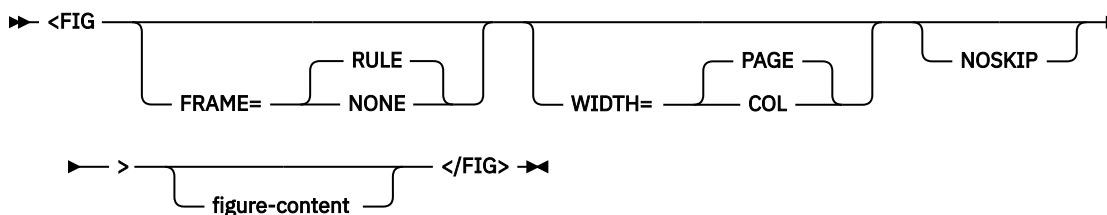
```
LOCATE function-name (Function Traces) or variable-name (Variable Traces)
LOC or   - The LOCATE command positions the scrollable display at the first
L        (or next) row containing the function name (Function Traces
          option) or the variable name (Variable Traces option).
```

Figure 115. Definition term segment

FIG (Figure)

The FIG tag defines the format of text so that it is set off from other text on the panel and retains the format of the enclosed text.

Syntax



Parameters

FRAME=RULE | NONE

This attribute specifies the type of frame to put around the figure.

RULE

Specifies dashed lines appears above and below the figure.

NONE

Specifies no frame; a blank line is left above and below the figure.

WIDTH=PAGE | COL

This attribute specifies where the figure should be aligned. The value PAGE (the default) formats the figure on the original left margin. The value COL formats the figure on the current left margin. The current left margin may be different than the original left margin of the panel if the FIG tag is nested within another tag that causes indenting; the UL tag, for example.

NOSKIP

This attribute causes the blank line normally placed before the figure to be skipped.

figure-content

This is the text of the figure definition.

Comments

The FIG tag defines the format of text so that it is set off from other text on the panel and retains the format of the enclosed text. Tags that normally cause word wrapping within an information region (such as P, NOTE, or PARML) do not cause word-wrapping when nested within a FIG definition. In addition, blank spaces and blank lines in the source are preserved in the figure.

If any DTL source text line is too long to fit in the remaining available formatting width, the data is truncated. A warning message is issued when the first line within the figure is truncated.

A figure can also contain a figure caption, defined with the FIGCAP tag (see [“FIGCAP \(Figure Caption\)”](#) on page 294).

Restrictions

- The FIG tag requires an end tag.
- You must code the FIG tag within an INFO definition. See [“INFO \(Information Region\)”](#) on page 317 for a complete description of this tag.

Processing

Table 34. The tags you can code within a FIG definition			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No

Table 34. The tags you can code within a FIG definition (continued)

Tag	Reference	Usage	Required
FIGCAP	“FIGCAP (Figure Caption)” on page 294	Single	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a figure definition with a ruled frame. The output of the text within the figure definition is identical to the *figure-content*. [Figure 116 on page 294](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=fig DEPTH=20>ShelfBrowse Help
<AREA>
<INFO>
  <FIG>

    We're your local library...

    CHECK US OUT!

  </FIG>
</INFO>
</AREA>
</HELP>
```

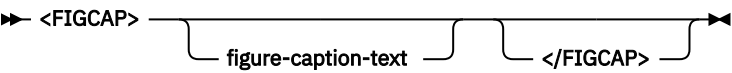


Figure 116. Figure

FIGCAP (Figure Caption)

The FIGCAP tag defines a caption for a figure defined with the FIG tag.

Syntax



Parameters

figure-caption-text

This is the text of the figure caption.

Comments

The FIGCAP tag defines a caption for a figure defined with the FIG tag. The figure caption is formatted below the frame of the figure when FRAME=RULE is specified on the FIG tag.

The conversion utility does not add any blank lines before or after the figure caption.

Restrictions

- You must code the FIGCAP tag within a FIG definition. See [“FIG \(Figure\)” on page 291](#) for a complete description of this tag.
- You can code only one FIGCAP within a FIG definition. Code the FIGCAP tag after the content of the figure, before the FIG end tag.

Processing

Table 35. The tags you can code within a FIGCAP definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No

Table 35. The tags you can code within a FIGCAP definition (continued)

Tag	Reference	Usage	Required
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains a figure definition with an enclosed figure caption. [Figure 117 on page 295](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=figcap DEPTH=20>ShelfBrowse Help
<AREA>
<INFO>
  <FIG>

    We're your local library...

    CHECK US OUT!

  <FIGCAP>Our Motto
  </FIG>
</INFO>
</AREA>
</HELP>
```

```
ShelfBrowse Help

-----

  We're your local library...

  CHECK US OUT!

-----

Our Motto

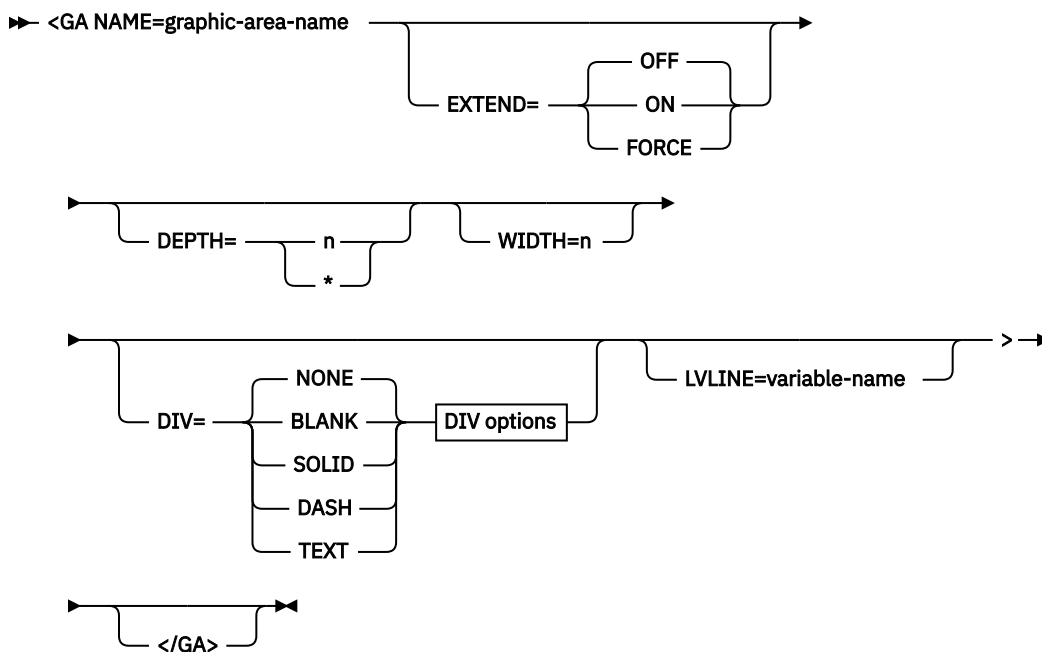

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrivTopic F8=NxtTopic
F10=PrivPage F11=NxtPage  F12=Cancel
```

Figure 117. Figure caption

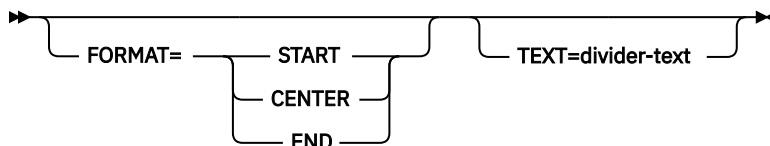
GA (Graphic Area)

The GA tag allows the creation of graphic areas on ISPF panels.

Syntax



DIV options



Parameters

NAME=graphic-area-name

This attribute defines the name of the graphic area. This name is the dialog variable specified by the application that provides the data for the graphic area.

The NAME attribute must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

EXTEND=OFF | ON | FORCE

This attribute defines the runtime display size of the graphic area. If EXTEND=ON is specified, the graphic area definition is expanded to the size of the logical screen. If you intend to display the panel in a pop-up window, use EXTEND=OFF (which is the default).

If EXTEND=FORCE is specified within a horizontal area or region, the EXTEND(ON) keyword is added to the graphic area attribute statement in the)ATTR panel section. The conversion utility issues a message to advise of a potential display error if other panel fields are formatted on or after the last defined line of the graphic area.

DEPTH=n | *

This attribute specifies the number of lines reserved for the graphic area definition. The DEPTH attribute value reserves space within the panel)BODY section. The minimum depth is one line. will reserve the remaining available panel depth for the graphic area.

WIDTH=n

This attribute specifies the number of columns reserved for the graphic area definition. The minimum width is the number of positions in the graphic area name plus 4 and the maximum is 2 positions less than the panel width. The conversion utility places attribute bytes on both sides of the graphic area.

DIV=NONE | BLANK | SOLID | DASH | TEXT

This attribute specifies the type of divider line to be placed before and after the graphic area. If this attribute is not specified or has the value NONE, no divider line is generated. The value BLANK produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line. A visible divider line formats with a non-displayable attribute byte on each end of the line.

FORMAT=START | CENTER | END

This attribute specifies the position of the *divider-text* within the divider line. You must specify both the FORMAT attribute and the TEXT attribute to create a divider line containing text.

TEXT=divider-text

This attribute specifies the text to be placed on the divider line. You must specify both the FORMAT attribute and the TEXT attribute to create a divider line containing text.

LVLIN=variable-name

This attribute allows you to specify the name of a variable which contains the result of the ISPF function LVLIN.

The LVLIN attribute must follow the standard naming convention described in [“Rules for variable names” on page 179](#).

Comments

The GA tag defines a graphic area in the panel)BODY section.

If you specify the CMDAREA tag within your DTL source file, it must appear before the GA tag when DEPTH=* is specified. The GA tag DEPTH may have to be adjusted to allow for additional lines which result from tags present within the panel definition following the end GA tag.

See [z/OS ISPF Dialog Developer's Guide and Reference](#) for a discussion of the graphic area in ISPF panels.

Restrictions

- You must code the GA tag within a PANEL, AREA, or REGION tag. If found anywhere else, an error is logged and the output panel is not saved.
- If NAME is not valid or not specified, an error is logged and the output panel is not saved.
- You can use the EXTEND=ON attribute only once within a panel. If EXTEND is already active, from another GA tag, or from an AREA, DA, SELFLD, or REGION tag, a warning message is logged and the EXTEND attribute is ignored.
- You can code only one GA tag within a PANEL definition.
- You cannot code the GA tag within a scrollable area.

Processing

None.

Examples

```

<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=ga KEYLIST=keylxml>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST> Type in patron's name and card number (if applicable)
<AREA>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
    <DTAFLD DATAVAR=curname USAGE=out ENTWIDTH=8>Name
    <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
    <DTAFLDD>(A 7-digit number)
    <DTAFLD DATAVAR=name>Name
    <DTAFLDD>(Last, First, M.I.)
    <DTAFLD DATAVAR=address>Address
  </DTACOL>
  <DIVIDER>
  <GA NAME=garea DIV=solid DEPTH=6 WIDTH=40>
  </GA>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

GENERATE (Generate)

The GENERATE tag provides direct formatting for)BODY and)AREA panel sections.

Syntax



Parameters

SUBSTITUTE=NO | YES

The SUBSTITUTE attribute specifies whether variable substitution is attempted within the pre-formatted panel text.

Comments

The GENERATE tag is used to add pre-formatted displayable panel contents into the)BODY or)AREA panel sections. These contents can contain any valid displayable information. It is the panel developer's responsibility to provide valid displayable data.

The pre-formatted information is coded within a nested SOURCE tag. The SOURCE tag TYPE attribute is automatically determined based on the position of the GENERATE tag within the DTL source file. When panel attributes are required, the ATTR tag can be used to define the necessary)ATTR section entries.

Restrictions

- The GENERATE tag requires an end tag.
- You must code the GENERATE tag within an AREA, HELP or PANEL tag definition.

Processing

Table 36. The tags you can code within a GENERATE definition			
Tag	Reference	Usage	Required
ATTR	“ATTR (Attribute)” on page 200	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is markup that shows contains a GENERATE tag with nested ATTR and SOURCE tags. [Figure 118 on page 300](#) shows the generated panel file.

```
<!DOCTYPE DM SYSTEM>
<PANEL NAME=* KEYLIST=keylxml applid=isr window=no>
    Generate Tag Example

<CMDAREA>
<pnlinst compact>
    Sample panel source to illustrate the GENERATE tag.

<divider type=solid gap=no>

<generate>
    <attr attrchar=! type=FP>
    <attr attrchar=_ type=NEF>
    <attr attrchar=+ type=NT>

<source>
!   Project ==>_PROJECT !
!   Group   ==>_GROUP1  !==>_GROUP2  !==>_GROUP3  !==>_GROUP4  +
!   Type    ==>_TYPE    !
!   Member  ==>_MEMBER  !

!   DS Name ==>_OTHERDSN
!   Volume  ==>_VOLUME+
</source>
</generate>

</panel>
```

```

)PANEL KEYLIST(KEYLXMP,ISR)
)ATTR DEFAULT(" ") FORMAT(MIX)
05 TYPE(PT)
06 TYPE(PIN)
09 TYPE(FP)
0A TYPE(NT)
13 TYPE(NEF)
22 TYPE(WASL) SKIP(ON) GE(ON)
! TYPE(FP)
- TYPE(NEF)
+ TYPE(NT)
)BODY CMD(ZCMD)

```

Generate Tag Example

Command ==> Z

Sample panel source to illustrate the GENERATE tag.

```

-----
! Project ==> _PROJECT !
! Group ==> _GROUP1 !==> _GROUP2 !==> _GROUP3 !==> _GROUP4 +
! Type ==> _TYPE !
! Member ==> _MEMBER !
! DS Name ==> _OTHERDSN +
! Volume ==> _VOLUME+
)INIT
.ZVARS = '(ZCMD)'
&ZCMD = ' '
)PROC
)END

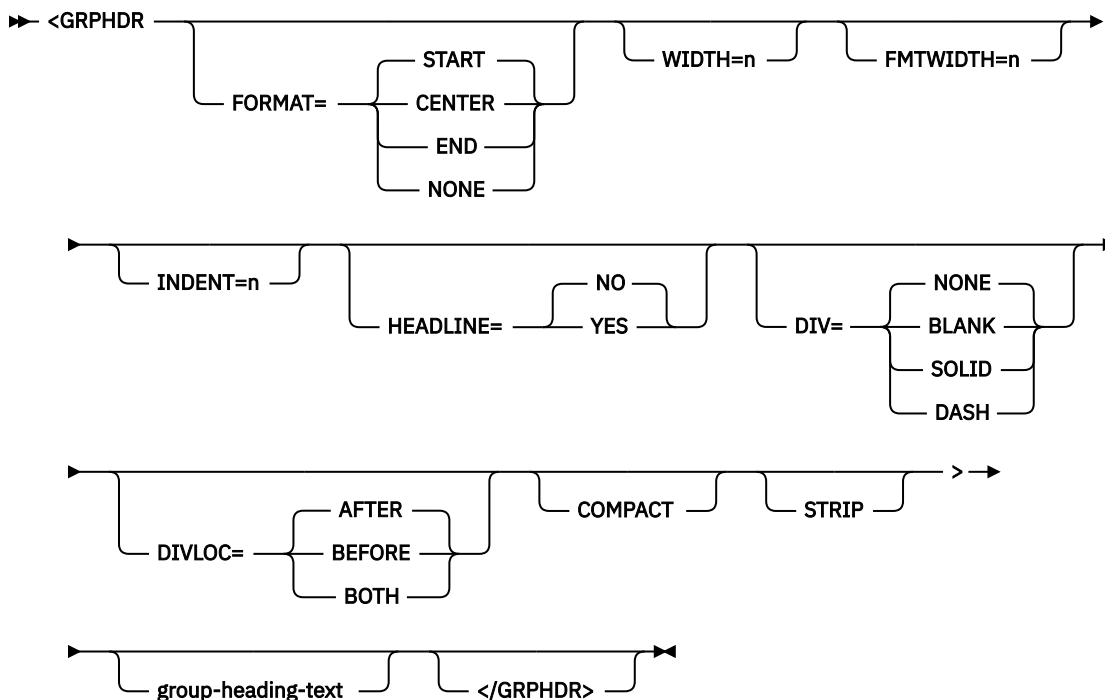
```

Figure 118. Generated panel

GRPHDR (Group Header)

The GRPHDR tag allows the creation of group headers on ISPF panels.

Syntax



Parameters

FORMAT=START | CENTER | END | NONE

This attribute specifies the type of group header formatting.

When FORMAT=NONE, the lines of *group-heading-text* are placed in the panel)BODY section without alteration. The processing is similar to the LINES tag.

When the values START, CENTER, or END are specified, the data is processed in a manner similar to the P tag. The *group-heading-text* is read and flowed to fit within the width limit specified by FMTWIDTH. Multiple lines may be added to the panel, depending on the length of the *group-heading-text*.

WIDTH=n

This attribute specifies the number of columns reserved for the group heading. The minimum width for a group heading is 4. The maximum value is the remaining panel width. If WIDTH is not specified, the default value is set to the remaining panel width. The conversion utility uses 2 positions from the specified or default WIDTH for attributes.

FMTWIDTH=n

This attribute specifies the number of columns to use for formatting the *group-heading-text*. The minimum formatting width is 2. The maximum value is the value specified or defaulted for WIDTH. If FMTWIDTH is not specified, the default value is set to the value of WIDTH.

INDENT=n

This attribute specifies that the group heading is to be indented from the current position.

HEADLINE=NO | YES

This attribute specifies whether dashes are added to span the width of the group heading not occupied by text. This allows a visual indication of the width of the group heading.

DIV=NONE | BLANK | SOLID | DASH

This attribute specifies the type of divider line to be placed before and after the group heading. If this attribute is not specified or has the value NONE, no divider line is generated. The value BLANK produces a blank line. You must specify SOLID or DASH to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

DIVLOC=AFTER | BEFORE | BOTH

This attribute specifies whether a divider line is to be added after the group heading, before the group heading or both before and after the group heading.

COMPACT

This attribute causes the group heading to format without a blank before the heading.

STRIP

This attribute causes leading and trailing blanks to be removed from the heading.

group-heading-text

This is the text of the group header. If no *group-heading-text* is provided, a blank line is added to the panel unless the COMPACT attribute is also specified.

Comments

The GRPHDR tag defines a group heading in the panel)BODY section.

The FMTWIDTH and HEADLINE attributes are not valid in combination with FORMAT=NONE. The DIVLOC attribute is not valid in combination with DIV=NONE.

You use the FMTWIDTH attribute to control the width of flowed text within the number of columns specified by WIDTH. The FORMAT attribute controls the placement of the resulting lines within the heading WIDTH. The FMTWIDTH attribute has no effect if the length of the *group-heading-text* is less than the value specified.

Because the group heading is formatted as text, a blank line is placed at the beginning of each group heading unless the COMPACT attribute has been specified. However, when the group heading is the first item in a scrollable region the blank line is not generated.

Restrictions

- You must code the GRPHDR tag within a PANEL, AREA, DTACOL, or REGION tag. If found anywhere else, an error is logged and the output panel is not saved.

Processing

Table 37. The tags you can code within a GRPHDR definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

```
<!DOCTYPE DM SYSTEM>
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=grphdr KEYLIST=keyl\mp>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST> Type in patron's name and card number (if applicable)
<AREA>
  <GRPHDR FORMAT=center WIDTH=50 FMTWIDTH=30 DIV=solid COMPACT>
    Data Field Group Heading
  </GRPHDR>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
    <DTAFLD DATAVAR=curname USAGE=out ENTWIDTH=8>Name
    <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
      <DTAFLDD>(A 7-digit number)
    <DTAFLD DATAVAR=name>Name
      <DTAFLDD>(Last, First, M.I.)
    <DTAFLD DATAVAR=address>Address
  </DTACOL>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

```

File  Search  Help
-----
                        Library Card Registration

Type in patron's name and card number (if applicable)

                        Data Field Group Heading
-----
Date . . . : _____
Card No. . . : _____ (A 7-digit number)
Name . . . : _____ (Last, First, M.I.)
Address . . : _____

Enter a command ==>
F1=Help      F3=Exit      F5=Display      F6=Keyshelp    F10=Actions
F12=Cancel

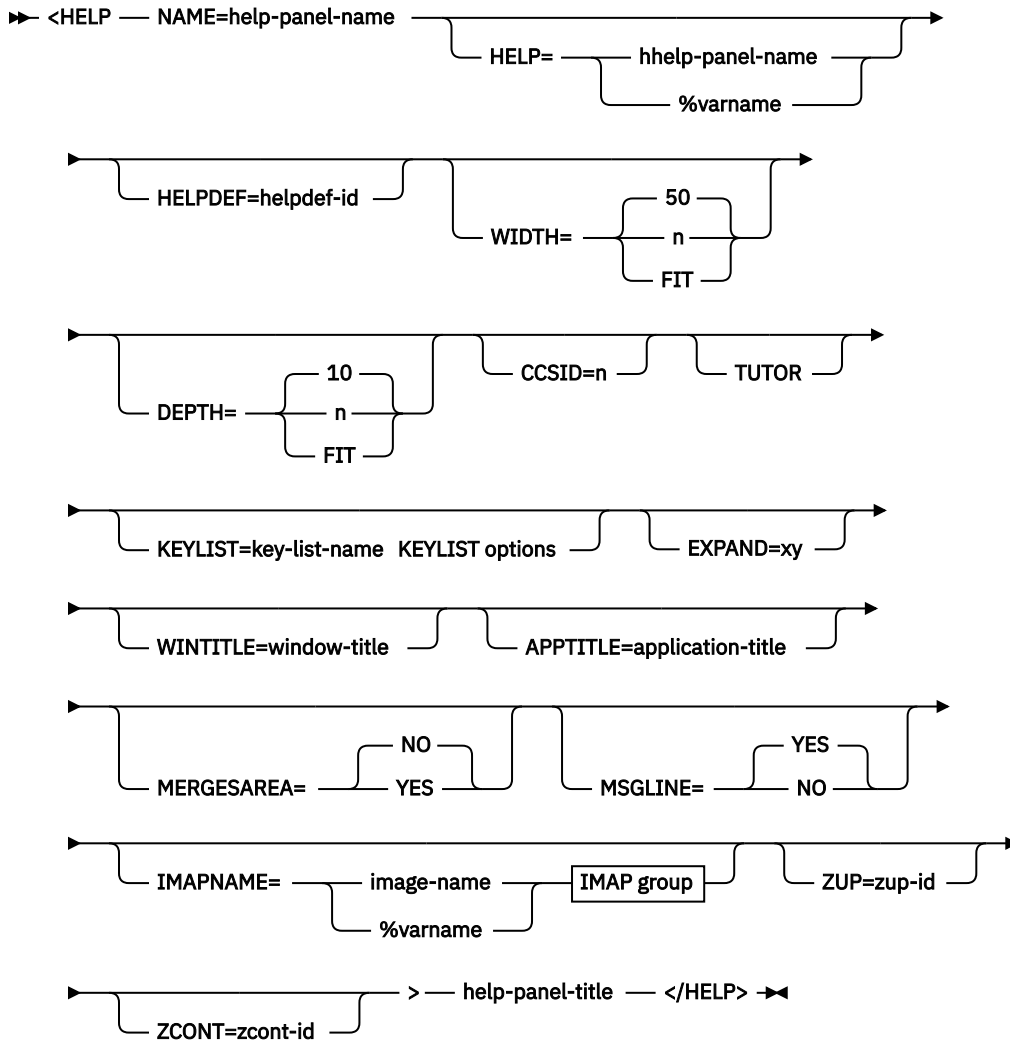
```

Figure 119. Group heading

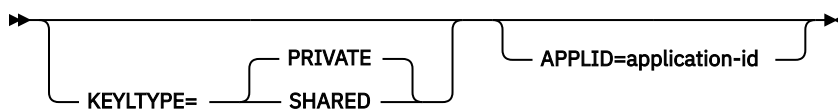
HELP (Help Panel)

The HELP tag defines a help panel.

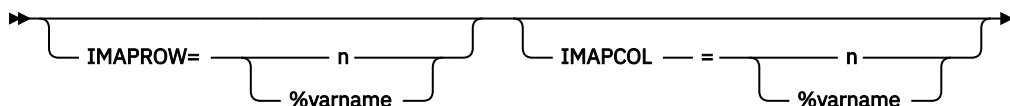
Syntax



KEYLIST options



IMAP group



Parameters

NAME=help-panel-name

This attribute specifies the name of the help panel. The *help-panel-name* must follow the standard naming convention described in “[Rules for variable names](#)” on page 179.

In addition, the *help-panel-name* is limited to 7 characters when the DTL source you are converting causes the conversion utility to build multiple panels. If you have specified an 8-position help name and multiple panels are required, the help name is truncated to 7 positions. If you are not creating a scrollable help panel, this allows additional panels to be built if the help text exceeds the limits of the original help panel. Up to 36 additional help panels are built to contain additional help text.

If the number of generated panels required exceeds 37, a warning message is issued and all help text after the 37th panel is discarded. The additional panel names are generated from the original *help-panel-name* by these rules:

- The character 'X' pads the *help-panel-name* to 8 characters in length if the original *help-panel-name* is less than 8 characters.
- The eighth character of the generated panel name increments from 0-9 and A-Z depending on the number of panels required to be generated. For example, if the original *help-panel-name* is 'HELP1' and the help text extends beyond the original panel, the second generated panel name would be 'HELP1XX0', and the third would be 'HELP1XX1'.

If you specify NAME=*, the *help-panel-name* is set to the input DTL source member name. If multiple dialog element definitions have been combined within a single source file, then this notation should be used for only one dialog element definition within the file. See [“Dialog elements” on page 5](#) for a description of dialog element types created by the conversion utility.

The *help-panel-name* is used to build the help panel output file name in which the conversion utility stores the converted help panel. The default name is “userid.PANELS(*help-panel-name*)”.

The output panel file name can be specified on the invocation panel for the conversion utility. You can specify the panel library of your choice. If the SCRIPT option was specified, the *help-panel-name* is also used to build the file name in which the conversion utility stores the image of the help panel. The default name is “userid.SCRIPT(*help-panel-name*)”.

See [Chapter 10, “Using the conversion utility,” on page 151](#) for complete information on invocation syntax.

The ISPF tutorial facility displays help panels. The user can scroll forward by pressing Enter or the RIGHT (F11) key, or scroll backward by pressing the LEFT (F10) key. The scrolling indicators “**More: +**”, “**More: -**”, and “**More: -+**” are added to the displayed panel to indicate more help is available.

HELP=hhelp-panel-name | %varname

This attribute specifies the name of a defined help for help panel. It identifies the help text that is associated with help processing.

The *hhelp-panel-name* must follow the standard naming convention described in [“Rules for variable names” on page 179](#).

Specification of the HELP attribute causes ISPDTLC to generate “.HHELP=*hhelp-panel-name*” (or “.HHELP=&varname”) in the)INIT section during help panel generation.

If no value is provided for the HELP attribute, the conversion utility adds the default “.HHELP = ISP00006” to the generated panel.

ISPF displays this panel when the application user requests help and the cursor is not on a panel field that is defined as a reference phrase.

HELPDEF=helpdef-id

This attribute specifies a defined help default. The *helpdef-id* value is the identifier specified on the HELPDEF tag. You can override any of the defaults from this HELPDEF tag by specifying that attribute on the HELP tag. See the description of the HELPDEF tag for information on defining help defaults.

WIDTH=50 | n | FIT

This attribute specifies the width of the help panel. The default width is 50. When you specify this attribute, it should be greater than or equal to the minimum width of 16 characters. The maximum is 156. Because there are set margins of 1 character on each side of the panel text to allow for 3270 attribute bytes, the effective width for text for a help panel defined as WIDTH=50 is 48 characters.

If you have specified WIDTH=FIT, the conversion utility formats the panel using the maximum available width. When formatting is completed the WIDTH value is reset to the minimum width used or to 16 if the formatted panel is less than 16 characters wide.

If the specified WIDTH exceeds the maximum minus 4 allowed by the display device, ISPF issues an error message at run time.

DEPTH=10 | n | FIT

This attribute specifies the depth of the HELP panel. The maximum depth is 60 and the minimum depth is 6. When the panel body does not end with a scrollable area, four lines at the bottom of each help panel are reserved for the function key area. Two lines are reserved at the top of the help panel for the *help-panel-title* and a separator line. You must include provisions for these 6 lines in the depth you specify.

The default help panel depth of 10 is used when the DEPTH attribute provided cannot be used or the DEPTH attribute is not specified.

If you have specified DEPTH=FIT, the conversion utility formats the panel using a depth of 22. When formatting is completed the DEPTH value is reset to the minimum depth used or to 6 if the formatted panel contains less than 6 lines.

If the specified DEPTH exceeds the maximum, minus 2, allowed by the display device, ISPF issues an error message at run time.

CCSID=n

CCSID specifies the coded-character-set identifier as defined by the Character Data Representation Architecture. CCSID should be entered as a five-position numeric value. For more information on using the CCSID attribute, refer to the [z/OS ISPF Dialog Developer's Guide and Reference](#).

TUTOR

This attribute specifies that the panel title be formatted with the word **Tutorial** (or its translated equivalent) on each end of the title line, similar to ISPF tutorial panels.

KEYLIST=key-list-name

KEYLIST is an ISPF extension to the Dialog Tag Language. This attribute specifies the name of the key mapping list associated with the help panel. If you do not specify a *key-list-name* in a HELP definition, the ISPF-provided key list (ISPHELP) is used. For information about defining key mapping list, see “KEYL (Key List)” on page 322. For information about the ISPF-provided key list, refer to the [z/OS ISPF User's Guide Vol I](#).

KEYLTYPE= PRIVATE | SHARED

This attribute is used to add the SHARED keyword to the KEYLIST parameter of the)PANEL statement. For information about the)PANEL statement, refer to the [z/OS ISPF Dialog Developer's Guide and Reference](#). The KEYLTYPE attribute is ignored if you have not provided the KEYLIST attribute as part of the HELP tag definition or as part of an associated HELPDEF tag definition.

APPLID=application-id

This attribute is used to add the application ID to the)PANEL statement. The *application-id* overrides the KEYLAPPL invocation option value. The APPLID attribute is ignored if you have not provided the KEYLIST attribute as part of the HELP tag definition or as part of an associated HELPDEF tag definition.

EXPAND=xy

This attribute adds the EXPAND(xy) attribute to the)BODY section of the panel. If only one character is present, the second character is set to the same value. If the EXPAND attribute is present with no value specified, the conversion utility uses a character from the range of low-order hex values available for panel attributes. This removes an available character from possible use as a panel attribute and may cause panel formatting errors.

WINTITLE=window-title

This attribute is used to add a title on the pop-up window border. The attribute value is placed in the ISPF ZWINTTL variable. The maximum length of the *window-title* text is the panel width minus 1.

APTITLE=application-title

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

MERGESAREA= NO | YES

This attribute controls an additional formatting step for panels with a single scrollable area. If the entire contents of the scrollable area fit within a standard 24-line panel (allowing 4 lines for the function keys display), the scrollable area content is moved into the panel body.

MSGLINE=YES | NO

This attribute controls the provision for a long message line in the generated panel. When MSGLINE=NO, the blank line for the long message is not added to the panel)BODY section. It is the panel designer's responsibility to ensure that critical panel areas are positioned so that the long message does not inhibit use of the resulting panel.

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPROW=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPCOL=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

ZUP=zup-id

This attribute provides the name of the Tutorial panel to be assigned to the ZUP variable. It is valid only when the TUTOR attribute has also been specified.

ZCONT=zcont-id

This attribute provides the name of the Tutorial panel to be assigned to the ZCONT variable. It is valid only when the TUTOR attribute has also been specified.

help-panel-title

This specifies the title that appears on the help panel.

The *help-panel-title* is centered within the specified help panel width in accordance to CUA rules. If the title text is wider than the WIDTH specified, the title is truncated with an ellipsis (...) appended. Two lines are reserved for the title and a separator which can include the scrolling indicator if there are more panels.

Comments

The HELP tag defines a help panel. A help panel can contain multiple information areas, which you use the INFO tag to define (see [“INFO \(Information Region\)”](#) on page 317).

ISPF always displays help panels defined with DTL in a pop-up window with a border. Therefore, the maximum value you can specify for the WIDTH attribute is 4 less than the maximum allowed by the display device. This allows for the left and right borders and their 3270 attribute characters. The maximum value for the DEPTH attribute is 2 less than the maximum allowed by the display device to allow for the top and bottom borders. Borders are added to the formatted help panel at run time.

If you are not creating a scrollable help panel and the text to be included in the)BODY section of the ISPF panel exceeds the specified DEPTH value, up to 36 additional panels are generated to contain the additional text. If the help text extends beyond the original help panel and 36 additional help panels, an error message is issued and the excess text is truncated. If the error occurs, and the DEPTH and WIDTH attributes are not set to their maximum values, the values should be increased or the amount of text to be included in the help panel should be reduced.

For nonscrollable HELP panels or for scrollable HELP panels which end with a nonscrollable section, a function key area of four lines is reserved at the bottom of the panel. The four lines are taken from the value specified for the DEPTH attribute.

If you do not specify the KEYLIST attribute, ISPF automatically associates the ISPF-provided key list "ISPHELP" with all DTL help panels.

This table shows the "ISPHELP" key list and assignments:

Table 38. ISPHELP keylist and assignments

Key	Command	Key Label	Format
F1	HELP	Help	Short
F2	SPLIT	Split	Long
F3	EXIT	Exit	Short
F4	RESIZE	Resize	Long
F5	EXHELP	Exhelp	Short
F6	KEYSHELP	Keyshelp	Short
F7	UP	PrvTopic	Short
F8	DOWN	NxtTopic	Short
F9	SWAP	Swap	Long
F10	LEFT	PrvPage	Short
F11	RIGHT	NxtPage	Short
F12	CANCEL	Cancel	Short

All ISPHELP function keys are active when the cursor is in the help panel. Display of keys in the function key area is controlled by the user through the ISPF FKA command.

Because help panels are displayed by the ISPF tutorial processor, the commands assigned to the keys are those supported by the ISPF tutorial. For more information on the ISPF tutorial, refer to the [z/OS ISPF User's Guide Vol I](#).

Since ISPDTLC generated panels are not normally used in a full Tutorial, the default ISPHELP keylist may result in confusion in the use of the F7 and F8 keys for scrolling. An alternate approach is the ISPHLP2 keylist. To use this keylist, add the KEYLIST=ISPHLP2 attribute to your help panel definition.

Table 39. ISPHLP2 keylist and assignments

Key	Command	Key Label	Format
F1	HELP	Help	Short
F2	SPLIT	Split	Long
F3	EXIT	Exit	Short
F4	RESIZE	Resize	Long
F5	EXHELP	Exhelp	Short
F6	KEYSHELP	Keyshelp	Short
F7	LEFT	PrvPage	Short
F8	RIGHT	NxtPage	Short
F9	SWAP	Swap	Long
F10	LEFT	PrvPage	Long
F11	RIGHT	NxtPage	Long
F12	CANCEL	Cancel	Short

Restrictions

- The HELP tag requires an end tag.
- You cannot code the HELP tag within any other tag definition.
- If the help panel does not have a panel body, the conversion utility issues an error message. The help panel must contain at least one INFO (information region) definition to qualify as a panel body. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- When a “%varname” notation is found on any of the attributes that allow a variable name, the “%varname” entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

Table 40. The tags you can code within a HELP definition			
Tag	Reference	Usage	Required
AREA	“AREA (Area)” on page 189	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
GENERATE	“GENERATE (Generate)” on page 298	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	Yes
INFO	“INFO (Information Region)” on page 317	Multiple	Yes
REGION	“REGION (Region)” on page 405	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No
TEXTLINE	“TEXTLINE (Text Line)” on page 439	Single	No

Examples

Here is help panel markup that contains an information region that contains a paragraph, a definition list, and two unordered lists nested within the definition list. Because all of the data does not fit in one help panel, the conversion utility created three panels HELP, HELPXXX0, and HELPXXX1. The panels are scrollable. Figures [Figure 120 on page 310](#), [Figure 121 on page 310](#), and [Figure 122 on page 310](#) show the formatted results with the function key area displayed in its short form.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=help WIDTH=46 DEPTH=16>ShelfBrowse for Kids
<AREA>
  <INFO>
    <P>ShelfBrowse can help you
    find any kind of book you are looking for.
    The two main categories for books are:
    <DL TSIZE=12>
      <DTHD>Book
      <DDHD>Description
      <DT>Fiction
      <DD>Fiction books are stories
      that never really happened.
      The writer made them up.
      For example:
      <UL>
        <LI>Fairy Tales
        <LI>Mysteries
        <LI>Science fiction stories
      </UL>
      <DT>Nonfiction
```

```
<DD>Nonfiction books are about
things that really exist.
For example:
<UL>
  <LI>History books
  <LI>Reference books
  <LI>How to books
</UL>
</DL>
</INFO>
</AREA>
</HELP>
```

ShelfBrowse for Kids

More: +

ShelfBrowse can help you find any kind of book you are looking for. The two main categories for books are:

Book	Description
Fiction	Fiction books are stories that never really happened. The writer made them up. For example:

F1=Help

F6=Keyshelp

F10=PrvPage

F3=Exit

F7=PrvTopic

F11=NxtPage

F5=Exhelp

F8=NxtTopic

F12=Cancel

Figure 120. Help panel (example 1 of 3)

ShelfBrowse for Kids

More: - +

- o Fairy Tales
- o Mysteries
- o Science fiction stories

Nonfiction	Nonfiction books are about things that really exist. For example:
------------	---

F1=Help

F6=Keyshelp

F10=PrvPage

F3=Exit

F7=PrvTopic

F11=NxtPage

F5=Exhelp

F8=NxtTopic

F12=Cancel

Figure 121. Help panel (example 2 of 3)

ShelfBrowse for Kids

More: -

- o History books
- o Reference books
- o How to books

F1=Help

F6=Keyshelp

F10=PrvPage

F3=Exit

F7=PrvTopic

F11=NxtPage

F5=Exhelp

F8=NxtTopic

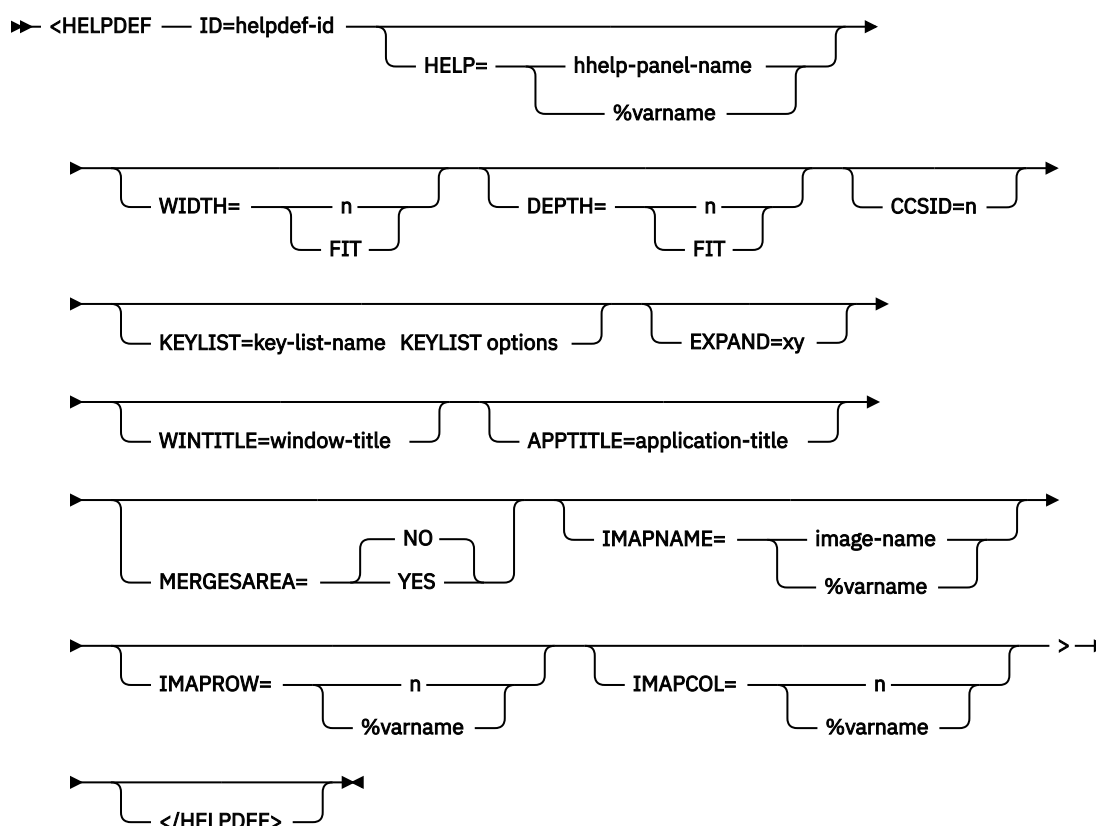
F12=Cancel

Figure 122. Help panel (example 3 of 3)

HELPDEF (Help default)

The HELPDEF tag defines default values for help panels.

Syntax



KEYLIST options



Parameters

ID=helpdef-id

This is the ID of the help panel default definition. The ID is used as the identifier of this set of default definitions on the HELP tag.

The *helpdef-id* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

HELP=hhelp-panel-name | %varname

This attribute specifies the default name of a defined help for help panel. It identifies the help text that is associated with help processing.

The *hhelp-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

WIDTH=n | FIT

This attribute specifies a default width value for a help panel that refers to this help default.

DEPTH=n | FIT

This attribute specifies a default depth value for a help panel that refers to this help default.

CCSID=n

CCSID specifies the coded-character-set identifier as defined by the Character Data Representation Architecture. CCSID should be entered as a five-position numeric value. For more information on using the CCSID attribute, refer to the [z/OS ISPF Dialog Developer's Guide and Reference](#).

KEYLIST=key-list-name

This attribute specifies the name of the key mapping list associated with the help panel. If you do not specify a *key-list-name* in a HELP definition, the ISPF-provided key list (ISPHelp) is used. For information about defining key mapping list, see “KEYL (Key List)” on page 322. For information on the ISPF-provided key list, refer to the [z/OS ISPF User's Guide Vol I](#).

KEYLTYPE=PRIVATE | SHARED

This attribute is used to add the SHARED keyword to the KEYLIST parameter of the)PANEL statement. For information about the)PANEL statement, refer to the [z/OS ISPF Dialog Developer's Guide and Reference](#).

APPLID=application-id

This attribute is used to add the application ID to the)PANEL statement. The *application-id* overrides the KEYLAPPL invocation option value.

EXPAND=xy

This attribute adds the EXPAND(xy) attribute to the)BODY section of the panel. If only one character is provided, the second character is set to the same value. If the EXPAND attribute is present with no value specified, the conversion utility uses a character from the range of low-order hex values available for panel attributes. This removes an available character from possible use as a panel attribute and may cause panel formatting errors.

WINTITLE=window-title

This attribute is used to add a title on the pop-up window border. The attribute value is placed in the ISPF ZWINTTL variable. The maximum length of the *window-title* is the panel width minus 1.

APPTITLE=application-title

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

MERGESAREA= NO | YES

This attribute controls an additional formatting step for panels with a single scrollable area. If the entire contents of the scrollable area fit within a standard 24-line panel (allowing 4 lines for the function keys display), the scrollable area content is moved into the panel body.

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPROW=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPCOL=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

Comments

The HELPDEF tag defines default values for help panels. When a HELP panel tag refers to a help panel default, the values specified by the associated HELPDEF tag are used for the help panel unless overridden by values specified in the HELP tag definition.

The HELP tag can override any of the HELPDEF values by specifying that value within its own definition. Therefore, it is possible for a HELP tag to select certain default values from the help panel default and override others.

See [“HELP \(Help Panel\)”](#) on page 303 for more information.

You can code multiple `HELPDEF` definitions in a single application. Each help default must have a unique *helpdef-id*.

Restrictions

- You cannot code the HELPDEF tag within any other tag definition.
- You must code the HELPDEF tag before you code any HELP tag that refers to it.

Processing

None.

Examples

Here is a source file example where the `HELPDEF` definition defines default `DEPTH` and `WIDTH` values. The help panels "help15" and "help16" both reference the help default. "help15" uses both default values and "help16" uses only the default `WIDTH` value, and overrides the default `DEPTH` value by specifying its own `DEPTH` value. The help panel "help17" does not reference the help default, and defines its own `DEPTH` and `WIDTH` values.

```
<!DOCTYPE DM SYSTEM>

<HELPDEF ID=helpdef1 DEPTH=10 WIDTH=40>

<HELP NAME=help15 HELPDEF=helpdef1>Help for This
:
</HELP>

<HELP NAME=help16 HELPDEF=helpdef1 DEPTH=15>Help for That
:
</HELP>

<HELP NAME=help17 DEPTH=15 WIDTH=25>Help for the Other
:
</HELP>
```

Hn (Heading)

The heading tags define main topics and subtopics of information within an information region.

Syntax



Parameters

COMPACT

This attribute causes the heading-text to be formatted without creating a blank line before the heading.

heading-text

This is the text of the heading.

Comments

The heading tags define main topics and subtopics of information within an information region. You can define up to four heading levels. The *n* in Hn indicates the heading level. The heading levels are formatted in this fashion:

H1

Identifies a main topic of information. The text is centered on the panel.

H2, H3, H4

The text is formatted against the left margin of the panel body.

Headings are formatted with one blank line before them.

Restrictions

- The Hn tag must be coded within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 41. The tags you can code only within an H2, H3, or H4 definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains two levels of headings. [Figure 123 on page 315](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=hn DEPTH=22>Department Descriptions Help
<AREA>
<INFO>
  <H1>Departments
  <H2>Entertainment
  <P>Our entertainment department carries the
  finest in home entertainment components.
  <H2>Exotic Pets
  <P>You can order from a wide variety of exotic
  pets and pet supplies in this department.
  <H2>Toys
  <P>Your kids will love our wide selection of
  toys, games, and play equipment.
</INFO>
</AREA>
</HELP>
```

Department Descriptions Help

Departments

Entertainment

Our entertainment department carries the finest in home entertainment components.

Exotic Pets

You can order from a wide variety of exotic pets and pet supplies in this department.

Toys

Your kids will love our wide selection of toys, games, and play equipment.

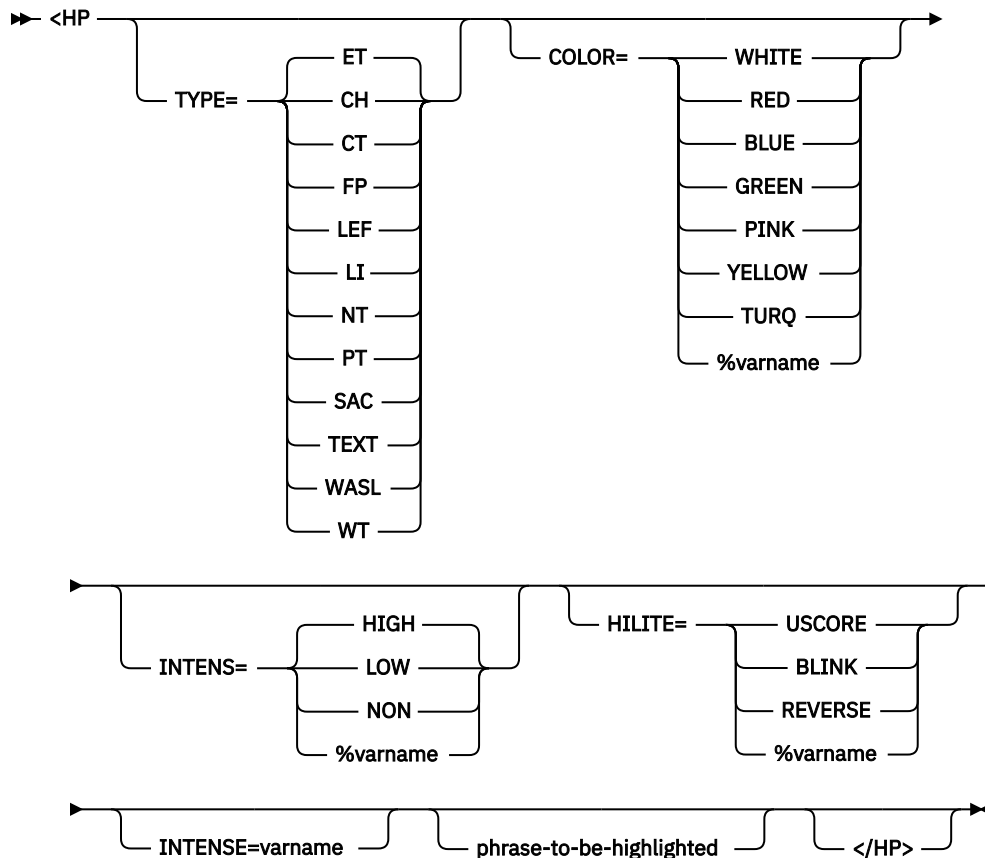
F1=Help F3=Exit F5=Exhelp
 F6=Keyshelp F7=PrvTopic F8=NxtTopic
 F10=PrvPage F11=NxtPage F12=Cancel

Figure 123. Headings

HP (Highlighted Phrase)

The HP tag identifies text to be displayed with highlighted emphasis.

Syntax



Parameters

TYPE= ET | CH | CT | FP | LEF | LI | NT | PT | SAC | TEXT | WASL | WT

This attribute defines the attribute type to be applied to the *phrase-to-be-highlighted*. Using a CUA attribute causes the text to appear in the associated color.

When TYPE=TEXT, a non-CUA attribute is generated and you can specify the color, intensity, and highlighting with the COLOR, INTENS, and HILITE attributes. These attributes are not valid for CUA types.

COLOR= WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENS= HIGH | LOW | NON | %varname

This attribute defines the intensity of a field. You can define this attribute as a variable name preceded by a percent (%) sign.

HILITE= USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute of a field. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENSE=varname

This attribute supplies a variable name that must contain a valid value for the INTENS keyword. The entire phrase is controlled by this value. For example, if the variable contains the value NON, the phrase is not visible.

phrase-to-be-highlighted

This text displays with highlighted emphasis.

Comments

The HP identifies text to be displayed with highlighted emphasis by ISPF. The HP end tag restores normal text.

Restrictions

- You can code the HP tag wherever the RP tag is valid.
- You can code the HP tag within the text following the CHDIV, CMDAREA, HELP, and PANEL tags.
- The HP tag requires an end tag.

Processing

None.

Examples

This markup shows the formatted result in [Figure 124 on page 317](#).

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=hp KEYLIST=keylxml>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST> Type in <HP>patron's name</HP> and <HP>card number</HP>
          (if applicable)
<TOPINST> Then select an action bar choice.
<AREA>
```

```

<DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
<DTAFLD DATAVAR=curname USAGE=out ENTWIDTH=8>Date
<DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
  <DTAFLDD>(A 7-digit number)
<DTAFLD DATAVAR=name>Name
  <DTAFLDD>(Last, First, M.I.)
<DTAFLD DATAVAR=address>Address
  </DTACOL>
<DIVIDER>
<REGION DIR=horiz>
<SELFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=card MATCH=new>New
  <CHOICE CHECKVAR=card MATCH=renew>Renewal
  <CHOICE CHECKVAR=card MATCH=replace>Replacement
</SELFLD>
<SELFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
  <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
  <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
  <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
  <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
</SELFLD>
</REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

```

File Search Help
-----
                        Library Card Registration

Type in patron's name and card number (if applicable).
Then select an action bar choice.

Date . . . :
Card No. . . : _____ (A 7-digit number)
Name . . . : _____ (Last, First, M.I.)
Address . . : _____

Choose one of the following          Check valid branches
--  1. New                          _ North Branch
    2. Renewal                      _ South Branch
    3. Replacement                  _ East Branch
                                   _ West Branch

Enter a command ==>
F1=Help      F2=Split      F3=Exit      F6=KEYSHELP      F9=Swap
F12=Cancel

```

Figure 124. HP (Highlighted Phrase)

INFO (Information Region)

The INFO tag defines an information region for a panel.

Syntax

```

➤ <INFO [WIDTH= format-width [INDENT=n]] > </INFO> ➤

```

Parameters

WIDTH=format-width | *

This attribute determines the width the conversion utility uses to format the text in the ISPF)BODY section of the panel. If WIDTH is not the value is set to the remaining available panel (or region) width. If specified, the WIDTH value cannot be larger than the defined width of the panel (or region) minus 2 characters. For example, a WIDTH value of 58 is acceptable for an information region within a panel with a defined width of 60.

Note: You should code the WIDTH attribute if the information region is part of an application panel definition that uses horizontal region capability. The actual width used in a horizontal region is 2 characters longer than the WIDTH attribute value to provide for attribute bytes that delimit the region.

INDENT=n

This attribute defines the number of columns to indent the current information region from the current left boundary.

Comments

The INFO tag defines an information region for a panel. The information region is used to display text such as paragraphs, lists, notes, examples, and figures. A typical use of the INFO tag is for the definition of text within help panels.

Restrictions

- The INFO tag requires an end tag.
- You must code the INFO tag within an AREA, HELP, or PANEL definition. See [“AREA \(Area\)” on page 189](#), [“HELP \(Help Panel\)” on page 303](#), and [“PANEL \(Panel\)” on page 376](#) for descriptions of these tags.

Processing

Table 42. The tags you can code within an INFO definition			
Tag	Reference	Usage	Required
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
Hn	“Hn (Heading)” on page 313	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No

Table 42. The tags you can code within an INFO definition (continued)

Tag	Reference	Usage	Required
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains an information region. The text of the information region is defined using two P (paragraph) tags and an unordered list (UL) tag with three LI (list item) tags. [Figure 125 on page 319](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=info WIDTH=60 DEPTH=22>ShelfBrowse Help
<AREA>
<INFO WIDTH=42>
  <P>When ShelfBrowse finds your book, it displays this
  information:
  <UL>
    <LI>Reference information about the book.
    <LI>The location of the book.
    <LI>If the book is in stock.
    <P>If the book is not in stock, see the librarian.
  </UL>
</INFO>
</AREA>
</HELP>
```

ShelfBrowse Help

When ShelfBrowse finds your book, it displays this information:

- o Reference information about the book.
- o The location of the book.
- o If the book is in stock.

If the book is not in stock, see the librarian.

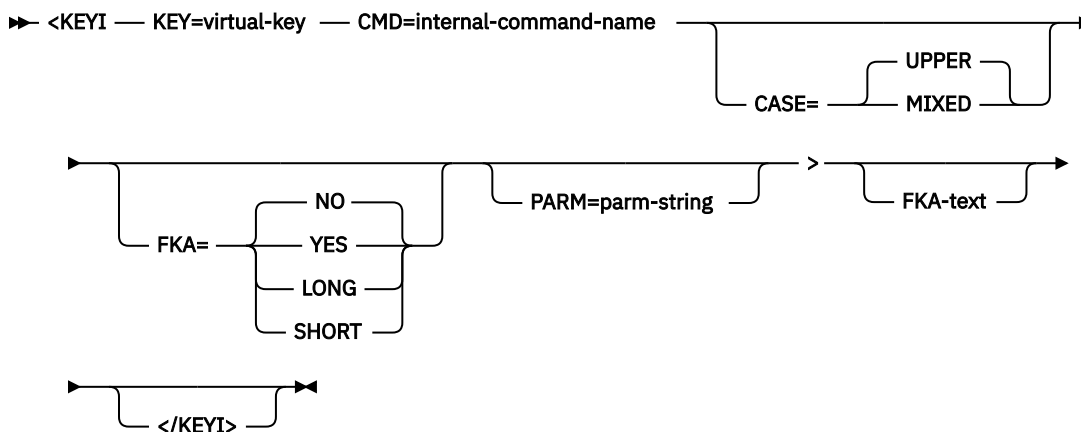
F1=Help F3=Exit F5=Exhelp F6=Keyshelp
 F7=PrvTopic F8=NxtTopic F10=PrvPage F11=NxtPage
 F12=Cancel

Figure 125. Information region

KEYI (Key Item)

The KEYI tag defines a key assignment within a key mapping list.

Syntax



Parameters

KEY=virtual-key

This attribute specifies the name of the key to assign to the command. The conversion utility supports F1-F24 only.

CMD=internal-command-name

This attribute specifies the command that ISPF runs when the user presses the key.

The *internal-command-name* must follow the standard naming convention described in [“Rules for variable names” on page 179](#).

As an extension to the Dialog Tag Language, the conversion utility supports special ISPF command syntax for *internal-command-name*. In this case, the *internal-command-name* must have these characteristics:

- 2-9 single-byte characters in length
- The first character must be a '>', ':', or '%'.

To code the > character you must use the **>sym** predefined entity. See [“Predefined entities” on page 23](#) for more information.

- The second character must be A-Z, a-z, @, #, or \$.
- Remaining characters, if any, must be A-Z, a-z, @, #, \$, or 0-9.

Lowercase characters are translated to their uppercase equivalents by default.

CASE=UPPER | MIXED

This attribute specifies whether the *internal-command-name* is converted to uppercase characters or stored as entered in the tag definition.

FKA=NO | YES | LONG | SHORT

This attribute specifies whether the key assignment is to appear in the function key area of an application panel. The default value NO indicates that the key is not to appear. You must specify FKA=YES, FKA=LONG, or FKA=SHORT if you want the key to be displayed in the function key area.

When FKA=NO is specified, the key is active even if it is not displayed.

PARM=parm-string

This attribute allows a parameter to be added to the command specified by the CMD attribute. The combined length of the command and the parameter is limited to 40 bytes. When the combined length exceeds 40 bytes, truncation of the PARM occurs at the end of the last complete word in the *parm-string*, for a *parm-string* containing multiple words. A *parm-string* which is a single word is truncated at position 40.

FKA-text

This is the text for the key which is to appear in the function key area of the panels that refer to the key list. This text is appended to the string "Fn=" (with no intervening space) to create the displayed format. Use initial caps for the *FKA-text* value.

If not specified, the *FKA-text* defaults to the *internal-command-name* specified for the key.

The function key area is formatted at run time based on the panel size. The maximum number of bytes allowed for *FKA-text* is 64. If the text exceeds 64 bytes, it is truncated and a warning message is issued. The conversion utility removes excess blanks from *FKA-text*. The first 8 bytes of the resulting text are used by ISPF.

Comments

The KEYI tag defines a key assignment within a key mapping list. Key assignments provide a means of associating commands with keys.

KEYI tags with the same assignment cause the conversion utility to issue a warning message and retain only the first occurrence.

Restrictions

- You must code the KEYI tag within a KEYL definition. See [“KEYL \(Key List\)” on page 322](#) for a complete description of this tag.
- Each KEYI definition can only have one command assigned to it. Additionally, CUA requires these conventions when assigning commands to certain keys:
 - If KEY=F1 or F13, then CMD must be HELP.
 - If KEY=F3 or F15, then CMD must be EXIT.
 - If KEY=F12 or F24, then CMD must be CANCEL.

ISPF lets you provide the name of your own command on these keys.

If you code the command HELP, EXIT, or CANCEL as part of your KEYI definition, then HELP must be assigned to key F1 or F13, EXIT must be assigned to F3 or F15, and CANCEL must be assigned to F12 or F24.

Processing

None.

Examples

Here is source file markup that contains a key mapping list and an application panel that refers to the key mapping list. The F7 and F8 keys do not appear on the panel because they both have an FKA value of NO. [Figure 126 on page 322](#) shows the formatted application panel with the displayed keys.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampabc system>
  <!entity sampbody system>)>
&sampvar1;

<KEYL NAME=keylxml>
  <KEYI KEY=f1 CMD=help FKA=yes>Help
  <KEYI KEY=f2 CMD=split FKA=yes>Split
  <KEYI KEY=f3 CMD=exit FKA=yes>Exit
  <KEYI KEY=f5 CMD=search FKA=no>Display
  <KEYI KEY=f6 CMD=keyhlp FKA=yes>Keyshelp
  <KEYI KEY=f7 CMD=backward FKA=no>Backward
  <KEYI KEY=f8 CMD=forward FKA=no>Forward
  <KEYI KEY=f9 CMD=swap FKA=yes>Swap
  <KEYI KEY=f10 CMD=actions FKA=no>Actions
  <KEYI KEY=f12 CMD=cancel FKA=yes>Cancel
</KEYL>

<PANEL NAME=keyi KEYLIST=keylxml>Library Card Registration
<AB>
&sampabc;
</AB>
&sampbody;
</PANEL>
```

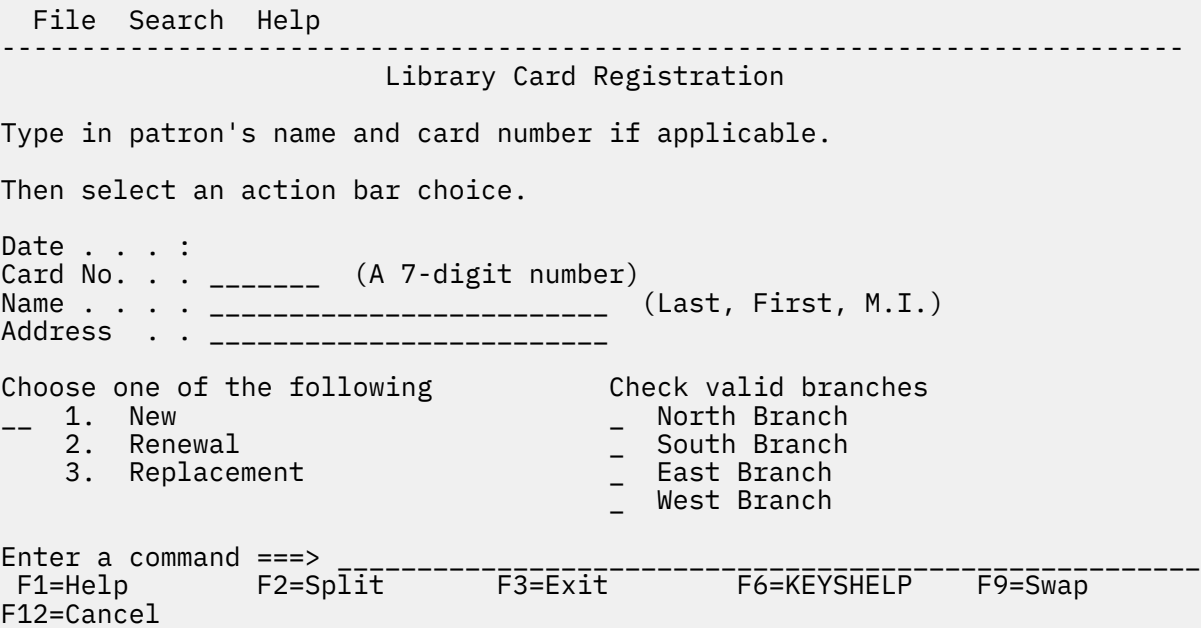
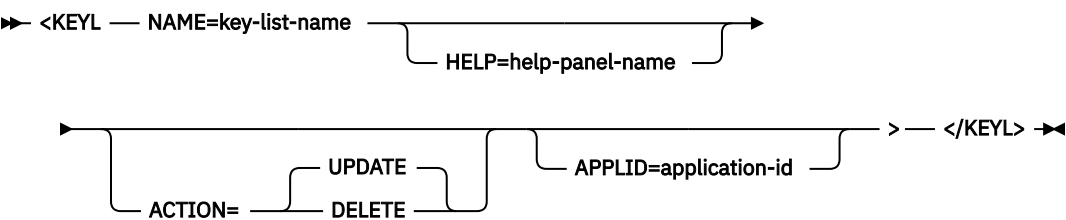


Figure 126. Key Items

KEYL (Key List)

The KEYL tag defines a key mapping list where keys can be mapped to commands.

Syntax



Parameters

NAME=key-list-name

This attribute specifies a name for a key list. The HELP, HELPDEF, PANEL, and PANDEF tag refer to the *key-list-name*.

The *key-list-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

The name of the keylist table is xxxxKEYS where xxxx represents the application identifier provided to ISPD TLC with the KEYLAPPL keyword when invoked, in the "Keylist Application ID" field on the invocation panel, or with the APPLID attribute of this tag.

The *key-list-name* is used to identify the entry in the keylist table. For example, if NAME=CONVLIST and KEYLAPPL=XYZ, then CONVLIST is written as a table entry to member XYZKEYS in the table library partitioned data set.

Keylists are updated using ISPF table services. Input is obtained from the ISPTLIB DDname allocation and output is written to the ISPTABL DDname allocation. See the description of how to allocate libraries before starting ISPF in the [z/OS ISPF User's Guide Vol I](#) for more information about the use of ISPTLIB and ISPTABL.

See Chapter 10, “Using the conversion utility,” on page 151 for more information on invocation parameters for the conversion utility.

HELP=help-panel-name

This attribute names a help panel that displays when the user requests help on a keylist display.

If a user requests help for a keylist and no help has been defined by the KEYL tag, the ZKEYHELP variable is checked for a help panel name. If the application has not set ZKEYHELP, a message that keyshelp is not available is displayed.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels.

ACTION=UPDATE | DELETE

This attribute specifies the type of action requested for the keylist specified by *key-list-name*.

When ACTION=DELETE is specified, it is not necessary to nest any KEYI tags within the KEYL tag definition.

APPLID=application-id

This attribute provides the application ID used to build the keylist name. The *application-id* overrides the KEYLAPPL invocation option value.

Comments

The KEYL tag defines a key mapping list where keys can be mapped to commands.

To display these keys on a panel requires that the PANEL or PANDEF tag refer to the *key-list-name*. ISPF uses the specified key mapping list when building the display dependent on the user's setting by the FKA command. For more information about displaying and formatting of the function key area, refer to the appropriate section in the [z/OS ISPF Dialog Developer's Guide and Reference](#).

Restrictions

- The KEYL tag requires an end tag.
- The KEYL tag cannot be nested within any other tag definition.
- When ACTION=UPDATE is specified (or defaulted), at least one KEYI tag must be included in the keylist definition.

Processing

Table 43. The tags you can code within a KEYL definition			
Tag	Reference	Usage	Required
KEYI	“KEYI (Key Item)” on page 319	Multiple	No

Examples

Here is source file markup that contains a key mapping list and an application panel that refers to the key mapping list. Figure 127 on page 324 shows the formatted application panel with the displayed keys.

```
<!DOCTYPE DM SYSTEM(  
  <!entity sampvar1 system>  
  <!entity sampabc system>  
  <!entity sampbody system>)>  
&sampvar1;  
  
<KEYL NAME=key1tb1>  
  <KEYI KEY=f1 CMD=help FKA=yes>Help  
  <KEYI KEY=f2 CMD=split FKA=yes>Split  
  <KEYI KEY=f3 CMD=exit FKA=yes>Exit  
  <KEYI KEY=f5 CMD=search FKA=no>Display  
  <KEYI KEY=f6 CMD=keyhlp FKA=no>Keyshelp  
  <KEYI KEY=f7 CMD=backward FKA=yes>Backward  
  <KEYI KEY=f8 CMD=forward FKA=yes>Forward  
  <KEYI KEY=f9 CMD=swap FKA=yes>Swap  
  <KEYI KEY=f10 CMD=actions FKA=no>Actions  
  <KEYI KEY=f12 CMD=cancel FKA=yes>Cancel  
</KEYL>  
  
<PANEL NAME=key1 KEYLIST=key1xmp>Library Card Registration  
<AB>  
&sampabc;  
</AB>  
&sampbody;  
</PANEL>
```

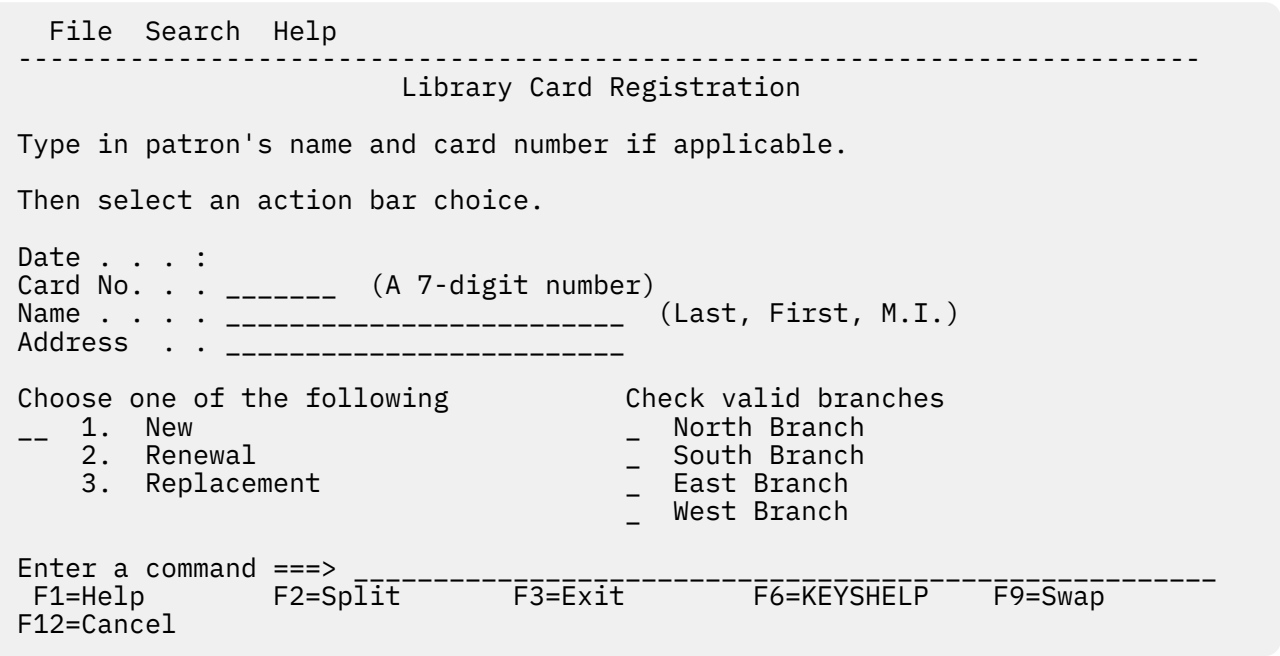
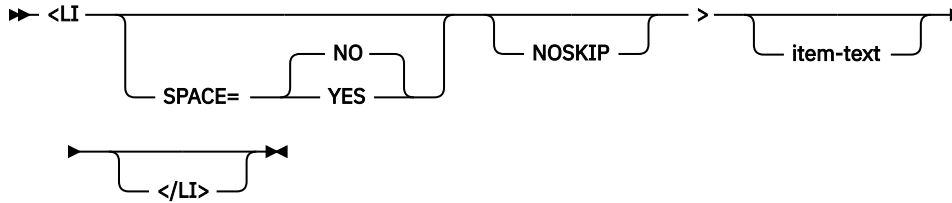


Figure 127. Function keys

LI (List Item)

The LI tag defines a list item within a note list, ordered list, unordered list, or simple list.

Syntax



Parameters

SPACE=NO | YES

The SPACE attribute controls the indentation space for the list item. When the SPACE attribute is not specified on the LI tag, the SPACE attribute from the enclosing list tag is used to set the indentation space for the *item-text*.

When SPACE=YES, the indentation is set to 3 spaces. When SPACE=NO (or SPACE is not specified), the indentation is set to 4 spaces.

The SPACE attribute can be used to control the alignment of list items when the first word of some list items is a DBCS word preceded by a shift-out character and the first word of other list items is a SBCS word.

NOSKIP

This attribute causes the list item to format without creating a blank line before the item.

item-text

This is the text of the list item.

Comments

The LI tag defines a list item within a note list, ordered list, unordered list, or simple list.

The formatting of the LI tag is dependent on the type of list you use it within and the level of nesting.

List

Formatting

Note

Formats with a 3-space or 4-space indentation (depending on the SPACE attribute) and is preceded by sequential numbers.

Ordered

Formats with a 3-space or 4-space indentation (depending on the SPACE attribute) within the level of the list in which it is defined and is preceded by sequential numbers or letters.

Simple

Formats with a 3-space or 4-space indentation (depending on the SPACE attribute) within the level of the list it is defined within.

Unordered

Formats with a 3-space or 4-space indentation (depending on the SPACE attribute) within the level of the list in which it is defined and is preceded by bullets or dashes.

The next list item implicitly ends the previous list item as do the NOTEL, OL, SL, and UL end tags.

If you do not specify text for a list item, a blank line is displayed for that item.

Restrictions

- You must code the LI tag within a NOTEL, OL, SL, or UL definition. See [“NOTEL \(Note List\)” on page 361](#), [“OL \(Ordered List\)” on page 367](#), [“SL \(Simple List\)” on page 433](#), and [“UL \(Unordered List\)” on page 443](#) for descriptions of these tags.

Processing

<i>Table 44. The tags you can code within an LI definition</i>			
Tag	Reference	Usage	Required
ATTENTION	“ATTENTION (Attention)” on page 198	Single	No
CAUTION	“CAUTION (Caution)” on page 205	Single	No
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
WARNING	“WARNING (Warning)” on page 454	Single	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains an unordered list with three list items. The last list item contains an additional paragraph of text. [Figure 128 on page 327](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM>

<HELP NAME=li DEPTH=20>ShelfBrowse Help
<AREA>
<INFO>
  <P>When ShelfBrowse finds your book,
  it displays this information:
  <UL>
    <LI>Reference information about the book.
    <LI>The location of the book.
    <LI>If the book is in stock.
    <P>If the book is not in stock, see the librarian.
  </UL>
  <P>Thank you for using ShelfBrowse.
</INFO>
</AREA>
</HELP>

```

ShelfBrowse Help

When ShelfBrowse finds your book, it displays this information:

- o Reference information about the book.
- o The location of the book.
- o If the book is in stock.

If the book is not in stock, see the librarian.

Thank you for using ShelfBrowse.

F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

Figure 128. List items

LINES (Lines)

The LINES tag defines unformatted text within an information region.

Syntax

➤ <— LINES —> —> </LINES> ➤

NOSKIP

text

Parameters

NOSKIP

This attribute causes the blank line normally placed before the lines to be skipped.

text

This is the unformatted text.

Comments

The LINES tag defines unformatted text within an information region. Tags that normally cause word-wrapping (such as the P, LI, or CAUTION) do not cause wrapping when nested within a LINES definition.

If the source text on any line is too long to fit in the remaining available formatting width, the conversion utility truncates that line. The conversion utility issues a warning message the first time that truncation occurs.

The formatting of the LINES tag is similar to that of the FIG tag, except that there is no border or caption capability.

Restrictions

- The LINES tag requires an end tag.
- You must code the LINES tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 45. The tags you can code within a LINES definition			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is application panel markup that contains a LINES definition. The formatted output of the LINES definition is identical to the input markup. [Figure 129 on page 329](#) shows the formatted results.

```
<!DOCTYPE DM SYSTEM>

<PANEL NAME=lines DEPTH=22 WIDTH=40>Lines Tag Example
<AREA>
<INFO WIDTH=38>
<P>The following area shows how the LINES tag formats.
<LINES>
First line, just at it was entered.
  Second line, ditto.

Notice we skipped a line here?

  You
    can
      even
        do
```



```

        this.
</LINES>
<P>The LINES tag formatting ends immediately above.
</INFO>
</AREA>
</PANEL>

```

Lines Tag Example

The following area shows how the LINES tag formats.

First line, just at it was entered.
Second line, ditto.

Notice we skipped a line here?

```

You
  can
    even
      do
        this.

```

The LINES tag formatting ends immediately above.

F1=Help F3=Exit F12=Cancel

Figure 129. LINES

LIT (Literal)

The LIT tag defines a string where all blanks are significant and included in the value.

Syntax

```
➡ <LIT> — literal-display-value — </LIT> ➡
```

Parameters

literal-display-value

This attribute specifies a string with all blanks preserved.

Comments

The LIT tag defines a string where all blanks are significant and included in the value. No stripping of leading, trailing, or embedded blanks is performed.

This is the only way to specify trailing blanks or a value of all blanks in the XLATI *displayed-value*.

The LIT start and end tags must be on the same line as the *literal-display-value* to preserve the original spacing of the value.

Restrictions

- The LIT tag requires an end tag.
- You must code the LIT tag only within an XLATI definition. See [“XLATI \(Translate Item\)” on page 456](#) for a complete description of this tag.

- Multiple LIT tags may be coded within a single XLATI definition, as long as they are not nested within each other. However, a better approach is to include the whole XLATI *displayed-value* within the LIT tag.

Processing

None.

Examples

Here is markup that contains a variable class definition with two translate lists. The last four translate items in the second list contain LIT definitions that preserve trailing blanks in the displayed value of their respective translate items.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=aa TYPE='char 2'>
<VARCLASS NAME=bb TYPE='char 9'>
<VARCLASS NAME=cc TYPE='char 9'>
  <XLATL FORMAT=upper>
  </XLATL>
  <XLATL>
    <XLATI VALUE=1>BIGCHARGE
    <XLATI VALUE=2><LIT>V I S T A</LIT>
    <XLATI VALUE=3><LIT>EZCARD </LIT>
    <XLATI VALUE=4><LIT>CHECK </LIT>
    <XLATI VALUE=5><LIT> CASH</LIT>
  </XLATL>

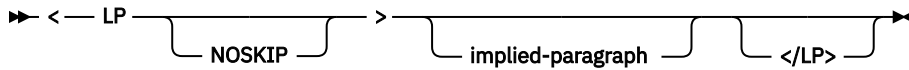
  <VARLIST>
    <VARDCL NAME=dispva VARCLASS=cc>
    <VARDCL NAME=inptva VARCLASS=bb>
    <VARDCL NAME=chckva VARCLASS=aa>
  </VARLIST>

  <PANEL NAME=lit>LIT translation
  <TOPINST>You can display this panel with ISPF option 7.2
  <TOPINST>For this example, enter the word 'BIGCHARGE', 'V I S T A',
    'EZCARD', 'CHECK', or ' CASH' in the "literal value"
    field (no quotes).
  <TOPINST>The literal will be translated to the corresponding number
    defined in the XLATL tag, and will be displayed in the
    "translated value" field.
  <TOPINST>The literal you enter will be displayed (left justified) in
    the "original value" field.
  <DTACOL>
  <!-- assign "literal value" to "original value" -->
  <SOURCE>
  &inptva = &dispva
  </SOURCE>
  <DTAFLD DATAVAR=dispva ENTWIDTH=9 PMTWIDTH=20 ALIGN=center>Literal value
  <DTAFLD DATAVAR=chckva ENTWIDTH=2 PMTWIDTH=20 USAGE=out>Translated value
  <DTAFLD DATAVAR=inptva ENTWIDTH=9 PMTWIDTH=20 USAGE=out>Original value
  <!-- assign translated "literal value" to "translated value" -->
  <SOURCE>
  &chckva = &dispva
  </SOURCE>
  </DTACOL>
  <CMDAREA>
</PANEL>
```

LP (List Part)

The LP tag defines a comment or explanation within a note list, ordered list, unordered list, or simple list.

Syntax



Parameters

NOSKIP

This attribute causes the list part to format without creating a line before the list part.

implied-paragraph

This is the text of the list part.

Comments

The LP tag defines a comment or explanation within an ordered list, unordered list, or simple list. You can code the LP tag anywhere within a list.

The text of the list part starts at the left margin of the current level of the list. It is not numbered or lettered. When you use it within a NOTEL or OL definition, LP does not interrupt or increment the sequence.

The next list item or the end of the list implicitly ends the list part.

Restrictions

- You must code the LP tag within a NOTEL, OL, SL, or UL definition. See [“NOTEL \(Note List\)” on page 361](#), [“OL \(Ordered List\)” on page 367](#), [“SL \(Simple List\)” on page 433](#), and [“UL \(Unordered List\)” on page 443](#) for descriptions of these tags.

Processing

<i>Table 46. The tags you can code within an LP definition</i>			
Tag	Reference	Usage	Required
ATTENTION	“ATTENTION (Attention)” on page 198	Single	No
CAUTION	“CAUTION (Caution)” on page 205	Single	No
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Table 46. The tags you can code within an LP definition (continued)			
Tag	Reference	Usage	Required
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
WARNING	“WARNING (Warning)” on page 454	Single	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains an ordered list with a nested list part tag. WARNING and P tags are nested within the list part definition. [Figure 130 on page 332](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=lp WIDTH=50 DEPTH=20>Help For Changing a File
<AREA>
<INFO>
  <OL>
    <LI>Type over the existing data
    in the entry fields with the new data.
    <LP>
      <WARNING>
        Performing the next step will save all changes
        and delete the existing data.
      <P>To quit this function without
        deleting the existing data, press F12=Cancel.
      </WARNING>
    <LI>Press Enter to save the
    updated data.
  </OL>
</INFO>
</AREA>
</HELP>
```

Help For Changing a File

1. Type over the existing data in the entry fields with the new data.

Warning: Performing the next step will save all changes and delete the existing data.

To quit this function without deleting the existing data, press F12=Cancel.

2. Press Enter to save the updated data.

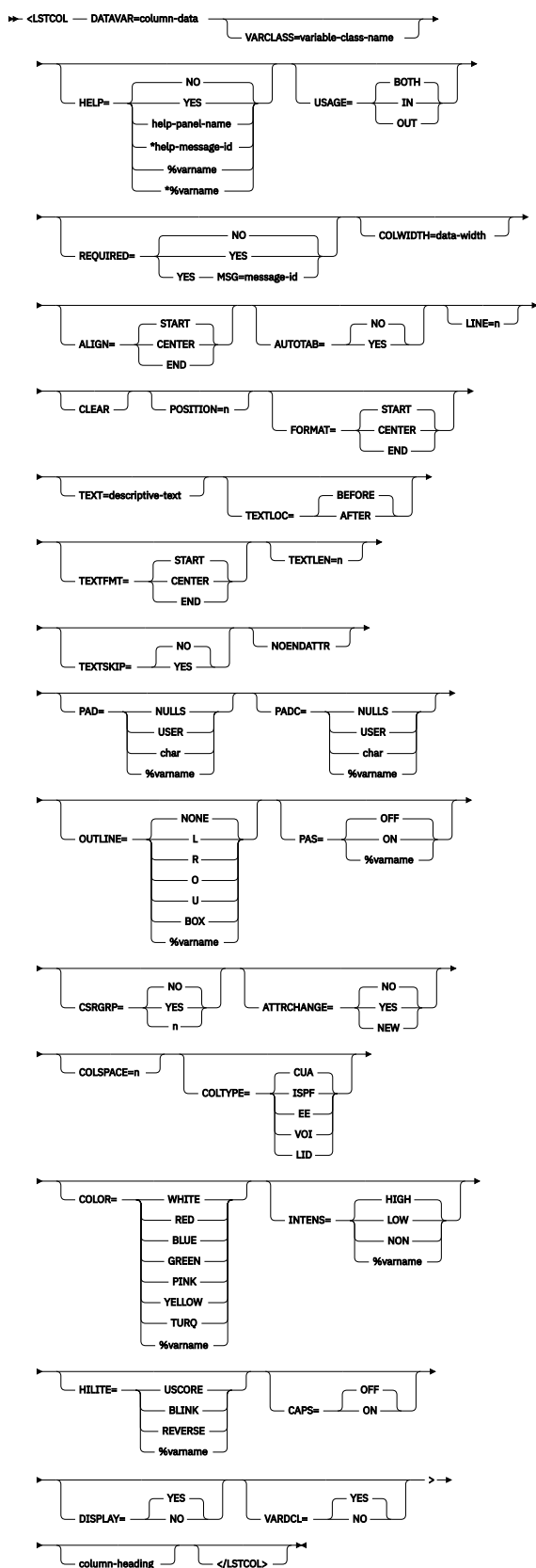
F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

Figure 130. List part

LSTCOL (List Column)

The LSTCOL tag defines a column of data from an ISPF table displayed in the ISPF table display area of a panel.

Syntax



Parameters

DATAVAR=column-data

This is the data which occupies the column. The *column-data* value must be an ISPF table variable name (without a leading % sign).

VARCLASS=variable-class-name

This is the name of a variable class, defined with a VARCLASS tag, that overrides the default variable class referred to by the VARDCL tag that declares the data variable for the list column.

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for this list column. This is field-level help.

When HELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help for a list column and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

USAGE=BOTH | IN | OUT

This attribute indicates if this column is for input, output, or both.

REQUIRED=NO | YES

This attribute indicates if this column is required to have input for each modified row. The default is NO. This attribute is valid only when USAGE=IN or BOTH.

If you specify REQUIRED=YES, a conditional VER(variable, NONBLANK) statement is built by the conversion utility and placed in the)PROC section of the ISPF panel generated. This results in the column variable being verified only when the row is selected or modified.

MSG=message-id

This attribute specifies the message that is displayed when the user does not complete a required entry (defined with the REQUIRED attribute). If you do not specify a *message-id*, ISPF displays a default message.

If you specify the MSG attribute and REQUIRED=YES, a VER(variable, NONBLANK, MSG=message-identifier) statement is built by the conversion utility and placed in the)PROC section of the ISPF panel generated. If you specify the MSG attribute and REQUIRED=NO (the default), the conversion utility issues a warning message.

See [“MSG \(Message\)”](#) on page 352 for information about creating messages.

Note: You can specify messages pertaining to other validations using XLATL and CHECKL tags within a VARCLASS definition. See the descriptions of these tags for additional information.

COLWIDTH=data-width

This attribute determines the data width to be used by the column. If you do not specify this attribute, the data width and column formatting width are determined by the actual length of the *column-heading*. If the width of the *column-heading* text is greater than the COLWIDTH, it is used as the column formatting width. The minimum width is 1 and the maximum is the remaining available panel (or region) width. If the *column-heading* and the COLWIDTH attribute are omitted, the data width and column formatting width are determined by the TYPE value of the associated VARCLASS. If a VARCLASS TYPE value is not available, the size of the column variable name (specified by the DATAVAR attribute) determines the width.

You should code the COLWIDTH attribute with a value equal to the length of the table data variable.

ALIGN=START | CENTER | END

This attribute specifies how the data value is to be displayed in the data field.

An attribute character is used for the field that specifies JUST(LEFT) if ALIGN=START, JUST(ASIS) if ALIGN=CENTER or JUST(RIGHT) if ALIGN=END. When ALIGN=END, no underscore padding is performed; blanks are used.

AUTOTAB=NO | YES

When AUTOTAB=YES, the cursor moves to the next field capable of input when the user enters the last character in the list column field.

AUTOTAB=YES is valid only when the value for USAGE is either BOTH or IN.

LINE=n

This attribute provides the ability to place LSTCOL fields on different model lines. ISPF defines the range of lines as 1 to 8. The default is 1. Column headings are generated on multiple lines to match the LSTCOL field placement.

CLEAR

This attribute adds a CLEAR (variable, ...) statement to the)MODEL line. CLEAR should be specified for table extension variables.

For more information about the)MODEL line, refer to the [*z/OS ISPF Dialog Developer's Guide and Reference*](#).

POSITION=n

This attribute specifies the starting position of the data column and related text or the column heading, if the heading is longer than the data column. If this attribute is not specified or is not valid, the conversion utility formats the column immediately to the right of the previous column on the specified or default model line. This attribute allows you to position fields on different model lines with vertical alignment. Column position is location of the attribute byte preceding the data column.

FORMAT=START | CENTER | END

This attribute specifies how the data column and its column heading are formatted. If you do not specify this attribute, or if you specify the attribute value START, then the column formats as in ISPF Version 3.1 and ISPF Version 3.2.

Formatting of the data in the column takes place within the column width, which is determined as described in the COLWIDTH attribute section.

When you specify the attribute value CENTER, the conversion utility centers a column heading that is shorter than the column width. If the column heading is longer than the column width, then the data column is centered under the column heading. When either the heading or the data column is centered, blank characters are added before and after the column heading or data column. The total amount of space to be added is divided by 2 and the resulting whole number is the number of blanks added in front of the column heading or data column. The difference between the total amount of space and the amount placed in front of the column heading or data column is used at the end.

When you specify the attribute value END, a column heading that is shorter than the column width is right-justified so it aligns with the end of the displayed data. If the column heading is longer than the column width, the data column is right-justified so that the displayed data and the column heading end at the same position.

If there is insufficient space available to format the column heading as requested, the conversion utility issues a message that the FORMAT attribute is ignored.

The FORMAT attribute does not affect the display of the field contents within the data column, which is determined by the ALIGN attribute.

TEXT=descriptive-text

This attribute specifies a short description of the data column. It can be placed before or after the data column. Text containing special characters or embedded blanks must be enclosed in quotes.

TEXTLOC=BEFORE | AFTER

This attribute specifies the location of the TEXT relative to the data column. Text can be placed on either side of the data column.

TEXTFMT=START | CENTER | END

This attribute specifies the format of the text within the length of the text area. The text can be left-justified, centered, right-justified.

TEXTLEN=n

This attribute specifies the amount of space to reserve for formatting the descriptive text. This attribute helps you line up text on different model lines, and if the space reserved is longer than the descriptive text, it permits formatting within the reserved space with the TEXTFMT attribute. If the descriptive text is longer than the space reserved by the TEXTLEN attribute, the descriptive text is not formatted and a warning message is issued.

TEXTSKIP=NO | YES

This attribute provides for skipping past the *descriptive text* when either the TEXTLOC=BEFORE and the previous LSTCOL tag includes the NOENDATTR attribute, or TEXTLOC=AFTER and the current LSTCOL tag includes the NOENDATTR attribute. If there is no other input field on the panel, the cursor moves to the first input field.

NOENDATTR

This attribute specifies that no ending attribute character is placed after the data column. NOENDATTR is ignored for the last data column on each model line.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a "%".

PADC=NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a "%".

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a "%".

PAS=OFF | ON | %varname

This attribute controls the availability of the point-and-shoot function for this table field. You can define this attribute as a variable name preceded by "%"

CSRGRP=NO | YES | n

When CSRGRP=YES, the conversion utility generates a cursor group number to be used for this table column. When CSRGRP=n, the number provided is used for this field. The PAS attribute must be specified as ON or %varname.

The conversion utility accepts the CSRGRP attribute for any table field definition. The CSRGRP attribute is used at runtime for output fields only.

ATTRCHANGE=NO | YES | NEW

When ATTRCHANGE=YES or ATTRCHANGE=NEW, the conversion utility formats an additional entry in the panel)ATTR section (that can apply to multiple list columns) instead of creating a unique ".ATTR(field-name)" entry in the)INIT section for this field. With this option, multiple LSTCOL tags with the same characteristic require fewer panel logic statements. ATTRCHANGE=NEW creates a new entry. ATTRCHANGE=YES uses an existing entry, if possible.

COLSPACE=n

The COLSPACE attribute specifies the total number of bytes for the column width, including the leading and trailing attributes, and any trailing blank following an input field. The use of the COLSPACE attribute causes column heading text longer than the COLSPACE value (minus the attribute bytes) to be flowed into multiple lines.

COLTYPE=CUA | ISPF | EE | VOI | LID

This attribute defines the attribute type to be applied to the table field. TYPE=CUA, the default, causes the field to display using the standard CUA attribute.

VOI and LID are valid only when USAGE=OUT.

EE is valid when USAGE=IN or USAGE=BOTH.

Using a CUA attribute causes the field to appear in the associated color.

When COLTYPE=ISPF, a non-CUA attribute is generated and you can specify the color, intensity, and highlighting of the field using the COLOR, INTENS, and HILITE attributes. These attributes are not valid for CUA types.

COLOR=WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENS=HIGH | LOW | NON | %varname

This attribute defines the intensity of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

HILITE=USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute of the field. You can define this attribute as a variable name preceded by a percent (%) sign.

CAPS=OFF | ON

When CAPS=ON, the data in the field is displayed in uppercase characters.

DISPLAY=YES | NO

This attribute specifies whether the data for the field is visible when the panel is displayed. This attribute is used to allow fields to contain information you do not want to appear on the screen.

VARDCL=YES | NO

When VARDCL=NO the list column name is not checked to the declared variable information provided with the VARCLASS and VARDCL tags.

column-heading

This is the text of the list column heading. If the length of the *column-heading* and the COLWIDTH values are not equal, the greater of the two is used to determine column formatting width. If the column-heading and the COLWIDTH attributes are omitted, the column formatting width is determined by the TYPE value of the associated VARCLASS. If a VARCLASS TYPE value is not available, the size of the column variable name (specified by the DATAVAR attribute) determines the width.

The *column-heading* text placement over the column is determined by the FORMAT attribute value.

Comments

In conjunction with the LSTFLD tag, LSTCOL tags provide a means of defining a vertically scrollable list display area that is made up of columns of data coming from ISPF table data. One or more ISPF)MODEL section statements is built to display the fields defined by the LSTCOL tags. The use of LSTCOL tags requires the use of the TBDISPL service in the application program.

If the ISPF panel width is smaller than the total width of the group of columns, columns that exceed the panel width are clipped from the right. A warning message is issued if this condition occurs.

You can use the LINE attribute to format your table to display on multiple lines.

If NOENDATTR is not specified, the conversion utility generates a beginning and ending attribute for each column of the table display)MODEL line. An additional blank is also inserted for fields with USAGE=IN or BOTH if AUTOTAB=NO. This characteristic results in these conditions:

- When USAGE=OUT, 2 extra spaces are added to the defined column formatting width.
- When AUTOTAB=YES and USAGE=IN or BOTH, 2 extra spaces are added to the defined column formatting width.
- When AUTOTAB=NO (the default) and USAGE=IN or BOTH, 3 extra spaces are added to the defined column formatting width.

It is important that you allow for this extra space when designing your panel. The extra space is added to the width value for the field as defined in the description of the COLWIDTH attribute.

When the maximum number of requested attributes for a panel is exceeded, the conversion utility issues error message ISPC804E. The number of requested attributes includes attribute override entries. These

are .ATTR entries that are added by the conversion utility for attributes that are specified on CHOFLD, DTACOL, DTAFLD, LSTCOL, and LSTFLD tags. If the same set of attributes is specified on multiple tags, duplicate .ATTR entries are added by default. Adding the parameter ATTRCHANGE=YES to the tags causes the compiler to instead add a single entry in the panel)ATTR section for each unique set of attributes specified. The entry for a set of attributes is then shared by all tags that specify that set of attributes.

Restrictions

- You must code the LSTCOL tag within a LSTFLD or LSTGRP tag. See [“LSTFLD \(List Field\)” on page 341](#) for a complete description of this tag.
- Each LSTCOL definition should have a VARDCL definition associated with the variable value specified with the DATAVAR attribute. See [“VARDCL \(Variable Declaration\)” on page 449](#) for a complete description of this tag.
- Only MODEL lines that actually are formatted with fields are written in the panel body. Thus, if some LSTCOL entries specify LINE=1 and others specify LINE=3, but there are no LSTCOL entries for LINE 2, only two MODEL lines are created.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

<i>Table 47. The tags you can code within a LSTCOL definition</i>			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SCRFLD	“SCRFLD (Scrollable Field)” on page 413	Single	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is source file markup where the application panel contains a list field with five list columns. The first three columns are defined as output-only, and are coded within the Subscriber Name list group. The Number column is an input/output column, and it is coded within the Phone list group. The last column is input-only, and it is coded within the Approved list group. This column requires input, so if it is not filled in, the error message MSGG886 is displayed. The variable declarations and classes associated with the list columns are also shown. [Figure 131 on page 339](#) shows the formatted result of the application panel.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=namecls TYPE='char 15'>
<VARCLASS NAME=midcls TYPE='char 1'>
<VARCLASS NAME=phoncls TYPE='char 12'>
<VARCLASS NAME=appcls TYPE='char 1'>
  <XLATL FORMAT=upper>
</XLATL>
<CHECKL>
  <CHECKI TYPE=values PARM1=EQ PARM2='Y N'>
</CHECKL>

<VARLIST>
  <VARDCL NAME=xfname VARCLASS=namecls>
  <VARDCL NAME=xlname VARCLASS=namecls>
  <VARDCL NAME=xmid VARCLASS=midcls>
  <VARDCL NAME=xphone VARCLASS=phoncls>
  <VARDCL NAME=xapp VARCLASS=appcls>
</VARLIST>

<PANEL NAME=lstcola KEYLIST=keyltbl>Subscriber List
<TOPINST>Enter phone number, if missing,
(format - nnn-xxx-nnnn) and approved
indicator (y or n) for each person.
<AREA>
  <LSTFLD>
    <LSTGRP HEADLINE=yes>Subscriber Name
      <LSTCOL DATAVAR=xfname USAGE=out COLWIDTH=15>First Name
      <LSTCOL DATAVAR=xlname USAGE=out COLWIDTH=15>Last Name
      <LSTCOL DATAVAR=xmid USAGE=out COLWIDTH=1>MI
    </LSTGRP>
    <LSTGRP>Phone
      <LSTCOL DATAVAR=xphone COLWIDTH=12>Number
    </LSTGRP>
    <LSTGRP>Approved
      <LSTCOL DATAVAR=xapp USAGE=in REQUIRED=yes
        COLWIDTH=1 MSG=msgg886>(Y or N)
    </LSTGRP>
  </LSTFLD>
</AREA>
<CMDAREA>
</PANEL>

```

Subscriber List ROW 1 TO 3 OF 3

Enter phone number, if missing, (format - nnn-xxx-nnnn) and approved indicator (y or n) for each person.

----- Subscriber Name -----			Phone	Approved
First Name	Last Name	MI	Number	(Y or N)
Pete	Moss	P	919-555-4444	-
Sally	Forth	N	-----	-
Melba	Toast	T	919-555-8888	-

***** BOTTOM OF DATA *****

Command ==> -----

F1=Help	F2=Split	F3=Exit	F7=Backward	F8=Forward
F9=Swap	F12=Cancel			

Figure 131. List columns

To display the same table in a different format, we can change the LSTCOL tags for name to include the LINE attribute. The DTL changes are reflected in the this example.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=namecls TYPE='char 15'>
<VARCLASS NAME=midcls TYPE='char 1'>
<VARCLASS NAME=phoncls TYPE='char 12'>
<VARCLASS NAME=appcls TYPE='char 1'>
  <XLATL FORMAT=upper>
</XLATL>
<CHECKL>
  <CHECKI TYPE=values PARM1=EQ PARM2='Y N'>
</CHECKL>

<VARLIST>
  <VARDCL NAME=xfname VARCLASS=namecls>
  <VARDCL NAME=xlname VARCLASS=namecls>
  <VARDCL NAME=xmid VARCLASS=midcls>
  <VARDCL NAME=xphone VARCLASS=phoncls>
  <VARDCL NAME=xapp VARCLASS=appcls>
</VARLIST>

<PANEL NAME=lstcolb KEYLIST=keyltbl>Subscriber List
<TOPINST>Enter phone number, if missing,
(format - nnn-xxx-nxxx) and approved
indicator (y or n) for each person.
<AREA>
  <LSTFLD DIV=solid>
    <LSTGRP HEADLINE=yes>Subscriber Name
      <LSTCOL DATAVAR=xfname USAGE=out LINE=1 COLWIDTH=15>First Name
      <LSTCOL DATAVAR=xlname USAGE=out LINE=2 COLWIDTH=15>Last Name
      <LSTCOL DATAVAR=xmid USAGE=out LINE=3 COLWIDTH=1>MI
    </LSTGRP>
    <LSTGRP>Phone
      <LSTCOL DATAVAR=xphone COLWIDTH=12>Number
    </LSTGRP>
    <LSTGRP>Approved
      <LSTCOL DATAVAR=xapp USAGE=in REQUIRED=yes
        COLWIDTH=1 MSG=msgg886>(Y or N)
    </LSTGRP>
  </LSTFLD>
</AREA>
<CMDAREA>
</PANEL>
```

Figure 132 on page 340 shows the formatted result of the application panel.

Subscriber List			ROW 1 TO 3 OF 3	
Enter phone number, if missing, (format - nnn-xxx-nxxx) and approved indicator (y or n) for each person.				
Subscriber Name	Phone	Approved		
First Name	Number	(Y or N)		
Last Name				
MI				
Pete	919-555-4444	-		
Moss				
P				

Sally	-----	-		
Forth				
N				

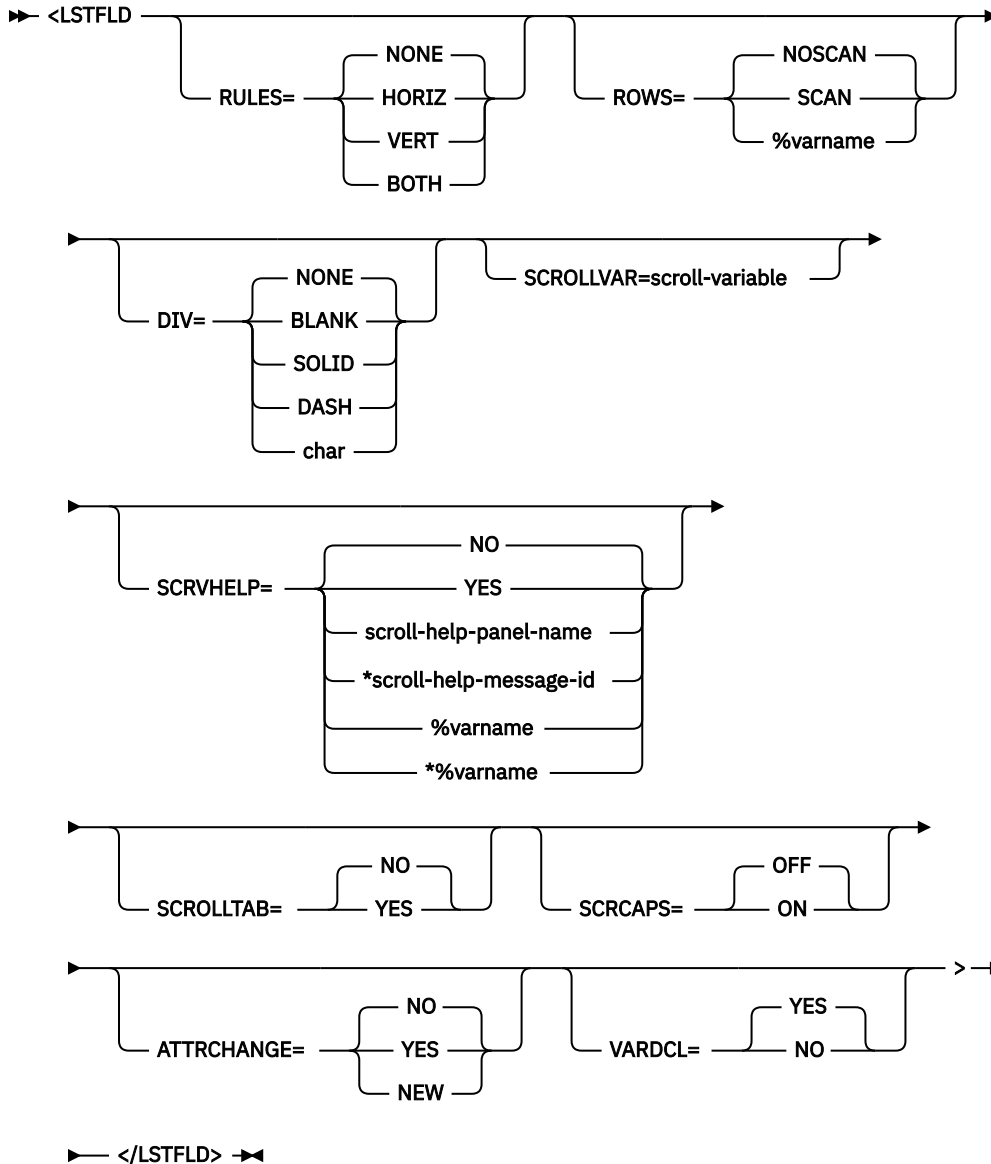
Melba	919-555-8888	-		
Toast				
Command ==>				
F1=Help	F2=Split	F3=Exit	F7=Backward	F8=Forward
F9=Swap	F12=Cancel			

Figure 132. List columns

LSTFLD (List Field)

The LSTFLD tag defines an ISPF table display area that is made up of columns of data coming from ISPF tables.

Syntax



Parameters

RULES=NONE | HORIZ | VERT | BOTH

This attribute specifies the type of interior rules that are drawn in the table display being defined within the LSTFLD tag. This applies to all the list columns within the context of this tag.

This attribute is supported by using the ISPF outline (L|R|O|U|Box|None) statement on panel definition statements. However, the lines around fields are only visible on double-byte character support terminals.

Note: Any list column field within the list field defining OUTLINE overrides the LSTFLD RULES value.

ROWS=NOSCAN | SCAN | %varname

This attribute provides support by TDISPL of rows previously selected by the TBSARG service. If you specify ROWS=SCAN, the conversion utility adds ROWS(SCAN) to the)MODEL line statement in the generated ISPF panel.

If you specify ROWS=%varname, ROWS(&varname) is added to the)MODEL line. The application must set the variable name to ALL or SCAN before the panel is displayed.

DIV=NONE | BLANK | SOLID | DASH | char

This attribute specifies the type of divider line to be added as the last line of a model set. If this attribute is omitted or specified as NONE, the divider line is not generated. If this attribute is specified as BLANK, a blank divider line is generated. You may specify either SOLID or DASH to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line. Alternately, you can specify a character or a character string of your choice. The character or characters provided are replicated to the available width of the panel (or region) to create the divider line.

If you have defined LSTCOL tags for all 8 of the available model lines, then the conversion utility issues a message and does not generate any divider line.

SCROLLVAR=scroll-variable

This attribute specifies the name of a variable that the application uses to obtain scrolling information. The *scroll-variable* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

If the attribute is specified, the conversion utility creates a scroll entry on the command line, providing that the resulting command area allows at least 8 bytes for a command entry.

SCRVHELP=NO | YES | scroll-help-panel-name | *scroll-help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for the field specified with the SCROLLVAR attribute.

When SCRHELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When %varname is coded, a panel variable name is created. When *%varname is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *scroll-help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

SCROLLTAB=NO | YES

When SCROLLTAB=YES, the cursor moves to the next input field when the user enters the last character in the scroll amount field. If there is no other input field on the panel, the cursor moves to the beginning of the command line.

SCRCAPS=OFF | ON

When SCRCAPS=ON, the data in the scroll field is displayed in uppercase characters.

ATTRCHANGE=NO | YES | NEW

When ATTRCHANGE=YES or ATTRCHANGE=NEW, the conversion utility formats an additional entry in the panel)ATTR section (that can apply to multiple list columns) instead of creating a unique ".ATTR(field-name)" entry in the)INIT section for each field. With this option, multiple LSTCOL tags with the same characteristics require fewer panel logic statements. ATTRCHANGE=NEW creates a new entry. ATTRCHANGE=YES uses an existing entry, if possible.

Note: Any list column field within the list field defining ATTRCHANGE overrides the LSTFLD ATTRCHANGE value.

VARDCL=YES | NO

When VARDCL=NO, the list field name is not checked to the declared variable information provided with the VARCLASS and VARDCL tags.

Note: Any list column field within the list field defining VARDCL overrides the LSTFLD VARDCL value.

Comments

The LSTFLD tag defines a scrollable list display area that is made up of columns of data coming from ISPF table data. The conversion utility creates a)MODEL line at the bottom of the)BODY section of the panel the list field is coded within.

The use of the LSTFLD tag causes all other tags that generate panel data and that are coded after the LSTFLD end tag to be moved before the)MODEL statement. This is because ISPF does not allow any panel body definition after the)MODEL statement.

When the maximum number of requested attributes for a panel is exceeded, the conversion utility issues error message ISPC804E. The number of requested attributes includes attribute override entries. These are .ATTR entries that are added by the conversion utility for attributes that are specified on CHOFLD, DTACOL, DTAFLD, LSTCOL, and LSTFLD tags. If the same set of attributes is specified on multiple tags, duplicate .ATTR entries are added by default. Adding the parameter ATTRCHANGE=YES to the tags causes the compiler to instead add a single entry in the panel)ATTR section for each unique set of attributes specified. The entry for a set of attributes is then shared by all tags that specify that set of attributes.

Restrictions

- The LSTFLD tag requires an end tag.
- You must code the LSTFLD tag within an AREA, REGION, or PANEL definition. See [“AREA \(Area\)” on page 189](#), [“REGION \(Region\)” on page 405](#), and [“PANEL \(Panel\)” on page 376](#) for descriptions of these tags.
- You can code only one list field on an application panel.
- You should code a CMDAREA on any panel that contains a LSTFLD definition. If you do not include the CMDAREA tag, the conversion utility inserts one and issues a message, unless the PANEL tag specifies CMDLINE=NO.
- You can use the SCROLLVAR attribute only once within a panel.
- The resulting scroll entry on the command line must leave at least 8 positions for the command entry field.
- If you specify the SCRHELP attribute, you must also specify the SCROLLVAR attribute.

Processing

<i>Table 48. The tags you can code within a LSTFLD definition</i>			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
LSTCOL	“LSTCOL (List Column)” on page 332	Multiple	No
LSTGRP	“LSTGRP (List Group)” on page 345	Multiple	No
LSTVAR	“LSTVAR (List Variable)” on page 348	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is an application panel in a source file markup that contains a list field with five list columns of data. In addition, three list groups are defined within the list field. The first three list columns are output-only columns. The fourth list column uses the default value *both*, which allows it to handle both input and output data. The last list column is an input-only column, and input by the user is required. [Figure 133 on page 345](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=namecls TYPE='char 15'>
<VARCLASS NAME=midcls TYPE='char 1'>
<VARCLASS NAME=phoncls TYPE='char 12'>
<VARCLASS NAME=appcls TYPE='char 1'>
  <XLATL FORMAT=upper>
</XLATL>
<CHECKL>
  <CHECKI TYPE=values PARM1=EQ PARM2='Y N'>
</CHECKL>

<VARLIST>
  <VARDCL NAME=xfname VARCLASS=namecls>
  <VARDCL NAME=xlname VARCLASS=namecls>
  <VARDCL NAME=xmid VARCLASS=midcls>
  <VARDCL NAME=xphone VARCLASS=phoncls>
  <VARDCL NAME=xapp VARCLASS=appcls>
</VARLIST>

<PANEL NAME=lstfld3 KEYLIST=keyltbl>Subscriber List
<TOPINST>Enter phone number, if missing,
(format - nnn-xxx-xxxx) and approved
indicator (y or n) for each person.
<AREA>
  <LSTFLD SCROLLVAR=scr1amt SCRHELP=scrhelp>
    <LSTGRP HEADLINE=yes>Subscriber Name
      <LSTCOL DATAVAR=xfname USAGE=out COLWIDTH=15>First Name
      <LSTCOL DATAVAR=xlname USAGE=out COLWIDTH=15>Last Name
      <LSTCOL DATAVAR=xmid USAGE=out COLWIDTH=1>MI
    </LSTGRP>
    <LSTGRP>Phone
      <LSTCOL DATAVAR=xphone COLWIDTH=12>Number
    </LSTGRP>
    <LSTGRP>Approved
      <LSTCOL DATAVAR=xapp USAGE=in REQUIRED=yes
        COLWIDTH=1 MSG=msgg886>(Y or N)
    </LSTGRP>
  </LSTFLD>
</AREA>
<CMDAREA>
</PANEL>
```

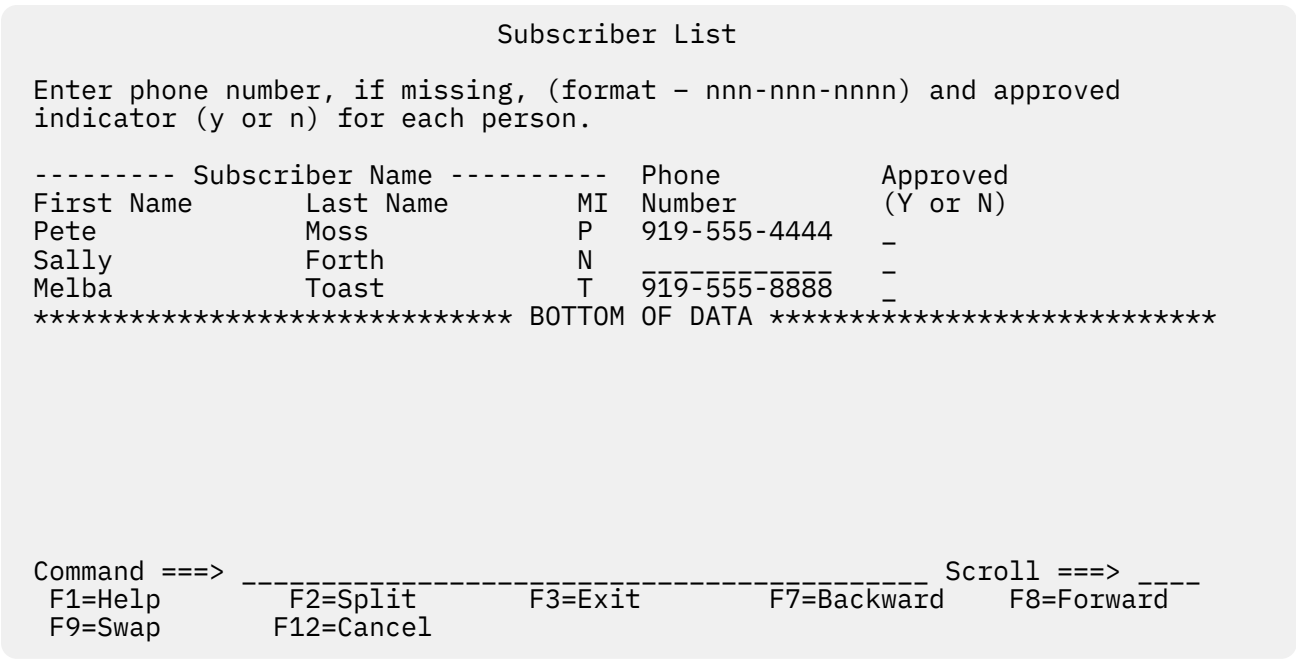
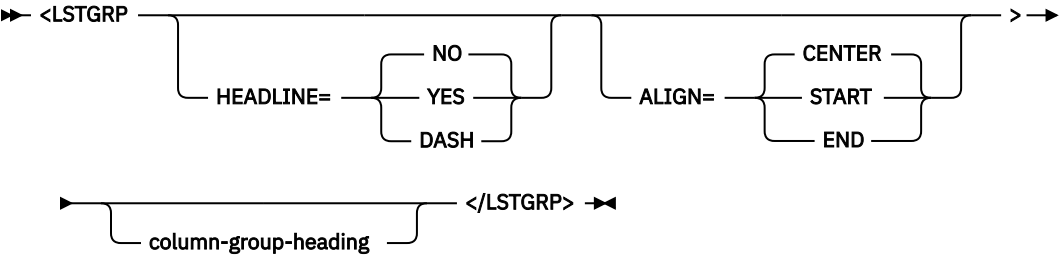



Figure 133. List field

LSTGRP (List Group)

The LSTGRP tag defines a heading for a single column or multiple columns within a list field.

Syntax



Parameters

HEADLINE=NO | YES | DASH

This attribute specifies whether the heading text is padded to span the width of the group heading not occupied by the text. This provides a visual indication of the columns that belong to a group heading. You must specify YES or DASH to produce the visible indicator. When the GRAPHIC invocation option is specified, YES produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both YES and DASH produce a dashed line.

ALIGN=CENTER | START | END

This attribute specifies how the list group heading is formatted. If you do not specify this attribute, or if you specify ALIGN=CENTER, then the heading is centered over multiple columns or a variable model line, or left-justified over a single column.

When ALIGN=START, the list group heading is left-justified. When ALIGN=END, the list group heading is right-justified.

column-group-heading

The heading is placed above the column group in the nonscrollable part of the list field. The heading must fit on one line above the column or columns in the group. If *column-group-heading* text is longer than the formatted width of the column or columns in the group, it is truncated. The *column-group-heading* appears on the line immediately above the group of columns.

If you do not specify *column-group-headings* for any of the columns within the group, the conversion utility reserves the area where the heading would be displayed and fill it with blanks. If the *column-group-heading* is not specified but HEADLINE=YES is specified, the heading contains only a dashed line.

Comments

The LSTGRP tag defines a heading for a single column or multiple columns within a list field. You can use the LSTGRP tag to group columns in a list field together under a single heading that applies to all of the columns. You create the columns using the LSTCOL or LSTVAR tag.

The list field can contain other columns that do not belong to the list column group. Only the LSTCOL or LSTVAR definitions nested within the LSTGRP tag belong to the group.

There must be at least one LSTCOL tag, nested LSTGRP tag, or LSTVAR tag defined within a column group. The column formatting widths, and the gutters between them, define how much space is allocated for the group heading. If this space is less than the space needed for the group heading, the conversion utility truncates the heading. If the LSTGRP definition contains only one LSTCOL tag, and the ALIGN attribute is not specified, the group heading is left-justified over the column.

You can use the LSTGRP tag to specify multiple lines of single column headings or multiple lines of multiple column headings.

Restrictions

- The LSTGRP tag requires an end tag.
- You must code the LSTGRP tag within a LSTFLD definition or another LSTGRP definition. See [“LSTFLD \(List Field\)”](#) on page 341 for a complete description of the LSTFLD tag.
- You can code multiple LSTGRP tags within a LSTFLD definition.
- A LSTGRP definition must contain a nested LSTCOL, LSTVAR, or LSTGRP tag, otherwise the conversion utility issues an error.
- The nested tags LSTCOL definitions must include at least one data column from the first displayable model line.

Processing

Table 49. The tags you can code within a LSTGRP definition			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LSTCOL	“LSTCOL (List Column)” on page 332	Multiple	Yes
LSTGRP	“LSTGRP (List Group)” on page 345	Multiple	No
LSTVAR	“LSTVAR (List Variable)” on page 348	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Table 49. The tags you can code within a LSTGRP definition (continued)

Tag	Reference	Usage	Required
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is source file markup where the application panel contains a list field with six list columns. The first three columns are placed under a common group, as are the last two columns. Also, for each of the first three columns, a second-level group heading is used in place of list column headings. This technique provides a blank space between the group headings and the data columns. [Figure 134 on page 348](#) shows the formatted result of the application panel.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=namecls TYPE='char 12'>
<VARCLASS NAME=midcls TYPE='char 1'>
<VARCLASS NAME=yearcls TYPE='char 9'>
<VARCLASS NAME=semcls TYPE='char 2'>

<VARLIST>
  <VARDCL NAME=xfname VARCLASS=namecls>
  <VARDCL NAME=xlname VARCLASS=namecls>
  <VARDCL NAME=xmid VARCLASS=midcls>
  <VARDCL NAME=xyear VARCLASS=yearcls>
  <VARDCL NAME=sem1 VARCLASS=semcls>
  <VARDCL NAME=sem2 VARCLASS=semcls>
</VARLIST>

<PANEL NAME=lstgrp WIDTH=66 KEYLIST=keyl1tbl>Class Roster
<AREA>
  <LSTFLD>
    <LSTGRP HEADLINE=yes>Student Name
      <LSTGRP>Last
        <LSTCOL DATAVAR=xlname USAGE=out COLWIDTH=12>
      </LSTGRP>
      <LSTGRP>First
        <LSTCOL DATAVAR=xfname USAGE=out COLWIDTH=12>
      </LSTGRP>
      <LSTGRP>M
        <LSTCOL DATAVAR=xmid USAGE=out COLWIDTH=1>
      </LSTGRP>
      <LSTGRP>
      <LSTGRP>Class
      <LSTGRP>Year
        <LSTCOL DATAVAR=xyear USAGE=out COLWIDTH=9>
      </LSTGRP>
      <LSTGRP>
      <LSTGRP HEADLINE=yes>Grade
        <LSTCOL DATAVAR=sem1 COLWIDTH=2>Sem 1
        <LSTCOL DATAVAR=sem2 COLWIDTH=2>Sem 2
      </LSTGRP>
    </LSTFLD>
  </AREA>
</CMDAREA>
</PANEL>
```

```

                                Class Roster                                ROW 1 TO 6 OF 6
----- Student Name ----- Class      -- Grade ---
Last          First          M Year          Sem 1  Sem 2
Scott         Dean           T  Junior          A      B+
Lewis         Dana           L  Freshman      B+      B
Roy           Sergio         J  Post-Grad      D      D
Romero        Maria          C  Post-Grad      A      A
Spencer       Alan            M  Freshman      A      B
Zhou          Alex            B  Senior         C+      B
***** BOTTOM OF DATA *****

```

Command ==> _____

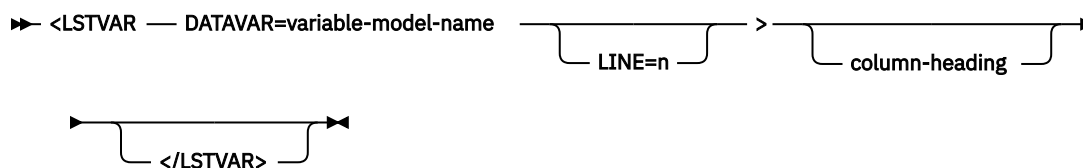
F1=Help	F2=Split	F3=Exit	F7=Backward	F8=Forward
F9=Swap	F12=Cancel			

Figure 134. List group

LSTVAR (List Variable)

The LSTVAR tag defines a)MODEL section variable model line displayed in the ISPF table display area of a panel.

Syntax



Parameters

DATAVAR=variable-model-name

This is the data that occupies the column. The *variable-model-name* value must be a variable model line name (without a leading % sign).

LINE=n

This attribute provides the application the ability to place a LSTVAR model variable on different model lines. ISPF defines the range of lines as 1 to 8. The default is 1. Headings are generated on multiple lines to match the LSTVAR field placement.

column-heading

This is the text of the model variable heading.

Comments

In conjunction with the LSTFLD and LSTCOL tags, LSTVAR tags provide a means of defining a vertically scrollable list display area that is made up of data coming from ISPF tables. One or more ISPF)MODEL

section statements are built to display the fields defined by the LSTVAR tags. The use of LSTVAR tags requires the use of the TBDISPL service in the application program.

The application must place valid data in the variable model line before the panel is displayed.

You can use the LINE attribute to format your table to display on multiple lines.

Restrictions

- You must code the LSTVAR tag within a LSTFLD tag. See [“LSTFLD \(List Field\)” on page 341](#) for a complete description of this tag.
- Only MODEL lines that are not blank fields are written in the panel body. Thus, if one LSTVAR entries specifies LINE=1 and another specifies LINE=3, but there are no entries for LINE 2, only two MODEL lines are created.

Processing

<i>Table 50. Tags you can code within an LSTVAR definition</i>			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is source file markup where the application panel contains a list field with five list columns and 2 variable model lines. The first three columns are defined as output-only, and are coded within the **Subscriber Name** list group. The **Number** column is an input/output column, and it is coded within the **Phone** list group. The last column is input-only, and it is coded within the **Approved** list group. This column requires input, so if it is not filled in, the error message **MSGG886** is displayed. The variable declarations and classes associated with the list columns are also shown.

Note: The variable model lines are shown in the formatted output to illustrate the formatting process. The application must provide valid values for these variables before the panel is displayed.

[Figure 135 on page 350](#) shows the formatted result of the application panel.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=namecls TYPE='char 15'>
<VARCLASS NAME=midcls TYPE='char 1'>
<VARCLASS NAME=phoncls TYPE='char 12'>
<VARCLASS NAME=appcls TYPE='char 1'>
  <XLATL FORMAT=upper>
</XLATL>
<CHECKL>
  <CHECKI TYPE=values PARM1=EQ PARM2='Y N'>
</CHECKL>

<VARLIST>
  <VARDCL NAME=xfname VARCLASS=namecls>
  <VARDCL NAME=xlname VARCLASS=namecls>
  <VARDCL NAME=xmid VARCLASS=midcls>
  <VARDCL NAME=xphone VARCLASS=phoncls>
  <VARDCL NAME=xapp VARCLASS=appcls>
</VARLIST>

<PANEL NAME=lstvar KEYLIST=keyltbl>Subscriber List
<TOPINST>Enter phone number, if missing,
(format - nnn-xxx-nnnn) and approved
indicator (y or n) for each person.
<AREA>
  <LSTFLD>
    <LSTVAR datavar=xmodelv1>Variable model line at top
    <LSTGRP HEADLINE=yes>Subscriber Name
      <LSTCOL DATAVAR=xfname USAGE=out line=2 COLWIDTH=15>First Name
      <LSTCOL DATAVAR=xlname USAGE=out line=2 COLWIDTH=15>Last Name
      <LSTCOL DATAVAR=xmid USAGE=out line=2 COLWIDTH=1>MI </LSTGRP>
    <LSTGRP>Phone
      <LSTCOL DATAVAR=xphone line=2 COLWIDTH=12>Number
    </LSTGRP>
    <LSTGRP>Approved
      <LSTCOL DATAVAR=xapp USAGE=in line=2 REQUIRED=yes
        COLWIDTH=1 MSG=msgf886>(Y or N)
    </LSTGRP>
    <LSTVAR datavar=xmodelv2 line=3>Variable model line at bottom
  </LSTFLD>
</AREA>
<CMDAREA>
</PANEL>

```

Subscriber List

Enter phone number, if missing, (format - nnn-xxx-nnnn)&cont.
and approved
indicator (y or n) for each person.

```

----- Subscriber Name ----- Phone
Approved
Variable model line at top
First Name      Last Name      MI   Number      (Y or N)
Variable model line at bottom
&XMODELV1
-----
&XMODELV2

```

```

Command ==> -----
F1=Help      F2=Split      F3=Exit      F7=Backward  F8=Forward
F9=Swap      F12=Cancel

```

Figure 135. List variable

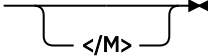
M (Mnemonic)

The M tag defines a single character to be used as a mnemonic selection for action bar choices.

Note: The M tag is accepted on pull-down choices in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

Syntax

➤ <M> — mnemonic-character — ➤



Parameters

mnemonic-character

The single-byte character following the mnemonic start tag specifies the mnemonic for the action bar choice. The mnemonic-character must be a single-byte alphabetic or numeric character; double-byte characters are not allowed.

If you want the mnemonic to be a character that is not part of the normal choice text, follow the choice text with the mnemonic character specified within parenthesis. This convention is particularly useful when you have a large number of choices, which makes it difficult to choose a unique mnemonic for each choice. For example, if you had the action bar choice Add, and the characters A and d were already used on other choices in the same action bar, you could choose another character for your mnemonic:

```
<abc>Add(<m>B)
```

In this case B becomes the mnemonic for Add.

Comments

Unless you have specified MNEMGEN=NO on the AB tag, the conversion utility automatically selects a mnemonic character for each action bar choice for SBCS conversions. The character selected as the mnemonic is the first alphabetic or numeric character in the choice description not previously used as a mnemonic for that set of choices.

Restrictions

When the conversion utility automatically generates mnemonics, the M tag selection is processed first, and if the specified mnemonic is valid, the automatic mnemonic generation is not used for that choice. If the specified mnemonic character is invalid, or a duplicate of a previously used mnemonic character (either specified or automatically selected), a message is issued and an attempt is made to select a different mnemonic character.

When processing DBCS conversions or when MNEMGEN=NO is coded on the AB tag, automatic mnemonic character selection is disabled and mnemonic characters are only specified by the M tag. The use of mnemonics should be consistent for all choices in an action bar:

- Code the M tag within the text following the ABC tags.
- Each mnemonic chosen must be unique. The conversion utility issues a message and discards duplicate mnemonics.
- If mnemonics are used for any action bar choice, they should be used for all of the choices. The conversion utility issues a message if any choice in a group does not have a mnemonic.

Processing

None.

Examples

Here is an example where all of the action bar choices have been coded to show the use of the M tag.

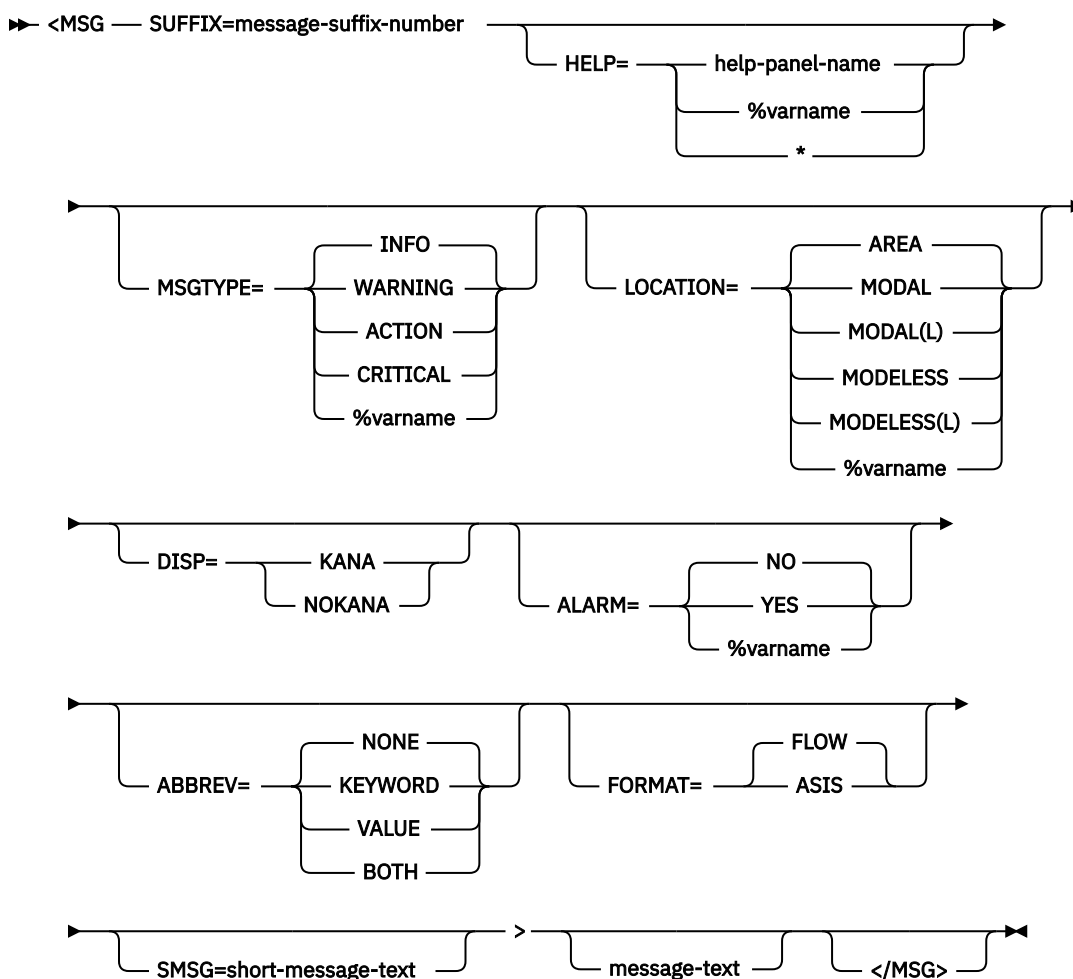
```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 sysem>
  <!entity sampbody system>)>
&sampvar1;

<PANEL NAME=m1 KEYLIST=keylxml>Library Card Registration
<AB>
<ABC><M>File
  <PDC>Add Entry
    <ACTION RUN=add>
  <PDC>Delete Entry
    <ACTION RUN=delete>
  <PDC>Update Entry
    <ACTION RUN=update>
  <PDC>Exit
    <ACTION RUN=exit>
<ABC><M>Search
  <PDC CHECKVAR=whchsrch MATCH=1>Search on name
    <ACTION SETVAR=whchsrch VALUE=1>
    <ACTION RUN=search>
  <PDC CHECKVAR=whchsrch MATCH=2>Search on card number
    <ACTION SETVAR=whchsrch VALUE=2>
    <ACTION RUN=search>
<ABC><M>Help
  <PDC>Extended Help...
    <ACTION RUN=exhelp>
  <PDC>Keys Help...
    <ACTION RUN=keyshelp>
</AB>
&sampbody;
</PANEL>
```

MSG (Message)

The MSG tag defines a message within a message member.

Syntax



Parameters

SUFFIX=message-suffix-number

This attribute specifies the suffix of the message. The suffix consists of either 1 numeric character (0-9) or a numeric character (0-9) and an optional alpha suffix character as defined for ISPF messages, which is added to the MSGMBR *message-member-name* to form the ISPF message ID.

Each *message-suffix-number* within a message member must be unique. Attempts to define duplicate suffixes result in a warning message and the duplicate MSG is ignored.

HELP=help-panel-name | %varname | *

Specifies the name of the help panel that is associated with this message and that is displayed if the user requests help for the message.

If you specify a help panel, ISPD TLC generates ".HELP=help-panel-name" (or ".HELP=&varname" or ".HELP=*") in the ISPF message ID definition. If you don't specify a help panel, no help is available for the message.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

MSGTYPE=INFO | WARNING | ACTION | CRITICAL | %varname

This attribute specifies the severity of the message. ISPF displays INFO messages without an alarm. ISPF displays WARNING, ACTION, and CRITICAL messages with an alarm.

ACTION and CRITICAL message types are used to identify the most severe errors. This level of error usually requires a user response. ISPF always displays CRITICAL messages in a pop-up. ACTION messages are displayed based on the value of the LOCATION attribute.

The %varname value specifies that the value INFO, WARNING, ACTION, or CRITICAL is provided in the named variable by the application before issuing the message.

The conversion utility changes INFO to .TYPE=NOTIFY when formatting the message member.

ISPF recognizes message types (.TYPE=) of NOTIFY, WARNING, ACTION, and CRITICAL. ISPF uses the TYPE value specified in conjunction with the value of .WINDOW to determine the display characteristics of the message. The .WINDOW value is generated from the value specified for the LOCATION attribute. For more information on ISPF messages, refer to *z/OS ISPF Dialog Developer's Guide and Reference*.

LOCATION=AREA | MODAL | MODAL(L) | MODELESS | MODELESS(L) | %varname

This attribute specifies how the message is displayed.

LOCATION=AREA (the default) specifies that the message is to appear in the panel message area. However, if the text of the message exceeds the length of the panel message area, ISPF displays the message in a pop-up.

LOCATION=MODAL specifies that the message is to appear in a pop-up which requires a user response. The conversion utility generates .WINDOW=RESP in the ISPF message definition.

LOCATION=MODAL(L) specifies that the long message is to appear in a pop-up which requires a user response. The conversion utility generates .WINDOW=LRESP in the ISPF message definition.

LOCATION=MODELESS specifies that the message is to appear in a pop-up which does not require a user response. The conversion utility generates .WINDOW=NORESP in the ISPF message definition.

LOCATION=MODELESS(L) specifies that the long message is to appear in a pop-up which does not require a user response. The conversion utility generates .WINDOW=LNORESP in the ISPF message definition.

LOCATION=%varname specifies that the value AREA, MODAL, or MODELESS is provided in the named variable by the application before issuing the message. The conversion utility generates .WINDOW=&VARNAME in the ISPF message definition.

DISP=KANA | NOKANA

This attribute specifies the addition of either the KANA or NOKANA keyword to the message control information.

ALARM=NO | YES | %varname

This attribute controls the use of the alarm when the message is displayed.

ALARM=%varname specifies that the value YES or NO is provided in the named variable by the application before issuing the message.

ABBREV=NONE | KEYWORD | VALUE | BOTH

This attribute specifies the format of the message control information. You may abbreviate the message control keyword, the message control keyword value, or both.

FORMAT=FLOW | ASIS

This attribute specifies the formatting of the *message-text*.

The default of **FLOW** means to flow the message text continuously within the WIDTH of the MSGMBR.

When FORMAT=ASIS, the generated message preserves embedded blanks, but drops leading or trailing blanks.

SMSG=short-message-text

You can provide a short message of up to 24 bytes which ISPF displays in the short message area of the panel.

The VARSUB tag is not supported within the *short-message-text*. If a substitution variable is required, you may code "&variable" to place the variable name in the message. A *short-message-text*

consisting of more than one word must be enclosed within quotation marks (" "). If the *short-message-text* contains a single apostrophe ('), the conversion utility generates double apostrophes as it does for *message-text*, as described for the next parameter (**message-text**).

The short message is not recommended by the CUA Architecture definition.

A short message cannot be created unless the *message-text* is also provided.

message-text

This is the text of the message. The *message-text* is placed in the long-message area of a message file. The *message-text* is limited to 512 characters. The conversion utility truncates all *message-text* after 512 characters and issues a warning message. If no *message-text* is coded, then no message is generated.

Several characters within the long message area have a special meaning to ISPF. If you use the apostrophe within *message-text*, the conversion utility generates double apostrophes so the single apostrophe is displayed when ISPF issues the message. If you use the ampersand (&) within the long message, it must be coded as "&" followed by a blank or semicolon to be interpreted as a literal ampersand character (through ENTITY substitution).

For ISPF substitution variables, you should code the VARSUB tag. ISPF does not perform output translation (specified in the associated VARCLASS tag) on ISPF runtime substitution variables.

See *z/OS ISPF Dialog Developer's Guide and Reference* for a description of the syntax rules you should use for defining consistent messages.

Comments

The MSG tag defines a message within a message member. Each MSG definition within a message member must have a unique *message-suffix-number*.

Restrictions

- You must code the MSG tag within a MSGMBR definition. See [“MSGMBR \(Message Member\)” on page 356](#) for a complete description of this tag.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).

Processing

Table 51. Tags you can code within a MSG definition			
Tag	Reference	Usage	Required
VARSUB	“VARSUB (Variable Substitution)” on page 452	Multiple	No

Examples

Here is markup that contains the message member MSGG88, which contains nine MSG definitions. The text of messages MSGG883 and MSGG888 contain variable substitutions. [Figure 136 on page 356](#) shows the generated ISPF message member.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=msgcls TYPE='char 20'>
<VARLIST>
  <VARDCL NAME=phoneno VARCLASS=msgcls>
  <VARDCL NAME=cnum VARCLASS=msgcls>
</VARLIST>

<MSGMBR NAME=msgg88>
  <MSG SUFFIX=1 disp=kana abbrev=keyword>Name must be alphabetic.
  <MSG SUFFIX=2 disp=nokana abbrev=value>Enter only number of days.
  <MSG SUFFIX=3 MSGTYPE=critical>The only rooms we have available
    are either SINGLE or DOUBLE. Please call the manager of the hotel
    who will arrange equivalent lodging at another
    hotel in the area. This is our mistake, and we will, of course,
    pick up the bill. Please call collect <VARSUB VAR=phoneno>.
  <MSG SUFFIX=4 MSGTYPE=action LOCATION=modal abbrev=both>
    Please enter either BIGCHARGE, V I S T A, EZCARD, CHECK, or CASH.
  <MSG SUFFIX=5 MSGTYPE=warning LOCATION=modeless>Please enter your name.
  <MSG SUFFIX=6>Please enter Y or N.
  <MSG SUFFIX=7>Card number is a seven-digit number.
  <MSG SUFFIX=8 MSGTYPE=warning>The card number you entered,
    <VARSUB VAR=cnum> is not valid.
  <MSG SUFFIX=9>Message '9' contains embedded quotes.
</MSGMBR>

```

```

MSGG881 .T=NOTIFY KANA
'Name must be alphabetic.'
MSGG882 .TYPE=N NOKANA
'Enter only number of days.'
MSGG883 .TYPE=CRITICAL
'The only rooms we have available are either SINGLE or DOUBLE. Please call th' +
'e manager of the hotel who will arrange equivalent lodging at another hotel ' +
'in the area. This is our mistake, and we will, of course, pick up the bill. ' +
'Please call collect &PHONENO.'
MSGG884 .T=A .W=R
'Please enter either BIGCHARGE, V I S T A, EZCARD, CHECK, or CASH.'
MSGG885 .TYPE=WARNING .WINDOW=NORESP
'Please enter your name.'
MSGG886 .TYPE=NOTIFY
'Please enter Y or N.'
MSGG887 .TYPE=NOTIFY
'Card number is a seven-digit number.'
MSGG888 .TYPE=WARNING .ALARM=YES
'The card number you entered, &CNUM is not valid.'
MSGG889 .TYPE=NOTIFY
'Message '9' contains embedded quotes.'

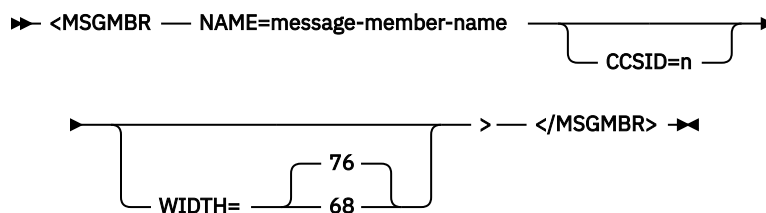
```

Figure 136. Messages

MSGMBR (Message Member)

The MSGMBR tag defines a message member.

Syntax



Parameters

NAME=message-member-name

This specifies the name of the message member, which also serves as the prefix for all identifiers of messages within the member.

The *message-member-name* can be specified as a 3-7 character name, conforming to ISPF message member standard naming convention. The last two positions must be numeric. The preceding characters can be A-Z, a-z, or #, \$, @.

Lowercase characters are translated to their uppercase equivalents.

If you specify NAME=*, the *message-member-name* is set to the input DTL source member name. If multiple dialog element definitions have been combined within a single source file, then this notation should be used for only one dialog element definition within the file. See “Dialog elements” on page 5 for a description of dialog element types created by the conversion utility.

The *message-member-name* is also used to build the name used for storing messages. For example, if NAME=MSGA12, the default name used to store the message members is userid.MSGS(MSGA12). This can be changed by specifying a message file on the conversion utility invocation panel. See Chapter 10, “Using the conversion utility,” on page 151 for more information about ISPD TLC syntax.

for information about allocating a message library at run time, refer to the [z/OS ISPF User's Guide Vol I](#).

CCSID=n

CCSID specifies the coded-character-set identifier as defined by the Character Data Representation Architecture. CCSID should be entered as a five-position numeric value. For more information on using the CCSID attribute, refer to the [z/OS ISPF Dialog Developer's Guide and Reference](#).

WIDTH=76 | 68

This attribute specifies the width of the formatted messages. When WIDTH=68, the resulting messages are formatted entirely within a normal Edit or View screen.

Comments

The MSGMBR tag defines a message member. You can code multiple message members for a single application.

The *message-member-name* is an explicit part of the identifier for messages coded in the message member. Each message member contains multiple messages. You use the MSG tag to define\ messages within a message member.

Restrictions

- The MSGMBR tag requires an end tag.
- You cannot code the MSGMBR tag within any other tag definition.

Processing

Table 52. Tags you can code within an MSGMBR definition			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
MSG	“MSG (Message)” on page 352	Multiple	Yes

Examples

Here is markup that defines the message member *MSGM88*, which contains nine MSG definitions. [Figure 137 on page 358](#) shows the generated ISPF message member.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=msgcls TYPE='char 20'>
<VARLIST>
  <VARDCL NAME=phoneno VARCLASS=msgcls>
  <VARDCL NAME=cnum VARCLASS=msgcls>
</VARLIST>

<MSGMBR NAME=msgm88>
  <MSG SUFFIX=1>Name must be alphabetic.
  <MSG SUFFIX=2>Enter only number of days.
  <MSG SUFFIX=3 MSGTYPE=critical>The only rooms we have available
  are either SINGLE or DOUBLE. Please call the manager of the hotel
  who will arrange equivalent lodging at another
  hotel in the area. This is our mistake, and we will, of course,
  pick up the bill. Please call collect <VARSUB VAR=phoneno>.
  <MSG SUFFIX=4 MSGTYPE=action LOCATION=modal>Please enter either
  BIGCHARGE, V I S T A, EZCARD, CHECK, or CASH.
  <MSG SUFFIX=5 MSGTYPE=warning LOCATION=modeless>Please enter your name.
  <MSG SUFFIX=6>Please enter Y or N.
  <MSG SUFFIX=7>Card number is a seven-digit number.
  <MSG SUFFIX=8 MSGTYPE=warning>The card number you
  entered, <VARSUB VAR=cnum> is not valid.
  <MSG SUFFIX=9>Message '9' contains embedded quotes.
</MSGMBR>
```

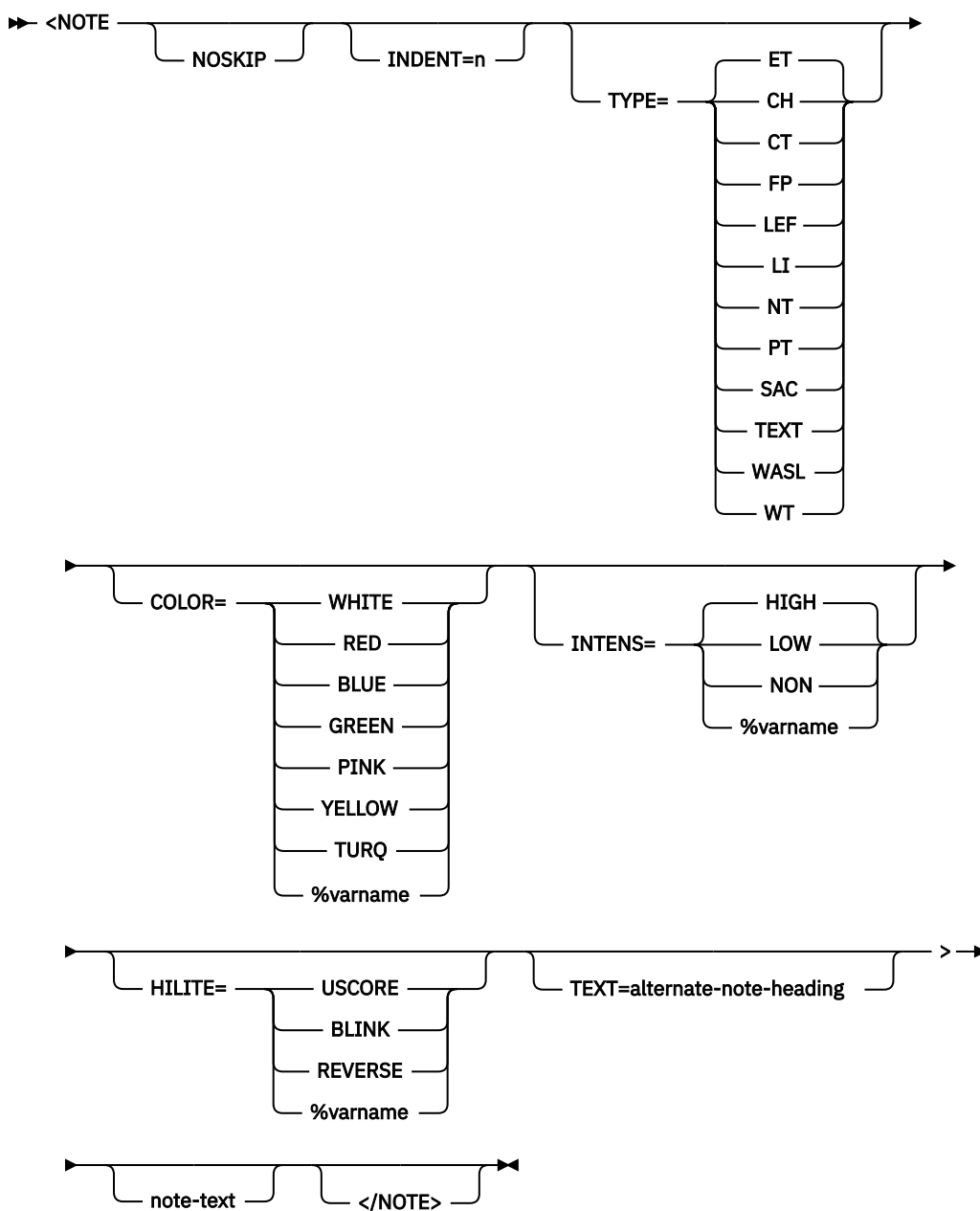
```
MSGM881 .TYPE=NOTIFY
'Name must be alphabetic.'
MSGM882 .TYPE=NOTIFY
'Enter only number of days.'
MSGM883 .TYPE=CRITICAL
'The only rooms we have available are either SINGLE or DOUBLE. Please call th' +
'e manager of the hotel who will arrange equivalent lodging at another hotel ' +
'in the area. This is our mistake, and we will, of course, pick up the bill. ' +
'Please call collect &PHONENO.'
MSGM884 .TYPE=ACTION .WINDOW=RESP
'Please enter either BIGCHARGE, V I S T A, EZCARD, CHECK, or CASH.'
MSGM885 .TYPE=WARNING .WINDOW=NORESP
'Please enter your name.'
MSGM886 .TYPE=NOTIFY
'Please enter Y or N.'
MSGM887 .TYPE=NOTIFY
'Card number is a seven-digit number.'
MSGM888 .TYPE=WARNING
'The card number you entered, &CNUM is not valid.'
MSGM889 .TYPE=NOTIFY
'Message '9'' contains embedded quotes.'
```

Figure 137. Message member

NOTE (Note)

The NOTE tag defines a single-paragraph note within an information region.

Syntax



Parameters

NOSKIP

This attribute causes the note to be formatted without creating a blank line before the note.

INDENT=n

This attribute specifies that the note be indented from the current left margin.

TYPE= **ET** | **CH** | **CT** | **FP** | **LEF** | **LI** | **NT** | **PT** | **SAC** | **TEXT** | **WASL** | **WT**

This attribute defines the attribute type to be applied to the note heading. Using a CUA attribute causes the text to appear in the associated color.

When TYPE=TEXT, a non-CUA attribute is generated and you can specify the color, intensity, and highlighting with the COLOR, INTENS, and HILITE attributes. These attributes are not valid for CUA types.

NOTE

COLOR= WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENS= HIGH | LOW | NON | %varname

This attribute defines the intensity of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

HILITE= USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute for the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

TEXT=alternate-note-heading

This attribute provides a text string to replace the standard "Note:" heading.

note-text

This is the text of the note.

Comments

The NOTE tag defines a single-paragraph note within an information region. You can code the NOTE tag anywhere within an INFO tag.

The text of the note formats as an implied paragraph, at the current left margin. The text "Note:" (or its translated equivalent), or the alternate note heading, begins the paragraph and is aligned with the text of a list item when you use it within a list.

Restrictions

- You must code the NOTE tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- You cannot nest a NOTE tag within another NOTE definition.

Processing

Table 53. Tags you can code within a NOTE definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains a note. [Figure 138 on page 361](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=note DEPTH=20>Book / Periodical Search Help
<AREA>
<INFO>
  <P>This entry screen allows you to locate a desired
  book or periodical by entering the title in the entry field.
  <NOTE>If the item you are trying to locate is not
  in stock and you would like to reserve it, please see the
  librarian at the front desk.
</INFO>
</AREA>
</HELP>
```


Book / Periodical Search Help

This entry screen allows you to locate a desired book or periodical by entering the title in the entry field.

Note: If the item you are trying to locate is not in stock and you would like to reserve it, please see the librarian at the front desk.

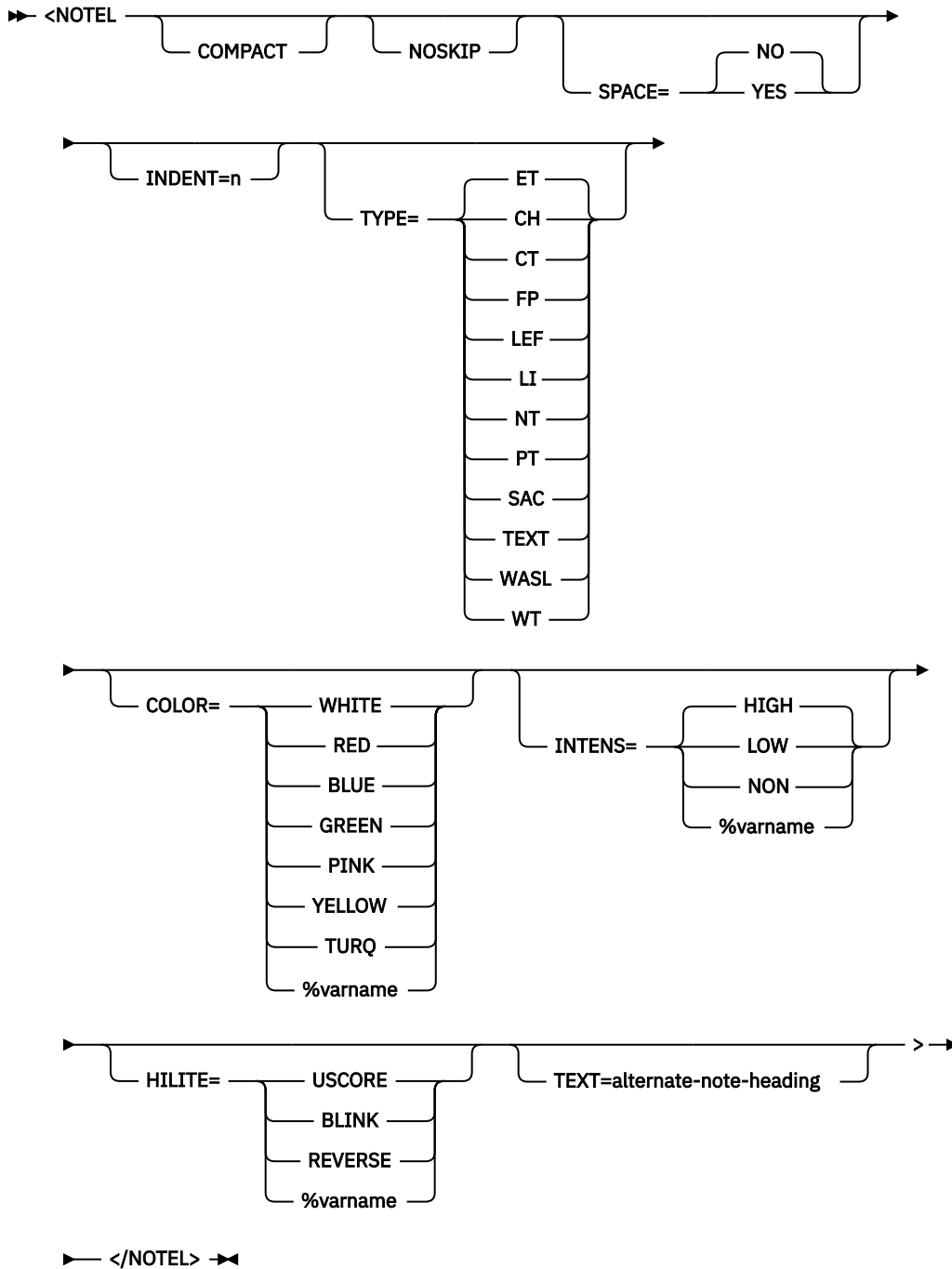
F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

Figure 138. Note

NOTEL (Note List)

The NOTEL tag defines a list of notes within an information region.

Syntax



Parameters

COMPACT

This attribute causes the list to be formatted without a blank line between the list items.

NOSKIP

This attribute causes the list to format without creating a blank line before the first line of the list.

SPACE=NO | YES

The SPACE attribute controls the indentation space for the list item. When the SPACE attribute is not specified on the LI tag, the SPACE attribute from the NOTEL tag is used to set the indentation space for the nested LI tag *item-text*.

When SPACE=YES, the indentation is set to 3 spaces.

When SPACE=NO (or SPACE is not specified), the indentation is set to 4 spaces.

The SPACE attribute can be used to control the alignment of list items when the first word of some list items is a DBCS word preceded by a shift-out character and the first word of other list items is an SBCS word.

INDENT=n

This attribute specifies that the note list be indented from the current left margin.

TYPE= ET | CH | CT | FP | LEF | LI | NT | PT | SAC | TEXT | WASL | WT

This attribute defines the attribute type to be applied to the note heading. Using a CUA attribute causes the text to appear in the associated color.

When TYPE=TEXT, a non-CUA attribute is generated and you can specify the color, intensity, and highlighting with the COLOR, INTENS, and HILITE attributes. These attributes are not valid for CUA types.

COLOR= WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENS= HIGH | LOW | NON | %varname

This attribute defines the intensity of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

HILITE= USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

TEXT=alternate-note-heading

This attribute provides a text string to replace the standard "Notes:" heading.

Comments

The NOTEL tag defines a numbered list of notes. You can code the NOTEL tag anywhere within an INFO tag.

The first line of the note list formats with the word "Notes:" (or its translated equivalent) or the alternate-note-heading.

Use the LI tag to denote each list item. See [“LI \(List Item\)” on page 325](#) for more information on the LI tag.

Restrictions

- You must code the NOTEL tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- You cannot nest a NOTEL tag within a NOTEL definition.

Processing

Table 54. Tags you can code within a NOTEL definition			
Tag	Reference	Usage	Required
LI	“LI (List Item)” on page 325	Multiple	No
LP	“LP (List Part)” on page 330	Multiple	No

Examples

Here is help panel markup that contains a multiple notes. Notice the numbered format for the content of the notes, which is different from the format generated with the NOTE or NT tag. A P tag is nested within the NOTEL definition to provide an additional paragraph of note text. [Figure 139 on page 364](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=notel DEPTH=20>Book / Periodical Search Help
<AREA>
<INFO>
  <P>This entry screen allows you to locate a desired
  book or periodical by entering the title in the entry field.
  <NOTEL>
    <LI>If the item you are trying to locate is not
      in stock and you would like to reserve it, please see the
      librarian at the front desk.
    <LI>If the librarian is not there, please do not yell for help.
    <P>This is a library!
  </NOTEL>
</INFO>
</AREA>
</HELP>
```

Book / Periodical Search Help

This entry screen allows you to locate a desired book or periodical by entering the title in the entry field.

Notes:

1. If the item you are trying to locate is not in stock and you would like to reserve it, please see the librarian at the front desk.
2. If the librarian is not there, please do not yell for help.

This is a library!

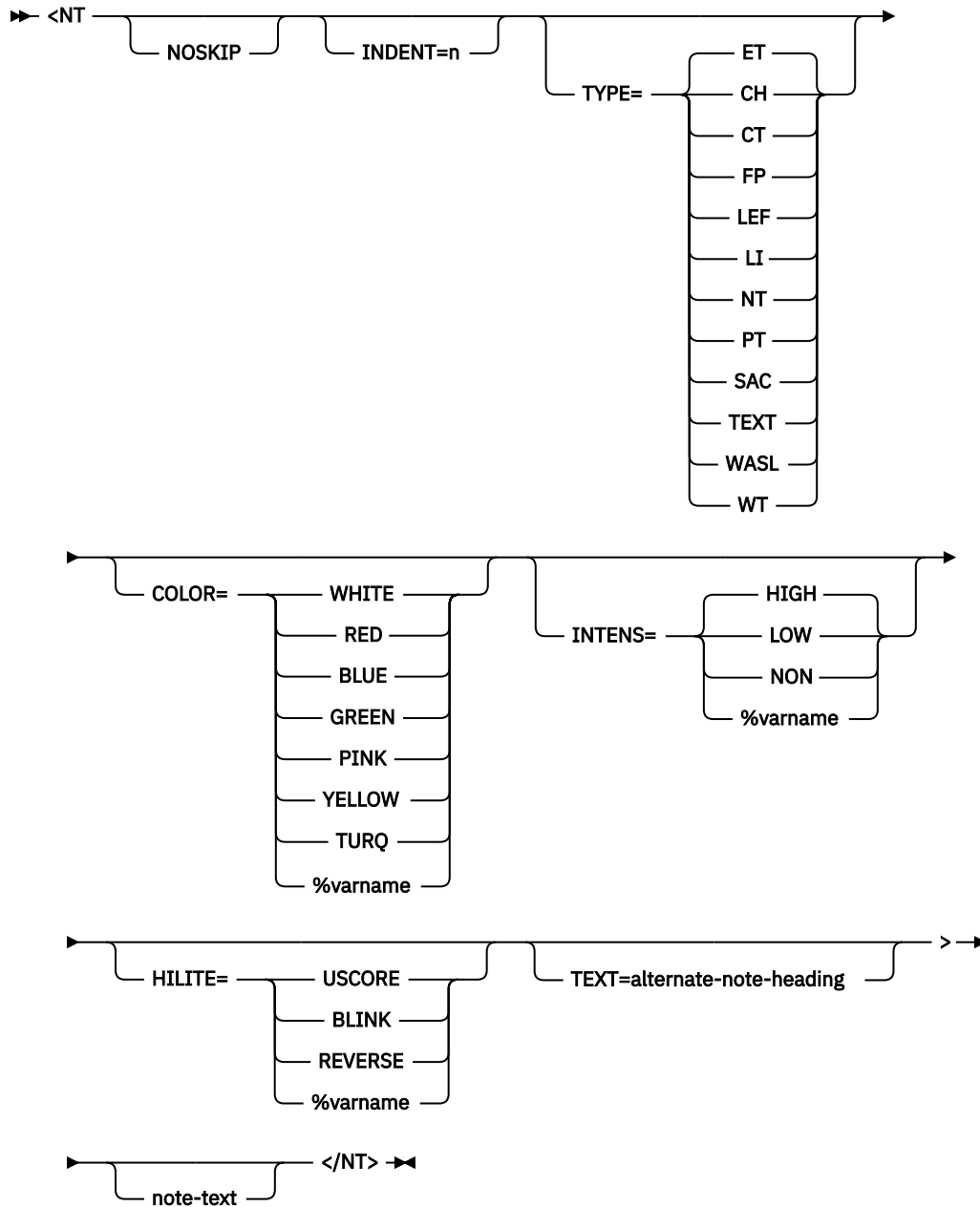
F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

Figure 139. NOTEL

NT (Note)

The NT tag defines a single- or multiple-paragraph note within an information region.

Syntax



Parameters

NOSKIP

This attribute causes the note to be formatted without creating a blank line before the note.

INDENT=n

This attribute specifies that the note be indented from the current left margin.

TYPE= **ET** | **CH** | **CT** | **FP** | **LEF** | **LI** | **NT** | **PT** | **SAC** | **TEXT** | **WASL** | **WT**

This attribute defines the attribute type to be applied to the note heading. Using a CUA attribute causes the text to appear in the associated color.

When `TYPE=TEXT`, a non-CUA attribute is generated and you can specify the color, intensity, and highlighting with the `COLOR`, `INTENS`, and `HILITE` attributes. These attributes are not valid for CUA types.

COLOR= WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute specifies the color of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

INTENS= HIGH | LOW | NON | %varname

This attribute defines the intensity of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

HILITE= USCORE | BLINK | REVERSE | %varname

This attribute specifies the extended highlighting attribute of the note heading. You can define this attribute as a variable name preceded by a percent (%) sign.

TEXT=alternate-note-heading

This attribute provides a text string to replace the standard "Note:" heading.

note-text

This is the text of the note. You can use the P tag to code additional paragraphs of text.

Comments

The NT tag defines a single- or multiple-paragraph note within an information region. You can code the NT tag anywhere within an INFO definition.

The text of the note formats as an indented block. The block of text is indented seven spaces from the current left margin. The text "Note:" (or its translated equivalent), or the alternate note heading, begins the paragraph. The note aligns with the text of a list item when you code it within a list.

Restrictions

- The NT tag requires an end tag.
- You must code the NT tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- You can nest text tags such as paragraphs and lists within a note, but you cannot nest NT and NOTE tags.

Processing

<i>Table 55. Tags you can code within an NT definition</i>			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No

Table 55. Tags you can code within an NT definition (continued)

Tag	Reference	Usage	Required
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a multiple-paragraph note. Notice the indented format for the content of the note, which is different from the format generated with the NOTE tag. A P tag is nested within the NT definition to provide an additional paragraph of note text. [Figure 140 on page 367](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=nt DEPTH=20>Book / Periodical Search Help
<AREA>
<INFO>
  <P>This entry screen allows you to locate a desired
  book or periodical by entering the title in the entry field.
  <NT>If the item you are trying to locate is not
  in stock and you would like to reserve it, please see the
  librarian at the front desk.
  <P>If the librarian is not there, please do not yell for help.
  This is a library!
  </NT>
</INFO>
</AREA>
</HELP>
```

Book / Periodical Search Help

This entry screen allows you to locate a desired book or periodical by entering the title in the entry field.

Note: If the item you are trying to locate is not in stock and you would like to reserve it, please see the librarian at the front desk.

If the librarian is not there, please do not yell for help. This is a library!

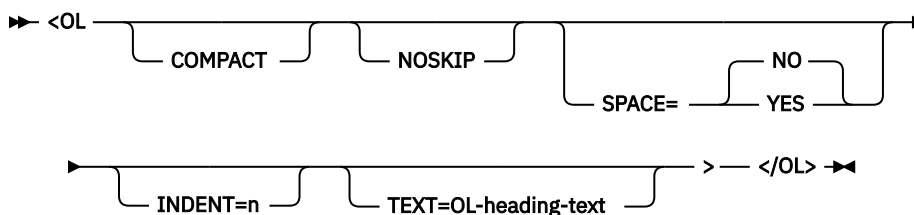
```
F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

Figure 140. NT

OL (Ordered List)

The OL tag defines an ordered list of items within an information region.

Syntax



Parameters

COMPACT

This attribute causes the list to be formatted without a blank line between the list items.

NOSKIP

This attribute causes the list to format without creating a blank line before the first line of the list.

SPACE=NO | YES

The SPACE attribute controls the indentation space for the list item. When the SPACE attribute is not specified on the LI tag, the SPACE attribute from the OL tag is used to set the indentation space for the nested LI tag *item-text*.

When SPACE=YES, the indentation is set to 3 spaces. When SPACE=NO (or SPACE is not specified), the indentation is set to 4 spaces.

The SPACE attribute can be used to control the alignment of list items when the first word of some list items is a DBCS word preceded by a shift-out

INDENT=n

This attribute specifies that the list be indented from the current left margin.

TEXT=OL-heading-text

This attribute causes the list to format with a heading line containing the *OL-heading-text*.

Comments

The OL tag defines an ordered list of items within an information region. You use ordered lists to indicate a set of sequential items or steps. You can code the OL tag anywhere within an information region.

Ordered lists are formatted as indented lists, with sequential numbers or letters at the left margin of the list items. Nested lists (lists embedded within other lists) indent four spaces to the right of the left margin of the list that contains them.

Note: The SPACE attribute does not affect the indentation of nested lists.

The conversion utility adds a blank line before the first item in the list.

Sequential numbers or letters, depending on the nesting level of the ordered list precede the list items. The levels are:

1. Level 1: 1., 2., 3., . . .
2. Level 2: a., b., c., . . .
3. Level 3: 1), 2), 3), . . .
4. Level 4: a), b), c), . . .

Any additional levels repeat the sequence from level 1.

Panels formatted with the DBCS option use uppercase alphabetic characters for the even-numbered nesting levels.

Use the LI tag to denote each list item. See [“LI \(List Item\)”](#) on page 325 for more information on the LI tag.

Restrictions

- The OL tag requires an end tag.
- You must code the OL tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 56. Tags you can code within an OL definition			
Tag	Reference	Usage	Required
LI	“LI (List Item)” on page 325	Multiple	No
LP	“LP (List Part)” on page 330	Multiple	No

Examples

Here is help panel markup that contains two ordered lists and a paragraph. The second ordered list and the paragraph are nested within the first list. [Figure 141 on page 370](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=ol DEPTH=22 WIDTH=60>Widget Assembly Help
<AREA>
<INFO>
  <P>To assemble your new Widget, you should:
  <OL>
    <LI>Attach the gizmo flexure component to the
    main steering mechanism of the doohickey.
    <OL COMPACT>
      <LI>If slot A fits snugly on retaining
      pin B, proceed to step 2.
      <LI>If slot A does not fit snugly on
      retaining pin B, throw the Widget away
      and buy a new one.
    </OL>
    <LI>Use a screwdriver to turn the power drive unit on.
    <LI>Stand back and watch the fun!
    <P>Wake up the kids and call the neighbors, they won't
    want to miss it!
  </OL>
</INFO>
</AREA>
</HELP>
```

Widget Assembly Help

To assemble your new Widget, you should:

1. Attach the gizmo flexure component to the main steering mechanism of the doohickey.
 - a. If slot A fits snugly on retaining pin B, proceed to step 2.
 - b. If slot A does not fit snugly on retaining pin B, throw the Widget away and buy a new one.
2. Use a screwdriver to turn the power drive unit on.
3. Stand back and watch the fun!

Wake up the kids and call the neighbors, they won't want to miss it!

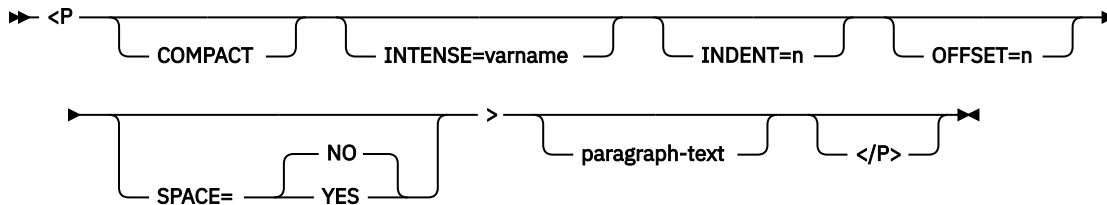
F1=Help F3=Exit F5=Exhelp F6=Keyshelp
 F7=PrvTopic F8=NxtTopic F10=PrvPage F11=NxtPage
 F12=Cancel

Figure 141. Ordered lists

P (Paragraph)

The P tag defines a paragraph of text within an information region.

Syntax



Parameters

COMPACT

This attribute causes the paragraph to format without a blank line before the paragraph.

INTENSE=varname

This attribute supplies a variable name that must contain a valid value for the INTENS keyword. The entire paragraph is controlled by this value. For example, if the variable contains the value NON, the paragraph is not visible.

INDENT=n

This attribute specifies that the paragraph be indented from the current left margin.

OFFSET=n

This attribute specifies that the formatted text following the first line of the paragraph should be indented an additional *n* bytes.

SPACE= NO | YES

This attribute is used when processing <P> tags coded within ENTITY definitions. When the ENTITY keyword SPACE is not specified, text following a paragraph tag within the ENTITY definition is processed as coded by default. This may result in unwanted spaces between words in the paragraph, which can be removed by specifying <p space=yes>.

paragraph-text

This is the text of the paragraph.

Comments

The P tag defines a paragraph of text within an information region. You can code the P tag anywhere within an INFO definition.

Each paragraph formats as an unindented block of text. A blank line is added before the paragraph unless the COMPACT attribute is specified.

Paragraphs within a list align with the text of the list item.

Restrictions

- You must code the P tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 57. Tags you can code within a P definition			
Tag	Reference	Usage	Required
ATTENTION	“ATTENTION (Attention)” on page 198	Single	No
CAUTION	“CAUTION (Caution)” on page 205	Single	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
WARNING	“WARNING (Warning)” on page 454	Single	No

Examples

Here is help panel markup that contains four paragraphs. The first three paragraphs are coded within an information region with a defined width of 40, so the text of the paragraphs is formatted according to this width. The last paragraph is coded within an information region with no defined width, so the paragraph text is formatted according to the width defined on the HELP tag. [Figure 142 on page 372](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=p DEPTH=22 WIDTH=60>P Tag Help
<AREA>
<INFO WIDTH=40>
  <P>Here's a paragraph.
  Lines are formatted to fill the width of the
  information region.
  <P>Here's another paragraph.
  Notice the line skip between the paragraphs.
  <P>Paragraphs are very versatile.
  You can use them within many other tags.
</INFO>
<INFO WIDTH=58>
  <P>The paragraphs above were formatted within an
  information region defined with a width of 40.
  This paragraph is formatted within the width specified
  for the panel.
</INFO>
</AREA>
</HELP>
```

P Tag Help

Here's a paragraph. Lines are formatted to fill the width of the information region.

Here's another paragraph. Notice the line skip between the paragraphs.

Paragraphs are very versatile. You can use them within many other tags.

The paragraphs above were formatted within an information region defined with a width of 40. This paragraph is formatted within the width specified for the panel.

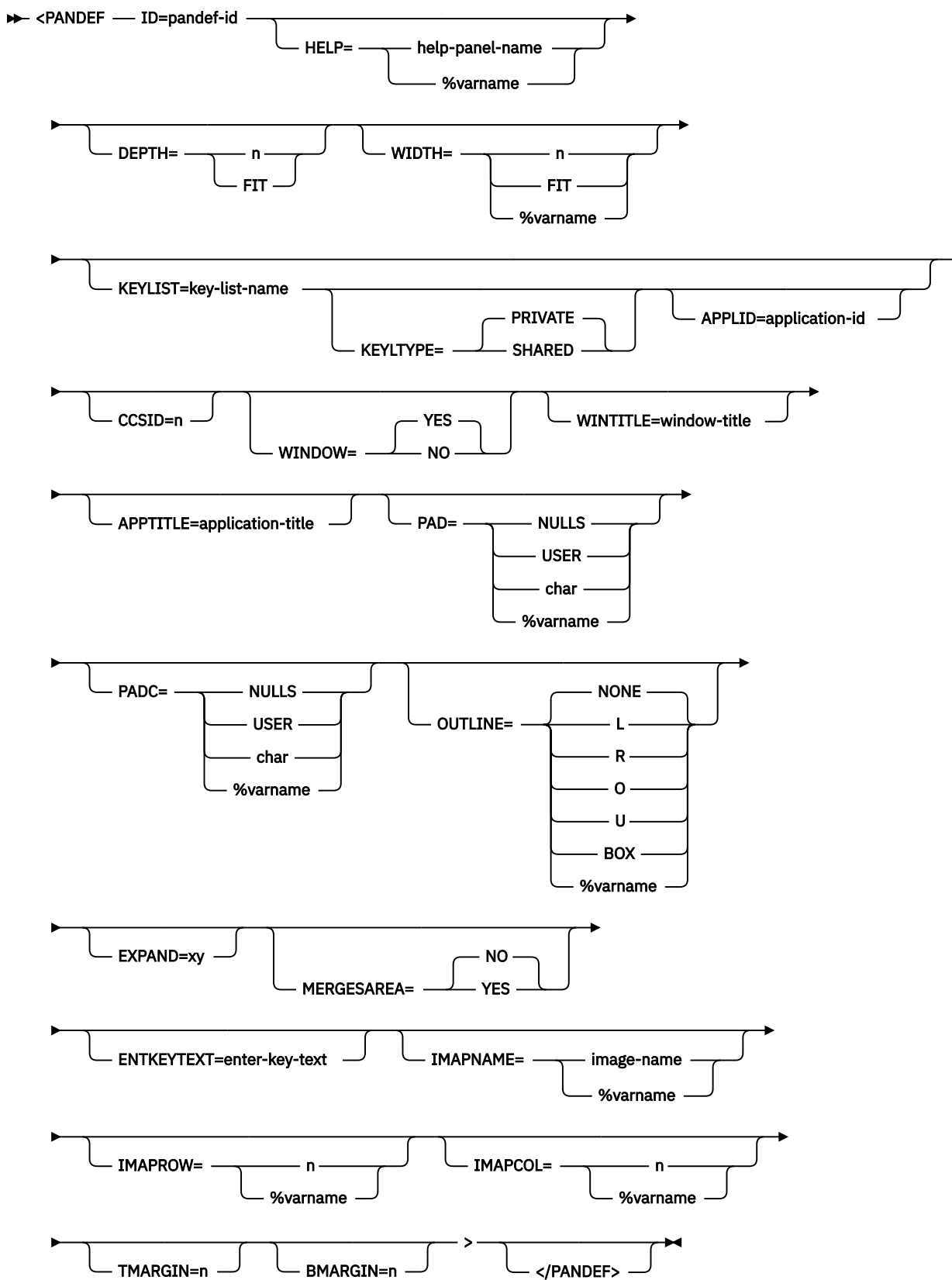
F1=Help	F3=Exit	F5=Exhelp	F6=Keyshelp
F7=PrvTopic	F8=NxtTopic	F10=PrvPage	F11=NxtPage
F12=Cancel			

Figure 142. Paragraphs

PANDEF (Panel Default)

The PANDEF tag defines default values for application panels.

Syntax



Parameters

ID=pandef-id

This attribute defines the identifier for the panel default definition. The *pandef-id* is the value you specify with the PANDEF attribute of PANEL tags that refer to the panel default.

The *pandef-id* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

HELP=help-panel-name | %varname

This attribute specifies the extended (panel help) help panel that displays when the user selects help on an application panel that specifies the panel default.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

Specification of the HELP attribute cause ISPD TLC to generate ".HELP=help-panel-name" (or ".HELP=&varname") in the)INIT section during panel generation.

ISPF displays this panel when the application user requests help and the cursor is not on a panel field that has its own field-level help specified. This help panel is also displayed when the user requests extended help.

DEPTH=n | FIT

This attribute specifies a default depth value for an application panel that refers to this panel default. See [“PANEL \(Panel\)”](#) on page 376, for more information.

WIDTH=n | FIT | %varname

This attribute specifies a default width value for an application panel that refers to this panel default. See [“PANEL \(Panel\)”](#) on page 376, for more information.

KEYLIST=key-list-name

This attribute specifies the name of a key mapping list associated with panels that refer to this panel default. See [“KEYL \(Key List\)”](#) on page 322 for more information.

KEYLTYPE= PRIVATE | SHARED

This attribute is used to add the SHARED keyword to the KEYLIST parameter of the)PANEL statement. For more information about the)PANEL statement, refer to the [z/OS ISPF Dialog Developer's Guide and Reference](#).

APPLID=application-id

This attribute is used to add the application ID to the)PANEL statement. The *application-id* overrides the KEYLAPPL invocation option value.

CCSID=n

This attribute specifies the default CCSID value for an application panel that refers to this panel default. See [“PANEL \(Panel\)”](#) on page 376 for more information.

WINDOW=YES | NO

The WINDOW attribute is used to control the generation of the WINDOW keyword on the panel)BODY section. The default is to create the WINDOW keyword. WINDOW=NO should be used when WIDTH=%varname is also used to create a panel.

WINTITLE=window-title

This attribute is used to add a title on the pop-up window border. The attribute value is placed in the ISPF ZWINTTL variable. The maximum length of the *window-title* text is the panel width minus 1.

APPTITLE=application-title

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

PADC= NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

EXPAND=xy

This attribute adds the EXPAND(xy) attribute to the)BODY section of the panel. If only one character is present, the second character is set to the same value. If the EXPAND attribute is present with no value specified, the conversion utility uses a character from the range of low-order hex values available for panel attributes. This removes an available character from possible use as a panel attribute and may cause panel formatting errors.

MERGESAREA= NO | YES

This attribute controls an additional formatting step for panels with a single scrollable area. If the entire contents of the scrollable area fit within a standard 24-line panel (allowing 2 lines for the function keys display), and no input or output fields are found in the panel body following the location of the scrollable area, the scrollable area content is moved into the panel body.

ENTKEYTEXT=enter-key-text

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPROW=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPCOL=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

TMARGIN=n

This attribute provides the number of blank lines to format at the top of the panel as a top margin.

BMARGIN=n

This attribute provides the number of blank lines to format at the bottom of the panel as a bottom margin.

Comments

The PANDEF tag defines default values for application panels.

PANEL tags refer to the panel default by specifying the *pandef-id* definition as the PANDEF attribute value. When a PANEL tag refers to a panel default, the values specified by the associated PANDEF tag are used for the panel unless overridden by values specified in the PANEL tag definition.

The PANEL tag can override any of the PANDEF values by specifying that value within its own definition. Thus, it is possible for a PANEL tag to select certain default values from the panel default and override others.

See “[PANEL \(Panel\)](#)” on page 376 for more information.

You can code multiple panel defaults for an application. Each panel default should have a unique *pandef-id*.

Restrictions

- You cannot code the PANDEF tag within any other tag definition.

PANEL

- You must code the PANDEF tag before you code any PANEL tag that refers to it.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).
- EXPAND can operate only when there are no trailing attributes on the line to be expanded. Panel lines formatted as part of a horizontal region require the use of attributes for field alignment. Therefore, the EXPAND feature is functional only for panel sections built within a vertical (or default) region that is not part of any horizontal region.

Processing

None.

Examples

Here is source file markup that contains two panel default definitions. The application panels *panel1* and *panel2* both refer to the panel default *pandef1*. The panel *panel1* uses all of the defined default values and *panel2* uses only the default DEPTH and WIDTH values, and overrides the default HELP and KEYLIST values by specifying those values in the PANEL definition. The third application panel, *panel3* refers to all of the default values specified in the panel default *pandef2*.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 sysem>
  <!entity sampbody system>)>
&sampvar1;

<PANDEF ID=panldef1 DEPTH=20 WIDTH=76 HELP=helpaaa KEYLIST=keylxml>

<PANDEF ID=panldef2 DEPTH=22 WIDTH=70 HELP=morehlp>

<PANEL NAME=pandef1 PANDEF=panldef1>First Panel
&sampbody;
</PANEL>

<PANEL NAME=pandef2 PANDEF=panldef1
  HELP=morehlp KEYLIST=keyltbl>Second Panel
&sampbody;
</PANEL>

<PANEL NAME=pandef3 PANDEF=panldef2>Third Panel
&sampbody;
</PANEL>

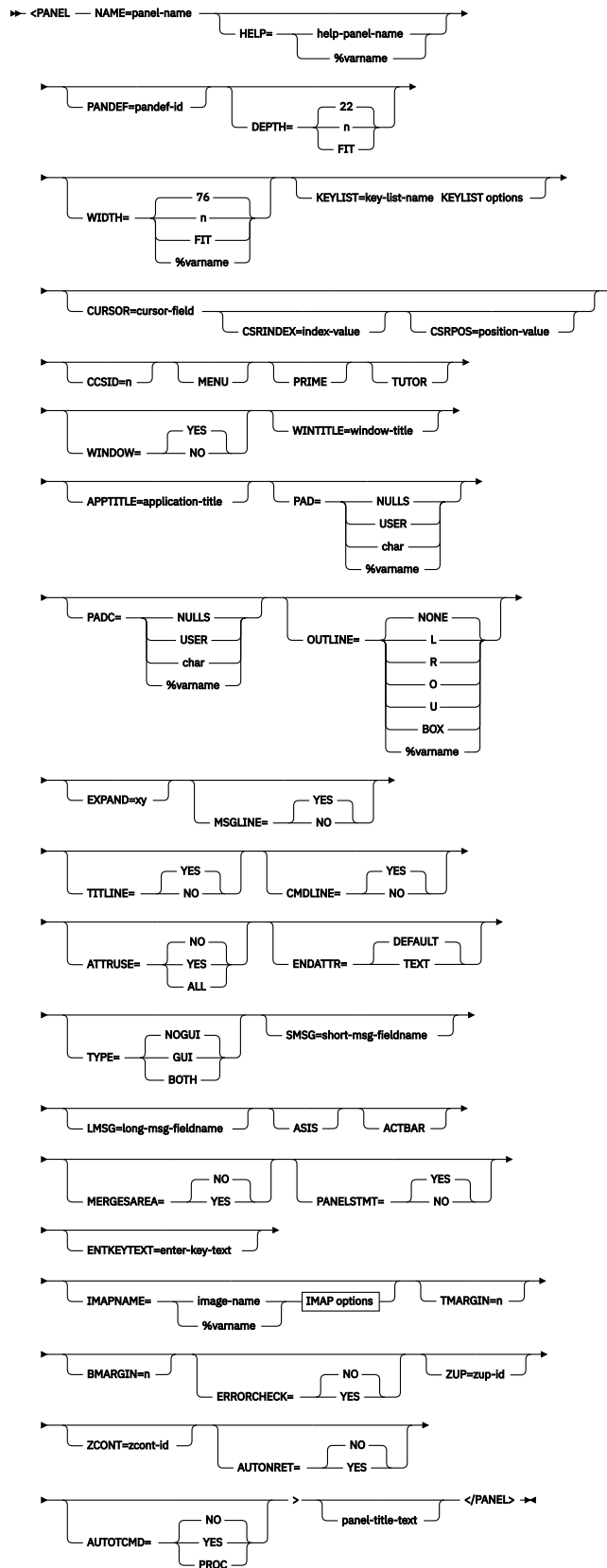
<HELP NAME=helpaaa>Help panel "helpaaa"
<AREA>
<INFO WIDTH=48>
<P>This is PANDEF help panel "helpaaa"
</INFO>
</AREA>
</HELP>

<HELP NAME=morehlp>Help panel "morehlp"
<AREA>
<INFO WIDTH=48>
<P>This is PANDEF help panel "morehlp"
</INFO>
</AREA>
</HELP>
```

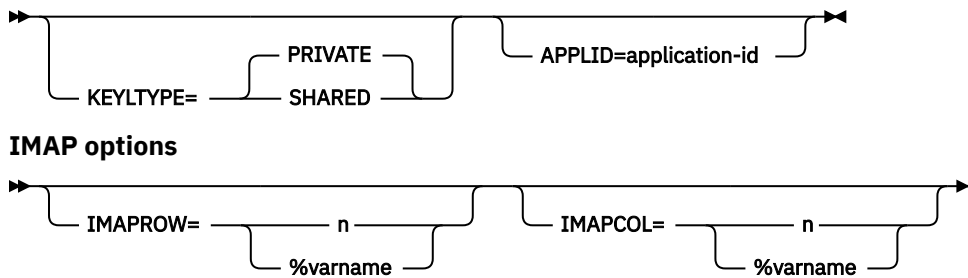
PANEL (Panel)

The PANEL tag defines an application panel.

Syntax



KEYLIST options



Parameters

NAME=panel-name

This attribute specifies the name of the panel. The *panel-name* is used in the ISPF DISPLAY or TBDISPL service call. The *panel-name* is also used as the panel ID, which the user can display. The *panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

If you specify NAME=*, the *panel-name* is set to the input DTL source member name. If multiple dialog element definitions have been combined within a single source file, then this notation should be used for only one dialog element definition within the file. See [“Dialog elements”](#) on page 5 for a description of dialog element types created by the conversion utility.

The *panel-name* is used to build the panel output file name in which the conversion utility stores the converted panel. The default is “userid.PANELS(*panel-name*)”.

You can specify the output panel library file name of your choice on the invocation panel for the conversion utility, or in the conversion utility profile as DDname DTLPAN for batch (or command syntax invocation) processing.

If the SCRIPT option has been specified, the *panel-name* is also used to build the file name in which the conversion utility stores the image of the panel. The default name is “userid.SCRIPT(*panel-name*)”.

You can specify the output SCRIPT library file name of your choice on the invocation panel for the conversion utility, or in the conversion utility profile as DDname DTLSCR for batch (or command syntax invocation) processing.

See [Chapter 10, “Using the conversion utility,”](#) on page 151 for complete information on invocation syntax.

HELP=help-panel-name | %varname

This attribute specifies the name of a defined extended (panel help) help panel. It identifies the help text that is associated with the panel definition.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

Specification of the HELP attribute causes ISPD TLC to generate “.HELP=help-panel-name” (or “.HELP=&varname”) in the)INIT section during panel generation.

ISPF displays this panel when the application user requests help and the cursor is not on a panel field that has its own field-level help specified. This help panel is also displayed when the user requests extended help.

PANDEF=pandef-id

This attribute specifies a defined panel default. The *pandef-id* value is the identifier specified on the PANDEF tag. You can override any of the defaults from this PANDEF tag by specifying that attribute on the PANEL tag. See [“PANDEF \(Panel Default\)”](#) on page 372 for information about defining panel defaults.

DEPTH=22 | n | FIT

This attribute defines the depth of the panel. The default depth is 22 when WINDOW=YES or 24 when WINDOW=NO. When the panel is displayed in a pop-up, ISPF adds two lines to the DEPTH value you specify to accommodate the borders at the top and bottom of the pop-up.

The value specified for the depth is the depth of the entire panel including the panel title, the action bar, the function key area, the message area, any scrollable areas, and the command area.

The maximum depth is 62 and the minimum depth is 5. If the DEPTH value is less than the minimum value allowed or exceeds the maximum value allowed, the conversion utility issues a warning message and sets the depth to the default.

The depth defined should be large enough to include all formatted text and input/output fields as well as the function key area, message area, any scrollable areas, and the command area. If the depth specified is not large enough to include these panel elements, ISPF overlays with the function keys if the function key display is on, or with the message area if the message is not displayed in a pop-up.

If DEPTH=FIT, The conversion utility formats the panel using a depth of 22. When formatting is completed the DEPTH value is reset to the minimum depth used or to 5 if the formatted panel contains less than 5 lines.

If the DEPTH value exceeds the maximum allowed to display the panel on the device, ISPF issues an error message at run time.

WIDTH=76 | n | FIT | %varname

This attribute defines the width (in characters) of the panel. The default width is 76 when WINDOW=YES or 80 when WINDOW=NO. When the panel is displayed in a pop-up, ISPF adds 4 to the WIDTH value you specify to accommodate the left and right borders of the pop-up.

The value specified for the width is the width of the entire panel (or region), including the margins.

The maximum width is 160 and the minimum width is 16.

Because there is a minimum margin width of 1 character on each side of the panel text, the effective width for text for a panel defined with WIDTH=76 is a maximum of 74 characters.

If the WIDTH value is less than the minimum value allowed or exceeds the maximum value allowed, ISPD TLC issues a warning message and sets the width to the default.

If WIDTH=FIT or WIDTH=%varname, the conversion utility formats the panel using the maximum available width as determined from the LRECL value of the output panel file.

If WIDTH=FIT, when formatting is completed the WIDTH value is reset to the minimum width used or to 16 if the formatted panel is less than 16 characters wide.

If WIDTH=%varname, when formatting is completed the WIDTH keyword on the)BODY panel statement is set to the variable name. WINDOW=NO must also be coded on the PANEL tag in order to use %varname.

Note: Panels that have the width specified as a variable cannot be preprocessed.

If WIDTH value exceeds the maximum allowed to display the panel on the device, ISPF issues an error message at run time.

KEYLIST=key-list-name

This attribute specifies the name of the key mapping list associated with the panel.

If you do not specify a *key-list-name* in a PANEL definition or an associated PANDEF definition, the ISPF-provided key list (ISPKYLIST) is used. For information about defining key mapping lists, see “KEYL (Key List)” on page 322. For information about the ISPF-provided key list, refer to the *z/OS ISPF Dialog Developer's Guide and Reference*.

KEYLTYPE=PRIVATE | SHARED

This attribute is used to add the SHARED keyword to the KEYLIST parameter of the)PANEL statement. For information about the)PANEL statement, refer to the *z/OS ISPF Dialog Developer's Guide and Reference*. The KEYLTYPE attribute is ignored if you have not provided the KEYLIST attribute as part of the PANEL tag definition or as part of an associated PANDEF tag definition.

APPLID=application-id

This attribute is used to add the application ID to the)PANEL statement. The *application-id* overrides the KEYLAPPL invocation option value. The APPLID attribute is ignored if you have not provided the KEYLIST attribute as part of the PANEL tag definition or as part of an associated PANDEF tag definition.

CURSOR=cursor-field

This attribute, together with CSRINDEX and CSRPOS, controls the initial placement of the cursor when the ISPF displays the panel. You can specify *cursor-field* as the value of:

- The NAME attribute of a CHOICE tag (for multiple-choice selection fields)
- The DATAVAR attribute of the CHOFLD tag.
- The DATAVAR attribute of a DTAFD tag
- The DATAVAR attribute of a LSTCOL tag
- The NAME attribute of a SELFLD tag (for single-choice selection fields).

The cursor can also be placed on the command area, when it is defined for a panel with the CMDAREA tag. Use the ISPF-reserved name *cmdarea* as the value for *cursor-field* to place the cursor on the command area.

CSRINDEX=index-value

This attribute, together with CURSOR and CSRPOS, controls the placement of the cursor when ISPF displays a table display panel. This attribute may be specified only when the CURSOR attribute refers to a list column.

CSRINDEX specifies the row in the)MODEL section where ISPF places the cursor when it displays the panel.

CSRPOS=position-value

This attribute, together with CURSOR and CSRINDEX, controls the placement of the cursor when ISPF displays the panel. This attribute may be specified only when the CURSOR attribute refers to a data field, list column, or the command area.

CSRPOS specifies the number of byte positions into the entry field that ISPF places the cursor when it displays the panel.

The first position of a field is denoted by 1. The maximum position that you can specify is the length of the underlying data.

If the value specified for this attribute is not valid, the default (1) is used.

CCSID=n

This attribute specifies the coded-character-set identifier as defined by the Character Data Representation Architecture. CCSID should be entered as a five-position numeric value. For more information about using the CCSID attribute, refer to the [*z/OS ISPF Dialog Developer's Guide and Reference*](#).

MENU

This attribute specifies that the panel is an ISPF menu selection or edit model selection panel. This type of panel does not allow a table display.

PRIME

This attribute is used together with MENU to specify a primary option menu.

TUTOR

This attribute specifies that the panel title be formatted with the word *Tutorial* (or its translated equivalent) on each end of the title line, similar to ISPF tutorial panels.

WINDOW=YES | NO

The WINDOW attribute is used to control the generation of the WINDOW keyword on the panel)BODY section. The default is to create the WINDOW keyword. WINDOW=NO should be used when WIDTH=%varname is also used to create a panel.

WINTITLE=window-title

This attribute is used to add a title on the pop-up window border. The attribute value is placed in the ISPF ZWINTTL variable. The maximum length of the *window-title* text is the panel width minus 1.

APPTITLE=application-title

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a "%".

PADC= NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a "%".

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a "%".

EXPAND=xy

This attribute adds the EXPAND(xy) attribute to the)BODY section of the panel. If only one character is present, the second character is set to the same value. If the EXPAND attribute is present with no value specified, the conversion utility uses a character from the range of low-order hex values available for panel attributes. This removes an available character from possible use as a panel attribute and may cause panel formatting errors.

MSGLINE=YES | NO

This attribute controls the provision for a long message line in the generated panel. When MSGLINE=NO, the blank line for the long message is not added to the panel)BODY section. It is the panel designer's responsibility to ensure that critical panel areas are positioned so that the long message does not inhibit use of the resulting panel.

TITLINE=YES | NO

This attribute controls the generation of the panel title line. When TITLINE=NO, the panel title is not added to the generated panel. This option is provided for applications that format a panel title as part of a dynamic area. It is the panel designer's responsibility to ensure that the resulting panel meets CUA requirements.

CMDLINE=YES | NO

This attribute controls the automatic generation of the command area on option menu panels and table display panels. When CMDLINE=NO, the command area is not automatically added to panels that do not include a CMDAREA tag within the panel definition.

ATTRUSE=NO | YES | ALL

This attribute controls the assignment of panel attributes within the range of x'01' through x'3F'. When ATTRUSE=YES or ATTRUSE=ALL, attributes for use in dynamic areas supplied by the ATTR tag can be assigned low-order hex values normally used by the conversion utility.

When ATTRUSE=YES, all of the attributes specified by the ATTR tag plus the required attributes used by the conversion utility must fit in the defined range of x'01' through x'2F'.

When ATTRUSE=ALL, all of the attributes specified by the ATTR tag plus the required attributes used by the conversion utility must fit in the defined range of x'01' through x'3F'.

ENDATTR=DEFAULT | TEXT

This attribute specifies that when the last attribute on any panel body line is "normal text" (CUA), it is replaced by the default "text" (ISPF) attribute. The effect is to force any text on subsequent lines not preceded by another attribute from the normal text color to blue.

TYPE=NOGUI | GUI | BOTH

This attribute specifies that the panel is used for either host display, by a client that is using the JSON API, or both. When NOGUI is specified, for example, the panel language control statements that enable check boxes and radio buttons are not added to the generated panel.

SMSG=short-msg-fieldname

This attribute provides the name of the field where the short message is to be placed. The *short-msg-fieldname* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

LMSG=long-msg-fieldname

This attribute provides the name of the field where the long message is to be placed. The *long-msg-fieldname* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

ASIS

This attribute specifies that the command and long message fields are to appear on the display as specified in the generated panel definition. When ASIS is specified, any user request specified on the Settings panel, or by setting the system variable ZPLACE is ignored.

ACTBAR

This attribute causes the action bar information for the panel to be generated, overriding the NOACTBAR invocation option.

MERGESAREA=NO | YES

This attribute controls an additional formatting step for panels with a single scrollable area. If the entire contents of the scrollable area fits within a standard 24-line panel (allowing two lines for the function keys display), and no input or output fields are found in the panel body following the location of the scrollable area, the scrollable area content is moved into the panel body.

PANELSTMT=YES | NO

This attribute controls the creation of the)PANEL statement. You can use this attribute to create a panel without keylist interaction.

ENTKEYTEXT=enter-key-text

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPROW=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPCOL=n | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

TMARGIN=n

This attribute provides the number of blank lines to format at the top of the panel as a top margin.

BMARGIN=n

This attribute provides the number of blank lines to format at the bottom of the panel as a bottom margin.

ERRORCHECK=NO | YES

This attribute specifies whether error checking logic is added to the end of the)PROC section. The extra logic prevents exit from the panel if any errors are present.

```
IF (.MSG ^= ' ')
  &ZVERB = ' '
  .RESP = ENTER
```

ZUP=zup-id

This attribute provides the name of the Tutorial panel to be assigned to the ZUP variable. It is valid only when the TUTOR attribute has been specified.

ZCONT=zcontid

This attribute provides the name of the Tutorial panel to be assigned to the ZCONT variable. It is valid only when the TUTOR attribute has been specified.

AUTONRET=NO | YES

This attribute specifies whether the .NRET = OFF panel statement is added to the)PROC section as part of the AUTOTYPE logic. When YES is specified, '.NRET = OFF' is the first AUTOTYPE panel logic statement created in the)PROC section.

AUTOTCMD=NO | YES | PROC

This attribute specifies whether the command field is refreshed during AUTOTYPE processing. When YES is specified, the command field name (normally ZCMD) is included with the AUTOTYPE variables added to the REFRESH statement in the)REINIT section of the panel. When PROC is specified, a REFRESH statement that references the command field name is included in the)PROC section of the panel. The REFRESH statement is inserted after the PANEXIT statement that invokes the AUTOTYPE panel exit.

panel-title-text

This is the text of the panel title.

Panel titles should be used when an application can display more than one panel. The *panel-title-text* is centered within the width defined for the panel in accordance with CUA rules. If the *panel-title-text* is wider than the WIDTH specified, the title is truncated from the right and an ellipsis (...) is appended. Two lines are reserved for the panel title and for a blank line between the panel title and the rest of the panel body.

Comments

The PANEL tag defines an application panel.

Tags coded within a PANEL definition (between the PANEL start tag and end tag) define the content of the panel.

Restrictions

- When the MENU attribute is specified, the LSTFLD tag cannot be nested under the PANEL tag.
- The PANEL tag requires an end tag.
- You cannot code a PANEL tag within any other tag definition.
- The PANEL definition must contain at least one of these tags:
 - BOTINST (See [“BOTINST \(Bottom Instruction\)” on page 204](#))
 - DA (See [“DA \(Dynamic Area\)” on page 250](#))
 - DTAFLD (See [“DTAFLD \(Data Field\)” on page 275](#))
 - GA (See [“GA \(Graphic Area\)” on page 295](#))
 - INFO (See [“INFO \(Information Region\)” on page 317](#))
 - LSTFLD (See [“LSTFLD \(List Field\)” on page 341](#))
 - PNLINST (See [“PNLINST \(Panel Instruction\)” on page 396](#))
 - SELFLD (See [“SELFLD \(Selection Field\)” on page 421](#))
 - TOPINST (See [“TOPINST \(Top Instruction\)” on page 441](#))
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names” on page 179](#).
- EXPAND can operate only when there are no trailing attributes on the line to be expanded. Panel lines formatted as part of a horizontal region require the use of attributes for field alignment. Therefore, the EXPAND feature is functional only for panel sections built with a vertical (or default) region that is not part of any horizontal region.

Processing

<i>Table 58. Tags you can code within a PANEL definition</i>			
Tag	Reference	Usage	Required
AB	“AB (Action Bar)” on page 179	Single	No
AREA	“AREA (Area)” on page 189	Multiple	No
BOTINST	“BOTINST (Bottom Instruction)” on page 204	Multiple	No
CMDAREA	“CMDAREA (Command Area)” on page 237	Single	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DA	“DA (Dynamic Area)” on page 250	Multiple	No
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
DTACOL	“DTACOL (Data Column)” on page 269	Multiple	No
DTAFLD	“DTAFLD (Data Field)” on page 275	Multiple	No
GA	“GA (Graphic Area)” on page 295	Single	No
GENERATE	“GENERATE (Generate)” on page 298	Multiple	No
GRPHDR	“GRPHDR (Group Header)” on page 300	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
INFO	“INFO (Information Region)” on page 317	Multiple	No
LSTFLD *	“LSTFLD (List Field)” on page 341	Single	No
PNLINST	“PNLINST (Panel Instruction)” on page 396	Multiple	No
REGION	“REGION (Region)” on page 405	Multiple	No
SELFLD	“SELFLD (Selection Field)” on page 421	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No
TEXTLINE	“TEXTLINE (Text Line)” on page 439	Single	No
TOPINST	“TOPINST (Top Instruction)” on page 441	Multiple	No
Note: Tags marked with * are not valid within an ISPF selection menu panel.			

Examples

Here is application panel markup that contains an action bar, a top instruction, two selection fields, and a command area. The PANEL KEYLIST attribute specifies a key mapping list, which is displayed below the command area. [Figure 143 on page 386](#) shows the formatted result.


```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=selcls TYPE='CHAR 2'>
<VARLIST>
  <VARDCL NAME=loc VARCLASS=selcls>
  <VARDCL NAME=mode VARCLASS=selcls>
</VARLIST>

<PANEL NAME=panel HELP=trvlhlp KEYLIST=keylmp
  DEPTH=22 WIDTH=60>Dream Vacation Guide
<AB>
  <ABC>File
    <PDC>Add Entry
      <ACTION RUN=add>
    <PDC>Delete Entry
      <ACTION RUN=delete>
    <PDC>Update Entry
      <ACTION RUN=update>
    <PDC>Exit
      <ACTION RUN=exit>
  <ABC>Help
    <PDC>Extended Help...
      <ACTION RUN=exhelp>
    <PDC>Keys Help...
      <ACTION RUN=keyshelp>
</AB>
<TOPINST>Choose one of the following exotic locations and
your preferred mode of travel, then press Enter.
<AREA>
  <REGION DIR=horiz>
  <SELFLD NAME=loc PMTWIDTH=23 SELWIDTH=25>Exotic Location:
    <CHOICE>Athens, GA
    <CHOICE>Berlin, CT
    <CHOICE>Cairo, IL
    <CHOICE>Lizard Lick, NC
    <CHOICE>Paris, TX
    <CHOICE>Rome, NY
    <CHOICE>Venice, FL
  </SELFLD>
  <DIVIDER>
  <SELFLD NAME=mode PMTWIDTH=25 SELWIDTH=25>Travel Mode:
    <CHOICE>Boxcar
    <CHOICE>Hitchhike
    <CHOICE>Mule
  </SELFLD>
</REGION>
</AREA>
<CMDAREA>
</PANEL>

<HELP NAME=trvlhlp>Sample help panel "trvlhlp"
<AREA>
<INFO WIDTH=48>
<P>This is help panel "trvlhlp"
</INFO>
</AREA>
</HELP>

```

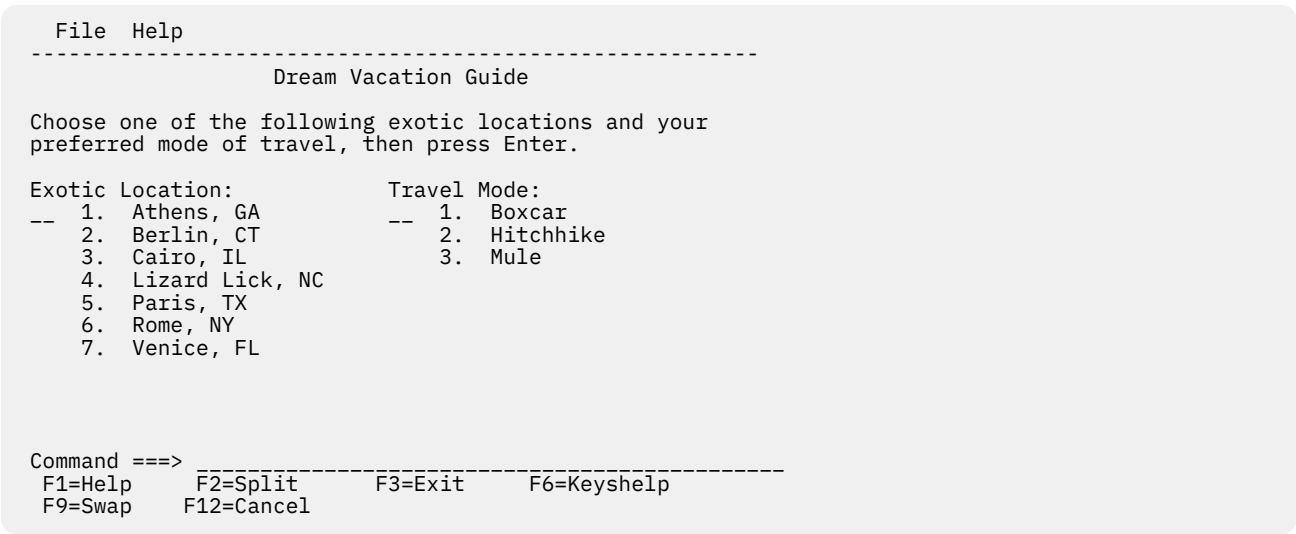
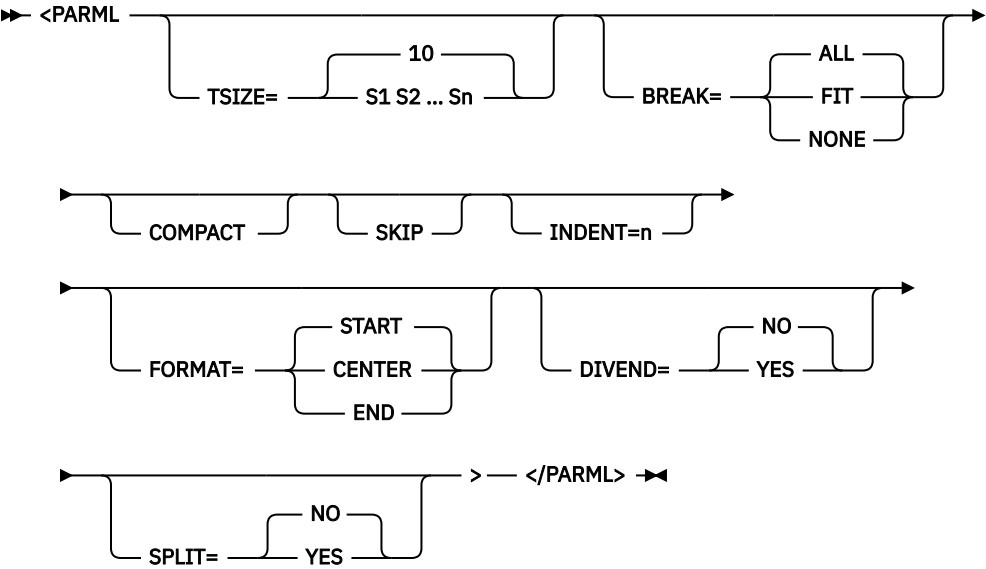


Figure 143. Application panel

PARML (Parameter List)

The PARML tag defines a parameter list within an information region.

Syntax



Parameters

TSIZE=**10** | S1 S2... Sn

This attribute defines the space allocated for the parameter term. The default is 10 characters. The minimum TSIZE value is 0 and the maximum is 40.

When multiple TSIZE values are specified, a PT tag must be coded for each value. The sizes are applied to the PT tags in the order the tags are encountered in the DTL source file.

BREAK=**ALL** | FIT | NONE

This attribute controls the formatting of the parameter terms and descriptions. If BREAK=ALL (the default), every description is on the line below the term. If BREAK=FIT, the description is on the line

below the term if the term is longer than the TSIZE value. If BREAK=NONE, the term is on the same line as the description, spilling into the description area if the length exceeds the TSIZE value.

COMPACT

This attribute causes the conversion utility to format the list without a blank line between the items.

SKIP

This attribute causes a blank line to be formatted before the first parameter term when COMPACT is also specified.

INDENT=n

This attribute specifies that the parameter list be indented from the current left margin.

FORMAT=START | CENTER | END

This attribute specifies the placement of the PT tag text within the space specified by TSIZE. The PARML tag FORMAT setting applies to all of the PT tags within the parameter list.

DIVEND=NO | YES

This attribute specifies whether a divider character is formatted following the PD tag text. When DIVEND=YES, the formatting width of the PD text is reduced to allow space for the divider character.

SPLIT=NO | YES

This attribute controls the format of the last PT tag in a multiple PT tag group. It is used only when BREAK=ALL or when BREAK=FIT and the PT tag text length exceeds the TSIZE value. When SPLIT=YES, the text following the last PT tag in the PT group (typically one or two dashes) is placed in front of the first line of the formatted PD tag text. The SPLIT setting on a PARML tag applies to all of the PT tag groups within the parameter list.

Comments

The PARML tag defines a parameter list within an information region.

Parameter lists are similar to definition lists. They involve three tags: PARML (parameter list) and a matching end tag, PT (parameter term), and PD (parameter description). As in definition lists, the term tag defines a term, and the definition tag defines the description associated with the term. The PD tag must immediately follow the PT tag that it is associated with.

Parameter lists can occur anywhere in an information region; you can nest them within other lists, and you can nest other lists within parameter lists.

Restrictions

- The PARML tag requires an end tag.
- You must code the PARML tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 59. Tags you can code within a PARML definition			
Tag	Reference	Usage	Required
PD	“PD (Parameter Description)” on page 388	Multiple	No
PLDIV	“PLDIV (Parameter List Divider)” on page 394	Multiple	No
PT	“PT (Parameter Term)” on page 400	Multiple	No
PTDIV	“PTDIV (Parameter Term Divider)” on page 402	Multiple	No

Examples

Here is help panel markup that contains two parameter lists. The second parameter list is nested within the second parameter description of the first list. [Figure 144 on page 388](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=parmls DEPTH=22>Part Number Code Help
<AREA>
<INFO>
  <P>Valid part numbers consist of a three-digit
  number followed by a 2-character suffix.
  <PARML TSIZE=6>
    <PT>123
    <PD>The first three digits represent
    the lot number of the part.
    <PT>AA
    <PD>The 2-character suffix represents the
    department the part originated from.
    The valid suffixes are:
      <PARML BREAK=none COMPACT>
        <PT>T0
        <PD>Tools
        <PT>EL
        <PD>Electrical
        <PT>ME
        <PD>Mechanical
      </PARML>
    </PARML>
  </INFO>
</AREA>
</HELP>
```

```

Part Number Code Help

Valid part numbers consist of a three-digit
number followed by a 2-character suffix.

123
    The first three digits represent the lot
    number of the part.

AA
    The 2-character suffix represents the
    department the part originated from. The
    valid suffixes are:

    T0      Tools
    EL      Electrical
    ME      Mechanical

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage   F12=Cancel
```

Figure 144. Parameter list

PD (Parameter Description)

The PD tag defines a parameter description in a parameter list.

Syntax

```

  >> <PD> _____ </PD> <<
           |               |
           | parameter-description |
           |               |
           +-----+

```

Parameters

parameter-description

This is the text of the parameter description.

Comments

The PD tag defines a parameter description in a parameter list.

Restrictions

- You must code the PD tag within a PARML definition. See [“PARML \(Parameter List\)” on page 386](#) for a complete description of this tag.
- Each PD tag must be paired with a PT tag. You can specify only one PD tag for each PT tag within a parameter list. The PD tag must immediately follow the PT tag it is associated with.

Processing

<i>Table 60. Tags you can code within a PD definition</i>			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a parameter list with three PD definitions. [Figure 145 on page 390](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=pd DEPTH=20>Help for Ordering Parts
<AREA>
<INFO>
  <P>Use one of the following codes when ordering
```

```

a part number from inventory:
<PARML TSIZE=5>
  <PT>ST
    <PD>Indicates that the part
    order is for stock replenishment.
  <PT>CU
    <PD>Indicates that the part
    order is for immediate customer shipment.
  <PT>EL
    <PD>Indicates that the part
    order is for shipment to an external location.
  </PARML>
</INFO>
</AREA>
</HELP>

```

Help for Ordering Parts

Use one of the following codes when ordering a part number from inventory:

```

ST      Indicates that the part order is for stock
        replenishment.

CU      Indicates that the part order is for
        immediate customer shipment.

EL      Indicates that the part order is for
        shipment to an external location.

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage   F12=Cancel

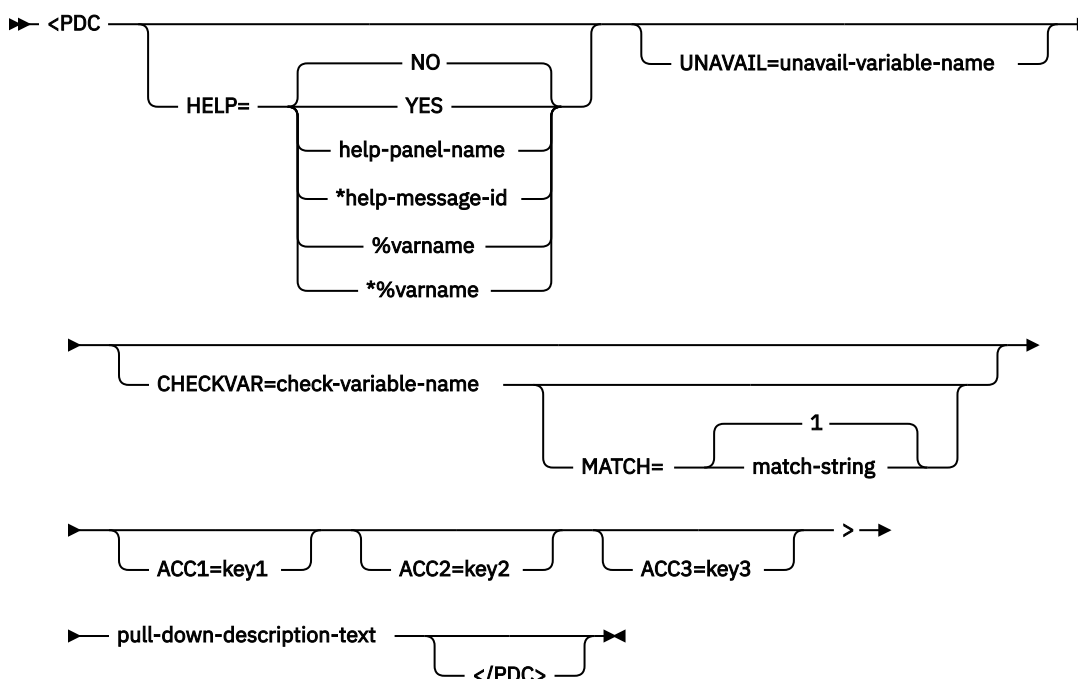
```

Figure 145. Parameter descriptions

PDC (Pull-Down Choice)

The PDC tag defines a pull-down choice for an action bar pull-down.

Syntax



Parameters

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for a pull-down choice selection.

When `HELP=YES`, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When `%varname` is coded, a panel variable name is created. When `*%varname` is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See [“HELP \(Help Panel\)”](#) on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

Note: This attribute is valid only when the `SELFLD` tag has been specified with `TYPE=MULTI`.

UNAVAIL=unavail-variable-name

This attribute specifies the name of a variable that is used by ISPF to determine the availability of the pull-down choice. When the variable value is 1, the pull-down choice is unavailable.

The *unavail-variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

CHECKVAR=check-variable-name

This attribute specifies a variable whose value indicates whether or not the pull-down choice is preselected when the pull-down is displayed. If the value of the variable is equivalent to the *match-string* you specify with the `MATCH` attribute, the pull-down choice appears preselected. Otherwise, it does not. The *check-variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

Note: Unlike selection fields, ISPF does not reset the *check-variable-name* to indicate the pull-down choice the user selects. Therefore, you should code the SETVAR attribute in an ACTION tag associated with the pull-down choices when the application needs to know which pull-down choice was selected.

MATCH=1 | match-string

This attribute defines the value that causes the pull-down choice to be preselected. The value of variable specified by the CHECKVAR attribute is compared to the *match-string* value, and if they are equal, the pull-down choice appears preselected.

ACC1=key1

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

ACC2=key2

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

ACC3=key3

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

pull-down-description-text

This is the text for the pull-down choice. The maximum length of the text is 64 bytes.

Each *pull-down-description-text* is prefixed with a sequential number beginning with 1 to allow selection by number.

Comments

The PDC tag defines a pull-down choice for an action bar pull-down. If you do not code any PDC tags within an ABC tag, that action bar choice does not appear on the action bar.

To provide for a pull-down selection, an input field is generated prior to the first *pull-down-description-text* that allows entry of the number of the selected pull-down choice. Since field names are being generated, the application developer should not use field names beginning with Z.

Restrictions

- You must code the PDC tag within an ABC definition. See [“ABC \(Action Bar Choice\)” on page 182](#) for a complete description of this tag.
- The maximum number of pull-down choices that is generated is 60. However, the depth specified on the enclosing PANEL tag can further reduce this maximum number.

Processing

Table 61. Tags you can code within a PDC definition			
Tag	Reference	Usage	Required
ACTION	“ACTION (Action)” on page 184	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
M	“M (Mnemonic)” on page 351	Single	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is application panel markup that produces the action bar and pull-down shown in [Figure 146](#) on page 393.

In this example, when the action bar choice **Search** is chosen, the variable *whchsrch* is tested to see if one of the pull-down choices should be preselected. If *whchsrch*=1 then the pull-down choice **Search on name** is preselected with a 1 in the pull-down selection entry field. If *whchsrch*=2 then the pull-down choice **Search on card number** is preselected with a 2 in the pull-down selection entry field. If *whchsrch* is not equal to 1 or 2, then neither pull-down choice is preselected. The example shows the **Search on name** choice preselected. If *srch2*=1, then the UNAVAIL attribute on the pull-down choice Search on card number would cause that choice to be unavailable. The example shows the result.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 system>
  <!entity sampbody system>)>
&sampvar1;

<PANEL NAME=pdc2 KEYLIST=keylxml>Library Card Registration
<AB>
<ABC>File
  <PDC>Add Entry
    <ACTION RUN=add>
  <PDC>Delete Entry
    <ACTION RUN=delete>
  <PDC>Update Entry
    <ACTION RUN=update>
  <PDC>Exit
    <ACTION RUN=exit>
<ABC>Search
  PDC CHECKVAR=whchsrch MATCH=1 UNAVAIL=srch1>Search on name
    <ACTION SETVAR=whchsrch VALUE=1>
    <ACTION RUN=search>
  PDC CHECKVAR=whchsrch MATCH=2 UNAVAIL=srch2>Search on card number
    <ACTION SETVAR=whchsrch VALUE=2>
    <ACTION RUN=search>
<ABC>Help
  <PDC>Extended Help...
    <ACTION RUN=exhelp>
  <PDC>Keys Help...
    <ACTION RUN=keyshelp>
</AB>
&sampbody;
</PANEL>
```

File Search Help

1 1. Search on name
* Search on card number

gistration

Type in client's name and case number (if applicable).

Then, select an action bar choice.

Date . . . : 11/09/89
Card No . . . (A 7-digit number)
Name . . . (Last, First, M.I.)
Address . . .

Choose one of the following Check valid branches

— 1. New	— North Branch
— 2. Renewal	— South Branch
— 3. Replacement	— East Branch
	— West Branch

Enter a command ==> _____

F1=Help F2=Split F3=Exit F6=KEYSHELP F9=Swap

Figure 146. Pull-down choices

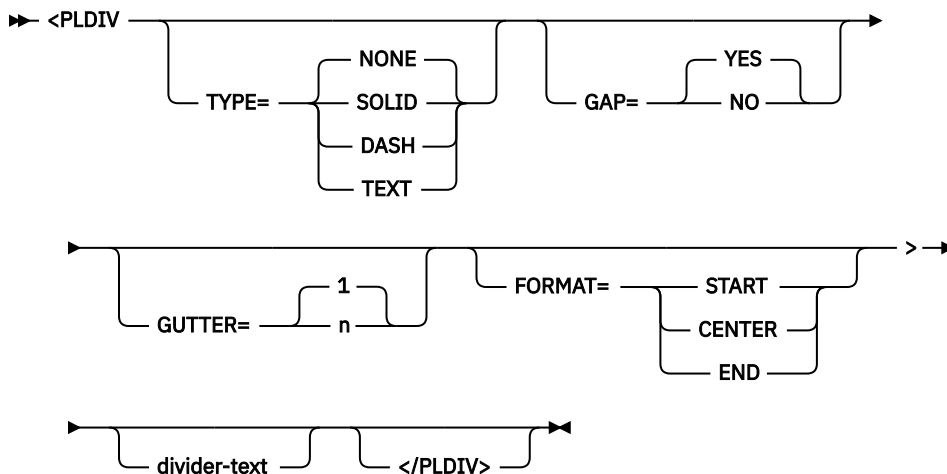
PDSEP (Pull-Down Separator)

The PDSEP tag is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PLDIV (Parameter List Divider)

The Parameter List Divider tag creates a blank or visible divider within the text portion of a parameter list.

Syntax



Parameters

TYPE=NONE | SOLID | DASH | TEXT

This attribute specifies the type of parameter list divider line.

The default value is NONE, which produces a blank line. You must specify SOLID, DASH, or TEXT to produce a visible divider line. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

GAP=YES | NO

When GAP=NO, the divider line completely crosses from one side of the text area to the other. When GAP=YES, a 1-character gap remains at each end of the divider line.

GUTTER=1 | n

This attribute specifies the total width of the parm list divider. If the GUTTER value is an even number, the conversion utility increases the number by 1 so that the divider is centered within the defined width.

The minimum GUTTER value, and the default, is 1.

FORMAT=START | CENTER | END

This attribute specifies the position of the divider text within the width of the divider line.

divider-text

This is the text of the area divider line.

Comments

The PLDIV tag creates a blank or solid divider within the text portion of an application panel. A horizontally formatted visible divider is created when you specify the TYPE attribute value as SOLID or DASH. When the GRAPHIC invocation option is specified, SOLID produces a solid line for host display and DASH produces a dashed line. When NOGRAPHIC is specified, both SOLID and DASH produce a dashed line.

The divider line can be formatted with descriptive text. When this feature is used, the FORMAT attribute must be specified. If FORMAT is not specified, the tag text is ignored. You control the text padding with the TYPE attribute. If TYPE=TEXT, the *divider-text* is padded with blanks. When TYPE=SOLID or TYPE=DASH, the *divider-text* is padded with the specified character.

Restrictions

- You must code the PLDIV tag within a PARML tag definition.

Processing

Table 62. Tags you can code within a PLDIV definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is an example that uses the PLDIV tag. [Figure 147 on page 396](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=pldiv DEPTH=22 WIDTH=60>Part Number Code Help
<AREA>
<INFO>
  <P>Valid part numbers consist of a three-digit
  number followed by a 2-character suffix.
  <DIVIDER>
  <PARML TSIZE=6 compact>
    <PLDIV TYPE=solid>
      <PT>123
      <PD>The first three digits represent
      the lot number of the part.
    <PLDIV TYPE=solid>
      <PT>AA
      <PD>The 2-character suffix represents the
      department the part originated from.
      The valid suffixes are:
      <PARML BREAK=none COMPACT SKIP>
        <PT>T0
        <PD>Tools
        <PT>EL
        <PD>Electrical
        <PT>ME
        <PD>Mechanical
      </PARML>
    </PARML>
  </INFO>
</AREA>
</HELP>
```

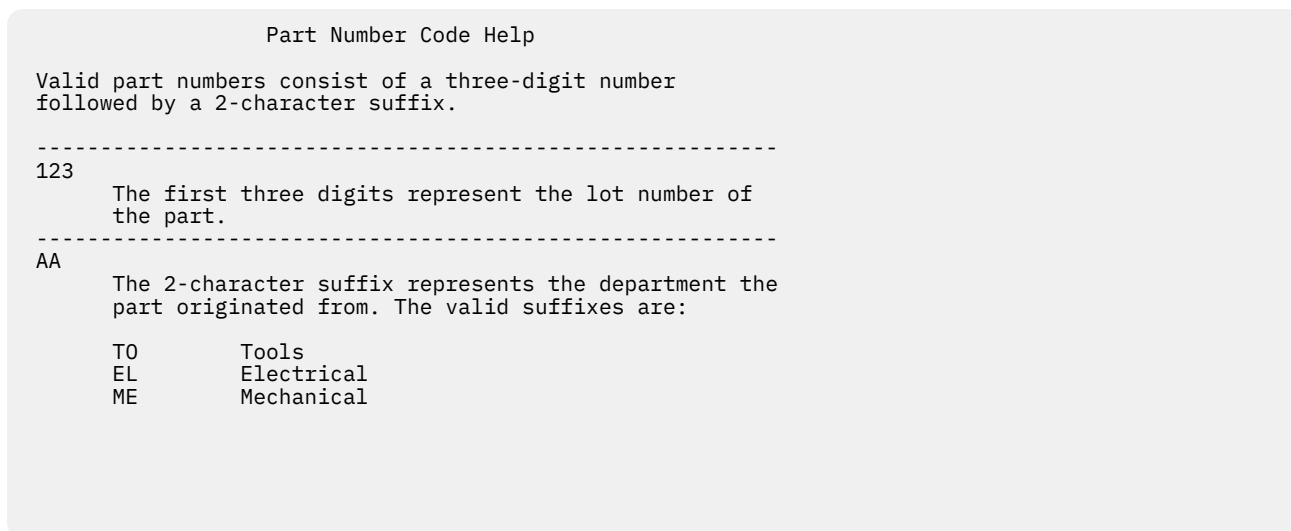


Figure 147. Parameter list divider

PNLINST (Panel Instruction)

The PNLINST tag defines panel instructions for an application panel.

Syntax



Parameters

COMPACT

This attribute causes the panel instruction to format without a blank line before the text.

instruction-text

This is the text of the panel instruction. The *instruction-text* must fit in the remaining panel depth.

Comments

The PNLINST tag defines panel instructions for an application panel. The *instruction-text* formats as a paragraph based on the width of the application panel, area, or region. You can code multiple paragraphs of instruction text by using a new panel instruction tag for each new paragraph.

If the COMPACT attribute is not specified, the conversion utility inserts a blank line before the panel instruction text.

Restrictions

- You must code the PNLINST within a PANEL, AREA, or REGION definition.

Processing

Table 63. Tags you can code within a PNLINST definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is application panel markup that contains one panel instruction. [Figure 148 on page 398](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=selcls TYPE='char 2'>
<VARLIST>
  <VARDECL NAME=loc VARCLASS=selcls>
  <VARDECL NAME=mode VARCLASS=selcls>
</VARLIST>

<PANEL NAME=pnlinst HELP=trvlhlp WIDTH=60 DEPTH=22 KEYLIST=keylxml>
Dream Vacation Guide
<AB>
  <ABC>File
    <PDC>Add Entry
      <ACTION RUN=add>
    <PDC>Delete Entry
      <ACTION RUN=delete>
    <PDC>Update Entry
      <ACTION RUN=update>
    <PDC>Exit
      <ACTION RUN=exit>
  <ABC>Help
    <PDC>Extended Help...
      <ACTION RUN=exhelp>
    <PDC>Keys Help...
      <ACTION RUN=keyshelp>
</AB>
<AREA>
  <PNLINST>Choose one of the following exotic locations and
your preferred mode of travel, then press Enter.
  <DIVIDER>
  <REGION DIR=horiz>
    <SEFLD NAME=loc PMTWIDTH=23 SELWIDTH=25>Exotic Location:
      <CHOICE>Athens, GA
      <CHOICE>Berlin, CT
      <CHOICE>Cairo, IL
      <CHOICE>Lizard Lick, NC
      <CHOICE>Paris, TX
      <CHOICE>Rome, NY
      <CHOICE>Venice, FL
    </SEFLD>
    <DIVIDER>
    <SEFLD NAME=mode PMTWIDTH=25 SELWIDTH=25>Travel Mode:
      <CHOICE>Boxcar
      <CHOICE>Hitchhike
      <CHOICE>Mule
    </SEFLD>
  </REGION>
</AREA>
<CMDAREA>
</PANEL>
```

```

                                Dream Vacation Guide
Command ==> -----

Choose one of the following exotic locations and your
preferred mode of travel, then press Enter.

Exotic Location:                Travel Mode:
-- 1. Athens, GA                -- 1. Boxcar
   2. Berlin, CT                -- 2. Hitchhike
   3. Cairo, IL                 -- 3. Mule
   4. Lizard Lick, NC
   5. Paris, TX
   6. Rome, NY
   7. Venice, FL

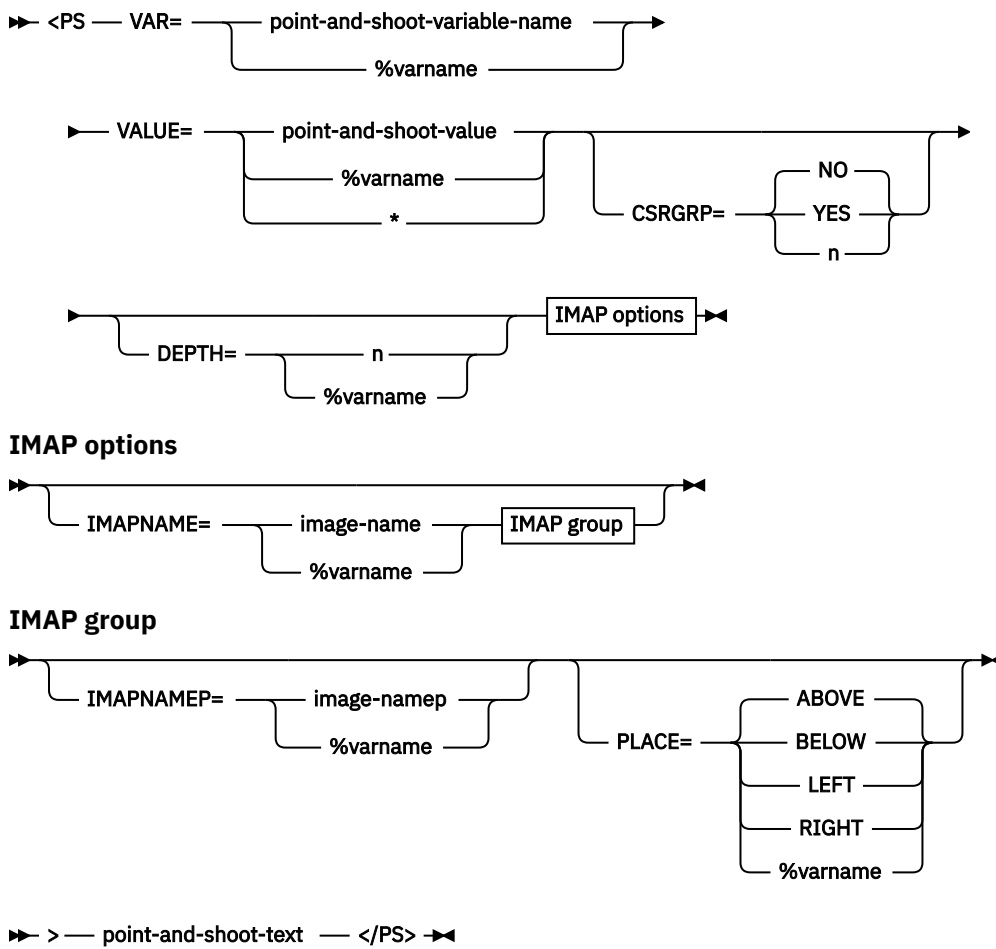
```

Figure 148. Panel instructions

PS (Point-and-Shoot)

The PS tag defines a text string that is to be enabled for point-and-shoot.

Syntax



Parameters

VAR=point-and-shoot-variable-name | %varname

This attribute provides the name of a variable which is to be set when a point-and-shoot phrase is clicked on for selection. You can define this attribute as a variable name preceded by a "%".

The *point-and-shoot-variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

VALUE=point-and-shoot-value | %varname | *

This attribute provides the value to be placed in the field specified by the VAR attribute. You can define this attribute as a variable name preceded by a "%". To specify a blank value, the " ' ' " (quotation mark, apostrophe, blank, apostrophe, quotation mark) coding notation should be used.

When the PS tag is used with the CHOICE tag, VALUE=* can be used to automatically use the current choice number (or SELCHAR value) as the point-and-shoot selection value.

CSRGRP=NO | YES | n

When CSRGRP=YES, the conversion utility generates a cursor group number to be used for this point-and-shoot text field. When CSRGRP=n, the number provided is used for this field.

DEPTH=n | %varname

This attribute defines the depth reserved for the point-and-shoot field. The minimum value is 1 and the maximum value is the remaining panel depth. This attribute is accepted in order to support existing DTL source files that use it. However, although the space is reserved, point-and-shoot does not function in the additional reserved space..

IMAPNAME=image-name | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

IMAPNAMEP=image-namep | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

PLACE=ABOVE | BELOW | LEFT | RIGHT | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

point-and-shoot-text

This is the text of a point-and-shoot entry.

Comments

The PS tag is valid as part of the text following these tags:

INFO TAGS

ATTENTION, CAUTION, DD, DDHD, DT, DTHD, FIG, FIGCAP, H2, H3, H4, LI, LINES, LP, NOTE, NT, P, PD, PT, WARNING, and XMP.

PANEL TAGS

BOTINST, CHOFLD, CHOICE, DTAFLD, DTAFLDD, GRPHDR, LSTCOL, LSTGRP, PNINST, SELFLD, and TOPINST.

The *point-and-shoot-text* is color emphasized within the text of the panel. The user places the cursor on the *point-and-shoot-text* and presses ENTER to select the option.

Restrictions

- The PS tag requires an end tag.
- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names”](#) on page 179.

Processing

None.

Examples

Here is an example that shows the use of point-and-shoot selection for a sample option menu. [Figure 149 on page 400](#) shows the formatted result.

```
<!doctype dm system ()>
<!-- Sample selection menu with point-and-shoot -->
<panel name=ps1 menu keylist=keylxml>Sample Point-and-Shoot
  <topinst>This is a selection panel.
    <selfld type=menu pmtloc=before
      selwidth=40 pmtwidth=10>Select an option
      <choice checkvar=xtest1 match=a>
        <PS VAR=zcmd VALUE=1>Selection #1 (Command Tstch1)
      </PS>
      <action run=tstch1 parm='1 2 3 4'
        passlib newpool suspend>
      <choice checkvar=xtest1 match=b>
        <PS VAR=zcmd VALUE=2>Selection #2 (Command Tstch2)
      </PS>
      <action run=tstch2 parm=1234>
      <choice checkvar=xtest1 match=c>
        <PS VAR=zcmd VALUE=3>Selection #3 (Command Tstch3)
      </PS>
      <action run=tstch3 parm=abcd>
      <choice checkvar=xtest1 match=d>
        <PS VAR=zcmd VALUE=4>Selection #4 (Command Tstch4)
      </PS>
      <action run=tstch4 parm='a b c d'>
    </selfld>
  <cmdarea>
</panel>
```

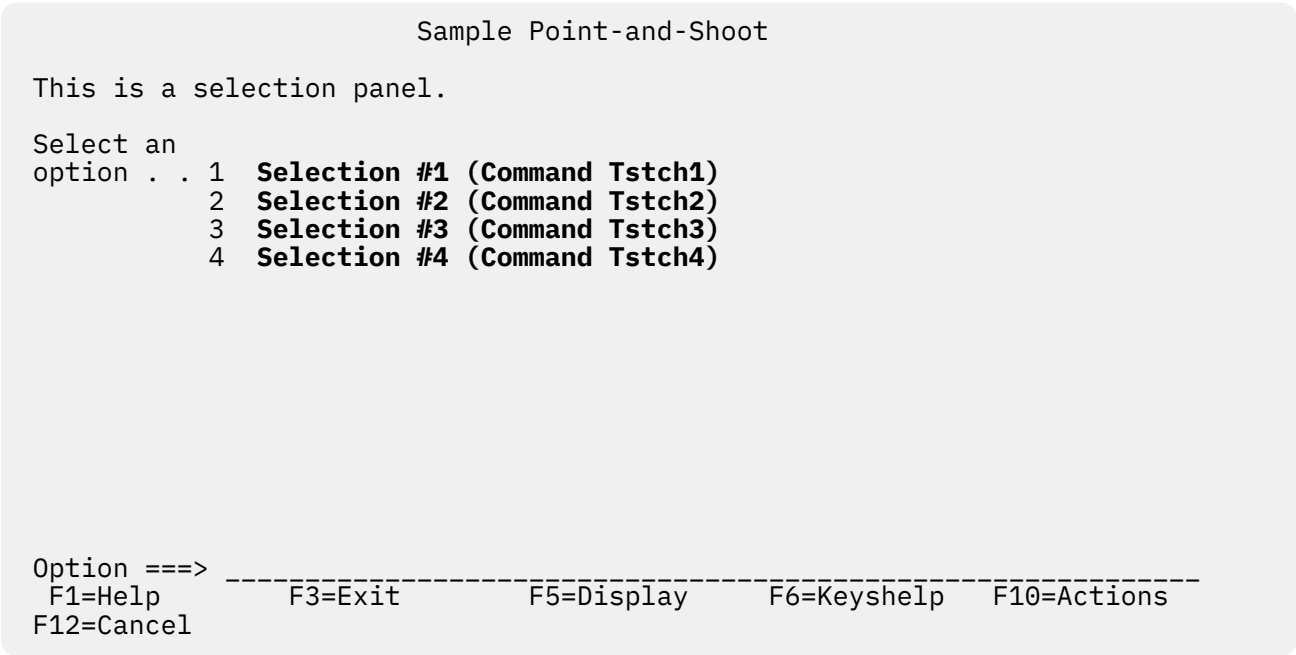
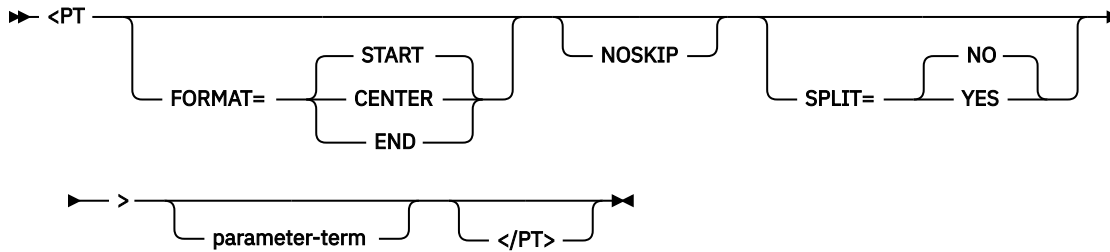


Figure 149. Point-and-shoot fields

PT (Parameter Term)

The PT tag defines a term in a parameter list.

Syntax



Parameters

FORMAT = START | CENTER | END

This attribute specifies the placement of the PT tag text within the space provided by TSIZE. The PT tag FORMAT setting overrides the FORMAT setting of the enclosing PARML tag.

NOSKIP

This attribute causes the definition term to be formatted without a blank line before the term. It is used to control the formatting of the parameter term when COMPACT has not been specified on the enclosing PARML tag. When the PARML tag TSIZE attribute specifies that multiple PT tags are to be formatted for each PD tag, NOSKIP should be coded on the first PT tag. It is ignored for the second and subsequent PT tags.

SPLIT=NO | YES

This attribute controls the format of the last PT tag in a multiple PT tag group. It is used only when BREAK=ALL or when BREAK=FIT and the PT tag text length exceeds the TSIZE value. When SPLIT=YES, the text following the last PT tag in the PT group (typically one or two dashes) is placed in front of the first line of the formatted PD tag text. The PT tag SPLIT setting overrides the SPLIT specified in the enclosing PARML tag.

parameter-term

This is the text of the parameter term.

Comments

The PT tag defines a parameter term in a parameter list.

Restrictions

- You must code the PT tag within a PARML definition. See [“PARML \(Parameter List\)”](#) on page 386 for a complete description of this tag.
- Each PT tag must be paired with an associated PD tag. You can specify only one PT tag for each PD tag within a parameter list. The PT tag must immediately precede the PD tag it is associated with.

Processing

Table 64. Tags you can code within a PT definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
PTSEG	“PTSEG (Parameter Term Segment)” on page 404	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is help panel markup that contains a parameter list with two parameter terms. [Figure 150 on page 402](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=pt WIDTH=40 DEPTH=18>Help for the Duplex Function
<AREA>
<INFO>
  <P>The two options associated with
  the DUPLEX function are:
  <PARML TSIZE=5>
    <PT>DCopies
    <PD>Which prints one-sided copies that
    are prepared for future duplex copying.
    <PT>DPrint
    <PD>Which prints two-sided copies.
  </PARML>
</INFO>
</AREA>
</HELP>
```

```

Help for the Duplex Function

The two options associated with the
DUPLEX function are:

DCopies
    Which prints one_sided copies
    that are prepared for future
    duplex copying.

DPrint
    Which prints two_sided copies.

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel

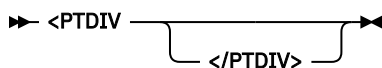
```

Figure 150. Parameter terms

PTDIV (Parameter Term Divider)

The PTDIV tag defines a visible vertical divider (|) between multiple PT tags.

Syntax



Comments

The PTDIV tag can be used to create a visual separation between the parameter terms. Each PTDIV tag adds a vertical bar (plus display control attributes) to the parameter list.

Restrictions

The PTDIV tag can be coded before the first PT tag, between PT tags, or following the last PT tag (before the PD tag definition).

Processing

None.

Examples

Here is an example that shows the PTDIV tag in combination with the DIVEND attribute of the PARML tag. [Figure 151 on page 403](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=ptdiv DEPTH=22 WIDTH=60>Part Number Code Help
<AREA>
<INFO>
  <P>Valid part numbers consist of a three-digit
  number followed by a 2-character suffix.
  <DIVIDER>
  <PARML TSIZE=6 compact>
    <PLDIV TYPE=solid>
      <PT>123
      <PD>The first three digits represent
      the lot number of the part.
      <PLDIV TYPE=solid>
      <PT>AA
      <PD>The 2-character suffix represents the
      department the part originated from.
      The valid suffixes are:
      <PARML BREAK=none COMPACT SKIP DIVEND=yes>
        <PLDIV TYPE=solid>
          <PTDIV>
          <PT>TO
          <PTDIV>
          <PD>Tools
          <PTDIV>
          <PT>EL
          <PTDIV>
          <PD>Electrical
          <PTDIV>
          <PT>ME
          <PTDIV>
          <PD>Mechanical
        </PARML>
      </PARML>
    </INFO>
  </AREA>
</HELP>
```

```
Part Number Code Help

Valid part numbers consist of a three-digit number
followed by a 2-character suffix.

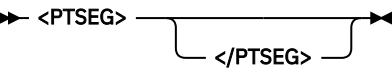
-----
123      The first three digits represent the lot number of
         the part.
-----
AA       The 2-character suffix represents the department the
         part originated from. The valid suffixes are:
         -----
         | TO          | Tools          |
         | EL          | Electrical     |
         | ME          | Mechanical     |
```

Figure 151. Parameter term divider

PTSEG (Parameter Term Segment)

The PTSEG tag defines a segment of the parameter term. It is used to provide vertical separation of the PT tag text.

Syntax



Comments

The PTSEG tag is used to create a vertical separation within the parameter term. The text following the PTSEG tag is formatted directly under any previous parameter term tag text. Multiple PTSEG tags create additional PT text lines.

Use of the PTSEG tag affects the PARML tag BREAK attribute. The first (or only) line of PT tag text is processed according to the BREAK attribute of the PARML tag. For additional lines, when TSIZE is large enough to accommodate the text segments, the PTSEG text is formatted in front of the associated PD tag text. When TSIZE is not large enough to accommodate the largest segment, all of the PT and PTSEG text is formatted above the associated PD tag text.

Restrictions

- The PTSEG tag can be coded within the text following a PT tag.
- When a PTSEG tag is coded, then all remaining PT tag text for the current PT tag set must follow a PTSEG tag.
- The PT nested tags RP and PS are not supported within PT tag text following any PTSEG tag in a PT/PD tag set.

Processing

Table 65. Tags you can code within a PTSEG definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is an example that shows the PTSEG tag in combination with a multiple PT tag set. The last PT tag includes the SPLIT=yes attribute to format the dash in front of the PD tag text. [Figure 152 on page 405](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM(>

<PANEL NAME=ptseg KEYLIST=ISRHELP APPLID=ISR WINDOW=no PADC=user
      TUTOR ZUP=ISP7R000>Traces - Primary Commands

<CMDAREA CAPS=on>
<AREA DEPTH=1 EXTEND=on>

  <INFO WIDTH=*>
    <P>
      Enter a <hp>Primary Command</hp> in the command input field.
      It is processed after all row modifications and all line commands
      are processed. The following primary commands are valid for the
      Traces options:

    <PARML TSIZE="8 1" INDENT=2>
```

```
<PT>
  LOCATE
  function-name
  (Function Traces) or variable name (Variable Traces)
  <PTSEG>
  LOC or
  <PTSEG>
  L
  <PT SPLIT=yes>-
  <PD>The LOCATE command positions the scrollable display at the
    first (or next) row containing the function name (Function
    Traces option) or the variable name (Variable Traces option).
  </PARML>
  </INFO>
  </AREA>
  </PANEL>
```

Tutorial ----- Traces - Primary Commands ----- Tutorial
Command ==> -----

Enter a Primary Command in the command input field. It is processed after all row modifications and all line commands are processed. The following primary commands are valid for the Traces options:

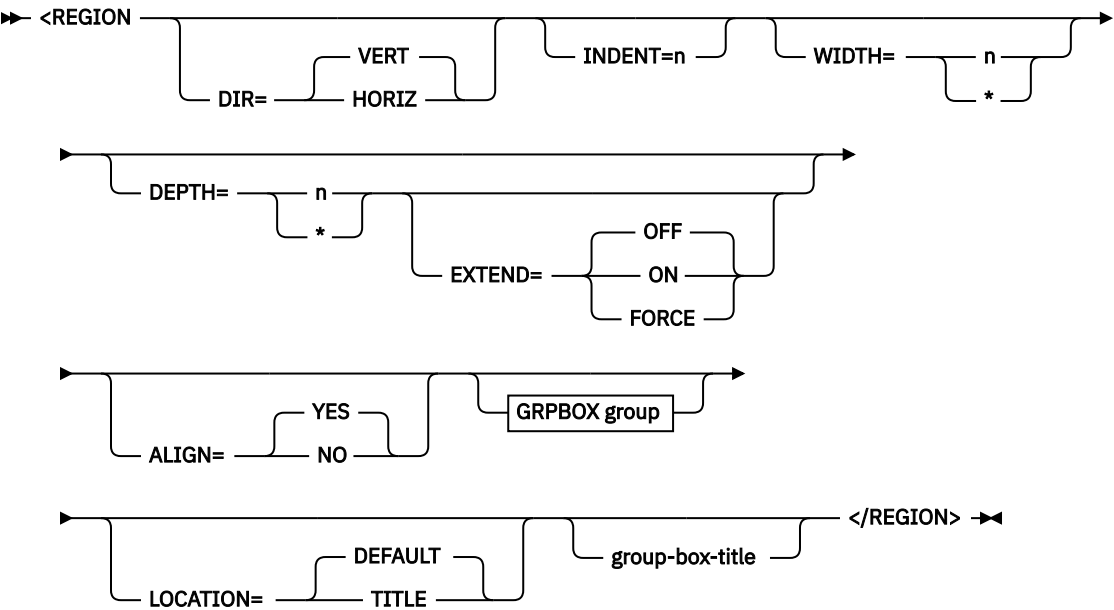
LOCATE function-name (Function Traces) or variable-name (Variable Traces)
LOC or - The LOCATE command positions the scrollable display at the first
L (or next) row containing the function name (Function Traces option) or the variable name (Variable Traces option).

Figure 152. Parameter term segment

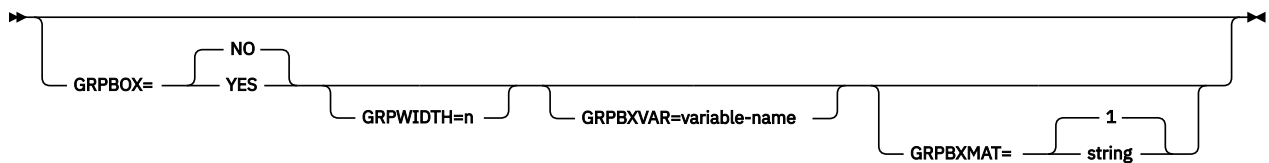
REGION (Region)

The REGION tag defines the characteristics of a panel section including the direction in which fields on an application panel are arranged.

Syntax



GRPBOX group



Parameters

DIR=VERT | HORIZ

This attribute specifies in which direction the contents of a region is arranged. The default value is VERT, which formats the contents of the region in a vertical direction; that is, top to bottom. If you specify the HORIZ value for DIR, the contents of the region are formatted horizontally; that is, left to right within the region.

INDENT=n

This attribute defines the number of columns to indent the current region from the current left region boundary.

WIDTH=n | *

This attribute defines the width of a panel region. If WIDTH is not specified or WIDTH=*, the default value is the remaining available panel width.

DEPTH=n | *

This attribute defines the size of a scrollable region. When EXTEND=OFF, the minimum value is 2 and the maximum value is the remaining panel depth. When EXTEND=ON, the minimum value is 1. If the DEPTH value is specified as "*", the conversion utility reserves the remaining available panel depth for the scrollable region.

If DEPTH is not specified the region is not scrollable.

EXTEND=OFF | ON | FORCE

This attribute defines the runtime display size for the scrollable region. If EXTEND=ON is specified, the panel definition is expanded from the minimum DEPTH to the size of the logical screen. Only one EXTEND=ON attribute value is allowed on a panel. The first tag (AREA, DA, GA, REGION, SELFLD) with EXTEND=ON is accepted; the EXTEND attribute on any subsequent tag is ignored.

If you intend to display the panels in a pop-up window, it is recommended that you code EXTEND=OFF.

If the EXTEND attribute is specified without the DEPTH attribute, a warning message is issued and the EXTEND attribute is ignored.

If EXTEND=FORCE is specified within a horizontal area or region, the EXTEND(ON) keyword is added to the scrollable area attribute statement in the)ATTR panel section. The conversion utility issues a message to advise of a potential display error if other panel fields are formatted on or after the last defined line of the scrollable area.

ALIGN=YES | NO

This attribute controls the horizontal alignment of the first fields in horizontal regions. The default is to align the fields to facilitate cursor movement by tabbing. This attribute is valid only when DIR=HORIZ.

GRPBOX=NO | YES

This attribute is used to specify a region title. The default value is NO.

When GRPBOX=YES is specified on the same REGION tag that defines a scrollable region, the region title is formatted as the first line within the)AREA panel section.

GRPWIDTH=n

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

GRPBXVAR=variable-name

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

GRPBXMAT=1 | string

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

LOCATION=DEFAULT | TITLE

This attribute is used to build a panel 'title' which requires data fields in specific column positions. A single line may be formatted to be placed in the panel title position by enclosing the appropriate tags within a horizontal region specifying LOCATION=TITLE. The resulting line displays with the colors associated with the tags used to format the line. This attribute is valid only when DIR=HORIZ.

group-box-title

This is the title for the region. The *group-box-title* should be supplied only when GRPBOX=YES. In other cases a warning message is issued.

Comments

The REGION tag defines the characteristics of a panel section. You can code multiple regions within an application panel.

Nonscrollable horizontal regions are normally aligned left-to-right using the first input field from each region. If a panel consists of both scrollable and nonscrollable regions formatted horizontally, scrollable regions are normally aligned with the first input fields of nonscrollable regions.

Regions containing data formatted from INFO tags or from the GRPHDR tag normally start with a blank line when formatted in the)BODY panel section. The blank line is omitted when these tags are formatted at the beginning of a scrollable area.

If you specify the CMDAREA tag within your DTL source file, it must appear before the REGION tag when DEPTH=* is specified. The REGION tag DEPTH may have to be adjusted to allow for additional lines which result from tags present within the panel definition following the end REGION tag.

Restrictions

- The REGION tag requires an end tag.
- You must code the REGION tag within an AREA or PANEL definition. See [“AREA \(Area\)” on page 189](#) and [“PANEL \(Panel\)” on page 376](#) for descriptions of these tags.
- You can also nest regions within other regions.
- You can code only one LSTFLD tag within a REGION definition.

Processing

Table 66. Tags you can code within a REGION definition			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
DA	“DA (Dynamic Area)” on page 250	Multiple	No
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
DTACOL	“DTACOL (Data Column)” on page 269	Multiple	No
DTAFLD	“DTAFLD (Data Field)” on page 275	Multiple	No
GA	“GA (Graphic Area)” on page 295	Single	No

Table 66. Tags you can code within a REGION definition (continued)

Tag	Reference	Usage	Required
GENERATE	“GENERATE (Generate)” on page 298	Multiple	No
GRPHDR	“GRPHDR (Group Header)” on page 300	Multiple	No
INFO	“INFO (Information Region)” on page 317	Multiple	No
LSTFLD	“LSTFLD (List Field)” on page 341	Single	No
PNLINST	“PNLINST (Panel Instruction)” on page 396	Multiple	No
REGION	“LSTFLD (List Field)” on page 341	Multiple	No
SELFLD	“SELFLD (Selection Field)” on page 421	Multiple	No

Help panel

Table 67. Tags you can code within a REGION tag on a help panel

Tag	Reference	Usage	Required
DIVIDER	“DIVIDER (Area Divider)” on page 258	Multiple	No
INFO	“INFO (Information Region)” on page 317	Multiple	No
REGION	“REGION (Region)” on page 405	Multiple	No

Examples

Here is application panel markup that contains horizontal and vertical regions. The first two horizontal regions arrange the fields coded within them in a horizontal format. The third horizontal region arranges the selection field and the contents of the vertical region nested within it in a horizontal format. In this example, the INDENT attribute has been used to indent all fields formatted within a region 2 positions under the previous text. The ALIGN attribute has adjusted the default placement of fields in the last vertical region. [Figure 153 on page 409](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=chr25 TYPE='char 25'>
<VARCLASS NAME=chr12 TYPE='char 12'>
<VARCLASS NAME=chr10 TYPE='char 10'>
<VARCLASS NAME=chr9 TYPE='char 9'>
<VARCLASS NAME=chr8 TYPE='char 8'>
<VARCLASS NAME=chr2 TYPE='char 2'>

<VARLIST>
  <VARDCL NAME=name VARCLASS=chr25>
  <VARDCL NAME=date VARCLASS=chr8>
  <VARDCL NAME=addr VARCLASS=chr25>
  <VARDCL NAME=city VARCLASS=chr10>
  <VARDCL NAME=state VARCLASS=chr9>
  <VARDCL NAME=zip VARCLASS=chr12>
  <VARDCL NAME=level VARCLASS=chr2>
  <VARDCL NAME=graddate VARCLASS=chr2>
  <VARDCL NAME=major VARCLASS=chr10>
</VARLIST>
<PANEL NAME=region1 keylist=keylxml>Application Form
<TOPINST>Complete all of the fields below, then press Enter.
<AREA>
  <REGION INDENT=2>
    <REGION DIR=horiz>
      <DTACOL PMTWIDTH=10>
        <DTAFLD DATAVAR=name ENTWIDTH=25>Name
        <DTAFLD DATAVAR=date ENTWIDTH=8 DESWIDTH=10>Date
        <DTAFLDD>(mm/dd/yy)
      </DTACOL>
    </REGION>
```



```

    <DTAFLD DATAVAR=addr ENTWIDTH=25 PMTWIDTH=10>Address
  <REGION DIR=horiz>
    <DTAFLD DATAVAR=city PMTWIDTH=10 ENTWIDTH=25>City
    <DTAFLD DATAVAR=state PMTWIDTH=9 ENTWIDTH=2>State
    <DTAFLD DATAVAR=zip PMTWIDTH=12 ENTWIDTH=5>Zip code
  </REGION>
</REGION>
<DIVIDER TYPE=solid GUTTER=3>
<REGION DIR=horiz INDENT=2 ALIGN=no>
  <SELFLD NAME=level SELWIDTH=35 PMTWIDTH=25>Highest education level:
    <CHOICE>Some high school
    <CHOICE>High school graduate
    <CHOICE>Some college
    <CHOICE>College graduate
    <CHOICE>Some post-graduate work
    <CHOICE>Post-graduate degree
  </SELFLD>
<DIVIDER TYPE=solid>
<REGION>
  <GRPHDR FORMAT=none COMPACT STRIP>
    For applicants who are
    high school or college
    graduates:
  <REGION INDENT=2>
    <DTACOL PMTWIDTH=20>
      <DTAFLD DATAVAR=graddate ENTWIDTH=2>Year of graduation
      <DTAFLD DATAVAR=major ENTWIDTH=10>Field of study
    </DTACOL>
  </REGION>
</REGION>
</REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

Application Form

Complete all of the fields below, then press Enter.

Name . . . _____ Date . . . _____ (mm/dd/yy)
 Address _____
 City . . . _____ State . . . __ Zip code . . . _____

```

-----
Highest education level:
-- 1. Some high school
   2. High school graduate
   3. Some college
   4. College graduate
   5. Some post-graduate work
   6. Post-graduate degree

```

```

For applicants who are
high school or college
graduates:
  Year of graduation  __
  Field of study . . . _____

```

Enter a command ==> _____
 F1=Help F3=Exit F5=Display F6=Keyshelp F10=Actions
 F12=Cancel

Figure 153. Regions

Here is an example that shows the WIDTH and DEPTH attributes. The first vertical region width reserves the space required for the second vertical region, which is also scrollable. [Figure 154 on page 411](#) shows the formatted result.

REGION

```
<!DOCTYPE DM SYSTEM(
  <!entity sampvar2 sysem>
  <!entity sampabc system>)>
&sampvar2;

<PANEL NAME=region3 KEYLIST=keylxmlp>File-A-Case
<AB>
&sampabc;
</AB>
<CMDAREA>Enter a command
<TOPINST COMPACT>
  Type in client's name and case number (if applicable).
<TOPINST>Then select an action bar choice.
<REGION DIR=horiz>
  <REGION WIDTH=50>
    <DTAFLD DATAVAR=caseno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=21>Case No
    <DTAFLDD>(A 7-digit number)
    <DTAFLD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=8>Name
    <DTAFLDD>(Last, First, M.I.)
    <DTAFLD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
    <DIVIDER>
    <SELFLD NAME=casesel PMTWIDTH=11 PMTLOC=before SELWIDTH=38>Choose
    one of the following
    <CHOICE CHECKVAR=case MATCH=civ>Civil
    <CHOICE CHECKVAR=case MATCH=estate>Real Estate
    <CHOICE CHECKVAR=case MATCH=environ>Environmental
    </SELFLD>
  </REGION>
  <REGION DEPTH=10>
    <SELFLD TYPE=multi PMTWIDTH=24 SELWIDTH=26>
      Check type of offense
      <CHOICE NAME=patin HELP=patin CHECKVAR=val>Patent Infringement
      <CHOICE NAME=defa HELP=defame CHECKVAR=def>Defamation
      <CHOICE NAME=cont HELP=cont CHECKVAR=con>Breach of Valid Contract
      <CHOICE NAME=priv HELP=priv CHECKVAR=pri>Invasion of Privacy
      <CHOICE NAME=incr HELP=incr CHECKVAR=icr>Interference with
      Contractual Relations
      <CHOICE NAME=disp HELP=disp CHECKVAR=dis>Improper Disposal of
      Medical By-Products
      <CHOICE NAME=fraud HELP=fraud CHECKVAR=fra>Fraud
    </SELFLD>
  </REGION>
</REGION>
</PANEL>
```

```
File  Search  Help
-----
                        File-A-Case

Type in client's name and case number (if applicable).
Then select an action bar choice.

Case No  . .  _____ (A 7-digit number)      #SAREA37      #
Name  . . . .  _____ (Last,                 #            #
                                   First,            #            #
                                   M.I.)             #            #
Address  . .  _____                          #            #
                                           #            #
Choose one of the following -- 1. Civil            #            #
                                2. Real Estate       #            #
                                3. Environmental      #            #

Enter a command ==> _____
F1=Help      F3=Exit      F5=Display      F6=Keyshelp      F10=Actions
F12=Cancel
```

Here are the contents of the scrollable area:

```
)AREA SAREA37

Check type of offense
- Patent Infringement
- Defamation
- Breach of Valid Contract
- Invasion of Privacy
- Interference with Contractual Relations
- Improper Disposal of Medical By-Products
- Fraud

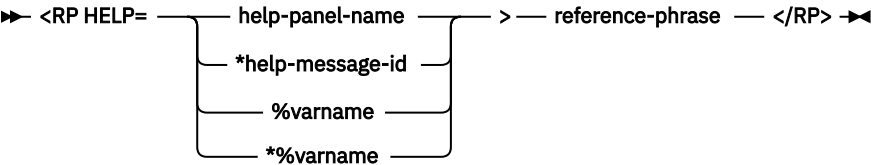
)AREA SAREA37
```

Figure 154. Using WIDTH and DEPTH attributes

RP (Reference Phrase)

The RP tag specifies a word or phrase within panel text that has additional help information associated with it.

Syntax



Parameters

HELP= help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the name of a panel that displays when the user requests help for the *reference-phrase*.

You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help on a choice and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See “HELP (Help Panel)” on page 303 for information about creating help panels. For information about creating messages, see [“MSG \(Message\)”](#) on page 352.

reference-phrase

This is the text of the phrase.

Comments

The RP tag specifies a word or phrase within panel text that has additional information associated with it.

The RP tag is valid as part of the text following these tags:

INFO tags

ATTENTION, CAUTION, DD, DDHD, DT, DTHD, FIG, FIGCAP, H2, H3, H4, LI, LINES, LP, NOTE, NT, P, PD, PT, WARNING, and XMP.

PANEL tags

BOTINST, CHOFLD, CHOICE, DTAFLD, DTAFLDD, GRPHDR, LSTCOL, LSTGRP, PNLINST, SELFLD, and TOPINST.

The *reference-phrase* is emphasized within the text of the panel to inform the user that additional information is available. The user positions the cursor on the reference phrase and presses F1=Help to obtain help on the phrase.

Each reference phrase is related to additional help panels in a manner similar to field-level help. The panel that appears when you request help from a reference phrase can also contain reference phrases.

Each *reference-phrase* results in one or more entries in the)HELP panel section. Multiple entries are required for phrases that span lines; a separate entry is created for each panel line used by the *reference-phrase*.

Restrictions

- The RP tag requires an end tag.

Processing

None.

Examples

Here is help panel markup that contains a reference phrase definition for the phrase, “lifetime warranty”. [Figure 155 on page 413](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM>

<HELP NAME=rp>HELP for Appliances
<AREA>
<INFO>
<p>In addition to our free delivery and installation program, we also
offer an exclusive <rp help=warrtyh>lifetime warranty</rp> on all
of our appliances.
</INFO>
</AREA>
</HELP>

<help name=warrtyh>Help for Lifetime Warranty
<AREA>
<INFO>
<p>Lifetime warranty covers the replacement of any part that breaks
or becomes non-functional while this product is used by the original
owner.
</INFO>
</AREA>
</HELP>

```

Help for Appliances

In addition to our free delivery and installation program, we also offer an exclusive **lifetime warranty** on all of our appliances.

F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

Figure 155. Reference phrase example

Accordingly, when the user selects the reference phrase **lifetime warranty**, the help panel specified by the HELP attribute (help=warrtyh) is displayed. [Figure 156 on page 413](#) shows the formatted result.

Help for Lifetime Warranty

Lifetime warranty covers the replacement of any part that breaks or becomes non-functional while this product is used by the original owner.

F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrvTopic	F8=NxtTopic
F10=PrvPage	F11=NxtPage	F12=Cancel

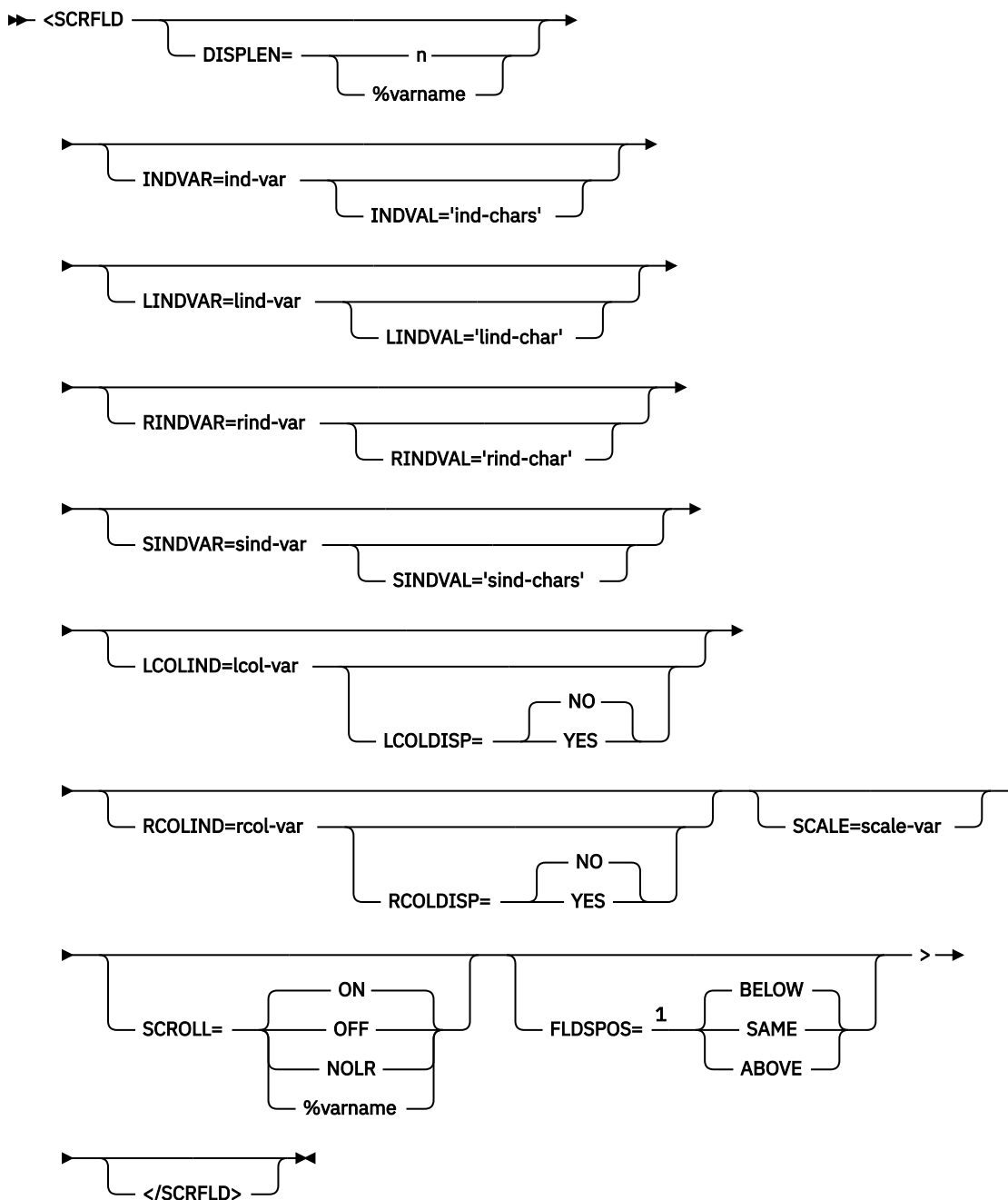
Figure 156. Reference phrase example

SCRFLD (Scrollable Field)

The SCRFLD tag defines a field on an application panel as being scrollable. The panel field is defined using either the DTAFLD or LSTCOL tag. The SCRFLD tag must be nested within either a DTAFLD or LSTCOL tag.

Using the SCRFLD tag causes the conversion utility to format an entry in the)FIELD section of the generated panel. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for a description of the)FIELD section.

Syntax



Notes:

¹ When the SCRFLD tag is nested in a DTAFLD tag, FLDSPOS can be BELOW or SAME. When the SCRFLD tag is nested in a LSTCOL tag, FLDSPOS can be BELOW or ABOVE.

Parameters

DISPLEN=*n* | %varname

This attribute is used to specify a length for the variable displayed in the scrollable field.

When DISPLEN=*n* is used, *n* specifies the initial length of the variable. *n* must be a value between 1 and 32 767.

%varname is a dialog variable that can contain a value between 1 and 32 767 to specify the initial length of the variable displayed in the scrollable field. After the panel is displayed %varname contains

the maximum of the length of the dialog variable displayed and the initial length specified. When the scrollable field has been defined using the LSTCOL tag, the length of the dialog variable displayed is the maximum of all instances on the current display for that variable.

INDVAR=ind-var

This attribute specifies the name of a dialog variable that contains the left and right scroll indicator.

ind-var is a 2-byte scroll indicator dialog variable that is updated with 1-byte indicators showing whether left and right scrolling can be performed.

INDVAL='ind-chars'

This attribute is used to override the default scroll indicator values of '-' and '+' where:

- + indicates you can scroll left or right
- indicates you can only scroll left
- + indicates you can only scroll right

ind-chars must be a 2-byte literal enclosed in quotes.

The INDVAL attribute can only be specified together with the INDVAR attribute.

LINDVAR=lind-var

This attribute specifies the name of a dialog variable that contains the left scroll indicator.

lind-var is a 1-byte left-scroll-indicator dialog variable that is updated with an indicator showing whether left scrolling can be performed. The LINDVAR attribute cannot be defined together with the INDVAR attribute.

LINDVAL='lind-char'

This attribute is used to override the default left-scroll-indicator value of '-'.

lind-char must be a 1-byte literal enclosed in quotes.

The LINDVAL attribute can only be specified together with the LINDVAR attribute.

RINDVAR=rind-var

This attribute specifies the name of a dialog variable that contains the right scroll indicator.

rind-var is a 1-byte right-scroll-indicator dialog variable that is updated with an indicator showing whether right scrolling can be performed. The RINDVAR attribute cannot be defined together with the INDVAR attribute.

RINDVAL='rind-char'

This attribute is used to override the default right-scroll-indicator value of '+'.

rind-char must be a 1-byte literal enclosed in quotes.

The RINDVAL attribute can only be specified together with the RINDVAR attribute.

SINDVAR=sind-var

This attribute specifies the name of a dialog variable that contains the separator scroll indicator.

sind-var is a separator scroll indicator dialog variable that is initialized with the value repeated for the length of the scrollable field displayed on the panel. If the field is scrollable to the left, the leftmost byte is the value of the left indicator (default: '<'). If the field is scrollable to the right, the rightmost byte is the value of the right indicator ('>').

SINDVAL='sind-chars'

This attribute is used to override the default separator scroll-indicator value of '<->'.

sind-chars must be a 3-byte literal enclosed in quotes.

The SINDVAL attribute can only be specified together with the SINDVAR attribute.

LCOLIND=icol-var

This attribute specifies the name of a dialog variable that contains the value of the left column position for the displayed scrollable field.

icol-var is a dialog variable that is updated when the field is scrolled to contain the value of the left column position. This dialog variable can be used to specify an initial left column position for the scrollable field.

Note: If the same *icol-var* is specified on multiple SCRFLD tags the associated panel fields scroll simultaneously. When the same *icol-var* is associated with multiple panel fields, the conversion utility only defines *icol-var* as a left column position indicator panel field for the first of those panel fields.

LCOLDISP=NO | YES

This attribute is used to specify whether the left column position indicator defined using the LCOLIND attribute is displayed on the panel.

When LCOLDISP=NO, the left column indicator is not generated as a panel field.

RCOLIND=rcol-var

This attribute specifies the name of a dialog variable that contains the value of the right column position for the displayed scrollable field.

rcol-var is a dialog variable that is updated when the field is scrolled to contain the value of the right column position.

Note: If the same *rcol-var* is specified on multiple SCRFLD tags the associated panel fields scroll simultaneously. When the same *rcol-var* is associated with multiple panel fields, the conversion utility only defines *rcol-var* as a right column position indicator panel field for the first of those panel fields.

RCOLDISP=NO | YES

This attribute is used to specify whether the right column position indicator defined using the LCOLIND attribute is displayed on the panel.

When RCOLDISP=NO, the right column indicator is not generated as a panel field.

SCALE=scale-var

This attribute specifies the name of a dialog variable that contains the scale indicator.

scale-var is a dialog variable that is updated with a scale line reflecting the current columns being displayed for the scrollable field.

SCROLL=ON | OFF | NOLR | %varname

This attribute is used to specify whether the field is scrollable or not.

When SCROLL=OFF, the field is not scrollable.

When SCROLL=NOLR, LEFT and RIGHT scrolling of the scrollable field is disabled.

%varname is used to specify the name of a scroll control dialog variable. This can be set to a value of ON or OFF to turn scrolling for the field either on or off. When SCROLL=NOLR, LEFT and RIGHT scrolling of the scrollable field is disabled.

FLDSPOS=BELOW | ABOVE | SAME

This attribute is used to specify where the scroll indicator panel fields are positioned in relation to the heading text for a table display field defined using the LSTCOL tag or in relation to the display field defined using the DTAFLD tag.

With FLDSPOS=BELOW, the conversion utility defines all scroll indicator panel fields for the scrollable table display field below the heading text or below the data field defined by the DTAFLD tag.

With FLDSPOS=SAME, the conversion utility attempts to define *ind-var*, *lind-var*, and *rind-var* for the data field on the same line as the data field. This option is not valid when the SCRFLD tag is nested within a LSTCOL tag.

With FLDSPOS=ABOVE, the conversion utility defines all scroll indicator panel fields for the scrollable table display field above the heading text. This option is not valid when the SCRFLD tag is nested within a DTAFLD tag.

Comments

The SCRFLD tag defines a field on an application panel as being scrollable. The panel field is defined using either the DTAFLD or LSTCOL tag. The SCRFLD tag must be nested within either a DTAFLD or LSTCOL tag.

Using the SCRFLD tag causes the conversion utility to format an entry in the)FIELD section of the generated panel. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for a description of the)FIELD section.

Scroll indicator fields

The conversion utility implicitly defines *ind-var*, *lind-var*, *rind-var*, and *sind-var* as scroll-indicator panel fields, and *scale-var* as a scale-indicator panel field. This topic describes where the scroll and scale indicator fields appear on the panel. Their position depends on whether the SCRFLD tag is nested within a DTAFLD or LSTCOL tag, and on the attributes specified on the SCRFLD tag.

Here is the order in which the scroll indicator fields are created by the conversion utility:

1. lcol_var
2. rcol_var
3. ind_var | lind_var
4. rind_var

Position of scroll indicator fields under the LSTCOL tag

Depending on the attributes specified on the SCRFLD tag, the conversion utility can create below or above the column heading text up to four panel lines containing scroll indicator fields. Here is a table that identifies the order in which the scroll indicator fields are created by the conversion utility, assuming FLDSPOS=BELOW is specified.

Table 68. Order in which scroll indicator fields are created when FLDSPOS=BELO is specified		
Relative Line from Column Heading ¹	Scroll Indicator Dialog Variables	Comments
+1	<i>ind-var</i> <i>lind-var</i> and <i>rind-var</i>	Displays either the left/right scroll indicator variable OR the left and right scroll indicator variables. The scroll indicator variables are positioned left-justified relative to the column.
+2	<i>lcol-var</i> and <i>rcol-var</i>	The left and right column position indicators are positioned left-justified relative to the column. The number of characters used for the left and right column indicators is one more than the larger of the dimension of the initial field display length or the dimension of the column width.
+3	<i>sind-var</i>	Separator scroll indicator field spans the width of the column.
+4	<i>scale-var</i>	Scale indicator field spans the width of the column.

¹If the associated scroll indicator dialog variables are not specified, the conversion utility uses the line for the next scroll indicator field.

Position of scroll indicator fields under the DTAFD tag

Depending on the attributes specified on the SCRFLD tag, the conversion utility can create below or on the same line as the data field up to four panel lines containing scroll indicator fields. The following table identifies the order in which the scroll indicator fields are created by the conversion utility, assuming FLDSPS=BELOW is specified.

The conversion utility defines, on the following panel lines, output fields for the scroll indicator variables specified using the SCRFLD tag attributes. Here is a table that identifies the order in which the scroll indicator fields are created by the conversion utility:

Table 69. Order in which scroll indicator fields are created		
Relative Line from DTAFD Field 1	Scroll Indicator Dialog Variables	Comments
+1	<i>scale-var</i>	Scale indicator field spans the width of the field.
+2	<i>sind-var</i>	Separator scroll indicator field spans the width of the field.
+3	<i>lcol-var</i> and <i>rcol-var</i>	The left and right column position indicators are positioned left-justified relative to the column. The number of characters used for the left and right column indicators is one more than the initial field display length.
+4	<i>ind-var</i> <i>lind-var</i> and <i>rind-var</i>	Displays either the left/right scroll indicator variable OR the left and right scroll indicator variables. The scroll indicator variables are positioned left-justified relative to the field.

1 If the associated scroll indicator dialog variables are not specified, the conversion utility uses the line for the next scroll indicator field.

When the SCRFLD tag is associated with a DTAFD tag that is immediately within a vertical region, scale and separator scroll indicators are not permitted.

Restrictions

- You must code the SCRFLD tag within a LSTCOL or DTAFD tag.

Processing

Table 70. Tags you can code within a SCRFLD definition			
Tag	Reference	Usage	Required
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is source file markup where the application panel contains two scrollable fields, the Address field and the Comments field. A scroll separator is displayed with the Address field and a scale line is displayed with the Comments field. [Figure 157 on page 420](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=date TYPE='char 8'>
<VARCLASS NAME=name TYPE='char 20'>
<VARCLASS NAME=addr TYPE='char 40'>
<VARCLASS NAME=prod TYPE='char 25'>
<VARCLASS NAME=comm TYPE='char 55'>

<VARLIST>
  <VARDCL NAME=curname VARCLASS=name>
  <VARDCL NAME=snamvar VARCLASS=name>
  <VARDCL NAME=fnamvar VARCLASS=name>
  <VARDCL NAME=sindvar VARCLASS=addr>
  <VARDCL NAME=addrvar VARCLASS=addr>
  <VARDCL NAME=prodvar VARCLASS=prod>
  <VARDCL NAME=scalvar VARCLASS=comm>
  <VARDCL NAME=commvar VARCLASS=comm>
</VARLIST>
<PANEL NAME=scrflld0 HELP=loghelp>Customer Feedback
<TOPINST>Complete the following fields, then press Enter.
<AREA>
  <DTACOL PMTWIDTH=15>
    <DIVIDER>
    <DTAFLD DATAVAR=curname USAGE=out ENTWIDTH=8 FLDSpace=27>Date
    <DTAFLDD>(Current Date)
    <DIVIDER>
    <DTAFLD DATAVAR=snamvar ENTWIDTH=20>Surname
    <DIVIDER>
    <DTAFLD DATAVAR=fnamvar ENTWIDTH=20>First Names
    <DIVIDER>
    <DTAFLD DATAVAR=addrvar ENTWIDTH=40 DESWIDTH=15>Address
    <DTAFLDD>(Optional)
    <SCRFLD DISPLEN=80 SINDVAR=sindvar>
    <DIVIDER>
    <DTAFLD DATAVAR=prodvar ENTWIDTH=25 DESWIDTH=25>Product
    <DTAFLDD>(Product Purchased)
    <DIVIDER>
    <DTAFLD DATAVAR=commvar ENTWIDTH=55>Comments
    <SCRFLD DISPLEN=110 SCALE=scalvar>
  </DTACOL>
</AREA>
<CMDAREA scrollvar=scrvar>Command
</PANEL>
```

Customer Feedback

Complete the following fields, then press Enter.

```

Date . . . . : 02/10/21                      (Current Date)
Surname . . . . Smith
First Names . . John Joseph
Address . . . . Apartment 10a, 100 Happiness Street, Ple (Optional)
                  ----->
Product . . . . Hammer                      (Product Purchased)
Comments . . . . An implement that has proved very useful for driving na
                  -----1-----2-----3-----4-----5-----+
Command ===> _____ Scroll ===> CSR
F1=Help   F3=Exit   F7=Up    F8=Down   F10=Left  F11=Right  F12=Cancel

```

Figure 157. List field

Here is source file markup that uses the LSTFLD and LSTCOL tags to display the data in an ISPF table. The SCRFLD tag is used to display the Customer and Comments data in scrollable fields. Left and right column indicators are displayed in the column headings for the Customer and Comments data. A separator scroll indicator is also displayed in the heading for the Customer column. A scale indicator is displayed in the heading for the Comments column. [Figure 158 on page 421](#) shows the formatted result.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=date TYPE='char 8'>
<VARCLASS NAME=cust TYPE='char 15'>
<VARCLASS NAME=prod TYPE='char 15'>
<VARCLASS NAME=comm TYPE='char 30'>

<VARLIST>
  <VARDCL NAME=datevar VARCLASS=date>
  <VARDCL NAME=custvar VARCLASS=cust>
  <VARDCL NAME=prodvar VARCLASS=prod>
  <VARDCL NAME=commvar VARCLASS=comm>
</VARLIST>

<PANEL NAME=scrfld1 HELP=loghelp>Customer Feedback Display
<AREA>
<LSTFLD SCROLLVAR=scr1amt SCRHELP=scrhelp>
  <LSTCOL DATAVAR=datevar USAGE=out COLWIDTH=8>Date
  <LSTCOL DATAVAR=custvar USAGE=out COLWIDTH=15>Customer
    <SCRFLD DISPLEN=50 SINDVAR=sindvar LCOLIND=cuslcol LCOLDISP=YES
      RCOLIND=cusicol RCOLDISP=YES>
  <LSTCOL DATAVAR=prodvar USAGE=out COLWIDTH=15>Product
  <LSTCOL DATAVAR=commvar USAGE=out COLWIDTH=30>Comments
    <SCRFLD DISPLEN=110 SCALE=scalvar LCOLIND=comlcol LCOLDISP=YES
      RCOLIND=comicol RCOLDISP=YES>
  </LSTFLD>
</AREA>
<CMDAREA>Command
</PANEL>

```

```

Customer Feedback Display
Row 1 to 6 of 6

Date      Customer      Product      Comments
1      15
----->      ----+----1----+----2----+----3

03/02/25  Big Bang Demoli  Jackhammer  We've used this piece of equip
03/02/25  Carpenter, Scot  Chisel      I'm able to shape pieces of wo
03/02/26  Acme Building C  Cement mixer Our bricklayers make heavy use
03/02/28  Smith, John Jos  Hammer     An implement that has proved v
03/02/28  Hole, Doug       Shovel      Is ideally suited to our holis
03/03/03  Picker, Sherry   Folding ladder This tool is highly recommende
***** Bottom of data *****

Command ==> _____ Scroll ==> CSR
F1=Help    F3=Exit    F7=Up      F8=Down    F10=Left   F11=Right  F12=Cancel
F13=Help   F15=Exit   F24=Cancel

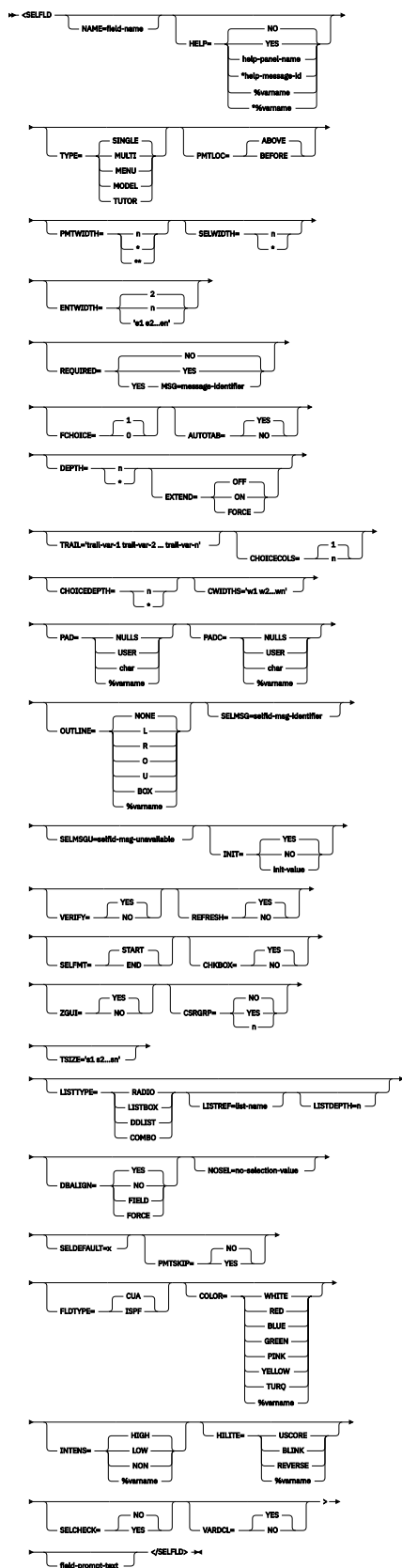
```

Figure 158. List variable

SELFID (Selection Field)

The SELFID tag defines a field that includes a list of choices.

Syntax



Parameters

NAME=field-name

This attribute specifies the name for the selection field. The *field-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

The NAME field is required if TYPE=SINGLE because the selection field name is used as the input field for single-choice selection fields. The NAME field is ignored if TYPE=MULTI.

The NAME field is optional for TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR. If present, it is used in place of the command field name in the construction of the option selection statement. However, because the input field is the command line, you must provide panel logic using the SOURCE tag to ensure that the selection choice is placed in the NAME field.

For single-choice selection fields, the *field-name* can be used to position the cursor on the field using the CURSOR attribute of the enclosing PANEL tag or the CURSOR parameter of the DISPLAY service call. In addition, you can use the *field-name* to position a pop-up using the POPLOC parameter of the ADDPOP service call.

HELP=NO | YES | help-panel-name | *help-message-id | %varname | *%varname

This attribute specifies the help action taken when the user requests help for a selection field. This is field-level help.

When HELP=YES, control is returned to the application. You can specify either a help panel or a message identifier. If a message identifier is used, it must be prefixed with an asterisk (*).

The help attribute value can be specified as a variable name. When **%varname** is coded, a panel variable name is created. When ***%varname** is coded, a message variable name is created.

If the user requests help on a field and no help is defined, the extended help panel is displayed. If an extended help panel is not defined for the panel, the application or ISPF tutorial is invoked.

The *help-panel-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

See “HELP (Help Panel)” on page 303 for information about creating help panels. For information about creating messages, see “MSG (Message)” on page 352.

Note: This attribute is valid only when TYPE=SINGLE.

TYPE=SINGLE | MULTI | MENU | MODEL | TUTOR

This attribute specifies whether the selection field is single-choice, multiple-choice, an ISPF selection menu, an edit model selection menu, or a tutorial selection menu.

Single-choice selection fields allow the user to select only one choice from the selection list. Choices in a single-choice selection field appear with sequential numbers before each choice. An input field precedes the first choice in the selection field.

Multiple-choice selection fields allow the user to select one or more choices from the selection list. Choices in a multiple-choice selection field appear with a single character input field in front of each choice.

The use of TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR is allowed only when the MENU keyword has been specified on the PANEL tag. ISPF selection menu, edit model, or tutorial selection menu fields are formatted in a manner similar to single-choice selection fields. Choices appear with sequential numbers in front of each choice and the user may select only one choice from the selection list. With these options, the command line is used as the entry choice field. Because the HELP attribute on the SELFID tag is not valid when TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR, help for selection menu or edit model menu choices must be entered on the CMDAREA tag.

Note: Because the selection menu, edit model menu, or tutorial menu panel uses the command line for choice selection, a command area is required. The conversion utility automatically generates a command area if no CMDAREA tag is provided.

PMTLOC=ABOVE | BEFORE

This attribute specifies whether the *field-prompt-text* appears above or in front of the selection field.

PMTWIDTH=n | * | **

This attribute specifies the number of bytes to be used by the prompt for the selection field. When you specify PMTWIDTH=*, the conversion utility uses the length of the prompt text as the prompt width. When you specify PMTWIDTH=**, the conversion utility uses the maximum available space as the prompt width. If any prompt is longer than this value, the prompt is word-wrapped to fit on multiple lines. The minimum value is 0 and the maximum is the remaining available panel (or region) value. This value overrides the PMTWIDTH value on an enclosing DTACOL tag.

SELWIDTH=n | *

This attribute specifies the number of bytes used for the choices in the selection field. It is useful for defining a consistent appearance for the selection choices. If you do not specify the SELWIDTH parameter on the SELFLD tag, the SELWIDTH parameter on any enclosing DTACOL tag is used. If you do not specify a SELWIDTH value and SELWIDTH is not specified on an enclosing DTACOL tag, then the remaining available width of the panel (or current region) determines the width used to format the choice text. If the SELWIDTH value is specified as “*”, the conversion utility uses the remaining available width.

If the width required by the *choice-description-text* and its entry-field exceeds the value specified for SELWIDTH, the text is word-wrapped to multiple lines.

Note: Because all of the remaining space is used if no SELWIDTH attribute is provided or if SELWIDTH=“*” is coded, you should specify a SELWIDTH value for fields defined:

- With PMTLOC=BEFORE, because PMTWIDTH is not part of the space reserved by SELWIDTH.
- Within a horizontal region if additional fields are to be formatted to the right of the SELFLD section.

SELWIDTH for selection fields defined within a horizontal region if additional fields are to be formatted to the right of the SELFLD section.

The width specified for a single-choice selection field should include all or a portion of the *choice-description-text* plus 8-13 positions, determined in this way:

- The choice selection entry-field (1-3 characters)
- The entry-field 3270 attributes (2 characters)
- The choice-number inserted by the conversion utility (3-5 characters)
- The 3270 attributes that enclose the *choice-description-text* (2 characters).

The width of a multiple-choice selection field should include all or a portion of the *choice-description-text* plus 5 positions, determined in this way:

- The choice selection entry-fields (1 character)
- The entry-field 3270 attributes (2 characters)
- The 3270 attributes that enclose the *choice-description-text* (2 characters).

The width specified for a menu-choice, model-choice, or tutorial-choice selection field should include all or a portion of the choice-description-text plus 4-19 positions, determined in this way:

- The choice selection entry-field (1-16 characters)
- The entry-field 3270 attribute (1 character)
- The 3270 attributes that enclose the choice-description-text (2 characters).

ENTWIDTH=2 | n | 'e1 e2...en'

This attribute is valid only when TYPE=SINGLE, TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR.

Multiple ENTWIDTH values can be used when TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR. For these types of selection lists, the ENTWIDTH is used only to format the amount of space used by the selection character(s). The multiple width format is used when CHOICECOLS is greater than 1 to customize the width required for each column of choices. If the number of ENTWIDTH values is less than the number of columns, the last (or only) ENTWIDTH value is used for the remaining columns. If more ENTWIDTH values are supplied than there are columns of choices, the excess ENTWIDTH values are ignored.

When TYPE=SINGLE and the value of LISTTYPE is not COMBO, ENTWIDTH specifies the number of bytes used for both the entry field and the space between the selection identifier and the selection text. The default width value is 2. The minimum width value is 1, which can be specified for any single-choice selection list. The maximum width value (when LISTTYPE is not COMBO) is 3, which can be specified for selection lists within a scrollable panel area. The width of 3 is provided for use when the number of CHOICE tags exceeds 99.

When LISTTYPE=COMBO, the maximum ENTWIDTH value is 2 bytes less than the SELWIDTH value.

Note: A width of 1 should only be used when the total number of CHOICE tags is less than 10. The conversion utility discards choices which cannot be selected with the specified entry width.

When TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR, the command area is used as the input field and the ENTWIDTH value is used only to determine the spacing between the selection identifier and the selection text. The maximum ENTWIDTH value for these types is 16.

REQUIRED=NO | YES

This attribute indicates if the field requires input.

If REQUIRED=YES is coded, a VER(variable, NONBLANK) statement is built by ISPD TLC and placed in the)PROC section of the generated ISPF panel.

Note: This attribute is valid only when TYPE=SINGLE.

MSG=message-identifier

This attribute specifies the message that is displayed when the user does not choose a selection (defined with the REQUIRED attribute). If you do not specify a *message-identifier*, ISPF displays a default message.

If you specify the MSG attribute and REQUIRED=YES, a VER(variable, NONBLANK, MSG=message-identifier) statement is built by ISPD TLC and placed in the)PROC section of the generated ISPF panel. If you specify the MSG attribute and REQUIRED=NO (the default), the conversion utility issues a warning message.

FCHOICE=1 | 0

The FCHOICE attribute controls the starting choice number for TYPE=SINGLE, TYPE=MENU, TYPE=MODEL or TYPE=TUTOR. When FCHOICE=0, the first choice is the number 0.

Note: This attribute is valid only when TYPE=SINGLE, TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR.

AUTOTAB=YES | NO

When AUTOTAB=YES, the cursor moves to the next field capable of input when the user enters the last character in this field. If no other field capable of user input exists on the panel, the cursor returns to the beginning of this field.

Note: This attribute is valid only when TYPE=SINGLE.

DEPTH=n | *

This attribute defines the minimum size of a scrollable selection list. If DEPTH is not specified, the selection list is not scrollable. If the DEPTH value is specified as "*", the conversion utility reserves the remaining available panel depth. When EXTEND=OFF, the minimum depth is 2. When EXTEND=ON, the minimum depth is 1. The DEPTH attribute is ignored when LISTTYPE=COMBO.

EXTEND=OFF | ON | FORCE

This attribute defines the runtime display size for the scrollable list area. If EXTEND=ON is specified, the panel definition is expanded from the minimum DEPTH to the size of the logical screen. Only one EXTEND=ON attribute value is allowed on a panel. The first tag (AREA, DA, GA, REGION, SELFLD) with EXTEND=ON is accepted; the EXTEND attribute on any subsequent tag is ignored.

If the EXTEND attribute is specified without the DEPTH attribute, a warning message is issued and the EXTEND attribute is ignored. The EXTEND attribute is ignored when LISTTYPE=COMBO.

If you intend to display the panels in a pop-up window, it is recommended that you code EXTEND=OFF.

If EXTEND=FORCE is specified within a horizontal area or region, the EXTEND(ON) keyword is added to the scrollable area attribute statement in the)ATTR panel section. The conversion utility issues a message to advise of a potential display error if other panel fields are formatted on or after the last defined line of the scrollable area.

TRAIL='trail-var-1 trail-var-2 ... trail-var-n'

This attribute specifies variable name(s) that the application uses to obtain the TRAIL information created by menu or model selection processing.

Each trail variable specified must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

Note: This attribute is valid only when TYPE=MENU or TYPE=MODEL.

CHOICECOLS=1 | n

This attribute specifies the number of columns to format with the CHOICE items. The default is 1. The CHOICECOLS attribute is ignored when LISTTYPE=COMBO.

CHOICEDEPTH=n | *

This attribute specifies the number of CHOICE entries to be placed in each column. The minimum CHOICEDEPTH value is 1. The normal maximum and default is the remaining panel depth. If the DEPTH attribute has been specified on the SELFLD tag, or an enclosing REGION or AREA tag, (and the corresponding tag attribute value for EXTEND is OFF) the most recently specified depth value is used as the maximum and default value. You may specify CHOICEDEPTH=“*” which tells the conversion utility to calculate the column depth based on the total number of CHOICE tags and the number of columns specified by the CHOICECOLS attribute.

If more CHOICE entries are specified than can be formatted in the available number of columns specified by the CHOICECOLS attribute, the remaining CHOICE entries are placed in the rightmost (or only) available column for the current SELFLD tag. The CHOICEDEPTH attribute is ignored when LISTTYPE=COMBO.

CWIDTHS='w1 w2...wn'

This attribute specifies the number of bytes to be allocated for each column of CHOICE entries. The 'w1 w2...wn' notation provides the number of bytes for each column. You may use an asterisk or a number combined with an asterisk to specify a proportional allocation of column space. For example, the specification of '2* * 3*' for 3 columns would result in a space calculation based on 6 units, with 2 units allocated to column 1, 1 unit allocated to column 2, and 3 units allocated to column 3. If more columns have been specified by CHOICECOLS than are accounted for by CWIDTHS, the remaining space is divided evenly between the remaining columns. If CWIDTHS is not specified, the available formatting space is divided evenly based on the CHOICECOLS value. The CWIDTHS attribute is ignored when LISTTYPE=COMBO.

PAD=NULLS | USER | char | %varname

This attribute specifies the pad character for initializing the field. You can define this attribute as a variable name preceded by a “%”.

Note: This attribute is valid only when TYPE=SINGLE.

PADC=NULLS | USER | char | %varname

This attribute specifies the conditional padding character to be used for initializing the field. You can define this attribute as a variable name preceded by a “%”.

Note: This attribute is valid only when TYPE=SINGLE.

OUTLINE=NONE | L | R | O | U | BOX | %varname

This attribute provides for displaying lines around the field on a DBCS terminal. You can define this attribute as a variable name preceded by a “%”.

Note: This attribute is valid only when TYPE=SINGLE.

SELMSG=selfld-msg-identifier

This attribute specifies the message that is displayed when an invalid single-choice entry is selected.

SELMSGU=selfld-msg-unavailable

This attribute specifies the message that is displayed when an unavailable single-choice entry is selected.

INIT=YES | NO | init-value

This attribute controls the single-choice and multi-choice selection field variables initialization in the panel)INIT section. When INIT = NO, the variables are not initialized to blank. When TYPE = SINGLE, you can alternatively provide a valid choice selection by specifying INIT = init-value.

CHOICE tag CHECKVAR processing can override the INIT value.

VERIFY=YES | NO

This attribute controls the single-choice verification and menu-choice, model-choice, or tutor-choice selection logic generation in the panel)PROC section. When TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR, VERIFY=NO bypasses the creation of the ZSEL statement. You can provide a replacement ZSEL statement with the <SOURCE> tag.

REFRESH=YES | NO

This attribute controls the creation of the REFRESH statement in the panel)REINIT section for multi-choice selection lists.

SELFMT=START | END

This attribute controls the placement of the choice selection character(s) within the width specified by ENTWIDTH. The default is to left justify the choice selection character(s).

Note: This attribute is valid only when TYPE=SINGLE, TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR.

CHKBX=YES | NO

This attribute controls the creation of panel keywords that enable check boxes on a client that is using the JSON API. The default value is YES.

The CHKBX attribute is not valid and is ignored for single-choice, menu-choice, and model-choice selection lists.

If the conversion utility has been invoked with the NOGUI option, specifying CHKBX=YES on the SELFID tag overrides the invocation option so that check-box controls are generated.

ZGUI=YES | NO

This attribute controls the creation of the "VGET (ZGUI)" statement in the panel)INIT section for multi-choice selection lists that specify the "&multipmt" ENTITY as *field-prompt-text*.

CSRGRP=NO | YES | n

The CSRGRP attribute is valid only when TYPE=MULTI and CHKBX=YES (either specified or defaulted). When CSRGRP=YES, the conversion utility generates a cursor group number to be used for this selection list. When CSRGRP=n, the number provided is used for the CHOICE fields within this SELFID tag.

TSIZE='s1 s2...sn'

The TSIZE attribute provides the number of bytes to indent multiple lines of CHOICE text. Multiple TSIZE values can be used to provide unique indentation amounts for multiple column lists (when CHOICECOLS is greater than 1). If the number of TSIZE values is less than the number of columns, the last (or only) TSIZE value is used for the remaining columns. If more TSIZE values are supplied than there are columns of choices, the extra TSIZE values are ignored.

LISTTYPE=RADIO | LISTBOX | DDLIST | COMBO

This attribute controls the creation of panel keywords that enable single-choice selection lists to be displayed with radio buttons on a client that is using the JSON API.

LISTTYPE=LISTBOX, LISTTYPE=DDLIST, and LISTTYPE=COMBO are accepted in order to support existing DTL source files that use them. However, they no longer affect the displayed panel.

Note: This attribute is valid only when TYPE=SINGLE.

LISTREF=list-name

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

LISTDEPTH=n

The LISTDEPTH attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

DBALIGN=YES | NO | FIELD | FORCE

This attribute defines the DBALIGN value. DBALIGN is used only for DBCS language conversions when PMTLOC=ABOVE and the DBALIGN invocation option is specified.

When DBALIGN=YES, and the field-prompt-text starts with a DBCS character or a single-choice or multi-choice selection list definition does not include field-prompt-text, the entry field for the choice is shifted 1 position to the right.

When DBALIGN=NO, no alignment adjustment is made.

When DBALIGN=FIELD, the entry field is shifted but no adjustment is done for the prompt. The FORCE and FIELD values are useful when alignment is required with other SELF or DTAFLD tags.

When DBALIGN=FORCE, the entry field is shifted and the field-prompt-text is also adjusted to match even if the field-prompt-text starts with a single byte character.

NOSEL=no-selection-value

This attribute provides a value to be placed the CHECKVAR variable (specified by the CHOICE tag) when no selection is chosen from the available list.

If REQUIRED=YES is specified, a message is issued and NOSEL is ignored.

If no CHOICE tag specifies a CHECKVAR attribute, the NOSEL attribute is ignored.

Note: This attribute is valid only when TYPE=SINGLE.

SELDEFAULT=x

This attribute is used to provide a default choice selection when TYPE=SINGLE, MENU, MODEL, or TUTOR. The value x must be a valid choice selection. If no selection is made by the user, the default value is returned to the application.

PMTSKIP=NO | YES

This attribute is used for horizontal formatting of input fields. When PMTSKIP=YES, and the previous DTAFLD definition includes the NOENDATTR attribute, the cursor moves past the prompt text to the input field when the user enters the last character in the previous field. If there is no other input field on the panel, the cursor returns to the first input field on the panel.

FLDTYPE=CUA | ISPF

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

COLOR=WHITE | RED | BLUE | GREEN | PINK | YELLOW | TURQ | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

INTENS=HIGH | LOW | NON | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

HILITE=USCORE | BLINK | REVERSE | %varname

This attribute is accepted in order to support existing DTL source files that use it. However, it no longer affects the displayed panel.

SELCHECK = NO | YES

This attribute is used with menu-choice selection to specify that panel logic is to be included in selection processing to check for valid selection entries. For example, a message is issued if a period (.) or a period followed by data (.xxx) is entered as a selection choice.

Note: This attribute is valid only when TYPE=MENU.

VARDCL=YES | NO

When VARDCL=NO the field name is not checked to the declared variable information provided with the VARCLASS and VARDCL tags.

Note: This attribute is only valid when TYPE=SINGLE.

field-prompt-text

This is the prompt text for the selection field. The prompt text can appear in front of or above the field, based on the value assigned to the PMTLOC attribute.

Multi-choice selections can be displayed as check boxes on a client that is using the JSON API. To support both host and JSON API client forms of multi-choice prompt text, a special pre-defined ENTITY name of "&multiplt" may be specified as the *field-prompt-text*. When the panel is displayed, the *field-prompt-text* is

```
Enter "/" to select option
```

(or its translated equivalent) for host display or

```
Check box to select option
```

(or its translated equivalent) for JSON API client display. The panel definition should specify a PMTWIDTH value large enough to format the prompt as a single line. If there is insufficient space to present the entire *field-prompt-text*, it is truncated to fit the available space.

Comments

The SELFLD tag defines a selection field that includes a list of choices. CHOICE tags coded within the SELFLD definition define the choices for the selection field.

The TYPE attribute of the SELFLD tag determines how the choices appear. If TYPE=SINGLE, the SELFLD NAME attribute is used as the selection input field. If TYPE=MULTI, the CHOICE NAME attribute is used as the selection input field for each choice. If TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR, the command line is used as the selection input field.

When a selection list is formatted as a scrollable list:

- The multi-choice list entry field scrolls with the choice descriptions.
- The single-choice entry field is formatted beside the choice list and remains visible when the choice descriptions scroll.
- Choice descriptions that are formatted in multiple columns (CHOICECOLS and CHOICEDEPTH attributes specified) result in a separate scrollable area for each column.

Note: If you specify the CMDAREA tag within your DTL source file, it must appear before the SELFLD tag when TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR and CHECKVAR or UNAVAIL attributes are specified on nested CHOICE tags.

If you specify the CMDAREA tag within your DTL source file, it must appear before the SELFLD tag when DEPTH=* is specified. The SELFLD tag DEPTH may have to be adjusted to allow for additional lines which result from tags present within the panel definition following the end SELFLD tag.

Restrictions

- The SELFLD tag requires an end tag.
- You must code the SELFLD tag within an AREA, DTACOL, REGION, or PANEL definition. See [“AREA \(Area\)” on page 189](#), [“DTACOL \(Data Column\)” on page 269](#), [“REGION \(Region\)” on page 405](#), and [“PANEL \(Panel\)” on page 376](#) for descriptions of these tags.
- Single-choice selection fields (the default TYPE value) should have an associated VARDCL definition for the *field-name* specified with the NAME attribute. See [“VARDCL \(Variable Declaration\)” on page 449](#) for a complete description of this tag.
- If both PAD and PADC have been specified, PAD is ignored and PADC is used.

- When a "%varname" notation is found on any of the attributes that allow a variable name, the "%varname" entry must follow the standard naming convention described in [“Rules for “%variable” names”](#) on page 179.
- You should code a CMDAREA on any panel that contains a SELFLD definition that specifies TYPE=MENU, TYPE=MODEL, or TYPE=TUTOR. If you do not include the CMDAREA tag, the conversion utility inserts one and issues a message, unless the PANEL tag specifies CMDLINE=NO.
- Only one menu-choice or model-choice list is formatted for any panel. If multiple menu-choice or model-choice lists are specified, the first one is formatted as a menu; subsequent menu-choice or model-choice lists are formatted as single-choice lists.

Processing

<i>Table 71. Tags you can code within a SELFLD definition</i>			
Tag	Reference	Usage	Required
CHDIV	“CHDIV (Choice Divider)” on page 207	Multiple	No
CHOICE	“CHOICE (Selection Choice)” on page 226	Multiple	No
COMMENT	“COMMENT (Comment)” on page 245	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SOURCE	“SOURCE (Source)” on page 435	Multiple	No

Examples

Here is application panel markup that contains two selection fields. The first selection field is a single-choice selection field with the prompt text located in front of the selection field. The single-choice selection field can be preselected depending on the value assigned to the variable *card*.

The second selection field is a multiple-choice selection field with the prompt text located above the selection field. Choices within this field may be preselected depending on the value assigned to the CHECKVAR attribute variable specified on the respective CHOICE tags.

```

<!DOCTYPE DM SYSTEM(
  <!entity sampvar1 sysem>
  <!entity sampabc system>)>
&sampvar1;

<PANEL NAME=selfld3 KEYLIST=keylxmlp>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST>Type in patron's name and card number (if applicable).
<TOPINST>Then select an action bar choice.
<AREA>
  <DTAFLD DATAVAR=curdate PMTWIDTH=12 ENTWIDTH=8 USAGE=out>Date
  <DTAFLD DATAVAR=cardno PMTWIDTH=12 ENTWIDTH=7 DESWIDTH=25>Card No
    <DTAFLDD>(A 7-digit number)
  <DTAFLD DATAVAR=name PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25>Name
    <DTAFLDD>(Last, First, M.I.)
  <DTAFLD DATAVAR=address PMTWIDTH=12 ENTWIDTH=25>Address
  <DIVIDER>
  <REGION DIR=horiz>
    <SELFID NAME=cardsel PMTWIDTH=30 SELWIDTH=40
      entwidth=1 required=yes autotab=yes>
      Choose one of the following
      <CHOICE CHECKVAR=card MATCH=new>New
      <CHOICE CHECKVAR=card MATCH=irenew>Renewal
      <CHOICE CHECKVAR=card MATCH=replace>Replacement
    </SELFID>
    <SELFID TYPE=multi PMTWIDTH=30 SELWIDTH=36
      depth=5 init=no>
      Check valid branches
      <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
      <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
      <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
      <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
      <CHOICE NAME=city HELP=ctyhlp CHECKVAR=cty>City Branch
      <CHOICE NAME=cnty HELP=cnthlp CHECKVAR=cnt>County Branch
    </SELFID>
  </REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

Figure 159 on page 432 shows the formatted result.

FileSearchHelp

Library Card Registration

Type in patron's name and card number (if applicable).

Then select an action bar choice.

Date . . . : -----

Card No . . : ----- (A 7-digit number)

Name . . . : ----- (Last, First, M.I.)

Address . . : -----

Choose one of the following

1. New

2. Renewal

3. Replacement

Check valid branches

#SAREA37

#

#

#

#

Enter a command ==> -----

F1=HelpF3=ExitF5=DisplayF6=KeyshelpF10=Actions

F12=Cancel

Here are the contents of the scrollable area:

)AREA SAREA37

- North Branch

- South Branch

- East Branch

- West Branch

- City Branch

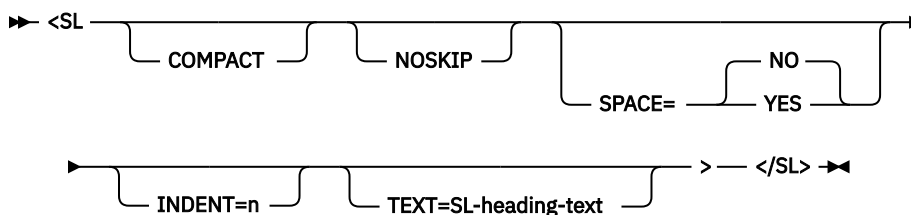
- County Branch

)AREA SAREA37

Figure 159. Selection fields

Here is an example that shows the creation of an ISPF selection menu. The FCHOICE attribute specifies that the first selection number is 0. The choice selection for Exit is specified on the CHOICE tag. The ACTION tag for the Exit choice selection specifies both the RUN and TYPE attributes because RUN is required on the ACTION tag and TYPE is necessary to specify the ISPF selection for the generated ZSEL panel statement.

Syntax



Parameters

COMPACT

This attribute causes the list to be formatted without a blank line between the list items.

NOSKIP

This attribute causes the list to format without creating a blank line before the first line of the list.

SPACE=NO | YES

The SPACE attribute controls the indentation space for the list item. When the SPACE attribute is not specified on the LI tag, the SPACE attribute from the SL tag is used to set the indentation space for the nested LI tag *item-text*.

When SPACE=YES, the indentation is set to 3 spaces. When SPACE=NO (or SPACE is not specified), the indentation is set to 4 spaces.

The SPACE attribute can be used to control the alignment of list items when the first word of some list items is a DBCS word preceded by a shift-out character and the first word of other list items is a SBCS word.

INDENT=n

This attribute specifies that the list be indented from the current left margin.

TEXT=SL-heading-text

This attribute causes the list to format with a heading line containing the *SL-heading-text*.

Comments

The SL tag defines a simple list of items within an information region.

Simple lists are indented lists, with no bullets, dashes, or hyphens preceding the list items. Nested lists indent four spaces to the right of the left margin of the list that contains them.

Note: The SPACE attribute does not affect the indentation of nested lists.

The conversion utility adds a blank line before the first item in the list.

Use the LI tag to denote each list item. See [“LI \(List Item\)” on page 325](#) for more information on the LI tag.

Restrictions

- The SL tag requires an end tag.
- You must code the SL tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 72. Tags you can code within an SL definition			
Tag	Reference	Usage	Required
LI	“LI (List Item)” on page 325	Multiple	No
LP	“LP (List Part)” on page 330	Multiple	No

Examples

Here is help panel markup that contains two simple lists. The second simple list is compact, and is nested within the first list. [Figure 161 on page 435](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=s1 WIDTH=40 DEPTH=22>Help for ShelfBrowse
<AREA>
<INFO>
  <P>Using ShelfBrowse, you can locate the following items:
  <SL>
    <LI>Audiotapes
    <LI>Books
    <LI>Periodicals
      <SL COMPACT>
        <LI>Newspapers
        <LI>Magazines
      </SL>
    <LI>Reference material
    <LI>Videotapes
  </SL>
</INFO>
</AREA>
</HELP>
```

```
Help for ShelfBrowse

Using ShelfBrowse, you can locate the
following items:

Audiotapes

Books

Periodicals

  Newspapers
  Magazines

Reference material

Videotapes

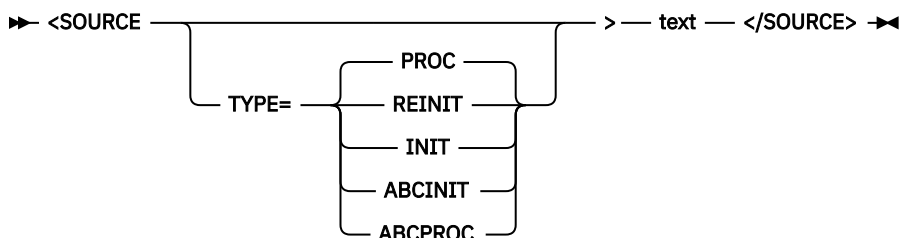
F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

Figure 161. Simple list

SOURCE (Source)

The SOURCE tag defines ISPF panel logic statements within an application panel.

Syntax



Parameters

TYPE=PROC | REINIT | INIT | ABCINIT | ABCPROC

This attribute specifies the panel section that is updated with the SOURCE tag *text*.

text

This is the unformatted ISPF panel statement.

Comments

The SOURCE tag defines ISPF panel statements within an application panel.

Lines of text from a SOURCE tag that follows an AREA, CHOICE, DA, DTACOL, DTAFLD, HELP, LSTCOL, LSTFLD, LSTGRP, PANEL, REGION, or SELFLD tag are added to the)INIT,)REINIT, or)PROC panel section when encountered in the DTL source file.

For example, if a SOURCE tag follows the DTAFLD tag, any logic or other entries normally generated by DTAFLD would be completed before the lines within SOURCE are added.

The use of a SOURCE tag within a SELFLD tag results in the placement of the SOURCE tag lines after any logic created by the previous CHOICE tag. Additional)INIT,)REINIT, or)PROC section entries may be added when the end SELFLD tag is processed. You can control the placement of the SOURCE tag entries by nesting the SELFLD tag definition within a DTACOL tag, and placing the SOURCE tag definition either before or after the SELFLD tag definition.

Lines of text from a SOURCE tag within an action bar definition are added to:

-)ABCINIT *following* all other generated statements for that PDC tag.
-)ABCPROC *before* any other generated statements for that PDC tag.

SOURCE tags within an action bar definition must specify the TYPE as ABCINIT or ABCPROC. SOURCE tags that follow the other listed tags cannot specify TYPE as ABCINIT or ABCPROC.

When the SOURCE tag is coded within a GENERATE tag, the TYPE attribute is ignored. TYPE is automatically determined from the placement of the GENERATE tag within the DTL source file.

If the length of any line exceeds the record length of the output panel file, the conversion utility truncates the line and issues a warning message.

Text found between the SOURCE and SOURCE end tags is placed in the specified panel section as coded; that is, no formatting except entity substitution is performed. To refer to an entity within <SOURCE> tag text, the entity name is preceded by a percent (%) instead of an ampersand (&). Using the percent (%) sign avoids conflict with variable names. A valid percent sign can be specified as "%amp;" to avoid an "entity not found" message. For example, you would refer to the TSO command "%xyz" as "%amp;xyz".

Restrictions

- The SOURCE tag requires an end tag.

- You must code the SOURCE tag within an ABC, AREA, CHOICE, DA, DTACOL, DTAFLD, GENERATE, HELP, INFO, LSTCOL, LSTFLD, LSTGRP, PANEL, PDC, REGION, or SELFLD tag definition.

Processing

None.

Examples

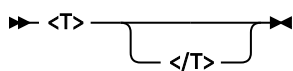
```
<!DOCTYPE DM SYSTEM(
<!entity sampvar1 sysem>
<!entity sampabc system>)>
&sampvar1;

<PANEL NAME=source1 KEYLIST=keyl&mp>Library Card Registration
<AB>
&sampabc;
</AB>
<TOPINST>Type in patron's name and card number (if applicable)
<TOPINST>Then select an action bar choice.
<AREA>
  <DTACOL PMTWIDTH=12 ENTWIDTH=25 DESWIDTH=25 SELWIDTH=25>
  <DTAFLD DATAVAR=curdate USAGE=out ENTWIDTH=8>Date
  <DTAFLD DATAVAR=cardno ENTWIDTH=7>Card No.
    <DTAFLDD>(A 7-digit number)
  <DTAFLD DATAVAR=name>Name
    <DTAFLDD>(Last, First, M.I.)
  <DTAFLD DATAVAR=address>Address
</DTACOL>
<DIVIDER>
<REGION DIR=horiz>
<SELFLD NAME=cardsel PMTWIDTH=30 SELWIDTH=38>Choose
one of the following
  <CHOICE CHECKVAR=card MATCH=new>New
  <CHOICE CHECKVAR=card MATCH=renew>Renewal
  <CHOICE CHECKVAR=card MATCH=replace>Replacement
  <SOURCE TYPE=proc>
    if (&cardsel = 1)
      VER (&name,nb)
      VER (&address,nb)
  </SOURCE>
</SELFLD>
<SELFLD TYPE=multi PMTWIDTH=30 SELWIDTH=25>Check valid branches
  <CHOICE NAME=north HELP=nthhlp CHECKVAR=nth>North Branch
  <CHOICE NAME=south HELP=sthhlp CHECKVAR=sth>South Branch
  <CHOICE NAME=east HELP=esthlp CHECKVAR=est>East Branch
  <CHOICE NAME=west HELP=wsthlp CHECKVAR=wst>West Branch
</SELFLD>
</REGION>
</AREA>
<CMDAREA>Enter a command
</PANEL>
```

T (Truncation)

The T tag designates the minimum command name that the user must enter to issue a command.

Syntax



Comments

You must code the T tag within the *external-command-name* of the CMD tag. For example, imagine this command is coded in an application command table:

```
<cmd name=compare>com<t>pare
```

Then you can enter com, comp, compa, compar, or compare to run the command.

The command name must be at least 2 bytes.

At run time, ISPF runs the first valid command in the command table that matches the character string entered in the command area.

You should be careful to avoid specifying values that conflict with other commands. For example:

```
<cmd name=compare>co<t>mpare
<cmd name=copy>co<t>py
```

In this situation, if the user enters co as a command, ISPF runs the COMPARE command.

Restrictions

- You must code the T tag within the *external-command-name* of a CMD definition. See “CMD (Command Definition)” on page 233 for a complete description of this tag.

Processing

None.

Examples

Here is source file markup that contains a command table. The commands DELETE and UPDATE have truncation definitions that allow the user to enter "del" and "upd", respectively, as the minimum command name.

```
<!DOCTYPE DM SYSTEM>

<CMDTBL APPLID=conv>
  <CMD NAME=update>Upd<T>ate
    <CMDACT ACTION='alias add'>
  <CMD NAME=add>Add
    <CMDACT ACTION=setverb>
  <CMD NAME=delete>Del<T>ete
    <CMDACT ACTION=passthru>
  <CMD NAME=search>Search
    <CMDACT ACTION=passthru>
</CMDTBL>
```

Here is a table that shows the resultant ISPF application command table.

Table 73. ISPF application command table

ZCTVERB	ZCTTRUNC	ZCTACT
UPDATE	3	ALIAS ADD
ADD	0	SETVERB
DELETE	3	PASSTHRU
SEARCH	0	PASSTHRU

TEXTLINE (Text Line)

The TEXTLINE tag generates a single line of text to replace the regular tag text for the HELP and PANEL tags.

Syntax

➤ <TEXTLINE> — </TEXTLINE> ➤

Comments

The TEXTLINE tag encloses one or more TEXTSEG tags, used to define the parts or segments of the replacement text. Text defined by the TEXTSEG tag(s) is accumulated in a left to right order. The resulting text is used to create or replace the text portion of the HELP or PANEL tag definition.

Restrictions

- The TEXTLINE tag requires an end tag.
- You must code the TEXTLINE tag within a HELP or PANEL tag definition.

Processing

Table 74. Tags you can code within a TEXTLINE definition			
Tag	Reference	Usage	Required
DTAFLD	“DTAFLD (Data Field)” on page 275	Multiple	No
TEXTSEG	“TEXTSEG (Text Segment)” on page 439	Multiple	Yes

Examples

See the example for [“TEXTSEG \(Text Segment\)” on page 439](#).

TEXTSEG (Text Segment)

The TEXTSEG tag creates a text segment to be accumulated for the replacement text line created by the TEXTLINE tag.

Syntax

➤ <TEXTSEG — EXPAND= — AFTER — BEFORE — BOTH — WIDTH= — n — > — text — ➤

➤ </TEXTSEG> ➤

Parameters

EXPAND=ABOVE | BEFORE | BOTH

This attribute specifies whether expand control is added to the provided text. Expand characters are obtained from the HELP or PANEL tag definition, if available. If no expand character(s) have been specified on those tags, the conversion utility generates the necessary character. You may place the expand control before, after, or both before and after the text.

WIDTH=n

This attribute specifies the number of bytes to reserve for the text. The default is to not allow space beyond the actual text length.

Text

This is the text of the segment.

Comments

The TEXTSEG tag defines a part or segment of a replacement text line. When multiple TEXTSEG tags are present within the TEXTLINE definition, the replacement text line is created from left to right in the order the TEXTSEG tags are coded.

Restrictions

- You must code the TEXTSEG tag within a TEXTLINE tag definition.
- If the EXPAND attribute is not specified and the resulting replacement text is less than the panel width, the text is centered as the panel title.

Processing

Table 75. Tags you can code within a TEXTSEG definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No

Examples

Here is an example that uses the TEXTLINE and TEXTSEG tags to create a special panel title that includes the system time and date. Because the EXPAND attribute is specified in the second TEXTSEG tag, the resulting title replacement text has the time and date fields placed at the left and right panel border.


```

<!doctype dm system ()>
<!-- Sample selection menu -->

<varclass name=vc1 type='char 80'>
  <xlat1 format=upper>
  </xlat1>

<varlist>
  <vardcl name=zcmd varclass=vc1>
</varlist>

<panel name=textseg1 menu keylist=keylxmlp>

  <textline>
    <textseg>&ztime
    <textseg expand=both>
      Sample Selection Panel with TEXTLINE tag
    <textseg>&zdate(8)
  </textline>

  <topinst>This is a selection panel.
  <selfld type=menu pmtloc=before fchoice=0 trail=nextsel
    selwidth=40 pmtwidth=10>Select an option
    <choice checkvar=xtest1 match=a>
      Selection #0 (Command Selch0)
    <action run=Selch0>
    <choice checkvar=xtext1 match=b>
      Selection #1 (Command Selch1)
    <action run=Selch1 parm='1 2 3 4'>
      passlib newpool suspend
    <choice checkvar=xtest1 match=c>
      Selection #2 (Command Selch2)
    <action run=Selch2 parm=1234>
    <choice checkvar=xtest1 match=d>
      Selection #3 (Command Selch3)
    <action run=Selch3 parm=abcd>
    <choice checkvar=xtest1 match=e>
      Selection #4 (Command Selch4)
    <action run=Selch4 parm='a b c d'>
  <chdiv>
  <choice selchar=x>
    Exit
  <action run=exit type=exit>
  </selfld>
  <cmdarea>
</panel>

```

```

07:30          Sample Selection Panel with TEXTLINE tag
Option ==> _

```

99/12/15

```

This is a selection panel.

```

```

Select an
option . . 0 Selection #0 (Command Selch0)
           1 Selection #1 (Command Selch1)
           2 Selection #2 (Command Selch2)
           3 Selection #3 (Command Selch3)
           4 Selection #4 (Command Selch4)

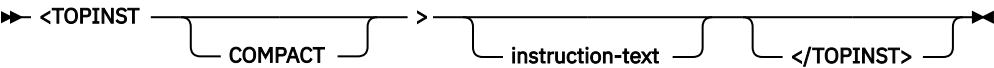
           X Exit

```

TOPINST (Top Instruction)

The TOPINST tag defines top instructions for an application panel.

Syntax



Parameters

COMPACT

This attribute causes the top instruction to format without a blank line after the text.

instruction-text

This is the text of the top instruction. The *instruction-text* must fit in the remaining panel depth.

Comments

The TOPINST tag defines top instructions for an application panel. The *instruction-text* formats as a paragraph based on the width of the application panel. You can code multiple paragraphs of instruction text by using a new top instruction tag for each new paragraph.

If the COMPACT attribute is not specified, the conversion utility inserts a blank line after the top instruction text.

Restrictions

- You must code the TOPINST within a PANEL definition. See [“PANEL \(Panel\)” on page 376](#) for a complete description of this tag.
- You cannot code a TOPINST tag within an AREA definition. If you define an area for the panel, code the TOPINST tag before the AREA start tag.

Processing

Table 76. Tags you can code within a TOPINST definition			
Tag	Reference	Usage	Required
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No

Examples

Here is application panel markup that contains top instructions. [Figure 162 on page 443](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=selcls TYPE='CHAR 2'>
<VARLIST>
  <VARDCL NAME=loc VARCLASS=selcls>
  <VARDCL NAME=mode VARCLASS=selcls>
</VARLIST>

<PANEL NAME=topinst HELP=trvlhlp WIDTH=60 DEPTH=22 KEYLIST=keylxml>
Dream Vacation Guide
<AB>
  <ABC>File
    <PDC>Add Entry
      <ACTION RUN=add>
    <PDC>Delete Entry
```

```

        <ACTION RUN=delete>
    <PDC>Update Entry
        <ACTION RUN=update>
    <PDC>Exit
        <ACTION RUN=exit>
    <ABC>Help
    <PDC>Extended Help...
        <ACTION RUN=exhelp>
    <PDC>Keys Help...
        <ACTION RUN=keyshelp>
</AB>
<TOPINST>Choose one of the following exotic locations and
your preferred mode of travel, then press Enter.
<AREA>
    <REGION DIR=horiz>
    <SELFLD NAME=loc PMTWIDTH=23 SELWIDTH=25>Exotic Location:
        <CHOICE>Athens, GA
        <CHOICE>Berlin, CT
        <CHOICE>Cairo, IL
        <CHOICE>Lizard Lick, NC
        <CHOICE>Paris, TX
        <CHOICE>Rome, NY
        <CHOICE>Venice, FL
    </SELFLD>
    <DIVIDER>
    <SELFLD NAME=mode PMTWIDTH=25 SELWIDTH=25>Travel Mode:
        <CHOICE>Boxcar
        <CHOICE>Hitchhike
        <CHOICE>Mule
    </SELFLD>
    </REGION>
</AREA>
<CMDAREA>
</PANEL>

```

```

File  Help
-----
                        Dream Vacation Guide

Choose one of the following exotic locations and your
preferred mode of travel, then press Enter.

Exotic Location:                Travel Mode:
--  1. Athens, GA                --  1. Boxcar
   2. Berlin, CT                --  2. Hitchhike
   3. Cairo, IL                 --  3. Mule
   4. Lizard Lick, NC
   5. Paris, TX
   6. Rome, NY
   7. Venice, FL

Command ==>-----
F1=Help      F2=Split      F3=Exit      F6=Keyshelp
F9=Swap      F12=Cancel

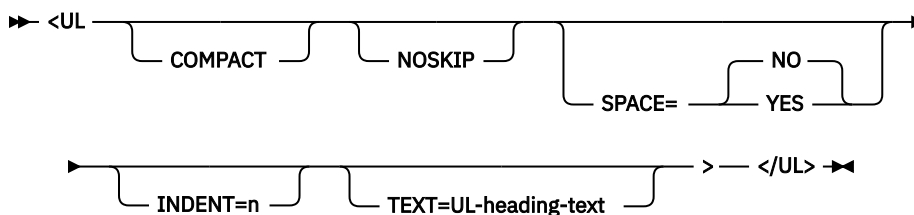
```

Figure 162. Top instructions

UL (Unordered List)

The UL tag defines an unordered list of items within an information region.

Syntax



Parameters

COMPACT

This attribute causes the list to be formatted without a blank line between the list items.

NOSKIP

This attribute causes the list to format without creating a blank line before the first line of the list.

SPACE=NO | YES

The SPACE attribute controls the indentation space for the list item. When the SPACE attribute is not specified on the LI tag, the SPACE attribute from the UL tag is used to set the indentation space for the nested LI tag *item-text*.

When SPACE=YES, the indentation is set to 3 spaces. When SPACE=NO (or SPACE is not specified), the indentation is set to 4 spaces.

The SPACE attribute can be used to control the alignment of list items when the first word of some list items is a DBCS word preceded by a shift-out character and the first word of other list items is a SBCS word.

INDENT=n

This attribute specifies that the list be indented from the current left margin.

TEXT=UL-heading-text

This attribute causes the list to format with a heading line containing the *UL-heading-text*.

Comments

The UL tag defines an unordered list of items within an information region. Unordered lists format as indented lists, with the list item identifier at the left margin. Nested lists indent four spaces to the right of the left margin of the list that contains them.

Note: The SPACE attribute does not affect the indentation of nested lists.

The conversion utility adds a blank line before the first item in the list. There are three levels of item identifiers: bullets (o), hyphens (-), and dashes (--). Each level is used successively when you nest unordered lists.

Panels formatted with the DBCS option use an uppercase 'O' as the bullet character.

Use the LI tag to denote each list item. See [“LI \(List Item\)” on page 325](#) for more information on the LI tag.

Restrictions

- The UL tag requires an end tag.
- You must code the UL tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.

Processing

Table 77. Tags you can code within a UL definition			
Tag	Reference	Usage	Required
LI	“LI (List Item)” on page 325	Multiple	No
LP	“LP (List Part)” on page 330	Multiple	No

Examples

Here is help panel markup that contains two unordered lists. The second unordered list is nested within the second list item of the first unordered list. [Figure 163 on page 445](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=ul DEPTH=22>Help for Reference Section
<AREA>
<INFO>
  <P>Learn everything about anything,
  and more, in our Reference section.
  Our Reference section includes:
  <UL>
    <LI>Atlases
    <LI>Dictionaries
      <UL COMPACT>
        <LI>English
        <LI>Other languages
      </UL>
    <LI>Encyclopedias
    <LI>How-to books
    <LI>Magazines and periodicals
  </UL>
</INFO>
</AREA>
</HELP>
```

```
Help for Reference Section

Learn everything about anything, and more, in
our Reference section. Our Reference section
includes:

o Atlases
o Dictionaries
  - English
  - Other languages
o Encyclopedias
o How-to books
o Magazines and periodicals

F1=Help      F3=Exit      F5=Exhelp
F6=Keyshelp  F7=PrvTopic  F8=NxtTopic
F10=PrvPage  F11=NxtPage  F12=Cancel
```

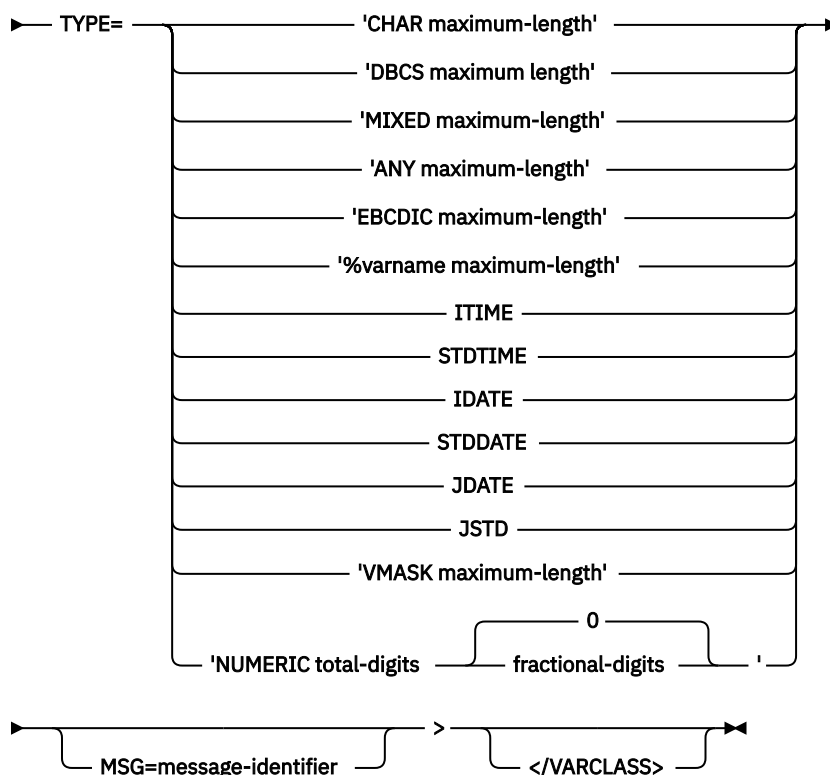
Figure 163. Unordered list

VARCLASS (Variable Class)

The VARCLASS tag defines information related to a class of variables.

Syntax

➤ <VARCLASS — NAME=variable-class-name ➔



Parameters

NAME=variable-class-name

This attribute specifies the name of this variable class.

The *variable-class-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

TYPE=type of data

This attribute specifies the data type and display length characteristics for variables that refer to the variable class.

For data fields and list columns, the conversion utility uses the lengths specified in this attribute when CHOFLD or DTAFLD ENTWIDTH or LSTCOL COLWIDTH attributes cannot otherwise be determined. The lengths specified control the width of the data field in the panel.

The allowable TYPE values are:

'CHAR maximum-length'

This specifies a character string for which the maximum length, in bytes, is given by *maximum-length*.

'DBCS maximum-length'

This is a double-byte character string for which the maximum length, in bytes, is given by *maximum-length*. The maximum length must be an even number.

'MIXED maximum-length'

This specifies a character string containing single-byte characters, double-byte characters, or both for which the maximum length, in bytes, is given by *maximum-length*. Strings of DBCS characters are delimited by shift-out (SO) and shift-in (SI) codes.

'ANY maximum-length'

This attribute is processed by the conversion utility as TYPE=MIXED.

'EBCDIC maximum-length'

This specifies a character string containing only single-byte characters for which the maximum length, in bytes, is given by *maximum-length*.

'%varname maximum-length'

This specifies that a variable name is used to define the type of character string. The maximum length, in bytes, is given by *maximum-length*. It is the responsibility of the application developer to ensure that **%varname** contains a valid TYPE value before attempting to display the panel.

ITIME

The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS. The default length value of ITIME is set by the conversion utility to 5.

STDTIME

The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS. The default length value of STDTIME is set by the conversion utility to 8.

IDATE

The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS. The default length value of IDATE is set by the conversion utility to 8.

STDDATE

The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS. The default length value of STDDATE is set by the conversion utility to 10.

JDATE

This attribute is supported as an ISPF extension to the Dialog Tag Language. The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS. The default length value of JDATE is set by the conversion utility to 6.

JSTD

This attribute is supported as an ISPF extension to the Dialog Tag Language. The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS. The default length value of JSTD is set by the conversion utility to 8.

'VMASK maximum-length'

This attribute is supported as an ISPF extension to the Dialog Tag Language. The VMASK attribute is provided to support the user mask option the ISPF VMASK service. The *maximum-length* value is limited to the ISPF maximum of 20. The conversion utility adds a "VEDIT (*variable*)" statement to the)PROC section of the panel for variables which are related to this VARCLASS.

'NUMERIC total-digits 0 | fractional-digits'

This attribute allows you to check to see if the user has entered a valid number. A valid number can include thousands separators, a decimal separator, and a sign. The conversion utility builds the VER(variable ENUM) statement to perform numeric validation. The value specified for *total-digits* must not be greater than 16.

The *total-digits* and *fractional-digits* are used to determine a *maximum-length* value which is used for field entry width, if necessary, in DTAFLD and LSTCOL processing. For example, 'NUMERIC 8 2' defines a width of 11, composed of 8 possible digits, a decimal point, a thousands separator, and a leading sign.

Note: ISPF does not check to verify proper positioning of the decimal point. See the discussion on VER(variable ENUM) in the [z/OS ISPF Dialog Developer's Guide and Reference](#) for more information.

MSG=message-identifier

This attribute indicates the default message to be displayed if the variable fails any of the enclosed checks. See ["MSG \(Message\)" on page 352](#) for information on creating messages.

Comments

The VARCLASS tag defines information related to a class of variables. You can group validation and translation checks you want ISPF to perform within one VARCLASS definition. You point to the VARCLASS definition from one or more VARDCL tags you code within the VARLIST definition.

Note: The ISPF Dialog Tag Language conversion utility does not require that you code the VARCLASS, VARDCL, or VARLIST tags for a successful generation of a panel, command table, or message member that includes variables. If the conversion utility finds a variable that does not have an associated VARDCL definition, it issues a warning message.

The use of the VARCLASS, VARDCL, and VARLIST tags is required if you want to use the facilities provided by the CHECKL and XLATL tags.

Restrictions

- You cannot code the VARCLASS tag within any other tag definition.
- You must code the VARCLASS tag before any other tag within the source file that refers to it.
- Within the variable class definition, you must code any and all XLATL tags before any CHECKL tags.

Processing

<i>Table 78. Tags you can code within a VARCLASS definition</i>			
Tag	Reference	Usage	Required
CHECKL	“CHECKL (Validity Check List)” on page 218	Multiple	No
XLATL	“XLATL (Translate List)” on page 458	Multiple	No

Examples

Here is an example that contains two variable classes. The first variable class provides an alphabetic validity check. The second variable class provides input translation to uppercase and validates that the input is one of the listed values. Also shown in the markup are two input data fields (within a PANEL definition) that refer to the variable declarations associated with the variable classes.


```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=namec TYPE='char 25'>
  <CHECKL>
    <CHECKI TYPE=alpha>
  </CHECKL>

  <VARCLASS NAME=officec TYPE='char 4'>
    <XLATL FORMAT=upper>
  </XLATL>
  <CHECKL>
    <CHECKI TYPE=values PARM1=EQ PARM2='A101 A108 B210 B214'>
  </CHECKL>

<VARLIST>
  <VARDCL NAME=reserver VARCLASS=namec>
  <VARDCL NAME=office VARCLASS=officec>
</VARLIST>

<PANEL NAME=varclass>Conference Room
  <TOPINST>Enter the required information to reserve a conference room.
  <AREA>
    <DTACOL PMTWIDTH=20>
      <DTAFD DATAVAR=reserver USAGE=in ENTWIDTH=25>Name
      <DTAFD DATAVAR=office USAGE=in ENTWIDTH=4>Office number
    </DTACOL>
  </AREA>
</PANEL>

```

VARDCL (Variable Declaration)

The VARDCL tag declares variables referred to in dialog element definitions.

Syntax

```

➤ <VARDCL — NAME=name — VARCLASS=variable-class-name — >
  └──────────────────────────────────────────────────────────┘
  </VARDCL>

```

Parameters

NAME=name

This attribute specifies the name of a variable used elsewhere in the DTL source file. The *name* must follow the standard naming convention described in [“Rules for variable names” on page 179](#).

VARCLASS=variable-class-name

This attribute specifies the default variable class associated with the variable. If you want to perform a different set of checks or translations on any data field or list column, you can specify an overriding variable class in the DTAFD or LSTCOL tags.

Comments

The VARDCL tag declares variables referred to in dialog element definitions.

Note: The ISPF Dialog Tag Language conversion utility does not require that you code the VARCLASS, VARDCL, or VARLIST tags for successful generation of a panel, command table, or message member that includes variables. If the conversion utility finds a variable that does not have an associated VARDCL definition, it issues a warning message.

The use of the VARCLASS, VARDCL, and VARLIST tags is required if you want to use the facilities provided by the CHECKL and XLATL tags.

Restrictions

- You must code the VARDCL tag within a VARLIST tag. See [“VARLIST \(Variable List\)”](#) on page 450 for a complete description of this tag.

Processing

None.

Examples

Here is source file markup that contains variable declarations for all of the variables defined in the panel definition. The declared variables include:

- The variable *whchsrch* specified in the CHECKVAR attributes associated with the pull-down choices of the **Search** action bar choice.
- The data field variables *curdate*, *cardno*, *name*, and *address*.
- The variable *cardsel*, which is the entry-field of the single-choice selection field.
- The variables *north*, *south*, *east*, and *west*, which are the entry-fields associated with the multiple-choice selection field.
- The variables defined as the check variables (CHECKVAR attribute) for the selection fields.

```
<!DOCTYPE DM SYSTEM(
  <!entity sampabc sysem>
  <!entity sampbody system>)>

<VARCLASS NAME=date      TYPE='char 8'>
<VARCLASS NAME=numcls    TYPE='numeric 7'>
<VARCLASS NAME=namecls   TYPE='char 25'>
<VARCLASS NAME=char1cls  TYPE='char 1'>
<VARCLASS NAME=char7cls  TYPE='char 7'>

<VARLIST>
  <VARDCL NAME=whchsrch VARCLASS=char1cls>
  <VARDCL NAME=curdate  VARCLASS=date>
  <VARDCL NAME=cardno   VARCLASS=numcls>
  <VARDCL NAME=name     VARCLASS=namecls>
  <VARDCL NAME=address  VARCLASS=namecls>
  <VARDCL NAME=cardsel  VARCLASS=char1cls>
  <VARDCL NAME=card     VARCLASS=char7cls>
  <VARDCL NAME=north    VARCLASS=char1cls>
  <VARDCL NAME=south    VARCLASS=char1cls>
  <VARDCL NAME=east     VARCLASS=char1cls>
  <VARDCL NAME=west     VARCLASS=char1cls>
  <VARDCL NAME=nth      VARCLASS=char1cls>
  <VARDCL NAME=sth      VARCLASS=char1cls>
  <VARDCL NAME=est      VARCLASS=char1cls>
  <VARDCL NAME=wst      VARCLASS=char1cls>
</VARLIST>

<PANEL NAME=vardcl>Library Card Registration
<AB>
&sampabc;
</AB>
&sampbody;
</PANEL>
```

VARLIST (Variable List)

The VARLIST tag provides the means to code VARDCL tags in a single list.

Syntax

➤ <VARLIST> — </VARLIST> ➤

Comments

The VARLIST tag provides the means to code VARDCL tags in a single list. The VARDCL tags coded within a VARLIST definition declare variables that are referred to in the dialog element definitions within a DTL source file.

Note: The ISPF Dialog Tag Language conversion utility does not require that you code the VARCLASS, VARDCL, or VARLIST tags for a successful generation of a panel, command table, or message member that includes variables. If the conversion utility finds a variable that does not have an associated VARDCL definition, it issues a warning message.

The use of the VARCLASS, VARDCL, and VARLIST tags is required if you want to use the facilities provided by the CHECKL and XLATL tags.

Restrictions

- The VARLIST tag requires an end tag.
- You cannot code the VARLIST tag within any other tag definition.
- You can code the VARLIST tag immediately after all VARCLASS tags within the DTL source file and before any tag definitions that refer to the variables declared in the variable list.

Processing

Table 79. Tags you can code within a VARLIST definition			
Tag	Reference	Usage	Required
VARDCL	“VARLIST (Variable List)” on page 450	Multiple	No

Examples

Here is source file markup that contains a variable list. The variable declarations within the list define variables for the fields within the PANEL definitions that refer to them.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=char8 TYPE='char 8'>
<VARCLASS NAME=name TYPE='char 25'>
<VARCLASS NAME=phoncls TYPE='char 12'>
<VARCLASS NAME=appcls TYPE='char 1'>
  <XLATL FORMAT=upper>
</XLATL>
<CHECKL>
  <CHECKI TYPE=values PARM1=EQ PARM2='Y N'>
</CHECKL>

<VARLIST>
  <VARDCL NAME=curdate VARCLASS=char8>
  <VARDCL NAME=namevar VARCLASS=name>
  <VARDCL NAME=passvar VARCLASS=char8>
  <VARDCL NAME=xlname VARCLASS=name>
  <VARDCL NAME=xphone VARCLASS=phoncls>
  <VARDCL NAME=xapp VARCLASS=appcls>
</VARLIST>

<PANEL NAME=varlist1 KEYLIST=keylxml>System Log On
<TOPINST>Complete the following fields, then press Enter.
<AREA>
  <DTACOL PMTWIDTH=12>
    <DTAFLD DATAVAR=curdate ENTWIDTH=8 USAGE=out>Date
    <DTAFLD DATAVAR=namevar ENTWIDTH=25 DESWIDTH=15>Name
    <DTAFLD DATAVAR=passvar ENTWIDTH=8 DISPLAY=no>Password
  </DTACOL>
</AREA>
</PANEL>

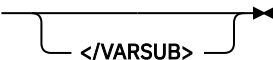
<PANEL NAME=varlist2 DEPTH=14 KEYLIST=keyltbl>Subscriber List
<TOPINST>Enter phone number, if missing,
(format - nnn-xxx-xxxx) and approved
indicator (y or n) for each person.
<AREA>
  <LSTFLD>
    <LSTCOL DATAVAR=xlname USAGE=out COLWIDTH=25>Last Name
    <LSTCOL DATAVAR=xphone COLWIDTH=12>Phone Number
  <LSTGRP>Approved
    <LSTCOL DATAVAR=xapp USAGE=in REQUIRED=yes
      COLWIDTH=1 MSG=msgv886>(Y or N)
  </LSTGRP>
</LSTFLD>
</AREA>
<CMDAREA>Enter a command
</PANEL>

```

VARSUB (Variable Substitution)

The VARSUB tag specifies a variable to substitute in message text.

Syntax

➤ <VARSUB — VAR=variable-name — >  ➤

Parameters

VAR=variable-name

This attribute specifies the variable whose value is substituted within the message.

The *variable-name* should be declared with a VARDCL tag.

The *variable-name* must follow the standard naming convention described in [“Rules for variable names”](#) on page 179.

Comments

The VARSUB tag specifies a variable to substitute in message text. You use the required VAR attribute to specify the variable whose value is resolved and inserted into the message text when the message is displayed. The value coded must be a variable name without leading % notation.

You can code the VARSUB tag in the *message-text* of a MSG tag. The variable value is inserted by ISPF at run time at the position in the message text where the VARSUB tag is coded.

For example, assume this VARSUB tag was coded within the text of this message:

```
<msgmbr name=abca00>
<msg suffix=1 msgtype=warning>Invalid name,
"<VARSUB VAR=LASTN>", specified.
The name may contain only alphabetic characters.
</msgmbr>
```

When a dialog refers to a message *abca001* (with a GETMSG, SETMSG, DISPLAY, or TBDISPL service call) or the message is displayed by ISPF during panel validation, the value of *lastn* is retrieved and inserted into the message text. Here is the message after substitution:

```
Invalid name, "Jones1", specified.
The name may contain only alphabetic characters.
```

Restrictions

- You must code the VARSUB tag within the text of a MSG definition. See [“MSG \(Message\)” on page 352](#) for a complete description of this tag.
- The value specified by the VAR attribute should be declared with a VARDCL tag. See [“VARDCL \(Variable Declaration\)” on page 449](#) for a complete description of this tag.

Processing

None.

Examples

Here is markup that contains a message member which contains nine MSG definitions. The text of messages *MSGV883* and *MSGV888* contain variable substitutions. [Figure 164 on page 454](#) shows the generated message member.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=msgcls TYPE='char 20'>
<VARLIST>
  <VARDCL NAME=phoneno VARCLASS=msgcls>
  <VARDCL NAME=cnum VARCLASS=msgcls>
</VARLIST>

<MSGMBR NAME=msgv88>
  <MSG SUFFIX=1>Name must be alphabetic.
  <MSG SUFFIX=2>Enter only number of days.
  <MSG SUFFIX=3 MSGTYPE=critical>The only rooms we have available
are either SINGLE or DOUBLE. Please call the manager of the hotel
who will arrange equivalent lodging at another
hotel in the area. This is our mistake, and we will, of course,
pick up the bill. Please call collect <VARSUB VAR=phoneno>.
  <MSG SUFFIX=4 MSGTYPE=action LOCATION=modal>Please enter either
BIGCHARGE, V I S T A, EZCARD, CHECK, or CASH.
  <MSG SUFFIX=5 MSGTYPE=warning LOCATION=modeless>Please enter your name.
  <MSG SUFFIX=6>Please enter Y or N.
  <MSG SUFFIX=7>Card number is a seven-digit number.
  <MSG SUFFIX=8 MSGTYPE=warning>The card number you
entered, <VARSUB VAR=cnum> is not valid.
  <MSG SUFFIX=9>Message '9' contains embedded quotes.
</MSGMBR>
```

```
MSGV881 .TYPE=NOTIFY
'Name must be alphabetic.'
MSGV882 .TYPE=NOTIFY
'Enter only number of days.'
MSGV883 .TYPE=CRITICAL
'The only rooms we have available are either SINGLE or DOUBLE. Please call th' +
'e manager of the hotel who will arrange equivalent lodging at another hotel ' +
'in the area. This is our mistake, and we will, of course, pick up the bill. ' +
'Please call collect &PHONENO.'
MSGV884 .TYPE=ACTION .WINDOW=RESP
'Please enter either BIGCHARGE, V I S T A, EZCARD, CHECK, or CASH.'
MSGV885 .TYPE=WARNING .WINDOW=NORESP
'Please enter your name.'
MSGV886 .TYPE=NOTIFY
'Please enter Y or N.'
MSGV887 .TYPE=NOTIFY
'Card number is a seven-digit number.'
MSGV888 .TYPE=WARNING
'The card number you entered, &CNUM is not valid.'
MSGV889 .TYPE=NOTIFY
'Message '9' contains embedded quotes.'
```

Figure 164. Variable substitution

WARNING (Warning)

The WARNING tag defines text that alerts the user to a risk of possible error conditions in the system.

Syntax

```
➤ <WARNING> _____ </WARNING> ➤
      |
      | text
```

Parameters

text

This is the text of the warning.

Comments

The WARNING tag defines text that alerts the user to a risk of possible error conditions in the system.

The WARNING tag is one of the tags that alert the user of a possible risk; the others are the CAUTION tag and the ATTENTION tag.

Code a warning statement before the text to which it pertains so that the user can read about the possible risks before reading the text.

When a warning statement is displayed, the string "Warning:" (or its translated equivalent) appears on the screen before the text of the warning statement.

You can code additional paragraphs of warning text by coding the P (paragraph) tag within a WARNING definition. You can also code other tags allowed in an information area within a WARNING definition.

Restrictions

- The WARNING tag requires an end tag.
- You must code the WARNING tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- The WARNING tag must be immediately preceded by a P, LI, or LP tag. If the WARNING tag is coded on the same line as one of these tags, there can be no intervening blanks. See [“P \(Paragraph\)” on page 370](#), [“LI \(List Item\)” on page 325](#), and [“LP \(List Part\)” on page 330](#) for descriptions of these tags.
- You cannot nest WARNING, ATTENTION, or CAUTION tags within each other.

Processing

<i>Table 80. Tags you can code within a WARNING definition</i>			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
FIG	“FIG (Figure)” on page 291	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
LINES	“LINES (Lines)” on page 327	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No
XMP	“XMP (Example)” on page 460	Multiple	No

Examples

Here is help panel markup that contains a warning statement. The warning statement starts at the left margin because it is embedded in the LP tag.

```

<!DOCTYPE DM SYSTEM>

<HELP NAME=warning DEPTH=20>Help For Changing a File
<AREA>
<INFO>
  <OL>
    <LI>Type over the existing data
    in the entry fields with the new data.
    <LP><WARNING>Performing the next step will save
    all changes and delete the existing data.
    <P>To quit this function without
    deleting the existing data, press F12.
    </WARNING>
    <LI>Press Enter to save the
    updated data.
  </OL>
</INFO>
</AREA>
</HELP>

```

Help For Changing a File

1. Type over the existing data in the entry fields with the new data.

Warning: Performing the next step will save all changes and delete the existing data.

To quit this function without deleting the existing data, press F12.

2. Press Enter to save the updated data.

F1=Help	F3=Exit	F5=Exhelp
F6=Keyshelp	F7=PrivTopic	F8=NxtTopic
F10=PrivPage	F11=NxtPage	F12=Cancel

Figure 165. Warning statement

XLATI (Translate Item)

The XLATI tag defines an individual list element in a translate list.

Syntax

```

➤ <XLATI [VALUE=internal-value] [displayed-value] </XLATI> ➤

```

Parameters

VALUE=internal-value

ISPF saves this value in the variable pool when translating on input and retrieves it from the variable pool when translating on output. If the *internal-value* contains characters other than A-Z, a-z, and 0-9, you must enclose the value in quotes.

Omitting this attribute indicates that any value is acceptable. When translating on input, ISPF does not translate the *displayed-value* before storing it in the pool. When translating on output, ISPF translates to the *displayed-value* any value that is not already matched.

displayed-value

This attribute specifies the displayed value that must be matched when doing a translation on input and the result when doing a translation on output. The test for a translation match is case-sensitive. Any characters, including embedded blanks, are allowed in the *displayed-value*. If the value has

blanks that you want preserved, or the value consists of only blanks, the value should be coded within the LIT (Literal) tag. If the LIT tag is not used, all blanks are stripped and any value with only blanks indicated that no value was specified.

Omitting this value indicates that any value is acceptable. When translating on output, this means that the *internal-value* is not to be translated before being displayed. When translating on input, it means that any value not already matched is to translate to the *internal-value*.

Comments

The XLATI tag defines an individual list element in a translate list. As many XLATI tags as are necessary (up to a limit of 126) to accomplish the desired translation can be included within the translation list.

Each XLATI tag provides information necessary to translate a *displayed-value* to an *internal-value* and vice versa. Translation is done in the order given by the tags. Translation stops when a match is found. An XLATI tag that omits both *internal-value* and *displayed-value* has this effect: when translating on output the variable value is displayed, and when translating on input the entered value is stored in the variable.

The ISPF TRANS() function is used for all translations. When translating on output, ISPF)INIT panel logic translates the *internal-value* to the *displayed-value*. When translating on input, ISPF)PROC panel logic translates the *displayed-value* to the *internal-value*. The test for a translation match is case-sensitive. You can code an XLATL FORMAT=UPPER definition before an XLATL definition that contains XLATI tags to convert user input to uppercase before the translate list is processed.

Restrictions

You must code the XLATI tag within an XLATL definition. See [“XLATI \(Translate Item\)” on page 456](#) for a complete description of this tag.

Processing

Table 81. Tags you can code within an XLATI definition			
Tag	Reference	Usage	Required
LIT	“LIT (Literal)” on page 329	Multiple	No

Examples

Here is source file markup that contains a variable class with a translate list that performs input and output translation on values assigned to the winter months. The associated variable declarations and fields are also shown.

```

<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=monthcls type='char 3'>
  <XLATL FORMAT=upper>
  </XLATL>
  <XLATL MSG=abcd003>
    <XLATI VALUE=11>NOV
    <XLATI VALUE=12>DEC
    <XLATI VALUE=01>JAN
    <XLATI VALUE=02>FEB
    <XLATI VALUE=03>MAR
  </XLATL>

<VARCLASS NAME=costcls TYPE='numeric 6' MSG=abcd001>

<VARCLASS NAME=typecls TYPE='char 4'>
  <XLATL FORMAT=upper>
  </XLATL>
  <CHECKL MSG=abcd002>
    <CHECKI TYPE=values Parm1=EQ Parm2= 'GAS OIL ELEC'>
  </CHECKL>

<VARLIST>
  <VARDCL NAME=month VARCLASS=monthcls>
  <VARDCL NAME=cost VARCLASS=costcls>
  <VARDCL NAME=heat VARCLASS=typecls>
</VARLIST>

<PANEL NAME=xlati KEYLIST=keylxml>Heating Cost Survey
  <TOPINST>Complete the information below and then press Enter.
  <AREA>
    <DTACOL PMTWIDTH=20 DESWIDTH=30>
      <DTAFLD DATAVAR=month ENTWIDTH=3>Winter month
      <DTAFLLD>Enter Nov, Dec, Jan, Feb, or Mar
      <DTAFLD DATAVAR=cost ENTWIDTH=6>Heating cost
      <DTAFLLD DATAVAR=heat ENTWIDTH=4>Type of heating
      <DTAFLLD>Enter Oil, Gas, or Elec
    </DTACOL>
  </AREA>
  <BOTINST>Thank you for your participation.
</PANEL>

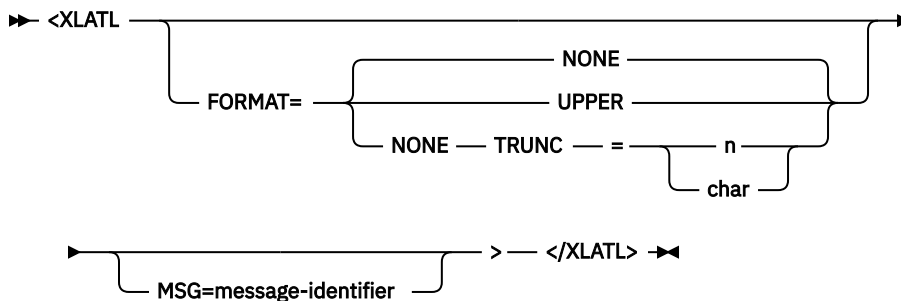
<MSGMBR NAME=abcd00>
  <MSG SUFFIX=1>Heating cost must be numeric
  <MSG SUFFIX=2>Type of heating must be "Gas", "Oil", or "Elec"
  <MSG SUFFIX=3>Winter month must be "Nov", "Dec", "Jan", "Feb", or "Mar"
</MSGMBR>

```

XLATL (Translate List)

The XLATL tag defines a translate list for a variable class.

Syntax



Parameters

FORMAT=NONE | UPPER

This attribute defines the type of translation. NONE specifies that enclosed XLATI tags are to be used to translate the value on an item for item basis. UPPER specifies that the variable value is translated to uppercase.

TRUNC=n | char

This attribute defines the type of truncation to be performed on input values. It is valid only when FORMAT=NONE. If a number is provided, truncation occurs at the length indicated. If a nonnumeric character is provided, truncation occurs at the first occurrence of that character.

MSG=message-identifier

This attribute specifies the ID of a message to be issued for the error condition that results when an input translation fails because the user entered a value not specified in the list. Specifying an XLATI tag with no *internal-value* and no *displayed-value* ensures that any value not in the list is accepted without error. If no message ID is specified and an error occurs, the *message-identifier* specified on the VARCLASS tag is used. If no *message-identifier* is specified on the XLATL tag or the VARCLASS tag, no message is displayed.

Note: This message is not used if translation on output fails. The variable value is displayed as is, subject to whatever size restrictions apply to the field.

Comments

The XLATL tag defines a translate list for a variable class. XLATI tags, which define the elements of the translation list, are coded within the XLATL tag. A translation list is defined within a VARCLASS tag.

If FORMAT=NONE is specified, it is expected that there are XLATI tags within the XLATL definition. If FORMAT=UPPER is specified, no XLATI tags are accepted in the XLATL definition.

Translation lists are optional and provide a means of translating between a displayed value and the internal value of the variable. Translation can occur on input (the translation result is stored in the variable pool), on output (the value from the pool is translated before the user sees it), or both, depending on the USAGE attribute of the DTAFD tag that is associated with the variable. Translation for table display is not supported by ISPF. See the [z/OS ISPF Dialog Developer's Guide and Reference](#) for additional information about the TRANS function.

Restrictions

- The XLATL tag requires an end tag.
- You must code the XLATL tag within a VARCLASS definition. See “[VARCLASS \(Variable Class\)](#)” on page 445 for a complete description of this tag.
- You must code all XLATL tags before any CHECKL tags in the same variable class.

Processing

Table 82. Tags you can code within an XLATL definition			
Tag	Reference	Usage	Required
XLATI	“XLATI (Translate Item)” on page 456	Multiple	Yes

Examples

Here is source file markup that includes translation of user input for *monthcls* to uppercase followed by a translation list of the abbreviated month to an internal value. If no match is found, message *abcd003*

is issued. The example also shows the use of uppercase translation before a check for a list of values for **Type of heating**.

```
<!DOCTYPE DM SYSTEM>

<VARCLASS NAME=monthcls type='char 3'>
  <XLATL FORMAT=upper>
  </XLATL>
  <XLATL MSG=abcd003>
    <XLATI VALUE=11>NOV
    <XLATI VALUE=12>DEC
    <XLATI VALUE=01>JAN
    <XLATI VALUE=02>FEB
    <XLATI VALUE=03>MAR
  </XLATL>

<VARCLASS NAME=costcls TYPE='numeric 6' MSG=abcd001>

<VARCLASS NAME=typecls TYPE='char 4'>
  <XLATL FORMAT=upper>
  </XLATL>
  <CHECKL MSG=abcd002>
    <CHECKI TYPE=values Parm1=EQ Parm2= 'GAS OIL ELEC'>
  </CHECKL>

<VARLIST>
  <VARDCL NAME=month VARCLASS=monthcls>
  <VARDCL NAME=cost VARCLASS=costcls>
  <VARDCL NAME=heat VARCLASS=typecls>
</VARLIST>

<PANEL NAME=xlatl KEYLIST=keylxmp>Heating Cost Survey
  <TOPINST>Complete the information below and then press Enter.
  <AREA>
    <DTACOL PMTWIDTH=20 DESWIDTH=30>
      <DTAFLD DATAVAR=month ENTWIDTH=3>Winter month
      <DTAFLDD>Enter Nov, Dec, Jan, Feb, or Mar
      <DTAFLD DATAVAR=cost ENTWIDTH=6>Heating cost
      <DTAFLD DATAVAR=heat ENTWIDTH=4>Type of heating
      <DTAFLDD>Enter Oil, Gas, or Elec
    </DTACOL>
  </AREA>
  <BOTINST>Thank you for your participation.
</PANEL>

<MSGMBR NAME=abcd00>
  <MSG SUFFIX=1>Heating cost must be numeric
  <MSG SUFFIX=2>Type of heating must be "Gas", "Oil", or "Elec"
  <MSG SUFFIX=3>Winter month must be "Nov", "Dec", "Jan", "Feb", or "Mar"
</MSGMBR>
```

XMP (Example)

The XMP tag defines unformatted text within an information region.

Syntax

```
➡ <XMP [NOSKIP] > text </XMP> ➡
```

Parameters

NOSKIP

This attribute causes the blank line normally placed before the example to be skipped.

text

This is the text of the example.

Comments

The XMP tag defines unformatted text within an information region.

Text coded between the XMP start and end tags is indented two spaces and formats from the current left margin. Tags which normally cause word-wrapping (for example, P and LI) do not cause word-wrapping when nested within an XMP tag.

When defining text for an example in your source file, you should be careful not to exceed the width of the information region it is coded within. If the source text on any line exceeds the formatting width, the conversion utility truncates the line. A warning message is issued the first time truncation occurs.

Restrictions

- The XMP tag requires an end tag.
- You must code the XMP tag within an INFO definition. See [“INFO \(Information Region\)” on page 317](#) for a complete description of this tag.
- You can code multiple XMP tags within an INFO definition, as long as they are not nested within each other.

Processing

<i>Table 83. Tags you can code within an XMP definition</i>			
Tag	Reference	Usage	Required
DL	“DL (Definition List)” on page 261	Multiple	No
HP	“HP (Highlighted Phrase)” on page 315	Multiple	No
NOTE	“NOTE (Note)” on page 358	Multiple	No
NOTEL	“NOTEL (Note List)” on page 361	Multiple	No
NT	“NT (Note)” on page 364	Multiple	No
OL	“OL (Ordered List)” on page 367	Multiple	No
P	“P (Paragraph)” on page 370	Multiple	No
PARML	“PARML (Parameter List)” on page 386	Multiple	No
PS	“PS (Point-and-Shoot)” on page 398	Multiple	No
RP	“RP (Reference Phrase)” on page 411	Multiple	No
SL	“SL (Simple List)” on page 433	Multiple	No
UL	“UL (Unordered List)” on page 443	Multiple	No

Examples

Here is help panel markup that contains an example. [Figure 166 on page 462](#) shows the formatted result.

```
<!DOCTYPE DM SYSTEM>

<HELP NAME=xmp WIDTH=40 DEPTH=20>Help for the Search Function
<AREA>
<INFO>
  <P>To locate a book, type the book
  title in the "Title" field and press Enter.
  <P>Example:
  <XMP>
```

```
Title:  THE JOY OF CODING
</XMP>
<P>You don't have to worry about using
upper or lowercase letters; all text is automatically
converted to uppercase for the search.
</INFO>
</AREA>
</HELP>
```

Help for the Search Function

To locate a book, type the book title
in the "Title" field and press Enter.

Example:

Title: THE JOY OF CODING

You don't have to worry about using
upper or lowercase letters; all text
is automatically converted to
uppercase for the search.

F1=Help F3=Exit F5=Exhelp
F6=Keyshelp F7=PrvTopic F8=NxtTopic
F10=PrvPage F11=NxtPage F12=Cancel

Figure 166. Example

Appendix A. Dialog Tag Language (DTL) tags

The following table is an alphabetic summary of the supported Dialog Tag Language (DTL) tags for z/OS ISPF. The table shows the tag, tells whether an end tag is required (Yes) or optional (No), and lists the tag's attributes (if any) and the tag content (if any) in *italics>. The table also lists which tags you can nest within the tag, as well as which tags you can code the tag within.*

Table 84. Tag summary

Tag	End tag	Attributes	Nested tags	Used within
AB	Yes	MNEMGEN=YES NO ABSEPSTR=ab-separator-string ABSEPCHAR=ab-separator-character	ABC	PANEL
ABC	No	HELP=NO YES help-panel-name *help-message-id %varname *%varname PDCVAR=cdc-variable-name <i>choice-description-text</i>	COMMENT M PDC PDSEP SOURCE	AB
ACTION	No	RUN=internal-command-name %varname PARM=parameters %varname APPLCMD=NO YES TYPE=CMD PGM PANEL EXIT NEWAPPL NEWAPPL=application-id NEWWINDOW PASSLIB NEWPOOL SUSPEND SCRNAME=screen-name NOCHECK ADDPPOP OPT=option %varname MODE=LINE FSCR LANG=APL CREX BARRIER NEST SETVAR=variable-name VALUE= <u>1</u> string %varname TOGVAR=variable-name VALUE1= <u>0</u> string %varname VALUE2= <u>1</u> string %varname		CHOICE PDC

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
AREA	Yes	MARGINW= <u>1</u> n MARGIND= <u>0</u> INDENT=n DEPTH=n * EXTEND= <u>OFF</u> ON FORCE DIV= <u>NONE</u> BLANK SOLID DASH TEXT DIVWIDTH= <u>MAX</u> MIN FORMAT=START CENTER END TEXT=divider-text WIDTH=n DIR= <u>VERT</u> HORIZ	COMMENT DA DIVIDER DTACOL DTAFLD GA GENERATE GRPHDR INFO LSTFLD PNLINST REGION SELFLD SOURCE	HELP PANEL
ASSIGNI	No	VALUE=test-value RESULT=assigned-value		ASSIGNL
ASSIGNL	Yes	DESTVAR=destination-variable-name	ASSIGNI	DTAFLD
ATTENTION	Yes	<i>text</i>	DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL XMP	LI LP P

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
ATTR	No	ATTRCHAR=code TYPE=DATAIN DATAOUT CHAR INTENS= <u>HIGH</u> LOW NON %varname CAPS=OFF ON IN OUT %varname JUST=ASIS LEFT RIGHT %varname PAD=NULLS USER char %varname PADC=NULLS USER char %varname SKIP= <u>OFF</u> ON %varname GE= <u>OFF</u> ON %varname COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname HILITE=USCORE BLINK REVERSE %varname NUMERIC= <u>OFF</u> ON %varname FORMAT=EBCDIC DBCS MIX %varname OUTLINE= <u>NONE</u> L R O U BOX %varname PAS= <u>OFF</u> ON %varname CKBOX= <u>OFF</u> ON %varname CUADYN=CEF EE LEF NEF VOI LID LI CH CT DT ET FP NT PIN PT SAC SI SUC WASL WT %varname CSRGRP= <u>NO</u> YES n ATTN= <u>OFF</u> ON %varname		DA
BOTINST	No	COMPACT <i>instruction-text</i>	HP PS RP	PANEL
CAUTION	Yes	<i>text</i>	DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL XMP	LI LP P
CHDIV	No	TYPE= <u>NONE</u> SOLID DASH TEXT GUTTER= <u>1</u> n FORMAT= <u>START</u> CENTER END <i>divider-text</i>	HP	SELF CHOICE

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
CHECKI	No	TYPE= RANGE PARM1=low-bound %varname PARM2=high-bound %varname ALPHA CHARS PARM1=EQ PARM2=character-set VALUES PARM1=EQ PARM2=value-list VALUESX PARM1=NE PARM2=value-list BIT NAME NAMEF PICT PARM1=EQ PARM2=pictstring PICTCN PARM1=mask-character PARM2=field-mask PARM3=string NUM DBCS LISTV PARM1=EQ PARM2=%varlist LISTVX PARM1=NE PARM2=%varlist ALPHAB LEN PARM1=operator %varname PARM2=length %varname EBCDIC ENUM DSNAMES DSNAMESF DSNAMESFM DSNAMESPQ DSNAMESQ MIX HEX FILEID INCLUDE PARM1=IMBLK PARM2=ALPHA ALPHAB NUM PARM3=ALPHA ALPHAB NUM		CHECKL

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
CHECKI	No	TYPE= IDATE STDDATE JDATE JSTD ITIME STDTIME IPADDR4		CHECKL
CHECKL	Yes	MSG=message-identifier	CHECKI	VARCLASS
CHOFLD	No	DATAVAR=field-data VARCLASS=variable-class-name HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname USAGE= <u>BOTH</u> IN OUT REQUIRED= <u>NO</u> YES MSG=message-identifier AUTOTAB= <u>NO</u> YES ENTWIDTH=n FLDSPACE=n ALIGN= <u>START</u> CENTER END DISPLAY= <u>YES</u> NO NOENDATTR PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname PSVAR=point-and-shoot-variable %varname PSVAL=point-and-shoot-value %varname PAS=%varname EXPAND ATTRCHANGE= <u>NO</u> YES NEW INIT=initial-value IMAPNAME=image-name %varname IMAPNAMEP=image-namep %varname PLACE= <u>ABOVE</u> BELOW LEFT RIGHT %varname ATTRCHAR=code CAPS= <u>OFF</u> ON <i>choice-description-text</i>	ACTION COMMENT HP PS RP SOURCE	CHOICE

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
CHOICE	No	NAME=choice-name HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname CHECKVAR=variable-name MATCH= <u>1</u> string NOMATCH= <u>0</u> string AUTOTAB= <u>YES</u> NO SELCHAR='char(s),n' PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname HIDE HIDEX UNAVAIL=variable-name UNAVAILMAT= <u>1</u> string TRUNC=n AUTOSEL= <u>YES</u> NO <i>choice-description-text</i>	ACTION CHOFLD COMMENT HP PS RP SOURCE	SELFLD
CMD	No	NAME=internal-command-name ALTDESCR=command-description <i>external-command-name</i>	CMDACT T	CMDTBL
CMDACT	No	ACTION= 'SELECT=select-parameters' 'ALIAS=internal-command-name parameters' PASSTHRU SETVERB BACKWARD CANCEL EXIT EXHELP FKA FORWARD HELP PANELID RETRIEVE %varname application-command ASIS		CMD

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
CMDAREA	No	HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname PMTLOC= <u>BEFORE</u> NOINIT PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname NAME=cmdarea-variable-name ENTWIDTH=n PMTTEXT= <u>YES</u> NO CMDLOC= <u>DEFAULT</u> ASIS CMDLEN= <u>DEFAULT</u> MAX AUTOTAB= <u>NO</u> YES SCROLLVAR=scroll-variable SCRVHELP= <u>NO</u> YES scroll-help-panel-name *scroll-help-message-id %varname *%varname SCROLLTAB= <u>NO</u> YES SCRCAPS= <u>OFF</u> ON PSBUTTON=cmd-pb-text PSVAR=point-and-shoot-variable %varname PSVAL=point-and-shoot-value %varname IMAPNAME=image-name %varname IMAPNAMEP=image-namep %varname PLACE= <u>ABOVE</u> BELOW LEFT RIGHT %varname CAPS= <u>OFF</u> ON NOJUMP= <u>OFF</u> ON VARDCL= <u>YES</u> NO <i>command-prompt-text</i>	HP	PANEL
CMDTBL	Yes	APPLID=application-identifier SORT= <u>NO</u> YES	CMD	
COMMENT	No	TYPE= <u>END</u> CCSID PANEL ATTR ABCINIT ABCPROC INIT REINIT PROC HELP PNTS LIST <i>comment-text</i>		ABC AREA CHOICE DA DTACOL DTAFLD HELP LSTCOL LSTFLD LSTGRP MSGMBR PANEL PDC REGION SELFELD

Table 84. Tag summary (continued)

Tag	End tag	Attributes	Nested tags	Used within
COMPOPT	No	<u>REPLACE</u> NOREPLACE <u>SCREEN</u> DISK <u>NODBCS</u> DBCS NOKANA KANA KEYLAPPL=xxxx <u>NOPANEL</u> PANEL <u>NOMSGSUPP</u> MSGSUPP <u>NOCUASUPP</u> CUASUPP <u>PREP</u> NOPREP <u>CUAATTR</u> NOCUAATTR <u>NOLSTVIEW</u> LSTVIEW <u>STATS</u> NOSTATS <u>NOSCRIP</u> SCRIPT <u>NOLISTING</u> LISTING <u>NOFORMAT</u> FORMAT <u>NOMSGEXPAND</u> MSGEXPAND <u>LOGREPL</u> NOLOGREPL <u>LISTREPL</u> NOLISTREPL <u>ACTBAR</u> NOACTBAR <u>GUI</u> NOGUI <u>VERSION</u> NOVERSION <u>NOMERGESAREA</u> MERGESAREA <u>NODISPLAY</u> DISPLAY <u>NODISPLAYW</u> DISPLAYW <u>DSNCHK</u> NODSNCHK <u>GRAPHIC</u> NOGRAPHIC <u>ZVARS</u> NOZVARS <u>NODBALIGN</u> DBALIGN <u>NOMCOMMENT</u> MCOMMENT <u>NOVPADC</u> PADC ADD RESET <i>national-language</i>	None	
COPYR	No	<i>copyright-text</i>		

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
DA	Yes	NAME=varname EXTEND= <u>OFF</u> ON FORCE LVLINE=variable-name SCROLL= <u>OFF</u> ON CMDLINE USERMOD=usermod-code %varname DATAMOD=datamod-code %varname DEPTH=n * WIDTH=n SHADOW=shadow-name DIV= <u>NONE</u> BLANK SOLID DASH TEXT FORMAT=START CENTER END TEXT=divider-text SCROLLVAR=scroll-variable SCRVHELP= <u>NO</u> YES scroll-help-panel-name *scroll-help-message-id %varname *%varname SCROLLTAB= <u>NO</u> YES SCRCAPS= <u>OFF</u> ON INITATTR= <u>NT</u> CT ET WT WASL HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname	ATTR COMMENT SOURCE	AREA PANEL REGION
DD	No	<i>definition-description</i>	DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL XMP	DL
DDHD	No	<i>definition-description-header</i>	HP PS RP	DL
DIVIDER	No	TYPE= <u>NONE</u> SOLID DASH TEXT GAP= <u>YES</u> NO GUTTER= <u>1</u> n NOENDATTR FORMAT=START CENTER END <i>divider-text</i>	HP	AREA DTACOL PANEL REGION

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
DL	Yes	TSIZE= <u>10</u> 'S1, S2,... Sn' BREAK= <u>NONE</u> FIT ALL COMPACT NOSKIP INDENT=n FORMAT=START CENTER END DIVEND= <u>NO</u> YES SPLIT= <u>NO</u> YES	DD DDHD DLDIV DT DTHD DTDIV DTHDIV	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING XMP
DLDIV	No	TYPE= <u>NONE</u> SOLID DASH TEXT GAP= <u>YES</u> NO GUTTER= <u>1</u> n FORMAT=START CENTER END <i>divider-text</i>	HP	DL
DT	No	FORMAT=START CENTER END NOSKIP SPLIT= <u>NO</u> YES <i>definition-term</i>	DTSEG HP PS RP	DL
DTACOL	Yes	PMTWIDTH=n * ** ENTWIDTH=n DESWIDTH=n * SELWIDTH=n * FLDSpace=n PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname PMTFMT= <u>CUA</u> ISPF NONE END AUTOTAB= <u>NO</u> YES ATTRCHANGE= <u>NO</u> YES NEW PMTLOC= <u>BEFORE</u> ABOVE DBALIGN= <u>YES</u> NO PROMPT FIELD FORCE VARCLASS=variable-class-name REQUIRED= <u>NO</u> YES CAPS= <u>OFF</u> ON VARDCL= <u>YES</u> NO	COMMENT DIVIDER DTAFLD GRPHDR SELFLD SOURCE	AREA PANEL REGION

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
DTAFLD	No	NAME=field-name DATAVAR=field-data VARCLASS=variable-class-name HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname USAGE= <u>BOTH</u> IN OUT REQUIRED= <u>NO</u> YES MSG=message-identifier AUTOTAB= <u>NO</u> YES ENTWIDTH=n PMTWIDTH=n * ** DESWIDTH=n * FLDSPACE=n ALIGN= <u>START</u> CENTER END PMTLOC= <u>BEFORE</u> ABOVE DISPLAY= <u>YES</u> NO NOENDATTR PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname PMTFMT= <u>CUA</u> ISPF NONE END PSVAR=point-and-shoot-variable %varname PSVAL=point-and-shoot-value %varname PAS=%varname CSRGRP= <u>NO</u> YES n EXPAND FLDWIDTH=n ATTRCHANGE= <u>NO</u> YES NEW INIT=initial-value DEPTH=n %varname IMAPNAME=image-name %varname IMAPNAMEP=image-namep %varname PLACE= <u>ABOVE</u> BELOW LEFT RIGHT %varname DBALIGN= <u>YES</u> NO PROMPT FIELD FORCE PMTSKIP= <u>NO</u> YES DESSKIP= <u>NO</u> YES FLDTYPE= <u>CUA</u> ISPF COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname ATTRCHAR=code CAPS= <u>OFF</u> ON NOJUMP= <u>OFF</u> ON AUTOTYPE=PROJECT GROUP1 GROUP2 GROUP3 GROUP4 TYPE MEMBER DSN AUTOVOL=volser-name AUTODMEM= <u>YES</u> NO VARDCL= <u>YES</u> NO prompt-text	ASSIGNL COMMENT DTAFLDD HP PS RP SOURCE SCRFLD	AREA DTACOL PANEL REGION

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
DTAFLDD	No	<i>description</i>	HP PS RP	DTAFLD
DTDIV	No			DL
DTHD	No	<i>definition-term-header</i>	HP PS RP	DL
DTHDIV	No			DL
DTSEG	No			DT
FIG	Yes	FRAME= <u>RULE</u> NONE WIDTH= <u>PAGE</u> COL NOSKIP <i>figure-content</i>	DL FIGCAP HP NOTE NOTEL NT OL P PARML PS RP SL UL XMP	ATTENTION CAUTION DD INFO LI LP NT PD WARNING
FIGCAP	No	<i>figure-caption-text</i>	HP PS RP	FIG
GA	No	NAME=graphic-area-name EXTEND= <u>OFF</u> ON FORCE DEPTH=n * WIDTH=n DIV= <u>NONE</u> BLANK SOLID DASH TEXT FORMAT=START CENTER END TEXT=divider-text LVLINE=variable-name		AREA PANEL REGION
GENERATE	Yes	SUBSTITUTE= <u>NO</u> YES	ATTR COMMENT SOURCE	AREA HELP PANEL REGION

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
GRPHDR	No	FORMAT= <u>START</u> CENTER END NONE WIDTH=n FMTWIDTH=n INDENT=n HEADLINE= <u>NO</u> YES DIV= <u>NONE</u> BLANK SOLID DASH DIVLOC= <u>AFTER</u> BEFORE BOTH COMPACT STRIP <i>group-heading-text</i>	HP PS RP	AREA DTACOL PANEL REGION
HELP	Yes	NAME=help-panel-name HELP=hhelp-panel-name %varname HELPDEF=helpdef-id WIDTH= <u>50</u> n FIT DEPTH= <u>10</u> n FIT CCSID=n TUTOR KEYLIST=key-list-name KEYLTYPE= <u>PRIVATE</u> SHARED APPLID=application-id EXPAND=xy WINTITLE=window-title APPTITLE=application-title MERGESAREA= <u>NO</u> YES MSGLINE= <u>YES</u> NO IMAPNAME=image-name %varname IMAPROW=n %varname IMAPCOL=n %varname ZUP=zup-id ZCONT=zcont-id <i>help-panel-title</i>	AREA COMMENT DIVIDER GENERATE HP INFO REGION SOURCE TEXTLINE	
HELPDEF	No	ID=helpdef-id HELP=hhelp-panel-name %varname WIDTH=n FIT DEPTH=n FIT CCSID=n KEYLIST=key-list-name KEYLTYPE= <u>PRIVATE</u> SHARED APPLID=application-id EXPAND=xy WINTITLE=window-title APPTITLE=application-title MERGESAREA= <u>NO</u> YES IMAPNAME=image-name %varname IMAPROW=n %varname IMAPCOL=n %varname		
H1	No	COMPACT <i>heading-text</i>		INFO

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
H2/H3/H4	No	COMPACT <i>heading-text</i>	HP PS RP	INFO
HP	Yes	TYPE= <u>ET</u> CH CT FP LEF LI NT PT SAC TEXT WASL WT COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname INTENSE=varname <i>phrase-to-be-highlighted</i>		ATTENTION BOTINST CAUTION CHDIV CHOICE CMDAREA DD DDHD DIVIDER DT DTAFLD DTAFLDD DTHD FIG FIGCAP GRPHDR H2 H3 H4 HELP LI LINES LP LSTCOL LSTGRP NOTE NT P PANEL PD PNLINST PT SELFLD TOPINST WARNING XMP

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
INFO	Yes	WIDTH=format-width * INDENT=n	DIVIDER DL FIG Hn LINES NOTE NOTEL NT OL P PARML SL SOURCE UL XMP	AREA HELP PANEL REGION
KEYI	No	KEY=virtual-key CMD=internal-command-name CASE= <u>UPPER</u> MIXED FKA= <u>NO</u> YES LONG SHORT PARM=parm-string <i>FKA-text</i>		KEYL
KEYL	Yes	NAME=key-list-name HELP=help-panel-name ACTION=UPDATE DELETE APPLID=application-id	KEYI	
LI	No	SPACE= <u>NO</u> YES NOSKIP <i>item-text</i>	ATTENTION CAUTION DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL WARNING XMP	NOTEL OL SL UL

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
LINES	Yes	NOSKIP <i>text</i>	DL HP NOTE NOTEL NT OL P PARML PS RP SL UL XMP	ATTENTION CAUTION DD INFO LI LP NT PD WARNING
LIT	Yes	<i>literal-display-value</i>		XLATI
LP	No	NOSKIP <i>implied-paragraph</i>	ATTENTION CAUTION DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL WARNING XMP	NOTEL OL SL UL

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
LSTCOL	No	DATAVAR=column-data VARCLASS=variable-class-name HELP= <u>NO</u> YES help-panel-name * help-message-id %varname *%varname USAGE= <u>BOTH</u> IN OUT REQUIRED= <u>NO</u> YES MSG=message-id COLWIDTH=data-width ALIGN= <u>START</u> CENTER END AUTOTAB= <u>NO</u> YES LINE=n CLEAR POSITION=n FORMAT= <u>START</u> CENTER END TEXT=descriptive-text TEXTLOC= <u>BEFORE</u> AFTER TEXTFMT= <u>START</u> CENTER END TEXTLEN=n TEXTSKIP= <u>NO</u> YES NOENDATTR PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname PAS= <u>OFF</u> ON %varname CSRGRP= <u>NO</u> YES n ATTRCHANGE= <u>NO</u> YES NEW COLSPACE=n COLTYPE= <u>CUA</u> ISPF EE VOI LID COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname CAPS= <u>OFF</u> ON DISPLAY= <u>YES</u> NO VARDCL= <u>YES</u> NO <i>column-heading</i>	COMMENT HP PS RP SOURCE SCRFLD	LSTFLD LSTGRP
LSTFLD	Yes	RULES= <u>NONE</u> HORIZ VERT BOTH ROWS= <u>NOSCAN</u> SCAN %varname DIV= <u>NONE</u> BLANK SOLID DASH char SCROLLVAR=scroll-variable SCRHELP= <u>NO</u> YES scroll-help-panel-name *scroll-help-message-id %varname *%varname SCROLLTAB= <u>NO</u> YES SCRCAPS= <u>OFF</u> ON ATTRCHANGE= <u>NO</u> YES NEW VARDCL= <u>YES</u> NO	COMMENTL STCOL LSTGRP LSTVAR SOURCE	AREA PANEL REGION

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
LSTGRP	Yes	HEADLINE= <u>NO</u> YES DASH ALIGN= <u>CENTER</u> START END <i>column-group-heading</i>	COMMENT HP LSTCOL LSTGRP LSTVAR PS RP SOURCE	LSTFLD LSTGRP
LSTVAR	No	DATAVAR=variable-model-name LINE=n <i>column-heading</i>	COMMENT HP PS RP SOURCE	LSTFLD LSTGRP
M	No	<i>mnemonic-character</i>		ABC PDC
MSG	No	SUFFIX=message-suffix-number HELP=help-panel-name %varname * MSGTYPE= <u>INFO</u> WARNING ACTION CRITICAL %varname LOCATION=AREA MODAL MODAL(L) MODELESS MODELESS (L) %varname DISP=KANA NOKANA ALARM= <u>NO</u> YES %varname ABBREV= <u>NONE</u> KEYWORD VALUE BOTH FORMAT= <u>FLOW</u> ASIS SMSG=short-message-text <i>message-text</i>	VARSUB	MSGMBR
MSGMBR	Yes	NAME=message-member-name CCSID=n WIDTH= <u>76</u> 68	COMMENTM SG	
NOTE	No	NOSKIP INDENT=n TYPE= <u>ET</u> CH CT FP LEF LI NT PT SAC TEXT WASL WT COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname TEXT=alternate-note-heading <i>note-text</i>	HP PS RP	ATTENTION CAUTION DD FIG INFO LI LINES LP PD WARNING XMP

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
NOTEL	Yes	COMPACT NOSKIP SPACE= <u>NO</u> YES INDENT= <u>n</u> TYPE= <u>ET</u> CH CT FP LEF LI NT PT SAC TEXT WASL WT COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname TEXT=alternate-note-heading	LI LP	ATTENTION CAUTION DD FIG INFO LI LINES LP PD WARNING XMP
NT	Yes	NOSKIP INDENT= <u>n</u> TYPE= <u>ET</u> CH CT FP LEF LI NT PT SAC TEXT WASL WT COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname TEXT=alternate-note-heading <i>note-text</i>	DL FIG HP LINES OL P PARML PS RP SL UL XMP	ATTENTION CAUTION DD FIG INFO LI LINES LP PD WARNING XMP
OL	Yes	COMPACT NOSKIP SPACE= <u>NO</u> YES INDENT= <u>n</u> TEXT=OL-heading-text	LI LP	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING XMP
P	No	COMPACT INTENSE=varname INDENT= <u>n</u> OFFSET= <u>n</u> SPACE= <u>NO</u> YES <i>paragraph-text</i>	ATTENTION CAUTION HP PS RP WARNING	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING XMP

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
PANDEF	No	ID=pandef-id HELP=help-panel-name %varname DEPTH=n FIT WIDTH=n FIT %varname KEYLIST=key-list-name KEYLTYPE= <u>PRIVATE</u> SHARED APPLID=application-id CCSID=n WINDOW= <u>YES</u> NO WINTITLE=window-title APPTITLE=application-title PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname EXPAND=xy MERGESAREA= <u>NO</u> YES ENTKEYTEXT=enter-key-text IMAPNAME=image-name %varname IMAPROW=n %varname IMAPCOL=n %varname TMARGIN=n BMARGIN=n		

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
PANEL	Yes	NAME=panel-name HELP=help-panel-name %varname PANDEF=pandef-id DEPTH= <u>22</u> n FIT WIDTH= <u>76</u> n FIT %varname KEYLIST=key-list-name KEYLTYPE= <u>PRIVATE</u> SHARED APPLID=application-id CURSOR=cursor-field CSRINDEX=index-value CSRPOS=position-value CCSID=n MENU PRIME TUTOR WINDOW= <u>YES</u> NO WINTITLE=window-title APPTITLE=application-title PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname EXPAND=xy MSGLINE= <u>YES</u> NO TITLINE= <u>YES</u> NO CMDLINE= <u>YES</u> NO ATTRUSE= <u>NO</u> YES ALL ENDATTR= <u>DEFAULT</u> TEXT TYPE= <u>BOTH</u> GUI NOGUI SMSG=short-msg-fieldname LMSG=long-msg-fieldname ASIS ACTBAR MERGESAREA= <u>NO</u> YES PANELSTMT= <u>YES</u> NO ENTKEYTEXT=enter-key-text IMAPNAME=image-name %varname IMAPROW=n %varname IMAPCOL=n %varname TMARGIN=n BMARGIN=n ERRORCHECK= <u>NO</u> YES ZUP=zup-id ZCONT=zcont-id AUTONRET= <u>NO</u> YES AUTOTCMD= <u>NO</u> YES PROC panel-title-text	AB AREA BOTINST CMDAREA COMMENT DA DIVIDER DTACOL DTAFLD GA GENERATE GRPHDR HP INFO LSTFLD PNLINST REGION SELFID SOURCE TEXTLINE TOPINST	

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
PARML	Yes	TSIZE= <u>10</u> 'S1 S2... Sn' BREAK= <u>ALL</u> FIT NONE COMPACT SKIP INDENT=n FORMAT=START CENTER END DIVEND= <u>NO</u> YES SPLIT= <u>NO</u> YES	PLDIV PT PTDIV PD	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING XMP
PD	No	<i>parameter-description</i>	DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL XMP	PARML
PDC	No	HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname UNAVAIL=unavail-variable-name CHECKVAR=check-variable-name MATCH= <u>1</u> match-string ACC1=key1 ACC2=key2 ACC3=key3 <i>pull-down-description-text</i>	ACTION COMMENT M SOURCE	ABC
PDSEP	No			PDC
PLDIV	No	TYPE= <u>NONE</u> SOLID DASH TEXT GAP= <u>YES</u> NO GUTTER= <u>1</u> n FORMAT=START CENTER END <i>divider-text</i>	HP	PARML
PNLINST	No	COMPACT <i>instruction-text</i>	HP PS RP	AREA REGION PANEL

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
PS	Yes	VAR=point-and-shoot-variable-name %varname VALUE=point-and-shoot-value %varname * CSRGRP= <u>NO</u> YES n DEPTH=n %varname IMAPNAME=image-name %varname IMAPNAMEP=image-namep %varname PLACE= <u>ABOVE</u> BELOW LEFT RIGHT %varname <i>point-and-shoot-text</i>		ATTENTION BOTINST CAUTION CHOFLD CHOICE DD DDHD DT DTAFLD DTAFLDD DTHD FIG FIGCAP GRPHDR H2 H3 H4 LI LINES LP LSTCOL LSTGRP NOTE NT P PD PNLINST PT SELFLD TOPINST WARNING XMP
PT	No	FORMAT= <u>START</u> CENTER END NOSKIP SPLIT= <u>NO</u> YES <i>parameter-term</i>	HP PS PTSEG RP	PARML
PTDIV	No			PARML
PTSEG	No			PT

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
REGION	Yes	DIR= <u>VERT</u> HORIZ INDENT=n WIDTH=n * DEPTH=n * EXTEND= <u>OFF</u> ON FORCE ALIGN= <u>YES</u> NO GRPBOX= <u>NO</u> YES GRPWIDTH=n GRPBXVAR=variable-name GRPBXMAT= <u>1</u> string LOCATION= <u>DEFAULT</u> TITLE <i>group-box-title</i>	COMMENT DA DIVIDER DTACOL DTAFLD GA GENERATE GRPHDR INFO LSTFLD PNLINST REGION SELFLD SOURCE	AREA HELP PANEL REGION
RP	Yes	HELP= help-panel-name help-message-id %varname *%varname <i>reference-phrase</i>		ATTENTION BOTINST CAUTION CHOFLD CHOICE DD DDHD DT DTAFLD DTAFLDD DTHD FIG FIGCAP GRPHDR H2 H3 H4 LI LINES LP LSTCOL LSTGRP NOTE NT P PD PNLINST PT SELFLD TOPINST WARNING XMP

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
SCRFLD	Yes	DISPLEN= n %varname INDVAR=ind-var INDVAL='ind-chars' LINDVAR=lind-var LINDVAL='lind-char' RINDVAR=rind-var RINDVAL='rind-char' SINDVAR=sind-var SINDVAL='sind-chars' LCOLIND=lcol-var LCOLDISP= <u>NO</u> YES RCOLIND=rcol-var RCOLDISP= <u>NO</u> YES SCALE=scale-var SCROLL= <u>ON</u> OFF %varname FLDSPOS= <u>BELOW</u> ABOVE	COMMENT SOURCE	DTAFLD LSTCOL

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
SELFLD	Yes	NAME=field-name HELP= <u>NO</u> YES help-panel-name *help-message-id %varname *%varname TYPE= <u>SINGLE</u> MULTI MENU MODEL TUTOR PMTLOC= <u>ABOVE</u> BEFORE PMTWIDTH=n * ** SELWIDTH=n * ENTWIDTH= <u>2</u> n 'e1 e2...en' REQUIRED= <u>NO</u> YES MSG=message-identifier FCHOICE= <u>1</u> 0 AUTOTAB= <u>YES</u> NO DEPTH=n * EXTEND= <u>OFF</u> ON FORCE TRAIL='trail-var-1 trail-var-2 ... trail-var-n' CHOICECOLS= <u>1</u> n CHOICEDEPTH=n * CWIDTHS='w1 w2...wn' PAD=NULLS USER char %varname PADC=NULLS USER char %varname OUTLINE= <u>NONE</u> L R O U BOX %varname SELMSG=selfld-msg-identifier SELMSGU=selfld-msg-unavailable INIT= <u>YES</u> NO init-value VERIFY= <u>YES</u> NO REFRESH= <u>YES</u> NO SELFMT= <u>START</u> END CHKBOX= <u>YES</u> NO ZGUI= <u>YES</u> NO CSRGRP= <u>NO</u> YES n TSIZE='s1 s2...sn' LISTTYPE=RADIO LISTBOX DDLIST COMBO LISTREF=list-name LISTDEPTH=n DBALIGN= <u>YES</u> NO FIELD FORCE NOSEL=no-selection-value SELDEFAULT=x PMTSKIP= <u>NO</u> YES FLDTYPE= <u>CUA</u> ISPF COLOR=WHITE RED BLUE GREEN PINK YELLOW TURQ %varname INTENS= <u>HIGH</u> LOW NON %varname HILITE=USCORE BLINK REVERSE %varname SELCHECK= <u>NO</u> YES VARDCL= <u>YES</u> NO <i>field-prompt-text</i>	CHDIV CHOICE COMMENT HP PS RP SOURCE	AREA DTACOL PANEL REGION

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
SL	Yes	COMPACT NOSKIP SPACE= <u>NO</u> YES INDENT=n TEXT='SL-heading-text'	LI LP	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING XMP
SOURCE	Yes	TYPE= <u>PROC</u> REINIT INIT ABCINIT ABCPROC <i>text</i>		ABC AREA CHOICE DA DTACOL DTAFLD HELP LSTCOL LSTFLD LSTGRP PANEL PDC REGION SELFELD
T	No			CMD
TEXTLINE	Yes		DTAFLD TEXTSEG	HELP PANEL
TEXTSEG	No	EXPAND=AFTER BEFORE BOTH WIDTH=n <i>text</i>	HP	TEXTLINE
TOPINST	No	COMPACT <i>instruction-text</i>	HP PS RP	PANEL

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
UL	Yes	COMPACT NOSKIP SPACE= <u>NO</u> YES INDENT=n TEXT=UL-heading-text	LI LP	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING XMP
VARCLASS	No	NAME=variable-class-name TYPE='CHAR maximum length' 'DBCS maximum length' 'MIXED maximum length' 'ANY maximum length' 'EBCDIC maximum length' '%varname maximum length' ITIME STDTIME IDATE STDDATE JDATE JSTD 'VMASK maximum-length' 'NUMERIC total-digits <u>0</u> fractional-digits' MSG=message-identifier	CHECKL XLATL	
VARDCL	No	NAME=name VARCLASS=variable-class-name		VARLIST
VARLIST	Yes		VARDCL	
VARSUB	No	VAR=variable-name		MSG
WARNING	Yes	<i>text</i>	DL FIG HP LINES NOTE NOTEL NT OL P PARML PS RP SL UL XMP	LI LP P

Table 84. Tag summary (continued)				
Tag	End tag	Attributes	Nested tags	Used within
XLATI	No	VALUE=internal-value <i>displayed-value</i>	LIT	XLATL
XLATL	Yes	FORMAT= <u>NONE</u> UPPER TRUNC=n char MSG=message-identifier	XLATI	VARCLASS
XMP	Yes	NOSKIP <i>text</i>	DL HP NOTE NOTEL NT OL P PARML PS RP SL UL	ATTENTION CAUTION DD FIG INFO LI LINES LP NT PD WARNING

Appendix B. Accessibility

Accessible publications for this product are offered through [IBM Documentation for z/OS \(www.ibm.com/docs/en/zos\)](http://www.ibm.com/docs/en/zos).

If you experience difficulty with the accessibility of any z/OS documentation see [How to Send Feedback to IBM](#) to leave documentation feedback.

Notices

This information was developed for products and services that are offered in the USA or elsewhere.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

*IBM Director of Licensing
IBM Corporation
North Castle Drive, MD-NC119
Armonk, NY 10504-1785
United States of America*

For license inquiries regarding double-byte character set (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

*Intellectual Property Licensing
Legal and Intellectual Property Law
IBM Japan Ltd.
19-21, Nihonbashi-Hakozakicho, Chuo-ku
Tokyo 103-8510, Japan*

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

This information could include missing, incorrect, or broken hyperlinks. Hyperlinks are maintained in only the HTML plug-in output for IBM Documentation. Use of hyperlinks in other output formats of this information is at your own risk.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

*IBM Corporation
Site Counsel
2455 South Road*

Poughkeepsie, NY 12601-5400
USA

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

Any performance data contained herein was determined in a controlled environment. Therefore, the results obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

All statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Terms and conditions for product documentation

Permissions for the use of these publications are granted subject to the following terms and conditions.

Applicability

These terms and conditions are in addition to any terms of use for the IBM website.

Personal use

You may reproduce these publications for your personal, noncommercial use provided that all proprietary notices are preserved. You may not distribute, display or make derivative work of these publications, or any portion thereof, without the express consent of IBM.

Commercial use

You may reproduce, distribute and display these publications solely within your enterprise provided that all proprietary notices are preserved. You may not make derivative works of these publications, or

reproduce, distribute or display these publications or any portion thereof outside your enterprise, without the express consent of IBM.

Rights

Except as expressly granted in this permission, no other permissions, licenses or rights are granted, either express or implied, to the publications or any information, data, software or other intellectual property contained therein.

IBM reserves the right to withdraw the permissions granted herein whenever, in its discretion, the use of the publications is detrimental to its interest or, as determined by IBM, the above instructions are not being properly followed.

You may not download, export or re-export this information except in full compliance with all applicable laws and regulations, including all United States export laws and regulations.

IBM MAKES NO GUARANTEE ABOUT THE CONTENT OF THESE PUBLICATIONS. THE PUBLICATIONS ARE PROVIDED "AS-IS" AND WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, AND FITNESS FOR A PARTICULAR PURPOSE.

IBM Online Privacy Statement

IBM Software products, including software as a service solutions, ("Software Offerings") may use cookies or other technologies to collect product usage information, to help improve the end user experience, to tailor interactions with the end user, or for other purposes. In many cases no personally identifiable information is collected by the Software Offerings. Some of our Software Offerings can help enable you to collect personally identifiable information. If this Software Offering uses cookies to collect personally identifiable information, specific information about this offering's use of cookies is set forth below.

Depending upon the configurations deployed, this Software Offering may use session cookies that collect each user's name, email address, phone number, or other personally identifiable information for purposes of enhanced user usability and single sign-on configuration. These cookies can be disabled, but disabling them will also eliminate the functionality they enable.

If the configurations deployed for this Software Offering provide you as customer the ability to collect personally identifiable information from end users via cookies and other technologies, you should seek your own legal advice about any laws applicable to such data collection, including any requirements for notice and consent.

For more information about the use of various technologies, including cookies, for these purposes, see IBM's Privacy Policy at ibm.com/privacy and IBM's Online Privacy Statement at ibm.com/privacy/details in the section entitled "Cookies, Web Beacons and Other Technologies," and the "IBM Software Products and Software-as-a-Service Privacy Statement" at ibm.com/software/info/product-privacy.

Policy for unsupported hardware

Various z/OS elements, such as DFSMSdfp, JES2, and MVS, contain code that supports specific hardware servers or devices. In some cases, this device-related element support remains in the product even after the hardware devices pass their announced End of Service date. z/OS may continue to service element code; however, it will not provide service related to unsupported hardware devices. Software problems related to these devices will not be accepted for service, and current service activity will cease if a problem is determined to be associated with out-of-support devices. In such cases, fixes will not be issued.

Minimum supported hardware

The minimum supported hardware for z/OS releases identified in z/OS announcements can subsequently change when service for particular servers or devices is withdrawn. Likewise, the levels of other software products supported on a particular release of z/OS are subject to the service support lifecycle of those

Trademarks

products. Therefore, z/OS and its product publications (for example, panels, samples, messages, and product documentation) can include references to hardware and software that is no longer supported.

- For information about software support lifecycle, see: [IBM Lifecycle Support for z/OS \(www.ibm.com/software/support/systemsz/lifecycle\)](http://www.ibm.com/software/support/systemsz/lifecycle)
- For information about currently-supported IBM hardware, contact your IBM representative.

Programming Interface Information

This publication primarily documents information that is NOT intended to be used as Programming Interfaces of ISPF.

This publication also documents intended Programming Interfaces that allow the customer to write programs to obtain the services of ISPF. This information is identified where it occurs, either by an introductory statement to a chapter or section or by the following marking:

```
+-----Programming Interface information-----+  
  
+-----End of Programming Interface information-----+
```

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at [Copyright and Trademark information \(www.ibm.com/legal/copytrade.shtml\)](http://www.ibm.com/legal/copytrade.shtml).

Index

Special Characters

- % keyword
 - on ENTITY statement [172](#)
- % notation [12](#)
- <: notation
 - in doctype declaration [15](#)

A

- AB (action bar) tag
 - conditions of usage [180](#)
 - defining an action bar [33](#)
 - description [180](#)
 - examples
 - adding HELP attribute [36](#)
 - application panel markup [181](#)
 - defining help panel [36](#)
 - markup [35](#)
 - using mnemonic selection [37](#)
 - how to code [34](#)
 - syntax [179](#)
 - See also action bar
- ABBREV attribute
 - on MSG tag [354](#)
- ABC (action bar choice) tag
 - conditions of usage [183](#)
 - description [183](#)
 - examples
 - defining help panel [36](#)
 - markup [35](#)
 - PDCVAR attribute [183](#)
 - using mnemonic selection [37](#)
 - syntax [182](#)
- ABSEPCHAR attribute on AB tag [180](#)
- ABSEPSTR attribute on AB tag [180](#)
- ACC1 attribute on PDC tag [392](#)
- ACC2 attribute on PDC tag [392](#)
- ACC3 attribute on PDC tag [392](#)
- accessibility
 - contact IBM [493](#)
- ACTBAR attribute
 - on PANEL tag [382](#)
- ACTBAR option [160](#)
- ACTION (action) tag
 - conditions of usage [188](#)
 - description [188](#)
 - examples
 - defining help panel [36](#)
 - markup [35](#)
 - PDC tags and SETVAR attribute [189](#)
 - using mnemonic selection [37](#)
 - for pull-down choice [35](#)
 - syntax [184](#)
- ACTION attribute
 - on CMDACT tag [235](#)
 - on KEYL tag [323](#)
- ACTION attribute (*continued*)
 - value ALIAS on CMDACT tag [235](#)
 - value application-command on CMDACT tag [236](#)
 - value BACKWARD on CMDACT tag [236](#)
 - value CANCEL on CMDACT tag [236](#)
 - value EXHELP on CMDACT tag [236](#)
 - value EXIT on CMDACT tag [236](#)
 - value FKA on CMDACT tag [236](#)
 - value FORWARD on CMDACT tag [236](#)
 - value HELP on CMDACT tag [236](#)
 - value PANELID on CMDACT tag [236](#)
 - value PASSTHRU on CMDACT tag [236](#)
 - value RETRIEVE on CMDACT tag [236](#)
 - value SELECT on CMDACT tag [235](#)
 - value SETVERB on CMDACT tag [236](#)
- action bar
 - defining [33](#)
 - description [5](#)
 - providing help for [35](#)
 - See also AB (action bar) tag
- action bar (AB) tag
 - conditions of usage [180](#)
 - description [180](#)
 - examples [181](#)
 - syntax [179](#)
- action bar choice [36](#)
 - See also ABC (action bar choice) tag
- action bar choice (ABC) tag
 - conditions of usage [183](#)
 - description [183](#)
 - examples [183](#)
 - syntax [182](#)
- action message
 - defining [138](#)
 - description [138](#)
- ADDPop attribute on ACTION tag [187](#)
- ALARM attribute
 - on MSG tag [354](#)
- alerting users
 - using the ATTENTION tag [121](#)
 - using the CAUTION tag [121](#)
 - using the NT tag [121](#)
 - using the WARNING tag [121](#)
- ALIAS command action
 - defining [145](#)
 - description [145](#)
- ALIGN attribute
 - on CHOFLD tag [222](#)
 - on DTAFLD tag [279](#)
 - on LSTCOL tag [335](#)
 - on LSTGRP tag [345](#)
 - on REGION tag [406](#)
 - using [77](#)
- ALPHA test
 - description [61](#)
 - example [61](#)
 - specifying on CHECKI tag [212](#)

- ALPHAB test
 - description [65](#)
 - example [65](#)
 - specifying on CHECKI tag [215](#)
- alphabetic test
 - description [61](#)
 - example [61](#)
- ALTDESCR attribute on CMD tag [233](#)
- APPLCMD attribute on ACTION tag [186](#)
- application command table
 - defining commands [143](#)
 - defining with CMDTBL tag [143](#)
 - overview [8](#)
 - See also* commands
- application panel
 - action bar [5](#)
 - bottom instruction [6](#)
 - command area [6](#)
 - defining a command area [48](#)
 - defining a panel ID [28](#)
 - defining a region [45](#)
 - defining action bar and pull-downs [33](#)
 - defining an area divider [44](#)
 - defining common attributes and values [50](#)
 - defining common elements [27](#)
 - defining cursor placement [30](#)
 - defining panel width and depth [29](#)
 - defining the panel title [29](#)
 - defining top and bottom instructions [37](#)
 - defining with the PANEL tag [27](#)
 - description of PANEL tag [383](#)
 - examples of PANEL tag [384](#)
 - function key area [6](#)
 - layout [5](#)
 - overview [5](#)
 - panel body [6](#)
 - panel title [5](#)
 - providing help for [29](#)
 - specifying a key mapping list [29](#)
 - syntax for PANEL tag [376](#)
 - top instruction [6](#)
- application-id
 - ISPx [143](#)
 - option [158](#)
 - using with CMDTBL tag [143](#)
- APPLID attribute
 - description [143](#)
 - on CMDTBL tag [244](#)
 - on HELP TAG [306](#)
 - on HELPDEF TAG [312](#)
 - on KEYL TAG [323](#)
 - on PANDEF TAG [374](#)
 - on PANEL TAG [380](#)
- APPTITLE attribute
 - on HELP TAG [306](#)
 - on HELPDEF TAG [312](#)
 - on PANDEF tag [374](#)
 - on PANEL tag [381](#)
- AREA (area) tag
 - conditions of usage [192](#)
 - defining panel text portion [38](#)
 - description [191](#)
 - examples [38](#), [193](#)
 - syntax [189](#)
- area divider (DIVIDER) tag
 - conditions of usage [260](#)
 - description [260](#)
 - examples [260](#)
 - syntax [258](#)
- area divider (DLDIV) tag
 - conditions of usage [266](#)
 - description [266](#)
 - examples [266](#)
- ASIS attribute
 - on PANEL tag [382](#)
- ASSIGNI (assignment list item) tag
 - conditions of usage [196](#)
 - description [196](#)
 - examples [196](#)
 - syntax [195](#)
- ASSIGNL (assignment list) tag
 - conditions of usage [197](#)
 - description [197](#)
 - examples [197](#)
 - syntax [196](#)
- assignment list (ASSIGNL) tag
 - conditions of usage [197](#)
 - description [197](#)
 - examples [197](#)
 - syntax [196](#)
- assignment list item (ASSIGNI) tag
 - conditions of usage [196](#)
 - description [196](#)
 - examples [196](#)
 - syntax [195](#)
- assistive technologies [493](#)
- ATTENTION (attention) tag
 - conditions of usage [198](#)
 - description [198](#)
 - syntax [198](#)
- attention message
 - description
 - format [124](#)
- ATTENTION tag
 - description [124](#)
- ATTN attribute
 - on ATTR tag [203](#)
- ATTR (attribute) tag
 - conditions of usage [203](#)
 - description [203](#)
 - examples [203](#)
 - syntax [200](#)
- ATTRCHANGE attribute
 - description
 - data field [78](#)
 - list field [94](#)
 - on CHOFLD tag [223](#)
 - on DTACOL tag [272](#)
 - on DTAFLD tag [280](#)
 - on LSTCOL tag [336](#)
 - on LSTFLD tag [342](#)
 - using [78](#), [94](#)
- ATTRCHAR attribute
 - description
 - data field [79](#)
 - on ATTR tag [202](#)
 - on DTAFLD tag [281](#)
 - using [79](#)

- attribute
 - description [11](#)
 - tag [11](#)
- attribute (ATTR) tag
 - conditions of usage [203](#)
 - description [203](#)
 - examples [203](#)
 - syntax [200](#)
- attributes and values, coding [11](#)
- ATTRUSE attribute
 - on PANEL tag [381](#)
- AUTODMEM attribute
 - description
 - data field [79](#)
 - on DTAFLD tag [282](#)
 - using [79](#)
- AUTONRET attribute
 - on PANEL tag [383](#)
- AUTOSEL attribute
 - on CHOICE tag [230](#)
- AUTOTAB [87](#)
- AUTOTAB attribute
 - description [89](#), [94](#)
 - on CHOFLD tag [222](#)
 - on CHOICE tag [229](#)
 - on CMDAREA tag [240](#)
 - on DTACOL tag [272](#)
 - on DTAFLD tag [278](#)
 - on LSTCOL tag [335](#)
 - on SELFLD tag [425](#)
 - using [94](#)
- AUTOTCMD attribute
 - on PANEL tag [383](#)
- AUTOTYPE attribute
 - description
 - data field [79](#)
 - on DTAFLD tag [281](#)
 - using [79](#)
- AUTOVOL attribute
 - description
 - data field [79](#)
 - on DTAFLD tag [281](#)
 - using [79](#)

B

- BARRIER attribute on ACTION tag [187](#)
- BIT test
 - description [67](#)
 - example [67](#)
 - specifying on CHECKI tag [214](#)
- BMARGIN attribute
 - on PANDEF TAG [375](#)
 - on PANEL TAG [382](#)
- BOTINST (bottom instruction) tag
 - conditions of usage [204](#)
 - defining instruction text [37](#)
 - description [204](#)
 - examples [38](#), [205](#)
 - syntax [204](#)
- bottom instruction
 - defining [37](#)
 - description [6](#)
- bottom instruction (BOTINST) tag

- bottom instruction (BOTINST) tag (*continued*)
 - conditions of usage [204](#)
 - description [204](#)
 - examples [205](#)
 - syntax [204](#)
- BREAK attribute
 - on DL tag [262](#)
 - on PARML tag [386](#)
- BREAK=ALL attribute in definition list [116](#)
- BREAK=FIT attribute in definition list [117](#)

C

- CAPS attribute
 - description
 - data field [79](#)
 - input-only [94](#)
 - input/output [94](#)
 - on ATTR tag [202](#)
 - on CMDAREA tag [241](#)
 - on DTACOL tag [272](#)
 - on DTAFLD tag [281](#)
 - on LSTCOL tag [337](#)
 - using [79](#)
- CASE attribute on KEYI tag [320](#)
- CAUTION (caution) tag
 - conditions of usage [206](#)
 - description [125](#), [206](#)
 - example [125](#)
 - examples [207](#)
 - syntax [205](#)
- CCSID attribute
 - on HELP tag [306](#), [357](#)
 - on HELPDEF tag [312](#)
 - on PANDEF tag [374](#)
 - on PANEL tag [380](#)
- CDATA keyword
 - on ENTITY statement [172](#)
- character variables [54](#)
- CHARS test
 - specifying on CHECKI tag [212](#)
- CHDIV (choice divider) tag
 - conditions of usage [208](#)
 - description [208](#)
 - examples
 - solid and blank [208](#)
 - syntax [207](#)
- check list
 - example [139](#)
 - specifying message for [139](#)
- CHECKI (validity check item) tag
 - conditions of usage [217](#)
 - description [217](#)
 - examples [217](#)
 - syntax [210](#)
- checking values within a numeric range, range test [61](#)
- CHECKL (validity check list) tag
 - conditions of usage [218](#)
 - description [218](#)
 - examples [218](#)
 - syntax [218](#)
 - using MSG attribute [139](#)
- CHECKVAR attribute
 - description [36](#)

- CHECKVAR attribute (*continued*)
 - example [37](#)
 - on CHOICE tag [228](#)
 - on PDC tag [391](#)
- CHKBOX attribute
 - description [88](#)
 - on SELFLD tag [427](#)
 - using [88](#)
- CHOFLD (choice data field) tag
 - conditions of usage [224](#)
 - description [224](#)
 - examples [224](#)
 - syntax [219](#)
- CHOFLD (choice field) tag
 - using MSG attribute [139](#)
- choice [80](#), [85](#)
 - See also* selection field
- CHOICE (selection choice) tag
 - conditions of usage [231](#)
 - defining a selection field [80](#)
 - description [230](#)
 - examples [231](#)
 - syntax [226](#)
- choice data field (CHOFLD) tag
 - conditions of usage [224](#)
 - description [224](#)
 - examples [224](#)
 - syntax [219](#)
- choice divider (CHDIV) tag
 - conditions of usage [208](#)
 - description [208](#)
 - examples [208](#)
 - syntax [207](#)
- CHOICECOLS attribute
 - description [87](#)
 - on SELFLD tag [426](#)
 - using [87](#)
- CHOICEDEPTH attribute
 - description [87](#)
 - on SELFLD tag [426](#)
 - using [87](#)
- CKBOX attribute
 - on ATTR tag [203](#)
- CLEAR attribute
 - description [94](#)
 - on LSTCOL tag [335](#)
- CMD (command definition) tag
 - assigning function key [147](#)
 - conditions of usage [234](#)
 - description [233](#)
 - examples [234](#)
 - syntax [233](#)
 - using to define a command [143](#)
- CMD attribute on KEYI tag [320](#)
- CMDACT (command action) tag
 - conditions of usage [237](#)
 - description [237](#)
 - examples [237](#)
 - syntax [234](#)
- CMDAREA (command area) tag
 - conditions of usage [242](#)
 - defining a command area [48](#)
 - description [242](#)
 - examples

- CMDAREA (command area) tag (*continued*)
 - examples (*continued*)
 - defining [48](#)
 - in an application panel markup [242](#)
 - specifying the command prompt text [49](#)
 - syntax [237](#)
- CMDLEN attribute
 - on CMDAREA tag [240](#)
- CMDLINE attribute
 - on PANEL tag [381](#)
- CMDLOC attribute
 - on CMDAREA tag [240](#)
- CMDTBL (command table) tag
 - assigning function key [147](#)
 - conditions of usage [244](#)
 - description [244](#)
 - examples [245](#)
 - syntax [243](#)
 - using to define a command table [143](#)
- coding an action bar definition [34](#)
- COLOR attribute
 - description
 - data field [79](#)
 - on ATTR tag [202](#)
 - on DTAFLD tag [281](#)
 - on HP tag [316](#)
 - on LSTCOL tag [337](#)
 - on NOTE tag [360](#)
 - on NOTEL tag [363](#)
 - on NT tag [366](#)
 - on SELFLD tag [428](#)
 - using [79](#), [88](#)
- COLSPACE attribute
 - description [94](#)
 - on LSTCOL tag [336](#)
- COLTYPE attribute
 - description [94](#)
 - on LSTCOL tag [336](#)
- column data [89](#)
- column, defining widths [89](#), [94](#)
- COLWIDTH attribute on LSTCOL tag [334](#)
- command [143](#)
 - See also* commands
- command action (CMDACT) tag
 - conditions of usage [237](#)
 - description [237](#)
 - examples [237](#)
 - syntax [234](#)
 - using to define a command action [144](#)
- command area
 - defining [48](#)
 - defining cursor placement [30](#)
 - description [6](#)
- command area (CMDAREA) tag
 - conditions of usage [242](#)
 - description [242](#)
 - examples [242](#)
 - syntax [237](#)
- command definition (CMD) tag
 - conditions of usage [234](#)
 - description [233](#)
 - examples [234](#)
 - syntax [233](#)
- command table [143](#)

- See also application command table
- command table (CMDTBL) tag
 - conditions of usage [244](#)
 - description [244](#)
 - examples [245](#)
 - syntax [243](#)
- command-prompt-text attribute
 - on CMDAREA tag [241](#)
- commands
 - application command table [143](#)
 - declaring variables for [53](#)
 - defining a command area [48](#)
 - defining application command table [143](#)
 - defining with CMDTBL tag [143](#)
 - description of application command table [8](#)
 - reading syntax diagrams [xxi](#)
 - specifying command action [144](#)
 - specifying with the RUN attribute [35](#)
 - truncating [145](#)
- comment (COMMENT) tag
 - examples [246](#)
 - syntax [245](#)
- COMMENT (comment) tag
 - examples [246](#)
 - syntax [245](#)
- comment delimiter
 - description [16](#)
 - example [16](#)
 - using [16](#)
- comments
 - examples [16](#)
 - including in generated panel [15](#)
 - including in source files [16](#)
- COMPACT attribute
 - on BOTINST tag [204](#)
 - on DL tag [262](#)
 - on GRPHDR tag [301](#)
 - on NOTEL tag [361](#)
 - on OL tag [368](#)
 - on P tag [370](#)
 - on PARML tag [387](#)
 - on PNLINST tag [396](#)
 - on SL tag [434](#)
 - on TOPINST tag [442](#)
 - on UL tag [444](#)
 - using in a note list [361](#)
 - using in a simple list [109](#)
 - using in an ordered list [113](#)
 - using in an unordered list [111](#)
- compact for simple list [109](#)
- compact lists
 - note [108](#)
 - ordered [113](#)
 - simple [109](#)
 - unordered [111](#)
- compiler options (COMPOPT) tag
 - syntax [247](#)
- COMPOPT (compiler options) tag
 - examples [249](#)
 - syntax [247](#)
- compopt (COMPOPT) tag
 - examples [249](#)
- considerations, compatibility for AREA tag [192](#)
- contact

- contact (*continued*)
 - z/OS [493](#)
- conversion utility
 - ACTBAR option [160](#)
 - converting multiple source files [156](#)
 - CUAATTR option [159](#)
 - CUASUPP option [159](#)
 - DBALIGN option [162](#)
 - DBCS option [158](#)
 - DISK option [158](#)
 - DISPLAY option [160](#)
 - DISPLAYW option [161](#)
 - DSNCHK option [161](#)
 - FORMAT option [160](#)
 - GRAPHIC option [161](#)
 - GUI option [160](#)
 - help [156](#)
 - installing [166](#)
 - invocation panel input fields [152](#)
 - invocation panel options [153](#)
 - invocation panels [151](#)
 - KANA option [158](#)
 - KEYAPPL option [158](#)
 - LISTING option [159](#)
 - LISTREPL option [160](#)
 - LOGREPL option [160](#)
 - LSTVIEW option [159](#)
 - MCOMMENT option [162](#)
 - MERGESAREA option [160](#)
 - messages [165](#)
 - MSGEXPAND option [160](#)
 - MSGSUPP option [159](#)
 - NOACTBAR option [160](#)
 - NOCUAATTR option [159](#)
 - NOCUASUPP option [159](#)
 - NODBALIGN option [162](#)
 - NODBCS option [158](#)
 - NODISPLAY option [160](#)
 - NODISPLAYW option [161](#)
 - NODSNCHK option [161](#)
 - NOFORMAT option [160](#)
 - NOGRAPHIC option [161](#)
 - NOGUI option [160](#)
 - NOKANA option [158](#)
 - NOLISTING option [159](#)
 - NOLISTREPL option [160](#)
 - NOLOGREPL option [160](#)
 - NOLSTVIEW option [159](#)
 - NOMCOMMENT option [162](#)
 - NOMERGESAREA option [160](#)
 - NOMSGEXPAND option [160](#)
 - NOMSGSUPP option [159](#)
 - NOPANEL option [159](#)
 - NOPLB option [162](#)
 - NOPREP option [159](#)
 - NOREPLACE option [158](#)
 - NOSCRIP option [159](#)
 - NOSTATS option [159](#)
 - NOV3PADC option [162](#)
 - NOVERSION option [160](#)
 - NOZVARS option [161](#)
 - overview [9](#)
 - PANEL option [159](#)
 - PLEB option [162](#)

conversion utility (*continued*)

- PREP option [159](#)
- PROFDDN option [162](#)
- PROFILE option [162](#)
- REPLACE option [158](#)
- SCREEN option [158](#)
- SCRIPT option [159](#)
- STATS option [159](#)
- supporting keys help [148](#)
- syntax [156](#)
- using [151](#)
- V3PADC option [162](#)
- VERSION option [160](#)
- ZVARS option [161](#)

converting DTL source files [151](#)

COPYR (copyright) tag

- examples [250](#)
- syntax [249](#)

copyright

- including in generated panel [15](#)

copyright (COPYR) tag

- examples [250](#)

copyright (COPYRIGHT) tag

- syntax [249](#)

CSRGRP attribute

- description
 - data field [78](#)
- on ATTR tag [203](#)
- on DTAFLD tag [280](#)
- on LSTCOL tag [336](#)
- on PS tag [399](#)
- on SELFLD tag [427](#)
- using [78, 88](#)

CSRINDEX attribute

- example [31](#)
- on PANEL tag [380](#)

CSRPOS attribute

- example [31](#)
- on PANEL tag [380](#)

CUAATTR option [159](#)

CUADYN attribute

- on ATTR tag [203](#)

CUASUPP option [159](#)

CURSOR attribute

- example [30](#)
- on PANEL tag [380](#)

cursor field

- on PANEL tag [380](#)

cursor placement

- ABC [30](#)
- characteristics [30](#)
- CHOICE [30](#)
- DTAFLD [30](#)
- in command area [30](#)
- in data field [30](#)
- in list field [30](#)
- in selection field [30](#)
- LSTCOL [30](#)
- SELFLD [30](#)

CWIDTHS attribute

- CWIDTHS attribute on SELFLD tag [426](#)
- description [87](#)
- using [87](#)

D

DA (dynamic area) tag

- conditions of usage [254](#)
- description [254](#)
- examples [255](#)
- syntax [250](#)
- using [43](#)

data column

- defining [89](#)
- example [89](#)

data column (DTACOL) tag

- conditions of usage [273](#)
- description [89, 273](#)
- examples [274](#)
- syntax [269](#)

data field

- attributes [77](#)
- defining a field prompt [71](#)
- defining alignment of data [77](#)
- defining an associated message [77, 87](#)
- defining cursor placement [30](#)
- defining data columns [89](#)
- defining help for [76](#)
- defining input/output [74](#)
- defining width [75](#)
- examples [74](#)
- providing descriptive text [75](#)
- tailoring [77, 86](#)

data field (DTAFLD) tag

- conditions of usage [282](#)
- description [282](#)
- examples [283](#)
- syntax [275](#)

data field description (DTAFLDD) tag

- conditions of usage [284](#)
- description [75, 284](#)
- example [75](#)
- examples [285](#)
- syntax [284](#)

data set names

- default used by conversion utility [167](#)

DATAMOD attribute

- on DA tag [252](#)

DATAVAR attribute

- description [53](#)
- on CHOFLD tag [221](#)
- on DTAFLD tag [277](#)
- on LSTCOL tag [334](#)
- on LSTVAR tag [348](#)

DBALIGN attribute

- description
 - data field [78](#)
- on DTAFLD tag [280](#)
- on SELFLD tag [428](#)
- using [78, 88](#)

DBALIGN option [162](#)

DBCS

- option [158](#)
- restrictions for leading and trailing blanks [13](#)

DBCS test

- specifying on CHECKI tag [214](#)
- test
 - description [64](#)

- DBCS test (*continued*)
 - test (*continued*)
 - example [64](#)
- DD (definition description) tag
 - conditions of usage [256](#)
 - description [114](#), [255](#)
 - examples
 - basic [114](#)
 - BREAK=ALL attribute [116](#)
 - BREAK=FIT attribute [117](#)
 - help panel markup [256](#)
 - syntax [255](#)
- DDHD (definition description header) tag
 - conditions of usage [257](#)
 - description [114](#), [257](#)
 - examples [117](#), [258](#)
 - syntax [257](#)
- declaring variables [53](#)
- default keylist, key mappings [147](#)
- defining help for key list [148](#)
- definition description (DD) tag
 - conditions of usage [256](#)
 - description [255](#)
 - examples
 - basic [114](#)
 - BREAK=ALL attribute [116](#)
 - BREAK=FIT attribute [117](#)
 - help panel markup [256](#)
 - syntax [255](#)
- definition description header (DDHD) tag
 - conditions of usage [257](#)
 - description [257](#)
 - examples [258](#)
 - syntax [257](#)
- definition list (DL) tag
 - conditions of usage [263](#)
 - description [263](#)
 - examples
 - basic [114](#)
 - BREAK=ALL attribute [116](#)
 - BREAK=FIT attribute [117](#)
 - default BREAK value of NONE [263](#)
 - syntax [261](#)
 - using to define a definition list [114](#)
- definition term (DT) tag
 - conditions of usage [268](#)
 - description [268](#)
 - examples
 - basic [114](#)
 - BREAK=ALL attribute [116](#)
 - BREAK=FIT attribute [117](#)
 - help panel markup [268](#)
 - syntax [267](#)
- definition term divider (DTDIV) tag
 - conditions [286](#)
 - description [286](#)
 - syntax [286](#)
- definition term header (DTHD) tag
 - conditions of usage [287](#)
 - description [287](#)
 - examples [288](#)
 - syntax [287](#)
- definition term header divider (DTHDIV) tag
 - conditions [289](#)
- definition term header divider (DTHDIV) tag (*continued*)
 - description [289](#)
 - syntax [288](#)
- definition term segment (DTSEG) tag
 - conditions [290](#)
 - syntax [290](#)
- delimiter symbol [11](#)
- depth and width, defining with PANDEF tag [50](#)
- DEPTH attribute
 - defining application panel depth [29](#)
 - description
 - data field [78](#)
 - on AREA tag [190](#)
 - on DA tag [252](#)
 - on DTAFLD tag [280](#)
 - on GA tag [296](#)
 - on HELP tag [306](#)
 - on HELPDEF tag [312](#)
 - on PANDEF tag [374](#)
 - on PANEL tag [379](#)
 - on PS tag [399](#)
 - on REGION tag [406](#)
 - on SELFLD tag [425](#)
 - using [78](#), [87](#)
- DESSKIP attribute
 - description
 - data field [79](#)
 - on DTAFLD tag [281](#)
 - using [79](#)
- DESTVAR attribute, on ASSIGNL tag [197](#)
- DESWIDTH attribute
 - description [89](#)
 - on DTACOL tag [271](#)
 - on DTAFLD tag [278](#)
- dialog element
 - creating [11](#)
 - help panel [7](#)
- dialog elements
 - application command table [8](#)
 - description [5](#)
 - description of application panel [5](#)
 - description of help panel [6](#)
 - key mapping list [8](#)
 - messages [7](#)
 - types [5](#)
 - variable classes [8](#)
 - variables [8](#)
- Dialog Tag Language (DTL)
 - advantages [3](#)
 - coding attributes and values [11](#)
 - coding tag text [12](#)
 - comments [15](#), [16](#)
 - copyright statements [15](#)
 - delimiters [11](#)
 - document type declaration [15](#)
 - entities [17](#)
 - introduction [3](#)
 - nesting tags [14](#)
 - parameter entities [21](#)
 - predefined symbols [23](#)
 - relationship to CUA [3](#)
 - results of converting [166](#)
 - return codes [166](#)
 - similarity to HTML [4](#)

- Dialog Tag Language (DTL) (*continued*)
 - source file [11](#)
 - syntax conventions [11](#)
 - using [11](#)
- dialog variable [53](#)
 - See also* variables
- DIR attribute
 - on AREA tag [191](#)
 - on REGION tag [406](#)
- DISK option [158](#)
- DISP attribute
 - on MSG tag [354](#)
- DISPLAY attribute
 - description [77](#)
 - input-only [94](#)
 - input/output [94](#)
 - on CHOFLD tag [222](#)
 - on DTAFLD tag [279](#)
 - on LSTCOL tag [337](#)
 - using [77](#)
- DISPLAY option [160](#)
- DISPLAYW option [161](#)
- DISPLEN attribute
 - on SCRFLD tag [414](#)
- DIV attribute
 - on AREA tag [191](#)
 - on DA tag [253](#)
 - on GA tag [297](#)
 - on GRPHDR tag [301](#)
 - on LSTFLD tag [342](#)
- DIVEND attribute
 - on DL tag [262](#)
 - on PARML tag [387](#)
- DIVIDER (area divider) tag
 - conditions of usage [260](#)
 - defining a divider [44](#)
 - description [260](#)
 - examples
 - solid and blank [47](#), [260](#)
 - TYPE attribute [44](#)
 - within a horizontal REGION tag [47](#)
 - within vertical REGION tag [45](#)
 - syntax [258](#)
 - within a REGION tag [47](#)
- DIVLOC attribute
 - on GRPHDR tag [301](#)
- DIVWIDTH attribute
 - on AREA tag [191](#)
- DL (definition list) tag
 - conditions of usage [263](#)
 - description [114](#), [263](#)
 - examples
 - basic [114](#)
 - BREAK=ALL attribute [116](#)
 - BREAK=FIT attribute [117](#)
 - default BREAK value of NONE [263](#)
 - syntax [261](#)
 - using to define a definition list [114](#)
- DLDIV (area divider) tag
 - conditions of usage [266](#)
 - description [266](#)
 - examples
 - solid and blank [266](#)
- DLDIV tag

- DLDIV tag (*continued*)
 - syntax [265](#)
- DM application, providing help for [130](#)
- DOCTYPE statement
 - declaring document type [15](#)
 - description [171](#)
 - document type declaration [15](#)
 - parameters [171](#)
 - syntax [171](#)
- document type declaration
 - description [171](#)
 - types supported by DTL [15](#)
- double-byte characters, permitting usage through variables [54](#)
- DSNAME test
 - description [63](#)
 - example [63](#)
 - specifying on CHECKI tag [215](#)
- DSNAMEF test
 - description [63](#)
 - example [63](#)
 - specifying on CHECKI tag [215](#)
- DSNAMEFM test
 - description [63](#)
 - example [63](#)
 - specifying on CHECKI tag [215](#)
- DSNAMEPQ test
 - description [63](#)
 - example [63](#)
 - specifying on CHECKI tag [216](#)
- DSNAMEQ test
 - description [64](#)
 - example [64](#)
 - specifying on CHECKI tag [216](#)
- DSNCHK option [161](#)
- DT (definition term) tag
 - conditions of usage [268](#)
 - description [114](#), [268](#)
 - examples
 - basic [114](#)
 - BREAK=ALL attribute [116](#)
 - BREAK=FIT attribute [117](#)
 - help panel markup [268](#)
 - syntax [267](#)
- DTACOL (data column) tag
 - conditions of usage [273](#)
 - description [273](#)
 - examples [274](#)
 - syntax [269](#)
- DTAFLD (data field) tag
 - conditions of usage [282](#)
 - description [282](#)
 - examples [283](#)
 - syntax [275](#)
 - using [74](#)
 - using MSG attribute [139](#)
 - See also* data field
- DTAFLDD (data field description) tag
 - conditions of usage [284](#)
 - description [284](#)
 - examples [285](#)
 - syntax [284](#)
 - See also* data field
- DTDIV (definition term divider) tag

- DTDIV (definition term divider) tag (*continued*)
 - conditions [286](#)
 - description [286](#)
 - syntax [286](#)
- DTHD (definition term header) tag
 - conditions of usage [287](#)
 - description [114](#), [287](#)
 - examples [117](#), [288](#)
 - syntax [287](#)
- DTHDIV (definition term header divider) tag
 - conditions [289](#)
 - description [289](#)
 - syntax [288](#)
- DTL
 - macros [177](#)
 - results of converting [166](#)
 - return codes [166](#)
 - source files, converting [151](#)
- DTSEG (definition term segment) tag
 - conditions [290](#)
 - syntax [290](#)
- dynamic area (DA) tag
 - conditions of usage [254](#)
 - description [254](#)
 - examples [255](#)
 - syntax [250](#)
 - using [43](#)

E

- EBCDIC test
 - description [65](#)
 - example [65](#)
 - specifying on CHECKI tag [215](#)
- embedding source files [22](#)
- emphasizing panel text
 - description [126](#)
 - HP (Highlighted phrase) [126](#)
 - restriction [126](#)
 - RP (reference phrase) [126](#)
- end tag delimiters [11](#)
- ENDATTR attribute
 - on PANEL tag [381](#)
- entity
 - defining [17](#)
 - description [17](#)
 - examples
 - changing text [18](#)
 - declaring a different name [20](#)
 - declaring a file [19](#)
 - declaring name and text string [17](#)
 - naming conventions [18](#)
 - specifying a name in text [18](#)
 - using a text string in source file [19](#)
 - using to embed external files [22](#)
 - naming conventions [18](#)
 - parameter [21](#)
 - predefined [23](#)
 - using to embed external files [22](#)
- entity declarations
 - conditions of usage [173](#)
 - description [172](#), [173](#)
 - example [174](#)
- entity definitions

- entity definitions (*continued*)
 - example [175](#)
- ENTITY keyword
 - on ENTITY statement [172](#)
- ENTITY statement
 - parameters [172](#)
 - syntax [172](#)
- entity-name
 - on ENTITY statement [172](#)
- entity-text
 - on ENTITY statement [173](#)
- ENTKEYTEXT attribute
 - on PANDEF TAG [375](#)
 - on PANEL TAG [382](#)
- ENTWIDTH attribute
 - description [86](#), [89](#)
 - on CHOFLD tag [222](#)
 - on CMDAREA tag [240](#)
 - on DTACOL tag [271](#)
 - on DTAFLD tag [278](#)
 - on SELFLD tag [424](#)
 - using [86](#)
- ENUM [67](#)
- ENUM test
 - specifying on CHECKI tag [215](#)
- error messages [165](#)
- ERRORCHECK attribute
 - on PANEL tag [382](#)
- example (XMP) tag
 - conditions of usage [461](#)
 - description [461](#)
 - examples [105](#), [461](#)
 - syntax [460](#)
 - using to define an example [105](#)
- EXPAND attribute
 - description
 - data field [78](#)
 - on CHOFLD tag [223](#)
 - on DTAFLD tag [280](#)
 - on HELP TAG [306](#)
 - on HELPDEF TAG [312](#)
 - on PANDEF TAG [375](#)
 - on PANEL tag [381](#)
 - using [78](#)
- EXTEND attribute
 - description [87](#)
 - on AREA tag [191](#)
 - on DA tag [252](#)
 - on GA tag [296](#)
 - on REGION tag [406](#)
 - on SELFLD tag [425](#)
 - using [87](#)

F

- FCHOICE attribute
 - description [87](#)
 - on SELFLD tag [425](#)
 - using [87](#)
- field
 - defining [71](#)
 - defining a data field message [77](#), [87](#)
 - defining a list field message [95](#)
 - defining data field [74](#)

- field (*continued*)
 - defining data field columns [89](#)
 - defining list field [91](#)
 - defining multiple-choice selection fields [81](#)
 - defining selection fields [79](#)
 - defining single-choice fields [80](#)
 - interactive [71](#)
 - providing additional information [129](#)
 - types [71](#)
- field prompt
 - attributes [71](#)
 - defining [71](#)
 - example [71](#)
 - specifying width [73](#)
- field-level help
 - on LSTCOL tag [334](#)
 - on SELFLD tag [423](#)
- FIG (figure) tag
 - conditions of usage [292](#)
 - description [292](#)
 - examples [106](#), [293](#)
 - syntax [291](#)
 - using to define a figure [106](#)
- FIGCAP (figure caption) tag
 - conditions of usage [294](#)
 - description [294](#)
 - examples [107](#), [295](#)
 - syntax [294](#)
- figure (FIG) tag
 - conditions of usage [292](#)
 - description [292](#)
 - examples [106](#), [293](#)
 - syntax [291](#)
 - using to define a figure [106](#)
- figure caption (FIGCAP) tag
 - conditions of usage [294](#)
 - description [294](#)
 - examples [107](#), [295](#)
 - syntax [294](#)
- FileID test [63](#)
- FILEID test
 - specifying on CHECKI tag [216](#)
- files for installing the product [166](#)
- filespec
 - on ENTITY statement [173](#)
- FKA attribute on KEYI tag [320](#)
- FLDSPACE attribute
 - description [77](#), [89](#)
 - on CHOFLD tag [222](#)
 - on DTACOL tag [271](#)
 - on DTAFLD tag [278](#)
 - using [77](#)
- FLDSPOS attribute
 - on SCRFLD tag [416](#)
- FLDTYPE attribute
 - description
 - data field [79](#)
 - on DTAFLD tag [281](#)
 - on SELFLD tag [428](#)
 - using [79](#), [88](#)
- FLDWIDTH attribute
 - description
 - data field [78](#)
 - on DTAFLD tag [280](#)

- FLDWIDTH attribute (*continued*)
 - using [78](#)
- FMTWIDTH attribute
 - on GRPHDR tag [301](#)
- FORMAT attribute
 - description [95](#)
 - on AREA tag [191](#)
 - on ATTR tag [202](#)
 - on CHDIV tag [208](#)
 - on DA tag [253](#)
 - on DIVIDER tag [259](#), [266](#)
 - on DL tag [262](#)
 - on GRPHDR tag [301](#)
 - on LSTCOL tag [335](#)
 - on MSG tag [354](#)
 - on PARML tag [387](#)
 - on PLDIV tag [394](#)
 - on XLATL tag [459](#)
- FORMAT option [160](#)
- formatting panel text
 - Asian rules [13](#)
 - English rules [13](#)
- fragments, syntax diagrams [xxi](#)
- FRAME attribute on FIG tag [292](#)
- function key area
 - defining [147](#)
 - description [6](#)

G

- GA (graphic area) tag
 - conditions of usage [297](#)
 - description [297](#)
 - examples [297](#)
 - syntax [295](#)
 - using [43](#)
- GAP attribute
 - on DIVIDER tag [259](#)
 - on DLDIV tag [265](#)
 - on PLDIV tag [394](#)
- GE attribute
 - on ATTR tag [202](#)
- generate (GENERATE) tag
 - conditions of usage [298](#)
 - description [298](#)
 - examples [299](#)
 - syntax [298](#)
- GENERATE (generate) tag
 - conditions of usage [298](#)
 - description [298](#)
 - syntax [298](#)
- GENERATE (generate)tag
 - examples [299](#)
- generated panel comments
 - comments [15](#)
- generated panel statements
 - copyright [15](#)
- graphic area (GA) tag
 - conditions of usage [297](#)
 - description [297](#)
 - examples [297](#)
 - syntax [295](#)
 - using [43](#)
- GRAPHIC option [161](#)

- group header (GRPHDR) tag
 - conditions of usage [302](#)
 - description [301](#)
 - examples [302](#)
 - syntax [300](#)
- GRPBOX attribute on REGION tag [406](#)
- GRPBXMAT attribute on REGION tag [407](#)
- GRPBXVAR attribute on REGION tag [407](#)
- GRPHDR (group header) tag
 - additional attributes for [96](#)
 - conditions of usage [302](#)
 - description [301](#)
 - description of attributes [96](#)
 - examples [302](#)
 - syntax [300](#)
- GRPWIDTH attribute on REGION tag [406](#)
- GUI option [160](#)
- GUTTER attribute
 - on CHDIV tag [208](#)
 - on DIVIDER tag [259](#)
 - on DLDIV tag [266](#)
 - on PLDIV tag [394](#)

H

- heading
 - in the information region [104](#)
 - levels [104](#)
- heading (Hn) tag
 - conditions of usage [314](#)
 - description [313](#)
 - examples [104](#), [314](#)
 - syntax [313](#)
 - using to define a heading [104](#)
- HEADLINE attribute
 - on GRPHDR tag [301](#)
 - on LSTGRP tag [345](#)
- help
 - defining a help pull-down [36](#)
 - defining for a data field [76](#)
 - defining help panels [130](#)
 - field-level [423](#)
 - for action bar [35](#)
 - for application panels [29](#)
 - for selection choice [85](#)
 - for selection field [85](#)
 - for the conversion utility [156](#)
- HELP (help panel) tag
 - conditions of usage [309](#)
 - description [307](#)
 - examples [309](#)
 - syntax [303](#)
- HELP attribute
 - on ABC tag [182](#)
 - on CHOFLD tag [221](#)
 - on CHOICE tag [228](#)
 - on CMDAREA tag [239](#)
 - on DA tag [254](#)
 - on DTAFLD tag [277](#)
 - on HELP tag [305](#)
 - on HELPDEF tag [311](#)
 - on KEYL tag [323](#)
 - on LSTCOL tag [334](#)
 - on MSG tag [353](#)

- HELP attribute (*continued*)
 - on PANDEF tag [374](#)
 - on PANEL tag [378](#)
 - on PDC tag [391](#)
 - on RP tag [412](#)
 - on SELFLD tag [423](#)
- help default (HELPDEF) tag
 - conditions of usage [313](#)
 - description [312](#)
 - examples [313](#)
 - syntax [311](#)
- help panel
 - defining for a data field [76](#)
 - defining help panel text [131](#)
 - function key area [7](#)
 - layout [6](#)
 - overview [6](#)
 - panel body [7](#)
 - panel title [7](#)
 - title [7](#)
 - types of help [6](#)
 - using HELP attribute of PANEL tag [29](#)
- help panel (HELP) tag
 - conditions of usage [309](#)
 - description [307](#)
 - examples [309](#)
 - syntax [303](#)
- help panel tag, using [130](#)
- help panels
 - defining [130](#)
 - defining areas and regions [130](#)
 - in sequence [134](#)
 - scrollable [132](#)
 - using the INFO tag with [130](#)
- help pull-down, defining [36](#)
- HELP tag [130](#)
- HELPDEF (help default) tag
 - conditions of usage [313](#)
 - description [312](#)
 - examples [313](#)
 - syntax [311](#)
- HELPDEF attribute on HELP tag [305](#)
- HEX test
 - description [68](#)
 - example [68](#)
 - specifying on CHECKI tag [216](#)
- HIDE attribute on CHOICE tag [229](#)
- HIDEX attribute on CHOICE tag [230](#)
- highlighted phrase (HP) tag
 - conditions of usage [316](#)
 - description [316](#)
 - examples [316](#)
 - syntax [315](#)
- HILITE attribute
 - description
 - data field [79](#)
 - on ATTR tag [202](#)
 - on DTAFLD tag [281](#)
 - on HP tag [316](#)
 - on LSTCOL tag [337](#)
 - on NOTE tag [360](#)
 - on NOTEL tag [363](#)
 - on NT tag [366](#)
 - on SELFLD tag [428](#)

HILITE attribute (*continued*)

using [79](#), [88](#)

Hn (heading) tag

conditions of usage [314](#)

description [313](#)

examples [104](#), [314](#)

syntax [313](#)

using to define a heading [104](#)

horizontal region [46](#)

HP (Highlighted phrase) tag

conditions of usage [316](#)

description [126](#), [316](#)

examples [126](#), [316](#)

restriction [126](#)

syntax [315](#)

I

ID attribute

on HELPDEF tag [311](#)

on PANDEF tag [374](#)

ID panel [28](#)

IDATE attribute

value IDATE on CHECKI tag [216](#)

IDATE test

description [68](#)

example [68](#)

IMAPCOL attribute

on HELP tag [307](#)

on HELPDEF tag [312](#)

on PANDEF tag [375](#)

on PANEL tag [382](#)

IMAPNAME attribute

description

data field [78](#)

on CHOFLD tag [223](#)

on CMDAREA tag [241](#)

on DTAFLD tag [280](#)

on HELP tag [307](#)

on HELPDEF tag [312](#)

on PANDEF tag [375](#)

on PANEL tag [382](#)

on PS tag [399](#)

using [78](#)

IMAPNAMEP attribute

description

data field [78](#)

on CMDAREA tag [241](#)

on DTAFLD tag [280](#)

on PS tag [399](#)

using [78](#)

IMAPROW attribute

on HELP tag [307](#)

on HELPDEF tag [312](#)

on PANDEF tag [375](#)

on PANEL tag [382](#)

immediate-action, for pull-down choice [35](#)

INCLUDE test

description [68](#)

example [68](#)

specifying on CHECKI tag [216](#)

INDENT attribute

on AREA tag [190](#)

on DL tag [262](#)

INDENT attribute (*continued*)

on GRPHDR tag [301](#)

on OL tag [368](#)

on P tag [370](#)

on PARML tag [387](#)

on REGION tag [406](#)

on SL tag [434](#)

on UL tag [444](#)

index value

on PANEL tag [380](#)

INDVAL attribute

on SCRFLD tag [415](#)

INDVAR attribute

on SCRFLD tag [415](#)

INFO (information region) tag

conditions of usage [318](#)

description [318](#)

examples [101](#), [319](#)

syntax [317](#)

using to define an information region [101](#)

information message

defining [138](#)

description [138](#)

providing for fields [129](#)

information region

defining [101](#)

defining text [102](#)

definition lists [114](#)

examples (XMP tag) [105](#)

figure captions [107](#)

figures [106](#)

headings [104](#)

list part [120](#)

ordered lists [112](#)

paragraphs [102](#)

parameter lists [118](#)

providing information for fields [129](#)

simple lists [109](#)

tags for text [101](#)

unformatted text [104](#)

unordered lists [110](#)

information region (INFO) tag

conditions of usage [318](#)

description [318](#)

examples [101](#), [319](#)

syntax [317](#)

using to define an information region [101](#)

INIT attribute

description

data field [78](#)

on CHOFLD tag [223](#)

on DTAFLD tag [280](#)

on SELFLD tag [427](#)

using [78](#), [87](#)

initialization syntax, source-filespec [158](#)

input/output data field [74](#)

installing the conversion utility [166](#)

instruction text [38](#)

instructions, top and bottom [37](#)

INTENS attribute

description

data field [79](#)

on ATTR tag [202](#)

on DTAFLD tag [281](#)

INTENS attribute (*continued*)

- on HP tag [316](#)
- on LSTCOL tag [337](#)
- on NOTE tag [360](#)
- on NOTEL tag [363](#)
- on NT tag [366](#)
- on SELFLD tag [428](#)
- using [79](#), [88](#)

INTENSE attribute

- on HP tag [316](#)
- on P tag [370](#)

IPADDR4

- description [69](#)
- example [69](#)

IPADDR4 attribute

- value IPADDR4 on CHECKI tag [217](#)

IPADDR4 test

- specifying on CHECKI tag [217](#)

ISP application identifier [143](#)

ISPCMDTB [143](#), [244](#)

ISPDTLC

- overview [9](#)
- using ? [156](#)
- using command [151](#)

ISPKYLST

- description [148](#)
- key mappings list [148](#)
- using [148](#)

ISPx application identifier [143](#)

item translate list

- description [56](#)
- example [57](#)

ITIME attribute

- value ITIME on CHECKI tag [217](#)

ITIME test

- description [69](#)
- example [69](#)

J

JDATE attribute

- value JDATE on CHECKI tag [216](#)

JDATE test

- description [69](#)
- example [69](#)

JSTD attribute

- value JSTD on CHECKI tag [216](#)

JSTD test

- description [69](#)
- example [69](#)

JUST attribute

- on ATTR tag [202](#)

K

KANA option [158](#)

KEY attribute on KEYI tag [320](#)

key item (KEYI) tag

- conditions of usage [321](#)
- description [321](#)
- examples [321](#)
- syntax [319](#)
- using to define a key item [147](#)

key list, defining help [148](#)

key mapping list

- defining [147](#)
- defining with PANDEF tag [50](#)
- overview [8](#)
- using KEYLIST attribute of PANEL tag [29](#)

key mapping list (KEYL) tag

- conditions of usage [323](#)
- description [323](#)
- examples [324](#)
- syntax [322](#)

KEYAPPL option [158](#)

keyboard

- navigation [493](#)
- PF keys [493](#)
- shortcut keys [493](#)

KEYI (key item) tag

- conditions of usage [321](#)
- description [147](#), [321](#)
- examples [321](#)
- syntax [319](#)
- using to define a key item [147](#)

KEYL (key mapping list) tag

- conditions of usage [323](#)
- description [147](#), [323](#)
- examples [324](#)
- syntax [322](#)
- using [147](#)

KEYLIST attribute

- examples [29](#)
- on HELP TAG [306](#)
- on HELPDEF TAG [312](#)
- on PANDEF tag [374](#)
- on PANEL tag [379](#)

KEYLTYPE attribute

- on HELP TAG [306](#)
- on HELPDEF tag [312](#)
- on PANDEF tag [374](#)
- on PANEL tag [379](#)

keys

- assigning actions [147](#)
- default keylist [147](#)
- defining [148](#)
- defining key mapping lists [147](#)
- displaying [148](#)
- See also* function key area

keywords, syntax diagrams [xxi](#)

L

LANG attribute on ACTION tag [187](#)

LCOLDISP attribute

- on SCRFLD tag [416](#)

LCOLIND attribute

- on SCRFLD tag [416](#)

LEN test

- description [67](#)
- example [67](#)
- specifying on CHECKI tag [215](#)

LI (list item) tag

- conditions of usage [326](#)
- description [325](#)
- examples
- basic unordered list [110](#)

- LI (list item) tag *(continued)*
 - examples *(continued)*
 - help panel with unordered list [326](#)
 - list part (LP) tag [120](#)
 - nested unordered [111](#)
 - nesting ordered list [112](#)
 - note list [108](#)
 - paragraph nested in a list [119](#)
 - simple list [109](#)
 - syntax [325](#)
- LINDVAL attribute
 - on SCRFLD tag [415](#)
- LINDVAR attribute
 - on SCRFLD tag [415](#)
- LINE attribute
 - description [95](#)
 - on LSTCOL tag [335](#)
 - on LSTVAR tag [348](#)
- LINES (lines) tag
 - conditions of usage [328](#)
 - description [327](#)
 - examples [105](#), [328](#)
 - syntax [327](#)
 - using to define unformatted text [104](#)
- list column
 - defining width [94](#)
 - truncating [94](#)
- list column (LSTCOL) tag
 - conditions of usage [338](#)
 - description [337](#)
 - examples [338](#)
 - syntax [332](#)
- list field
 - additional attributes for [94](#)
 - auto-tab attribute [94](#)
 - defining [91](#)
 - defining alignment of data [94](#)
 - defining an associated message [95](#)
 - defining cursor placement [30](#)
 - defining required input for [95](#)
 - description [91](#)
 - example [91](#)
 - specifying help for [95](#)
 - tailoring [94](#)
- list field (LSTFLD) tag
 - conditions of usage [343](#)
 - description [343](#)
 - examples [344](#)
 - syntax [341](#)
- list group (LSTGRP) tag
 - conditions of usage [346](#)
 - description [346](#)
 - example [347](#)
 - syntax [345](#)
- list item (LI) tag
 - conditions of usage [326](#)
 - description [325](#)
 - examples
 - basic [109](#)
 - COMPACT attribute on simple list [109](#)
 - compact ordered list [113](#)
 - help panel with unordered list [326](#)
 - syntax [325](#)
- list part (LP) tag

- list part (LP) tag *(continued)*
 - conditions of usage [331](#)
 - description [331](#)
 - examples [120](#), [332](#)
 - syntax [330](#)
 - using to define a list part [120](#)
- list variable (LSTVAR) tag
 - conditions of usage [349](#)
 - description [348](#)
 - example [349](#)
 - syntax [348](#)
- LISTDEPTH attribute
 - description [88](#)
 - on SELFLD tag [428](#)
 - using [88](#)
- LISTING option [159](#)
- LISTREF attribute
 - description [88](#)
 - on SELFLD tag [427](#)
 - using [88](#)
- LISTREPL option [160](#)
- lists
 - definition [114](#)
 - list part [120](#)
 - nesting lists within lists [120](#)
 - note [108](#)
 - ordered [112](#)
 - parameter [118](#)
 - simple [109](#)
 - types [108](#)
 - unordered [110](#)
- LISTTYPE attribute
 - description [88](#)
 - on SELFLD tag [427](#)
 - using [88](#)
- LISTV test
 - description [66](#)
 - example [66](#)
 - specifying on CHECKI tag [215](#)
- LISTVX test
 - description [66](#)
 - example [66](#)
 - specifying on CHECKI tag [215](#)
- LIT (literal) tag
 - conditions of usage [329](#)
 - description [329](#)
 - examples [330](#)
 - syntax [329](#)
- literal (LIT) tag
 - conditions of usage [329](#)
 - description [329](#)
 - examples [330](#)
 - syntax [329](#)
- LMSG attribute
 - on PANEL tag [382](#)
- LOCATION attribute
 - on MSG tag [354](#)
 - on REGION tag [407](#)
- LOGREPL option [160](#)
- LP (list part) tag
 - conditions of usage [331](#)
 - description [331](#)
 - examples [120](#), [332](#)
 - syntax [330](#)

- LP (list part) tag *(continued)*
 - using to define a list part [120](#)
- LSTCOL (list column) tag
 - additional attributes for [94](#)
 - conditions of usage [338](#)
 - description [91](#), [337](#)
 - description of attributes [94](#)
 - examples [338](#)
 - syntax [332](#)
 - using MSG attribute [139](#)
- LSTFLD (list field) tag
 - conditions of usage [343](#)
 - description [91](#), [343](#)
 - examples [344](#)
 - syntax [341](#)
- LSTGRP (list group) tag
 - conditions of usage [346](#)
 - description [91](#), [346](#)
 - example [347](#)
 - syntax [345](#)
- LSTVAR (list variable) tag
 - conditions of usage [349](#)
 - description [91](#), [348](#)
 - example [349](#)
 - syntax [348](#)
- LSTVIEW option [159](#)
- LVLIN attribute
 - on DA tag [252](#)
 - on GA tag [297](#)

M

- M (mnemonic) tag
 - conditions of usage [351](#)
 - description [351](#)
 - example [352](#)
 - parameters [351](#)
 - syntax [351](#)
- macros
 - DTL [177](#)
- MARGIN attribute on AREA tag [190](#)
- MARGINW attribute
 - example [39](#)
 - on AREA tag [190](#)
- markup declarations
 - comments [16](#)
 - defining entities and parameter entities [17](#)
 - document type declaration [15](#)
 - entity declarations [17](#)
 - types supported by DTL [15](#)
- markup language
 - advantages [4](#)
 - description [3](#)
- markup, coding [11](#)
- MATCH attribute
 - description [36](#)
 - example [37](#)
 - on CHOICE tag [229](#)
 - on PDC tag [392](#)
- MCOMMENT option [162](#)
- MENU attribute
 - on PANEL tag [380](#)
- MERGESAREA attribute
 - on HELP TAG [306](#)

- MERGESAREA attribute *(continued)*
 - on HELPDEF TAG [312](#)
 - on PANDEF TAG [375](#)
 - on PANEL tag [382](#)
- MERGESAREA option [160](#)
- message
 - declaring variables for [53](#)
 - defining [137](#)
 - defining for a data field [77](#), [87](#)
 - defining for a list field [95](#)
 - example [8](#)
 - for check list [139](#)
 - member [137](#)
 - specifying a variable in text [141](#)
 - specifying type [138](#)
 - types [137](#)
- message (MSG) tag
 - conditions of usage [355](#)
 - description [355](#)
 - examples
 - MSG SUFFIX attribute [137](#)
 - syntax [352](#)
 - using to define a message [137](#)
- message identifier [138](#)
- message member [137](#)
- message member (MSGMBR) tag
 - conditions of usage [357](#)
 - description [357](#)
 - examples
 - basic [137](#)
 - defining a message member [357](#)
 - specifying type [138](#)
 - syntax [356](#)
 - using to define a message member [137](#)
- messages
 - assigning for check list [139](#)
 - assigning for data field [139](#)
 - assigning for failing specified translation [139](#)
 - assigning for failing validity check [139](#)
 - assigning for list column [139](#)
 - conversion utility [165](#)
 - defining [137](#)
 - description [137](#)
 - error [165](#)
 - overview [7](#)
 - warning [165](#)
- MIX test
 - description [65](#)
 - example [65](#)
 - specifying on CHECKI tag [216](#)
- MIXC attribute on CMDACT tag [235](#)
- MNEMGEN attribute on AB tag [180](#)
- mnemonic (M) tag
 - conditions of usage [351](#)
 - description [351](#)
 - example [352](#)
 - parameters [351](#)
 - syntax [351](#)
- mnemonic choice selection
 - from pull-downs and action bars [36](#)
 - support of [36](#)
- MODE attribute on ACTION tag [187](#)
- MSG (message) tag
 - conditions of usage [355](#)

- MSG (message) tag *(continued)*
 - description [355](#)
 - examples
 - MSG SUFFIX attribute [137](#)
 - syntax [352](#)
 - using to define a message [137](#)
- MSG attribute
 - on CHECKL tag [218](#)
 - on DTAFLD tag [278](#)
 - on LSTCOL tag [334](#)
 - on SELFLD tag [425](#)
 - on VARCLASS tag [447](#)
 - on XLATL tag [459](#)
 - using [77](#), [87](#)
- MSGEXPAND option [160](#)
- MSGLINE attribute
 - on HELP tag [307](#)
 - on PANEL tag [381](#)
- MSGMBR (message member) tag
 - conditions of usage [357](#)
 - description [357](#)
 - examples
 - basic [137](#)
 - defining a message member [357](#)
 - specifying type [138](#)
 - syntax [356](#)
 - using to define a message member [137](#)
- MSGSUPP option [159](#)
- MSGTYPE attribute on MSG tag [353](#)
- multicultural support [4](#), [24](#)
- multiple-choice selection field
 - defining [81](#)
 - discussion [81](#)
 - example [81](#)
- MVS naming conventions [166](#)

N

- NAME attribute
 - identifying variables [53](#)
 - on CHOICE tag [227](#)
 - on CMD tag [233](#)
 - on CMDAREA tag [240](#)
 - on DA tag [252](#)
 - on DTAFLD tag [277](#)
 - on GA tag [296](#)
 - on HELP tag [304](#)
 - on KEYL tag [323](#)
 - on MSGMBR tag [357](#)
 - on PANEL tag [378](#)
 - on SELFLD tag [423](#)
 - on VARCLASS tag [446](#)
 - on VARDCL tag [449](#)
 - rules for variable names [179](#)
 - using with the PANEL tag [28](#)
- NAME test
 - description [64](#)
 - example [64](#)
 - specifying on CHECKI tag [214](#)
- NAMEF test
 - description [64](#)
 - example [64](#)
 - specifying on CHECKI tag [214](#)
- naming conventions for MVS [166](#)

- National Language Support [4](#)
 - See also* multicultural support
- navigation
 - keyboard [493](#)
- NEST attribute on ACTION tag [187](#)
- nesting
 - lists within lists [120](#)
 - ordered lists [112](#), [113](#)
 - simple list [109](#)
 - tags within lists [119](#)
 - unordered lists [111](#)
- NEWAPPL attribute on ACTION tag [187](#)
- NEWPOOL attribute on ACTION tag [187](#)
- NEWWINDOW attribute on ACTION tag [187](#)
- NLS [4](#)
 - See also* multicultural support
- NOACTBAR option [160](#)
- NOCHECK attribute on ACTION tag [187](#)
- NOCUAATTR option [159](#)
- NOCUASUPP option [159](#)
- NODBALIGN option [162](#)
- NODBCS option [158](#)
- NODISPLAY option [160](#)
- NODISPLAYW option [161](#)
- NODSNCHK option [161](#)
- NOENDATTR attribute
 - description [78](#), [95](#)
 - on CHOFLD tag [222](#)
 - on DIVIDER tag [259](#)
 - on DTAFLD tag [279](#)
 - on LSTCOL tag [336](#)
 - using [78](#)
- NOFORMAT option [160](#)
- NOGRAPHIC option [161](#)
- NOGUI option [160](#)
- NOINIT attribute
 - on CMDAREA tag [239](#)
- NOJUMP attribute
 - description
 - data field [79](#)
 - on CMDAREA tag [241](#)
 - on DTAFLD tag [281](#)
 - using [79](#)
- NOKANA option [158](#)
- NOLISTING option [159](#)
- NOLISTREPL option [160](#)
- NOLOGREPL option [160](#)
- NOLSTVIEW option [159](#)
- NOMATCH attribute on CHOICE tag [229](#)
- NOMCOMMENT option [162](#)
- NOMERGESAREA option [160](#)
- NOMSGEXPAND option [160](#)
- NOMSGSUPP option [159](#)
- NOPANEL option [159](#)
- NOPLEB option [162](#)
- NOPREP option [159](#)
- NOREPLACE option [158](#)
- NOSCRIP option [159](#)
- NOSEL attribute
 - description [88](#)
 - on SELFLD tag [428](#)
 - using [88](#)
- NOSKIP attribute
 - on DL tag [262](#)

NOSKIP attribute (*continued*)

- on FIG tag [292](#)
- on LINES tag [327](#)
- on NOTE tag [359](#)
- on NOTEL tag [362](#)
- on NT tag [365](#)
- on OL tag [368](#)
- on SL tag [434](#)
- on UL tag [444](#)
- on XMP tag [460](#)

NOSTATS option [159](#)

NOTE (note) tag

- conditions of usage [360](#)
- description [360](#)
- examples [360](#)
- syntax [358](#)

note (NT) tag

- conditions of usage [366](#)
- description [366](#)
- examples [367](#)
- syntax [364](#)

note list (NOTEL) tag

- conditions of usage [363](#)
- description [363](#)
- examples [364](#)
- syntax [361](#)

NOTE tag

- description [121](#)
- example [122](#)

NOTEL (note list) tag

- conditions of usage [363](#)
- description [121](#), [363](#)
- examples [122](#), [364](#)
- syntax [361](#)

NOV3PADC option [162](#)

NOVERSION option [160](#)

NOZVARS option [161](#)

NT (note) tag

- conditions of usage [366](#)
- description [121](#), [366](#)
- example [123](#)
- examples [367](#)
- syntax [364](#)

NUM test

- description [67](#)
- example [67](#)
- specifying on CHECKI tag [214](#)

NUMERIC attribute

- on ATTR tag [202](#)

numeric variables

- converting [56](#)
- description [56](#)
- uses [56](#)

O

OFFSET attribute

- on P tag [370](#)

OL (ordered list) tag

- conditions of usage [369](#)
- description [368](#)
- examples
 - COMPACT attribute [113](#)
 - list part (LP) tag [120](#)

OL (ordered list) tag (*continued*)

examples (*continued*)

- nested with paragraph [369](#)
- nesting [112](#)
- paragraph nested in a list [119](#)
- syntax [367](#)
- using to define an ordered list [112](#)

OPT attribute on ACTION tag [187](#)

ordered list (OL) tag

- conditions of usage [369](#)
- description [368](#)
- examples
 - COMPACT attribute [113](#)
 - list part (LP) tag [120](#)
 - nested with paragraph [369](#)
 - nesting [112](#)
 - paragraph nested in a list [119](#)
- syntax [367](#)
- using to define an ordered list [112](#)

OUTLINE attribute

- description
 - data field [78](#)
 - DTACOL tag [89](#)
 - LSTCOL tag [95](#)
 - selection field [87](#)
- on ATTR tag [202](#)
- on CHOFLD tag [223](#)
- on CHOICE tag [229](#)
- on CMDAREA tag [239](#)
- on DTACOL tag [272](#)
- on DTAFLD tag [279](#)
- on LSTCOL tag [336](#)
- on PANDEF tag [375](#)
- on PANEL tag [381](#)
- on SELFLD tag [426](#)
- using [78](#), [87](#)

output data field [74](#)

overriding variable classes [70](#)

P

P (paragraph) tag

- conditions of usage [371](#)
- description [371](#)
- examples
 - basic [102](#)
 - defining information region width [371](#)
 - formatting of [103](#)
 - nested in an ordered list [119](#)
- syntax [370](#)
- using to define a paragraph [102](#)

PAD attribute

- description
 - data field [78](#)
 - DTACOL tag [89](#)
 - LSTCOL tag [95](#)
 - selection field [87](#)
- on ATTR tag [202](#)
- on CHOFLD tag [222](#)
- on CHOICE tag [229](#)
- on CMDAREA tag [239](#)
- on DTACOL tag [271](#)
- on DTAFLD tag [279](#)
- on LSTCOL tag [336](#)

- PAD attribute (*continued*)
 - on PANDEF tag [374](#)
 - on PANEL tag [381](#)
 - on SELFLD tag [426](#)
 - using [78](#), [87](#)
- PADC attribute
 - description
 - data field [78](#)
 - DTACOL tag [89](#)
 - LSTCOL tag [95](#)
 - selection field [87](#)
 - on ATTR tag [202](#)
 - on CHOFLD tag [222](#)
 - on CHOICE tag [229](#)
 - on CMDAREA tag [239](#)
 - on DTACOL tag [271](#)
 - on DTAFLD tag [279](#)
 - on LSTCOL tag [336](#)
 - on PANDEF tag [375](#)
 - on PANEL tag [381](#)
 - on SELFLD tag [426](#)
 - using [78](#), [87](#)
- PANDEF (panel default) tag
 - conditions of usage [375](#)
 - defining panel defaults [50](#)
 - description [375](#)
 - examples
 - overriding a value [52](#)
 - referring to default definitions [376](#)
 - shared panel dimensions only [51](#)
 - shared panel values [51](#)
 - syntax [372](#)
- PANDEF attribute
 - on PANEL tag [378](#)
- panel
 - declaring variables for [53](#)
 - defining fields [71](#)
 - defining with the PANEL tag [27](#)
- PANEL (panel) tag
 - conditions of usage [383](#)
 - defining a panel ID [28](#)
 - defining an application panel [27](#)
 - defining cursor placement [30](#)
 - defining panel NAME value [28](#)
 - defining the panel title [29](#)
 - defining the panel width and depth [29](#)
 - description [383](#)
 - examples
 - CURSOR attribute [30](#)
 - HELP attribute [30](#)
 - KEYLIST attribute [29](#), [384](#)
 - start and end tags [28](#)
 - WIDTH and DEPTH attributes [29](#)
 - specifying a key mapping list [29](#)
 - specifying a KEYLIST attribute [29](#)
 - specifying associated help panel [29](#)
 - syntax [376](#)
- panel body [6](#)
- panel default (PANDEF) tag
 - conditions of usage [375](#)
 - description [375](#)
 - examples [376](#)
 - syntax [372](#)
- panel defaults [50](#)
- panel instruction (PNLINST) tag
 - conditions of usage [396](#)
 - description [396](#)
 - examples [397](#)
 - syntax [396](#)
- PANEL option [159](#)
- panel region, defining [45](#)
- panel title
 - defining [29](#)
 - description [5](#)
 - for help panels [7](#)
- panel-title-text attribute
 - on PANEL tag [383](#)
- panels
 - converting multiple [165](#)
- PANELSTMT attribute
 - on PANEL tag [382](#)
- paragraph (P) tag
 - conditions of usage [371](#)
 - description [371](#)
 - examples
 - basic [102](#)
 - defining information region width [371](#)
 - formatting of [103](#)
 - nested in an ordered list [119](#)
 - syntax [370](#)
 - using to define a paragraph [102](#)
- parameter description (PD) tag
 - conditions of usage [389](#)
 - description [389](#)
 - examples [118](#), [389](#)
 - syntax [388](#)
- parameter entity
 - description [21](#)
 - examples [21](#)
 - naming conventions [22](#)
 - syntax [172](#)
- parameter list (PARML) tag
 - conditions of usage [387](#)
 - description [387](#)
 - examples [388](#)
 - syntax [386](#)
 - using to define a parameter list [118](#)
- parameter list divider (PLDIV) tag
 - syntax [394](#)
- parameter term (PT) tag
 - conditions of usage [401](#)
 - description [401](#)
 - examples [118](#), [402](#)
 - syntax [400](#)
- parameter term divider (PTDIV) tag
 - conditions of usage [402](#)
 - description [402](#)
 - examples [403](#)
 - syntax [402](#)
- parameter term segment (PTSEG) tag
 - conditions [404](#)
 - syntax [404](#)
- PARM attribute
 - on ACTION tag [186](#)
 - on KEYI tag [320](#)
- parm list divider (PLDIV) tag
 - conditions of usage [395](#)
 - description [394](#)

- parm list divider (PLDIV) tag (*continued*)
 - examples [395](#)
- PARML (parameter list) tag
 - conditions of usage [387](#)
 - description [387](#)
 - examples [118](#), [388](#)
 - syntax [386](#)
 - using to define a parameter list [118](#)
- PAS attribute
 - description
 - data field [78](#)
 - on ATTR tag [202](#)
 - on CHOFLD tag [223](#)
 - on DTAFLD tag [280](#)
 - on LSTCOL tag [336](#)
 - using [78](#)
- PASSLIB attribute on ACTION tag [187](#)
- PASSTHRU [145](#)
- PD (parameter description) tag
 - conditions of usage [389](#)
 - description [389](#)
 - examples [118](#), [389](#)
 - syntax [388](#)
- PDC (pull-down choice) tag
 - conditions of usage [392](#)
 - description [392](#)
 - examples
 - basic [392](#)
 - defining help panel [36](#)
 - markup [35](#)
 - using mnemonic selection [37](#)
 - syntax [390](#)
- PDCVAR attribute on ABC tag [183](#)
- PDSEP (pull-down separator) tag
 - syntax [394](#)
- phrase-to-be-highlighted attribute
 - on HP tag [316](#)
- PICT test
 - description [65](#)
 - example [65](#)
 - specifying on CHECKI tag [214](#)
- PICTCN test
 - description [66](#)
 - example [66](#)
 - specifying on CHECKI tag [214](#)
- PLACE attribute
 - description
 - data field [79](#)
 - on CMDAREA tag [241](#)
 - on DTAFLD tag [280](#)
 - on PS tag [399](#)
 - using [79](#)
- PLDIV (parm list divider) tag
 - conditions of usage [395](#)
 - description [394](#)
 - examples
 - solid and blank [395](#)
 - syntax [394](#)
- PLEB option [162](#)
- PMTFMT attribute
 - description
 - data field [78](#)
 - on DTACOL tag [272](#)
 - on DTAFLD tag [279](#)

- PMTFMT attribute (*continued*)
 - using [78](#)
- PMTLOC attribute
 - description [89](#)
 - on CMDAREA tag [239](#)
 - on DTAFLD tag [279](#)
 - on SELFLD tag [423](#)
 - using [71](#)
- PMTSKIP attribute
 - description
 - data field [79](#)
 - on DTAFLD tag [281](#)
 - on SELFLD tag [428](#)
 - using [79](#), [88](#)
- PMTTEXT attribute
 - on CMDAREA tag [240](#)
- PMTWIDTH attribute
 - description [89](#)
 - on DTACOL tag [271](#)
 - on DTAFLD tag [278](#)
 - on SELFLD tag [424](#)
 - using [71](#)
- PNLINST (panel instruction) tag
 - conditions of usage [396](#)
 - description [396](#)
 - examples [397](#)
 - syntax [396](#)
- point-and-shoot (PS) tag
 - conditions of usage [399](#)
 - description [399](#)
 - example [400](#)
 - syntax [398](#)
- pop-up window, displaying messages on [137](#)
- POSITION attribute
 - description [95](#)
 - on LSTCOL tag [335](#)
- position value
 - on PANEL tag [380](#)
- predefined entities [23](#)
- predetermined tag attributes [11](#)
- PREP option [159](#)
- preselected pull-down choice [36](#)
- PRIME attribute
 - on PANEL tag [380](#)
- PROFDDN option [162](#)
- PROFILE option [162](#)
- prompt [71](#)
 - See also* field prompt
- prompt-width, specifying for data field [73](#)
- PS (point-and-shoot) tag
 - conditions of usage [399](#)
 - description [399](#)
 - example [400](#)
 - syntax [398](#)
- PSBUTTON attribute
 - on CMDAREA tag [241](#)
- PSVAL attribute
 - description
 - data field [78](#)
 - on CHOFLD tag [223](#)
 - on CMDAREA tag [241](#)
 - on DTAFLD tag [280](#)
 - using [78](#)
- PSVAR attribute

PSVAR attribute (*continued*)

- description
 - data field [78](#)
- on CHOFLD tag [223](#)
- on CMDAREA tag [241](#)
- on DTAFLD tag [279](#)
- using [78](#)

PT (parameter term) tag

- conditions of usage [401](#)
- description [401](#)
- examples [118](#), [402](#)
- syntax [400](#)

PTDIV (parameter term divider) tag

- conditions of usage [402](#)
- description [402](#)
- examples [403](#)
- syntax [402](#)

PTSEG (parameter term segment) tag

- conditions [404](#)
- syntax [404](#)

pull-down choice (PDC) tag

- conditions of usage [392](#)
- description [392](#)
- examples [392](#)
- syntax [390](#)

- actions [35](#)
- defining [33](#)
- example [36](#)
- preselected [36](#)
- providing help for [35](#)

pull-down separator (PDSEP) tag

- syntax [394](#)

pull-down, defining [33](#)

R

RANGE test

- checking values within a numeric range [61](#)
- description [61](#)
- specifying on CHECKI tag [212](#)

RCOLDISP attribute

- on SCRFLD tag [416](#)

RCOLIND attribute

- on SCRFLD tag [416](#)

reference phrase (RP) tag

- conditions of usage [412](#)
- description [412](#)
- examples [412](#)
- syntax [411](#)

REFRESH attribute

- description [88](#)
- on SELFLD tag [427](#)
- using [88](#)

REGION (region) tag

- conditions of usage [407](#)
- defining a region [45](#)
- description [407](#)
- examples
 - DIR attribute [45](#)
 - horizontal and vertical [47](#), [408](#)
- syntax [405](#)

repeatable items, syntax diagrams [xxi](#)

REPLACE keyword

REPLACE keyword (*continued*)

- on ENTITY statement [173](#)

REPLACE option [158](#)

REQUIRED attribute

- description
 - data field [77](#)
 - LSTCOL tag [95](#)
 - selection field [86](#)
- on CHOFLD tag [221](#)
- on DTACOL tag [272](#)
- on DTAFLD tag [278](#)
- on LSTCOL tag [334](#)
- on SELFLD tag [425](#)
- using [77](#), [86](#)

RESULT attribute on ASSIGNI tag [195](#)

return codes, results of converting with DTL [166](#)

RINDVAL attribute

- on SCRFLD tag [415](#)

RINDVAR attribute

- on SCRFLD tag [415](#)

risk (attention statement) [124](#)

risk (warning statement) [124](#)

ROWS attribute

- on LSTFLD tag [342](#)

RP (Reference phrase) tag

- conditions of usage [412](#)
- description [127](#), [412](#)
- example [127](#)
- examples [412](#)
- restriction [127](#)
- syntax [411](#)

rules

- for variable names [179](#)
- formatting Asian panel text [13](#)
- formatting English panel text [13](#)

RULES attribute

- on LSTFLD tag [341](#)

RUN attribute

- example [144](#)
- on ACTION tag [186](#)
- specifying a command [35](#), [144](#)

S

SCALE attribute

- on SCRFLD tag [416](#)

SCRCAPS attribute

- on CMDAREA tag [241](#)
- on DA tag [253](#)
- on LSTFLD tag [342](#)

SCREEN option [158](#)

SCRFLD (scrollable field) tag

- conditions of usage [418](#)
- description [417](#)
- examples [418](#)
- syntax [413](#)

SCRIPT option [159](#)

SCRNAME attribute on ACTION tag [187](#)

SCROLL attribute

- on DA tag [252](#)
- on SCRFLD tag [416](#)

scrollable fields

- defining [97](#)
- SCRFLD tag [413](#)

- SCROLLTAB attribute
 - on CMDAREA tag [240](#)
 - on DA tag [253](#)
 - on LSTFLD tag [342](#)
- SCROLLVAR attribute
 - on CMDAREA tag [240](#)
 - on DA tag [253](#)
 - on LSTFLD tag [342](#)
- SCRVHELP attribute
 - on CMDAREA tag [240](#)
 - on DA tag [253](#)
 - on LSTFLD tag [342](#)
- SELCHAR attribute on CHOICE tag [229](#)
- SELCHECK parameter [89](#)
- SELDEFAULT attribute
 - description [88](#)
 - on SELFLD tag [428](#)
 - using [88](#)
- selection choice (CHOICE) tag
 - conditions of usage [231](#)
 - description [230](#)
 - examples [231](#)
 - syntax [226](#)
- selection choice, defining space for [85](#)
- selection field
 - attributes [86](#)
 - defining [79](#)
 - defining a field prompt [71](#)
 - defining cursor placement [30](#)
 - defining space for choice [85](#)
 - help for [85](#)
 - using the CHOICE tag [80](#)
- selection field (SELFLD) tag
 - conditions of usage [429](#)
 - description [429](#)
 - examples [430](#)
 - syntax [421](#)
- selection list, defining cursor placement [31](#)
- selection width, defining [86](#)
- SELFLD (selection field) tag
 - conditions of usage [429](#)
 - description [429](#)
 - examples [430](#)
 - syntax [421](#)
 - using MSG attribute [139](#)
- SELFMT attribute
 - description [88](#)
 - on SELFLD tag [427](#)
 - using [88](#)
- SELMSG attribute
 - description [87](#)
 - on SELFLD tag [426](#)
 - using [87](#)
- SELMSGU attribute
 - description [87](#)
 - on SELFLD tag [427](#)
 - using [87](#)
- SELWIDTH attribute
 - defining space for [85](#)
 - description [85](#), [89](#)
 - on DTACOL tag [271](#)
 - on SELFLD tag [424](#)
 - using [85](#)
- SETVAR attribute on ACTION tag [187](#)
- SETVERB [145](#)
- SHADOW attribute
 - on DA tag [253](#)
- shortcut keys [493](#)
- simple list (SL) tag
 - conditions of usage [434](#)
 - description [434](#)
 - examples
 - basic [109](#)
 - compact and nested [435](#)
 - COMPACT attribute [109](#)
 - syntax [433](#)
 - using to define a simple list [109](#)
- SINDVAL attribute
 - on SCRFLD tag [415](#)
- SINDVAR attribute
 - on SCRFLD tag [415](#)
- single-choice selection field
 - defining [80](#)
 - discussion [80](#)
 - example [80](#)
- SKIP attribute
 - on ATTR tag [202](#)
 - on PARML tag [387](#)
- SL (simple list) tag
 - conditions of usage [434](#)
 - description [434](#)
 - examples
 - basic [109](#)
 - compact and nested [435](#)
 - COMPACT attribute [109](#)
 - syntax [433](#)
 - using to define a simple list [109](#)
- SMSG attribute
 - on MSG tag [354](#)
 - on PANEL tag [382](#)
- SORT attribute on CMDTBL tag [244](#)
- source (SOURCE) tag
 - example [437](#)
- SOURCE (source) tag
 - example [437](#)
- SOURCE (Source) tag
 - conditions of usage [436](#)
 - description [436](#)
 - syntax [435](#)
- source file
 - defining dialog elements [11](#)
 - defining entities and parameter entities [17](#)
 - DOCTYPE declaration [171](#)
 - embedding [22](#)
 - frequently used words (entities) [17](#)
 - including comments [15](#), [16](#)
 - including copyright statements [15](#)
- source-filespec, for system [158](#)
- SPACE attribute
 - on LI tag [325](#)
 - on OL tag [368](#)
 - on P tag [370](#)
 - on SL tag [434](#)
 - on UL tag [444](#)
- SPACE keyword
 - on ENTITY statement [173](#)
- specifying
 - a list of values to match value of user input [62](#)

- specifying (*continued*)
 - a list of valuesx to match value of user input [62](#)
 - calling the conversion utility [158](#)
 - source-filespec [158](#)
- SPLIT attribute
 - on DL tag [262](#)
 - on PARML tag [387](#)
- start tag delimiters [11](#)
- STATS option [159](#)
- STDDATE attribute
 - value STDDATE on CHECKI tag [216](#)
- STDDATE test
 - description [68](#)
 - example [68](#)
- STDTIME attribute
 - value STDTIME on CHECKI tag [217](#)
- STDTIME test
 - description [69](#)
 - example [69](#)
- STRIP attribute
 - on GRPHDR tag [301](#)
- SUBSTITUTE attribute
 - on GENERATE tag [298](#)
- SUFFIX attribute on MSG tag [353](#)
- summary of changes xxix
- SUSPEND attribute on ACTION tag [187](#)
- syntax diagrams, how to read [xxi](#)
- SYSTEM keyword
 - on ENTITY statement [173](#)

T

- T (truncation) tag
 - conditions of usage [438](#)
 - description [437](#)
 - examples [145](#), [438](#)
 - syntax [437](#)
 - using to define command truncation [145](#)
- tag
 - attributes [11](#)
 - coding text [12](#)
 - delimiters [11](#)
 - description [11](#)
 - nesting [14](#)
 - rules for coding values [12](#)
 - text [12](#)
 - values [11](#)
- tags
 - AB [179](#)
 - ABC [182](#)
 - ACTION [184](#)
 - ASSIGNI [195](#)
 - ASSIGNL [196](#)
 - ATTENTION [121](#), [198](#)
 - ATTR [200](#)
 - BOTINST [204](#)
 - CAUTION [121](#)
 - CHDIV [207](#)
 - CHECKI [210](#)
 - CHECKL [218](#)
 - CHOFLD [219](#)
 - CHOICE [226](#)
 - CMD [233](#)
 - CMDACT [234](#)

- tags (*continued*)
 - CMDAREA [237](#)
 - CMDTBL [243](#)
 - COMMENT [245](#)
 - COMPOPT [247](#)
 - COPYRIGHT [249](#)
 - DA [250](#)
 - DD [255](#)
 - DDHD [257](#)
 - DIVIDER [258](#)
 - DL (definition list) [261](#)
 - DLDIV [265](#)
 - DT [267](#)
 - DTACOL [269](#)
 - DTAFLD [275](#)
 - DTAFLDD [284](#)
 - DTDIV [286](#)
 - DTHD [287](#)
 - DTHDIV [288](#)
 - DTSEG [290](#)
 - FIG [291](#)
 - FIGCAP [294](#)
 - for information region [101](#)
 - GA [295](#)
 - GENERATE [298](#)
 - GRPHDR [300](#)
 - HELP [303](#)
 - HELPDEF [311](#)
 - Hn [313](#)
 - HP [315](#)
 - INFO [317](#)
 - KEYI [319](#)
 - KEYL [322](#)
 - LI [325](#)
 - LINES [327](#)
 - LIT [329](#)
 - LP [330](#)
 - LSTCOL [332](#)
 - LSTFLD [341](#)
 - LSTGRP [345](#)
 - LSTVAR [348](#)
 - M [351](#)
 - mnemonic [351](#)
 - MSG [352](#)
 - MSGMBR [356](#)
 - nesting [120](#)
 - NOTE [358](#)
 - NOTEL [361](#)
 - NT [121](#), [364](#)
 - OL [367](#)
 - P [370](#)
 - PANDEF [372](#)
 - PANEL [376](#)
 - PARML [386](#)
 - PD [388](#)
 - PDC [390](#)
 - PDSEP [394](#)
 - PLDIV [394](#)
 - PNLINST [396](#)
 - PS [398](#)
 - PT [400](#)
 - PTDIV [402](#)
 - PTSEG [404](#)
 - REGION [405](#)

tags (*continued*)

- RP [411](#)
- SCRFLD [413](#)
- SELFLD [421](#)
- SL [433](#)
- SOURCE [435](#)
- T [437](#)
- TEXTLINE [439](#)
- TEXTSEG [439](#)
- TOPINST [441](#)
- UL [443](#)
- VARCLASS [445](#)
- VARDCL [449](#)
- VARLIST [450](#)
- VARSUB [452](#)
- WARNING [121](#), [454](#)
- XLATI [456](#)
- XLATL [458](#)
- XMP [460](#)

text

- adding to a list [120](#)
- defining for information region [102](#)
- help [131](#)
- providing attention to user [124](#)
- providing caution [125](#)
- providing notes to user [121](#)
- providing warning to user [124](#)
- static [101](#)
- tag [12](#)
- tags used to define [101](#)
- using the example tag [105](#)
- using the figure tag [106](#)
- using the heading tags [104](#)
- using the LINES tag [104](#)
- using the paragraph tag [102](#)

TEXT attribute

- description [95](#)
- on AREA tag [191](#)
- on LSTCOL tag [335](#)
- on NOTE tag [360](#)
- on NOTEL tag [363](#)
- on NT tag [366](#)
- on OL tag [368](#)
- on SL tag [434](#)
- on UL tag [444](#)

text line (TEXTLINE) tag

- conditions of usage [439](#)
- description [439](#)
- examples [439](#)
- syntax [439](#)

text segment (TEXTSEG) tag

- conditions of usage [440](#)
- description [440](#)
- examples [440](#)
- syntax [439](#)

TEXTFMT attribute

- description [96](#)
- on LSTCOL tag [336](#)

TEXTLEN attribute

- description [96](#)
- on LSTCOL tag [336](#)

TEXTLINE (text line) tag

- conditions of usage [439](#)
- description [439](#)

TEXTLINE (text line) tag (*continued*)

- syntax [439](#)

TEXTLINE (text line)tag

- examples [439](#)

TEXTLOC attribute

- description [95](#)
- on LSTCOL tag [335](#)

TEXTSEG (text segment) tag

- conditions of usage [440](#)
- description [440](#)
- syntax [439](#)

TEXTSEG(text segment)tag

- examples [440](#)

TEXTSKIP attribute

- description [96](#)
- on LSTCOL tag [336](#)

title, panel [29](#)

TITLE attribute

- on PANEL tag [381](#)

TMARGIN attribute

- on PANDEF TAG [375](#)
- on PANEL TAG [382](#)

TOGVAR attribute on ACTION tag [188](#)

top instruction

- defining [37](#)
- description [6](#)

top instruction (TOPINST) tag

- conditions of usage [442](#)
- description [442](#)
- examples [442](#)
- syntax [441](#)

TOPINST (top instruction) tag

- conditions of usage [442](#)
- defining instruction text [37](#)
- description [442](#)
- examples [38](#), [442](#)
- syntax [441](#)

trademarks [498](#)

TRAIL attribute

- description [87](#)
- on SELFLD tag [426](#)
- using [87](#)

translate item (XLATI) tag

- conditions of usage [457](#)
- description [457](#)
- examples [457](#)
- syntax [456](#)

translate list (XLATL) tag

- conditions of usage [459](#)
- description [459](#)
- examples [459](#)
- syntax [458](#)

translate lists

- defining [56](#)
- types [56](#)

translation

- defining items for [56](#)
- example [57](#)

TRUNC attribute

- on CHOICE tag [230](#)
- on XLATL tag [459](#)

truncation (T) tag

- conditions of usage [438](#)
- description [437](#)

- truncation (T) tag (*continued*)
 - examples [145](#), [438](#)
 - syntax [437](#)
 - using to define command truncation [145](#)
- TSIZE attribute
 - description [88](#)
 - on DL tag [262](#)
 - on PARML tag [386](#)
 - on SELFLD tag [427](#)
 - using [88](#)
- TUTOR attribute
 - on HELP tag [306](#)
 - on PANEL tag [380](#)
- tutor-choice fields [84](#)
- TYPE attribute
 - description [79](#)
 - on ACTION tag [187](#)
 - on ATTR tag [202](#)
 - on CHDIV tag [208](#)
 - on CHECKI tag [212](#)
 - on COMMENT tag [246](#)
 - on DIVIDER tag [259](#)
 - on DLDIV tag [265](#)
 - on HP tag [316](#)
 - on NOTE tag [359](#)
 - on NOTEL tag [363](#)
 - on NT tag [365](#)
 - on PANEL tag [381](#)
 - on PLDIV tag [394](#)
 - on SELFLD tag [423](#)
 - on SOURCE tag [436](#)
 - on VARCLASS tag [446](#)
 - using [79](#)
 - value %varname on VARCLASS tag [447](#)
 - value ALPHA on CHECKI tag [212](#)
 - value ALPHAB on CHECKI tag [215](#)
 - value ANY on VARCLASS tag [446](#)
 - value BIT on CHECKI tag [214](#)
 - value CHAR on VARCLASS tag [446](#)
 - value CHARS on CHECKI tag [212](#)
 - value DBCS on CHECKI tag [214](#)
 - value DBCS on VARCLASS tag [446](#)
 - value DSNAME on CHECKI tag [215](#)
 - value DSNAMEF on CHECKI tag [215](#)
 - value DSNAMEFM on CHECKI tag [215](#)
 - value DSNAMEPQ on CHECKI tag [216](#)
 - value DSNAMEQ on CHECKI tag [216](#)
 - value EBCDIC on CHECKI tag [215](#)
 - value EBCDIC on VARCLASS tag [447](#)
 - value ENUM on CHECKI tag [215](#)
 - value FILEID on CHECKI tag [216](#)
 - value HEX on CHECKI tag [216](#)
 - value IDATE on VARCLASS tag [447](#)
 - value INCLUDE on CHECKI tag [216](#)
 - value IPADDR4 on CHECKI tag [217](#)
 - value ITIME on VARCLASS tag [447](#)
 - value JDATE on VARCLASS tag [447](#)
 - value JSTD on VARCLASS tag [447](#)
 - value LEN on CHECKI tag [215](#)
 - value LISTV on CHECKI tag [215](#)
 - value LISTVX on CHECKI tag [215](#)
 - value MIX on CHECKI tag [216](#)
 - value MIXED on VARCLASS tag [446](#)
 - value NAME on CHECKI tag [214](#)

- TYPE attribute (*continued*)
 - value NAMEF on CHECKI tag [214](#)
 - value NUM on CHECKI tag [214](#)
 - value NUMERIC on VARCLASS tag [447](#)
 - value PICT on CHECKI tag [214](#)
 - value PICTCN on CHECKI tag [212](#)
 - value RANGE on CHECKI tag [212](#)
 - value STDDATE on VARCLASS tag [447](#)
 - value STDTIME on VARCLASS tag [447](#)
 - value VALUES on CHECKI tag [213](#)
 - value VALUESX on CHECKI tag [213](#)
 - value VMASK on VARCLASS tag [447](#)

U

- UL (unordered list) tag
 - conditions of usage [444](#)
 - description [444](#)
 - examples
 - basic [110](#)
 - basic and nested [445](#)
 - nested [111](#)
 - syntax [443](#)
 - using to define an unordered list [110](#)
- UNAVAIL attribute
 - on CHOICE tag [230](#)
 - on PDC tag [391](#)
- UNAVAILMAT attribute on CHOICE tag [230](#)
- unformatted text
 - using the example tag [105](#)
 - using the figure tag [106](#)
 - using the LINES tag [104](#)
- unordered list (UL) tag
 - conditions of usage [444](#)
 - description [444](#)
 - examples
 - basic [110](#)
 - basic and nested [445](#)
 - nested [111](#)
 - syntax [443](#)
 - using to define an unordered list [110](#)
- upper translate list
 - example [56](#)
 - translating a value to uppercase [56](#)
- USAGE attribute
 - description [91](#)
 - on CHOFLD tag [221](#)
 - on DTAFLD tag [277](#)
 - on LSTCOL tag [334](#)
- user interface
 - ISPF [493](#)
 - TSO/E [493](#)
- USERMOD attribute
 - on DA tag [252](#)

V

- V3PADC option [162](#)
- validity check item (CHECKI) tag
 - conditions of usage [217](#)
 - description [217](#)
 - examples [217](#)
 - syntax [210](#)

- validity check list (CHECKL) tag
 - conditions of usage [218](#)
 - description [218](#)
 - examples [139](#), [218](#)
 - syntax [218](#)
- validity checks
 - description [60](#)
 - types [60](#)
 - using [60](#)
 - using CHECKL and CHECKI tags [60](#)
- value
 - description [11](#)
 - tag [11](#)
- VALUE attribute
 - on ACTION tag [187](#)
 - on ASSIGNI tag [195](#)
 - on PS tag [399](#)
 - on XLATI tag [456](#)
- VALUE1 attribute on ACTION tag [188](#)
- VALUE2 attribute on ACTION tag [188](#)
- VALUES test
 - description [62](#)
 - example [62](#)
 - specifying on CHECKI tag [213](#)
- VALUESX test
 - description [62](#)
 - example [62](#)
 - specifying on CHECKI tag [213](#)
- VAR attribute
 - on PS tag [399](#)
 - on VARSUB tag [452](#)
- VARCLASS (variable class) tag
 - conditions of usage [448](#)
 - description [448](#)
 - examples [448](#)
 - syntax [445](#)
 - using MSG attribute [139](#)
 - using to define a variable class [54](#)
- VARCLASS attribute
 - description [77](#), [89](#)
 - input-only [96](#)
 - input/output [96](#)
 - on CHOFLD tag [221](#)
 - on DTAFLD tag [277](#)
 - on LSTCOL tag [334](#)
 - on VARDCL tag [449](#)
 - using [77](#)
- VARDCL (variable declaration) tag
 - conditions of usage [450](#)
 - description [449](#)
 - examples [53](#), [450](#)
 - syntax [449](#)
- VARDCL attribute
 - description
 - data field [79](#)
 - on CMDAREA tag [241](#)
 - on DTACOL tag [272](#)
 - on DTAFLD tag [282](#)
 - on LSTCOL tag [337](#)
 - on LSTFLD tag [343](#)
 - on SELFLD tag [428](#)
 - using [79](#)
- variable class
 - associating translate list [56](#)
- variable class (*continued*)
 - defining [54](#)
 - defining character variables [54](#)
 - defining translate list [458](#)
 - description [53](#)
 - overriding [70](#)
 - overriding the data variable [96](#)
 - overview [8](#)
- variable class (VARCLASS) tag
 - conditions of usage [448](#)
 - description [448](#)
 - examples [448](#)
 - syntax [445](#)
- variable data
 - aligning [77](#), [94](#)
 - description [94](#)
- variable declaration (VARDCL) tag
 - conditions of usage [450](#)
 - description [449](#)
 - examples [450](#)
 - syntax [449](#)
- variable list
 - defining [53](#)
- variable list (VARLIST) tag
 - conditions of usage [451](#)
 - description [451](#)
 - examples [451](#)
 - syntax [450](#)
- variable names
 - as a tag value, assigning [12](#)
 - rules [179](#)
 - rules for %varname as a tag value [179](#)
- variable substitution (VARSUB) tag
 - conditions of usage [453](#)
 - description [453](#)
 - examples [141](#), [453](#)
 - syntax [452](#)
 - using to substitute a variable [141](#)
- variable translate list
 - defining [56](#)
 - relation to VARCLASS definition [56](#)
 - types [56](#)
- variable validation [56](#)
- variables
 - character [54](#)
 - declaring [53](#)
 - defining a variable class [54](#)
 - in data fields [74](#)
 - item translating [57](#)
 - numeric [56](#)
 - overview [8](#)
 - rules for naming [179](#)
 - specifying in message text [141](#)
 - translate list [56](#)
 - translating [56](#), [57](#)
 - validating [56](#)
 - validity checks [60](#)
- variables, syntax diagrams [xxi](#)
- VARLIST (variable list) tag
 - conditions of usage [451](#)
 - description [451](#)
 - examples [53](#), [451](#)
 - syntax [450](#)
 - using [53](#)

VARSUB (variable substitution) tag
conditions of usage [453](#)
description [453](#)
examples [453](#)
syntax [452](#)
VERIFY attribute
description [88](#)
on SELFLD tag [427](#)
using [88](#)
VERSION option [160](#)
vertical region [45](#)

W

WARNING (warning) tag
conditions of usage [455](#)
description [454](#)
examples [199](#), [455](#)
syntax [454](#)
warning message
defining [138](#)
description
format [124](#)
information messages [138](#)
ISPF conversion utility messages [165](#)
example [124](#)
WARNING tag
description [124](#)
example [124](#)
width and depth, defining with PANDEF tag [50](#)
WIDTH attribute
defining application panel width [29](#)
on AREA tag [191](#)
on DA tag [253](#)
on FIG tag [292](#)
on GA tag [296](#)
on GRPHDR tag [301](#)
on HELP tag [305](#)
on HELPDEF tag [311](#)
on INFO TAG [318](#)
on MSGMBR tag [357](#)
on PANDEF tag [374](#)
on PANEL tag [379](#)
on REGION tag [406](#)
window
layout [5](#)
WINDOW attribute
on PANDEF tag [374](#)
on PANEL tag [380](#)
WINTITLE attribute
on HELP TAG [306](#)
on HELPDEF TAG [312](#)
on PANDEF tag [374](#)
on PANEL tag [381](#)
word-wrapping
data field width [75](#)
on DESWIDTH attribute [75](#)

X

XLATI (translate item) tag
conditions of usage [457](#)
description [457](#)

XLATI (translate item) tag (*continued*)
examples [457](#)
syntax [456](#)
XLATL (translate list) tag
conditions of usage [459](#)
description [459](#)
examples [459](#)
syntax [458](#)
using MSG attribute [139](#)
XMP (example) tag
conditions of usage [461](#)
description [461](#)
examples [105](#), [461](#)
syntax [460](#)
using to define an example [105](#)

Z

ZCMD system variable
for PASSTHRU [145](#)
for SETVERB [145](#)
ZCONT attribute
on PANEL tag [382](#)
ZGUI attribute
description [88](#)
on SELFLD tag [427](#)
using [88](#)
ZKEYHELP, using [148](#)
ZUP attribute
on PANEL tag [382](#)
ZVARS option [161](#)
ZVERB system variable for SETVERB [145](#)



Product Number: 5655-ZOS

SC19-3620-60

